

PROJECT 3 · SRS IMPLEMENTATION

MARKETLINK AGRI-HUB

B2B Agricultural Marketplace connecting Pakistani farmers (kisan) directly with wholesalers, supermarket chains, exporters and food processors.

Complete Project Book

- ▶ Hosting guide: local, temporary public link and free cloud
- ▶ Database ERD & DDL (PostgreSQL + MySQL)
- ▶ REST API endpoint specification
- ▶ Business logic workflows (Bidding · Escrow · Mandi Sync)
- ▶ Full source code: Farmer, Buyer & Admin dashboards

Tech stack

Next.js 16 (React 19) · Tailwind CSS 4 · PostgreSQL · Drizzle ORM · Recharts · REST API · RBAC · Docker

Live preview (temporary)

<https://3000-ia4hpf9h9j4g803r3j2ey.e2b.app>

Demo logins (password: password123)

Farmer: farmer@marketlink.pk Buyer: buyer@marketlink.pk Admin: ahmed.mustafa@admin.com

Contents

- 0** Hosting Guide (local · temporary link · free cloud)
- 1** Project Overview & Features
- 2** Database: ERD & DDL Scripts
- 3** REST API Endpoint Specification
- 4** Business Logic Workflows
- 5** Project Structure
- 6** Complete Source Code (178 files, 15,632 lines)

Tip: use your PDF viewer's bookmarks panel to jump to any section or source file.

0 · Hosting Guide

Why <http://localhost:3000> does not open by itself

localhost means 'this computer'. The link only works AFTER you start the MarketLink server on your own PC, and only while its server window stays open. Opening the link without starting the server always shows 'This site can't be reached'.

Option 1: Run on your PC (recommended, only Node.js needed)

The app includes a built-in PostgreSQL-compatible database (PGlite) stored in the .data folder, so there's no PostgreSQL or Docker to install. Tables and demo data are created automatically.

- 1 Install Node.js 20 LTS or newer from nodejs.org (green LTS button, default options).
- 2 Download MarketLink-AgriHub-Pakistan.zip from the /deploy page, right-click → Extract All. Don't run it from inside the zip.
- 3 Windows: double-click start-windows.bat. Mac/Linux: run 'bash start.sh' in Terminal. The first run installs and builds (3-6 minutes).
- 4 Wait for 'MarketLink is running!'. The browser opens <http://localhost:3000> automatically. Keep the window open; press Ctrl+C to stop.
- 5 Phone on the same Wi-Fi: open the 'Phone / same Wi-Fi' address printed in the window (allow Windows Firewall access if asked).

Troubleshooting

- 'node' is not recognized: Install Node.js, restart the PC, try again.
- Browser: can't be reached: The server window was closed or is still starting. Wait for the message.
- Port 3000 busy: Handled automatically: the window shows the port used (e.g. 3001).
- Fresh demo data: Stop the server, delete the .data folder, start again.
- Updated code: Run start-windows.bat --rebuild
- Own PostgreSQL: Set DATABASE_URL in .env. The database is created automatically.

Option 2: Temporary public link from your PC

- 1 Keep Option 1 running.
- 2 In a second terminal run: `npx cloudflared tunnel --url http://localhost:3000`
- 3 Share the printed <https://xxxx.trycloudflare.com> link. It works while your PC stays on.
Alternative: `npx localtunnel --port 3000`

Option 3: Docker (with a real PostgreSQL server)

- 1 Install Docker Desktop.
- 2 Double-click start-docker.bat, or run: `docker compose up --build`
- 3 Open <http://localhost:3000>

Option 4: Free cloud hosting (24/7)

- Render.com: push to GitHub → New → Blueprint (render.yaml creates the web service + PostgreSQL). Free tier sleeps after 15 min idle.

- Vercel + Neon: free Neon Postgres, import the repo on Vercel, set DATABASE_URL, SESSION_SECRET, COOKIE_SECURE=true.
- Railway.app: deploy from GitHub + PostgreSQL plugin, set DATABASE_URL and SESSION_SECRET.

Admin panel access

- 1 Open /admin-login (e.g. http://localhost:3000/admin-login) and sign in: ahmed.mustafa@admin.com / password123.
- 2 Change the default password right away: sidebar → Account settings. The admin dashboard shows a red warning until you do.
- 3 Admin pages: /admin (analytics), /admin/users (create admins & inspectors, reset passwords, change roles), /admin/verification, /admin/mandi, /admin/orders.
- 4 Own admin login from the first start: set ADMIN_EMAIL and ADMIN_PASSWORD in .env before the first run (or delete the .data folder to start over).

Environment variables (.env)

DATABASE_URL empty = built-in database; or a PostgreSQL connection string
ADMIN_EMAIL / ADMIN_PASSWORD admin login created on first start
SESSION_SECRET long random text that signs login cookies
COOKIE_SECURE 'true' when served over HTTPS
PORT preferred port (default 3000, auto-increments if busy)

1 · Project Overview & Features

MarketLink Agri-Hub is a hyper-local and regional B2B agri-commerce platform for Pakistan. It lets farmers sell graded produce directly to commercial buyers without arhti (commission agent) middlemen, with transparent mandi pricing, PKR escrow payments, quality inspection and truck tracking.

Pakistan localisation

- Currency: Pakistani Rupee (Rs. / PKR), with Lakh / Crore short forms for large amounts.
- Mandi rates per kg and per maund (40 kg), the unit Pakistani farmers and arhtis use.
- 38 cities across Punjab, Sindh, Khyber Pakhtunkhwa, Balochistan, Islamabad, Gilgit-Baltistan and Azad Kashmir, grouped by province.
- 10 real wholesale mandis: Badami Bagh Lahore, Super Highway Karachi, Ghalla Mandi Multan, Faisalabad, I-11 Islamabad, Hyderabad, Peshawar, Quetta, Sukkur, Gujranwala.
- Local crops: Wheat, Basmati, Maize, Chana, Tomato, Potato, Onion, Chaunsa Mango, Kinnow, Khajoor, Cotton (Phutti), Sugarcane, Kunri Red Chilli.
- English / Urdu toggle with right-to-left layout, Pakistan Standard Time clock, CNIC validation (12345-1234567-1), JazzCash / Easypaisa / IBFT top-up (simulated).

User roles (RBAC)

Farmer (Kisan / Seller)

- Dashboard: inventory value, active listings, incoming bids, earnings & escrow, monthly sales chart, sales-by-crop donut.
- Crop listing: photos, grade A/B/C, harvest date, quantity in kg / maund / tons, price suggested from live mandi rates.
- Live Mandi Insights and Negotiation Hub: accept / reject / counter from a slide-in drawer with instant feedback.
- Order dispatch, transit checkpoints, escrow payouts wallet.

Buyer (Wholesaler / Supermarket / Exporter / Processor)

- Marketplace filtered by category, grade, price and distance from any Pakistani city.
- Bidding & RFQ: price, quantity, target delivery date. Respond to counter-offers.
- Order pipeline with a 5-stage logistics stepper and checkpoint timeline.
- Escrow wallet: lock funds at deal time, release on delivery, raise disputes.

Admin / Quality Inspector

- Verification portal: verify farmer CNIC/farm, upload inspection reports (grade, moisture %, soil pH, PDF/image).
- Mandi price controller: manual daily rates and a mock government feed sync.
- System analytics: volume, active bids, top regions, category mix, logistics pipeline, dispute log.
- Pre-dispatch quality inspection and dispute resolution (release to farmer / refund buyer).

Architecture

- Presentation: React Server Components + Client Components (drawers, modals, Recharts charts). Tailwind CSS 4, dark/light mode, mobile navigation.
- API: Next.js Route Handlers exposing REST endpoints with a { success, data | error } envelope.
- Domain: service modules (crops, bids, orders, mandi, analytics, admin) with transactional row locking.
- Data: PostgreSQL (server) or built-in PGLite (local, zero-setup) via Drizzle ORM. 9 tables, enums, foreign keys, indexes. Schema + demo data are created automatically on an empty database.

- Security: script password hashing, HMAC-signed httpOnly session cookie, role checks on every endpoint and page.

2 · Database: ERD & DDL Scripts

Entity relationships

```
users 1—* crops_inventory      (farmer_id)
users 1—* bids_negotiations    (buyer_id)
crops_inventory 1—* bids_negotiations (crop_id)
bids_negotiations 1—0..1 orders_logistics (bid_id UNIQUE)
users 1—* orders_logistics      (farmer_id, buyer_id)
crops_inventory 1—* orders_logistics (crop_id)
orders_logistics 1—* order_events (order_id)
crops_inventory 1—* quality_inspections (crop_id)
users 1—* quality_inspections    (inspector_id)
users 1—* wallet_transactions    (user_id)
orders_logistics 1—* disputes    (order_id)
orders_logistics 1—0..1 reviews (order_id UNIQUE) → users (buyer_id, farmer_id)
users 1—* notifications · auth_tokens · payment_intents · payout_requests · db_audit_log
```

PostgreSQL DDL (runtime, generated from src/db/schema.ts)

```
1  CREATE TYPE "public"."bid_status" AS ENUM('pending', 'accepted', 'rejected', 'countered', 'withdrawn');
2  CREATE TYPE "public"."crop_category" AS ENUM('grains', 'vegetables', 'fruits', 'cash_crops');
3  CREATE TYPE "public"."crop_status" AS ENUM('active', 'pending_inspection', 'sold_out', 'archived');
4  CREATE TYPE "public"."delivery_stage" AS ENUM('confirmed', 'quality_checked', 'dispatched', 'in_transit',
5  'delivered');
6  CREATE TYPE "public"."dispute_status" AS ENUM('open', 'investigating', 'resolved_release', 'resolved_refun
7  d');
8  CREATE TYPE "public"."inspection_status" AS ENUM('pending', 'passed', 'failed');
9  CREATE TYPE "public"."payment_status" AS ENUM('awaiting_escrow', 'escrow_locked', 'released', 'refunded',
10 'disputed');
11 CREATE TYPE "public"."quality_grade" AS ENUM('A', 'B', 'C');
12 CREATE TYPE "public"."user_role" AS ENUM('farmer', 'buyer', 'admin', 'inspector');
13 CREATE TYPE "public"."wallet_txn_type" AS ENUM('deposit', 'escrow_lock', 'escrow_release', 'payout', 'refu
14 nd', 'platform_fee', 'withdrawal');
15 CREATE TABLE "auth_tokens" (
16   "token_id" serial PRIMARY KEY NOT NULL,
17   "user_id" integer NOT NULL,
18   "purpose" varchar(20) NOT NULL,
19   "code_hash" varchar(128) NOT NULL,
20   "attempts" integer DEFAULT 0 NOT NULL,
21   "expires_at" timestamp with time zone NOT NULL,
22   "used_at" timestamp with time zone,
23   "created_at" timestamp with time zone DEFAULT now() NOT NULL
24 );
25
26 CREATE TABLE "bids_negotiations" (
27   "bid_id" serial PRIMARY KEY NOT NULL,
28   "crop_id" integer NOT NULL,
29   "buyer_id" integer NOT NULL,
30   "bid_price_per_kg" numeric(14, 2) NOT NULL,
31   "bid_quantity_kg" numeric(14, 2) NOT NULL,
32   "target_delivery_date" date,
33   "message" text,
34   "status" "bid_status" DEFAULT 'pending' NOT NULL,
35   "counter_price" numeric(14, 2),
36   "counter_note" text,
37   "created_at" timestamp with time zone DEFAULT now() NOT NULL,
38   "updated_at" timestamp with time zone DEFAULT now() NOT NULL
39 );
40
41 CREATE TABLE "crops_inventory" (
42   "crop_id" serial PRIMARY KEY NOT NULL,
43   "farmer_id" integer NOT NULL,
44   "crop_name" varchar(80) NOT NULL,
45   "category" "crop_category" NOT NULL,
46   "quality_grade" "quality_grade" NOT NULL,
47   "total_quantity_kg" numeric(14, 2) NOT NULL,
```

```

44     "base_price_per_kg" numeric(14, 2) NOT NULL,
45     "harvest_date" date NOT NULL,
46     "images_json" jsonb DEFAULT '[]'::jsonb NOT NULL,
47     "description" text,
48     "farm_location" varchar(80) NOT NULL,
49     "latitude" double precision,
50     "longitude" double precision,
51     "status" "crop_status" DEFAULT 'active' NOT NULL,
52     "created_at" timestamp with time zone DEFAULT now() NOT NULL
53 );
54
55 CREATE TABLE "db_audit_log" (
56     "audit_id" serial PRIMARY KEY NOT NULL,
57     "admin_id" integer,
58     "mode" varchar(10) NOT NULL,
59     "query" text NOT NULL,
60     "row_count" integer,
61     "duration_ms" integer,
62     "error" text,
63     "created_at" timestamp with time zone DEFAULT now() NOT NULL
64 );
65
66 CREATE TABLE "disputes" (
67     "dispute_id" serial PRIMARY KEY NOT NULL,
68     "order_id" integer NOT NULL,
69     "raised_by" integer NOT NULL,
70     "reason" text NOT NULL,
71     "status" "dispute_status" DEFAULT 'open' NOT NULL,
72     "resolution" text,
73     "created_at" timestamp with time zone DEFAULT now() NOT NULL,
74     "resolved_at" timestamp with time zone
75 );
76
77 CREATE TABLE "mandi_rates" (
78     "rate_id" serial PRIMARY KEY NOT NULL,
79     "crop_name" varchar(80) NOT NULL,
80     "market_location" varchar(80) NOT NULL,
81     "min_price_per_kg" numeric(14, 2) NOT NULL,
82     "max_price_per_kg" numeric(14, 2) NOT NULL,
83     "avg_price_per_kg" numeric(14, 2) NOT NULL,
84     "source" varchar(20) DEFAULT 'manual' NOT NULL,
85     "date_updated" date NOT NULL
86 );
87
88 CREATE TABLE "notifications" (
89     "notification_id" serial PRIMARY KEY NOT NULL,
90     "user_id" integer NOT NULL,
91     "type" varchar(40) NOT NULL,
92     "title" varchar(200) NOT NULL,
93     "body" text,
94     "link" varchar(300),
95     "channels" varchar(60) DEFAULT 'in_app' NOT NULL,
96     "is_read" boolean DEFAULT false NOT NULL,
97     "created_at" timestamp with time zone DEFAULT now() NOT NULL
98 );
99
100 CREATE TABLE "order_events" (
101     "event_id" serial PRIMARY KEY NOT NULL,
102     "order_id" integer NOT NULL,
103     "stage" varchar(40) NOT NULL,
104     "location" varchar(120),
105     "lat" double precision,
106     "lng" double precision,
107     "note" text,
108     "actor_id" integer,
109     "created_at" timestamp with time zone DEFAULT now() NOT NULL
110 );
111
112 CREATE TABLE "orders_logistics" (
113     "order_id" serial PRIMARY KEY NOT NULL,
114     "bid_id" integer NOT NULL,
115     "crop_id" integer NOT NULL,
116     "farmer_id" integer NOT NULL,
117     "buyer_id" integer NOT NULL,
118     "quantity_kg" numeric(14, 2) NOT NULL,
119     "price_per_kg" numeric(14, 2) NOT NULL,

```

```

120     "total_amount" numeric(14, 2) NOT NULL,
121     "payment_status" "payment_status" DEFAULT 'awaiting_escrow' NOT NULL,
122     "delivery_stage" "delivery_stage" DEFAULT 'confirmed' NOT NULL,
123     "transit_location" varchar(120),
124     "current_lat" double precision,
125     "current_lng" double precision,
126     "tracking_number" varchar(40) NOT NULL,
127     "expected_delivery_date" date,
128     "created_at" timestamp with time zone DEFAULT now() NOT NULL,
129     "updated_at" timestamp with time zone DEFAULT now() NOT NULL
130 );
131
132 CREATE TABLE "payment_intents" (
133     "intent_id" serial PRIMARY KEY NOT NULL,
134     "reference" varchar(60) NOT NULL,
135     "user_id" integer NOT NULL,
136     "provider" varchar(20) NOT NULL,
137     "amount" numeric(14, 2) NOT NULL,
138     "status" varchar(20) DEFAULT 'pending' NOT NULL,
139     "provider_ref" varchar(200),
140     "raw_callback" text,
141     "created_at" timestamp with time zone DEFAULT now() NOT NULL,
142     "completed_at" timestamp with time zone
143 );
144
145 CREATE TABLE "payout_requests" (
146     "payout_id" serial PRIMARY KEY NOT NULL,
147     "farmer_id" integer NOT NULL,
148     "amount" numeric(14, 2) NOT NULL,
149     "method" varchar(20) NOT NULL,
150     "account_title" varchar(120) NOT NULL,
151     "account_number" varchar(60) NOT NULL,
152     "status" varchar(20) DEFAULT 'pending' NOT NULL,
153     "admin_note" text,
154     "created_at" timestamp with time zone DEFAULT now() NOT NULL,
155     "processed_at" timestamp with time zone
156 );
157
158 CREATE TABLE "quality_inspections" (
159     "inspection_id" serial PRIMARY KEY NOT NULL,
160     "crop_id" integer NOT NULL,
161     "inspector_id" integer NOT NULL,
162     "grade_assigned" "quality_grade" NOT NULL,
163     "moisture_level_percentage" numeric(5, 2) NOT NULL,
164     "soil_ph" numeric(4, 2),
165     "inspection_notes" text,
166     "report_attachment" text,
167     "report_file_name" varchar(200),
168     "certificate_no" varchar(40),
169     "status" "inspection_status" DEFAULT 'pending' NOT NULL,
170     "verified_at" timestamp with time zone
171 );
172
173 CREATE TABLE "reviews" (
174     "review_id" serial PRIMARY KEY NOT NULL,
175     "order_id" integer NOT NULL,
176     "buyer_id" integer NOT NULL,
177     "farmer_id" integer NOT NULL,
178     "rating" integer NOT NULL,
179     "comment" text,
180     "farmer_reply" text,
181     "replied_at" timestamp with time zone,
182     "created_at" timestamp with time zone DEFAULT now() NOT NULL
183 );
184
185 CREATE TABLE "users" (
186     "id" serial PRIMARY KEY NOT NULL,
187     "full_name" varchar(120) NOT NULL,
188     "email" varchar(160) NOT NULL,
189     "password_hash" varchar(255) NOT NULL,
190     "phone" varchar(30) NOT NULL,
191     "role" "user_role" NOT NULL,
192     "cnic_id" varchar(20),
193     "business_name" varchar(160),
194     "city" varchar(80) NOT NULL,
195     "address" text,

```

```

196     "is_verified" boolean DEFAULT false NOT NULL,
197     "email_verified" boolean DEFAULT false NOT NULL,
198     "title" varchar(120),
199     "user_code" varchar(40),
200     "avatar_url" text,
201     "google_id" varchar(64),
202     "auth_provider" varchar(20) DEFAULT 'password' NOT NULL,
203     "trust_score" numeric(3, 2) DEFAULT 0 NOT NULL,
204     "rating_avg" numeric(3, 2) DEFAULT 0 NOT NULL,
205     "rating_count" integer DEFAULT 0 NOT NULL,
206     "wallet_balance" numeric(14, 2) DEFAULT 0 NOT NULL,
207     "escrow_balance" numeric(14, 2) DEFAULT 0 NOT NULL,
208     "created_at" timestamp with time zone DEFAULT now() NOT NULL
209 );
210
211 CREATE TABLE "wallet_transactions" (
212     "txn_id" serial PRIMARY KEY NOT NULL,
213     "user_id" integer NOT NULL,
214     "order_id" integer,
215     "type" "wallet_txn_type" NOT NULL,
216     "amount" numeric(14, 2) NOT NULL,
217     "description" text,
218     "created_at" timestamp with time zone DEFAULT now() NOT NULL
219 );
220
221 ALTER TABLE "auth_tokens" ADD CONSTRAINT "auth_tokens_user_id_users_id_fk" FOREIGN KEY ("user_id") REFEREN
    ↵ CES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
222 ALTER TABLE "bids_negotiations" ADD CONSTRAINT "bids_negotiations_crop_id_crops_inventory_crop_id_fk" FORE
    ↵ IGN KEY ("crop_id") REFERENCES "public"."crops_inventory"("crop_id") ON DELETE cascade ON UPDATE no action
    ↵ ;
223 ALTER TABLE "bids_negotiations" ADD CONSTRAINT "bids_negotiations_buyer_id_users_id_fk" FOREIGN KEY ("buye
    ↵ r_id") REFERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
224 ALTER TABLE "crops_inventory" ADD CONSTRAINT "crops_inventory_farmer_id_users_id_fk" FOREIGN KEY ("farmer_
    ↵ id") REFERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
225 ALTER TABLE "db_audit_log" ADD CONSTRAINT "db_audit_log_admin_id_users_id_fk" FOREIGN KEY ("admin_id") REF
    ↵ ERENCES "public"."users"("id") ON DELETE set null ON UPDATE no action;
226 ALTER TABLE "disputes" ADD CONSTRAINT "disputes_order_id_orders_logistics_order_id_fk" FOREIGN KEY ("order
    ↵ _id") REFERENCES "public"."orders_logistics"("order_id") ON DELETE cascade ON UPDATE no action;
227 ALTER TABLE "disputes" ADD CONSTRAINT "disputes_raised_by_users_id_fk" FOREIGN KEY ("raised_by") REFERENCE
    ↵ S "public"."users"("id") ON DELETE restrict ON UPDATE no action;
228 ALTER TABLE "notifications" ADD CONSTRAINT "notifications_user_id_users_id_fk" FOREIGN KEY ("user_id") REF
    ↵ ERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
229 ALTER TABLE "order_events" ADD CONSTRAINT "order_events_order_id_orders_logistics_order_id_fk" FOREIGN KEY
    ↵ ("order_id") REFERENCES "public"."orders_logistics"("order_id") ON DELETE cascade ON UPDATE no action;
230 ALTER TABLE "order_events" ADD CONSTRAINT "order_events_actor_id_users_id_fk" FOREIGN KEY ("actor_id") REF
    ↵ ERENCES "public"."users"("id") ON DELETE set null ON UPDATE no action;
231 ALTER TABLE "orders_logistics" ADD CONSTRAINT "orders_logistics_bid_id_bids_negotiations_bid_id_fk" FOREIG
    ↵ N KEY ("bid_id") REFERENCES "public"."bids_negotiations"("bid_id") ON DELETE restrict ON UPDATE no action;
232 ALTER TABLE "orders_logistics" ADD CONSTRAINT "orders_logistics_crop_id_crops_inventory_crop_id_fk" FOREIG
    ↵ N KEY ("crop_id") REFERENCES "public"."crops_inventory"("crop_id") ON DELETE restrict ON UPDATE no action;
233 ALTER TABLE "orders_logistics" ADD CONSTRAINT "orders_logistics_farmer_id_users_id_fk" FOREIGN KEY ("farme
    ↵ r_id") REFERENCES "public"."users"("id") ON DELETE restrict ON UPDATE no action;
234 ALTER TABLE "orders_logistics" ADD CONSTRAINT "orders_logistics_buyer_id_users_id_fk" FOREIGN KEY ("buyer_
    ↵ id") REFERENCES "public"."users"("id") ON DELETE restrict ON UPDATE no action;
235 ALTER TABLE "payment_intents" ADD CONSTRAINT "payment_intents_user_id_users_id_fk" FOREIGN KEY ("user_id")
    ↵ REFERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
236 ALTER TABLE "payout_requests" ADD CONSTRAINT "payout_requests_farmer_id_users_id_fk" FOREIGN KEY ("farmer_
    ↵ id") REFERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
237 ALTER TABLE "quality_inspections" ADD CONSTRAINT "quality_inspections_crop_id_crops_inventory_crop_id_fk"
    ↵ FOREIGN KEY ("crop_id") REFERENCES "public"."crops_inventory"("crop_id") ON DELETE cascade ON UPDATE no ac
    ↵ tion;
238 ALTER TABLE "quality_inspections" ADD CONSTRAINT "quality_inspections_inspector_id_users_id_fk" FOREIGN KE
    ↵ Y ("inspector_id") REFERENCES "public"."users"("id") ON DELETE restrict ON UPDATE no action;
239 ALTER TABLE "reviews" ADD CONSTRAINT "reviews_order_id_orders_logistics_order_id_fk" FOREIGN KEY ("order_i
    ↵ d") REFERENCES "public"."orders_logistics"("order_id") ON DELETE cascade ON UPDATE no action;
240 ALTER TABLE "reviews" ADD CONSTRAINT "reviews_buyer_id_users_id_fk" FOREIGN KEY ("buyer_id") REFERENCES "p
    ↵ ublic"."users"("id") ON DELETE cascade ON UPDATE no action;
241 ALTER TABLE "reviews" ADD CONSTRAINT "reviews_farmer_id_users_id_fk" FOREIGN KEY ("farmer_id") REFERENCES
    ↵ "public"."users"("id") ON DELETE cascade ON UPDATE no action;
242 ALTER TABLE "wallet_transactions" ADD CONSTRAINT "wallet_transactions_user_id_users_id_fk" FOREIGN KEY ("u
    ↵ ser_id") REFERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
243 ALTER TABLE "wallet_transactions" ADD CONSTRAINT "wallet_transactions_order_id_orders_logistics_order_id_f
    ↵ k" FOREIGN KEY ("order_id") REFERENCES "public"."orders_logistics"("order_id") ON DELETE set null ON UPDAT
    ↵ E no action;
244 CREATE INDEX "auth_tokens_user_idx" ON "auth_tokens" USING btree ("user_id", "purpose");
245 CREATE INDEX "bids_crop_idx" ON "bids_negotiations" USING btree ("crop_id");

```

```

246 CREATE INDEX "bids_buyer_idx" ON "bids_negotiations" USING btree ("buyer_id");
247 CREATE INDEX "bids_status_idx" ON "bids_negotiations" USING btree ("status");
248 CREATE INDEX "crops_farmer_idx" ON "crops_inventory" USING btree ("farmer_id");
249 CREATE INDEX "crops_category_idx" ON "crops_inventory" USING btree ("category");
250 CREATE INDEX "crops_status_idx" ON "crops_inventory" USING btree ("status");
251 CREATE INDEX "crops_name_idx" ON "crops_inventory" USING btree ("crop_name");
252 CREATE INDEX "disputes_order_idx" ON "disputes" USING btree ("order_id");
253 CREATE INDEX "disputes_status_idx" ON "disputes" USING btree ("status");
254 CREATE UNIQUE INDEX "mandi_crop_market_date_uq" ON "mandi_rates" USING btree ("crop_name","market_location",
  ~ "date_updated");
255 CREATE INDEX "mandi_date_idx" ON "mandi_rates" USING btree ("date_updated");
256 CREATE INDEX "notifications_user_idx" ON "notifications" USING btree ("user_id","is_read");
257 CREATE INDEX "order_events_order_idx" ON "order_events" USING btree ("order_id");
258 CREATE UNIQUE INDEX "orders_bid_uq" ON "orders_logistics" USING btree ("bid_id");
259 CREATE UNIQUE INDEX "orders_tracking_uq" ON "orders_logistics" USING btree ("tracking_number");
260 CREATE INDEX "orders_farmer_idx" ON "orders_logistics" USING btree ("farmer_id");
261 CREATE INDEX "orders_buyer_idx" ON "orders_logistics" USING btree ("buyer_id");
262 CREATE INDEX "orders_stage_idx" ON "orders_logistics" USING btree ("delivery_stage");
263 CREATE UNIQUE INDEX "payment_intents_ref_uq" ON "payment_intents" USING btree ("reference");
264 CREATE INDEX "payment_intents_user_idx" ON "payment_intents" USING btree ("user_id");
265 CREATE INDEX "payout_requests_farmer_idx" ON "payout_requests" USING btree ("farmer_id");
266 CREATE INDEX "payout_requests_status_idx" ON "payout_requests" USING btree ("status");
267 CREATE INDEX "inspections_crop_idx" ON "quality_inspections" USING btree ("crop_id");
268 CREATE INDEX "inspections_status_idx" ON "quality_inspections" USING btree ("status");
269 CREATE UNIQUE INDEX "reviews_order_uq" ON "reviews" USING btree ("order_id");
270 CREATE INDEX "reviews_farmer_idx" ON "reviews" USING btree ("farmer_id");
271 CREATE UNIQUE INDEX "users_email_uq" ON "users" USING btree ("email");
272 CREATE UNIQUE INDEX "users_google_id_uq" ON "users" USING btree ("google_id");
273 CREATE UNIQUE INDEX "users_user_code_uq" ON "users" USING btree ("user_code");
274 CREATE INDEX "users_role_idx" ON "users" USING btree ("role");
275 CREATE INDEX "users_city_idx" ON "users" USING btree ("city");
276 CREATE INDEX "wallet_user_idx" ON "wallet_transactions" USING btree ("user_id");
277

```

MySQL 8 DDL (portable equivalent)

```

1  -- MarketLink Agri-Hub · MySQL 8.0 compatible DDL (InnoDB, utf8mb4)
2  -- Production runtime uses PostgreSQL (see PostgreSQL tab); this script is the portable equivalent.
3
4  CREATE TABLE users (
5      id                INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
6      full_name         VARCHAR(120) NOT NULL,
7      email             VARCHAR(160) NOT NULL,
8      password_hash     VARCHAR(255) NOT NULL,
9      phone             VARCHAR(30) NOT NULL,
10     role               ENUM('farmer','buyer','admin','inspector') NOT NULL,
11     cnic_id            VARCHAR(20) NULL,
12     business_name      VARCHAR(160) NULL,
13     city               VARCHAR(80) NOT NULL,
14     address            TEXT NULL,
15     is_verified        BOOLEAN NOT NULL DEFAULT FALSE,
16     email_verified     BOOLEAN NOT NULL DEFAULT FALSE,
17     title              VARCHAR(120) NULL, -- e.g. 'Super Admin & Systems Controller'
18     user_code          VARCHAR(40) NULL, -- e.g. 'admin_01'
19     avatar_url         TEXT NULL,
20     google_id          VARCHAR(64) NULL, -- Google OAuth subject
21     auth_provider       VARCHAR(20) NOT NULL DEFAULT 'password',
22     trust_score         DECIMAL(3,2) NOT NULL DEFAULT 0,
23     rating_avg         DECIMAL(3,2) NOT NULL DEFAULT 0,
24     rating_count        INT NOT NULL DEFAULT 0,
25     wallet_balance      DECIMAL(14,2) NOT NULL DEFAULT 0,
26     escrow_balance      DECIMAL(14,2) NOT NULL DEFAULT 0,
27     created_at          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
28     UNIQUE KEY users_email_uq (email),
29     UNIQUE KEY users_google_id_uq (google_id),
30     UNIQUE KEY users_user_code_uq (user_code),
31     KEY users_role_idx (role),
32     KEY users_city_idx (city)
33 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
34
35 CREATE TABLE crops_inventory (
36     crop_id            INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
37     farmer_id          INT UNSIGNED NOT NULL,
38     crop_name          VARCHAR(80) NOT NULL,
39     category           ENUM('grains','vegetables','fruits','cash_crops') NOT NULL,

```

```

40     quality_grade          ENUM('A','B','C') NOT NULL,
41     total_quantity_kg     DECIMAL(14,2) NOT NULL CHECK (total_quantity_kg >= 0),
42     base_price_per_kg     DECIMAL(14,2) NOT NULL CHECK (base_price_per_kg > 0),
43     harvest_date          DATE          NOT NULL,
44     images_json            JSON          NOT NULL,
45     description            TEXT          NULL,
46     farm_location          VARCHAR(80)   NOT NULL,
47     latitude               DOUBLE        NULL,
48     longitude              DOUBLE        NULL,
49     status                 ENUM('active','pending_inspection','sold_out','archived') NOT NULL DEFAULT 'active',
50     created_at             TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
51     KEY crops_farmer_idx (farmer_id),
52     KEY crops_category_idx (category),
53     KEY crops_status_idx (status),
54     KEY crops_name_idx (crop_name),
55     CONSTRAINT fk_crops_farmer FOREIGN KEY (farmer_id) REFERENCES users(id) ON DELETE CASCADE
56 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
57
58 CREATE TABLE mandi_rates (
59     rate_id                INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
60     crop_name               VARCHAR(80)   NOT NULL,
61     market_location        VARCHAR(80)   NOT NULL,
62     min_price_per_kg       DECIMAL(14,2) NOT NULL,
63     max_price_per_kg       DECIMAL(14,2) NOT NULL,
64     avg_price_per_kg       DECIMAL(14,2) NOT NULL,
65     source                  VARCHAR(20)   NOT NULL DEFAULT 'manual',
66     date_updated            DATE          NOT NULL,
67     UNIQUE KEY mandi_crop_market_date_uq (crop_name, market_location, date_updated),
68     KEY mandi_date_idx (date_updated),
69     CHECK (min_price_per_kg <= avg_price_per_kg AND avg_price_per_kg <= max_price_per_kg)
70 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
71
72 CREATE TABLE bids_negotiations (
73     bid_id                 INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
74     crop_id                INT UNSIGNED NOT NULL,
75     buyer_id              INT UNSIGNED NOT NULL,
76     bid_price_per_kg       DECIMAL(14,2) NOT NULL,
77     bid_quantity_kg        DECIMAL(14,2) NOT NULL,
78     target_delivery_date   DATE          NULL,
79     message                TEXT          NULL,
80     status                 ENUM('pending','accepted','rejected','countered','withdrawn') NOT NULL DEFAULT 'pending',
81     counter_price           DECIMAL(14,2) NULL,
82     counter_note            TEXT          NULL,
83     created_at              TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
84     updated_at              TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
85     KEY bids_crop_idx (crop_id),
86     KEY bids_buyer_idx (buyer_id),
87     KEY bids_status_idx (status),
88     CONSTRAINT fk_bids_crop FOREIGN KEY (crop_id) REFERENCES crops_inventory(crop_id) ON DELETE CASCADE,
89     CONSTRAINT fk_bids_buyer FOREIGN KEY (buyer_id) REFERENCES users(id) ON DELETE CASCADE
90 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
91
92 CREATE TABLE orders_logistics (
93     order_id               INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
94     bid_id                 INT UNSIGNED NOT NULL,
95     crop_id                INT UNSIGNED NOT NULL,
96     farmer_id              INT UNSIGNED NOT NULL,
97     buyer_id              INT UNSIGNED NOT NULL,
98     quantity_kg            DECIMAL(14,2) NOT NULL,
99     price_per_kg           DECIMAL(14,2) NOT NULL,
100    total_amount            DECIMAL(14,2) NOT NULL,
101    payment_status          ENUM('awaiting_escrow','escrow_locked','released','refunded','disputed') NOT NULL
102    DEFAULT 'awaiting_escrow',
103    delivery_stage          ENUM('confirmed','quality_checked','dispatched','in_transit','delivered') NOT NULL
104    DEFAULT 'confirmed',
105    transit_location        VARCHAR(120)   NULL,
106    current_lat             DOUBLE          NULL,
107    current_lng             DOUBLE          NULL,
108    tracking_number         VARCHAR(40)     NOT NULL,
109    expected_delivery_date  DATE            NULL,
110    created_at              TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
111    updated_at              TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
112    UNIQUE KEY orders_bid_uq (bid_id),
113    UNIQUE KEY orders_tracking_uq (tracking_number),
114    KEY orders_farmer_idx (farmer_id),

```

```

113     KEY orders_buyer_idx (buyer_id),
114     KEY orders_stage_idx (delivery_stage),
115     CONSTRAINT fk_orders_bid FOREIGN KEY (bid_id) REFERENCES bids_negotiations(bid_id),
116     CONSTRAINT fk_orders_crop FOREIGN KEY (crop_id) REFERENCES crops_inventory(crop_id),
117     CONSTRAINT fk_orders_farmer FOREIGN KEY (farmer_id) REFERENCES users(id),
118     CONSTRAINT fk_orders_buyer FOREIGN KEY (buyer_id) REFERENCES users(id)
119 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
120
121 CREATE TABLE quality_inspections (
122     inspection_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
123     crop_id INT UNSIGNED NOT NULL,
124     inspector_id INT UNSIGNED NOT NULL,
125     grade_assigned ENUM('A','B','C') NOT NULL,
126     moisture_level_percentage DECIMAL(5,2) NOT NULL,
127     soil_ph DECIMAL(4,2) NULL,
128     inspection_notes TEXT NULL,
129     report_attachment LONGTEXT NULL,
130     report_file_name VARCHAR(200) NULL,
131     certificate_no VARCHAR(40) NULL,
132     status ENUM('pending','passed','failed') NOT NULL DEFAULT 'pending',
133     verified_at TIMESTAMP NULL,
134     KEY inspections_crop_idx (crop_id),
135     KEY inspections_status_idx (status),
136     CONSTRAINT fk_insp_crop FOREIGN KEY (crop_id) REFERENCES crops_inventory(crop_id) ON DELETE CA
137     SCADE,
138     CONSTRAINT fk_insp_inspector FOREIGN KEY (inspector_id) REFERENCES users(id)
139 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
140
141 -- Supporting tables: order_events (tracking audit), wallet_transactions (escrow ledger), disputes
142 CREATE TABLE order_events (
143     event_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
144     order_id INT UNSIGNED NOT NULL,
145     stage VARCHAR(40) NOT NULL,
146     location VARCHAR(120) NULL,
147     lat DOUBLE NULL,
148     lng DOUBLE NULL,
149     note TEXT NULL,
150     actor_id INT UNSIGNED NULL,
151     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
152     KEY order_events_order_idx (order_id),
153     CONSTRAINT fk_events_order FOREIGN KEY (order_id) REFERENCES orders_logistics(order_id) ON DELETE CASCAD
154     E,
155     CONSTRAINT fk_events_actor FOREIGN KEY (actor_id) REFERENCES users(id) ON DELETE SET NULL
156 ) ENGINE=InnoDB;
157
158 CREATE TABLE wallet_transactions (
159     txn_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
160     user_id INT UNSIGNED NOT NULL,
161     order_id INT UNSIGNED NULL,
162     type ENUM('deposit','escrow_lock','escrow_release','payout','refund','platform_fee','withdrawal')
163     NOT NULL,
164     amount DECIMAL(14,2) NOT NULL,
165     description TEXT NULL,
166     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
167     KEY wallet_user_idx (user_id),
168     CONSTRAINT fk_wallet_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
169     CONSTRAINT fk_wallet_order FOREIGN KEY (order_id) REFERENCES orders_logistics(order_id) ON DELETE SET NU
170     LL
171 ) ENGINE=InnoDB;
172
173 CREATE TABLE disputes (
174     dispute_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
175     order_id INT UNSIGNED NOT NULL,
176     raised_by INT UNSIGNED NOT NULL,
177     reason TEXT NOT NULL,
178     status ENUM('open','investigating','resolved_release','resolved_refund') NOT NULL DEFAULT 'open',
179     resolution TEXT NULL,
180     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
181     resolved_at TIMESTAMP NULL,
182     KEY disputes_order_idx (order_id),
183     KEY disputes_status_idx (status),
184     CONSTRAINT fk_disputes_order FOREIGN KEY (order_id) REFERENCES orders_logistics(order_id) ON DELETE CASC
185     ADE,
186     CONSTRAINT fk_disputes_user FOREIGN KEY (raised_by) REFERENCES users(id)
187 ) ENGINE=InnoDB;

```

```

184 CREATE TABLE reviews (
185     review_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
186     order_id INT UNSIGNED NOT NULL,
187     buyer_id INT UNSIGNED NOT NULL,
188     farmer_id INT UNSIGNED NOT NULL,
189     rating TINYINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
190     comment TEXT NULL,
191     farmer_reply TEXT NULL,
192     replied_at TIMESTAMP NULL,
193     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
194     UNIQUE KEY reviews_order_uq (order_id),
195     KEY reviews_farmer_idx (farmer_id),
196     CONSTRAINT fk_reviews_order FOREIGN KEY (order_id) REFERENCES orders_logistics(order_id) ON DELETE CAS
197     CADE,
198     CONSTRAINT fk_reviews_buyer FOREIGN KEY (buyer_id) REFERENCES users(id) ON DELETE CASCADE,
199     CONSTRAINT fk_reviews_farmer FOREIGN KEY (farmer_id) REFERENCES users(id) ON DELETE CASCADE
200 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
201
202 CREATE TABLE notifications (
203     notification_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
204     user_id INT UNSIGNED NOT NULL,
205     type VARCHAR(40) NOT NULL,
206     title VARCHAR(200) NOT NULL,
207     body TEXT NULL,
208     link VARCHAR(300) NULL,
209     channels VARCHAR(60) NOT NULL DEFAULT 'in_app',
210     is_read BOOLEAN NOT NULL DEFAULT FALSE,
211     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
212     KEY notifications_user_idx (user_id, is_read),
213     CONSTRAINT fk_notifications_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
214 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
215
216 CREATE TABLE auth_tokens (
217     token_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
218     user_id INT UNSIGNED NOT NULL,
219     purpose VARCHAR(20) NOT NULL, -- verify_email | reset_link (15-min password reset token)
220     code_hash VARCHAR(128) NOT NULL, -- SHA-256 of the 6-digit code + secret
221     attempts INT NOT NULL DEFAULT 0,
222     expires_at TIMESTAMP NOT NULL,
223     used_at TIMESTAMP NULL,
224     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
225     KEY auth_tokens_user_idx (user_id, purpose),
226     CONSTRAINT fk_auth_tokens_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
227 ) ENGINE=InnoDB;
228
229 CREATE TABLE payment_intents (
230     intent_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
231     reference VARCHAR(60) NOT NULL,
232     user_id INT UNSIGNED NOT NULL,
233     provider VARCHAR(20) NOT NULL, -- stripe | jazzcash | easypaisa | sandbox
234     amount DECIMAL(14,2) NOT NULL,
235     status VARCHAR(20) NOT NULL DEFAULT 'pending',
236     provider_ref VARCHAR(200) NULL,
237     raw_callback TEXT NULL,
238     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
239     completed_at TIMESTAMP NULL,
240     UNIQUE KEY payment_intents_ref_uq (reference),
241     KEY payment_intents_user_idx (user_id),
242     CONSTRAINT fk_payment_intents_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
243 ) ENGINE=InnoDB;
244
245 CREATE TABLE payout_requests (
246     payout_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
247     farmer_id INT UNSIGNED NOT NULL,
248     amount DECIMAL(14,2) NOT NULL,
249     method VARCHAR(20) NOT NULL, -- bank | jazzcash | easypaisa
250     account_title VARCHAR(120) NOT NULL,
251     account_number VARCHAR(60) NOT NULL,
252     status VARCHAR(20) NOT NULL DEFAULT 'pending',
253     admin_note TEXT NULL,
254     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
255     processed_at TIMESTAMP NULL,
256     KEY payout_requests_farmer_idx (farmer_id),
257     KEY payout_requests_status_idx (status),
258     CONSTRAINT fk_payout_farmer FOREIGN KEY (farmer_id) REFERENCES users(id) ON DELETE CASCADE
259 ) ENGINE=InnoDB;

```

```

259
260 CREATE TABLE db_audit_log (
261     audit_id    INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
262     admin_id    INT UNSIGNED NULL,
263     mode        VARCHAR(10) NOT NULL,
264     query       TEXT        NOT NULL,
265     row_count   INT          NULL,
266     duration_ms INT          NULL,
267     error       TEXT        NULL,
268     created_at  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
269     CONSTRAINT fk_audit_admin FOREIGN KEY (admin_id) REFERENCES users(id) ON DELETE SET NULL
270 ) ENGINE=InnoDB;
271

```

3 · REST API Endpoint Specification

Authentication uses the httpOnly ml_session cookie set by /api/auth/login. Responses: { "success": true, "data": ... } or { "success": false, "error": "message" }.

Errors: 400 bad JSON · 401 unauthenticated · 402 insufficient balance · 403 wrong role / not owner · 404 not found · 409 invalid state transition · 422 validation.

Authentication

POST /api/auth/register

Roles: public

Register a farmer or buyer. Sets httpOnly session cookie.

REQUEST

```
{ "fullName": "Ali Raza", "email": "ali@farm.pk", "password": "*****", "phone": "+92 300 1234567", "role": "farmer", "city": "Multan", "cnicId": "36302-1234567-1", "businessName": "Raza Farms" }
```

RESPONSE

```
{ "success": true, "data": { "id": 11, "role": "farmer", "redirectTo": "/farmer" } }
```

POST /api/auth/login

Roles: public

Email + password login (scrypt hash, HMAC-signed cookie).

REQUEST

```
{ "email": "farmer@marketlink.pk", "password": "password123" }
```

RESPONSE

```
{ "success": true, "data": { "id": 2, "fullName": "Muhammad Aslam", "role": "farmer", "redirectTo": "/farmer" } }
```

POST /api/auth/logout

Roles: any

Clears the session cookie.

GET /api/auth/me

Roles: any

Current user profile (no password hash).

Crop Inventory

GET /api/crops?category=&grade=&minPrice=&maxPrice=&q=&originCity=&radiusKm=&sort=&mine=1

Roles: any

Marketplace catalogue with haversine radius filtering, open-bid counts, highest bid and inspection grade.

RESPONSE

```
{ "success": true, "data": [{ "cropId": 1, "cropName": "Wheat", "category": "grains", "qualityGrade": "A", "totalQuantityKg": 24000, "basePricePerKg": 96, "farmLocation": "Multan", "distanceKm": 312, "openBids": 2, "highestBid": 94, "farmerVerified": true } ] }
```

POST /api/crops

Roles: farmer

Create listing. Unverified farmers' listings enter pending_inspection.

REQUEST

```
{ "cropName": "Wheat", "category": "grains", "qualityGrade": "A", "totalQuantityKg": 5000, "basePricePerKg": 96.5, "harvestDate": "2025-04-20", "farmLocation": "Multan", "description": "Galaxy-2013", "imagesJson": [ "data:image/jpeg;base64,..." ] }
```

RESPONSE

```
{ "success": true, "data": { "cropId": 14, "status": "active", ... } }
```

GET /api/crops/:id

Roles: any

Listing detail with inspection history.

PATCH /api/crops/:id

Roles: farmer (owner), admin

Update price, quantity, photos, status (active|archived).

REQUEST

```
{ "basePricePerKg": 98, "totalQuantityKg": 20000 }
```

DELETE /api/crops/:id

Roles: farmer (owner), admin

Hard delete, or soft-archive if orders reference it (open bids auto-rejected).

RESPONSE

```
{ "success": true, "data": { "archived": true } }
```

Bidding & Negotiation

GET /api/bids?status=&cropId=

Roles: farmer | buyer | admin

Bids scoped to role (farmer: bids on own crops; buyer: own bids).

POST /api/bids

Roles: buyer

Submit bulk offer / RFQ. Validates stock, lowball guard ($\geq 50\%$ of ask), one open bid per crop per buyer.

REQUEST

```
{ "cropId": 1, "bidPricePerKg": 92, "bidQuantityKg": 10000, "targetDeliveryDate": "2025-06-01", "message": "Pickup from farm" }
```

RESPONSE

```
{ "success": true, "data": { "bidId": 41, "status": "pending", ... } }
```

PATCH /api/bids/:id

Roles: farmer | buyer

State machine action: accept | reject | counter (farmer) · accept_counter | reject_counter | revise | withdraw (buyer). Accepting creates an order atomically.

REQUEST

```
{ "action": "counter", "counterPrice": 98, "note": "Includes bagging" }
```

RESPONSE

```
{ "success": true, "data": { "bid": { "status": "accepted" }, "order": { "orderId": 37, "trackingNumber": "ML-2025-1038-X7K", "paymentStatus": "awaiting_escrow" } } }
```

Orders, Logistics & Escrow

GET /api/orders?active=1

Roles: any

Orders with joined crop/party info, tracking events and disputes.

GET /api/orders/:id

Roles: party | admin

Single order detail.

PATCH /api/orders/:id

Roles: role-dependent

lock_escrow (buyer) · advance (admin→quality_checked, farmer→dispatched/in_transit, buyer→delivered + auto escrow release) · update_location (farmer).

REQUEST

```
{ "action": "advance", "location": "Khanewal Bypass, N-5", "note": "Truck LES-4521" }
```

RESPONSE

```
{ "success": true, "data": { "orderId": 1, "deliveryStage": "in_transit", "paymentStatus": "escrow_locked" } }
```

POST /api/orders/:id/dispute

Roles: buyer | farmer

Freeze escrow and open a dispute.

REQUEST

```
{ "reason": "Moisture 17% vs agreed 14%" }
```

GET /api/wallet

Roles: farmer | buyer

Balances + ledger.

RESPONSE

```
{ "success": true, "data": { "walletBalance": 4500000, "escrowBalance": 1044000, "pendingInEscrow": 1044000, "transactions": [...] } }
```

POST /api/wallet

Roles: buyer

Simulated bank top-up.

REQUEST

```
{ "amount": 500000 }
```

Mandi Rates

GET /api/mandi-rates?view=latest&crop=&market=

Roles: any

Latest rate per crop × mandi with day-on-day % change (window function).

GET /api/mandi-rates?view=trend&crop=Wheat&days=14

Roles: any

Pivoted daily series for charting.

RESPONSE

```
{ "success": true, "data": { "cropName": "Wheat", "markets": ["Lahore", ...], "series": [{ "date": "2025-05-01", "Lahore": 98.2, "Multan": 91.4 }] } }
```

GET /api/mandi-rates?view=compare

Roles: any

Mandi avg vs platform ask vs realised platform deal price per crop.

POST /api/mandi-rates

Roles: admin

Manual upsert (unique crop+market+date).

REQUEST

```
{ "cropName": "Wheat", "marketLocation": "Lahore", "minPricePerKg": 92, "maxPricePerKg": 101, "dateUpdated": "2025-05-14" }
```

POST /api/mandi-rates/sync

Roles: admin

Mock government feed sync (bounded $\pm 3\%$ random walk, source=feed).

RESPONSE

```
{ "success": true, "data": { "updated": 60, "date": "2025-05-14" } }
```

Payments, Payouts & Reviews

GET /api/payments/checkout

Roles: buyer | farmer

Enabled gateways (stripe / jazzcash / easypaisa / sandbox) and their mode.

POST /api/payments/checkout

Roles: buyer

Create a payment intent; returns a redirect URL (Stripe Checkout / sandbox) or an auto-submit form (JazzCash / Easypaisa). The wallet is credited only by a verified callback.

REQUEST

```
{ "provider": "stripe", "amount": 500000 }
```

RESPONSE

```
{ "success": true, "data": { "reference": "MLT1790...", "redirectUrl": "https://checkout.stripe.com/..." } }
```

POST /api/payments/webhook/stripe

Roles: Stripe (signed)

Stripe webhook; verifies Stripe-Signature (HMAC-SHA256, 5-min tolerance) and settles checkout.session.* events idempotently.

POST /api/payments/callback/jazzcash

Roles: JazzCash (signed)

JazzCash return URL; verifies pp_SecureHash with the integrity salt, pp_ResponseCode 000 = success.

GET /api/payments/callback/easypaisa

Roles: Easypaisa

Easypaisa postBack; result is confirmed server-to-server via Inquire Transaction before crediting.

GET /api/payments/:reference

Roles: owner

Poll payment status after returning from the gateway.

POST /api/payouts

Roles: farmer

Withdraw wallet balance to bank / JazzCash / Easypaisa.

REQUEST

```
{ "amount": 50000, "method": "jazzcash", "accountTitle": "Bashir Soomro", "accountNumber": "03451234567" }
```

PATCH /api/payouts/:id

Roles: admin

Mark payout paid, or reject (funds returned).

REQUEST

```
{ "status": "paid" }
```

POST /api/reviews

Roles: buyer

Rate the seller of a delivered & paid order (1 per order); recalculates the seller trust score.

REQUEST

```
{ "orderId": 33, "rating": 5, "comment": "Fresh onions, perfect weight" }
```

RESPONSE

```
{ "success": true, "data": { "trust": { "trustScore": 4.41, "ratingAvg": 4.6, "ratingCount": 5 } } }
```

GET /api/reviews?farmerId=2 · ?pending=1

Roles: public · buyer

Seller reviews (including farmer replies), or the buyer's delivered orders awaiting review.

POST /api/reviews/:id/reply

Roles: farmer (seller on that order)

Respond once to a verified buyer review. Buyer receives an in-app notification; response appears on crop details.

REQUEST

```
{ "reply": "Thank you for your feedback. We look forward to supplying you again." }
```

GET /api/reports/summary

Roles: farmer | admin

Download a live, role-scoped CSV with KPIs, six monthly periods, and crop or regional breakdowns.

Buyers and inspectors cannot access it.

Tracking, Notifications & Auth

GET /api/tracking/:trackingNumber

Roles: public

Route, live/estimated GPS position, progress, ETA and milestones. Party names & notes only for order parties/staff. Fires the 'arriving soon' milestone at 85%.

POST /api/orders/:id/gps

Roles: farmer | admin

Driver GPS ping while dispatched / in transit.

REQUEST

```
{ "lat": 25.9, "lng": 68.1, "label": "M-9 near Nooriabad" }
```

GET /api/notifications

Roles: any

Latest 30 notifications + unread count.

PATCH /api/notifications

Roles: any

Mark given ids (or all) as read.

REQUEST

```
{ "ids": [12, 13] }
```

POST /api/auth/verify-email

Roles: public

Verify the 6-digit signup code (signs in), or { resend: true }.

REQUEST

```
{ "email": "ali@farm.pk", "code": "816958" }
```

POST /api/auth/forgot-password

Roles: public

Generates a single-use reset token (15-minute expiry, stored as SHA-256) and emails a /reset-password?token=... link. Same response whether or not the account exists.

REQUEST

```
{ "email": "buyer@marketlink.pk" }
```

RESPONSE

```
{ "success": true, "data": { "sent": true, "message": "If an account exists..." } }
```

GET /api/auth/reset-password?token=...

Roles: public

Checks a reset link before the form is shown (masked email + expiry).

POST /api/auth/reset-password

Roles: public

Validates the token, stores the new password as a bcrypt hash and deletes the token.

REQUEST

```
{ "token": "...", "newPassword": "NewPass#2026" }
```

GET /api/auth/google?role=buyer|farmer

Roles: public

Starts Google OAuth 2.0 (authorization code + PKCE, signed state cookie) → Google consent screen.

GET /api/auth/google/callback

Roles: Google

Verifies state, exchanges the code, requires a verified Google email; signs in existing users (linking Google) or creates a new buyer/farmer with name, email and profile picture.

PATCH /api/account/profile

Roles: any

Update name, phone, city, business, CNIC, address (used to complete Google sign-ups).

REQUEST

```
{ "fullName": "Ali Raza", "phone": "0300 1234567", "city": "Multan" }
```

Database Explorer (admin)

GET /api/db/schema

Roles: admin

Tables, columns, PK/FK, indexes, row counts and recent audit log.

POST /api/db/query

Roles: admin

Single SQL statement. read = READ ONLY transaction + 8s timeout + 500-row cap; write requires confirm: true. Dangerous statements blocked; every query audited in db_audit_log.

REQUEST

```
{ "sql": "SELECT full_name, trust_score FROM users WHERE role = 'farmer'", "mode": "read" }
```

RESPONSE

```
{ "success": true, "data": { "columns": ["full_name", "trust_score"], "rows": [{"Ghulam Rasool", 4.53}], "rowCount": 1, "durationMs": 4 } }
```

AI Shopping Assistant

POST /api/assistant

Roles: public (visitors + buyers)

LLM assistant (OpenAI or Gemini when OPENAI_API_KEY / GEMINI_API_KEY is set; rule engine otherwise) grounded in live DB results: top-rated dealers (rating from deliveries, verification, inspections, disputes), high-quality listings by crop/category, cheapest/nearest produce, mandi rates and FAQ (escrow, bidding, tracking). Understands English + Roman Urdu. Rate-limited to 30 msgs/min.

REQUEST

```
{ "message": "Where can I find top-rated dealers?" }
```

RESPONSE

```
{ "success": true, "data": { "reply": "Top-rated sellers...", "cards": [{ "kind": "dealer", "name": "Muhammad Aslam", "business": "Aslam Agri Farms", "city": "Multan", "rating": 4.6, "deliveries": 8, "href": "/buyer?farmerId=2" }], "suggestions": ["High-quality vegetables", "Cheapest wheat"] } }
```

Admin & Quality

GET /api/admin/users?role=farmer

Roles: admin

Users with listing/order counts.

PATCH /api/admin/users/:id

Roles: admin

Verify / revoke. Verifying a farmer activates held listings.

REQUEST

```
{ "isVerified": true }
```

RESPONSE

```
{ "success": true, "data": { "id": 4, "isVerified": true, "listingsActivated": 1 } }
```

GET /api/inspections

Roles: admin

Inspection report log.

POST /api/inspections

Roles: admin

Upload soil/crop inspection. passed → grade updated & listing active; failed → archived & bids rejected.

REQUEST

```
{ "cropId": 7, "gradeAssigned": "B", "moistureLevelPercentage": 68, "soilPh": 7.8, "inspectionNotes": "Brix 19.2", "status": "passed", "reportAttachment": "data:application/pdf;base64,..." }
```

GET /api/inspections/:id/report

Roles: admin

Download attached report file.

GET /api/admin/disputes

Roles: admin

Dispute resolution log.

PATCH /api/admin/disputes/:id

Roles: admin

investigating | release (pay farmer) | refund (return to buyer).

REQUEST

```
{ "outcome": "refund", "resolution": "Lab retest confirmed 17% moisture" }
```

GET **/api/analytics**

Roles: any

Role-aware KPIs: farmer earnings, buyer spend, or platform-wide volume/regions/stages.

GET **/api/health**

Roles: public

DB connectivity healthcheck.

4 · Business Logic Workflows

Role-Based Marketplace Flow

- 1 Authenticate with password or Google OAuth; the signed session determines whether the user enters Admin, Farmer/Vendor, Buyer/Customer or Inspector modules. Every page and API repeats the role/ownership check.
- 2 Farmer: manage profile and graded crop listings → review and counter bids → fulfil accepted orders → receive escrow payout → respond once to each verified buyer review.
- 3 Buyer: search the marketplace and inspect crop/quality details → place bids and negotiate → fund escrow after acceptance → track shipment → confirm delivery and submit a verified rating.
- 4 Inspector: certify crop quality and record moisture, soil pH, grade and report before dispatch. Admin: approve users, monitor bids and transactions, manage mandi rates, and resolve disputes.
- 5 Data processing: accepting a bid creates an order and decrements inventory transactionally; escrow lock precedes inspection, dispatch and tracking; verified delivery releases payment and enables a review.
- 6 Output: Admin and Farmer dashboards show live metrics and charts; their role-scoped CSV summary downloads reflect current sales, orders and trading regions/crops.

Bidding & Negotiation

- 1 Buyer submits bid (price, qty, target date). Guards: listing active, $qty \leq stock$, $price \geq 50\%$ of ask, one open bid per buyer per listing.
- 2 Farmer reviews in Negotiation Hub drawer → Accept, Reject, or Counter (counter must exceed bid price).
- 3 On counter: buyer may accept_counter, reject_counter, or revise (bid returns to pending with new price) — unlimited rounds.
- 4 On acceptance (SELECT ... FOR UPDATE on bid + crop): stock decremented, listing → sold_out at 0, other open bids exceeding remaining stock auto-rejected, order created with unique tracking number in 'awaiting_escrow'.

Escrow & Order Status Progression

- 1 Buyer locks full order value: $wallet_balance -= total$, $escrow_balance += total$, ledger entry 'escrow_lock'. Nothing progresses before this.
- 2 Quality inspector records pre-dispatch inspection → stage 'quality_checked' (creates a quality_inspections row).
- 3 Farmer marks 'dispatched' then 'in_transit', posting checkpoint locations (order_events audit trail).
- 4 Buyer confirms delivery → 'delivered'; escrow released atomically: farmer credited total – 1.5% platform fee, ledger entries $payout + platform_fee + escrow_release$.
- 5 Either party can raise a dispute while escrow is locked → progression frozen; admin resolves by release (farmer paid) or refund (buyer credited).

Mandi Rate Sync

- 1 Rates are stored per (crop, market, date) with a UNIQUE constraint → all writes are idempotent upserts.
- 2 Admin manual entry: validates $\min \leq \text{avg} \leq \max$ (avg defaults to midpoint), source='manual'.
- 3 Mock feed sync: reads the latest rate for every crop × mandi, applies a bounded $\pm 3\%$ random walk, upserts today's row with source='feed' in one transaction; missing pairs are seeded from catalogue base prices.
- 4 Insights: day-on-day change via ROW_NUMBER() window; platform comparison joins latest mandi avg vs active listing ask vs 90-day realised deal price; farmer listing form suggests grade-adjusted price (A ×1.06, B ×1.00, C ×0.90).

Verification & Quality

- 1 New farmers register unverified; their listings enter 'pending_inspection' and are hidden from the marketplace.
- 2 Admin verifies CNIC/farm → farmer verified and held listings activated.
- 3 Inspection passed → listing grade overwritten with assigned grade and activated; failed → listing archived and open bids rejected; pending → held.

State machines

BID

```
pending —farmer:accept→ accepted → order (awaiting_escrow)
pending —farmer:counter→ countered
countered —buyer:accept_counter→ accepted → order @ counter price
countered —buyer:reject_counter→ rejected
countered —buyer:revise→ pending (new price)
pending —farmer:reject→ rejected
pending|countered —buyer:withdraw→ withdrawn
```

ORDER / LOGISTICS

```
confirmed —(buyer locks escrow)→ quality_checked [inspector]
    → dispatched [farmer] → in_transit [farmer, checkpoints]
    → delivered [buyer] → escrow released: farmer gets total – 1.5% fee
```

PAYMENT

```
awaiting_escrow → escrow_locked → released
escrow_locked —(dispute)→ disputed —(admin)→ released | refunded
```

5 · Project Structure

All files reproduced in section 6. (src/lib/ddl-postgres.ts is auto-generated from the schema and appears in section 2.)

```
├─ README.md (142 lines)
├─ schema.sql (287 lines)
├─ schema.mysql.sql (271 lines)
├─ package.json (38 lines)
├─ tsconfig.json (43 lines)
├─ next.config.ts (33 lines)
├─ drizzle.config.json (8 lines)
├─ postcss.config.mjs (8 lines)
├─ eslint.config.mjs (9 lines)
├─ Dockerfile (27 lines)
├─ docker-compose.yml (35 lines)
├─ render.yaml (29 lines)
├─ .env.example (73 lines)
├─ start-windows.bat (24 lines)
├─ start-docker.bat (9 lines)
├─ start.sh (14 lines)
├─ src/app/(app)/account/page.tsx (67 lines)
├─ src/app/(app)/account/password-form.tsx (62 lines)
├─ src/app/(app)/account/profile-form.tsx (45 lines)
├─ src/app/(app)/admin/dashboard/page.tsx (129 lines)
├─ src/app/(app)/admin/db-explorer/page.tsx (205 lines)
├─ src/app/(app)/admin/layout.tsx (9 lines)
├─ src/app/(app)/admin/mandi/page.tsx (111 lines)
├─ src/app/(app)/admin/orders/page.tsx (8 lines)
├─ src/app/(app)/admin/page.tsx (6 lines)
├─ src/app/(app)/admin/users/page.tsx (172 lines)
├─ src/app/(app)/admin/verification/page.tsx (191 lines)
├─ src/app/(app)/buyer/bids/page.tsx (6 lines)
├─ src/app/(app)/buyer/dashboard/page.tsx (94 lines)
├─ src/app/(app)/buyer/layout.tsx (8 lines)
├─ src/app/(app)/buyer/orders/page.tsx (6 lines)
├─ src/app/(app)/buyer/page.tsx (6 lines)
├─ src/app/(app)/farmer/bids/page.tsx (6 lines)
├─ src/app/(app)/farmer/dashboard/page.tsx (207 lines)
├─ src/app/(app)/farmer/layout.tsx (8 lines)
├─ src/app/(app)/farmer/listings/listings-client.tsx (224 lines)
├─ src/app/(app)/farmer/listings/page.tsx (7 lines)
├─ src/app/(app)/farmer/orders/page.tsx (6 lines)
├─ src/app/(app)/farmer/page.tsx (6 lines)
├─ src/app/(app)/farmer/reviews/page.tsx (20 lines)
├─ src/app/(app)/inspector/dashboard/page.tsx (168 lines)
├─ src/app/(app)/inspector/layout.tsx (8 lines)
├─ src/app/(app)/inspector/page.tsx (6 lines)
├─ src/app/(app)/layout.tsx (18 lines)
├─ src/app/(app)/notifications/page.tsx (37 lines)
├─ src/app/(app)/wallet/page.tsx (13 lines)
├─ src/app/(app)/wallet/wallet-client.tsx (219 lines)
├─ src/app/(site)/crop/[id]/page.tsx (254 lines)
├─ src/app/(site)/layout.tsx (33 lines)
├─ src/app/(site)/mandi-rates/page.tsx (8 lines)
├─ src/app/(site)/marketplace/page.tsx (17 lines)
├─ src/app/(site)/tracking/page.tsx (15 lines)
├─ src/app/admin-login/page.tsx (14 lines)
├─ src/app/api/account/password/route.ts (22 lines)
├─ src/app/api/account/profile/route.ts (27 lines)
├─ src/app/api/admin/disputes/[id]/route.ts (15 lines)
├─ src/app/api/admin/disputes/route.ts (13 lines)
├─ src/app/api/admin/users/[id]/route.ts (26 lines)
├─ src/app/api/admin/users/route.ts (34 lines)
├─ src/app/api/analytics/route.ts (16 lines)
├─ src/app/api/assistant/route.ts (54 lines)
├─ src/app/api/auth/forgot-password/route.ts (24 lines)
├─ src/app/api/auth/google/callback/route.ts (27 lines)
├─ src/app/api/auth/google/route.ts (19 lines)
```

- └ src/app/api/auth/google/test/route.ts (22 lines)
- └ src/app/api/auth/login/route.ts (43 lines)
- └ src/app/api/auth/logout/route.ts (11 lines)
- └ src/app/api/auth/me/route.ts (9 lines)
- └ src/app/api/auth/register/route.ts (36 lines)
- └ src/app/api/auth/reset-password/route.ts (18 lines)
- └ src/app/api/auth/verify-email/route.ts (39 lines)
- └ src/app/api/bids/[id]/route.ts (20 lines)
- └ src/app/api/bids/route.ts (30 lines)
- └ src/app/api/crops/[id]/route.ts (40 lines)
- └ src/app/api/crops/route.ts (51 lines)
- └ src/app/api/db/query/route.ts (72 lines)
- └ src/app/api/db/schema/route.ts (57 lines)
- └ src/app/api/health/route.ts (14 lines)
 - └ src/app/api/inspections/[id]/report/route.ts (19 lines)
- └ src/app/api/inspections/route.ts (31 lines)
- └ src/app/api/mandi-rates/route.ts (39 lines)
 - └ src/app/api/mandi-rates/sync/route.ts (12 lines)
- └ src/app/api/notifications/route.ts (32 lines)
 - └ src/app/api/orders/[id]/dispute/route.ts (15 lines)
 - └ src/app/api/orders/[id]/gps/route.ts (15 lines)
- └ src/app/api/orders/[id]/route.ts (35 lines)
- └ src/app/api/orders/route.ts (15 lines)
 - └ src/app/api/payments/[reference]/route.ts (12 lines)
 - └ src/app/api/payments/callback/easypaisa/route.ts (38 lines)
 - └ src/app/api/payments/callback/jazzcash/route.ts (25 lines)
 - └ src/app/api/payments/checkout/route.ts (23 lines)
 - └ src/app/api/payments/sandbox/[reference]/route.ts (24 lines)
 - └ src/app/api/payments/webhook/stripe/route.ts (25 lines)
 - └ src/app/api/payouts/[id]/route.ts (15 lines)
- └ src/app/api/payouts/route.ts (24 lines)
 - └ src/app/api/reports/summary/route.ts (88 lines)
 - └ src/app/api/reviews/[id]/reply/route.ts (17 lines)
- └ src/app/api/reviews/route.ts (26 lines)
 - └ src/app/api/tracking/[trackingNumber]/route.ts (15 lines)
- └ src/app/api/wallet/route.ts (17 lines)
- └ src/app/auth/google-test/page.tsx (13 lines)
- └ src/app/auth/google-test/test-form.tsx (64 lines)
- └ src/app/deploy/page.tsx (138 lines)
- └ src/app/docs/ddl-tabs.tsx (32 lines)
- └ src/app/docs/page.tsx (125 lines)
- └ src/app/error.tsx (27 lines)
 - └ src/app/forgot-password/page.tsx (9 lines)
- └ src/app/globals.css (111 lines)
- └ src/app/layout.tsx (41 lines)
 - └ src/app/login/login-form.tsx (205 lines)
 - └ src/app/login/page.tsx (41 lines)
- └ src/app/page.tsx (179 lines)
 - └ src/app/pay/sandbox/[reference]/page.tsx (67 lines)
 - └ src/app/register/page.tsx (14 lines)
 - └ src/app/reset-password/page.tsx (11 lines)
 - └ src/app/verify-email/page.tsx (11 lines)
- └ src/components/ChatBot.tsx (354 lines)
- └ src/components/Logo.tsx (58 lines)
 - └ src/components/admin/orders-table.tsx (478 lines)
 - └ src/components/admin/payouts-panel.tsx (70 lines)
- └ src/components/app-shell.tsx (211 lines)
 - └ src/components/auth/google-button.tsx (42 lines)
 - └ src/components/auth/otp-forms.tsx (194 lines)
- └ src/components/charts.tsx (145 lines)
- └ src/components/city-options.tsx (23 lines)
 - └ src/components/crop/bid-panel.tsx (77 lines)
 - └ src/components/crop/price-chart.tsx (28 lines)
- └ src/components/dashboard/workflow-trail.tsx (36 lines)
- └ src/components/mandi-insights.tsx (120 lines)
- └ src/components/mandi-ticker.tsx (33 lines)
 - └ src/components/marketplace/marketplace-client.tsx (215 lines)
- └ src/components/nav/mark-all-read.tsx (13 lines)
- └ src/components/nav/nav-actions.tsx (173 lines)
- └ src/components/nav/site-nav.tsx (92 lines)
- └ src/components/negotiation.tsx (265 lines)
- └ src/components/orders-board.tsx (254 lines)
- └ src/components/providers.tsx (125 lines)

- └─ src/components/reviews/farmer-review-board.tsx (124 lines)
- └─ src/components/reviews/review-prompts.tsx (70 lines)
- └─ src/components/stepper.tsx (51 lines)
- └─ src/components/tracking/tracking-client.tsx (134 lines)
- └─ src/components/tracking/tracking-map.tsx (69 lines)
- └─ src/components/ui.tsx (179 lines)
- └─ src/db/index.ts (101 lines)
- └─ src/db/schema.ts (406 lines)
- └─ src/lib/api.ts (80 lines)
- └─ src/lib/auth.ts (93 lines)
- └─ src/lib/base-url.ts (266 lines)
- └─ src/lib/constants.ts (244 lines)
- └─ src/lib/docs-content.ts (454 lines)
- └─ src/lib/google.ts (165 lines)
- └─ src/lib/llm.ts (71 lines)
- └─ src/lib/migrate.ts (102 lines)
- └─ src/lib/notify.ts (136 lines)
- └─ src/lib/otp.ts (56 lines)
- └─ src/lib/password.ts (28 lines)
- └─ src/lib/reset.ts (86 lines)
- └─ src/lib/seed.ts (335 lines)
- └─ src/lib/services/admin.ts (171 lines)
- └─ src/lib/services/analytics.ts (147 lines)
- └─ src/lib/services/assistant.ts (297 lines)
- └─ src/lib/services/bids.ts (246 lines)
- └─ src/lib/services/crops.ts (173 lines)
- └─ src/lib/services/mandi.ts (174 lines)
- └─ src/lib/services/orders.ts (349 lines)
- └─ src/lib/services/payments.ts (235 lines)
- └─ src/lib/services/reviews.ts (121 lines)
- └─ src/lib/services/tracking.ts (132 lines)
- └─ src/lib/ticker.ts (25 lines)
- └─ src/lib/viewer.ts (17 lines)
- └─ src/proxy.ts (75 lines)
- └─ database/mysql_schema.sql (271 lines)
- └─ database/schema.sql (277 lines)
- └─ scripts/generate-brand-assets.cjs (52 lines)
- └─ scripts/generate-pdf.cjs (269 lines)
- └─ scripts/package-project.cjs (218 lines)
- └─ scripts/serve.cjs (151 lines)
- └─ scripts/verify-payment-redirect.cjs (118 lines)

6 • Complete Source Code

178 files • 15,632 lines, with line numbers. Long lines wrap with ↵.

README.md

```
1  # MarketLink Agri-Hub Pakistan
2
3  B2B agricultural marketplace connecting Pakistani farmers (kisan) directly with wholesalers, supermarket c
↵ hains, exporters and food processors: live mandi rates (PKR / maund), bidding & counter-offers, escrow pay
↵ ments, quality inspection and truck tracking.
4
5  **Demo logins** (password `password123`): `farmer@marketlink.pk` • `buyer@marketlink.pk` • `inspector@mark
↵ etlink.pk` • `ahmed.mustafa@admin.com`
6
7  ## Routes
8
9  | Area | Route | Access |
10 |---|---|---|
11 | Home • Marketplace • Crop details • Mandi rates • Tracking | `/`, `/marketplace`, `/crop/:id`, `/mandi-r
↵ ates`, `/tracking` | Public |
12 | Farmer / Buyer / Inspector / Admin dashboards | `/farmer/dashboard`, `/buyer/dashboard`, `/inspector/das
↵ hboard`, `/admin/dashboard` | Role only (enforced in `src/proxy.ts` + server) |
13 | Database explorer | `/admin/db-explorer` (API `/api/db/query`, `/api/db/schema`) | Admin |
14 | Auth | `/login`, `/register`, `/verify-email`, `/forgot-password`, `/admin-login` | Public |
15
16 ## Branding & marketplace workflow
17
18 - **MARKETLINK AGRI-HUB** uses the infinity-leaf and wheat identity throughout the header, role dashboards
↵ , authentication, chatbot, favicon, and notification emails. Editable SVGs are in `public/brand/`; the ema
↵ il image is a PNG of the same lookup.
19 - **Farmer / Vendor:** manage profile and crops → negotiate bids → dispatch orders → receive escrow payout
↵ s → respond to verified buyer reviews at `/farmer/reviews`.
20 - **Buyer / Customer:** browse and inspect crops → submit offers → fund escrow → track delivery → rate sel
↵ lers after delivery.
21 - **Admin / System Controller:** manage users → monitor transactions and reports → control mandi rates → r
↵ esolve disputes. Quality inspectors certify crops before dispatch.
22 - Role dashboards show the steps and live metrics. Admin and Farmer dashboards can download their own curr
↵ ent CSV summary report from `/api/reports/summary`.
23
24 ## Integrations (configure in `.env`)
25
26 - **Payments:** Stripe Checkout + webhook, JazzCash & Easypaisa hosted checkout with signed callbacks. A b
↵ uilt-in test gateway is used until keys are set (`PAYMENTS_SANDBOX=false` disables it).
27 - **Email / SMS:** Resend and Twilio. Without keys, messages go to the server log and OTP codes are shown
↵ on screen (development mode).
28 - **Google sign-in:** set `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET` and `GOOGLE_CALLBACK_URL` (`<your-sit
↵ e>/api/auth/google/callback`, added as an Authorized redirect URI in Google Cloud Console). Until then a c
↵ learly-labelled test mode is used (`GOOGLE_OAUTH MOCK=false` disables it).
29 - **Password reset:** `/forgot-password` emails a 15-minute single-use link to `/reset-password`; password
↵ s are stored with bcrypt.
30 - **AI assistant:** `OPENAI_API_KEY` or `GEMINI_API_KEY`; otherwise the built-in rule engine answers.
31 - **Database schema:** `schema.sql` (PostgreSQL, runtime) and `schema.mysql.sql` (MySQL 8).
32
33 ## [Urdu]Admin panel (Super Admin: Ahmed Mustafa)
34
35 Only `role = admin` can open `/admin/*` or call `/api/admin/*` and `/api/db/*`. Everyone else is redirecte
↵ d to `/login` with "Access Denied: Admin privileges required" (APIs return 403).
36
37 - Open **`/admin-login`** (e.g. `http://localhost:3000/admin-login`) and sign in with `ahmed.mustafa@admin
↵ .com` / `password123`.
38 - **Change the default password immediately:** sidebar → **Account settings** (the dashboard shows a red w
↵ arning until you do).
39 - Admin pages: Analytics `/admin` • Users & Access `/admin/users` (create admins/inspectors, reset passwor
↵ ds, change roles) • Verification `/admin/verification` • Price Controller `/admin/mandi` • Shipments & Dis
↵ putes `/admin/orders`.
40 - Want your own admin email/password from the very first start? Set `ADMIN_EMAIL` and `ADMIN_PASSWORD` in
↵ `.env` **before** the first run (or delete the `.data` folder to start over).
41 - Forgot the admin password (local install)? Stop the server, set `ADMIN_PASSWORD` in `.env`, delete the `
↵ .data` folder, start again (demo data is recreated).
42
```

```

43 ---
44
45 ## Folder contents
46
47 | Item | What it is |
48 |---|---|
49 | `START-HERE.html` | Offline visual guide: open it first |
50 | `start-windows.bat` / `start.sh` / `start-docker.bat` | One-click start files |
51 | `Documentation/` | Project book PDF, PostgreSQL & MySQL scripts, API spec, workflows, ERD, admin guide,
52 | demo accounts |
53 | `src/` | Application source code (pages, dashboards, REST API, business logic) |
54 | `database/` · `public/` · `scripts/` | DDL scripts · images & downloads · launcher and generators |
55
56 ## ⚠️[Urdu]Why "http://localhost:3000" doesn't open by itself
57
58 `localhost` means "this computer". The link only works after you start the server on your own PC,
59 and only while the black server window stays open. Follow Option 1 below.
60
61 ---
62
63 ## Option 1: Run on your PC (recommended, only Node.js needed)
64
65 No PostgreSQL or Docker required: the app includes a built-in database (stored in the .data` folder).
66
67 1. Install Node.js 20 LTS or newer from https://nodejs.org (click the big green "LTS" button, install
68 with default options).
69 2. Unzip MarketLink-AgriHub-Pakistan.zip` (right-click → Extract All) and open the MarketLink-AgriH
70 ub-Pakistan folder. Open START-HERE.html for a visual guide. Don't run it from inside the zip.
71 3. Open the extracted folder and:
72   - Windows: double-click start-windows.bat`
73   - macOS / Linux: open Terminal in the folder and run bash start.sh`
74 4. First run: it installs packages and builds (3-6 minutes). When you see MarketLink is running!, you
75 r browser opens http://localhost:3000` automatically.
76 5. Keep the black window open while using the site. Press Ctrl + C` in it to stop.
77
78 Next time, step 3 starts in a few seconds.
79
80 ### Open on your phone
81
82 Connect the phone to the same Wi-Fi and open the "Phone / same Wi-Fi" address shown in the window (e.g
83 . http://192.168.1.5:3000`). If Windows Firewall asks, click Allow access.
84
85 ### Troubleshooting
86
87 | Problem | Fix |
88 |---|---|
89 | 'node' is not recognized` | Node.js not installed, or restart the PC after installing. |
90 | Browser says "can't be reached" | The server window was closed, or it hasn't finished starting. Wait for
91 MarketLink is running! |
92 | Port 3000 busy | Handled automatically: the window shows the port it used (e.g. 3001). |
93 | Want a fresh start with new demo data | Stop the server, delete the .data` folder, start again. |
94 | Changed code / updated files | Run start-windows.bat --rebuild` (or node scripts/serve.cjs --rebuild`)
95 . |
96 | Want to use your own PostgreSQL | Set DATABASE_URL=postgresql://postgres:PASSWORD@localhost:5432/app_db`
97 in .env` (the database is created automatically). |
98 | Page loads but looks broken / nothing clickable behind a tunnel or cloud preview (npm run dev`) | Dev m
99 ode blocks its own scripts for unknown hosts. Either start the production server (npm run build && npm st
100 art`, recommended – no such check) or add your host: ALLOWED_DEV_ORIGINS=my-host.example.com npm run dev`
101 . Cloud-preview hosts (*.e2b.app`, cloudflared`, localtunnel`, ngrok`) are already allowed in next.config.t
102 s`. |
103 | Payment redirect says "Could not build a valid payment redirect URL" | The proxy in front of the app is
104 not forwarding the original host. Set APP_URL=https://your-domain` in .env` and restart. |
105
106 ---
107
108 ## Option 2: Temporary public link (share with anyone, free)
109
110 While Option 1 is running, open a second terminal in the folder and run:
111
112 ```bash
113 npx cloudflared tunnel --url http://localhost:3000
114 ```
115
116 It prints a link like https://green-mango-1234.trycloudflare.com`. Anyone can open it while your PC and b
117 oth windows stay on. Alternative: npx localtunnel --port 3000`.
118
119 ## Option 3: Docker (includes a real PostgreSQL server)

```

```

104
105 Install Docker Desktop, then double-click `start-docker.bat` (or run `docker compose up --build`). Open ht
  tp://localhost:3000.
106
107 ## Option 4: Free cloud hosting (online 24/7)
108
109 - **Render.com:** push the folder to GitHub → Render → **New → Blueprint** → select repo (`render.yaml` se
  ts up the web service and PostgreSQL). Free tier sleeps after 15 min idle.
110 - **Vercel + Neon:** free Postgres on neon.tech → import repo on vercel.com → set `DATABASE_URL`, `SESSION
  _SECRET`, `COOKIE_SECURE=true`.
111 - **Railway.app:** deploy from GitHub + PostgreSQL plugin, set `DATABASE_URL` and `SESSION_SECRET`.
112
113 ---
114
115 ## Environment variables (`.env`)
116
117 | Variable | Description |
118 |---|---|
119 | `DATABASE_URL` | Empty = built-in database. Or a PostgreSQL connection string. |
120 | `ADMIN_EMAIL` / `ADMIN_PASSWORD` | Admin login created on first start (default `ahmed.mustafa@admin.com`
  / `password123`). |
121 | `SESSION_SECRET` | Long random text used to sign login cookies. |
122 | `COOKIE_SECURE` | `true` when served over HTTPS. |
123 | `APP_URL` | Public base URL used in payment redirects, OAuth callbacks and email links. **Leave empty**
  (recommended): the app derives it from the request (`x-forwarded-host` / `host`), so links work on localho
  st, LAN, tunnels and cloud hosts. Set it only if your proxy does not forward the original host. Bind addre
  sses such as `http://0.0.0.0:3000` are ignored. `NEXT_PUBLIC_APP_URL`, `PUBLIC_URL` and `BASE_URL` are acc
  epted as aliases. |
124 | `PORT` | Preferred port (default 3000). |
125
126 ## Developer commands
127
128 ```bash
129 node scripts/serve.cjs      # build if needed + start + open browser
130 npm run dev                 # development mode with hot reload
131 npm run build && npm start   # plain production build / start
132 node scripts/verify-payment-redirect.cjs --full # while the server runs: check the buyer escrow redirect
  end-to-end
133 ```
134
135 `verify-payment-redirect.cjs` signs in with the demo buyer, starts a test-gateway top-up and asserts the
136 redirect is an absolute https URL on the host you are using (never `0.0.0.0`, `:::` or a hardcoded
137 `localhost`) – the regression guard for the `ERR_ADDRESS_INVALID` escrow redirect bug. Pass
138 `--host=my-domain.example` to simulate the public Host header your proxy forwards.
139
140 Health check: `GET /api/health` → `{ "ok": true }` · Docs: `/docs` · Hosting page: `/deploy`
141 Stack: Next.js 16 · React 19 · Tailwind CSS 4 · PostgreSQL / PGLite · Drizzle ORM · Recharts
142

```

schema.sql

```

1  -- =====
2  -- MarketLink Agri-Hub Pakistan: database schema (PostgreSQL 14+, runtime DDL)
3  -- Generated from src/db/schema.ts (drizzle-kit export). 15 tables:
4  --   users, crops_inventory, mandi_rates, bids_negotiations, orders_logistics,
5  --   order_events, quality_inspections, wallet_transactions, disputes, reviews,
6  --   notifications, auth_tokens, payment_intents, payout_requests, db_audit_log
7  -- MySQL 8 equivalent: schema.mysql.sql
8  -- Apply: psql "$DATABASE_URL" -f schema.sql (the app also creates it automatically)
9  -- =====
10
11 CREATE TYPE "public"."bid_status" AS ENUM('pending', 'accepted', 'rejected', 'countered', 'withdrawn');
12 CREATE TYPE "public"."crop_category" AS ENUM('grains', 'vegetables', 'fruits', 'cash_crops');
13 CREATE TYPE "public"."crop_status" AS ENUM('active', 'pending_inspection', 'sold_out', 'archived');
14 CREATE TYPE "public"."delivery_stage" AS ENUM('confirmed', 'quality_checked', 'dispatched', 'in_transit',
  'delivered');
15 CREATE TYPE "public"."dispute_status" AS ENUM('open', 'investigating', 'resolved_release', 'resolved_refun
  d');
16 CREATE TYPE "public"."inspection_status" AS ENUM('pending', 'passed', 'failed');
17 CREATE TYPE "public"."payment_status" AS ENUM('awaiting_escrow', 'escrow_locked', 'released', 'refunded',
  'disputed');
18 CREATE TYPE "public"."quality_grade" AS ENUM('A', 'B', 'C');
19 CREATE TYPE "public"."user_role" AS ENUM('farmer', 'buyer', 'admin', 'inspector');
20 CREATE TYPE "public"."wallet_txn_type" AS ENUM('deposit', 'escrow_lock', 'escrow_release', 'payout', 'refu
  nd', 'platform_fee', 'withdrawal');

```

```

21 CREATE TABLE "auth_tokens" (
22     "token_id" serial PRIMARY KEY NOT NULL,
23     "user_id" integer NOT NULL,
24     "purpose" varchar(20) NOT NULL,
25     "code_hash" varchar(128) NOT NULL,
26     "attempts" integer DEFAULT 0 NOT NULL,
27     "expires_at" timestamp with time zone NOT NULL,
28     "used_at" timestamp with time zone,
29     "created_at" timestamp with time zone DEFAULT now() NOT NULL
30 );
31
32 CREATE TABLE "bids_negotiations" (
33     "bid_id" serial PRIMARY KEY NOT NULL,
34     "crop_id" integer NOT NULL,
35     "buyer_id" integer NOT NULL,
36     "bid_price_per_kg" numeric(14, 2) NOT NULL,
37     "bid_quantity_kg" numeric(14, 2) NOT NULL,
38     "target_delivery_date" date,
39     "message" text,
40     "status" "bid_status" DEFAULT 'pending' NOT NULL,
41     "counter_price" numeric(14, 2),
42     "counter_note" text,
43     "created_at" timestamp with time zone DEFAULT now() NOT NULL,
44     "updated_at" timestamp with time zone DEFAULT now() NOT NULL
45 );
46
47 CREATE TABLE "crops_inventory" (
48     "crop_id" serial PRIMARY KEY NOT NULL,
49     "farmer_id" integer NOT NULL,
50     "crop_name" varchar(80) NOT NULL,
51     "category" "crop_category" NOT NULL,
52     "quality_grade" "quality_grade" NOT NULL,
53     "total_quantity_kg" numeric(14, 2) NOT NULL,
54     "base_price_per_kg" numeric(14, 2) NOT NULL,
55     "harvest_date" date NOT NULL,
56     "images_json" jsonb DEFAULT '[]'::jsonb NOT NULL,
57     "description" text,
58     "farm_location" varchar(80) NOT NULL,
59     "latitude" double precision,
60     "longitude" double precision,
61     "status" "crop_status" DEFAULT 'active' NOT NULL,
62     "created_at" timestamp with time zone DEFAULT now() NOT NULL
63 );
64
65 CREATE TABLE "db_audit_log" (
66     "audit_id" serial PRIMARY KEY NOT NULL,
67     "admin_id" integer,
68     "mode" varchar(10) NOT NULL,
69     "query" text NOT NULL,
70     "row_count" integer,
71     "duration_ms" integer,
72     "error" text,
73     "created_at" timestamp with time zone DEFAULT now() NOT NULL
74 );
75
76 CREATE TABLE "disputes" (
77     "dispute_id" serial PRIMARY KEY NOT NULL,
78     "order_id" integer NOT NULL,
79     "raised_by" integer NOT NULL,
80     "reason" text NOT NULL,
81     "status" "dispute_status" DEFAULT 'open' NOT NULL,
82     "resolution" text,
83     "created_at" timestamp with time zone DEFAULT now() NOT NULL,
84     "resolved_at" timestamp with time zone
85 );
86
87 CREATE TABLE "mandi_rates" (
88     "rate_id" serial PRIMARY KEY NOT NULL,
89     "crop_name" varchar(80) NOT NULL,
90     "market_location" varchar(80) NOT NULL,
91     "min_price_per_kg" numeric(14, 2) NOT NULL,
92     "max_price_per_kg" numeric(14, 2) NOT NULL,
93     "avg_price_per_kg" numeric(14, 2) NOT NULL,
94     "source" varchar(20) DEFAULT 'manual' NOT NULL,
95     "date_updated" date NOT NULL
96 );

```

```

97
98 CREATE TABLE "notifications" (
99     "notification_id" serial PRIMARY KEY NOT NULL,
100     "user_id" integer NOT NULL,
101     "type" varchar(40) NOT NULL,
102     "title" varchar(200) NOT NULL,
103     "body" text,
104     "link" varchar(300),
105     "channels" varchar(60) DEFAULT 'in_app' NOT NULL,
106     "is_read" boolean DEFAULT false NOT NULL,
107     "created_at" timestamp with time zone DEFAULT now() NOT NULL
108 );
109
110 CREATE TABLE "order_events" (
111     "event_id" serial PRIMARY KEY NOT NULL,
112     "order_id" integer NOT NULL,
113     "stage" varchar(40) NOT NULL,
114     "location" varchar(120),
115     "lat" double precision,
116     "lng" double precision,
117     "note" text,
118     "actor_id" integer,
119     "created_at" timestamp with time zone DEFAULT now() NOT NULL
120 );
121
122 CREATE TABLE "orders_logistics" (
123     "order_id" serial PRIMARY KEY NOT NULL,
124     "bid_id" integer NOT NULL,
125     "crop_id" integer NOT NULL,
126     "farmer_id" integer NOT NULL,
127     "buyer_id" integer NOT NULL,
128     "quantity_kg" numeric(14, 2) NOT NULL,
129     "price_per_kg" numeric(14, 2) NOT NULL,
130     "total_amount" numeric(14, 2) NOT NULL,
131     "payment_status" "payment_status" DEFAULT 'awaiting_escrow' NOT NULL,
132     "delivery_stage" "delivery_stage" DEFAULT 'confirmed' NOT NULL,
133     "transit_location" varchar(120),
134     "current_lat" double precision,
135     "current_lng" double precision,
136     "tracking_number" varchar(40) NOT NULL,
137     "expected_delivery_date" date,
138     "created_at" timestamp with time zone DEFAULT now() NOT NULL,
139     "updated_at" timestamp with time zone DEFAULT now() NOT NULL
140 );
141
142 CREATE TABLE "payment_intents" (
143     "intent_id" serial PRIMARY KEY NOT NULL,
144     "reference" varchar(60) NOT NULL,
145     "user_id" integer NOT NULL,
146     "provider" varchar(20) NOT NULL,
147     "amount" numeric(14, 2) NOT NULL,
148     "status" varchar(20) DEFAULT 'pending' NOT NULL,
149     "provider_ref" varchar(200),
150     "raw_callback" text,
151     "created_at" timestamp with time zone DEFAULT now() NOT NULL,
152     "completed_at" timestamp with time zone
153 );
154
155 CREATE TABLE "payout_requests" (
156     "payout_id" serial PRIMARY KEY NOT NULL,
157     "farmer_id" integer NOT NULL,
158     "amount" numeric(14, 2) NOT NULL,
159     "method" varchar(20) NOT NULL,
160     "account_title" varchar(120) NOT NULL,
161     "account_number" varchar(60) NOT NULL,
162     "status" varchar(20) DEFAULT 'pending' NOT NULL,
163     "admin_note" text,
164     "created_at" timestamp with time zone DEFAULT now() NOT NULL,
165     "processed_at" timestamp with time zone
166 );
167
168 CREATE TABLE "quality_inspections" (
169     "inspection_id" serial PRIMARY KEY NOT NULL,
170     "crop_id" integer NOT NULL,
171     "inspector_id" integer NOT NULL,
172     "grade_assigned" "quality_grade" NOT NULL,

```

```

173     "moisture_level_percentage" numeric(5, 2) NOT NULL,
174     "soil_ph" numeric(4, 2),
175     "inspection_notes" text,
176     "report_attachment" text,
177     "report_file_name" varchar(200),
178     "certificate_no" varchar(40),
179     "status" "inspection_status" DEFAULT 'pending' NOT NULL,
180     "verified_at" timestamp with time zone
181 );
182
183 CREATE TABLE "reviews" (
184     "review_id" serial PRIMARY KEY NOT NULL,
185     "order_id" integer NOT NULL,
186     "buyer_id" integer NOT NULL,
187     "farmer_id" integer NOT NULL,
188     "rating" integer NOT NULL,
189     "comment" text,
190     "farmer_reply" text,
191     "replied_at" timestamp with time zone,
192     "created_at" timestamp with time zone DEFAULT now() NOT NULL
193 );
194
195 CREATE TABLE "users" (
196     "id" serial PRIMARY KEY NOT NULL,
197     "full_name" varchar(120) NOT NULL,
198     "email" varchar(160) NOT NULL,
199     "password_hash" varchar(255) NOT NULL,
200     "phone" varchar(30) NOT NULL,
201     "role" "user_role" NOT NULL,
202     "cnic_id" varchar(20),
203     "business_name" varchar(160),
204     "city" varchar(80) NOT NULL,
205     "address" text,
206     "is_verified" boolean DEFAULT false NOT NULL,
207     "email_verified" boolean DEFAULT false NOT NULL,
208     "title" varchar(120),
209     "user_code" varchar(40),
210     "avatar_url" text,
211     "google_id" varchar(64),
212     "auth_provider" varchar(20) DEFAULT 'password' NOT NULL,
213     "trust_score" numeric(3, 2) DEFAULT 0 NOT NULL,
214     "rating_avg" numeric(3, 2) DEFAULT 0 NOT NULL,
215     "rating_count" integer DEFAULT 0 NOT NULL,
216     "wallet_balance" numeric(14, 2) DEFAULT 0 NOT NULL,
217     "escrow_balance" numeric(14, 2) DEFAULT 0 NOT NULL,
218     "created_at" timestamp with time zone DEFAULT now() NOT NULL
219 );
220
221 CREATE TABLE "wallet_transactions" (
222     "txn_id" serial PRIMARY KEY NOT NULL,
223     "user_id" integer NOT NULL,
224     "order_id" integer,
225     "type" "wallet_txn_type" NOT NULL,
226     "amount" numeric(14, 2) NOT NULL,
227     "description" text,
228     "created_at" timestamp with time zone DEFAULT now() NOT NULL
229 );
230
231 ALTER TABLE "auth_tokens" ADD CONSTRAINT "auth_tokens_user_id_users_id_fk" FOREIGN KEY ("user_id") REFEREN
232     CES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
233 ALTER TABLE "bids_negotiations" ADD CONSTRAINT "bids_negotiations_crop_id_crops_inventory_crop_id_fk" FORE
234     IGN KEY ("crop_id") REFERENCES "public"."crops_inventory"("crop_id") ON DELETE cascade ON UPDATE no action
235     ;
236 ALTER TABLE "bids_negotiations" ADD CONSTRAINT "bids_negotiations_buyer_id_users_id_fk" FOREIGN KEY ("buye
237     r_id") REFERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
238 ALTER TABLE "crops_inventory" ADD CONSTRAINT "crops_inventory_farmer_id_users_id_fk" FOREIGN KEY ("farmer_
239     id") REFERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
240 ALTER TABLE "db_audit_log" ADD CONSTRAINT "db_audit_log_admin_id_users_id_fk" FOREIGN KEY ("admin_id") REF
241     ERENCES "public"."users"("id") ON DELETE set null ON UPDATE no action;
242 ALTER TABLE "disputes" ADD CONSTRAINT "disputes_order_id_orders_logistics_order_id_fk" FOREIGN KEY ("order
243     _id") REFERENCES "public"."orders_logistics"("order_id") ON DELETE cascade ON UPDATE no action;
244 ALTER TABLE "disputes" ADD CONSTRAINT "disputes_raised_by_users_id_fk" FOREIGN KEY ("raised_by") REFERENCE
245     S "public"."users"("id") ON DELETE restrict ON UPDATE no action;
246 ALTER TABLE "notifications" ADD CONSTRAINT "notifications_user_id_users_id_fk" FOREIGN KEY ("user_id") REF
247     ERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
248 ALTER TABLE "order_events" ADD CONSTRAINT "order_events_order_id_orders_logistics_order_id_fk" FOREIGN KEY

```

```

    ("order_id") REFERENCES "public"."orders_logistics"("order_id") ON DELETE cascade ON UPDATE no action;
240 ALTER TABLE "order_events" ADD CONSTRAINT "order_events_actor_id_users_id_fk" FOREIGN KEY ("actor_id") REF
    ERENCES "public"."users"("id") ON DELETE set null ON UPDATE no action;
241 ALTER TABLE "orders_logistics" ADD CONSTRAINT "orders_logistics_bid_id_bids_negotiations_bid_id_fk" FOREIG
    N KEY ("bid_id") REFERENCES "public"."bids_negotiations"("bid_id") ON DELETE restrict ON UPDATE no action;
242 ALTER TABLE "orders_logistics" ADD CONSTRAINT "orders_logistics_crop_id_crops_inventory_crop_id_fk" FOREIG
    N KEY ("crop_id") REFERENCES "public"."crops_inventory"("crop_id") ON DELETE restrict ON UPDATE no action;
243 ALTER TABLE "orders_logistics" ADD CONSTRAINT "orders_logistics_farmer_id_users_id_fk" FOREIGN KEY ("farme
    r_id") REFERENCES "public"."users"("id") ON DELETE restrict ON UPDATE no action;
244 ALTER TABLE "orders_logistics" ADD CONSTRAINT "orders_logistics_buyer_id_users_id_fk" FOREIGN KEY ("buyer_
    id") REFERENCES "public"."users"("id") ON DELETE restrict ON UPDATE no action;
245 ALTER TABLE "payment_intents" ADD CONSTRAINT "payment_intents_user_id_users_id_fk" FOREIGN KEY ("user_id")
    REFERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
246 ALTER TABLE "payout_requests" ADD CONSTRAINT "payout_requests_farmer_id_users_id_fk" FOREIGN KEY ("farmer_
    id") REFERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
247 ALTER TABLE "quality_inspections" ADD CONSTRAINT "quality_inspections_crop_id_crops_inventory_crop_id_fk"
    FOREIGN KEY ("crop_id") REFERENCES "public"."crops_inventory"("crop_id") ON DELETE cascade ON UPDATE no ac
    tion;
248 ALTER TABLE "quality_inspections" ADD CONSTRAINT "quality_inspections_inspector_id_users_id_fk" FOREIGN KE
    Y ("inspector_id") REFERENCES "public"."users"("id") ON DELETE restrict ON UPDATE no action;
249 ALTER TABLE "reviews" ADD CONSTRAINT "reviews_order_id_orders_logistics_order_id_fk" FOREIGN KEY ("order_i
    d") REFERENCES "public"."orders_logistics"("order_id") ON DELETE cascade ON UPDATE no action;
250 ALTER TABLE "reviews" ADD CONSTRAINT "reviews_buyer_id_users_id_fk" FOREIGN KEY ("buyer_id") REFERENCES "p
    ublic"."users"("id") ON DELETE cascade ON UPDATE no action;
251 ALTER TABLE "reviews" ADD CONSTRAINT "reviews_farmer_id_users_id_fk" FOREIGN KEY ("farmer_id") REFERENCES
    "public"."users"("id") ON DELETE cascade ON UPDATE no action;
252 ALTER TABLE "wallet_transactions" ADD CONSTRAINT "wallet_transactions_user_id_users_id_fk" FOREIGN KEY ("u
    ser_id") REFERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
253 ALTER TABLE "wallet_transactions" ADD CONSTRAINT "wallet_transactions_order_id_orders_logistics_order_id_f
    k" FOREIGN KEY ("order_id") REFERENCES "public"."orders_logistics"("order_id") ON DELETE set null ON UPDAT
    E no action;
254 CREATE INDEX "auth_tokens_user_idx" ON "auth_tokens" USING btree ("user_id","purpose");
255 CREATE INDEX "bids_crop_idx" ON "bids_negotiations" USING btree ("crop_id");
256 CREATE INDEX "bids_buyer_idx" ON "bids_negotiations" USING btree ("buyer_id");
257 CREATE INDEX "bids_status_idx" ON "bids_negotiations" USING btree ("status");
258 CREATE INDEX "crops_farmer_idx" ON "crops_inventory" USING btree ("farmer_id");
259 CREATE INDEX "crops_category_idx" ON "crops_inventory" USING btree ("category");
260 CREATE INDEX "crops_status_idx" ON "crops_inventory" USING btree ("status");
261 CREATE INDEX "crops_name_idx" ON "crops_inventory" USING btree ("crop_name");
262 CREATE INDEX "disputes_order_idx" ON "disputes" USING btree ("order_id");
263 CREATE INDEX "disputes_status_idx" ON "disputes" USING btree ("status");
264 CREATE UNIQUE INDEX "mandi_crop_market_date_uq" ON "mandi_rates" USING btree ("crop_name","market_location
    ","date_updated");
265 CREATE INDEX "mandi_date_idx" ON "mandi_rates" USING btree ("date_updated");
266 CREATE INDEX "notifications_user_idx" ON "notifications" USING btree ("user_id","is_read");
267 CREATE INDEX "order_events_order_idx" ON "order_events" USING btree ("order_id");
268 CREATE UNIQUE INDEX "orders_bid_uq" ON "orders_logistics" USING btree ("bid_id");
269 CREATE UNIQUE INDEX "orders_tracking_uq" ON "orders_logistics" USING btree ("tracking_number");
270 CREATE INDEX "orders_farmer_idx" ON "orders_logistics" USING btree ("farmer_id");
271 CREATE INDEX "orders_buyer_idx" ON "orders_logistics" USING btree ("buyer_id");
272 CREATE INDEX "orders_stage_idx" ON "orders_logistics" USING btree ("delivery_stage");
273 CREATE UNIQUE INDEX "payment_intents_ref_uq" ON "payment_intents" USING btree ("reference");
274 CREATE INDEX "payment_intents_user_idx" ON "payment_intents" USING btree ("user_id");
275 CREATE INDEX "payout_requests_farmer_idx" ON "payout_requests" USING btree ("farmer_id");
276 CREATE INDEX "payout_requests_status_idx" ON "payout_requests" USING btree ("status");
277 CREATE INDEX "inspections_crop_idx" ON "quality_inspections" USING btree ("crop_id");
278 CREATE INDEX "inspections_status_idx" ON "quality_inspections" USING btree ("status");
279 CREATE UNIQUE INDEX "reviews_order_uq" ON "reviews" USING btree ("order_id");
280 CREATE INDEX "reviews_farmer_idx" ON "reviews" USING btree ("farmer_id");
281 CREATE UNIQUE INDEX "users_email_uq" ON "users" USING btree ("email");
282 CREATE UNIQUE INDEX "users_google_id_uq" ON "users" USING btree ("google_id");
283 CREATE UNIQUE INDEX "users_user_code_uq" ON "users" USING btree ("user_code");
284 CREATE INDEX "users_role_idx" ON "users" USING btree ("role");
285 CREATE INDEX "users_city_idx" ON "users" USING btree ("city");
286 CREATE INDEX "wallet_user_idx" ON "wallet_transactions" USING btree ("user_id");
287

```

schema.mysql.sql

```

1  -- MarketLink Agri-Hub · MySQL 8.0 compatible DDL (InnoDB, utf8mb4)
2  -- Production runtime uses PostgreSQL (see PostgreSQL tab); this script is the portable equivalent.
3
4  CREATE TABLE users (
5      id                INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
6      full_name          VARCHAR(120) NOT NULL,

```

```

7     email                VARCHAR(160) NOT NULL,
8     password_hash        VARCHAR(255) NOT NULL,
9     phone                VARCHAR(30) NOT NULL,
10    role                  ENUM('farmer','buyer','admin','inspector') NOT NULL,
11    cnic_id               VARCHAR(20) NULL,
12    business_name         VARCHAR(160) NULL,
13    city                  VARCHAR(80) NOT NULL,
14    address               TEXT NULL,
15    is_verified           BOOLEAN NOT NULL DEFAULT FALSE,
16    email_verified        BOOLEAN NOT NULL DEFAULT FALSE,
17    title                 VARCHAR(120) NULL, -- e.g. 'Super Admin & Systems Controller'
18    user_code             VARCHAR(40) NULL, -- e.g. 'admin_01'
19    avatar_url            TEXT NULL,
20    google_id             VARCHAR(64) NULL, -- Google OAuth subject
21    auth_provider         VARCHAR(20) NOT NULL DEFAULT 'password',
22    trust_score           DECIMAL(3,2) NOT NULL DEFAULT 0,
23    rating_avg            DECIMAL(3,2) NOT NULL DEFAULT 0,
24    rating_count          INT NOT NULL DEFAULT 0,
25    wallet_balance        DECIMAL(14,2) NOT NULL DEFAULT 0,
26    escrow_balance        DECIMAL(14,2) NOT NULL DEFAULT 0,
27    created_at            TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
28    UNIQUE KEY users_email_uq (email),
29    UNIQUE KEY users_google_id_uq (google_id),
30    UNIQUE KEY users_user_code_uq (user_code),
31    KEY users_role_idx (role),
32    KEY users_city_idx (city)
33 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
34
35 CREATE TABLE crops_inventory (
36     crop_id              INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
37     farmer_id            INT UNSIGNED NOT NULL,
38     crop_name            VARCHAR(80) NOT NULL,
39     category             ENUM('grains','vegetables','fruits','cash_crops') NOT NULL,
40     quality_grade        ENUM('A','B','C') NOT NULL,
41     total_quantity_kg    DECIMAL(14,2) NOT NULL CHECK (total_quantity_kg >= 0),
42     base_price_per_kg    DECIMAL(14,2) NOT NULL CHECK (base_price_per_kg > 0),
43     harvest_date         DATE NOT NULL,
44     images_json          JSON NOT NULL,
45     description          TEXT NULL,
46     farm_location        VARCHAR(80) NOT NULL,
47     latitude             DOUBLE NULL,
48     longitude            DOUBLE NULL,
49     status               ENUM('active','pending_inspection','sold_out','archived') NOT NULL DEFAULT 'active',
50     created_at           TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
51     KEY crops_farmer_idx (farmer_id),
52     KEY crops_category_idx (category),
53     KEY crops_status_idx (status),
54     KEY crops_name_idx (crop_name),
55     CONSTRAINT fk_crops_farmer FOREIGN KEY (farmer_id) REFERENCES users(id) ON DELETE CASCADE
56 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
57
58 CREATE TABLE mandi_rates (
59     rate_id              INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
60     crop_name            VARCHAR(80) NOT NULL,
61     market_location      VARCHAR(80) NOT NULL,
62     min_price_per_kg     DECIMAL(14,2) NOT NULL,
63     max_price_per_kg     DECIMAL(14,2) NOT NULL,
64     avg_price_per_kg     DECIMAL(14,2) NOT NULL,
65     source               VARCHAR(20) NOT NULL DEFAULT 'manual',
66     date_updated         DATE NOT NULL,
67     UNIQUE KEY mandi_crop_market_date_uq (crop_name, market_location, date_updated),
68     KEY mandi_date_idx (date_updated),
69     CHECK (min_price_per_kg <= avg_price_per_kg AND avg_price_per_kg <= max_price_per_kg)
70 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
71
72 CREATE TABLE bids_negotiations (
73     bid_id              INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
74     crop_id             INT UNSIGNED NOT NULL,
75     buyer_id           INT UNSIGNED NOT NULL,
76     bid_price_per_kg    DECIMAL(14,2) NOT NULL,
77     bid_quantity_kg     DECIMAL(14,2) NOT NULL,
78     target_delivery_date DATE NULL,
79     message             TEXT NULL,
80     status              ENUM('pending','accepted','rejected','countered','withdrawn') NOT NULL DEFAULT 'pending',
81     counter_price       DECIMAL(14,2) NULL,

```

```

82     counter_note          TEXT          NULL,
83     created_at            TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
84     updated_at            TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
85     KEY bids_crop_idx (crop_id),
86     KEY bids_buyer_idx (buyer_id),
87     KEY bids_status_idx (status),
88     CONSTRAINT fk_bids_crop FOREIGN KEY (crop_id) REFERENCES crops_inventory(crop_id) ON DELETE CASCADE,
89     CONSTRAINT fk_bids_buyer FOREIGN KEY (buyer_id) REFERENCES users(id) ON DELETE CASCADE
90 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
91
92 CREATE TABLE orders_logistics (
93     order_id              INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
94     bid_id                INT UNSIGNED NOT NULL,
95     crop_id               INT UNSIGNED NOT NULL,
96     farmer_id             INT UNSIGNED NOT NULL,
97     buyer_id             INT UNSIGNED NOT NULL,
98     quantity_kg           DECIMAL(14,2) NOT NULL,
99     price_per_kg           DECIMAL(14,2) NOT NULL,
100    total_amount           DECIMAL(14,2) NOT NULL,
101    payment_status         ENUM('awaiting_escrow','escrow_locked','released','refunded','disputed') NOT NULL
102    DEFAULT 'awaiting_escrow',
103    delivery_stage         ENUM('confirmed','quality_checked','dispatched','in_transit','delivered') NOT NULL
104    DEFAULT 'confirmed',
105    transit_location       VARCHAR(120) NULL,
106    current_lat            DOUBLE        NULL,
107    current_lng            DOUBLE        NULL,
108    tracking_number        VARCHAR(40)   NOT NULL,
109    expected_delivery_date DATE          NULL,
110    created_at             TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
111    updated_at             TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
112    UNIQUE KEY orders_bid_uq (bid_id),
113    UNIQUE KEY orders_tracking_uq (tracking_number),
114    KEY orders_farmer_idx (farmer_id),
115    KEY orders_buyer_idx (buyer_id),
116    KEY orders_stage_idx (delivery_stage),
117    CONSTRAINT fk_orders_bid FOREIGN KEY (bid_id) REFERENCES bids_negotiations(bid_id),
118    CONSTRAINT fk_orders_crop FOREIGN KEY (crop_id) REFERENCES crops_inventory(crop_id),
119    CONSTRAINT fk_orders_farmer FOREIGN KEY (farmer_id) REFERENCES users(id),
120    CONSTRAINT fk_orders_buyer FOREIGN KEY (buyer_id) REFERENCES users(id)
121 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
122
123 CREATE TABLE quality_inspections (
124     inspection_id         INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
125     crop_id               INT UNSIGNED NOT NULL,
126     inspector_id          INT UNSIGNED NOT NULL,
127     grade_assigned        ENUM('A','B','C') NOT NULL,
128     moisture_level_percentage DECIMAL(5,2) NOT NULL,
129     soil_ph               DECIMAL(4,2) NULL,
130     inspection_notes      TEXT          NULL,
131     report_attachment     LONGTEXT     NULL,
132     report_file_name      VARCHAR(200) NULL,
133     certificate_no        VARCHAR(40)   NULL,
134     status                ENUM('pending','passed','failed') NOT NULL DEFAULT 'pending',
135     verified_at           TIMESTAMP     NULL,
136     KEY inspections_crop_idx (crop_id),
137     KEY inspections_status_idx (status),
138     CONSTRAINT fk_insp_crop FOREIGN KEY (crop_id) REFERENCES crops_inventory(crop_id) ON DELETE CASCADE,
139     CONSTRAINT fk_insp_inspector FOREIGN KEY (inspector_id) REFERENCES users(id)
140 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
141
142 -- Supporting tables: order_events (tracking audit), wallet_transactions (escrow ledger), disputes
143 CREATE TABLE order_events (
144     event_id              INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
145     order_id              INT UNSIGNED NOT NULL,
146     stage                 VARCHAR(40)   NOT NULL,
147     location              VARCHAR(120) NULL,
148     lat                   DOUBLE        NULL,
149     lng                   DOUBLE        NULL,
150     note                  TEXT          NULL,
151     actor_id              INT UNSIGNED NULL,
152     created_at            TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
153     KEY order_events_order_idx (order_id),
154     CONSTRAINT fk_events_order FOREIGN KEY (order_id) REFERENCES orders_logistics(order_id) ON DELETE CASCADE,
155     KEY order_events_actor_idx (actor_id),
156     CONSTRAINT fk_events_actor FOREIGN KEY (actor_id) REFERENCES users(id) ON DELETE SET NULL

```

```

154 ) ENGINE=InnoDB;
155
156 CREATE TABLE wallet_transactions (
157     txn_id      INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
158     user_id     INT UNSIGNED NOT NULL,
159     order_id    INT UNSIGNED NULL,
160     type        ENUM('deposit','escrow_lock','escrow_release','payout','refund','platform_fee','withdrawal')
161     NOT NULL,
162     amount      DECIMAL(14,2) NOT NULL,
163     description TEXT          NULL,
164     created_at  TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
165     KEY wallet_user_idx (user_id),
166     CONSTRAINT fk_wallet_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
167     CONSTRAINT fk_wallet_order FOREIGN KEY (order_id) REFERENCES orders_logistics(order_id) ON DELETE SET NU
168     LL
169 ) ENGINE=InnoDB;
170
171 CREATE TABLE disputes (
172     dispute_id  INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
173     order_id    INT UNSIGNED NOT NULL,
174     raised_by   INT UNSIGNED NOT NULL,
175     reason      TEXT          NOT NULL,
176     status      ENUM('open','investigating','resolved_release','resolved_refund') NOT NULL DEFAULT 'open',
177     resolution  TEXT          NULL,
178     created_at  TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
179     resolved_at TIMESTAMP     NULL,
180     KEY disputes_order_idx (order_id),
181     KEY disputes_status_idx (status),
182     CONSTRAINT fk_disputes_order FOREIGN KEY (order_id) REFERENCES orders_logistics(order_id) ON DELETE CASC
183     ADE,
184     CONSTRAINT fk_disputes_user FOREIGN KEY (raised_by) REFERENCES users(id)
185 ) ENGINE=InnoDB;
186
187 CREATE TABLE reviews (
188     review_id   INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
189     order_id    INT UNSIGNED NOT NULL,
190     buyer_id   INT UNSIGNED NOT NULL,
191     farmer_id   INT UNSIGNED NOT NULL,
192     rating      TINYINT      NOT NULL CHECK (rating BETWEEN 1 AND 5),
193     comment     TEXT          NULL,
194     farmer_reply TEXT        NULL,
195     replied_at  TIMESTAMP     NULL,
196     created_at  TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
197     UNIQUE KEY reviews_order_uq (order_id),
198     KEY reviews_farmer_idx (farmer_id),
199     CONSTRAINT fk_reviews_order FOREIGN KEY (order_id) REFERENCES orders_logistics(order_id) ON DELETE CAS
200     CADE,
201     CONSTRAINT fk_reviews_buyer FOREIGN KEY (buyer_id) REFERENCES users(id) ON DELETE CASCADE,
202     CONSTRAINT fk_reviews_farmer FOREIGN KEY (farmer_id) REFERENCES users(id) ON DELETE CASCADE
203 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
204
205 CREATE TABLE notifications (
206     notification_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
207     user_id         INT UNSIGNED NOT NULL,
208     type            VARCHAR(40)  NOT NULL,
209     title           VARCHAR(200) NOT NULL,
210     body            TEXT         NULL,
211     link            VARCHAR(300) NULL,
212     channels        VARCHAR(60)  NOT NULL DEFAULT 'in_app',
213     is_read         BOOLEAN      NOT NULL DEFAULT FALSE,
214     created_at      TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
215     KEY notifications_user_idx (user_id, is_read),
216     CONSTRAINT fk_notifications_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
217 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
218
219 CREATE TABLE auth_tokens (
220     token_id      INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
221     user_id       INT UNSIGNED NOT NULL,
222     purpose        VARCHAR(20)  NOT NULL,          -- verify_email | reset_link (15-min password reset token)
223     code_hash      VARCHAR(128) NOT NULL,          -- SHA-256 of the 6-digit code + secret
224     attempts       INT          NOT NULL DEFAULT 0,
225     expires_at     TIMESTAMP     NOT NULL,
226     used_at        TIMESTAMP     NULL,
227     created_at     TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
228     KEY auth_tokens_user_idx (user_id, purpose),
229     CONSTRAINT fk_auth_tokens_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE

```

```

226 ) ENGINE=InnoDB;
227
228 CREATE TABLE payment_intents (
229   intent_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
230   reference VARCHAR(60) NOT NULL,
231   user_id INT UNSIGNED NOT NULL,
232   provider VARCHAR(20) NOT NULL, -- stripe | jazzcash | easypaisa | sandbox
233   amount DECIMAL(14,2) NOT NULL,
234   status VARCHAR(20) NOT NULL DEFAULT 'pending',
235   provider_ref VARCHAR(200) NULL,
236   raw_callback TEXT NULL,
237   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
238   completed_at TIMESTAMP NULL,
239   UNIQUE KEY payment_intents_ref_uq (reference),
240   KEY payment_intents_user_idx (user_id),
241   CONSTRAINT fk_payment_intents_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
242 ) ENGINE=InnoDB;
243
244 CREATE TABLE payout_requests (
245   payout_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
246   farmer_id INT UNSIGNED NOT NULL,
247   amount DECIMAL(14,2) NOT NULL,
248   method VARCHAR(20) NOT NULL, -- bank | jazzcash | easypaisa
249   account_title VARCHAR(120) NOT NULL,
250   account_number VARCHAR(60) NOT NULL,
251   status VARCHAR(20) NOT NULL DEFAULT 'pending',
252   admin_note TEXT NULL,
253   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
254   processed_at TIMESTAMP NULL,
255   KEY payout_requests_farmer_idx (farmer_id),
256   KEY payout_requests_status_idx (status),
257   CONSTRAINT fk_payout_farmer FOREIGN KEY (farmer_id) REFERENCES users(id) ON DELETE CASCADE
258 ) ENGINE=InnoDB;
259
260 CREATE TABLE db_audit_log (
261   audit_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
262   admin_id INT UNSIGNED NULL,
263   mode VARCHAR(10) NOT NULL,
264   query TEXT NOT NULL,
265   row_count INT NULL,
266   duration_ms INT NULL,
267   error TEXT NULL,
268   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
269   CONSTRAINT fk_audit_admin FOREIGN KEY (admin_id) REFERENCES users(id) ON DELETE SET NULL
270 ) ENGINE=InnoDB;
271

```

package.json

```

1  {
2    "name": "nextjs-postgresql-template",
3    "private": true,
4    "scripts": {
5      "dev": "next dev",
6      "build": "next build",
7      "start": "next start",
8      "lint": "eslint .",
9      "typecheck": "tsc --noEmit"
10   },
11   "dependencies": {
12     "@electric-sql/pglite": "^0.5.8",
13     "@types/leaflet": "^1.9.22",
14     "bcryptjs": "^3.0.3",
15     "dotenv": "17.3.1",
16     "drizzle-orm": "0.45.2",
17     "leaflet": "^1.9.4",
18     "next": "16.2.6",
19     "pg": "8.20.0",
20     "react": "19.2.6",
21     "react-dom": "19.2.6",
22     "recharts": "^3.10.1"
23   },
24   "devDependencies": {
25     "@tailwindcss/postcss": "4.1.17",
26     "@types/node": "22.19.15",

```

```

27     "@types/pg": "8.18.0",
28     "@types/react": "19.2.14",
29     "@types/react-dom": "19.2.3",
30     "drizzle-kit": "0.31.10",
31     "eslint": "9.39.4",
32     "eslint-config-next": "16.2.6",
33     "postcss": "8.5.8",
34     "tailwindcss": "4.1.17",
35     "typescript": "5.9.3"
36   }
37 }
38

```

tsconfig.json

```

1  {
2    "compilerOptions": {
3      "target": "ES2017",
4      "lib": [
5        "dom",
6        "dom.iterable",
7        "esnext"
8      ],
9      "allowJs": false,
10     "skipLibCheck": true,
11     "strict": true,
12     "noEmit": true,
13     "esModuleInterop": true,
14     "module": "esnext",
15     "moduleResolution": "bundler",
16     "resolveJsonModule": true,
17     "isolatedModules": true,
18     "jsx": "react-jsx",
19     "incremental": true,
20     "baseUrl": ".",
21     "paths": {
22       "@/*": [
23         "./src/*"
24       ]
25     },
26     "plugins": [
27       {
28         "name": "next"
29       }
30     ]
31   },
32   "include": [
33     "next-env.d.ts",
34     "**/*.ts",
35     "**/*.tsx",
36     ".next/types/**/*.ts",
37     ".next/dev/types/**/*.ts"
38   ],
39   "exclude": [
40     "node_modules"
41   ]
42 }
43

```

next.config.ts

```

1  import type { NextConfig } from "next";
2
3  /**
4   * Hosts that may load Next.js *development* resources (HMR, dev chunks).
5   * Without this, running `npm run dev` behind a tunnel / cloud preview proxy makes the browser get
6   * "Blocked cross-origin request to Next.js dev resource ..." for every script and stylesheet, so the
7   * page looks broken. Nothing here affects production builds (`npm run build && npm start`) – the
8   * check only exists in dev mode. Add your own host with ALLOWED_DEV_ORIGINS=a.example.com,b.example.com
9   */
10 const devOrigins = [
11   ".*e2b.app", // cloud sandbox live previews (https://<port>-<sandbox>.e2b.app)
12   ".*e2b.dev",
13   ".*trycloudflare.com", // npx cloudflared tunnel --url http://localhost:3000

```

```

14  "*.localt", // npx localtunnel --port 3000
15  "*.ngrok-free.app",
16  ...(process.env.ALLOWED_DEV_ORIGINS ?? "").split(",").map((origin) => origin.trim()).filter(Boolean),
17  ];
18
19  const nextConfig: NextConfig = {
20    // Embedded PostgreSQL (WASM) must be loaded from node_modules at runtime, not bundled.
21    serverExternalPackages: ["@electric-sql/pglite", "pg"],
22    allowedDevOrigins: devOrigins,
23    async redirects() {
24      return [
25        { source: "/mandi", destination: "/mandi-rates", permanent: false },
26        { source: "/dashboard", destination: "/login", permanent: false },
27        { source: "/signup", destination: "/register", permanent: false },
28      ];
29    },
30  };
31
32  export default nextConfig;
33

```

drizzle.config.json

```

1  {
2    "dialect": "postgresql",
3    "schema": "./src/db/schema.ts",
4    "dbCredentials": {
5      "url": "postgresql://postgres:postgres@127.0.0.1:5432/app_db"
6    }
7  }
8

```

postcss.config.mjs

```

1  const postcssConfig = {
2    plugins: {
3      "@tailwindcss/postcss": {},
4    },
5  };
6
7  export default postcssConfig;
8

```

eslint.config.mjs

```

1  import { defineConfig, globalIgnores } from "eslint/config";
2  import nextCoreWebVitals from "eslint-config-next/core-web-vitals";
3
4  export default defineConfig([
5    // Keep the starter on the flat config export that actually runs under the pinned ESLint/Next toolchain.
6    ...nextCoreWebVitals,
7    globalIgnores([".next/**", "out/**", "build/**", "next-env.d.ts"]),
8  ]);
9

```

Dockerfile

```

1  # MarketLink Agri-Hub Pakistan – production container
2  FROM node:20-alpine
3
4  WORKDIR /app
5  ENV NEXT_TELEMETRY_DISABLED=1
6
7  # Install dependencies (cached layer)
8  COPY package.json package-lock.json ./
9  RUN npm ci
10
11 # Build the app. A placeholder DATABASE_URL is enough at build time –
12 # the real one is supplied at runtime (docker-compose / cloud host).
13 COPY . .
14 ARG DATABASE_URL=postgresql://build:build@localhost:5432/build
15 ENV DATABASE_URL=${DATABASE_URL}

```

```

16 RUN npm run build
17
18 ENV NODE_ENV=production
19 ENV PORT=3000
20 EXPOSE 3000
21
22 HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 \
23   CMD wget -qO- http://127.0.0.1:3000/api/health || exit 1
24
25 # Tables and demo data are created automatically on first request.
26 CMD ["npm", "start"]
27

```

docker-compose.yml

```

1  # One-command local hosting: docker compose up --build
2  # Then open http://localhost:3000
3  services:
4    db:
5      image: postgres:16-alpine
6      restart: unless-stopped
7      environment:
8        POSTGRES_USER: postgres
9        POSTGRES_PASSWORD: postgres
10       POSTGRES_DB: app_db
11      volumes:
12        - marketlink_pgdata:/var/lib/postgresql/data
13      ports:
14        - "5432:5432"
15      healthcheck:
16        test: ["CMD-SHELL", "pg_isready -U postgres -d app_db"]
17        interval: 5s
18        timeout: 5s
19        retries: 10
20
21    app:
22      build: .
23      restart: unless-stopped
24      depends_on:
25        db:
26          condition: service_healthy
27      environment:
28        DATABASE_URL: postgresql://postgres:postgres@db:5432/app_db
29        SESSION_SECRET: change-this-to-a-long-random-string
30      ports:
31        - "3000:3000"
32
33    volumes:
34      marketlink_pgdata:
35

```

render.yaml

```

1  # Render.com Blueprint – free web service + free PostgreSQL.
2  # Push this project to GitHub, then on Render: New → Blueprint → select the repo.
3  databases:
4    - name: marketlink-db
5      plan: free
6      databaseName: app_db
7      user: marketlink
8
9  services:
10   - type: web
11     name: marketlink-agri-hub
12     runtime: node
13     plan: free
14     region: singapore
15     buildCommand: npm ci && npm run build
16     startCommand: npm start
17     healthCheckPath: /api/health
18     envVars:
19       - key: NODE_VERSION
20         value: "20"
21       - key: DATABASE_URL

```

```

22         fromDatabase:
23             name: marketlink-db
24             property: connectionString
25         - key: SESSION_SECRET
26           generateValue: true
27         - key: COOKIE_SECURE
28           value: "true"
29

```

.env.example

```

1  # MarketLink Agri-Hub Pakistan: settings
2  #
3  # DATABASE
4  #   Leave DATABASE_URL empty to use the BUILT-IN database (no installation needed).
5  #   Data is stored in the ".data" folder next to this file.
6  #
7  #   To use your own PostgreSQL server instead, uncomment and edit:
8  #   DATABASE_URL=postgresql://postgres:YOUR_PASSWORD@localhost:5432/app_db
9  #   (the app_db database is created automatically if it doesn't exist)
10 DATABASE_URL=
11
12 # ADMIN LOGIN (applied on the FIRST start, when the database is created).
13 #   Leave empty to use the default Super Admin: Ahmed Mustafa · ahmed.mustafa@admin.com / password123
14 #   After that, change the password in the app: Account settings.
15 ADMIN_EMAIL=
16 ADMIN_PASSWORD=
17
18 # Any long random text, used to sign login cookies.
19 SESSION_SECRET=change-this-to-a-long-random-string
20
21 # Set to "true" only when the site is served over HTTPS (cloud hosting).
22 COOKIE_SECURE=false
23
24 # -----
25 # PUBLIC URL (used in payment redirect/return URLs, OAuth callbacks and email links)
26 #
27 #   Leave EMPTY (recommended): the app derives its public origin from the request itself
28 #   (x-forwarded-host / host + x-forwarded-proto), so payment redirects and links always match
29 #   the domain the visitor is using – local, LAN, tunnel, Docker or cloud. Set it only when your
30 #   proxy does not forward the original host (e.g. APP_URL=https://marketlink.example.pk).
31 #
32 #   Wildcard/bind addresses (http://0.0.0.0:3000, http://[:]:3000) and malformed values are
33 #   ignored, because a browser cannot open them (ERR_ADDRESS_INVALID).
34 #   NEXT_PUBLIC_APP_URL / PUBLIC_URL / BASE_URL are accepted as aliases, in that order.
35 APP_URL=
36
37 # PAYMENTS: wallet top-ups are credited only by verified gateway callbacks.
38 PAYMENTS_MODE=sandbox           # sandbox | live (JazzCash/Easypaisa endpoints)
39 PAYMENTS_SANDBOX=true           # built-in test gateway; set false in production
40 STRIPE_SECRET_KEY=              # sk_test_... / sk_live_...
41 STRIPE_WEBHOOK_SECRET=          # whsec_... endpoint: /api/payments/webhook/stripe
42 JAZZCASH_MERCHANT_ID=
43 JAZZCASH_PASSWORD=
44 JAZZCASH_INTEGRITY_SALT=
45 EASYPAISA_STORE_ID=
46 EASYPAISA_HASH_KEY=
47 EASYPAISA_USERNAME=             # for server-to-server transaction inquiry
48 EASYPAISA_PASSWORD=
49 EASYPAISA_ACCOUNT_NUM=
50
51 # NOTIFICATIONS: without keys, emails/SMS are written to the server log (outbox)
52 RESEND_API_KEY=                 # email (resend.com)
53 EMAIL_FROM=MarketLink <noreply@yourdomain.pk>
54 TWILIO_ACCOUNT_SID=            # SMS
55 TWILIO_AUTH_TOKEN=
56 TWILIO_FROM=
57
58 # AI ASSISTANT: one of these enables the LLM (bilingual English / Roman Urdu)
59 OPENAI_API_KEY=
60 OPENAI_MODEL=gpt-4o-mini
61 GEMINI_API_KEY=
62 GEMINI_MODEL=gemini-1.5-flash
63
64 # GOOGLE SIGN-IN (OAuth 2.0): create credentials at console.cloud.google.com → APIs & Services → Credentia
65 ls

```

```

65 # Authorized redirect URI must exactly match GOOGLE_CALLBACK_URL.
66 GOOGLE_CLIENT_ID=
67 GOOGLE_CLIENT_SECRET=
68 GOOGLE_CALLBACK_URL=          # leave empty to auto-detect (APP_URL or the request host)
69 GOOGLE_OAUTH MOCK=true        # test-mode chooser while Google keys are missing; set false in productio
70 n
71 # PASSWORD RESET: without an email provider, reset links for non-staff accounts are shown on screen.
72 AUTH_DEV_LINKS=true           # set false (or configure RESEND_API_KEY) before real users sign up
73

```

start-windows.bat

```

1 @echo off
2 title MarketLink Agri-Hub Pakistan
3 cd /d "%~dp0"
4 echo =====
5 echo    MarketLink Agri-Hub Pakistan - Local Server
6 echo    Keep this window OPEN while using the website.
7 echo =====
8 echo.
9
10 where node >nul 2>nul
11 if %errorlevel% neq 0 (
12     echo [ERROR] Node.js is not installed.
13     echo Download the LTS version from https://nodejs.org , install it,
14     echo then double-click this file again.
15     start "" https://nodejs.org
16     pause
17     exit /b 1
18 )
19
20 node scripts\serve.cjs %*
21 echo.
22 echo Server stopped.
23 pause
24

```

start-docker.bat

```

1 @echo off
2 title MarketLink Agri-Hub Pakistan (Docker)
3 echo Starting MarketLink with Docker (PostgreSQL included)...
4 echo First run downloads images and builds - may take 3-5 minutes.
5 echo When you see "Ready", open http://localhost:3000
6 echo.
7 docker compose up --build
8 pause
9

```

start.sh

```

1 #!/usr/bin/env bash
2 # MarketLink Agri-Hub Pakistan: local server (macOS / Linux)
3 #   bash start.sh           → built-in database, opens browser automatically
4 #   bash start.sh docker    → Docker + PostgreSQL
5 cd "$(dirname "$0")"
6
7 if [ "$1" = "docker" ]; then
8     echo "Starting with Docker (PostgreSQL included) → http://localhost:3000"
9     exec docker compose up --build
10 fi
11
12 command -v node >/dev/null || { echo "Node.js 20+ is required: https://nodejs.org"; exit 1; }
13 exec node scripts/serve.cjs "$@"
14

```

src/app/(app)/account/page.tsx

```
1  import Link from "next/link";
2  import { requirePageUser } from "@lib/auth";
3  import { PageHeader } from "@components/ui";
4  import { CITIES, formatDate } from "@lib/constants";
5  import { ChangePasswordForm } from "../password-form";
6  import { ProfileForm } from "../profile-form";
7
8  export const dynamic = "force-dynamic";
9
10 const ROLE_NAME = { admin: "Administrator", inspector: "Quality Inspector", farmer: "Farmer (Kisan)", buyer: "Buyer / Wholesaler" } as const;
11
12 export default async function AccountPage({ searchParams }: { searchParams: Promise<{ complete?: string }> }) {
13   const [u, sp] = await Promise.all([requirePageUser(), searchParams]);
14   const incomplete = !u.phone || !u.city;
15   const google = u.authProvider.startsWith("google") || !!u.googleId;
16   const initials = u.fullName.split(" ").map((p) => p[0]).slice(0, 2).join("");
17   return (
18     <div className="space-y-6">
19       <PageHeader title="Account Settings" urdu="[Urdu]" subtitle="Your profile, sign-in methods and security" />
20
21       {(incomplete || sp.complete) && (
22         <div className="card flex items-start gap-3 border-amber-300 bg-amber-50 p-4 dark:border-amber-700 dark:bg-amber-950/30">
23           <span className="text-2xl"></span>
24           <div className="text-sm">
25             <p className="font-bold text-amber-900 dark:text-amber-200">{incomplete ? "Complete your profile to start trading" : "Profile complete, you're all set!"}</p>
26             <p className="text-amber-800/90 dark:text-amber-300/80">{incomplete ? "Add your mobile number and city. Buyers and farmers need them for deals, deliveries and SMS alerts." : "Head to your dashboard to get started."}</p>
27           </div>
28         </div>
29       )}
30
31       <section className="card flex flex-wrap items-center gap-5 p-6">
32         {u.avatarUrl ? (
33           // eslint-disable-next-line @next/next/no-img-element
34           <img src={u.avatarUrl} alt="" referrerPolicy="no-referrer" className="h-16 w-16 rounded-full object-cover ring-4 ring-brand-100 dark:ring-brand-900" />
35         ) : (
36           <span className="grid h-16 w-16 place-items-center rounded-full bg-brand-900 text-xl font-extrabold text-gold-400">{initials}</span>
37         )}
38         <div className="min-w-0 flex-1">
39           <p className="text-xl font-extrabold">{u.fullName}</p>
40           <p className="text-sm text-slate-500">{u.role === "admin" ? (u.title ?? ROLE_NAME.admin) : ROLE_NAME[u.role]} · {u.email}</p>
41           <div className="mt-2 flex flex-wrap gap-1.5 text-[11px] font-semibold">
42             {u.role === "admin" && <span className="rounded-full bg-gold-400 px-2 py-0.5 text-brand-950">★ Super Admin</span>}
43             {u.userCode && <span className="rounded-full bg-slate-100 px-2 py-0.5 font-mono dark:bg-white/10">ID {u.userCode}</span>}
44             <span className="rounded-full bg-emerald-100 px-2 py-0.5 text-emerald-800 dark:bg-emerald-500/15 dark:text-emerald-300">{u.emailVerified ? "✓ Email verified" : "Email unverified"}</span>
45             {google && <span className="rounded-full bg-blue-100 px-2 py-0.5 text-blue-800 dark:bg-blue-500/15 dark:text-blue-300">G · Google linked</span>}
46             <span className="rounded-full bg-slate-100 px-2 py-0.5 dark:bg-white/10">Member since {formatDate(u.createdAt)}</span>
47           </div>
48         </div>
49         {!incomplete && <p className="text-sm text-slate-500"> {u.city}{CITIES[u.city] ? `, ${CITIES[u.city].province}` : ""}</p>}
50       </section>
51
52       <div className="grid gap-5 lg:grid-cols-2">
53         <ProfileForm initial={{ fullName: u.fullName, phone: u.phone, city: u.city, businessName: u.businessName ?? "", cnicId: u.cnicId ?? "", address: u.address ?? "" }} highlight={incomplete} />
54         {google && u.authProvider.startsWith("google") ? (
55           <div className="card space-y-3 p-6">
56             <h2 className="font-bold"> Password</h2>
```

```

57         <p className="text-sm text-slate-600 dark:text-slate-300">You signed up with Google, so there&
↵      apos;s no MarketLink password on this account yet. Keep using <b>Continue with Google</b>, or set a passwo
↵      rd to also sign in with email.</p>
58         <Link href={` /forgot-password?email=${encodeURIComponent(u.email)}`} className="btn-secondary"
↵      >Set a password by email link</Link>
59     </div>
60     ) : (
61         <ChangePasswordForm />
62     )}
63 </div>
64 </div>
65 );
66 }
67

```

src/app/(app)/account/password-form.tsx

```

1  "use client";
2
3  import { useState } from "react";
4  import { api, Spinner } from "@components/ui";
5  import { useToast } from "@components/providers";
6
7  export function ChangePasswordForm() {
8      const { push } = useToast();
9      const [f, setF] = useState({ currentPassword: "", newPassword: "", confirm: "" });
10     const [busy, setBusy] = useState(false);
11     const [show, setShow] = useState(false);
12
13     const strength = () => {
14         const p = f.newPassword;
15         let s = 0;
16         if (p.length >= 8) s++;
17         if (p.length >= 12) s++;
18         if (/[A-Z]/.test(p) && /[a-z]/.test(p)) s++;
19         if (/d/.test(p)) s++;
20         if (/^[A-Za-z0-9]/.test(p)) s++;
21         return s;
22     }();
23
24     async function submit(e: React.FormEvent) {
25         e.preventDefault();
26         if (f.newPassword !== f.confirm) return push({ kind: "error", title: "New passwords don't match" });
27         setBusy(true);
28         try {
29             await api("/api/account/password", { method: "PATCH", json: { currentPassword: f.currentPassword, ne
↵      wPassword: f.newPassword } });
30             push({ kind: "success", title: "Password changed", body: "Use your new password next time you sign i
↵      n." });
31             setF({ currentPassword: "", newPassword: "", confirm: "" });
32         } catch (err) {
33             push({ kind: "error", title: "Could not change password", body: (err as Error).message });
34         } finally {
35             setBusy(false);
36         }
37     }
38
39     const type = show ? "text" : "password";
40     return (
41         <form onSubmit={submit} className="card space-y-4 p-6">
42             <div>
43                 <h2 className="font-bold"> Change password</h2>
44                 <p className="text-xs text-slate-500">Minimum 8 characters. Mix letters, numbers and symbols for a
↵      strong password.</p>
45             </div>
46             <div><label className="label">Current password</label><input type={type} className="input" required
↵      value={f.currentPassword} onChange={(e) => setF({ ...f, currentPassword: e.target.value })} autoComplete="
↵      current-password" /></div>
47             <div>
48                 <label className="label">New password</label>
49                 <input type={type} className="input" required minLength={8} value={f.newPassword} onChange={(e) =>
↵      setF({ ...f, newPassword: e.target.value })} autoComplete="new-password" />
50                 <div className="mt-2 flex gap-1">
51                     {[0, 1, 2, 3, 4].map((i) => (
52                         <span key={i} className={`h-1.5 flex-1 rounded-full ${i < strength ? (strength <= 2 ? "bg-rose

```

```

52 -500" : strength <= 3 ? "bg-amber-500" : "bg-brand-600") : "bg-slate-200 dark:bg-slate-700"}`} />
53   )})
54   </div>
55   </div>
56   <div><label className="label">Confirm new password</label><input type={type} className="input" requi
57   red value={f.confirm} onChange={(e) => setF({ ...f, confirm: e.target.value })} autoComplete="new-password
58   " /></div>
59   <label className="flex items-center gap-2 text-sm text-slate-600 dark:text-slate-400"><input type="c
60   heckbox" checked={show} onChange={(e) => setShow(e.target.checked)} /> Show passwords</label>
61   <button className="btn-primary w-full" disabled={busy}>{busy && <Spinner />} Update password</button
62   >
63   </form>
64   );
65   }
66 }

```

src/app/(app)/account/profile-form.tsx

```

1  "use client";
2
3  import { useRouter } from "next/navigation";
4  import { useState } from "react";
5  import { api, Spinner } from "@components/ui";
6  import { CityOptions } from "@components/city-options";
7  import { useToast } from "@components/providers";
8
9  type P = { fullName: string; phone: string; city: string; businessName: string; cnicId: string; address: s
10  tring };
11
12  export function ProfileForm({ initial, highlight }: { initial: P; highlight: boolean }) {
13    const router = useRouter();
14    const { push } = useToast();
15    const [f, setF] = useState<P>(initial);
16    const [busy, setBusy] = useState(false);
17    const set = (k: keyof P) => (e: React.ChangeEvent<HTMLInputElement | HTMLSelectElement>) => setF({ ...f,
18    [k]: e.target.value });
19
20    async function save(e: React.FormEvent) {
21      e.preventDefault();
22      setBusy(true);
23      try {
24        await api("/api/account/profile", { method: "PATCH", json: { ...f, cnicId: f.cnicId || undefined, bu
25        sinessName: f.businessName || undefined, address: f.address || undefined } });
26        push({ kind: "success", title: "Profile saved" });
27        router.refresh();
28      } catch (err) {
29        push({ kind: "error", title: "Couldn't save profile", body: (err as Error).message });
30      } finally {
31        setBusy(false);
32      }
33    }
34
35    return (
36      <form onSubmit={save} className={`card grid grid-cols-2 gap-3 p-6 ${highlight ? "ring-2 ring-amber-400
37      " : ""}`>
38        <h2 className="col-span-2 font-bold">Profile</h2>
39        <div className="col-span-2"><label className="label">Full name</label><input className="input" requi
40        red value={f.fullName} onChange={set("fullName")} /></div>
41        <div className="col-span-2 sm:col-span-1"><label className="label">Mobile number</label><input class
42        Name="input" required value={f.phone} onChange={set("phone")} placeholder="03xx xxxxxx" /></div>
43        <div className="col-span-2 sm:col-span-1"><label className="label">City</label><select className="in
44        put" required value={f.city} onChange={set("city")}><option value="" disabled>Select city...</option><CityOp
45        tions /></select></div>
46        <div className="col-span-2 sm:col-span-1"><label className="label">Business / farm name</label><inpu
47        t className="input" value={f.businessName} onChange={set("businessName")} /></div>
48        <div className="col-span-2 sm:col-span-1"><label className="label">CNIC</label><input className="inp
49        ut" value={f.cnicId} onChange={set("cnicId")} placeholder="12345-1234567-1" /></div>
50        <div className="col-span-2"><label className="label">Address</label><input className="input" value={
51        f.address} onChange={set("address")} /></div>
52        <div className="col-span-2 flex justify-end"><button className="btn-primary" disabled={busy}>{busy &
53        & <Spinner />} Save profile</button></div>
54      </form>
55    );
56  }
57 }

```

src/app/(app)/admin/dashboard/page.tsx

```
1  import Link from "next/link";
2  import { eq } from "drizzle-orm";
3  import { db } from "@/db";
4  import { users } from "@/db/schema";
5  import { requireAdminPage, verifyPassword } from "@/lib/auth";
6  import { adminStats } from "@/lib/services/analytics";
7  import { listDisputes } from "@/lib/services/orders";
8  import { PayoutsPanel } from "@/components/admin/payouts-panel";
9  import { WorkflowTrail } from "@/components/dashboard/workflow-trail";
10 import { Badge, PageHeader, StatCard } from "@/components/ui";
11 import { DonutChart, SimpleBarChart, VolumeAreaChart } from "@/components/charts";
12 import { DELIVERY_STAGES, categoryLabel, formatKg, formatPKR, formatPKRShort } from "@/lib/constants";
13
14 export const dynamic = "force-dynamic";
15 export const metadata = { title: "Admin overview" };
16
17 export default async function AdminDashboard() {
18   const me = await requireAdminPage();
19   const [s, disputes] = await Promise.all([adminStats(), listDisputes()]);
20   const [pw] = await db.select({ hash: users.passwordHash }).from(users).where(eq(users.id, me.id)).limit(
21     1);
22   const usingDefaultPassword = !!pw && verifyPassword("password123", pw.hash);
23   const stageCounts = DELIVERY_STAGES.map((st) => ({ ...st, count: s.stages.find((x) => x.stage === st.key)
24     }?.count ?? 0 ));
25   const maxStage = Math.max(1, ...stageCounts.map((x) => x.count));
26
27   return (
28     <div className="space-y-6">
29       <PageHeader
30         title="System Analytics" urdu="[Urdu]"
31         subtitle={`Welcome back, ${me.fullName} · ${me.title ?? "Administrator"}`}
32         actions={
33           <>
34             <Link href="/admin/users" className="btn-secondary"> Users</Link>
35             <Link href="/admin/db-explorer" className="btn-secondary">[Urdu]Database</Link>
36             <Link href="/admin/verification" className="btn-secondary">[Urdu]Verify ({s.unverifiedFarmers
37               + s.pendingInspections})</Link>
38             <Link href="/admin/mandi" className="btn-primary"> Update mandi rates</Link>
39             <a href="/api/reports/summary" className="btn-secondary"> Platform report</a>
40           </>
41         }
42       </PageHeader>
43       {usingDefaultPassword && (
44         <div className="card flex flex-wrap items-center gap-3 border-rose-300 bg-rose-50 p-4 dark:border-
45           rose-800 dark:bg-rose-950/30">
46           <span className="text-2xl"></span>
47           <div className="flex-1 text-sm">
48             <p className="font-bold text-rose-900 dark:text-rose-200">You are using the default admin pass
49               word (password123)</p>
50             <p className="text-rose-800/80 dark:text-rose-300/80">Anyone who has read the demo instruction
51               s can sign in as admin. Change it now to secure your admin panel.</p>
52           </div>
53           <Link href="/account" className="btn-danger">Change password</Link>
54         </div>
55       )}
56       <div className="grid grid-cols-2 gap-3 sm:gap-4 xl:grid-cols-4">
57         <StatCard label="Transaction volume" value={formatPKRShort(s.totalVolume)} sub={` ${s.totalOrders}
58           orders · ${formatKg(s.totalKg)} `} icon="" />
59         <StatCard label="Active bids" value={s.activeBids} sub={` ${s.activeShipments} shipments in progres
60           s `} icon="" tone="amber" />
61         <StatCard label="Escrow held" value={formatPKRShort(s.escrowHeld)} sub={` Fees earned ${formatPKR(s
62           .platformFees)} `} icon="" tone="sky" />
63         <StatCard label="Open disputes" value={s.openDisputes} sub={` ${s.farmers} farmers · ${s.buyers} bu
64           yers `} icon="👁️[Urdu]" tone="rose" />
65       </div>
66       <WorkflowTrail title="Admin / System Controller workflow" steps=[
67         { label: "Manage access", detail: "Users and role approvals", href: "/admin/users", pending: s.unv
68           erifiedFarmers },
69         { label: "Monitor trading", detail: "Orders, escrow and reports", href: "/admin/dashboard", pendin
70           g: s.activeBids },
71         { label: "Certify quality", detail: "Inspection workflow", href: "/inspector/dashboard", pending:
72           s.pendingInspections },
73       ] />
74     </div>
75   );
76 }
```

```

61      { label: "Control rates", detail: "Daily mandi prices", href: "/admin/mandi" },
62      { label: "Resolve cases", detail: "Shipments and disputes", href: "/admin/orders", pending: s.open
63    Disputes },
64    ]} />
65
66    <div className="grid gap-4 lg:grid-cols-3">
67      <div className="card p-5 lg:col-span-2">
68        <h2 className="mb-3 font-semibold">Monthly transaction volume</h2>
69        <VolumeAreaChart data={s.monthly} />
70      </div>
71      <div className="card p-5">
72        <h2 className="mb-3 font-semibold">Volume by category</h2>
73        <DonutChart data={s.categories.map((c) => ({ name: categoryLabel(c.name), value: c.value }))) />
74      </div>
75    </div>
76
77    <div className="grid gap-4 lg:grid-cols-2">
78      <div className="card p-5">
79        <h2 className="mb-3 font-semibold">Top trading regions</h2>
80        <SimpleBarChart data={s.topRegions} xKey="region" yKey="volume" horizontal height={280} />
81      </div>
82      <div className="card p-5">
83        <h2 className="mb-4 font-semibold">Logistics pipeline</h2>
84        <div className="space-y-3">
85          {stageCounts.map((st) => (
86            <div key={st.key}>
87              <div className="mb-1 flex justify-between text-sm"><span>{st.label}</span><span className=
88                "font-semibold">{st.count}</span></div>
89              <div className="h-2.5 overflow-hidden rounded-full bg-slate-100 dark:bg-slate-800">
90                <div className="h-full rounded-full bg-gradient-to-r from-brand-500 to-brand-700" style=
91                  {{ width: `${(st.count / maxStage) * 100}%` }} />
92              </div>
93            </div>
94          ))}
95        </div>
96        <div className="mt-5 grid grid-cols-2 gap-3 text-sm">
97          <div className="rounded-xl bg-amber-50 p-3 dark:bg-amber-950/30"><p className="text-xs text-am
98            ber-700 dark:text-amber-300">Farmers awaiting verification</p><p className="text-xl font-bold">{s.unverifi
99            edFarmers}</p></div>
100          <div className="rounded-xl bg-sky-50 p-3 dark:bg-sky-950/30"><p className="text-xs text-sky-70
101            0 dark:text-sky-300">Listings pending inspection</p><p className="text-xl font-bold">{s.pendingInspections
102            }</p></div>
103        </div>
104      </div>
105    </div>
106    <PayoutsPanel />
107
108    <div className="card overflow-hidden">
109      <div className="flex items-center justify-between border-b border-slate-200 px-5 py-3.5 dark:borde
110        r-slate-800">
111        <h2 className="font-semibold">Dispute resolution log</h2>
112        <Link href="/admin/orders" className="text-sm font-semibold text-brand-600 hover:underline">Reso
113        lve disputes </Link>
114      </div>
115      <div className="overflow-x-auto">
116        <table className="w-full text-sm">
117          <thead className="table-head"><tr><th className="px-5 py-2.5">Order</th><th className="px-5 py
118            -2.5">Raised by</th><th className="px-5 py-2.5">Reason</th><th className="px-5 py-2.5">Amount</th><th clas
119            sName="px-5 py-2.5">Status</th><th className="px-5 py-2.5">Resolution</th></tr></thead>
120          <tbody className="divide-y divide-slate-100 dark:divide-slate-800">
121            {disputes.map((d) => (
122              <tr key={d.disputeId}>
123                <td className="px-5 py-3 font-mono text-xs">{d.trackingNumber}<br /><span className="fon
124                  t-sans text-slate-500">{d.cropName}</span></td>
125                <td className="px-5 py-3">{d.raisedByName} <span className="text-xs text-slate-500">({d.
126                  raisedByRole})</span></td>
127                <td className="max-w-xs px-5 py-3 text-slate-600 dark:text-slate-300">{d.reason}</td>
128                <td className="px-5 py-3 font-semibold">{formatPKR(d.totalAmount)}</td>
129                <td className="px-5 py-3"><Badge status={d.status} /></td>
130                <td className="max-w-xs px-5 py-3 text-xs text-slate-500">{d.resolution ?? "-"}</td>
131              </tr>
132            ))}
133            {disputes.length === 0 && <tr><td colspan={6} className="p-8 text-center text-slate-500">No
134              disputes logged</td></tr>}
135          </tbody>
136        </table>
137      </div>
138    </div>

```

```

123         </table>
124     </div>
125 </div>
126 </div>
127 );
128 }
129

```

src/app/(app)/admin/db-explorer/page.tsx

```

1  "use client";
2
3  import { useCallback, useEffect, useMemo, useState } from "react";
4  import { api, PageHeader, Spinner } from "@components/ui";
5  import { useToast } from "@components/providers";
6
7  type Column = { name: string; type: string; nullable: boolean; default: string | null; pk: boolean; fk: st
  ↵  ring | null };
8  type Table = { name: string; rows: number; columns: Column[]; indexes: { name: string; def: string }[] };
9  type Schema = { engine: string; tables: Table[]; audit: { id: number; mode: string; query: string; duratio
  ↵  nMs: number | null; error: string | null; at: string; admin: string | null }[] };
10 type Result = { columns: string[]; rows: (string | number | boolean | null)[][]; rowCount: number; affecte
  ↵  d: number; truncated: boolean; command: string; durationMs: number };
11
12 const SAMPLES: { label: string; sql: string }[] = [
13   { label: "Top sellers by trust", sql: "SELECT id, full_name, business_name, city, trust_score, rating_av
  ↵  g, rating_count\nFROM users WHERE role = 'farmer'\nORDER BY trust_score DESC" },
14   { label: "Revenue by month", sql: "SELECT to_char(created_at, 'YYYY-MM') AS month, count(*) AS orders, s
  ↵  um(total_amount) AS volume_pkr\nFROM orders_logistics WHERE payment_status <> 'refunded'\nGROUP BY 1 ORDER
  ↵  BY 1 DESC" },
15   { label: "Escrow exposure", sql: "SELECT payment_status, count(*) AS orders, sum(total_amount) AS amount
  ↵  _pkr\nFROM orders_logistics GROUP BY 1 ORDER BY 3 DESC" },
16   { label: "Open disputes", sql: "SELECT d.dispute_id, o.tracking_number, d.status, d.reason, d.created_at
  ↵  \nFROM disputes d JOIN orders_logistics o ON o.order_id = d.order_id\nWHERE d.status IN ('open','investiga
  ↵  ting')" },
17   { label: "Latest reviews", sql: "SELECT r.rating, r.comment, u.business_name AS seller, r.created_at\nFR
  ↵  OM reviews r JOIN users u ON u.id = r.farmer_id\nORDER BY r.created_at DESC LIMIT 20" },
18   { label: "Pending payments", sql: "SELECT reference, provider, amount, status, created_at FROM payment_i
  ↵  ntents ORDER BY created_at DESC LIMIT 50" },
19 ];
20
21 export default function DbExplorerPage() {
22   const { push } = useToast();
23   const [schema, setSchema] = useState<Schema | null>(null);
24   const [filter, setFilter] = useState("");
25   const [openTable, setOpenTable] = useState<string | null>(null);
26   const [sqlText, setSqlText] = useState(SAMPLES[0].sql);
27   const [mode, setMode] = useState<"read" | "write">("read");
28   const [result, setResult] = useState<Result | null>(null);
29   const [error, setError] = useState<string | null>(null);
30   const [busy, setBusy] = useState(false);
31   const [tab, setTab] = useState<"results" | "schema" | "audit">("results");
32
33   const loadSchema = useCallback(() => api<Schema>("/api/db/schema").then(setSchema).catch((e) => push({ k
  ↵  ind: "error", title: (e as Error).message })), [push]);
34   useEffect(() => { loadSchema(); }, [loadSchema]);
35
36   const run = useCallback(async (q = sqlText, m = mode) => {
37     if (m === "write" && !confirm("Run this WRITE statement against the live database? This is audited and
  ↵  cannot be undone.)) return;
38     setBusy(true);
39     setError(null);
40     try {
41       const r = await api<Result>("/api/db/query", { method: "POST", json: { sql: q, mode: m, confirm: m =
  ↵  == "write" } });
42       setResult(r);
43       setTab("results");
44       if (m === "write") { push({ kind: "success", title: `${r.command} · ${r.affected} row(s) affected` }
  ↵  ); loadSchema(); }
45     } catch (e) {
46       setError((e as Error).message);
47       setResult(null);
48       setTab("results");
49     } finally {
50       setBusy(false);
51     }

```

```

52   }, [sqlText, mode, push, loadSchema]);
53
54   function browse(t: Table) {
55     const pk = t.columns.find((c) => c.pk)?.name;
56     const q = `SELECT * FROM "${t.name}"${pk ? ` ORDER BY "${pk}" DESC` : ""} LIMIT 100`;
57     setSqlText(q);
58     setMode("read");
59     run(q, "read");
60   }
61
62   function exportCsv() {
63     if (!result) return;
64     const esc = (v: unknown) => (v === null ? "" : /[",\n]/.test(String(v)) ? `${String(v).replace(/"/g,
65     ↵   `""`)}` : String(v));
66     const csv = [result.columns.join(","), ...result.rows.map((r) => r.map(esc).join(",")).join("\n");
67     const a = document.createElement("a");
68     a.href = URL.createObjectURL(new Blob([csv], { type: "text/csv" }));
69     a.download = `query-${Date.now()}.csv`;
70     a.click();
71   }
72
73   const tables = useMemo(() => (schema?.tables ?? []).filter((t) => t.name.includes(filter.toLowerCase()))
74   ↵   , [schema, filter]);
75   const selected = schema?.tables.find((t) => t.name === openTable);
76
77   return (
78     <div className="space-y-5">
79       <PageHeader title="Database Explorer" urdu="Urdu" subtitle={schema ? `${schema.engine} · ${schema.
80       ↵   tables.length} tables · every query is audited` : "Loading schema..." } />
81
82       <div className="grid gap-5 xl:grid-cols-[280px_1fr]">
83         <div className="card h-fit overflow-hidden xl:sticky xl:top-24">
84           <div className="border-b border-slate-100 p-3 dark:border-white/5"><input className="input" plac
85           ↵   eholder="Filter tables..." value={filter} onChange={(e) => setFilter(e.target.value)} /></div>
86           <ul className="max-h-[60vh] overflow-y-auto p-2 text-sm">
87             {tables.map((t) => (
88               <li key={t.name}>
89                 <div className={`group flex items-center gap-2 rounded-lg px-2 py-1.5 ${openTable === t.name ? "bg-brand-50 dark:bg-brand-500/10" : "hover:bg-slate-50 dark:hover:bg-white/5"}`}>
90                   <button className="flex min-w-0 flex-1 items-center gap-2 text-left" onClick={() => { se
91                   ↵   tOpenTable(openTable === t.name ? null : t.name); setTab("schema"); }}>
92                     <span className="text-xs text-slate-400">{openTable === t.name ? "▼" : "▶"}</span>
93                     <span className="truncate font-mono text-xs font-semibold">{t.name}</span>
94                   </button>
95                   <span className="text-[10px] text-slate-400 tabular-nums">{t.rows.toLocaleString()}</spa
96                   ↵   n>
97                   <button onClick={() => browse(t)} className="rounded px-1.5 text-[10px] font-bold text-b
98                   ↵   rand-700 opacity-0 group-hover:opacity-100 hover:bg-brand-100 dark:text-brand-300" title="Browse rows">▶</
99                   ↵   button>
100                 </div>
101                 {openTable === t.name && (
102                   <ul className="mb-1 ml-6 border-l border-slate-200 pl-2 dark:border-white/10">
103                     {t.columns.map((c) => (
104                       <li key={c.name} className="flex items-center gap-1.5 py-0.5 font-mono text-[11px]">
105                         <span className={c.pk ? "text-gold-500" : c.fk ? "text-sky-600" : "text-slate-400"}>
106                         ↵   {c.pk ? "" : c.fk ? "" : "."}</span>
107                         <span className="truncate">{c.name}</span>
108                         <span className="ml-auto shrink-0 text-slate-400">{c.type}</span>
109                       </li>
110                     )})
111                   </ul>
112                 )})
113               </li>
114             )})
115           </ul>
116         </div>
117         <div className="min-w-0 space-y-4">
118           <div className="card overflow-hidden">
119             <div className="flex flex-wrap items-center gap-2 border-b border-slate-100 px-4 py-2.5 dark:b
120             ↵   order-white/5">
121               <div className="flex rounded-lg border border-slate-200 p-0.5 text-xs font-semibold dark:bor
122               ↵   der-white/10" role="group" aria-label="Mode">
123                 <button onClick={() => setMode("read")} className={`rounded-md px-3 py-1 ${mode === "read"

```

```

116     ? "bg-brand-900 text-white" : "text-slate-500"`)> Read-only</button>
117     <button onClick={() => setMode("write")} className={`rounded-md px-3 py-1 ${mode === "write" ? "bg-rose-600 text-white" : "text-slate-500"}`}>[Urdu]Write</button>
118     </div>
119     <select className="input w-auto py-1 text-xs" value="" onChange={(e) => { const s = SAMPLES.find((x) => x.label === e.target.value); if (s) { setSqlText(s.sql); setMode("read"); } }} aria-label="Sample queries">
120         <option value="">Sample queries...</option>
121         {SAMPLES.map((s) => <option key={s.label}>{s.label}</option>)}
122     </select>
123     <span className="ml-auto hidden text-[11px] text-slate-400 sm:block">Ctrl + Enter to run</span>
124     <button className={mode === "write" ? "btn-danger py-1.5" : "btn-primary py-1.5"} disabled={busy || !sqlText.trim()} onClick={() => run()}>{busy ? <Spinner /> : "▶"} Run</button>
125     </div>
126     <textarea value={sqlText} onChange={(e) => setSqlText(e.target.value)} onKeyDown={(e) => { if ((e.ctrlKey || e.metaKey) && e.key === "Enter") { e.preventDefault(); run(); } }} spellCheck={false} rows={7} aria-label="SQL query"
127     className={`block w-full resize-y bg-brand-950 p-4 font-mono text-[13px] leading-6 text-emerald-100 outline-none ${mode === "write" ? "ring-2 ring-rose-500 ring-inset" : ""}`} />
128     {mode === "write" && <p className="bg-rose-50 px-4 py-2 text-xs text-rose-800 dark:bg-rose-950 dark:text-rose-200">Write mode: INSERT / UPDATE / DELETE / DDL run for real after confirmation. Dangerous statements (DROP DATABASE, GRANT, file access...) are blocked.</p>}
129     </section>
130
131     { /* Output */ }
132     <section className="card overflow-hidden">
133     <div className="flex items-center gap-1 border-b border-slate-100 px-3 pt-2 dark:border-white/5">
134         {(["results", "schema", "audit"] as const).map((t) => (
135             <button key={t} onClick={() => setTab(t)} className={`rounded-t-lg px-3 py-2 text-xs font-semibold capitalize ${tab === t ? "border-b-2 border-brand-600 text-brand-800 dark:text-brand-300" : "text-slate-500"}`}>{t === "schema" ? `Schema${selected ? ` ` : ` `}` : t === "audit" ? "Audit log" : "Results"}</button>
136         ))}
137         {tab === "results" && result && (
138             <div className="ml-auto flex items-center gap-3 pb-1 text-[11px] text-slate-500">
139                 <span>{result.command} · {result.rowCount} row(s){result.truncated ? " (first 500 shown)" : ""} · {result.durationMs} ms</span>
140                 {result.rows.length > 0 && <button onClick={exportCsv} className="rounded-md border border-slate-200 px-2 py-0.5 font-semibold hover:bg-slate-50 dark:border-white/10"> CSV</button>}
141             </div>
142         )}
143     </div>
144     {tab === "results" && (
145         error ? <pre className="m-4 overflow-x-auto rounded-xl bg-rose-50 p-4 text-xs whitespace-pre-wrap text-rose-800 dark:bg-rose-950/30 dark:text-rose-200">{error}</pre>
146         : !result ? <p className="p-8 text-center text-sm text-slate-500">Run a query or click ▶ next to a table to browse its rows.</p>
147         : result.columns.length === 0 ? <p className="p-8 text-center text-sm text-slate-500">{result.command} completed · {result.affected} row(s) affected.</p>
148         : (
149             <div className="max-h-[520px] overflow-auto">
150                 <table className="w-full font-mono text-xs">
151                     <thead className="table-head sticky top-0"><tr><th className="px-3 py-2 text-slate-400">#</th>{result.columns.map((c) => <th key={c} className="px-3 py-2 whitespace-nowrap normal-case">{c}</th>)}</tr></thead>
152                     <tbody className="divide-y divide-slate-100 dark:divide-white/5">
153                         {result.rows.map((r, i) => (
154                             <tr key={i} className="hover:bg-slate-50 dark:hover:bg-white/[0.03]">
155                                 <td className="px-3 py-1.5 text-slate-400">{i + 1}</td>
156                                 {r.map((v, j) => <td key={j} className={`max-w-[320px] truncate px-3 py-1.5 whitespace-nowrap ${v === null ? "text-slate-400 italic" : typeof v === "number" ? "text-right tabular-nums" : ""}`}>{v === null ? "NULL" : String(v)}>{v === null ? "NULL" : String(v)}</td>)}
157                             </tr>
158                         ))}
159                     </tbody>
160                 </table>
161             </div>
162         )
163     )}
164     {tab === "schema" && (
165         !selected ? <p className="p-8 text-center text-sm text-slate-500">Select a table on the left to inspect its structure.</p> : (
166             <div className="space-y-4 p-4">
167                 <div className="overflow-x-auto">

```

```

168         <table className="w-full text-xs">
169             <thead className="table-head"><tr><th className="px-3 py-2">Column</th><th className
    ↵   ="px-3 py-2">Type</th><th className="px-3 py-2">Null</th><th className="px-3 py-2">Default</th><th classNa
    ↵   me="px-3 py-2">Key</th></tr></thead>
170             <tbody className="divide-y divide-slate-100 font-mono dark:divide-white/5">
171                 {selected.columns.map((c) => (
172                     <tr key={c.name}><td className="px-3 py-1.5 font-semibold">{c.name}</td><td clas
    ↵   sName="px-3 py-1.5">{c.type}</td><td className="px-3 py-1.5">{c.nullable ? "YES" : "NO"}</td><td className
    ↵   ="max-w-[220px] truncate px-3 py-1.5 text-slate-500">{c.default ?? ""}</td><td className="px-3 py-1.5">{c.
    ↵   pk ? "PK" : c.fk ? `FK → ${c.fk}` : ""}</td></tr>
173                 )))
174             </tbody>
175         </table>
176     </div>
177     <div>
178         <p className="label">Indexes</p>
179         <ul className="space-y-1 font-mono text-[11px] text-slate-600 dark:text-slate-400">{se
    ↵   lected.indexes.map((i) => <li key={i.name}>{i.def}</li>)}</ul>
180     </div>
181 </div>
182 )
183 })
184 {tab === "audit" && (
185     <ul className="max-h-[520px] divide-y divide-slate-100 overflow-y-auto text-xs dark:divide-w
    ↵   hite/5">
186         {schema?.audit.map((a) => (
187             <li key={a.id} className="px-4 py-2.5">
188                 <div className="flex items-center gap-2">
189                     <span className={`rounded px-1.5 py-0.5 text-[10px] font-bold uppercase ${a.mode ===
    ↵   "write" ? "bg-rose-100 text-rose-700" : "bg-slate-100 text-slate-600 dark:bg-white/10 dark:text-slate-300
    ↵   "`}>{a.mode}</span>
190                     <span className="text-slate-500">{a.admin} · {new Date(a.at).toLocaleString("en-GB")}
    ↵   } · {a.durationMs} ms</span>
191                     {a.error && <span className="text-rose-600">error</span>}
192                 </div>
193                 <pre className="mt-1 truncate font-mono text-slate-700 dark:text-slate-300">{a.query}<
    ↵   /pre>
194             </li>
195         )))
196         {!schema?.audit.length && <li className="p-6 text-center text-slate-500">No queries yet</li>
    ↵   }
197     </ul>
198 )}
199 </section>
200 </div>
201 </div>
202 </div>
203 );
204 }
205

```

src/app/(app)/admin/layout.tsx

```

1  import type { ReactNode } from "react";
2  import { requireAdminPage } from "@/lib/auth";
3
4  /** Every /admin/* page is admin-only (also enforced earlier in src/proxy.ts). */
5  export default async function AdminLayout({ children }: { children: ReactNode }) {
6      await requireAdminPage();
7      return <>{children}</>;
8  }
9

```

src/app/(app)/admin/mandi/page.tsx

```

1  "use client";
2
3  import { useCallback, useEffect, useMemo, useState } from "react";
4  import { api, LoadingBlock, PageHeader, Spinner } from "@components/ui";
5  import { useToast } from "@components/providers";
6  import { CROP_CATALOGUE, MANDIS, MANDI_MARKETS, formatPKR, mandiName, todayISO } from "@lib/constants";
7
8  type Latest = { rateId: number; cropName: string; marketLocation: string; minPricePerKg: number; maxPriceP
    ↵   erKg: number; avgPricePerKg: number; changePct: number | null; source: string; dateUpdated: string };
9

```

```

9
10 export default function MandiController() {
11   const { push } = useToast();
12   const [rows, setRows] = useState<Latest[] | null>(null);
13   const [crop, setCrop] = useState("");
14   const [form, setForm] = useState({ cropName: "Wheat", marketLocation: "Lahore", minPricePerKg: "", maxPricePerKg: "", avgPricePerKg: "", dateUpdated: todayISO() });
15   const [busy, setBusy] = useState<string | null>(null);
16   const [lastSync, setLastSync] = useState<string | null>(null);
17
18   const load = useCallback(() => api<Latest[]>("/api/mandi-rates?view=latest").then(setRows).catch((e) =>
19     push({ kind: "error", title: e.message })), [push]);
20   useEffect(() => { load(); }, [load]);
21
22   function prefill(r: Latest) {
23     setForm({ cropName: r.cropName, marketLocation: r.marketLocation, minPricePerKg: String(r.minPricePerKg), maxPricePerKg: String(r.maxPricePerKg), avgPricePerKg: String(r.avgPricePerKg), dateUpdated: todayISO() });
24     window.scrollTo({ top: 0, behavior: "smooth" });
25   }
26
27   async function save(e: React.FormEvent) {
28     e.preventDefault();
29     setBusy("save");
30     try {
31       await api("/api/mandi-rates", {
32         method: "POST",
33         json: { cropName: form.cropName, marketLocation: form.marketLocation, minPricePerKg: Number(form.minPricePerKg), maxPricePerKg: Number(form.maxPricePerKg), avgPricePerKg: form.avgPricePerKg ? Number(form.avgPricePerKg) : undefined, dateUpdated: form.dateUpdated },
34       });
35       push({ kind: "success", title: "Rate published", body: `${form.cropName} @ ${form.marketLocation}` });
36       load();
37     } catch (err) {
38       push({ kind: "error", title: (err as Error).message });
39     } finally { setBusy(null); }
40   }
41
42   async function sync() {
43     setBusy("sync");
44     try {
45       const r = await api<{ updated: number; date: string; syncedAt: string }>("/api/mandi-rates/sync", {
46         method: "POST" });
47       setLastSync(new Date(r.syncedAt).toLocaleTimeString());
48       push({ kind: "success", title: "Mock feed synced", body: `${r.updated} rates updated for ${r.date}` });
49       load();
50     } catch (err) {
51       push({ kind: "error", title: (err as Error).message });
52     } finally { setBusy(null); }
53   }
54
55   const shown = useMemo(() => (rows ?? []).filter((r) => !crop || r.cropName === crop), [rows, crop]);
56
57   return (
58     <div className="space-y-6">
59       <PageHeader title="Mandi Price Controller" subtitle="Publish and sync daily mandi rates" />
60
61       <div className="grid gap-4 lg:grid-cols-3">
62         <form onSubmit={save} className="card grid grid-cols-2 gap-3 p-5 lg:col-span-2">
63           <h2 className="col-span-2 font-semibold">[Urdu]Manual rate entry</h2>
64           <div><label className="label">Crop</label><select className="input" value={form.cropName} onChange={(e) => setForm({ ...form, cropName: e.target.value })}>{CROP_CATALOGUE.map((c) => <option key={c.name}>{c.name}</option>)}</select></div>
65           <div><label className="label">Mandi</label><select className="input" value={form.marketLocation} onChange={(e) => setForm({ ...form, marketLocation: e.target.value })}>{MANDIS.map((m) => <option key={m.name}>{m.name}</option>)}</select></div>
66           <div><label className="label">Min Rs./kg</label><input type="number" step="0.01" className="input" value={form.minPricePerKg} onChange={(e) => setForm({ ...form, minPricePerKg: e.target.value })} required /></div>
67           <div><label className="label">Max Rs./kg</label><input type="number" step="0.01" className="input" value={form.maxPricePerKg} onChange={(e) => setForm({ ...form, maxPricePerKg: e.target.value })} required /></div>
68           <div><label className="label">Avg Rs./kg (optional)</label><input type="number" step="0.01" className="input" value={form.avgPricePerKg} onChange={(e) => setForm({ ...form, avgPricePerKg: e.target.value

```

```

    }} placeholder="auto = (min+max)/2" /></div>
67     <div><label className="label">Date</label><input type="date" max={todayISO()} className="input"
    value={form.dateUpdated} onChange={(e) => setForm({ ...form, dateUpdated: e.target.value })} /></div>
68     <div className="col-span-2 flex justify-end"><button className="btn-primary" disabled={busy ===
    "save"}>{busy === "save" && <Spinner />} Publish rate</button></div>
69   </form>
70   <div className="card flex flex-col p-5">
71     <h2 className="font-semibold">Market feed sync</h2>
72     <p className="mt-2 flex-1 text-sm text-slate-600 dark:text-slate-400">
73       Pull today's rates for every crop and mandi from the connected market feed. Existing entr
    ies for today are updated.
74     </p>
75     {lastSync && <p className="mt-2 text-xs text-brand-600">Last synced at {lastSync}</p>}
76     <button className="btn-amber mt-4" onClick={sync} disabled={busy === "sync"}>{busy === "sync" ?
    <Spinner /> : "<"} Sync {CROP_CATALOGUE.length * MANDI_MARKETS.length} rates now</button>
77   </div>
78 </div>
79
80   <div className="card overflow-hidden">
81     <div className="flex flex-wrap items-center justify-between gap-3 border-b border-slate-200 px-5 p
    y-3.5 dark:border-slate-800">
82       <h2 className="font-semibold">Latest published rates</h2>
83       <select className="input w-44" value={crop} onChange={(e) => setCrop(e.target.value)}><option va
    lue="">All crops</option>{CROP_CATALOGUE.map((c) => <option key={c.name}>{c.name}</option>)}</select>
84     </div>
85     {!rows ? <div className="p-5"><LoadingBlock /></div> : (
86       <div className="max-h-[560px] overflow-auto">
87         <table className="w-full text-sm">
88           <thead className="table-head sticky top-0"><tr><th className="px-4 py-2.5">Crop</th><th clas
    sName="px-4 py-2.5">Mandi</th><th className="px-4 py-2.5">Min</th><th className="px-4 py-2.5">Max</th><th
    className="px-4 py-2.5">Avg</th><th className="px-4 py-2.5">/maund</th><th className="px-4 py-2.5">Δ</th><th
    className="px-4 py-2.5">Source</th><th className="px-4 py-2.5">Date</th><th className="px-4 py-2.5"></th></tr></thead>
89           <tbody className="divide-y divide-slate-100 dark:divide-slate-800">
90             {shown.map((r) => (
91               <tr key={r.rateId}>
92                 <td className="px-4 py-2.5 font-medium">{r.cropName}</td>
93                 <td className="px-4 py-2.5">{mandiName(r.marketLocation)}</td>
94                 <td className="px-4 py-2.5">{formatPKR(r.minPricePerKg, 2)}</td>
95                 <td className="px-4 py-2.5">{formatPKR(r.maxPricePerKg, 2)}</td>
96                 <td className="px-4 py-2.5 font-bold">{formatPKR(r.avgPricePerKg, 2)}</td><td classNam
    e="px-4 py-2.5">{formatPKR(r.avgPricePerKg * 40)}</td>
97                 <td className={`px-4 py-2.5 text-xs font-semibold ${r.changePct === null ? "" : r.chan
    gePct >= 0 ? "text-emerald-600" : "text-rose-600"}`}>{r.changePct === null ? "-" : ` ${r.changePct >= 0 ? "
    +" : ""} ${r.changePct.toFixed(2)}%`}</td>
98                 <td className="px-4 py-2.5"><span className="badge bg-slate-100 dark:bg-slate-800">{r.
    source}</span></td>
99                 <td className="px-4 py-2.5 text-slate-500">{r.dateUpdated}</td>
100                 <td className="px-4 py-2.5 text-right"><button className="text-xs font-semibold text-b
    rand-600 hover:underline" onClick={() => prefill(r)}>Edit</button></td>
101               </tr>
102             )})
103           </tbody>
104         </table>
105       </div>
106     )}
107   </div>
108 </div>
109 );
110 }
111

```

src/app/(app)/admin/orders/page.tsx

```

1  import { AdminOrdersTable } from "@components/admin/orders-table";
2
3  export const metadata = { title: "Admin orders" };
4
5  export default function AdminOrdersPage() {
6    return <AdminOrdersTable />;
7  }
8

```

```
src/app/(app)/admin/page.tsx
```

```
1 import { redirect } from "next/navigation";
2
3 export default function Page() {
4   redirect("/admin/dashboard");
5 }
6
```

```
src/app/(app)/admin/users/page.tsx
```

```

1  "use client";
2
3  import { useCallback, useEffect, useMemo, useState } from "react";
4  import { api, Badge, LoadingBlock, Modal, PageHeader, Spinner, StatCard } from "@components/ui";
5  import { CityOptions } from "@components/city-options";
6  import { useToast } from "@components/providers";
7  import { formatDate, formatPKRShort } from "@lib/constants";
8
9  type U = { id: number; fullName: string; email: string; phone: string; role: "farmer" | "buyer" | "admin"
  | "inspector"; cnicId: string | null; businessName: string | null; city: string; isVerified: boolean; wall
  etBalance: number; listings: number; orders: number; createdAt: string };
10
11  const ROLE_BADGE: Record<string, string> = {
12    admin: "bg-brand-900 text-white",
13    farmer: "bg-emerald-100 text-emerald-800 dark:bg-emerald-500/15 dark:text-emerald-300",
14    buyer: "bg-sky-100 text-sky-800 dark:bg-sky-500/15 dark:text-sky-300",
15    inspector: "bg-violet-100 text-violet-800 dark:bg-violet-500/15 dark:text-violet-300",
16  };
17  const emptyNew = { fullName: "", email: "", password: "", phone: "", role: "admin" as U["role"], city: "La
  hore", cnicId: "", businessName: "" };
18
19  export default function AdminUsersPage() {
20    const { push } = useToast();
21    const [users, setUsers] = useState<U[] | null>(null);
22    const [role, setRole] = useState<"" | U["role"]>("");
23    const [q, setQ] = useState("");
24    const [creating, setCreating] = useState(false);
25    const [nu, setNu] = useState(emptyNew);
26    const [resetFor, setResetFor] = useState<U | null>(null);
27    const [newPw, setNewPw] = useState("");
28    const [busy, setBusy] = useState<string | null>(null);
29
30    const load = useCallback(() => api<U[]>("/api/admin/users").then(setUsers).catch((e) => push({ kind: "er
  ror", title: e.message })), [push]);
31    useEffect(() => { load(); }, [load]);
32
33    const shown = useMemo(
34      () => (users ?? []).filter((u) => (!role || u.role === role) && (!q || `${u.fullName} ${u.email} ${u.c
  ity} ${u.businessName} ? ""` .toLowerCase().includes(q.toLowerCase()))),
35      [users, role, q],
36    );
37
38    async function create(e: React.FormEvent) {
39      e.preventDefault();
40      setBusy("create");
41      try {
42        await api("/api/admin/users", { method: "POST", json: { ...nu, cnicId: nu.cnicId || undefined, busin
  essName: nu.businessName || undefined } });
43        push({ kind: "success", title: `${nu.role === "admin" ? "Admin" : "User"} account created`, body: `
  ${nu.email} can now sign in.` });
44        setCreating(false);
45        setNu(emptyNew);
46        load();
47      } catch (err) {
48        push({ kind: "error", title: "Could not create account", body: (err as Error).message });
49      } finally { setBusy(null); }
50    }
51
52    async function patch(u: U, json: Record<string, unknown>, ok: string) {
53      setBusy(`u${u.id}`);
54      try {
55        await api(`/api/admin/users/${u.id}`, { method: "PATCH", json });
56        push({ kind: "success", title: ok });

```

```

57     load();
58     return true;
59   } catch (err) {
60     push({ kind: "error", title: (err as Error).message });
61     return false;
62   } finally { setBusy(null); }
63 }
64
65 function genPassword() {
66   const chars = "ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz23456789@#\$";
67   return Array.from({ length: 12 }, () => chars[Math.floor(Math.random() * chars.length)]).join("");
68 }
69
70 if (!users) return <LoadingBlock />;
71 const count = (r: string) => users.filter((u) => u.role === r).length;
72
73 return (
74   <div className="space-y-6">
75     <PageHeader title="Users & Access" urdu="[Urdu]" subtitle="Accounts, roles and passwords" actions={
76       <button className="btn-primary" onClick={() => { setNu({ ...emptyNew, password: genPassword() }); setCreati
77         ng(true); }}> New account</button>
78     } />
79     <div className="grid grid-cols-2 gap-3 sm:gap-4 xl:grid-cols-4">
80       <StatCard label="Total accounts" value={users.length} icon="" />
81       <StatCard label="Admins / inspectors" value={
82         {count("admin")} / {count("inspector")}
83       } icon="[Urdu]" tone="violet" />
84       <StatCard label="Farmers" value={count("farmer")} sub={`
85         ${users.filter((u) => u.role === "farmer"
86         && !u.isVerified).length} unverified`} icon="" tone="amber" />
87       <StatCard label="Buyers" value={count("buyer")} icon="" tone="sky" />
88     </div>
89     <div className="card overflow-hidden">
90       <div className="flex flex-wrap items-center gap-2 border-b border-slate-100 p-4 dark:border-white/
91         5">
92         <input className="input max-w-xs" placeholder="Search name, email, city..." value={q} onChange={
93           (e) => setQ(e.target.value)} />
94         {[ "", "admin", "inspector", "farmer", "buyer" ] as const}.map((r) => (
95           <button key={r || "all"} onClick={() => setRole(r)} className={`rounded-full px-3.5 py-1.5 tex
96             t-xs font-semibold capitalize ${role === r ? "bg-brand-900 text-white" : "bg-slate-100 text-slate-600 dark
97             :bg-white/5 dark:text-slate-300"}>{r || "all"}</button>
98         ))
99       </div>
100       <div className="overflow-x-auto">
101         <table className="w-full text-sm">
102           <thead className="table-head"><tr><th className="px-4 py-2.5">User</th><th className="px-4 py-
103             2.5">Role</th><th className="px-4 py-2.5">City</th><th className="px-4 py-2.5">Status</th><th className="p
104             x-4 py-2.5">Activity</th><th className="px-4 py-2.5">Joined</th><th className="px-4 py-2.5 text-right">Act
105             ions</th></tr></thead>
106           <tbody className="divide-y divide-slate-100 dark:divide-white/5">
107             {shown.map((u) => (
108               <tr key={u.id}>
109                 <td className="px-4 py-3"><p className="font-semibold">{u.fullName}</p><p className="tex
110                   t-xs text-slate-500">{u.email}{u.businessName ? ` · ${u.businessName}` : ""}</p></td>
111                 <td className="px-4 py-3">
112                   <select
113                     className={`rounded-lg border-0 px-2 py-1 text-xs font-bold capitalize ${ROLE_BADGE[
114                       u.role]}`}
115                     value={u.role}
116                     disabled={busy === `u${u.id}`}
117                     onChange={(e) => {
118                       const r = e.target.value as U["role"];
119                       if (confirm(`Change ${u.fullName}'s role to ${r}?`)) patch(u, { role: r }, `Role c
120                         hanged to ${r}`);
121                     }}
122                   >
123                     <option value="admin">admin</option><option value="inspector">inspector</option><opt
124                       ion value="farmer">farmer</option><option value="buyer">buyer</option>
125                   </select>
126                 </td>
127                 <td className="px-4 py-3">{u.city}</td>
128                 <td className="px-4 py-3"><Badge status={u.isVerified ? "accepted" : "pending"} label={u
129                   .isVerified ? "verified" : "unverified"} /></td>
130                 <td className="px-4 py-3 text-xs text-slate-500">{u.listings} listings · {u.orders} orde
131                   rs{u.role === "buyer" ? ` · ${formatPKRShort(u.walletBalance)}` : ""}</td>
132                 <td className="px-4 py-3 text-xs text-slate-500">{formatDate(u.createdAt)}</td>
133                 <td className="px-4 py-3">

```

```

116         <div className="flex justify-end gap-1.5">
117             <button className="btn-secondary px-2.5 py-1 text-xs" disabled={busy === `u${u.id}`}
118             ↵ onClick={() => patch(u, { isVerified: !u.isVerified }, u.isVerified ? "Verification revoked" : "User veri
119             ↵ fied")}>{u.isVerified ? "Unverify" : "✓ Verify"}</button>
120             <button className="btn-secondary px-2.5 py-1 text-xs" onClick={() => { setResetFor(u
121             ↵ ); setNewPw(genPassword()); }}> Reset password</button>
122         </div>
123     </td>
124 </tr>
125 ))}
126 {shown.length === 0 && <tr><td colSpan={7} className="p-8 text-center text-slate-500">No use
127 ↵ rs match</td></tr>}
128 </tbody>
129 </table>
130 </div>
131 </div>
132
133 <Modal open={creating} onClose={() => setCreating(false)} title="Create account">
134     <form onSubmit={create} className="grid grid-cols-2 gap-3">
135         <div className="col-span-2 grid grid-cols-2 gap-2 sm:grid-cols-4">
136             {[ "admin", "inspector", "farmer", "buyer" ] as const}.map((r) => (
137                 <button type="button" key={r} onClick={() => setNu({ ...nu, role: r })} className={`rounded-
138                 ↵ xl border-2 p-2.5 text-xs font-bold capitalize ${nu.role === r ? "border-brand-600 bg-brand-50 dark:bg-bra
139                 ↵ nd-500/10" : "border-slate-200 dark:border-slate-700"}`>
140                     {r === "admin" ? "[Urdu]Admin" : r === "inspector" ? "Inspector" : r === "farmer" ? "Far
141                     ↵ mer" : " Buyer"}
142                 </button>
143             ))}
144         </div>
145         <div className="col-span-2"><label className="label">Full name</label><input className="input" r
146         ↵ equired value={nu.fullName} onChange={(e) => setNu({ ...nu, fullName: e.target.value })} /></div>
147         <div className="col-span-2 sm:col-span-1"><label className="label">Email (login)</label><input t
148         ↵ ype="email" className="input" required value={nu.email} onChange={(e) => setNu({ ...nu, email: e.target.va
149         ↵ lue })} /></div>
150         <div className="col-span-2 sm:col-span-1"><label className="label">Password</label><div classNam
151         ↵ e="flex gap-1"><input className="input font-mono" required minLength={8} value={nu.password} onChange={(e)
152         ↵ => setNu({ ...nu, password: e.target.value })} /><button type="button" className="btn-secondary px-2" tit
153         ↵ le="Generate" onClick={() => setNu({ ...nu, password: genPassword() )}></button></div></div>
154         <div className="col-span-2 sm:col-span-1"><label className="label">Phone</label><input className
155         ↵ ="input" required value={nu.phone} onChange={(e) => setNu({ ...nu, phone: e.target.value })} placeholder="
156         ↵ +92 3xx xxxxxxxx" /></div>
157         <div className="col-span-2 sm:col-span-1"><label className="label">City</label><select classNam
158         ↵ e="input" value={nu.city} onChange={(e) => setNu({ ...nu, city: e.target.value })}><CityOptions /></select>
159         </div>
160         <div className="col-span-2 sm:col-span-1"><label className="label">CNIC (optional)</label><input
161         ↵ className="input" value={nu.cnicId} onChange={(e) => setNu({ ...nu, cnicId: e.target.value })} placeholde
162         ↵ r="12345-1234567-1" /></div>
163         <div className="col-span-2 sm:col-span-1"><label className="label">Organisation (optional)</labe
164         ↵ l><input className="input" value={nu.businessName} onChange={(e) => setNu({ ...nu, businessName: e.target.
165         ↵ value })} /></div>
166         <p className="col-span-2 text-xs text-slate-500">Copy the password now and share it securely wit
167         ↵ h the user. They can change it under Account settings.</p>
168         <div className="col-span-2 flex justify-end gap-2">
169             <button type="button" className="btn-secondary" onClick={() => setCreating(false)}>Cancel</but
170             ↵ ton>
171             <button className="btn-primary" disabled={busy === "create"}>{busy === "create" && <Spinner />
172             ↵ } Create account</button>
173         </div>
174     </form>
175 </Modal>
176
177 <Modal open={!resetFor} onClose={() => setResetFor(null)} title={resetFor ? `Reset password: ${rese
178 ↵ tFor.fullName}` : ""}>
179     {resetFor && (
180         <div className="space-y-4">
181             <p className="text-sm text-slate-600 dark:text-slate-400">Set a new password for <b>{resetFor.
182             ↵ email}</b>. Share it with the user securely.</p>
183             <div className="flex gap-2">
184                 <input className="input font-mono" value={newPw} minLength={8} onChange={(e) => setNewPw(e.t
185                 ↵ arget.value)} />
186                 <button type="button" className="btn-secondary" onClick={() => setNewPw(genPassword())}></bu
187                 ↵ tton>
188                 <button type="button" className="btn-secondary" onClick={() => { navigator.clipboard.writeTe
189                 ↵ xt(newPw); push({ kind: "info", title: "Copied" }); }}></button>
190             </div>
191         </div>
192     )}
193 </Modal>

```

```

163         <button className="btn-secondary" onClick={() => setResetFor(null)}>Cancel</button>
164         <button className="btn-primary" disabled={busy === `u${resetFor.id}` || newPw.length < 8} on
    ↵ Click={async () => { if (await patch(resetFor, { newPassword: newPw }, "Password reset")) setResetFor(null
    ↵ ); }}>Set password</button>
165     </div>
166 </div>
167   )}
168 </Modal>
169 </div>
170 );
171 }
172

```

src/app/(app)/admin/verification/page.tsx

```

1  "use client";
2
3  import { useCallback, useEffect, useState } from "react";
4  import { api, Badge, GradeBadge, LoadingBlock, Modal, PageHeader, Spinner } from "@components/ui";
5  import { useToast } from "@components/providers";
6  import { formatDate, formatKg, formatPKR } from "@lib/constants";
7
8  type U = { id: number; fullName: string; email: string; phone: string; role: string; cnicId: string | null
    ↵ ; businessName: string | null; city: string; address: string | null; isVerified: boolean; listings: number
    ↵ ; orders: number; createdAt: string };
9  type Crop = { cropId: number; cropName: string; farmerName: string; farmLocation: string; qualityGrade: st
    ↵ ring; totalQuantityKg: number; basePricePerKg: number; status: string; harvestDate: string; inspectionGrad
    ↵ e: string | null };
10 type Insp = { inspectionId: number; cropName: string; farmerName: string; farmLocation: string; inspectorN
    ↵ ame: string; gradeAssigned: string; moistureLevelPercentage: number; soilPh: number | null; inspectionNote
    ↵ s: string | null; reportAttachment: string | null; status: string; verifiedAt: string | null };
11
12 export default function VerificationPortal() {
13   const { push } = useToast();
14   const [tab, setTab] = useState<"farmers" | "inspections">("farmers");
15   const [users, setUsers] = useState<U[] | null>(null);
16   const [crops, setCrops] = useState<Crop[]>([]);
17   const [insps, setInsps] = useState<Insp[]>([]);
18   const [target, setTarget] = useState<Crop | null>(null);
19   const [form, setForm] = useState({ gradeAssigned: "A", moistureLevelPercentage: "12", soilPh: "7.2", ins
    ↵ pectionNotes: "", status: "passed", reportAttachment: "", fileName: "" });
20   const [busy, setBusy] = useState<number | string | null>(null);
21
22   const load = useCallback(async () => {
23     try {
24       const [u, c, i] = await Promise.all([api<U[]>("/api/admin/users?role=farmer"), api<Crop[]>("/api/cro
    ↵ ps"), api<Insp[]>("/api/inspections")]);
25       setUsers(u); setCrops(c); setInsps(i);
26     } catch (e) {
27       push({ kind: "error", title: (e as Error).message });
28     }
29   }, [push]);
30   useEffect(() => { load(); }, [load]);
31
32   async function verify(u: U, v: boolean) {
33     setBusy(u.id);
34     try {
35       const r = await api<{ listingsActivated: number }>(`/api/admin/users/${u.id}`, { method: "PATCH", js
    ↵ on: { isVerified: v } });
36       push({ kind: "success", title: v ? `${u.fullName} verified` : `${u.fullName} unverified`, body: r.li
    ↵ stingsActivated ? `${r.listingsActivated} held listing(s) activated` : undefined });
37       load();
38     } catch (e) {
39       push({ kind: "error", title: (e as Error).message });
40     } finally { setBusy(null); }
41   }
42
43   function onFile(file: File | undefined) {
44     if (!file) return;
45     if (file.size > 1_400_000) return push({ kind: "error", title: "File too large (max 1.4MB)" });
46     const r = new FileReader();
47     r.onload = () => setForm((f) => ({ ...f, reportAttachment: r.result as string, fileName: file.name }));
    ↵ ;
48     r.readAsDataURL(file);
49   }

```

```

50
51   async function submitInspection(e: React.FormEvent) {
52     e.preventDefault();
53     if (!target) return;
54     setBusy("insp");
55     try {
56       await api("/api/inspections", {
57         method: "POST",
58         json: { cropId: target.cropId, gradeAssigned: form.gradeAssigned, moistureLevelPercentage: Number(
59           form.moistureLevelPercentage), soilPh: form.soilPh ? Number(form.soilPh) : undefined, inspectionNotes: for
60           m.inspectionNotes, status: form.status, reportAttachment: form.reportAttachment || undefined },
61       });
62       push({ kind: "success", title: "Inspection report saved", body: form.status === "passed" ? "Listing
63         activated with assigned grade." : form.status === "failed" ? "Listing archived; open bids rejected." : "Li
64         sting held for inspection." });
65       setTarget(null);
66       load();
67     } catch (err) {
68       push({ kind: "error", title: (err as Error).message });
69     } finally { setBusy(null); }
70   }
71
72   if (!users) return <LoadingBlock />;
73   const pendingCrops = crops.filter((c) => c.status === "pending_inspection");
74   const otherCrops = crops.filter((c) => c.status !== "pending_inspection" && c.status !== "archived");
75
76   return (
77     <div>
78       <PageHeader title="Verification Portal" subtitle="Farmer verification and crop inspections" />
79       <div className="mb-5 flex gap-2">
80         <button onClick={() => setTab("farmers")} className={`rounded-full px-4 py-2 text-sm font-semibold
81           ${tab === "farmers" ? "bg-brand-600 text-white" : "bg-white ring-1 ring-slate-200 dark:bg-slate-900 dark:
82           ring-slate-700"}> Farmer accounts ({users.filter((u) => !u.isVerified).length} pending)</button>
83         <button onClick={() => setTab("inspections")} className={`rounded-full px-4 py-2 text-sm font-semi
84           bold ${tab === "inspections" ? "bg-brand-600 text-white" : "bg-white ring-1 ring-slate-200 dark:bg-slate-9
85           00 dark:ring-slate-700"}> Crop inspections ({pendingCrops.length} pending)</button>
86       </div>
87
88       {tab === "farmers" ? (
89         <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-3">
90           {users.map((u) => (
91             <div key={u.id} className={`card p-5 ${!u.isVerified ? "ring-2 ring-amber-400/60" : ""}>
92               <div className="flex items-start justify-between gap-2">
93                 <div>
94                   <p className="font-semibold">{u.fullName}</p>
95                   <p className="text-xs text-slate-500">{u.businessName ?? "-"}</p>
96                 </div>
97                 <Badge status={u.isVerified ? "accepted" : "pending"} label={u.isVerified ? "verified" : "
98                 unverified"} />
99               </div>
100               <dl className="mt-3 space-y-1 text-sm">
101                 <div className="flex justify-between"><dt className="text-slate-500">CNIC</dt><dd classNam
102                 e="font-mono">{u.cnid ?? "Not provided"}</dd></div>
103                 <div className="flex justify-between"><dt className="text-slate-500">Phone</dt><dd>{u.phon
104                 e}</dd></div>
105                 <div className="flex justify-between"><dt className="text-slate-500">Location</dt><dd>{u.c
106                 ity}</dd></div>
107                 <div className="flex justify-between"><dt className="text-slate-500">Listings / Orders</dt>
108                 <dd>{u.listings} / {u.orders}</dd></div>
109                 <div className="flex justify-between"><dt className="text-slate-500">Joined</dt><dd>{forma
110                 tDate(u.createdAt)}</dd></div>
111               </dl>
112               {u.address && <p className="mt-2 text-xs text-slate-500"> {u.address}</p>}
113               <div className="mt-4">
114                 {u.isVerified ? (
115                   <button className="btn-secondary w-full text-rose-600" disabled={busy === u.id} onClick=
116                   {() => verify(u, false)}>Revoke verification</button>
117                 ) : (
118                   <button className="btn-primary w-full" disabled={busy === u.id} onClick={() => verify(u,
119                   true)}>{busy === u.id && <Spinner />} ✓ Verify farmer</button>
120                 )}
121               </div>
122             </div>
123           ))}
124         </div>
125       ) : (
126

```

```

110     <div className="space-y-6">
111       <div className="card overflow-hidden">
112         <h2 className="border-b border-slate-200 px-5 py-3.5 font-semibold dark:border-slate-800">List
113         ings awaiting inspection</h2>
114         <CropTable rows={pendingCrops} onInspect={(c) => { setTarget(c); setForm({ ...form, gradeAssign
115         ned: c.qualityGrade, inspectionNotes: "", reportAttachment: "", fileName: "" }); }} empty="No listings pen
116         ding inspection " />
117       </div>
118       <details className="card overflow-hidden">
119         <summary className="cursor-pointer px-5 py-3.5 font-semibold">Re-inspect an active listing ({o
120         therCrops.length})</summary>
121         <CropTable rows={otherCrops} onInspect={(c) => { setTarget(c); setForm({ ...form, gradeAssigne
122         d: c.qualityGrade, inspectionNotes: "", reportAttachment: "", fileName: "" }); }} empty="None" />
123       </details>
124       <div className="card overflow-hidden">
125         <h2 className="border-b border-slate-200 px-5 py-3.5 font-semibold dark:border-slate-800">Insp
126         ection reports log</h2>
127         <div className="overflow-x-auto">
128           <table className="w-full text-sm">
129             <thead className="table-head"><tr><th className="px-4 py-2.5">#</th><th className="px-4 py
130             -2.5">Crop / Farmer</th><th className="px-4 py-2.5">Grade</th><th className="px-4 py-2.5">Moisture</th><th
131             className="px-4 py-2.5">Soil pH</th><th className="px-4 py-2.5">Notes</th><th className="px-4 py-2.5">Sta
132             tus</th><th className="px-4 py-2.5">Report</th></tr></thead>
133             <tbody className="divide-y divide-slate-100 dark:divide-slate-800">
134               {insps.map((i) => (
135                 <tr key={i.inspectionId}>
136                   <td className="px-4 py-3 text-slate-500">{i.inspectionId}</td>
137                   <td className="px-4 py-3"><p className="font-medium">{i.cropName}</p><p class="t
138                   ext-xs text-slate-500">{i.farmerName} · {i.farmLocation}</p></td>
139                   <td className="px-4 py-3"><GradeBadge grade={i.gradeAssigned} /></td>
140                   <td className="px-4 py-3">{i.moistureLevelPercentage}%</td>
141                   <td className="px-4 py-3">{i.soilPh ?? "-"}</td>
142                   <td className="max-w-xs px-4 py-3 text-xs text-slate-600 dark:text-slate-400">{i.ins
143                   pectionNotes}</td>
144                   <td className="px-4 py-3"><Badge status={i.status} /><p className="mt-1 text-[11px]
145                   text-slate-500">{i.verifiedAt ? formatDate(i.verifiedAt) : ""}</p></td>
146                   <td className="px-4 py-3">{i.reportAttachment ? <a className="font-semibold text-bra
147                   nd-600 hover:underline" href={` /api/inspections/${i.inspectionId}/report`} target="_blank" rel="noreferrer
148                   "> View</a> : "-"</td>
149                 </tr>
150               ))}
151             </tbody>
152           </table>
153         </div>
154       </div>
155     </div>
156   )}
157
158   <Modal open={!!target} onClose={() => setTarget(null)} title={target ? `Inspection report · ${target
159   .cropName} (${target.farmerName})` : ""}>
160     {target && (
161       <form onSubmit={submitInspection} className="grid grid-cols-2 gap-3">
162         <div><label className="label">Grade assigned</label><select className="input" value={form.grad
163         eAssigned} onChange={(e) => setForm({ ...form, gradeAssigned: e.target.value })}><option>A</option><option
164         >B</option><option>C</option></select></div>
165         <div><label className="label">Result</label><select className="input" value={form.status} onCh
166         ange={(e) => setForm({ ...form, status: e.target.value })}><option value="passed"> Passed</option><option
167         value="pending"> Pending lab</option><option value="failed"> Failed</option></select></div>
168         <div><label className="label">Moisture %</label><input type="number" step="0.1" min="0" max="1
169         00" className="input" value={form.moistureLevelPercentage} onChange={(e) => setForm({ ...form, moistureLev
170         elPercentage: e.target.value })} required /></div>
171         <div><label className="label">Soil pH</label><input type="number" step="0.1" min="0" max="14"
172         className="input" value={form.soilPh} onChange={(e) => setForm({ ...form, soilPh: e.target.value })} /></d
173         iv>
174         <div className="col-span-2"><label className="label">Inspection notes</label><textarea rows={3
175         } className="input" value={form.inspectionNotes} onChange={(e) => setForm({ ...form, inspectionNotes: e.ta
176         rget.value })} placeholder="Foreign matter %, pest damage, grain size, colour..." /></div>
177         <div className="col-span-2">
178           <label className="label">Soil / crop lab report (PDF or image)</label>
179           <input type="file" accept="application/pdf,image/*" className="input" onChange={(e) => onFil
180           e(e.target.files?.[0])} />
181           {form.fileName && <p className="mt-1 text-xs text-brand-600"> {form.fileName}</p>}
182         </div>
183         <div className="col-span-2 flex justify-end gap-2">
184           <button type="button" className="btn-secondary" onClick={() => setTarget(null)}>Cancel</butt
185           on>

```

```

159         <button className="btn-primary" disabled={busy === "insp"}>{busy === "insp" && <Spinner />}
160     </button>
161 </div>
162 </form>
163 </div>
164 </div>
165 </div>
166 </div>
167 </div>
168 function CropTable({ rows, onInspect, empty }: { rows: Crop[]; onInspect: (c: Crop) => void; empty: string
169 }) {
170     if (!rows.length) return <p className="p-8 text-center text-sm text-slate-500">{empty}</p>;
171     return (
172         <div className="overflow-x-auto">
173             <table className="w-full text-sm">
174                 <thead className="table-head"><tr><th className="px-4 py-2.5">Crop</th><th className="px-4 py-2.5">
175                 >Farmer</th><th className="px-4 py-2.5">Declared grade</th><th className="px-4 py-2.5">Qty</th><th classNa
176                 me="px-4 py-2.5">Price</th><th className="px-4 py-2.5">Status</th><th className="px-4 py-2.5"></th></tr></thead>
177                 <tbody className="divide-y divide-slate-100 dark:divide-slate-800">
178                     {rows.map((c) => (
179                         <tr key={c.cropId}>
180                             <td className="px-4 py-3 font-medium">{c.cropName}<p className="text-xs text-slate-500">Harv
181                             est {formatDate(c.harvestDate)}</p></td>
182                             <td className="px-4 py-3">{c.farmerName}<p className="text-xs text-slate-500">{c.farmLocatio
183                             n}</p></td>
184                             <td className="px-4 py-3"><GradeBadge grade={c.qualityGrade} /></td>
185                             <td className="px-4 py-3">{formatKg(c.totalQuantityKg)}</td>
186                             <td className="px-4 py-3">{formatPKR(c.basePricePerKg, 2)}</td>
187                             <td className="px-4 py-3"><Badge status={c.status} /></td>
188                             <td className="px-4 py-3 text-right"><button className="btn-primary py-1.5 text-xs" onClick=
189                             {() => onInspect(c)}> Inspect</button></td>
190                         </tr>
191                     ))}
192                 </tbody>
193             </table>
194         </div>
195     );
196 }

```

src/app/(app)/buyer/bids/page.tsx

```

1 import { NegotiationBoard } from "@components/negotiation";
2
3 export default function BuyerBidsPage() {
4     return <NegotiationBoard role="buyer" />;
5 }
6

```

src/app/(app)/buyer/dashboard/page.tsx

```

1 import Link from "next/link";
2 import { requirePageUser } from "@lib/auth";
3 import { buyerStats } from "@lib/services/analytics";
4 import { listOrders } from "@lib/services/orders";
5 import { listBids } from "@lib/services/bids";
6 import { pendingReviews } from "@lib/services/reviews";
7 import { Badge, PageHeader, StatCard } from "@components/ui";
8 import { VolumeAreaChart } from "@components/charts";
9 import { LogisticsStepper } from "@components/stepper";
10 import { ReviewPrompts } from "@components/reviews/review-prompts";
11 import { WorkflowTrail } from "@components/dashboard/workflow-trail";
12 import { formatKg, formatPKR, formatPKRShort, type DeliveryStage } from "@lib/constants";
13
14 export const dynamic = "force-dynamic";
15 export const metadata = { title: "Buyer dashboard" };
16
17 export default async function BuyerDashboard() {
18     const user = await requirePageUser(["buyer"]);
19     const [s, orders, bids, toReview] = await Promise.all([buyerStats(user.id), listOrders(user, { active: t
20     rue, limit: 6 }), listBids(user), pendingReviews(user.id)]);
21     const countered = bids.filter((b) => b.status === "countered").slice(0, 4);

```

```

21     const awaiting = orders.filter((o) => o.paymentStatus === "awaiting_escrow");
22
23     return (
24       <div className="space-y-6">
25         <PageHeader title="Buyer Dashboard" urdu="[Urdu]" subtitle={user.businessName ?? user.fullName}
26           actions={<><Link href="/marketplace" className="btn-primary"> Browse marketplace</Link><Link href=
27             "/wallet" className="btn-secondary"> Add funds</Link></> } />
28         <div className="grid grid-cols-2 gap-3 sm:gap-4 xl:grid-cols-4">
29           <StatCard label="Wallet balance" value={formatPKRShort(s.walletBalance)} sub={` ${formatPKRShort(s.
30             escrowBalance)} in escrow`} icon="" tone="sky" />
31           <StatCard label="Active orders" value={s.activeOrders} sub={s.awaitingEscrow ? ` ${s.awaitingEscrow
32             } need escrow funding` : "All funded"} icon="" />
33           <StatCard label="Open bids" value={s.pendingBids + s.counteredBids} sub={s.counteredBids ? ` ${s.co
34             unteredBids} counter-offer(s)` : "No counter-offers"} icon="" tone="amber" />
35           <StatCard label="Total procured" value={formatKg(s.totalKg)} sub={` ${formatPKRShort(s.totalSpend)}
36             lifetime`} icon="" tone="violet" />
37         </div>
38         <WorkflowTrail title="Buyer / Customer workflow" steps={[
39           { label: "Browse produce", detail: "Search graded crops", href: "/marketplace" },
40           { label: "Submit bids", detail: "Make or revise offers", href: "/buyer/bids", pending: s.countered
41             Bids },
42           { label: "Fund escrow", detail: "Secure accepted deals", href: "/buyer/orders", pending: s.awaitin
43             gEscrow },
44           { label: "Track delivery", detail: "Follow your shipments", href: "/buyer/orders", pending: s.acti
45             veOrders },
46           { label: "Rate the farmer", detail: "Review delivered orders", href: "/buyer/dashboard", pending:
47             toReview.length },
48         ]} />
49         {(awaiting.length > 0 || countered.length > 0 || toReview.length > 0) && (
50           <section className="card border-amber-200 p-5 dark:border-amber-900">
51             <h2 className="font-bold">Needs your attention</h2>
52             <ul className="mt-3 space-y-2 text-sm">
53               {awaiting.map((o) => (
54                 <li key={o.orderId} className="flex flex-wrap items-center justify-between gap-2 rounded-xl
55                 bg-amber-50 px-3 py-2 dark:bg-amber-500/10">
56                   <span> Fund escrow for <b>{o.cropName}</b> ({o.trackingNumber}): {formatPKR(o.totalAmount)
57                     }</span>
58                   <Link href="/buyer/orders" className="text-xs font-semibold text-brand-700 hover:underline
59                     dark:text-brand-400">Pay now →</Link>
60                 </li>
61               ))}
62               {countered.map((b) => (
63                 <li key={b.bidId} className="flex flex-wrap items-center justify-between gap-2 rounded-xl bg
64                 -sky-50 px-3 py-2 dark:bg-sky-500/10">
65                   <span> Counter-offer on <b>{b.cropName}</b>: {formatPKR(b.counterPrice ?? 0, 2)}/kg (you b
66                     id {formatPKR(b.bidPricePerKg, 2)})</span>
67                   <Link href="/buyer/bids" className="text-xs font-semibold text-brand-700 hover:underline d
68                     ark:text-brand-400">Respond →</Link>
69                 </li>
70               ))}
71             </ul>
72             {toReview.length > 0 && <div className="mt-4"><ReviewPrompts items={toReview} /></div>}
73           </section>
74         )}
75         <div className="grid gap-4 lg:grid-cols-5">
76           <section className="card overflow-hidden lg:col-span-3">
77             <div className="flex items-center justify-between border-b border-slate-100 px-5 py-3.5 dark:bor
78             der-white/5">
79               <h2 className="font-bold">Active shipments</h2>
80               <Link href="/buyer/orders" className="text-sm font-semibold text-brand-700 hover:underline dar
81                 k:text-brand-400">All orders →</Link>
82             </div>
83             {orders.length === 0 ? (
84               <p className="p-8 text-center text-sm text-slate-500">No active shipments. <Link href="/market
85                 place" className="font-semibold text-brand-700">Find produce →</Link></p>
86             ) : (
87               <ul className="divide-y divide-slate-100 dark:divide-white/5">
88                 {orders.map((o) => (
89                   <li key={o.orderId} className="px-5 py-4">
90                     <div className="mb-3 flex flex-wrap items-center justify-between gap-2 text-sm">
91                       <span className="font-semibold">{o.cropName} · {formatKg(o.quantityKg)} <span classNam
92                         e="font-mono text-xs font-normal text-slate-500">{o.trackingNumber}</span></span>

```

```

78         <span className="flex items-center gap-2"><Badge status={o.paymentStatus} /><Link href
↳      ={'/tracking?no=${o.trackingNumber}`' } className="text-xs font-semibold text-brand-700 hover:underline dark
↳      :text-brand-400"> Track</Link></span>
79     </div>
80     <LogisticsStepper stage={o.deliveryStage as DeliveryStage} compact disputed={o.paymentSt
↳      atus === "disputed"} />
81     </li>
82   )}}
83 </ul>
84   )}
85 </section>
86 <section className="card p-5 lg:col-span-2">
87   <h2 className="mb-3 font-bold">Monthly procurement</h2>
88   <VolumeAreaChart data={s.monthly} height={240} />
89 </section>
90 </div>
91 </div>
92   );
93 }
94

```

src/app/(app)/buyer/layout.tsx

```

1  import type { ReactNode } from "react";
2  import { requirePageUser } from "@lib/auth";
3
4  export default async function BuyerLayout({ children }: { children: ReactNode }) {
5    await requirePageUser(["buyer"]);
6    return <>{children}</>;
7  }
8

```

src/app/(app)/buyer/orders/page.tsx

```

1  import { OrdersBoard } from "@components/orders-board";
2
3  export default function BuyerOrdersPage() {
4    return <OrdersBoard role="buyer" title="Order Pipeline & Logistics" subtitle="Fund escrow, track shipmen
↳      ts, confirm delivery" />;
5  }
6

```

src/app/(app)/buyer/page.tsx

```

1  import { redirect } from "next/navigation";
2
3  export default function Page() {
4    redirect("/buyer/dashboard");
5  }
6

```

src/app/(app)/farmer/bids/page.tsx

```

1  import { NegotiationBoard } from "@components/negotiation";
2
3  export default function FarmerBidsPage() {
4    return <NegotiationBoard role="farmer" />;
5  }
6

```

src/app/(app)/farmer/dashboard/page.tsx

```

1  import Link from "next/link";
2  import { requirePageUser } from "@lib/auth";
3  import { farmerStats } from "@lib/services/analytics";
4  import { listBids } from "@lib/services/bids";
5  import { listOrders } from "@lib/services/orders";
6  import { listCrops } from "@lib/services/crops";
7  import { platformComparison } from "@lib/services/mandi";
8  import { listReviews } from "@lib/services/reviews";
9  import { Badge, PageHeader, StatCard } from "@components/ui";

```

```

10 import { DonutChart, VolumeAreaChart } from "@components/charts";
11 import { LogisticsStepper } from "@components/stepper";
12 import { WorkflowTrail } from "@components/dashboard/workflow-trail";
13 import { formatKg, formatPKR, formatPKRShort, type DeliveryStage } from "@lib/constants";
14
15 export const dynamic = "force-dynamic";
16 export const metadata = { title: "Farmer dashboard" };
17
18 export default async function FarmerDashboard() {
19   const user = await requirePageUser(["farmer"]);
20   const [stats, bids, orders, crops, compare] = await Promise.all([
21     farmerStats(user.id),
22     listBids(user),
23     listOrders(user, { active: true }),
24     listCrops({ farmerId: user.id, status: "all" }),
25     platformComparison(),
26   ]);
27   const reviews = await listReviews(user.id, 100);
28   const unansweredReviews = reviews.filter((review) => !review.farmerReply).length;
29   const openBids = bids.filter((b) => b.status === "pending" || b.status === "countered").slice(0, 5);
30   const myCropNames = [...new Set(crops.map((c) => c.cropName))];
31   const insights = compare.filter((c) => myCropNames.includes(c.cropName));
32
33   return (
34     <div className="space-y-6">
35       <PageHeader
36         title="Kisan Dashboard" urdu="[Urdu]"
37         subtitle={` ${user.businessName ?? user.fullName} · inventory, bids & earnings at a glance` }
38         actions={
39           <>
40             <Link href="/farmer/listings?new=1" className="btn-primary"> New Listing</Link>
41             <Link href="/mandi-rates" className="btn-secondary"> Mandi Rates</Link>
42             <a href="/api/reports/summary" className="btn-secondary"> Sales report</a>
43           </>
44         }
45       </div>
46
47       {!user.isVerified && (
48         <div className="card flex items-start gap-3 border-amber-300 bg-amber-50 p-4 dark:border-amber-700
49           dark:bg-amber-950/30">
50           <span className="text-2xl"></span>
51           <div className="text-sm">
52             <p className="font-semibold text-amber-900 dark:text-amber-200">Account verification pending</p>
53             <p className="text-amber-800 dark:text-amber-300/80">New listings are held as "pending inspection" until a MarketLink quality inspector verifies your CNIC and farm. This usually takes 24 hours.</p>
54           </div>
55         </div>
56       )}
57
58       <div className="grid grid-cols-2 gap-3 sm:gap-4 lg:grid-cols-3 2xl:grid-cols-5">
59         <StatCard label="Active listings" value={stats.activeListings} sub={` ${formatKg(stats.inventoryKg)} `} · {formatPKR(stats.inventoryValue)} value` icon="" />
60         <StatCard label="Incoming bids" value={stats.pendingBids} sub={` ${stats.counteredBids} awaiting buyer reply` icon="" tone="amber" />
61         <StatCard label="Total earnings" value={formatPKRShort(stats.totalEarned)} sub={` ${formatPKR(stats.earnedThisMonth)} this month` icon="" tone="violet" />
62         <StatCard label="Trust score" value={user.trustScore ? `${Number(user.trustScore).toFixed(2)} / 5` : "New"} sub={user.ratingCount ? `★ ${Number(user.ratingAvg).toFixed(1)} from ${user.ratingCount} reviews` : "No reviews yet"} icon="" tone="amber" />
63         <StatCard label="Held in escrow" value={formatPKRShort(stats.inEscrow)} sub={` ${stats.activeOrders} active orders` icon="" tone="sky" />
64       </div>
65
66       <WorkflowTrail title="Farmer / Vendor workflow" steps={[
67         { label: "List produce", detail: "Manage crops and grades", href: "/farmer/listings", pending: stats.pendingInspection },
68         { label: "Negotiate bids", detail: "Review buyer offers", href: "/farmer/bids", pending: stats.pendingBids },
69         { label: "Fulfil orders", detail: "Dispatch and track", href: "/farmer/orders", pending: stats.activeOrders },
70         { label: "Receive payment", detail: "Escrow and payouts", href: "/wallet" },
71         { label: "Reply to reviews", detail: "Verified buyer feedback", href: "/farmer/reviews", pending: unansweredReviews },
72         { label: "Manage profile", detail: "Farm and account details", href: "/account" },

```

```

72     }} />
73
74     <div className="grid gap-4 lg:grid-cols-3">
75         <div className="card p-5 lg:col-span-2">
76             <div className="mb-3 flex items-center justify-between">
77                 <h2 className="font-semibold">Monthly sales trend</h2>
78                 <span className="text-xs text-slate-500">Last 6 months · order value</span>
79             </div>
80             <VolumeAreaChart data={stats.monthly} />
81         </div>
82         <div className="card p-5">
83             <h2 className="mb-3 font-semibold">Sales by crop</h2>
84             {stats.byCrop.length ? <DonutChart data={stats.byCrop} /> : <p className="py-16 text-center text
85 -sm text-slate-500">No sales yet</p>}
86         </div>
87
88     <div className="grid gap-4 lg:grid-cols-2">
89         <div className="card overflow-hidden">
90             <div className="flex items-center justify-between border-b border-slate-200 px-5 py-3.5 dark:bor
91 der-slate-800">
92                 <h2 className="font-semibold">Incoming buyer bids</h2>
93                 <Link href="/farmer/bids" className="text-sm font-semibold text-brand-600 hover:underline">Neg
94 otiation hub </Link>
95             </div>
96             {openBids.length === 0 ? (
97                 <p className="p-8 text-center text-sm text-slate-500">No open bids right now.</p>
98             ) : (
99                 <ul className="divide-y divide-slate-100 dark:divide-slate-800">
100                     {openBids.map((b) => {
101                         const diff = ((b.bidPricePerKg - b.basePricePerKg) / b.basePricePerKg) * 100;
102                         return (
103                             <li key={b.bidId} className="flex items-center gap-3 px-5 py-3">
104                                 <div className="grid h-10 w-10 shrink-0 place-items-center rounded-xl bg-brand-50 text
105 -lg dark:bg-brand-500/10"></div>
106                                 <div className="min-w-0 flex-1">
107                                     <p className="truncate text-sm font-semibold">{b.buyerBusiness ?? b.buyerName}</p>
108                                     <p className="text-xs text-slate-500">{b.cropName} · {formatKg(b.bidQuantityKg)} · {
109 b.buyerCity}</p>
110                                 </div>
111                                 <div className="text-right">
112                                     <p className="text-sm font-bold">{formatPKR(b.bidPricePerKg, 2)}/kg</p>
113                                     <p className="text-xs font-semibold ${diff} >= 0 ? "text-emerald-600" : "text-rose-6
114 00">`>{diff} >= 0 ? "+" : "">{diff.toFixed(1)}% vs ask</p>
115                                 </div>
116                                 <Badge status={b.status} />
117                             </li>
118                         );
119                     })}
120                 </ul>
121             )}
122         </div>
123
124         <div className="card overflow-hidden">
125             <div className="flex items-center justify-between border-b border-slate-200 px-5 py-3.5 dark:bor
126 der-slate-800">
127                 <h2 className="font-semibold">Active shipments</h2>
128                 <Link href="/farmer/orders" className="text-sm font-semibold text-brand-600 hover:underline">A
129 ll orders </Link>
130             </div>
131             {orders.length === 0 ? (
132                 <p className="p-8 text-center text-sm text-slate-500">No active shipments.</p>
133             ) : (
134                 <ul className="divide-y divide-slate-100 dark:divide-slate-800">
135                     {orders.slice(0, 4).map((o) => (
136                         <li key={o.orderId} className="px-5 py-4">
137                             <div className="mb-3 flex items-center justify-between gap-2 text-sm">
138                                 <span className="font-semibold">{o.cropName} → {o.buyerBusiness ?? o.buyerName}</span>
139                                 <Badge status={o.paymentStatus} />
140                             </div>
141                             <LogisticsStepper stage={o.deliveryStage as DeliveryStage} compact disputed={o.paymentSt
142 atus === "disputed"} />
143                         </li>
144                     ))}
145                 </ul>
146             )}
147         </div>
148     </div>
149 }

```

```

139     </div>
140 </div>
141
142     {reviews.length > 0 && (
143       <section className="card p-5">
144         <div className="flex items-center justify-between gap-3">
145           <h2 className="font-semibold">Latest buyer reviews</h2>
146           <Link href="/farmer/reviews" className="text-sm font-semibold text-brand-700 hover:underline d
147             ark:text-brand-400">
148             {unansweredReviews > 0 ? `Respond to ${unansweredReviews} →` : "All reviews →"}
149           </Link>
150         </div>
151         <ul className="mt-3 grid gap-3 md:grid-cols-3">
152           {reviews.slice(0, 3).map((r) => (
153             <li key={r.reviewId} className="rounded-2xl border border-slate-200 p-4 text-sm dark:border-
154             white/10">
155               <p className="text-gold-500">{"★".repeat(r.rating)}<span className="text-slate-300 dark:te
156               xt-slate-600">{"★".repeat(5 - r.rating)}</span></p>
157               {r.comment && <p className="mt-1">&ldquo;{r.comment}&rdquo;</p>}
158               <p className="mt-1 text-xs text-slate-500">{r.buyerName ?? r.buyerFullName} · {r.cropName}
159             </p>
160               <p className={`mt-2 text-xs font-semibold ${r.farmerReply ? "text-brand-700 dark:text-bran
161               d-400" : "text-amber-700 dark:text-amber-400"}`}>
162                 {r.farmerReply ? "✓ Responded" : "Reply pending"}
163               </p>
164             </li>
165           )})
166         </ul>
167       </section>
168     )}
169
170     <div className="card overflow-hidden">
171       <div className="flex items-center justify-between border-b border-slate-200 px-5 py-3.5 dark:borde
172       r-slate-800">
173         <h2 className="font-semibold">Mandi price insight for your crops</h2>
174         <Link href="/mandi-rates" className="text-sm font-semibold text-brand-600 hover:underline">Full
175         insights →</Link>
176       </div>
177       <div className="overflow-x-auto">
178         <table className="w-full text-sm">
179           <thead className="table-head">
180             <tr>
181               <th className="px-5 py-2.5">Crop</th>
182               <th className="px-5 py-2.5">Mandi avg</th>
183               <th class
184               Name="px-5 py-2.5">Your ask</th>
185               <th className="px-5 py-2.5">Platform deals</th>
186               <th className="px-5 py-2.5">Signal</th>
187             </tr>
188           </thead>
189           <tbody className="divide-y divide-slate-100 dark:divide-slate-800">
190             {insights.map((i) => {
191               const mine = crops.filter((c) => c.cropName === i.cropName && c.status === "active");
192               const ask = mine.length ? mine.reduce((s, c) => s + c.basePricePerKg, 0) / mine.length : n
193               ull;
194               const gap = ask ? ((ask - i.mandiAvg) / i.mandiAvg) * 100 : null;
195               return (
196                 <tr key={i.cropName}>
197                   <td className="px-5 py-3 font-medium">{i.cropName}</td>
198                   <td className="px-5 py-3">{formatPKR(i.mandiAvg, 2)}</td>
199                   <td className="px-5 py-3">{ask ? formatPKR(ask, 2) : "-"}</td>
200                   <td className="px-5 py-3">{i.platformDeal ? formatPKR(i.platformDeal, 2) : "-"}</td>
201                   <td className="px-5 py-3">
202                     {gap === null ? (
203                       <span className="text-slate-400">No active listing</span>
204                     ) : gap > 8 ? (
205                       <span className="font-semibold text-amber-600">▲ {gap.toFixed(1)}% above mandi – m
206                       ay slow bids</span>
207                     ) : gap < -5 ? (
208                       <span className="font-semibold text-rose-600">▼ {Math.abs(gap).toFixed(1)}% below
209                       mandi – consider raising</span>
210                     ) : (
211                       <span className="font-semibold text-emerald-600">● Competitive ({gap >= 0 ? "+" :
212                       ""}{gap.toFixed(1)}%)</span>
213                     )
214                   }
215                 </td>
216               </tr>
217             )}
218           </tbody>
219         </table>

```

```

202     </div>
203     </div>
204   </div>
205   );
206 }
207

```

src/app/(app)/farmer/layout.tsx

```

1  import type { ReactNode } from "react";
2  import { requirePageUser } from "@lib/auth";
3
4  export default async function FarmerLayout({ children }: { children: ReactNode }) {
5    await requirePageUser(["farmer"]);
6    return <>{children}</>;
7  }
8

```

src/app/(app)/farmer/listings/listings-client.tsx

```

1  "use client";
2
3  import { useCallback, useEffect, useMemo, useState } from "react";
4  import { api, Badge, compressImage, EmptyState, GradeBadge, LoadingBlock, Modal, PageHeader, Spinner } from "@components/ui";
5  import { CityOptions } from "@components/city-options";
6  import { useToast } from "@components/providers";
7  import { CATEGORIES, CROP_CATALOGUE, perMaund, cropImage, formatDate, formatKg, formatPKR, todayISO, type Category, type Grade } from "@lib/constants";
8
9  type Crop = {
10    cropId: number; cropName: string; category: Category; qualityGrade: Grade; totalQuantityKg: number; basePricePerKg: number;
11    harvestDate: string; imagesJson: string[]; status: string; farmLocation: string; description: string | null; openBids: number; highestBid: number | null; inspectionGrade: string | null;
12  };
13  type Rate = { marketLocation: string; minPricePerKg: number; maxPricePerKg: number; avgPricePerKg: number };
14
15  const GRADE_FACTOR: Record<Grade, number> = { A: 1.06, B: 1.0, C: 0.9 };
16  const empty = { cropName: "Wheat", category: "grains" as Category, qualityGrade: "A" as Grade, quantity: "", unit: "kg" as "kg" | "maund" | "tons", basePricePerKg: "", harvestDate: todayISO(), farmLocation: "Multan", description: "", imagesJson: [] as string[] };
17
18  export function ListingsClient({ openNew }: { openNew: boolean }) {
19    const { push } = useToast();
20    const [crops, setCrops] = useState<Crop[] | null>(null);
21    const [open, setOpen] = useState(openNew);
22    const [editing, setEditing] = useState<Crop | null>(null);
23    const [form, setForm] = useState(empty);
24    const [rates, setRates] = useState<Rate[]>([]);
25    const [saving, setSaving] = useState(false);
26    const [filter, setFilter] = useState("all");
27
28    const load = useCallback(() => api<Crop[]>("/api/crops?mine=1").then(setCrops).catch((e) => push({ kind: "error", title: e.message })), [push]);
29    useEffect(() => { load(); }, [load]);
30
31    useEffect(() => {
32      if (!open) return;
33      api<Rate[]>(`/api/mandi-rates?view=latest&crop=${encodeURIComponent(form.cropName)}`).then(setRates).catch(() => setRates([]));
34    }, [form.cropName, open]);
35
36    const suggestion = useMemo(() => {
37      if (!rates.length) return null;
38      const avg = rates.reduce((s, r) => s + r.avgPricePerKg, 0) / rates.length;
39      const min = Math.min(...rates.map((r) => r.minPricePerKg));
40      const max = Math.max(...rates.map((r) => r.maxPricePerKg));
41      return { avg, min, max, recommended: Math.round(avg * GRADE_FACTOR[form.qualityGrade] * 100) / 100 };
42    }, [rates, form.qualityGrade]);
43
44    function startNew() { setEditing(null); setForm(empty); setOpen(true); }
45    function startEdit(c: Crop) { setEditing(c); }
46

```

```

47     setForm({ cropName: c.cropName, category: c.category, qualityGrade: c.qualityGrade, quantity: String(c
    ↵ .totalQuantityKg), unit: "kg", basePricePerKg: String(c.basePricePerKg), harvestDate: c.harvestDate, farmL
    ↵ ocation: c.farmLocation, description: c.description ?? "", imagesJson: c.imagesJson });
48     setOpen(true);
49   }
50
51   async function onFiles(files: FileList | null) {
52     if (!files) return;
53     const room = 4 - form.imagesJson.length;
54     const list = Array.from(files).slice(0, room);
55     try {
56       const imgs = await Promise.all(list.map((f) => compressImage(f)));
57       setForm((f) => ({ ...f, imagesJson: [...f.imagesJson, ...imgs] }));
58     } catch (e) {
59       push({ kind: "error", title: (e as Error).message });
60     }
61   }
62
63   async function save(e: React.FormEvent) {
64     e.preventDefault();
65     setSaving(true);
66     const qtyKg = Number(form.quantity) * (form.unit === "tons" ? 1000 : form.unit === "maund" ? 40 : 1);
67     const payload = { cropName: form.cropName, category: form.category, qualityGrade: form.qualityGrade, t
    ↵ otalQuantityKg: qtyKg, basePricePerKg: Number(form.basePricePerKg), harvestDate: form.harvestDate, farmLoc
    ↵ ation: form.farmLocation, description: form.description, imagesJson: form.imagesJson };
68     try {
69       if (editing) {
70         await api(`/api/crops/${editing.cropId}`, { method: "PATCH", json: payload });
71         push({ kind: "success", title: "Listing updated" });
72       } else {
73         const c = await api<{ status: string }>("/api/crops", { method: "POST", json: payload });
74         push({ kind: "success", title: "Listing published", body: c.status === "pending_inspection" ? "Hel
    ↵ d for quality inspection until your account is verified." : "Now visible to buyers in the marketplace." })
    ↵ ;
75       }
76       setOpen(false);
77       load();
78     } catch (err) {
79       push({ kind: "error", title: "Could not save", body: (err as Error).message });
80     } finally {
81       setSaving(false);
82     }
83   }
84
85   async function remove(c: Crop) {
86     if (!confirm(`Remove ${c.cropName} listing? Open bids will be rejected.`)) return;
87     try {
88       const r = await api<{ archived?: boolean }>(`/api/crops/${c.cropId}`, { method: "DELETE" });
89       push({ kind: "success", title: r.archived ? "Listing archived (has order history)" : "Listing delete
    ↵ d" });
90       load();
91     } catch (err) {
92       push({ kind: "error", title: (err as Error).message });
93     }
94   }
95
96   const shown = (crops ?? []).filter((c) => filter === "all" || c.status === filter);
97
98   return (
99     <div>
100       <PageHeader title="My Crop Listings" urdu="[Urdu]" subtitle="Publish produce, manage stock and prici
    ↵ ng" actions={<button className="btn-primary" onClick={startNew}> New Listing</button>} />
101
102       <div className="mb-4 flex flex-wrap gap-2">
103         {[ "all", "active", "pending_inspection", "sold_out" ].map((s) => (
104           <button key={s} onClick={() => setFilter(s)} className={`rounded-full px-3.5 py-1.5 text-xs font
    ↵ -semibold capitalize ${filter === s ? "bg-slate-900 text-white dark:bg-white dark:text-slate-900" : "bg-wh
    ↵ ite text-slate-600 ring-1 ring-slate-200 dark:bg-slate-900 dark:text-slate-300 dark:ring-slate-700"}`>
105           {s.replace("-", " ")} {crops && <span className="opacity-60">{s === "all" ? crops.length : cr
    ↵ ops.filter((c) => c.status === s).length}</span>}
106         </button>
107       )]}
108     </div>
109
110     {!crops ? <LoadingBlock /> : shown.length === 0 ? (
111       <EmptyState icon="" title="No listings here" body="Create your first listing to start receiving bi

```

```

112     ) : (
113       <div className="grid gap-4 sm:grid-cols-2 xl:grid-cols-3">
114         {shown.map((c) => (
115           <article key={c.cropId} className="card overflow-hidden">
116             <div className="relative h-40">
117               {/* eslint-disable-next-line @next/next/no-img-element */}
118               <img src={cropImage(c.imagesJson, c.cropName, c.category)} alt={c.cropName} className="h-full w-full object-cover" />
119               <div className="absolute top-3 left-3 flex gap-1.5"><GradeBadge grade={c.qualityGrade} />{
120                 c.inspectionGrade && <span className="badge bg-white/90 text-emerald-700"> Inspected</span></div>
121               <div className="absolute top-3 right-3"><Badge status={c.status} /></div>
122             </div>
123             <div className="p-4">
124               <div className="flex items-start justify-between gap-2">
125                 <div>
126                   <h3 className="font-semibold">{c.cropName}</h3>
127                   <p className="text-xs text-slate-500"> {c.farmLocation} · Harvested {formatDate(c.harvestDate)}</p>
128                 </div>
129                 <div className="text-right"><p className="text-lg font-bold text-brand-700 dark:text-brand-400">{formatPKR(c.basePricePerKg, 2)}<span className="text-xs font-normal text-slate-500">/kg</span></p><p className="text-[11px] text-slate-500">{perMaund(c.basePricePerKg)}</p></div>
130               </div>
131               <div className="mt-3 grid grid-cols-3 gap-2 rounded-xl bg-slate-50 p-2.5 text-center text-xs dark:bg-slate-800/60">
132                 <div><p className="text-slate-500">Available</p><p className="font-semibold">{formatKg(c.totalQuantityKg)}</p></div>
133                 <div><p className="text-slate-500">Open bids</p><p className="font-semibold">{c.openBids}</p></div>
134                 <div><p className="text-slate-500">Top bid</p><p className="font-semibold">{c.highestBid ? formatPKR(c.highestBid, 1) : "-"}</p></div>
135               </div>
136               <div className="mt-3 flex gap-2">
137                 <button className="btn-secondary flex-1 py-1.5 text-xs" onClick={() => startEdit(c)}>[U
138                   rdu]Edit</button>
139                 <button className="btn-secondary py-1.5 text-xs text-rose-600" onClick={() => remove(c)}>
140                   ></button>
141               </div>
142             </div>
143           )})
144         </div>
145       <Modal open={open} onClose={() => setOpen(false)} title={editing ? `Edit ${editing.cropName}` : "New crop listing"} wide>
146         <form onSubmit={save} className="grid gap-4 sm:grid-cols-2">
147           <div>
148             <label className="label">Crop</label>
149             <select className="input" value={form.cropName} onChange={(e) => { const c = CROP_CATALOGUE.find((x) => x.name === e.target.value); setForm({ ...form, cropName: e.target.value, category: c?.category ? form.category : c?.category }); }}>
150               {CROP_CATALOGUE.map((c) => <option key={c.name}>{c.name}</option>)}
151             </select>
152           </div>
153           <div>
154             <label className="label">Category</label>
155             <select className="input" value={form.category} onChange={(e) => setForm({ ...form, category: e.target.value as Category })}>
156               {CATEGORIES.map((c) => <option key={c.value} value={c.value}>{c.emoji} {c.label}</option>)}
157             </select>
158           </div>
159           <div>
160             <label className="label">Quality grade</label>
161             <div className="grid grid-cols-3 gap-2">
162               {[["A", "B", "C"] as Grade[]].map((g) => (
163                 <button type="button" key={g} onClick={() => setForm({ ...form, qualityGrade: g })} className={
164                   `rounded-xl border-2 py-2 text-sm font-bold ${form.qualityGrade === g ? "border-brand-500 bg-brand-50 text-brand-700 dark:bg-brand-500/10 dark:text-brand-300" : "border-slate-200 dark:border-slate-700"}>{g}
165                 </button>
166               ))}
167             </div>
168           </div>
169         </form>

```

```

168         <label className="label">Harvest date</label>
169         <input type="date" className="input" max={todayISO()} value={form.harvestDate} onChange={(e) =
170         < > setForm({ ...form, harvestDate: e.target.value })} required />
171         </div>
172         <div>
173         <label className="label">Available quantity</label>
174         <div className="flex gap-2">
175         <input type="number" min="1" step="any" className="input" value={form.quantity} onChange={(e
176         < > => setForm({ ...form, quantity: e.target.value })} required placeholder="e.g. 5000" />
177         <select className="input w-28" value={form.unit} onChange={(e) => setForm({ ...form, unit: e
178         < .target.value as "kg" | "maund" | "tons" }}><option value="kg">Kg</option><option value="maund">Maund (40
179         < kg)</option><option value="tons">Tons</option></select>
180         </div>
181         </div>
182         <div>
183         <label className="label">Base price (Rs. / kg)</label>
184         <input type="number" min="0.5" step="0.01" className="input" value={form.basePricePerKg} onCha
185         < nge={(e) => setForm({ ...form, basePricePerKg: e.target.value })} required />
186         <{form.basePricePerKg && <p className="mt-1 text-xs text-slate-500">= {perMaund(Number(form.bas
187         < ePricePerKg))}</p>}
188         </div>
189         {suggestion && (
190         <div className="rounded-xl border border-brand-200 bg-brand-50 p-3 text-sm sm:col-span-2 dark:
191         < border-brand-800 dark:bg-brand-950/30">
192         <p className="font-semibold text-brand-800 dark:text-brand-300"> Live mandi insight for {for
193         < m.cropName}</p>
194         <p className="mt-1 text-brand-900/80 dark:text-brand-200/80">
195         < Today across {rates.length} mandis: {formatPKR(suggestion.min, 2)} – {formatPKR(suggestion
196         < .max, 2)} (avg {formatPKR(suggestion.avg, 2)}). Recommended for Grade {form.qualityGrade}: <b>{formatPKR(s
197         < uggestion.recommended, 2)}/kg ({perMaund(suggestion.recommended)})</b>
198         <button type="button" className="ml-2 font-semibold text-brand-700 underline dark:text-bra
199         < nd-300" onClick={() => setForm({ ...form, basePricePerKg: String(suggestion.recommended) }}>Use this</but
200         < ton>
201         </p>
202         </div>
203         )}
204         <div>
205         <label className="label">Farm location</label>
206         <select className="input" value={form.farmLocation} onChange={(e) => setForm({ ...form, farmLo
207         < cation: e.target.value })}><CityOptions /></select>
208         </div>
209         <div>
210         <label className="label">Crop photos (max 4)</label>
211         <input type="file" accept="image/*" multiple className="input file:mr-3 file:rounded-lg file:b
212         < order-0 file:bg-brand-50 file:px-3 file:py-1 file:text-brand-700" onChange={(e) => onFiles(e.target.files)
213         < } disabled={form.imagesJson.length >= 4} />
214         </div>
215         {form.imagesJson.length > 0 && (
216         <div className="flex flex-wrap gap-2 sm:col-span-2">
217         {form.imagesJson.map((src, i) => (
218         <div key={i} className="relative">
219         </* eslint-disable-next-line @next/next/no-img-element */>
220         <img src={src} alt="" className="h-20 w-28 rounded-lg object-cover" />
221         <button type="button" className="absolute -top-2 -right-2 grid h-6 w-6 place-items-cente
222         < r rounded-full bg-rose-600 text-xs text-white" onClick={() => setForm({ ...form, imagesJson: form.imagesJs
223         < on.filter((_, j) => j !== i) }}>x</button>
224         </div>
225         ))}
226         </div>
227         )}
228         <div className="sm:col-span-2">
229         <label className="label">Description</label>
230         <textarea className="input" rows={3} value={form.description} onChange={(e) => setForm({ ...fo
231         < rm, description: e.target.value })} placeholder="Variety, packing, moisture, pickup terms..." />
232         </div>
233         <div className="flex justify-end gap-2 sm:col-span-2">
234         <button type="button" className="btn-secondary" onClick={() => setOpen(false)}>Cancel</button>
235         <button className="btn-primary" disabled={!saving}>{saving && <Spinner />} {editing ? "Save cha
236         < nges" : "Publish listing"</button>
237         </div>
238         </form>
239         </Modal>
240         </div>
241         );
242     }
243 }
244

```

src/app/(app)/farmer/listings/page.tsx

```
1 import { ListingsClient } from "../listings-client";
2
3 export default async function ListingsPage({ searchParams }: { searchParams: Promise<{ new?: string }> }) {
4   const sp = await searchParams;
5   return <ListingsClient openNew={sp.new === "1"} />;
6 }
7
```

src/app/(app)/farmer/orders/page.tsx

```
1 import { OrdersBoard } from "@components/orders-board";
2
3 export default function FarmerOrdersPage() {
4   return <OrdersBoard role="farmer" title="Orders & Dispatch" subtitle="Dispatch orders and track payouts"
5   />;
6 }
6
```

src/app/(app)/farmer/page.tsx

```
1 import { redirect } from "next/navigation";
2
3 export default function Page() {
4   redirect("/farmer/dashboard");
5 }
6
```

src/app/(app)/farmer/reviews/page.tsx

```
1 import { requirePageUser } from "@lib/auth";
2 import { listReviews } from "@lib/services/reviews";
3 import { FarmerReviewBoard } from "@components/reviews/farmer-review-board";
4
5 export const dynamic = "force-dynamic";
6 export const metadata = { title: "Buyer reviews" };
7
8 export default async function FarmerReviewsPage() {
9   const farmer = await requirePageUser(["farmer"]);
10  const reviews = await listReviews(farmer.id, 100);
11  return (
12    <FarmerReviewBoard
13      initialReviews={reviews}
14      trustScore={Number(farmer.trustScore)}
15      ratingAverage={Number(farmer.ratingAvg)}
16      ratingCount={farmer.ratingCount}
17    />
18  );
19 }
20
```

src/app/(app)/inspector/dashboard/page.tsx

```
1 "use client";
2
3 import { useCallback, useEffect, useState } from "react";
4 import { api, Badge, GradeBadge, Modal, PageHeader, Spinner, StatCard } from "@components/ui";
5 import { useToast } from "@components/providers";
6 import { formatDate, formatKg } from "@lib/constants";
7
8 type Crop = { cropId: number; cropName: string; farmerName: string; farmName: string | null; farmLocation:
9   string; qualityGrade: string; totalQuantityKg: number; status: string; harvestDate: string; inspectionGra
10  de: string | null };
11 type Order = { orderId: number; trackingNumber: string; cropName: string; quantityKg: number; qualityGrade
12   : string; farmLocation: string; farmName: string | null; farmerName: string; buyerBusiness: string | null;
13   buyerName: string; paymentStatus: string; deliveryStage: string; createdAt: string };
14 type Insp = { inspectionId: number; cropName: string; farmerName: string; farmLocation: string; inspectorN
15   ame: string; gradeAssigned: string; moistureLevelPercentage: number; soilPh: number | null; inspectionNote
16   s: string | null; reportAttachment: string | null; status: string; verifiedAt: string | null; certificateN
17   o?: string | null };
18
```

```

11 type Target = { kind: "listing"; crop: Crop } | { kind: "order"; order: Order };
12
13 export default function InspectorDashboard() {
14   const { push } = useToast();
15   const [crops, setCrops] = useState<Crop[]>([]);
16   const [orders, setOrders] = useState<Order[]>([]);
17   const [reports, setReports] = useState<Insp[]>([]);
18   const [loading, setLoading] = useState(true);
19   const [target, setTarget] = useState<Target | null>(null);
20   const [f, setF] = useState({ grade: "A", moisture: "12", ph: "7.2", notes: "", status: "passed", file: ""
  ↵  ", fileName: "" });
21   const [busy, setBusy] = useState(false);
22
23   const load = useCallback(async () => {
24     try {
25       const [c, o, r] = await Promise.all([api<Crop[]>("/api/crops"), api<Order[]>("/api/orders?active=1")
  ↵  , api<Insp[]>("/api/inspections")]);
26       setCrops(c.filter((x) => x.status === "pending_inspection"));
27       setOrders(o.filter((x) => x.paymentStatus === "escrow_locked" && x.deliveryStage === "confirmed"));
28       setReports(r);
29     } catch (e) {
30       push({ kind: "error", title: (e as Error).message });
31     } finally {
32       setLoading(false);
33     }
34   }, [push]);
35   useEffect(() => { load(); }, [load]);
36
37   function open(t: Target) {
38     setTarget(t);
39     setF({ grade: t.kind === "listing" ? t.crop.qualityGrade : t.order.qualityGrade, moisture: "12", ph: "
  ↵  7.2", notes: "", status: "passed", file: "", fileName: "" });
40   }
41   function onFile(file?: File) {
42     if (!file) return;
43     if (!["application/pdf", "image/png", "image/jpeg"].includes(file.type)) return push({ kind: "error",
  ↵  title: "Upload a PDF, PNG or JPG lab report" });
44     if (file.size > 1_400_000) return push({ kind: "error", title: "File too large (max 1.4 MB)" });
45     const r = new FileReader();
46     r.onload = () => setF(x => ({ ...x, file: r.result as string, fileName: file.name }));
47     r.readAsDataURL(file);
48   }
49
50   async function submit(e: React.FormEvent) {
51     e.preventDefault();
52     if (!target) return;
53     setBusy(true);
54     try {
55       if (target.kind === "listing") {
56         await api("/api/inspections", { method: "POST", json: { cropId: target.crop.cropId, gradeAssigned:
  ↵  f.grade, moistureLevelPercentage: Number(f.moisture), soilPh: f.ph ? Number(f.ph) : undefined, inspection
  ↵  Notes: f.notes, status: f.status, reportAttachment: f.file || undefined, reportFileName: f.fileName || und
  ↵  efined } });
57         push({ kind: "success", title: f.status === "passed" ? "Certified: listing is now live" : f.status
  ↵  === "failed" ? "Listing rejected" : "Saved as pending lab result" });
58       } else {
59         await api(`/api/orders/${target.order.orderId}`, { method: "PATCH", json: { action: "advance", gra
  ↵  de: f.grade, moisture: Number(f.moisture), note: f.notes || `Pre-dispatch inspection · pH ${f.ph}` } });
60         push({ kind: "success", title: "Order certified for dispatch", body: "Farmer and buyer have been n
  ↵  otified." });
61       }
62       setTarget(null);
63       load();
64     } catch (err) {
65       push({ kind: "error", title: "Could not save", body: (err as Error).message });
66     } finally {
67       setBusy(false);
68     }
69   }
70
71   const passed = reports.filter((r) => r.status === "passed").length;
72   return (
73     <div className="space-y-6">
74       <PageHeader title="Quality Inspection Desk" urdu="[Urdu]" subtitle="Lab tests, grade certification a
  ↵  nd pre-dispatch checks" />
75       <div className="grid grid-cols-2 gap-3 sm:gap-4 xl:grid-cols-4">

```

```

76     <StatCard label="Orders awaiting QC" value={loading ? "-" : orders.length} sub="Escrow funded, ready to inspect" icon="" tone="amber" />
77     <StatCard label="Listings to certify" value={loading ? "-" : crops.length} sub="New / unverified sellers" icon="" />
78     <StatCard label="Reports filed" value={reports.length} sub={` ${passed} passed`} icon="" tone="violet" />
79     <StatCard label="Pass rate" value={reports.length ? `${Math.round((passed / reports.length) * 100)}%` : "-"} sub="All inspections" icon="" tone="sky" />
80   </div>
81
82   <div className="grid gap-5 lg:grid-cols-2">
83     <section className="card overflow-hidden">
84       <h2 className="border-b border-slate-100 px-5 py-3.5 font-bold dark:border-white/5">Pre-dispatch inspections</h2>
85       {orders.length === 0 ? <p className="p-8 text-center text-sm text-slate-500">{loading ? "Loading..." : "No orders waiting for inspection"}</p> : (
86         <ul className="divide-y divide-slate-100 dark:divide-white/5">
87           {orders.map((o) => (
88             <li key={o.orderId} className="flex items-center gap-3 px-5 py-3">
89               <div className="min-w-0 flex-1">
90                 <p className="truncate text-sm font-semibold">{o.cropName} · {formatKg(o.quantityKg)}</p>
91                 <p className="font-mono text-xs font-normal text-slate-500">{o.trackingNumber}</p>
92                 <p className="truncate text-xs text-slate-500">{o.farmName ?? o.farmerName} ({o.farmLocation}) → {o.buyerBusiness ?? o.buyerName}</p>
93               </div>
94               <GradeBadge grade={o.qualityGrade} />
95               <button className="btn-primary px-3 py-1.5 text-xs" onClick={() => open({ kind: "order", order: o })}>Inspect</button>
96             </li>
97           ))}
98         </ul>
99       )}
100     </section>
101     <section className="card overflow-hidden">
102       <h2 className="border-b border-slate-100 px-5 py-3.5 font-bold dark:border-white/5">Listing certification</h2>
103       {crops.length === 0 ? <p className="p-8 text-center text-sm text-slate-500">{loading ? "Loading..." : "No listings pending certification"}</p> : (
104         <ul className="divide-y divide-slate-100 dark:divide-white/5">
105           {crops.map((c) => (
106             <li key={c.cropId} className="flex items-center gap-3 px-5 py-3">
107               <div className="min-w-0 flex-1">
108                 <p className="truncate text-sm font-semibold">{c.cropName} · {formatKg(c.totalQuantityKg)}</p>
109                 <p className="truncate text-xs text-slate-500">{c.farmName ?? c.farmerName} · {c.farmLocation} · harvested {formatDate(c.harvestDate)}</p>
110               </div>
111               <GradeBadge grade={c.qualityGrade} />
112               <button className="btn-primary px-3 py-1.5 text-xs" onClick={() => open({ kind: "listing", crop: c })}>Lab test</button>
113             </li>
114           ))}
115         </ul>
116       )}
117     </section>
118   </div>
119
120   <section className="card overflow-hidden">
121     <h2 className="border-b border-slate-100 px-5 py-3.5 font-bold dark:border-white/5">Inspection reports</h2>
122     <div className="overflow-x-auto">
123       <table className="w-full text-sm">
124         <thead className="table-head"><tr><th className="px-4 py-3">Crop / farmer</th><th className="px-4 py-3">Soil pH</th><th className="px-4 py-3">Moisture</th><th className="px-4 py-3">Result</th><th className="px-4 py-3">Inspector</th><th className="px-4 py-3">Report</th></tr></thead>
125         <tbody className="divide-y divide-slate-100 dark:divide-white/5">
126           {reports.map((r) => (
127             <tr key={r.inspectionId}>
128               <td className="px-4 py-3"><p className="font-medium">{r.cropName}</p><p className="text-xs text-slate-500">{r.farmerName} · {r.farmLocation}</p></td>
129               <td className="px-4 py-3"><GradeBadge grade={r.gradeAssigned} /></td>
130               <td className="px-4 py-3 tabular-nums">{r.moistureLevelPercentage}%</td>
131               <td className="px-4 py-3 tabular-nums">{r.soilPh ?? "-"}</td>
132               <td className="px-4 py-3"><Badge status={r.status} /><p className="mt-1 text-[11px] text-slate-500">{r.verifiedAt ? formatDate(r.verifiedAt) : ""}</p></td>
133             </tr>
134           ))}
135         </tbody>
136       </table>
137     </div>
138   </section>

```

```

132         <td className="px-4 py-3 text-xs text-slate-500">{r.inspectorName}</td>
133         <td className="px-4 py-3">{r.reportAttachment ? <a href={` /api/inspections/${r.inspectio
    nId}/report`} target="_blank" rel="noreferrer" className="text-xs font-semibold text-brand-700 hover:under
    line dark:text-brand-400"> Open</a> : <span className="text-xs text-slate-400">—</span></td>
134     </tr>
135   )}}
136 </tbody>
137 </table>
138 </div>
139 </section>
140
141   <Modal open={!target} onClose={() => setTarget(null)} title={target ? (target.kind === "order" ? `P
    re-dispatch QC · ${target.order.trackingNumber}` : `Lab certification · ${target.crop.cropName}`) : ""}>
142     {target && (
143       <form onSubmit={submit} className="grid grid-cols-2 gap-3">
144         <div><label className="label">Grade certification</label><select className="input" value={f.gr
    ade} onChange={(e) => setF({ ...f, grade: e.target.value })}><option value="A">A · Premium / export</optio
    n><option value="B">B · Standard</option><option value="C">C · Processing</option></select></div>
145         {target.kind === "listing" ? (
146           <div><label className="label">Result</label><select className="input" value={f.status} onCha
    nge={(e) => setF({ ...f, status: e.target.value })}><option value="passed"> Passed</option><option value="
    pending"> Awaiting lab</option><option value="failed"> Failed</option></select></div>
147         ) : <div><label className="label">Result</label><input className="input" value=" Passed → rele
    ase for dispatch" readOnly /></div>
148         <div><label className="label">Moisture %</label><input type="number" step="0.1" min="0" max="1
    00" className="input" value={f.moisture} onChange={(e) => setF({ ...f, moisture: e.target.value })} requir
    ed /></div>
149         <div><label className="label">Soil pH</label><input type="number" step="0.1" min="0" max="14"
    className="input" value={f.ph} onChange={(e) => setF({ ...f, ph: e.target.value })} /></div>
150         <div className="col-span-2"><label className="label">Lab notes</label><textarea rows={3} class
    Name="input" value={f.notes} onChange={(e) => setF({ ...f, notes: e.target.value })} placeholder="Foreign
    matter %, broken grains, pest damage, colour, brix..." /></div>
151         {target.kind === "listing" && (
152           <div className="col-span-2">
153             <label className="label">Lab report (PDF)</label>
154             <input type="file" accept="application/pdf,image/png,image/jpeg" className="input" onChang
    e={(e) => onFile(e.target.files?.[0])} />
155             {f.fileName && <p className="mt-1 text-xs text-brand-700"> {f.fileName}</p>}
156           </div>
157         )}
158         <div className="col-span-2 flex justify-end gap-2">
159           <button type="button" className="btn-secondary" onClick={() => setTarget(null)}>Cancel</butt
    on>
160           <button className="btn-primary" disabled={busy}>{busy && <Spinner />} {target.kind === "orde
    r" ? "Certify & release" : "Save report"}</button>
161         </div>
162       </form>
163     )}
164   </Modal>
165 </div>
166 );
167 }
168

```

src/app/(app)/inspector/layout.tsx

```

1 import type { ReactNode } from "react";
2 import { requirePageUser } from "@lib/auth";
3
4 export default async function InspectorLayout({ children }: { children: ReactNode }) {
5   await requirePageUser(["inspector", "admin"]);
6   return <>{children}</>;
7 }
8

```

src/app/(app)/inspector/page.tsx

```

1 import { redirect } from "next/navigation";
2
3 export default function Page() {
4   redirect("/inspector/dashboard");
5 }
6

```

src/app/(app)/layout.tsx

```
1 import type { ReactNode } from "react";
2 import { AppShell } from "@components/app-shell";
3 import { requirePageUser } from "@lib/auth";
4 import { getTicker } from "@lib/ticker";
5 import { getViewer } from "@lib/viewer";
6
7 export const dynamic = "force-dynamic";
8
9 export default async function AppLayout({ children }: { children: ReactNode }) {
10   const user = await requirePageUser();
11   const [ticker, viewer] = await Promise.all([getTicker(), getViewer(user)]);
12   return (
13     <AppShell user={{ fullName: user.fullName, role: user.role, businessName: user.businessName, isVerified: user.isVerified, city: user.city, title: user.title, avatarUrl: user.avatarUrl, userCode: user.userCode }} ticker={ticker} viewer={viewer}>
14       {children}
15     </AppShell>
16   );
17 }
18
```

src/app/(app)/notifications/page.tsx

```
1 import Link from "next/link";
2 import { desc, eq } from "drizzle-orm";
3 import { db } from "@db";
4 import { notifications } from "@db/schema";
5 import { requirePageUser } from "@lib/auth";
6 import { PageHeader } from "@components/ui";
7 import { MarkAllRead } from "@components/nav/mark-all-read";
8
9 export const dynamic = "force-dynamic";
10
11 export default async function NotificationsPage() {
12   const user = await requirePageUser();
13   const items = await db.select().from(notifications).where(eq(notifications.userId, user.id)).orderBy(desc(notifications.createdAt)).limit(100);
14   return (
15     <div className="space-y-6">
16       <PageHeader title="Notifications" urdu="[Urdu]" subtitle="Bids, escrow, shipments, inspections and p
17       ayouts" actions={<MarkAllRead />} />
18       <div className="card overflow-hidden">
19         {items.length === 0 ? <p className="p-10 text-center text-sm text-slate-500">No notifications yet.
20         </p> : (
21           <ul className="divide-y divide-slate-100 dark:divide-white/5">
22             {items.map((n) => (
23               <li key={n.notificationId} className={`flex gap-4 px-5 py-4 ${n.isRead ? "" : "bg-brand-50/50 dark:bg-brand-500/5"}`}>
24                 <span className={`mt-1.5 h-2 w-2 shrink-0 rounded-full ${n.isRead ? "bg-transparent" : "bg-brand-600"}`> />
25                 <div className="min-w-0 flex-1">
26                   <p className="font-semibold">{n.title}</p>
27                   {n.body && <p className="text-sm text-slate-600 dark:text-slate-300">{n.body}</p>}
28                   <p className="mt-1 text-xs text-slate-400">{new Date(n.createdAt).toLocaleString("en-GB"
29                   , { dateStyle: "medium", timeStyle: "short" })} · via {n.channels.replace(/in_app/, "app").replace(/:logged/g, " (outbox)").replace(/,/g, ", ")}</p>
30                 </div>
31                 {n.link && <Link href={n.link} className="self-center text-sm font-semibold text-brand-700
32                 hover:underline dark:text-brand-400">Open </Link>}
33               </li>
34             )]}
35           </ul>
36         )}
37     </div>
38   );
39 }
```

src/app/(app)/wallet/page.tsx

```
1 import { requirePageUser } from "@lib/auth";
2 import { Suspense } from "react";
3 import { WalletClient } from "../wallet-client";
4
5 export default async function WalletPage() {
6   const user = await requirePageUser(["farmer", "buyer"]);
7   return (
8     <Suspense>
9       <WalletClient role={user.role as "farmer" | "buyer"} />
10    </Suspense>
11  );
12 }
13
```

src/app/(app)/wallet/wallet-client.tsx

```
1 "use client";
2
3 import { useSearchParams, useRouter } from "next/navigation";
4 import { useCallback, useEffect, useState } from "react";
5 import { api, Badge, LoadingBlock, PageHeader, Spinner, StatCard } from "@components/ui";
6 import { useToast } from "@components/providers";
7 import { resolveBrowserRedirect } from "@lib/base-url";
8 import { formatDate, formatPKR, formatPKRShort } from "@lib/constants";
9
10 type Wallet = {
11   walletBalance: number; escrowBalance: number; pendingInEscrow: number;
12   transactions: { txnId: number; type: string; amount: number; description: string | null; createdAt: string }[];
13 };
14 type Providers = Record<"stripe" | "jazzcash" | "easypaisa" | "sandbox", { enabled: boolean; label: string; mode: string }>;
15 type Payout = { payoutId: number; amount: number; method: string; accountNumber: string; status: string; adminNote: string | null; createdAt: string };
16
17 const TX: Record<string, { icon: string; sign: 1 | -1 | 0; label: string }> = {
18   deposit: { icon: "", sign: 1, label: "Top-up" },
19   escrow_lock: { icon: "", sign: -1, label: "Escrow lock" },
20   escrow_release: { icon: "", sign: 0, label: "Escrow released" },
21   payout: { icon: "", sign: 1, label: "Sale proceeds" },
22   refund: { icon: "↩️[Urdu]", sign: 1, label: "Refund" },
23   platform_fee: { icon: "", sign: 0, label: "Platform fee" },
24   withdrawal: { icon: "", sign: -1, label: "Withdrawal" },
25 };
26 const PROVIDER_ICON = { stripe: "", jazzcash: "", easypaisa: "", sandbox: "" };
27
28 function submitForm(action: string, fields: Record<string, string>) {
29   const form = document.createElement("form");
30   form.method = "POST";
31   form.action = action;
32   Object.entries(fields).forEach(([k, v]) => {
33     const i = document.createElement("input");
34     i.type = "hidden";
35     i.name = k;
36     i.value = v;
37     form.appendChild(i);
38   });
39   document.body.appendChild(form);
40   form.submit();
41 }
42
43 /**
44  * The API returns absolute URLs. Validate them before use and, if the server ever hands back a
45  * bind/loopback address (http://0.0.0.0:3000/...) while this page runs on a public origin, send the
46  * browser to the same path on this origin instead of failing with ERR_ADDRESS_INVALID.
47  */
48 function safeTarget(raw: string): string {
49   const resolved = resolveBrowserRedirect(raw, window.location.origin);
50   if (!resolved.ok) throw new Error(resolved.error);
51   if (resolved.rewritten) console.warn(`[wallet] payment URL host was not reachable, using this origin instead: ${raw} → ${resolved.url}`);
52   return resolved.url;
53 }
```

```

53 }
54
55 export function WalletClient({ role }: { role: "farmer" | "buyer" }) {
56   const { push } = useToast();
57   const sp = useSearchParams();
58   const router = useRouter();
59   const [w, setW] = useState<Wallet | null>(null);
60   const [providers, setProviders] = useState<Providers | null>(null);
61   const [provider, setProvider] = useState<keyof Providers>("sandbox");
62   const [amount, setAmount] = useState("500000");
63   const [busy, setBusy] = useState(false);
64   const [payment, setPayment] = useState<{ reference: string; status: string; amount: number } | null>(null);
65   const [payouts, setPayouts] = useState<Payout[]>([]);
66   const [po, setPo] = useState({ amount: "", method: "bank", accountTitle: "", accountNumber: "" });
67
68   const load = useCallback(async () => {
69     try {
70       setW(await api<Wallet>("/api/wallet"));
71       if (role === "buyer") {
72         const p = await api<Providers>("/api/payments/checkout");
73         setProviders(p);
74         const first = ([ "stripe", "jazzcash", "easypaisa", "sandbox" ] as const).find((k) => p[k].enabled);
75         if (first) setProvider(first);
76       } else setPayouts(await api<Payout[]>("/api/payouts"));
77     } catch (e) {
78       push({ kind: "error", title: (e as Error).message });
79     }
80   }, [push, role]);
81   useEffect(() => { load(); }, [load]);
82
83   // Returning from a gateway: poll the payment status until the webhook/callback settles it
84   const ref = sp.get("payment");
85   useEffect(() => {
86     if (!ref) return;
87     let n = 0;
88     let stop = false;
89     const tick = async () => {
90       try {
91         const p = await api<{ reference: string; status: string; amount: number }>(`/api/payments/${ref}`)
92         ;
93         setPayment(p);
94         if (p.status !== "pending" || n++ > 20) {
95           if (p.status === "succeeded") { push({ kind: "success", title: "Payment received", body: `${form
96             atPKR(p.amount)} added to your wallet` }); load(); }
97           return;
98         } catch { return; }
99         if (!stop) setTimeout(tick, 2000);
100       };
101       tick();
102       return () => { stop = true; };
103     }, [ref, push, load]);
104
105     async function topUp() {
106       setBusy(true);
107       try {
108         const r = await api<{ redirectUrl?: string; form?: { action: string; fields: Record<string, string>
109           } }>("/api/payments/checkout", { method: "POST", json: { provider, amount: Number(amount) } });
110         if (r.redirectUrl) { window.location.assign(safeTarget(r.redirectUrl)); return; }
111         if (r.form) submitForm(safeTarget(r.form.action), r.form.fields);
112         else throw new Error("The payment gateway did not return a checkout link. Please try again.");
113       } catch (e) {
114         push({ kind: "error", title: "Could not start payment", body: (e as Error).message });
115         setBusy(false);
116       }
117     }
118
119     async function requestPayout(e: React.FormEvent) {
120       e.preventDefault();
121       setBusy(true);
122       try {
123         await api("/api/payouts", { method: "POST", json: { ...po, amount: Number(po.amount) } });
124         push({ kind: "success", title: "Payout requested", body: "An admin will process it within 1 business
125         day." });
126         setPo({ amount: "", method: "bank", accountTitle: "", accountNumber: "" });
127       }
128     }
129   }

```

```

124     load();
125   } catch (err) {
126     push({ kind: "error", title: (err as Error).message });
127   } finally {
128     setBusy(false);
129   }
130 }
131
132 if (!w) return <LoadingBlock />;
133 return (
134   <div className="space-y-6">
135     <PageHeader title="Escrow Wallet" urdu="[Urdu]" subtitle={role === "buyer" ? "Add funds through a pa
136 yment gateway and pay farmers safely through escrow" : "Sale proceeds arrive here when buyers confirm deli
137 very"} />
138     {payment && (
139       <div className={`card flex flex-wrap items-center gap-3 p-4 ${payment.status === "succeeded" ? "bo
140 rder-emerald-300 bg-emerald-50 dark:bg-emerald-950/30" : payment.status === "pending" ? "border-sky-300 bg
141 -sky-50 dark:bg-sky-950/30" : "border-rose-300 bg-rose-50 dark:bg-rose-950/30"}`}>
142         <span className="text-2xl">{payment.status === "succeeded" ? "" : payment.status === "pending" ?
143 "" : "△[Urdu]}</span>
144         <p className="flex-1 text-sm">
145           {payment.status === "succeeded" ? <>Payment <b>{payment.reference}</b> received: {formatPKR(pa
146 yment.amount)} added.</> : payment.status === "pending" ? <>Waiting for the gateway to confirm payment <b>
147 {payment.reference}</b>...</> : <>Payment <b>{payment.reference}</b> {payment.status}. No money was deducted
148 .</>}
149         </p>
150         <button className="text-xs font-semibold text-slate-500 hover:text-slate-800" onClick={() => { s
151 etPayment(null); router.replace("/wallet"); }}>Dismiss</button>
152       </div>
153     )}
154     <div className="grid gap-4 sm:grid-cols-3">
155       <StatCard label="Available balance" value={formatPKRShort(w.walletBalance)} icon="" />
156       <StatCard label={role === "buyer" ? "Locked in escrow" : "Incoming (in escrow)"} value={formatPKRS
157 hort(role === "buyer" ? w.escrowBalance : w.pendingInEscrow)} icon="" tone="sky" />
158       <StatCard label="Transactions" value={w.transactions.length} icon="" tone="violet" />
159     </div>
160     <div className="grid gap-4 lg:grid-cols-3">
161       {role === "buyer" ? (
162         <section className="card space-y-4 p-5">
163           <h2 className="font-bold">Add funds</h2>
164           <div className="grid gap-2">
165             {providers && (Object.keys(providers) as (keyof Providers)[]).map((k) => (
166               <label key={k} className={`flex cursor-pointer items-center gap-3 rounded-xl border-2 p-3
167 text-sm transition ${!providers[k].enabled ? "cursor-not-allowed opacity-45" : provider === k ? "border-br
168 and-600 bg-brand-50 dark:bg-brand-500/10" : "border-slate-200 dark:border-white/10"}`}>
169                 <input type="radio" name="provider" className="sr-only" disabled={!providers[k].enabled}
170 checked={provider === k} onChange={() => setProvider(k)} />
171                 <span className="text-xl">{PROVIDER_ICON[k]}</span>
172                 <span className="flex-1 font-semibold">{providers[k].label}</span>
173                 <span className="text-[10px] font-bold text-slate-500 uppercase">{providers[k].enabled ?
174 providers[k].mode : "not configured"}</span>
175               </label>
176             ))}
177           </div>
178           <div>
179             <label className="label">Amount (PKR)</label>
180             <input type="number" min={1000} className="input" value={amount} onChange={(e) => setAmount(
181 e.target.value)} />
182             <div className="mt-2 flex flex-wrap gap-1.5">
183               {[100000, 500000, 2500000].map((a) => <button key={a} className="rounded-full bg-slate-100
184 px-3 py-1 text-xs font-semibold dark:bg-white/10" onClick={() => setAmount(String(a))}>{formatPKRShort(a)
185 }</button>)}
186             </div>
187           </div>
188           <button className="btn-primary w-full" disabled={busy || !providers?.[provider]?.enabled} onCl
189 ick={topUp}>{busy && <Spinner />} Continue to {providers?.[provider]?.label ?? "payment"}</button>
190           <p className="text-[11px] text-slate-500">Your wallet is credited only after the gateway confi
191 rms the payment with a signed callback.</p>
192         </section>
193       ) : (
194         <section className="card p-5">
195           <h2 className="font-bold">Request payout</h2>
196           <form onSubmit={requestPayout} className="mt-3 space-y-3">

```

```

181     <div><label className="label">Amount (PKR)</label><input type="number" min={1000} max={w.wal
    letBalance} className="input" required value={po.amount} onChange={(e) => setPo({ ...po, amount: e.target.
    value }}} /><button type="button" className="mt-1 text-xs font-semibold text-brand-700" onClick={() => set
    Po({ ...po, amount: String(Math.floor(w.walletBalance)) }}>Withdraw all ({formatPKR(w.walletBalance)})</b
    utton></div>
182     <div><label className="label">Method</label><select className="input" value={po.method} onCh
    ange={(e) => setPo({ ...po, method: e.target.value }}><option value="bank">Bank account (IBFT)</option><o
    ption value="jazzcash">JazzCash wallet</option><option value="easypaisa">Easypaisa wallet</option></select
    ></div>
183     <div><label className="label">Account title</label><input className="input" required value={
    po.accountTitle} onChange={(e) => setPo({ ...po, accountTitle: e.target.value }} /></div>
184     <div><label className="label">{po.method === "bank" ? "IBAN" : "Mobile number"}</label><input
    t className="input font-mono" required value={po.accountNumber} onChange={(e) => setPo({ ...po, accountNum
    ber: e.target.value }} placeholder={po.method === "bank" ? "PK36SCBL0000001123456702" : "03xx xxxxxxxx"} /
    ></div>
185     <button className="btn-primary w-full" disabled={busy || w.walletBalance < 1000}>{busy && <S
    pinner />} Request payout</button>
186   </form>
187   {payouts.length > 0 && (
188     <ul className="mt-5 space-y-2 border-t border-slate-100 pt-4 text-sm dark:border-white/5">
189       {payouts.slice(0, 5).map((p) => (
190         <li key={p.payoutId} className="flex items-center justify-between gap-2">
191           <span><b>{formatPKR(p.amount)}</b> <span className="text-xs text-slate-500">· {p.metho
    d} · {formatDate(p.createdAt)}</span></span>
192           <Badge status={p.status === "paid" ? "released" : p.status === "rejected" ? "rejected"
    : "pending"} label={p.status} />
193         </li>
194       )}}
195     </ul>
196   )}
197 </section>
198 )}
199 <section className="card overflow-hidden lg:col-span-2">
200   <h2 className="border-b border-slate-100 px-5 py-3.5 font-bold dark:border-white/5">Ledger</h2>
201   <ul className="max-h-[560px] divide-y divide-slate-100 overflow-y-auto dark:divide-white/5">
202     {w.transactions.map((t) => {
203       const meta = TX[t.type] ?? { icon: "•", sign: 0, label: t.type };
204       return (
205         <li key={t.txnId} className="flex items-center gap-3 px-5 py-3">
206           <span className="grid h-9 w-9 place-items-center rounded-xl bg-slate-100 dark:bg-white/5
    >{meta.icon}</span>
207           <div className="min-w-0 flex-1"><p className="text-sm font-semibold">{meta.label}</p><p
    className="truncate text-xs text-slate-500">{t.description} · {formatDate(t.createdAt)}</p></div>
208           <p className={`text-sm font-bold tabular-nums ${meta.sign > 0 ? "text-emerald-600" : met
    a.sign < 0 ? "text-rose-600" : "text-slate-500"}`}>{meta.sign > 0 ? "+" : meta.sign < 0 ? "-" : ""}{format
    PKR(t.amount)}</p>
209         </li>
210       );
211     })}
212     {w.transactions.length === 0 && <li className="p-8 text-center text-sm text-slate-500">No tran
    sactions yet</li>
213   </ul>
214 </section>
215 </div>
216 </div>
217 );
218 }
219

```

src/app/(site)/crop/[id]/page.tsx

```

1  import Link from "next/link";
2  import { notFound } from "next/navigation";
3  import { sql } from "drizzle-orm";
4  import { db } from "@/db";
5  import { getCurrentUser } from "@/lib/auth";
6  import { getCrop } from "@/lib/services/crops";
7  import { rateTrend } from "@/lib/services/mandi";
8  import { listReviews } from "@/lib/services/reviews";
9  import { Badge, GradeBadge } from "@/components/ui";
10 import { CropPriceChart } from "@/components/crop/price-chart";
11 import { BidPanel } from "@/components/crop/bid-panel";
12 import { CITIES, categoryLabel, cropImage, cropUrdu, formatDate, formatKg, formatPKR, perMaund } from "@/l
    ib/constants";
13

```

```

14 export const dynamic = "force-dynamic";
15
16 function Stars({ value, size = "text-sm" }: { value: number; size?: string }) {
17   return (
18     <span className={` ${size} tracking-tight text-gold-500`} aria-label={` ${value.toFixed(1)} out of 5`} >
19       {"*****".slice(0, Math.round(value))}
20       <span className="text-slate-300 dark:text-slate-600">{"*****".slice(Math.round(value))}</span>
21     </span>
22   );
23 }
24
25 export async function generateMetadata({ params }: { params: Promise<{ id: string }> }) {
26   const id = Number((await params).id);
27   try {
28     const c = await getCrop(id);
29     return { title: `${c.cropName} · Grade ${c.qualityGrade} · ${c.farmLocation}` };
30   } catch {
31     return { title: "Crop not found" };
32   }
33 }
34
35 export default async function CropDetailsPage({ params }: { params: Promise<{ id: string }> }) {
36   const id = Number((await params).id);
37   if (!Number.isFinite(id)) notFound();
38   let crop: Awaited<ReturnType<typeof getCrop>>;
39   try {
40     crop = await getCrop(id);
41   } catch {
42     notFound();
43   }
44   const [user, trend, reviews, dealRows, bidStats] = await Promise.all([
45     getCurrentUser(),
46     rateTrend(crop.cropName, 30),
47     listReviews(crop.farmerId, 6),
48     db.execute(sql`
49       select to_char(o.created_at, 'YYYY-MM-DD') as d, round(sum(o.total_amount) / nullif(sum(o.quantity_k
50 g), 0), 2) as p
51       from orders_logistics o join crops_inventory c on c.crop_id = o.crop_id
52       where c.crop_name = ${crop.cropName} and o.created_at > now() - interval '30 days' and o.payment_sta
53 tus <> 'refunded'
54       group by 1 order by 1`),
55     db.execute(sql`select count(*)::int as n, max(bid_price_per_kg) as hi from bids_negotiations where cro
56 p_id = ${id} and status in ('pending','countered')`),
57   ]);
58   // Chart series: mandi average across markets vs platform deals vs this listing's ask
59   const deals = new Map((dealRows.rows as { d: string; p: string }[]).map((r) => [r.d, Number(r.p)]));
60   const series = trend.series.map((row) => {
61     const vals = trend.markets.map((m) => Number(row[m])).filter((v) => Number.isFinite(v) && v > 0);
62     const date = String(row.date);
63     return { date, mandi: vals.length ? Math.round((vals.reduce((a, b) => a + b, 0) / vals.length) * 100)
64 / 100 : null, platform: deals.get(date) ?? null, ask: crop.basePricePerKg };
65   });
66   const lastMandi = [...series].reverse().find((s) => s.mandi)?.mandi ?? null;
67   const vsMandi = lastMandi ? ((crop.basePricePerKg - lastMandi) / lastMandi) * 100 : null;
68   const passed = crop.inspections.filter((i) => i.status === "passed");
69   const latest = passed[0];
70   const bids = bidStats.rows[0] as { n: number; hi: string | null };
71   const province = CITIES[crop.farmLocation]?.province;
72   const available = crop.status === "active";
73
74   return (
75     <div className="space-y-8">
76       <nav className="text-sm text-slate-500" aria-label="Breadcrumb">
77         <Link href="/marketplace" className="hover:text-brand-700">Marketplace</Link> <span className="mx-
78 1"></span>
79         <Link href={` /marketplace?category=${crop.category}`} className="hover:text-brand-700">{categoryLa
80 bel(crop.category)}</Link> <span className="mx-1"></span>
81         <span className="text-slate-800 dark:text-slate-200">{crop.cropName}</span>
82       </nav>
83
84       <div className="grid gap-8 lg:grid-cols-[1.1fr_1fr]">
85         <div>
86           <div className="relative overflow-hidden rounded-3xl">
87             <div>
88               <div>
89                 <div>
90                   <div>
91                     <div>
92                       <div>
93                         <div>
94                           <div>
95                             <div>
96                               <div>
97                                 <div>
98                                   <div>
99                                     <div>
100                   </div>
101                 </div>
102               </div>
103             </div>
104           </div>
105         </div>
106       </div>
107     </div>
108   );
109 }

```

```

84         <img src={cropImage(crop.imagesJson, crop.cropName, crop.category)} alt={crop.cropName} classN
    ame="aspect-[4/3] w-full object-cover" />
85         <div className="absolute top-4 left-4 flex gap-2">
86             <GradeBadge grade={crop.qualityGrade} />
87             {latest && <span className="badge bg-white/95 text-emerald-700 shadow"> Lab certified</span>
    }
88         </div>
89         {!available && <div className="absolute inset-0 grid place-items-center bg-black/50 text-lg fo
    nt-bold text-white">{crop.status === "sold_out" ? "Sold out" : "Not available"}</div>}
90     </div>
91     {crop.imagesJson.length > 1 && (
92         <div className="grid grid-cols-4 gap-2">
93             {crop.imagesJson.slice(1, 5).map((src, i) => (
94                 // eslint-disable-next-line @next/next/no-img-element
95                 <img key={i} src={src} alt="" className="aspect-square w-full rounded-xl object-cover" />
96             )}}
97         </div>
98     )}
99 </div>
100
101     {/* Summary + CTA */}
102     <div className="space-y-5">
103         <div>
104             <p className="text-sm font-semibold text-brand-700 dark:text-brand-400">{categoryLabel(crop.ca
    tegory)} · {crop.farmLocation}{province ? `, ${province}` : ""}</p>
105             <h1 className="mt-1 text-3xl font-extrabold tracking-tight sm:text-4xl">{crop.cropName} <span
    className="text-xl font-semibold text-slate-400">{cropUrdu(crop.cropName)}</span></h1>
106             <div className="mt-3 flex flex-wrap items-baseline gap-x-4 gap-y-1">
107                 <p className="text-3xl font-extrabold text-brand-700 dark:text-brand-400">{formatPKR(crop.ba
    sePricePerKg, 2)}<span className="text-base font-medium text-slate-500">/kg</span></p>
108                 <p className="text-slate-500">{perMaund(crop.basePricePerKg)}</p>
109                 {vsMandi !== null && <span className={`badge ${vsMandi <= 0 ? "bg-emerald-100 text-emerald-8
    00 dark:bg-emerald-500/15 dark:text-emerald-300" : "bg-amber-100 text-amber-800 dark:bg-amber-500/15 dark:
    text-amber-300"}`}>{vsMandi <= 0 ? "▼" : "▲"} {Math.abs(vsMandi).toFixed(1)}% vs mandi avg</span>}
110             </div>
111         </div>
112
113         <dl className="grid grid-cols-2 gap-3 sm:grid-cols-4">
114             {[
115                 ["Available", formatKg(crop.totalQuantityKg)],
116                 ["Harvested", formatDate(crop.harvestDate)],
117                 ["Open bids", String(bids.n)],
118                 ["Top bid", bids.hi ? `${formatPKR(Number(bids.hi), 2)}/kg` : "-"],
119             ]}.map(([k, v]) => (
120                 <div key={k} className="rounded-2xl border border-slate-200 p-3 dark:border-white/10">
121                     <dt className="text-[11px] font-semibold tracking-wider text-slate-500 uppercase">{k}</dt>
122                     <dd className="mt-1 text-sm font-bold">{v}</dd>
123                 </div>
124             )}}
125         </dl>
126
127         <BidPanel
128             crop={{ cropId: crop.cropId, cropName: crop.cropName, basePricePerKg: crop.basePricePerKg, tot
    alQuantityKg: crop.totalQuantityKg, available }}
129             viewerRole={user?.role ?? null}
130             isOwner={user?.id === crop.farmerId}
131         />
132
133         {crop.description && <p className="text-sm leading-relaxed text-slate-600 dark:text-slate-300">{
    crop.description}</p>}
134     </div>
135 </div>
136
137 <div className="grid gap-6 lg:grid-cols-3">
138     {/* Specifications */}
139     <section className="card p-6">
140         <h2 className="font-bold">Specifications</h2>
141         <dl className="mt-4 divide-y divide-slate-100 text-sm dark:divide-white/5">
142             {[
143                 ["Crop", crop.cropName],
144                 ["Category", categoryLabel(crop.category)],
145                 ["Declared grade", `Grade ${crop.qualityGrade}`],
146                 ["Quantity", `${formatKg(crop.totalQuantityKg)} (${Math.round(crop.totalQuantityKg / 40).toL
    ocaleString("en-US")} maund)`],
147                 ["Base price", `${formatPKR(crop.basePricePerKg, 2)}/kg`],
148                 ["Harvest date", formatDate(crop.harvestDate)],

```

```

149         ["Farm location", `${crop.farmLocation}${province ? `, ${province}` : ""}`],
150         ["Listed", formatDate(crop.createdAt)],
151         ["Status", <Badge key="s" status={crop.status} />],
152     ].map(([k, v]) => (
153         <div key={String(k)} className="flex justify-between gap-4 py-2.5"><dt className="text-slate
154         -500">{k}</dt><dd className="text-right font-medium">{v}</dd></div>
155     )))
156 </dl>
157 </section>
158
159 {/* Quality reports */}
160 <section className="card p-6">
161     <h2 className="font-bold">Quality inspection</h2>
162     {crop.inspections.length === 0 ? (
163         <p className="mt-4 rounded-xl bg-slate-50 p-4 text-sm text-slate-500 dark:bg-white/5">No lab r
164         eport yet. Every order is inspected before dispatch.</p>
165     ) : (
166         <ul className="mt-4 space-y-3">
167             {crop.inspections.slice(0, 3).map((i) => (
168                 <li key={i.inspectionId} className={`rounded-2xl border p-4 ${i.status === "passed" ? "bor
169                 der-emerald-200 bg-emerald-50/60 dark:border-emerald-900 dark:bg-emerald-950/20" : "border-slate-200 dark:
170                 border-white/10"}`}>
171                     <div className="flex items-center justify-between gap-2">
172                         <GradeBadge grade={i.gradeAssigned} />
173                         <Badge status={i.status} />
174                     </div>
175                     <div className="mt-3 grid grid-cols-2 gap-2 text-xs">
176                         <p><span className="text-slate-500">Moisture</span><br /><b>{i.moistureLevelPercentage
177                         }%</b></p>
178                         <p><span className="text-slate-500">Soil pH</span><br /><b>{i.soilPh ?? "-"}</b></p>
179                     </div>
180                     {i.inspectionNotes && <p className="mt-2 text-xs text-slate-600 dark:text-slate-400">{i.
181                     inspectionNotes}</p>}
182                     <p className="mt-2 text-[11px] text-slate-500">{i.certificateNo ? `Certificate ${i.certi
183                     ficateNo} · ` : ""}{i.verifiedAt ? formatDate(i.verifiedAt) : "Pending lab"}{i.reportAttachment ? " · PDF
184                     on file" : ""}</p>
185                 </li>
186             ))}
187         </ul>
188     )}
189 </section>
190
191 {/* Seller */}
192 <section className="card p-6">
193     <h2 className="font-bold">Seller</h2>
194     <div className="mt-4 flex items-center gap-3">
195         <span className="grid h-12 w-12 place-items-center rounded-full bg-brand-900 text-sm font-bold
196         text-gold-400">{crop.farmerName.split(" ").map((p) => p[0]).slice(0, 2).join("")}</span>
197         <div className="min-w-0">
198             <p className="truncate font-semibold">{crop.farmName ?? crop.farmerName}</p>
199             <p className="text-xs text-slate-500">{crop.farmerName} · {crop.farmerCity} · since {new Dat
200             e(crop.memberSince).getFullYear()}</p>
201         </div>
202     </div>
203     <div className="mt-4 grid grid-cols-2 gap-3">
204         <div className="rounded-2xl bg-brand-50 p-3 dark:bg-brand-500/10">
205             <p className="text-[11px] font-semibold tracking-wider text-brand-800 uppercase dark:text-br
206             and-300">Trust score</p>
207             <p className="mt-1 text-2xl font-extrabold text-brand-900 dark:text-white">{crop.trustScore
208             ? Number(crop.trustScore).toFixed(1) : "New"}<span className="text-sm font-medium text-slate-500">/5</span>
209             </p>
210         </div>
211         <div className="rounded-2xl bg-amber-50 p-3 dark:bg-amber-500/10">
212             <p className="text-[11px] font-semibold tracking-wider text-amber-800 uppercase dark:text-am
213             ber-300">Reviews</p>
214             <p className="mt-1 text-lg font-bold">{crop.ratingCount ? <>Stars value={Number(crop.rating
215             Avg)} /> <span className="text-sm">{Number(crop.ratingAvg).toFixed(1)}</span></> : "-"}</p>
216             <p className="text-[11px] text-slate-500">{crop.ratingCount} verified</p>
217         </div>
218     </div>
219     <ul className="mt-4 space-y-1.5 text-sm">
220         <li className={crop.farmerVerified ? "text-emerald-700 dark:text-emerald-400" : "text-amber-70
221         0"}>{crop.farmerVerified ? "✓ Identity (CNIC) verified" : "Identity verification pending"}</li>
222         <li className={passed.length ? "text-emerald-700 dark:text-emerald-400" : "text-slate-500"}>{p
223         assed.length ? `✓ ${passed.length} passed lab inspection${passed.length > 1 ? "s" : ""}` : "• No lab inspe
224         ctions yet"}</li>

```

```

207         <li className="text-emerald-700 dark:text-emerald-400">✓ Escrow-protected payment</li>
208     </ul>
209     <Link href={` /marketplace?farmerId=${crop.farmerId}`} className="btn-secondary mt-4 w-full">More
    ↵   from this seller</Link>
210 </section>
211 </div>
212
213     { /* Price history */ }
214     <section className="card p-6">
215         <div className="flex flex-wrap items-end justify-between gap-2">
216             <div>
217                 <h2 className="font-bold">Price history: last 30 days</h2>
218                 <p className="text-xs text-slate-500">Government mandi average vs MarketLink deal price vs thi
    ↵   s listing (Rs./kg)</p>
219             </div>
220             <Link href="/mandi-rates" className="text-sm font-semibold text-brand-700 hover:underline dark:t
    ↵   ext-brand-400">All mandi rates -></Link>
221         </div>
222         <div className="mt-4"><CropPriceChart data={series} /></div>
223     </section>
224
225     { /* Reviews */ }
226     <section className="card p-6">
227         <h2 className="font-bold">Buyer reviews for {crop.farmName ?? crop.farmerName}</h2>
228         {reviews.length === 0 ? (
229             <p className="mt-4 text-sm text-slate-500">No reviews yet. Reviews can only be written by buyers
    ↵   after a delivered order.</p>
230         ) : (
231             <ul className="mt-4 grid gap-4 md:grid-cols-2">
232                 {reviews.map((r) => (
233                     <li key={r.reviewId} className="rounded-2xl border border-slate-200 p-4 dark:border-white/10
    ↵   ">
234                     <div className="flex items-center justify-between gap-2">
235                         <Stars value={r.rating} />
236                         <span className="text-xs text-slate-500">{formatDate(r.createdAt)}</span>
237                     </div>
238                     {r.comment && <p className="mt-2 text-sm">&ldquo;{r.comment}&rdquo;</p>}
239                     <p className="mt-2 text-xs text-slate-500">{r.buyerName ?? r.buyerFullName} · bought {r.cr
    ↵   opName} · ✓ Verified purchase</p>
240                     {r.farmerReply && (
241                         <div className="mt-3 rounded-lg border-l-2 border-brand-600 bg-brand-50 px-3 py-2 text-s
    ↵   m dark:bg-brand-950/30">
242                             <p className="text-xs font-semibold text-brand-800 dark:text-brand-300">Seller respons
    ↵   e · {formatDate(r.repliedAt)}</p>
243                             <p className="mt-1 whitespace-pre-wrap">{r.farmerReply}</p>
244                         </div>
245                     )}
246                 </li>
247             )}
248         </ul>
249     )}
250 </section>
251 </div>
252 );
253 }
254

```

src/app/(site)/layout.tsx

```

1  import Link from "next/link";
2  import type { ReactNode } from "react";
3  import { SiteNav } from "@components/nav/site-nav";
4  import { ChatBot } from "@components/ChatBot";
5  import { MandiTicker } from "@components/mandi-ticker";
6  import { getViewer } from "@lib/viewer";
7  import { getTicker } from "@lib/ticker";
8
9  export const dynamic = "force-dynamic";
10
11  export default async function SiteLayout({ children }: { children: ReactNode }) {
12      const [viewer, ticker] = await Promise.all([getViewer(), getTicker()]);
13      return (
14          <div className="flex min-h-screen flex-col">
15              <SiteNav viewer={viewer} />
16              <MandiTicker items={ticker} />

```

```

17     <main className="mx-auto w-full max-w-7xl flex-1 px-4 py-8 sm:px-6">{children}</main>
18     <footer className="border-t border-brand-900/10 dark:border-white/5">
19       <div className="mx-auto flex max-w-7xl flex-col items-center justify-between gap-3 px-4 py-6 text-
  ↳ sm text-slate-500 sm:flex-row sm:px-6">
20         <p>📅 {new Date().getFullYear()} MarketLink Agri-Hub Pakistan</p>
21         <nav className="flex gap-5">
22           <Link href="/marketplace" className="hover:text-brand-700">Marketplace</Link>
23           <Link href="/mandi-rates" className="hover:text-brand-700">Mandi rates</Link>
24           <Link href="/tracking" className="hover:text-brand-700">Tracking</Link>
25           <Link href="/deploy" className="hover:text-brand-700">Developers</Link>
26         </nav>
27       </div>
28     </footer>
29     {!!viewer || viewer.role === "buyer"} && <ChatBot placement="public" />
30   </div>
31   );
32 }
33

```

src/app/(site)/mandi-rates/page.tsx

```

1  import { MandiInsights } from "@components/mandi-insights";
2
3  export const metadata = { title: "Live mandi rates" };
4
5  export default function MandiRatesPage() {
6    return <MandiInsights />;
7  }
8

```

src/app/(site)/marketplace/page.tsx

```

1  import { getCurrentUser } from "@lib/auth";
2  import { MarketplaceClient } from "@components/marketplace/marketplace-client";
3
4  export const metadata = { title: "Marketplace" };
5  export const dynamic = "force-dynamic";
6
7  export default async function MarketplacePage({ searchParams }: { searchParams: Promise<Record<string, str
  ↳ ing | undefined>> }) {
8    const [sp, user] = await Promise.all([searchParams, getCurrentUser()]);
9    return (
10      <MarketplaceClient
11        homeCity={user?.city ?? "Lahore"}
12        viewerRole={user?.role ?? null}
13        initial={{ q: sp.q, category: sp.category, grade: sp.grade, sort: sp.sort, farmerId: sp.farmerId }}
14      />
15    );
16  }
17

```

src/app/(site)/tracking/page.tsx

```

1  import { getCurrentUser } from "@lib/auth";
2  import { listOrders } from "@lib/services/orders";
3  import { TrackingClient } from "@components/tracking/tracking-client";
4
5  export const metadata = { title: "Shipment tracking" };
6  export const dynamic = "force-dynamic";
7
8  export default async function TrackingPage({ searchParams }: { searchParams: Promise<{ no?: string }> }) {
9    const [{ no }, user] = await Promise.all([searchParams, getCurrentUser()]);
10    const mine = user && user.role !== "inspector"
11      ? (await listOrders(user, { active: true, limit: 12 })).map((o) => ({ trackingNumber: o.trackingNumber
  ↳ , cropName: o.cropName, stage: o.deliveryStage }))
12      : [];
13    return <TrackingClient initial={no ?? mine[0]?.trackingNumber ?? ""} mine={mine} />;
14  }
15

```

src/app/admin-login/page.tsx

```
1 import { redirect } from "next/navigation";
2 import { getCurrentUser, homeFor } from "@lib/auth";
3 import { LoginForm } from "@app/login/login-form";
4
5 export const dynamic = "force-dynamic";
6 export const metadata = { title: "Admin login" };
7
8 export default async function AdminLoginPage({ searchParams }: { searchParams: Promise<{ next?: string }>
9 }) {
10   const { next } = await searchParams;
11   const user = await getCurrentUser();
12   if (user) redirect(homeFor(user.role));
13   return <LoginForm initialEmail="ahmed.mustafa@admin.com" initialRole="farmer" startOnRegister={false} ad
14     minMode next={next} />;
15 }
```

src/app/api/account/password/route.ts

```
1 import { eq } from "drizzle-orm";
2 import { db } from "@db";
3 import { users } from "@db/schema";
4 import { ApiError, handle, readJson, str } from "@lib/api";
5 import { hashPassword, requireUser, verifyPassword } from "@lib/auth";
6
7 /** PATCH /api/account/password { currentPassword, newPassword }: any signed-in user */
8 export async function PATCH(req: Request) {
9   return handle(async () => {
10     const me = await requireUser();
11     const b = await readJson(req);
12     const current = str(b, "currentPassword", { required: true, max: 200 });
13     const next = str(b, "newPassword", { required: true, max: 200 });
14     if (next.length < 8) throw new ApiError(422, "New password must be at least 8 characters");
15     if (next === current) throw new ApiError(422, "New password must be different from the current one");
16     const [row] = await db.select({ hash: users.passwordHash }).from(users).where(eq(users.id, me.id)).lim
17     it(1);
18     if (!row || !verifyPassword(current, row.hash)) throw new ApiError(401, "Current password is incorrect
19     ");
20     await db.update(users).set({ passwordHash: hashPassword(next) }).where(eq(users.id, me.id));
21     return { changed: true };
22   });
23 }
```

src/app/api/account/profile/route.ts

```
1 import { eq } from "drizzle-orm";
2 import { db } from "@db";
3 import { users } from "@db/schema";
4 import { ApiError, handle, oneOf, readJson, str } from "@lib/api";
5 import { requireUser } from "@lib/auth";
6 import { CITY_NAMES } from "@lib/constants";
7
8 /** PATCH /api/account/profile { fullName, phone, city, businessName?, cnicId?, address? } */
9 export async function PATCH(req: Request) {
10   return handle(async () => {
11     const me = await requireUser();
12     const b = await readJson(req);
13     const fullName = str(b, "fullName", { required: true, max: 120 });
14     const phone = str(b, "phone", { required: true, max: 30 });
15     const city = oneOf(b, "city", CITY_NAMES, true);
16     const cnicId = str(b, "cnicId", { max: 20 });
17     if (!/^(+92|0092|0)3\d{9}$/.test(phone.replace(/[\s-]/g, ""))) throw new ApiError(422, "Enter a valid
18     Pakistani mobile number (03xx xxxxxxx)");
19     if (cnicId && !/^d{5}-d{7}-d$/.test(cnicId)) throw new ApiError(422, "CNIC must be in format 12345-
20     1234567-1");
21     const [u] = await db
22       .update(users)
23       .set({ fullName, phone, city, cnicId: cnicId ?? null, businessName: str(b, "businessName", { max: 16
24         0 }) ?? null, address: str(b, "address", { max: 500 }) ?? null })
25       .where(eq(users.id, me.id))
26     ;
27   });
28 }
```

```

23     .returning({ id: users.id, fullName: users.fullName, city: users.city });
24     return u;
25   });
26 }
27

```

src/app/api/admin/disputes/[id]/route.ts

```

1  import { handle, oneOf, readJson, str } from "@lib/api";
2  import { requireUser } from "@lib/auth";
3  import { resolveDispute } from "@lib/services/orders";
4
5  type Ctx = { params: Promise<{ id: string }> };
6
7  /** PATCH /api/admin/disputes/:id { outcome: 'investigating' | 'release' | 'refund', resolution } */
8  export async function PATCH(req: Request, { params }: Ctx) {
9    return handle(async () => {
10      const admin = await requireUser(["admin"]);
11      const b = await readJson(req);
12      return resolveDispute(admin, Number((await params).id), oneOf(b, "outcome", ["investigating", "release", "refund"] as const, true!), str(b, "resolution", { max: 1000 }));
13    });
14  }
15

```

src/app/api/admin/disputes/route.ts

```

1  import { handle } from "@lib/api";
2  import { requireUser } from "@lib/auth";
3  import { listDisputes } from "@lib/services/orders";
4
5  export const dynamic = "force-dynamic";
6
7  export async function GET() {
8    return handle(async () => {
9      await requireUser(["admin"]);
10     return listDisputes();
11   });
12 }
13

```

src/app/api/admin/users/[id]/route.ts

```

1  import { ApiError, handle, oneOf, readJson, str } from "@lib/api";
2  import { requireUser } from "@lib/auth";
3  import { changeRole, resetUserPassword, setVerified } from "@lib/services/admin";
4
5  type Ctx = { params: Promise<{ id: string }> };
6
7  /**
8   * PATCH /api/admin/users/:id
9   * { isVerified: boolean }          verify / revoke
10  * { newPassword: "..." }         reset password
11  * { role: "admin"|"farmer"|"buyer" } change role
12  */
13  export async function PATCH(req: Request, { params }: Ctx) {
14    return handle(async () => {
15      const admin = await requireUser(["admin"]);
16      const id = Number((await params).id);
17      const b = await readJson(req);
18      if (typeof b.isVerified === "boolean") return setVerified(id, b.isVerified);
19      const newPassword = str(b, "newPassword", { max: 200 });
20      if (newPassword) return resetUserPassword(id, newPassword);
21      const role = oneOf(b, "role", ["farmer", "buyer", "admin", "inspector"] as const);
22      if (role) return changeRole(admin, id, role);
23      throw new ApiError(422, "Provide isVerified, newPassword or role");
24    });
25  }
26

```

src/app/api/admin/users/route.ts

```
1 import { handle, oneOf, readJson, str } from "@lib/api";
2 import { requireUser } from "@lib/auth";
3 import { CITY_NAMES } from "@lib/constants";
4 import { createUserByAdmin, listUsers } from "@lib/services/admin";
5
6 export const dynamic = "force-dynamic";
7
8 /** GET /api/admin/users?role=farmer|buyer|admin */
9 export async function GET(req: Request) {
10   return handle(async () => {
11     await requireUser(["admin"]);
12     const role = new URL(req.url).searchParams.get("role");
13     return listUsers(role === "farmer" || role === "buyer" || role === "admin" || role === "inspector" ? role : undefined);
14   });
15 }
16
17 /** POST /api/admin/users: admin creates an account (admin / inspector / farmer / buyer) */
18 export async function POST(req: Request) {
19   return handle(async () => {
20     await requireUser(["admin"]);
21     const b = await readJson(req);
22     return createUserByAdmin({
23       fullName: str(b, "fullName", { required: true, max: 120 })!,
24       email: str(b, "email", { required: true, max: 160 })!.toLowerCase(),
25       password: str(b, "password", { required: true, max: 200 })!,
26       phone: str(b, "phone", { required: true, max: 30 })!,
27       role: oneOf(b, "role", ["farmer", "buyer", "admin", "inspector"] as const, true)!,
28       city: oneOf(b, "city", CITY_NAMES, true)!,
29       cnicId: str(b, "cnicId", { max: 20 }),
30       businessName: str(b, "businessName", { max: 160 }),
31     });
32   }, 201);
33 }
34
```

src/app/api/analytics/route.ts

```
1 import { handle } from "@lib/api";
2 import { requireUser } from "@lib/auth";
3 import { adminStats, buyerStats, farmerStats } from "@lib/services/analytics";
4
5 export const dynamic = "force-dynamic";
6
7 /** GET /api/analytics - role-aware dashboard KPIs */
8 export async function GET() {
9   return handle(async () => {
10     const user = await requireUser();
11     if (user.role === "farmer") return farmerStats(user.id);
12     if (user.role === "buyer") return buyerStats(user.id);
13     return adminStats();
14   });
15 }
16
```

src/app/api/assistant/route.ts

```
1 import { ApiError, handle, readJson, str } from "@lib/api";
2 import { getCurrentUser } from "@lib/auth";
3 import { llmReply, type ChatTurn } from "@lib/llm";
4 import { answer } from "@lib/services/assistant";
5 import { latestRates } from "@lib/services/mandi";
6
7 export const dynamic = "force-dynamic";
8
9 const hits = new Map<string, number[]>();
10 function limited(key: string) {
11   const now = Date.now();
12   const recent = (hits.get(key) ?? []).filter((t) => now - t < 60_000);
13   recent.push(now);
14   hits.set(key, recent);
15 }
```

```

15     if (hits.size > 5000) hits.clear();
16     return recent.length > 30;
17 }
18
19 /**
20  * POST /api/assistant { message, history? } → { reply, cards?, suggestions?, link?, engine }
21  * 1) Rule engine retrieves live DB results (listings / dealers / rates / FAQ) → cards
22  * 2) If OPENAI_API_KEY or GEMINI_API_KEY is set, the LLM writes the reply grounded in those results
23  *    (bilingual English / Roman Urdu); otherwise the rule-engine reply is returned.
24  */
25 export async function POST(req: Request) {
26     return handle(async () => {
27         const ip = req.headers.get("x-forwarded-for")?.split(",")[0].trim() || "local";
28         if (limited(ip)) throw new ApiError(429, "Too many messages. Please wait a moment.");
29         const body = await readJson(req);
30         const message = str(body, "message", { required: true, max: 500 });
31         const history: ChatTurn[] = Array.isArray(body.history)
32             ? (body.history as { role?: string; content?: string }[])
33               .filter((t) => (t.role === "user" || t.role === "assistant") && typeof t.content === "string")
34               .slice(-6)
35             .map((t) => ({ role: t.role as "user" | "assistant", content: String(t.content).slice(0, 600) }));
36         : [];
37         const user = await getCurrentUser();
38         const base = await answer(message, { isBuyer: user?.role === "buyer", city: user?.city });
39
40         const rates = await latestRates().catch(() => []);
41         const snapshot = Object.values(
42             rates.reduce<Record<string, { crop: string; avgPerMaund: number; n: number }>>((acc, r) => {
43                 const a = (acc[r.cropName] ??= { crop: r.cropName, avgPerMaund: 0, n: 0 });
44                 a.avgPerMaund += r.avgPricePerKg * 40;
45                 a.n += 1;
46                 return acc;
47             }, {}),
48         ).map((a) => ({ crop: a.crop, avgPerMaund: Math.round(a.avgPerMaund / a.n) }));
49
50         const llm = await llmReply(message, { userCity: user?.city ?? null, signedInAs: user?.role ?? "visitor",
51             draftAnswer: base.reply, cards: base.cards ?? [], mandiSnapshot: snapshot }, history);
52         return { ...base, reply: llm?.text ?? base.reply, engine: llm?.engine ?? "rules" };
53     });
54 }

```

src/app/api/auth/forgot-password/route.ts

```

1  import { ApiError, handle, readJson, str } from "@lib/api";
2  import { requestPasswordReset } from "@lib/reset";
3
4  // Per-IP throttle: 8 requests / 10 minutes
5  const hits = new Map<string, number[]>();
6
7  /**
8   * POST /api/auth/forgot-password { email }
9   * Generates a secure single-use reset token (15-minute expiry) and emails a /reset-password?token=... link.
10  * Always responds with the same message (no account enumeration).
11  */
12  export async function POST(req: Request) {
13      return handle(async () => {
14          const ip = req.headers.get("x-forwarded-for")?.split(",")[0].trim() || "local";
15          const recent = (hits.get(ip) ?? []).filter((t) => Date.now() - t < 10 * 60_000);
16          if (recent.length >= 8) throw new ApiError(429, "Too many reset requests. Try again in a few minutes.");
17          hits.set(ip, [...recent, Date.now()]);
18          const b = await readJson(req);
19          const email = str(b, "email", { required: true, max: 160 });
20          if (!/^[^\s]+@[^\s]+\.[^\s]+$/.test(email)) throw new ApiError(422, "Enter a valid email address");
21          return requestPasswordReset(email, req);
22      });
23  }
24

```

src/app/api/auth/google/callback/route.ts

```
1 import { cookies } from "next/headers";
2 import { OAUTH_COOKIE, exchangeCode, openState, provisionGoogleUser, redirectTo } from "@lib/google";
3
4 export const dynamic = "force-dynamic";
5
6 const fail = (msg: string) => redirectTo(`/login?error=google_failed&msg=${encodeURIComponent(msg)}`);
7
8 /** GET /api/auth/google/callback?code&state → verifies state, exchanges the code, provisions the user, starts a session */
9 export async function GET(req: Request) {
10   const sp = new URL(req.url).searchParams;
11   const store = await cookies();
12   const saved = openState(store.get(OAUTH_COOKIE)?.value);
13   store.delete({ name: OAUTH_COOKIE, path: "/api/auth/google" });
14
15   if (sp.get("error")) return fail(sp.get("error") === "access_denied" ? "Google sign-in was cancelled" : `Google returned: ${sp.get("error")}`);
16   const code = sp.get("code");
17   if (!saved || !code || sp.get("state") !== saved.state) return fail("Your sign-in session expired or was tampered with. Please try again.");
18   try {
19     const profile = await exchangeCode(req, code, saved.verifier);
20     const r = await provisionGoogleUser(profile, saved.role);
21     return redirectTo(r.redirectTo.startsWith("/account") ? r.redirectTo : (saved.next ?? r.redirectTo));
22   } catch (e) {
23     console.error("[google oauth]", e);
24     return fail((e as Error).message || "Google sign-in failed");
25   }
26 }
27
```

src/app/api/auth/google/route.ts

```
1 import { cookies } from "next/headers";
2 import { OAUTH_COOKIE, buildAuthRequest, googleConfigured, googleMockEnabled, redirectTo, safeNext } from "@lib/google";
3
4 export const dynamic = "force-dynamic";
5
6 /** GET /api/auth/google?role=buyer|farmer&next=/path → redirects to Google's consent screen */
7 export async function GET(req: Request) {
8   const sp = new URL(req.url).searchParams;
9   const role = sp.get("role") === "farmer" ? "farmer" : "buyer";
10   const next = safeNext(sp.get("next"));
11   if (!googleConfigured()) {
12     if (googleMockEnabled()) return redirectTo(`/auth/google-test?role=${role}${next ? `&next=${encodeURIComponent(next)}` : ""}`);
13     return redirectTo("/login?error=google_not_configured");
14   }
15   const { url, cookie } = buildAuthRequest(req, role, next);
16   (await cookies()).set(OAUTH_COOKIE, cookie, { httpOnly: true, sameSite: "lax", secure: process.env.COOKIE_SECURE === "true", path: "/api/auth/google", maxAge: 600 });
17   return redirectTo(url);
18 }
19
```

src/app/api/auth/google/test/route.ts

```
1 import { ApiError, handle, oneOf, readJson, str } from "@lib/api";
2 import { googleMockEnabled, mockSub, provisionGoogleUser, safeNext } from "@lib/google";
3
4 /**
5  * POST /api/auth/google/test { email, name, role, next }
6  * Google test mode (only when Google credentials are NOT configured and GOOGLE_OAUTH MOCK=false).
7  * Runs the same provisioning code; can only create/sign into accounts created by test mode.
8  */
9 export async function POST(req: Request) {
10   return handle(async () => {
11     if (!googleMockEnabled()) throw new ApiError(404, "Not available");
12     const b = await readJson(req);
13     const email = str(b, "email", { required: true, max: 160 }).toLowerCase();
14   });
15 }
```

```

14     if (!/^[^@\\s]+@[^@\\s]+\\.^[^@\\s]+$/ .test(email)) throw new ApiError(422, "Enter a valid email address");
15     const name = str(b, "name", { max: 120 }) || email.split("@")[0];
16     const role = oneOf(b, "role", ["buyer", "farmer"] as const) ?? "buyer";
17     const r = await provisionGoogleUser({ sub: mockSub(email), email, email_verified: true, name }, role,
18     true);
19     const next = safeNext(str(b, "next", { max: 300 }));
20     return { created: r.created, redirectTo: r.redirectTo.startsWith("/account") ? r.redirectTo : (next ??
21     r.redirectTo) };
22 }

```

src/app/api/auth/login/route.ts

```

1  import { eq } from "drizzle-orm";
2  import { db } from "@db";
3  import { users } from "@db/schema";
4  import { ApiError, handle, readJson, str } from "@lib/api";
5  import { hashPassword, homeFor, startSession, verifyPassword } from "@lib/auth";
6  import { isLegacyHash } from "@lib/password";
7  import { issueOtp } from "@lib/otp";
8  import { ensureSeeded } from "@lib/seed";
9
10 // Brute-force protection: 10 failed attempts / 15 min per email.
11 const failures = new Map<string, number[]>();
12
13 export async function POST(req: Request) {
14   return handle(async () => {
15     await ensureSeeded();
16     const body = await readJson(req);
17     const email = str(body, "email", { required: true })!.toLowerCase();
18     const password = str(body, "password", { required: true })!;
19     const recent = (failures.get(email) ?? []).filter((t) => Date.now() - t < 15 * 60_000);
20     if (recent.length >= 10) throw new ApiError(429, "Too many failed attempts. Try again in 15 minutes or
21     reset your password.");
22
23     const [u] = await db.select().from(users).where(eq(users.email, email)).limit(1);
24     if (!u || !verifyPassword(password, u.passwordHash)) {
25       failures.set(email, [...recent, Date.now()]);
26       throw new ApiError(401, "Invalid email or password");
27     }
28     failures.delete(email);
29     if (isLegacyHash(u.passwordHash)) await db.update(users).set({ passwordHash: hashPassword(password) })
30     .where(eq(users.id, u.id)); // upgrade to bcrypt
31
32     if (!u.emailVerified) {
33       let devCode: string | undefined;
34       try {
35         devCode = (await issueOtp(u, "verify_email")).devCode;
36       } catch {
37         /* cooldown: an earlier code is still valid */
38       }
39       throw new ApiError(403, "Please verify your email address first. We sent you a code.", { code: "EMAI
40       L_NOT_VERIFIED", email: u.email, devCode });
41     }
42     await startSession(u.id, u.role);
43     return { id: u.id, fullName: u.fullName, role: u.role, redirectTo: homeFor(u.role) };
44   });
45 }

```

src/app/api/auth/logout/route.ts

```

1  import { cookies } from "next/headers";
2  import { handle } from "@lib/api";
3  import { SESSION_COOKIE } from "@lib/auth";
4
5  export async function POST() {
6    return handle(async () => {
7      (await cookies()).delete(SESSION_COOKIE);
8      return { loggedOut: true };
9    });
10 }
11

```

src/app/api/auth/me/route.ts

```
1 import { handle } from "@lib/api";
2 import { requireUser } from "@lib/auth";
3
4 export const dynamic = "force-dynamic";
5
6 export async function GET() {
7   return handle(() => requireUser());
8 }
9
```

src/app/api/auth/register/route.ts

```
1 import { eq } from "drizzle-orm";
2 import { db } from "@db";
3 import { users } from "@db/schema";
4 import { ApiError, handle, oneOf, readJson, str } from "@lib/api";
5 import { hashPassword } from "@lib/auth";
6 import { CITY_NAMES } from "@lib/constants";
7 import { issueOtp } from "@lib/otp";
8
9 /** Self-registration for farmers and buyers. Account must verify its email (6-digit OTP) before first log
  in. */
10 export async function POST(req: Request) {
11   return handle(async () => {
12     const body = await readJson(req);
13     const fullName = str(body, "fullName", { required: true, max: 120 });
14     const email = str(body, "email", { required: true, max: 160 }).toLowerCase();
15     const password = str(body, "password", { required: true, max: 200 });
16     const phone = str(body, "phone", { required: true, max: 30 });
17     const role = oneOf(body, "role", ["farmer", "buyer"] as const, true);
18     const city = oneOf(body, "city", CITY_NAMES, true);
19     const cnicId = str(body, "cnicId", { max: 20 });
20     const businessName = str(body, "businessName", { max: 160 });
21     const address = str(body, "address", { max: 500 });
22     if (!/^^[^@\\s]+@[^@\\s]+\\.^[^@\\s]+$/i.test(email)) throw new ApiError(422, "Invalid email address");
23     if (password.length < 8) throw new ApiError(422, "Password must be at least 8 characters");
24     if (!/^(\+92|0092|0)3\d{9}$/.test(phone.replace(/[/\s-]/g, ""))) throw new ApiError(422, "Enter a valid
  Pakistani mobile number (03xx xxxxxxx)");
25     if (cnicId && !/^d{5}-d{7}-d$/i.test(cnicId)) throw new ApiError(422, "CNIC must be in format 12345-
  1234567-1");
26     const [exists] = await db.select({ id: users.id }).from(users).where(eq(users.email, email)).limit(1);
27     if (exists) throw new ApiError(409, "An account with this email already exists");
28     const [u] = await db
29       .insert(users)
30       .values({ fullName, email, passwordHash: hashPassword(password), phone, role, city, cnicId, business
  Name, address, isVerified: role === "buyer", emailVerified: false, walletBalance: 0 })
31       .returning({ id: users.id, email: users.email, phone: users.phone });
32     const otp = await issueOtp(u, "verify_email");
33     return { needsVerification: true, email: u.email, devCode: otp.devCode };
34   }, 201);
35 }
36
```

src/app/api/auth/reset-password/route.ts

```
1 import { handle, readJson, str } from "@lib/api";
2 import { inspectResetToken, resetPassword } from "@lib/reset";
3
4 export const dynamic = "force-dynamic";
5
6 /** GET /api/auth/reset-password?token=... → { valid, email (masked), expiresAt } */
7 export async function GET(req: Request) {
8   return handle(async () => inspectResetToken(new URL(req.url).searchParams.get("token") ?? ""));
9 }
10
11 /** POST /api/auth/reset-password { token, newPassword } → bcrypt-hashes the new password and clears the t
  oken */
12 export async function POST(req: Request) {
13   return handle(async () => {
14     const b = await readJson(req);
15     return resetPassword(str(b, "token", { required: true, max: 200 }), str(b, "newPassword", { required:
```

```

15     true, max: 200 }));
16   });
17 }
18

```

src/app/api/auth/verify-email/route.ts

```

1  import { eq } from "drizzle-orm";
2  import { db } from "@db";
3  import { users } from "@db/schema";
4  import { ApiError, handle, readJson, str } from "@lib/api";
5  import { homeFor, startSession } from "@lib/auth";
6  import { notify } from "@lib/notify";
7  import { consumeOtp, issueOtp } from "@lib/otp";
8
9  /**
10   * POST /api/auth/verify-email { email, code } → verifies, signs the user in
11   * POST /api/auth/verify-email { email, resend: true } → sends a new code
12   */
13  export async function POST(req: Request) {
14    return handle(async () => {
15      const b = await readJson(req);
16      const email = str(b, "email", { required: true, max: 160 })!.toLowerCase();
17      const [u] = await db.select().from(users).where(eq(users.email, email)).limit(1);
18      if (!u) throw new ApiError(400, "Code expired or not found. Request a new one.");
19      if (b.resend === true) {
20        if (u.emailVerified) return { alreadyVerified: true };
21        return issueOtp(u, "verify_email");
22      }
23      const code = str(b, "code", { required: true, max: 10 })!;
24      if (!u.emailVerified) {
25        await consumeOtp(u.id, "verify_email", code);
26        await db.update(users).set({ emailVerified: true }).where(eq(users.id, u.id));
27        await notify(u.id, {
28          type: "welcome",
29          title: "Welcome to MarketLink",
30          body: u.role === "farmer" ? "Your email is verified. List your first crop. It goes live after a quick identity check." : "Your email is verified. Add funds to your wallet and start sourcing directly from farms.",
31          link: homeFor(u.role),
32          email: true,
33        });
34      }
35      await startSession(u.id, u.role);
36      return { verified: true, redirectTo: homeFor(u.role) };
37    });
38  }
39

```

src/app/api/bids/[id]/route.ts

```

1  import { handle, num, oneOf, readJson, str } from "@lib/api";
2  import { requireUser } from "@lib/auth";
3  import { actOnBid } from "@lib/services/bids";
4
5  type Ctx = { params: Promise<{ id: string }> };
6
7  /** PATCH /api/bids/:id { action, counterPrice?, bidPricePerKg?, note? } */
8  export async function PATCH(req: Request, { params }: Ctx) {
9    return handle(async () => {
10      const user = await requireUser(["farmer", "buyer"]);
11      const b = await readJson(req);
12      const action = oneOf(b, "action", ["accept", "reject", "counter", "accept_counter", "reject_counter", "revise", "withdraw"] as const, true)!;
13      return actOnBid(user, Number((await params).id), action, {
14        counterPrice: num(b, "counterPrice", { min: 0.5 }),
15        bidPricePerKg: num(b, "bidPricePerKg", { min: 0.5 }),
16        note: str(b, "note", { max: 500 }),
17      });
18    });
19  }
20

```

src/app/api/bids/route.ts

```
1 import { handle, num, readJson, str } from "@lib/api";
2 import { requireUser } from "@lib/auth";
3 import { createBid, listBids } from "@lib/services/bids";
4
5 export const dynamic = "force-dynamic";
6
7 /** GET /api/bids?status=&cropId= – scoped to the caller's role */
8 export async function GET(req: Request) {
9   return handle(async () => {
10     const user = await requireUser();
11     const sp = new URL(req.url).searchParams;
12     return listBids(user, { status: sp.get("status") || undefined, cropId: sp.get("cropId") ? Number(sp.get("cropId")) : undefined });
13   });
14 }
15
16 /** POST /api/bids – buyer submits a bulk purchase offer / RFQ */
17 export async function POST(req: Request) {
18   return handle(async () => {
19     const user = await requireUser(["buyer"]);
20     const b = await readJson(req);
21     return createBid(user, {
22       cropId: num(b, "cropId", { required: true, min: 1 })!,
23       bidPricePerKg: num(b, "bidPricePerKg", { required: true, min: 0.5 })!,
24       bidQuantityKg: num(b, "bidQuantityKg", { required: true, min: 1 })!,
25       targetDeliveryDate: str(b, "targetDeliveryDate"),
26       message: str(b, "message", { max: 500 }),
27     });
28   }, 201);
29 }
30
```

src/app/api/crops/[id]/route.ts

```
1 import { handle, num, oneOf, readJson, str } from "@lib/api";
2 import { requireUser } from "@lib/auth";
3 import { CITY_NAMES } from "@lib/constants";
4 import { deleteCrop, getCrop, updateCrop } from "@lib/services/crops";
5
6 type Ctx = { params: Promise<{ id: string }> };
7 export const dynamic = "force-dynamic";
8
9 export async function GET(_req: Request, { params }: Ctx) {
10   return handle(async () => {
11     return getCrop(Number((await params).id));
12   });
13 }
14
15 export async function PATCH(req: Request, { params }: Ctx) {
16   return handle(async () => {
17     const user = await requireUser(["farmer", "admin"]);
18     const b = await readJson(req);
19     return updateCrop(user, Number((await params).id), {
20       cropName: str(b, "cropName", { max: 80 }),
21       category: oneOf(b, "category", ["grains", "vegetables", "fruits", "cash_crops"] as const),
22       qualityGrade: oneOf(b, "qualityGrade", ["A", "B", "C"] as const),
23       totalQuantityKg: num(b, "totalQuantityKg", { min: 0 }),
24       basePricePerKg: num(b, "basePricePerKg", { min: 0.5 }),
25       harvestDate: str(b, "harvestDate"),
26       farmLocation: oneOf(b, "farmLocation", CITY_NAMES),
27       description: str(b, "description", { max: 1000 }),
28       imagesJson: Array.isArray(b.imagesJson) ? (b.imagesJson as string[]) : undefined,
29       status: oneOf(b, "status", ["active", "archived"] as const),
30     });
31   });
32 }
33
34 export async function DELETE(_req: Request, { params }: Ctx) {
35   return handle(async () => {
36     const user = await requireUser(["farmer", "admin"]);
37     return deleteCrop(user, Number((await params).id));
38   });
39 }

```

```
39 }
40
```

src/app/api/crops/route.ts

```
1 import { handle, num, oneOf, readJson, str } from "@lib/api";
2 import { getCurrentUser, requireUser } from "@lib/auth";
3 import { CITY_NAMES } from "@lib/constants";
4 import { createCrop, listCrops } from "@lib/services/crops";
5
6 export const dynamic = "force-dynamic";
7 const CATS = ["grains", "vegetables", "fruits", "cash_crops"] as const;
8 const GRADES = ["A", "B", "C"] as const;
9
10 /** GET /api/crops?category=&grade=&minPrice=&maxPrice=&q=&originCity=&radiusKm=&sort=&mine=1 */
11 export async function GET(req: Request) {
12   return handle(async () => {
13     const user = await getCurrentUser(); // public catalogue; "mine" requires a farmer session
14     const sp = Object.fromEntries(new URL(req.url).searchParams.entries());
15     const mine = sp.mine === "1" && user?.role === "farmer";
16     return listCrops({
17       category: oneOf(sp, "category", CATS),
18       grade: oneOf(sp, "grade", GRADES),
19       minPrice: num(sp, "minPrice", { min: 0 }),
20       maxPrice: num(sp, "maxPrice", { min: 0 }),
21       q: str(sp, "q", { max: 80 }),
22       originCity: oneOf(sp, "originCity", CITY_NAMES),
23       radiusKm: num(sp, "radiusKm", { min: 1 }),
24       sort: oneOf(sp, "sort", ["newest", "price_asc", "price_desc", "distance"] as const),
25       farmerId: mine ? user!.id : sp.farmerId ? Number(sp.farmerId) : undefined,
26       minQty: num(sp, "minQty", { min: 0 }),
27       province: str(sp, "province", { max: 40 }),
28       status: mine || user?.role === "admin" || user?.role === "inspector" ? "all" : "active",
29     });
30   });
31 }
32
33 /** POST /api/crops - farmer creates a produce listing */
34 export async function POST(req: Request) {
35   return handle(async () => {
36     const user = await requireUser(["farmer"]);
37     const b = await readJson(req);
38     return createCrop(user, {
39       cropName: str(b, "cropName", { required: true, max: 80 })!,
40       category: oneOf(b, "category", CATS, true)!,
41       qualityGrade: oneOf(b, "qualityGrade", GRADES, true)!,
42       totalQuantityKg: num(b, "totalQuantityKg", { required: true, min: 1, max: 10_000_000 })!,
43       basePricePerKg: num(b, "basePricePerKg", { required: true, min: 0.5, max: 100_000 })!,
44       harvestDate: str(b, "harvestDate", { required: true })!,
45       farmLocation: oneOf(b, "farmLocation", CITY_NAMES, true)!,
46       description: str(b, "description", { max: 1000 }),
47       imagesJson: Array.isArray(b.imagesJson) ? (b.imagesJson as string[]) : [],
48     });
49   }, 201);
50 }
51
```

src/app/api/db/query/route.ts

```
1 import { sql } from "drizzle-orm";
2 import { db } from "@db";
3 import { dbAuditLog } from "@db/schema";
4 import { ApiError, handle, oneOf, readJson, str } from "@lib/api";
5 import { requireUser } from "@lib/auth";
6
7 export const dynamic = "force-dynamic";
8 const MAX_ROWS = 500;
9
10 /** Strip string literals, quoted identifiers and comments so keyword / semicolon checks aren't fooled. */
11 function normalise(q: string) {
12   return q.replace(/'(?![^']*')'/g, "'").replace(/"(?:[^"]|")"/g, "\"").replace(/--[\n]*/g, "").replace(/\/\s*\S*\?\/g, "");
13 }
14
15 const BLOCKED = /\b(drop\s+database|create\s+database|alter\s+system|copy\b\s*\S*\bprogram|pg_read_file|p
```

```

15  g_read_binary_file|pg_ls_dir|lo_import|lo_export|dblink|pg_terminate_backend|pg_cancel_backend|create\s+(o
16  r\s+replace\s+)?function|create\s+extension|set\s+role|reset\s+role|grant|revoke)\b/i;
17
18  /**
19   * POST /api/db/query { sql, mode: "read" | "write", confirm? }
20   * Admin-only SQL console (also enforced by proxy.ts).
21   * • read → runs inside a READ ONLY transaction with an 8s statement timeout, max 500 rows returned
22   * • write → requires confirm: true; blocked dangerous statements; always audited in db_audit_log
23   */
24  export async function POST(req: Request) {
25    return handle(async () => {
26      const admin = await requireUser(["admin"]);
27      const b = await readJson(req);
28      const query = str(b, "sql", { required: true, max: 20000 })!.trim().replace(/;\s*$/, "");
29      const mode = oneOf(b, "mode", ["read", "write"] as const) ?? "read";
30      const norm = normalise(query);
31      if (norm.includes(";")) throw new ApiError(422, "Run one statement at a time");
32      if (BLOCKED.test(norm)) throw new ApiError(403, "This statement is blocked in the web console for safe
33      ty");
34      if (mode === "read" && WRITE.test(norm)) throw new ApiError(422, "This is a write statement. Switch to
35      write mode and confirm to run it.");
36      if (mode === "write" && b.confirm !== true) throw new ApiError(428, "Write queries require confirmatio
37      n");
38
39      const started = Date.now();
40      let error: string | null = null;
41      try {
42        const res = (await db.transaction(async (tx) => {
43          if (mode === "read") await tx.execute(sql`SET TRANSACTION READ ONLY`);
44          await tx.execute(sql.raw(`SET LOCAL statement_timeout = '8s'`)).catch(() => undefined);
45          return tx.execute(sql.raw(query));
46        })) as unknown as { rows?: Record<string, unknown>[]; fields?: { name: string }[]; rowCount?: number
47        | null; affectedRows?: number; command?: string };
48        const rows = res.rows ?? [];
49        const columns = res.fields?.map((f) => f.name) ?? (rows[0] ? Object.keys(rows[0]) : []);
50        const affected = res.rowCount ?? res.affectedRows ?? rows.length;
51        const out = {
52          columns,
53          rows: rows.slice(0, MAX_ROWS).map((r) => columns.map((c) => {
54            const v = r[c];
55            if (v instanceof Date) return v.toISOString();
56            if (typeof v === "string" && v.length > 400) return v.slice(0, 400) + "...";
57            if (v !== null && typeof v === "object") return JSON.stringify(v).slice(0, 400);
58            return v as string | number | boolean | null;
59          })),
60          rowCount: rows.length,
61          affected,
62          truncated: rows.length > MAX_ROWS,
63          command: res.command ?? (WRITE.exec(norm)?.[1] ?? "SELECT").toUpperCase(),
64          durationMs: Date.now() - started,
65        };
66        return out;
67      } catch (e) {
68        const cause = (e as { cause?: { message?: string } }).cause?.message;
69        error = cause ?? (e as Error).message.replace(/^Failed query: [\s\S]*$/, "Query failed");
70        throw new ApiError(400, error.replace(/^error: \s*/i, ""));
71      } finally {
72        await db.insert(dbAuditLog).values({ adminId: admin.id, mode, query: query.slice(0, 5000), durationM
73        s: Date.now() - started, error }).catch(() => undefined);
74      }
75    });
76  }

```

src/app/api/db/schema/route.ts

```

1  import { desc, eq } from "drizzle-orm";
2  import { sql } from "drizzle-orm";
3  import { db } from "@db";
4  import { dbAuditLog, users } from "@db/schema";
5  import { handle } from "@lib/api";
6  import { requireUser } from "@lib/auth";
7

```

```

8   export const dynamic = "force-dynamic";
9
10  /** GET /api/db/schema: tables, columns, keys, indexes, row counts + recent audit log (admin only) */
11  export async function GET() {
12    return handle(async () => {
13      await requireUser(["admin"]);
14      const cols = (await db.execute(sql`
15        select c.table_name, c.column_name, c.data_type, c.udt_name, c.is_nullable, c.column_default, c.ordinal_position
16        from information_schema.columns c
17        join information_schema.tables t on t.table_name = c.table_name and t.table_schema = c.table_schema
18        where c.table_schema = 'public' and t.table_type = 'BASE TABLE'
19        order by c.table_name, c.ordinal_position`)).rows as { table_name: string; column_name: string; data
20        _type: string; udt_name: string; is_nullable: string; column_default: string | null }[];
21      const keys = (await db.execute(sql`
22        select tc.table_name, kcu.column_name, tc.constraint_type, ccu.table_name as ref_table, ccu.column_n
23        ame as ref_column
24        from information_schema.table_constraints tc
25        join information_schema.key_column_usage kcu on kcu.constraint_name = tc.constraint_name and kcu.tab
26        le_schema = tc.table_schema
27        left join information_schema.constraint_column_usage ccu on ccu.constraint_name = tc.constraint_name
28        and tc.constraint_type = 'FOREIGN KEY'
29        where tc.table_schema = 'public' and tc.constraint_type in ('PRIMARY KEY', 'FOREIGN KEY'))).rows as
30        { table_name: string; column_name: string; constraint_type: string; ref_table: string | null; ref_column:
31        string | null }[];
32      const idx = (await db.execute(sql`select tablename, indexname, indexdef from pg_indexes where schemana
33        me = 'public' order by tablename, indexname`)).rows as { tablename: string; indexname: string; indexdef: s
34        tring }[];
35
36      const tables = [...new Set(cols.map((c) => c.table_name))];
37      const counts: Record<string, number> = {};
38      for (const t of tables) {
39        const r = await db.execute(sql.raw(`select count(*)::int as n from "${t.replace(/"/g, '"')}");`));
40        counts[t] = Number((r.rows[0] as { n: number }).n);
41      }
42
43      const audit = await db
44        .select({ id: dbAuditLog.auditId, mode: dbAuditLog.mode, query: dbAuditLog.query, durationMs: dbAudi
45        tLog.durationMs, error: dbAuditLog.error, at: dbAuditLog.createdAt, admin: users.fullName })
46        .from(dbAuditLog)
47        .leftJoin(users, eq(users.id, dbAuditLog.adminId))
48        .orderBy(desc(dbAuditLog.createdAt))
49        .limit(25);
50
51      return {
52        engine: process.env.DATABASE_URL?.startsWith("postgres") ? "PostgreSQL" : "PostgreSQL (embedded PGli
53        te)",
54        tables: tables.map((t) => ({
55          name: t,
56          rows: counts[t],
57          columns: cols.filter((c) => c.table_name === t).map((c) => {
58            const pk = keys.some((k) => k.table_name === t && k.column_name === c.column_name && k.constrain
59            t_type === "PRIMARY KEY");
60            const fk = keys.find((k) => k.table_name === t && k.column_name === c.column_name && k.constrain
61            t_type === "FOREIGN KEY");
62            return { name: c.column_name, type: c.data_type === "USER-DEFINED" ? c.udt_name : c.data_type, n
63            ullable: c.is_nullable === "YES", default: c.column_default, pk, fk: fk ? `${fk.ref_table}.${fk.ref_column
64            }` : null };
65          })),
66          indexes: idx.filter((i) => i.tablename === t).map((i) => ({ name: i.indexname, def: i.indexdef })),
67        })),
68        audit,
69      };
70    });
71  }
72

```

src/app/api/health/route.ts

```

1   import { db } from "@db";
2   import { sql } from "drizzle-orm";
3
4   export const dynamic = "force-dynamic";
5
6   export async function GET() {

```

```

7     try {
8         await db.execute(sql`select 1`);
9         return Response.json({ ok: true });
10    } catch {
11        return Response.json({ ok: false }, { status: 500 });
12    }
13 }
14

```

src/app/api/inspections/[id]/report/route.ts

```

1  import { fail } from "@lib/api";
2  import { requireUser } from "@lib/auth";
3  import { getInspectionReport } from "@lib/services/admin";
4
5  type Ctx = { params: Promise<{ id: string }> };
6
7  /** GET /api/inspections/:id/report – streams the uploaded report file */
8  export async function GET(_req: Request, { params }: Ctx) {
9      try {
10         await requireUser(["inspector", "admin"]);
11         const dataUrl = await getInspectionReport(Number((await params).id));
12         const m = /^data:([^\;]+);base64,(.*)$/i.exec(dataUrl);
13         if (!m) return new Response(dataUrl, { headers: { "content-type": "text/plain" } });
14         return new Response(Buffer.from(m[2], "base64"), { headers: { "content-type": m[1] } });
15     } catch (e) {
16         return fail(e);
17     }
18 }
19

```

src/app/api/inspections/route.ts

```

1  import { handle, num, oneOf, readJson, str } from "@lib/api";
2  import { requireUser } from "@lib/auth";
3  import { createInspection, listInspections } from "@lib/services/admin";
4
5  export const dynamic = "force-dynamic";
6
7  export async function GET() {
8      return handle(async () => {
9          await requireUser(["inspector", "admin"]);
10         return listInspections();
11     });
12 }
13
14 /** POST /api/inspections – quality inspector uploads soil/crop inspection report */
15 export async function POST(req: Request) {
16     return handle(async () => {
17         const user = await requireUser(["inspector", "admin"]);
18         const b = await readJson(req);
19         return createInspection(user, {
20             cropId: num(b, "cropId", { required: true, min: 1 })!,
21             gradeAssigned: oneOf(b, "gradeAssigned", ["A", "B", "C"] as const, true)!,
22             moistureLevelPercentage: num(b, "moistureLevelPercentage", { required: true, min: 0, max: 100 })!,
23             soilPh: num(b, "soilPh", { min: 0, max: 14 }),
24             inspectionNotes: str(b, "inspectionNotes", { max: 2000 }),
25             reportAttachment: typeof b.reportAttachment === "string" ? b.reportAttachment : undefined,
26             reportFileName: str(b, "reportFileName", { max: 200 }),
27             status: oneOf(b, "status", ["pending", "passed", "failed"] as const, true)!,
28         });
29     }, 201);
30 }
31

```

src/app/api/mandi-rates/route.ts

```

1  import { handle, num, oneOf, readJson, str } from "@lib/api";
2  import { requireUser } from "@lib/auth";
3  import { MANDI_MARKETS } from "@lib/constants";
4  import { latestRates, platformComparison, rateTrend, upsertRate } from "@lib/services/mandi";
5
6  export const dynamic = "force-dynamic";
7

```

```

8  /**
9   * GET /api/mandi-rates?view=latest|trend|compare&crop=&market=&days=
10  * latest - latest rate per crop x market with day-on-day change
11  * trend - daily avg per market for one crop (chart series)
12  * compare - mandi avg vs platform ask vs platform deal price
13  */
14  export async function GET(req: Request) {
15    return handle(async () => {
16      const sp = new URL(req.url).searchParams; // public market data
17      const view = sp.get("view") ?? "latest";
18      if (view === "trend") return rateTrend(sp.get("crop") || "Wheat", Math.min(60, Number(sp.get("days") ||
19  | 14)));
20      if (view === "compare") return platformComparison();
21      return latestRates({ crop: sp.get("crop") || undefined, market: sp.get("market") || undefined });
22    });
23  }
24  /** POST /api/mandi-rates - admin manual daily rate entry (upsert by crop+market+date) */
25  export async function POST(req: Request) {
26    return handle(async () => {
27      await requireUser(["admin"]);
28      const b = await readJson(req);
29      return upsertRate({
30        cropName: str(b, "cropName", { required: true, max: 80 })!,
31        marketLocation: oneOf(b, "marketLocation", MANDI_MARKETS, true)!,
32        minPricePerKg: num(b, "minPricePerKg", { required: true, min: 0.1 })!,
33        maxPricePerKg: num(b, "maxPricePerKg", { required: true, min: 0.1 })!,
34        avgPricePerKg: num(b, "avgPricePerKg", { min: 0.1 }),
35        dateUpdated: str(b, "dateUpdated"),
36      });
37    }, 201);
38  }
39

```

src/app/api/mandi-rates/sync/route.ts

```

1  import { handle } from "@lib/api";
2  import { requireUser } from "@lib/auth";
3  import { syncMockFeed } from "@lib/services/mandi";
4
5  /** POST /api/mandi-rates/sync - pull today's rates from the (mock) government mandi feed */
6  export async function POST() {
7    return handle(async () => {
8      await requireUser(["admin"]);
9      return syncMockFeed();
10    });
11  }
12

```

src/app/api/notifications/route.ts

```

1  import { and, desc, eq, sql } from "drizzle-orm";
2  import { db } from "@db";
3  import { notifications } from "@db/schema";
4  import { handle, readJson } from "@lib/api";
5  import { requireUser } from "@lib/auth";
6
7  export const dynamic = "force-dynamic";
8
9  /** GET: latest 30 notifications + unread count */
10 export async function GET() {
11   return handle(async () => {
12     const u = await requireUser();
13     const items = await db.select().from(notifications).where(eq(notifications.userId, u.id)).orderBy(desc
14 | (notifications.createdAt)).limit(30);
15     const [{ n }] = await db.select({ n: sql<number>`count(*)`.mapWith(Number) }).from(notifications).wher
16 | e(and(eq(notifications.userId, u.id), eq(notifications.isRead, false)));
17     return { unread: n, items };
18   });
19 }
20
21 /** PATCH { ids?: number[] }: mark given (or all) notifications as read */
22 export async function PATCH(req: Request) {
23   return handle(async () => {
24

```

```

22     const u = await requireUser();
23     const b = await readJson(req).catch(() => ({})) as Record<string, unknown>;
24     const ids = Array.isArray(b.ids) ? (b.ids as unknown[]).map(Number).filter(Number.isFinite) : null;
25     await db
26       .update(notifications)
27       .set({ isRead: true })
28       .where(ids && ids.length ? and(eq(notifications.userId, u.id), sql`${notifications.notificationId} i
  ↵ n (${sql.join(ids.map((i) => sql`${i}`), sql`, `))}`) : eq(notifications.userId, u.id));
29     return { ok: true };
30   });
31 }
32

```

src/app/api/orders/[id]/dispute/route.ts

```

1  import { handle, readJson, str } from "@lib/api";
2  import { requireUser } from "@lib/auth";
3  import { raiseDispute } from "@lib/services/orders";
4
5  type Ctx = { params: Promise<{ id: string }> };
6
7  /** POST /api/orders/:id/dispute { reason } – freezes escrow pending admin review */
8  export async function POST(req: Request, { params }: Ctx) {
9    return handle(async () => {
10      const user = await requireUser(["buyer", "farmer"]);
11      const b = await readJson(req);
12      return raiseDispute(user, Number((await params).id), str(b, "reason", { required: true, max: 1000 }!))
  ↵ ;
13    }, 201);
14  }
15

```

src/app/api/orders/[id]/gps/route.ts

```

1  import { handle, num, readJson, str } from "@lib/api";
2  import { requireUser } from "@lib/auth";
3  import { recordGpsPing } from "@lib/services/tracking";
4
5  type Ctx = { params: Promise<{ id: string }> };
6
7  /** POST { lat, lng, label? }: GPS ping from the driver's phone / tracker while the shipment is moving */
8  export async function POST(req: Request, { params }: Ctx) {
9    return handle(async () => {
10      const user = await requireUser(["farmer", "admin"]);
11      const b = await readJson(req);
12      return recordGpsPing(user, Number((await params).id), num(b, "lat", { required: true }!), num(b, "lng"
  ↵ , { required: true }!), str(b, "label", { max: 120 }));
13    }, 201);
14  }
15

```

src/app/api/orders/[id]/route.ts

```

1  import { ApiError, handle, num, oneOf, readJson, str } from "@lib/api";
2  import { requireUser } from "@lib/auth";
3  import { advanceOrder, listOrders, lockEscrow, updateTransitLocation } from "@lib/services/orders";
4
5  type Ctx = { params: Promise<{ id: string }> };
6  export const dynamic = "force-dynamic";
7
8  export async function GET(_req: Request, { params }: Ctx) {
9    return handle(async () => {
10      const user = await requireUser();
11      const id = Number((await params).id);
12      const order = (await listOrders(user, { limit: 1000 })).find((o) => o.orderId === id);
13      if (!order) throw new ApiError(404, "Order not found");
14      return order;
15    });
16  }
17
18  /** PATCH /api/orders/:id { action: 'lock_escrow' | 'advance' | 'update_location', location?, note?, grad
  ↵ e?, moisture? } */
19  export async function PATCH(req: Request, { params }: Ctx) {

```

```

20     return handle(async () => {
21         const user = await requireUser();
22         const id = Number((await params).id);
23         const b = await readJson(req);
24         const action = oneOf(b, "action", ["lock_escrow", "advance", "update_location"] as const, true!);
25         if (action === "lock_escrow") return lockEscrow(user, id);
26         if (action === "update_location") return updateTransitLocation(user, id, str(b, "location", { required
27 : true, max: 120 }));
28         return advanceOrder(user, id, {
29             location: str(b, "location", { max: 120 }),
30             note: str(b, "note", { max: 500 }),
31             grade: oneOf(b, "grade", ["A", "B", "C"] as const),
32             moisture: num(b, "moisture", { min: 0, max: 100 }),
33         });
34     });
35 }

```

src/app/api/orders/route.ts

```

1  import { handle } from "@lib/api";
2  import { requireUser } from "@lib/auth";
3  import { listOrders } from "@lib/services/orders";
4
5  export const dynamic = "force-dynamic";
6
7  /** GET /api/orders?active=1 - orders with logistics events, scoped by role */
8  export async function GET(req: Request) {
9      return handle(async () => {
10         const user = await requireUser();
11         const active = new URL(req.url).searchParams.get("active") === "1";
12         return listOrders(user, { active });
13     });
14 }
15

```

src/app/api/payments/[reference]/route.ts

```

1  import { handle } from "@lib/api";
2  import { requireUser } from "@lib/auth";
3  import { getIntent } from "@lib/services/payments";
4
5  export const dynamic = "force-dynamic";
6  type Ctx = { params: Promise<{ reference: string }> };
7
8  /** GET /api/payments/:reference: poll payment status after returning from the gateway */
9  export async function GET(_req: Request, { params }: Ctx) {
10     return handle(async () => getIntent(await requireUser(), (await params).reference));
11 }
12

```

src/app/api/payments/callback/easypaisa/route.ts

```

1  import { absoluteUrl } from "@lib/base-url";
2  import { completeIntent, easypaisaInquire } from "@lib/services/payments";
3
4  export const dynamic = "force-dynamic";
5
6  /**
7   * Easypaisa redirects the customer here (GET with orderRefNumber & status, or POST with auth_token on step 1).
8   * The result is confirmed server-to-server via the Inquire Transaction API before the wallet is credited.
9   */
10 async function handleCallback(req: Request, params: URLSearchParams) {
11     const ref = params.get("orderRefNumber") || params.get("orderRefNum") || "";
12     const authToken = params.get("auth_token");
13     if (authToken) {
14         // Step 1 of the hosted flow: forward the token to Easypaisa's confirm page
15         const confirm = process.env.PAYMENTS_MODE === "live" ? "https://easypay.easypaisa.com.pk/easypay/Confirm.jsf" : "https://easypaystg.easypaisa.com.pk/easypay/Confirm.jsf";
16         const postBackURL = absoluteUrl(req, "/api/payments/callback/easypaisa");
17         return Response.redirect(`${confirm}?auth_token=${encodeURIComponent(authToken)}&postBackURL=${encodeURIComponent(postBackURL)}`, 303);
18     }
19 }

```

```

18     }
19     if (ref) {
20         try {
21             const paid = await easypaisaInquire(ref);
22             if (paid) await completeIntent(ref, true, ref, { status: params.get("status") });
23             else if (params.get("status") && params.get("status") !== "0000") await completeIntent(ref, false, r
24         ef, { status: params.get("status"), desc: params.get("desc") });
25         } catch (e) {
26             console.error("[easypaisa callback]", e);
27         }
28     }
29     return Response.redirect(absoluteUrl(req, `/wallet?payment=${encodeURIComponent(ref)}`), 303);
30 }
31 export async function GET(req: Request) {
32     return handleCallback(req, new URL(req.url).searchParams);
33 }
34 export async function POST(req: Request) {
35     const form = await req.formData();
36     return handleCallback(req, new URLSearchParams([...form.entries()].map(([k, v]) => [k, String(v)])));
37 }
38

```

src/app/api/payments/callback/jazzcash/route.ts

```

1  import { timingSafeEqual } from "crypto";
2  import { absoluteUrl } from "@lib/base-url";
3  import { completeIntent, jazzcashHash } from "@lib/services/payments";
4
5  export const dynamic = "force-dynamic";
6
7  /** JazzCash posts the transaction result (form-encoded) to pp_ReturnURL. The secure hash is verified befo
8  re settling. */
9  export async function POST(req: Request) {
10     const form = await req.formData();
11     const fields = Object.fromEntries([...form.entries()].map(([k, v]) => [k, String(v)]));
12     // Gateway return URLs must point at the host the buyer used, never at the server bind address.
13     const ref = fields.pp_TxnRefNo ?? "";
14     const expected = process.env.JAZZCASH_INTEGRITY_SALT ? jazzcashHash(fields) : "";
15     const got = (fields.pp_SecureHash ?? "").toUpperCase();
16     if (!expected || expected.length !== got.length || !timingSafeEqual(Buffer.from(expected), Buffer.from(g
17     ot))) {
18         return Response.redirect(absoluteUrl(req, `/wallet?payment=${encodeURIComponent(ref)}&error=signature`
19         ), 303);
20     }
21     try {
22         await completeIntent(ref, fields.pp_ResponseCode === "000", fields.pp_RetreivalReferenceNo || fields.p
23         p_TxnRefNo, { code: fields.pp_ResponseCode, message: fields.pp_ResponseMessage });
24     } catch (e) {
25         console.error("[jazzcash callback]", e);
26     }
27     return Response.redirect(absoluteUrl(req, `/wallet?payment=${encodeURIComponent(ref)}`), 303);
28 }
29

```

src/app/api/payments/checkout/route.ts

```

1  import { handle, num, oneOf, readJson } from "@lib/api";
2  import { requireUser } from "@lib/auth";
3  import { createCheckout, providerStatus } from "@lib/services/payments";
4
5  export const dynamic = "force-dynamic";
6
7  /** GET: which gateways are enabled on this server */
8  export async function GET() {
9     return handle(async () => {
10         await requireUser(["buyer", "farmer"]);
11         return providerStatus();
12     });
13 }
14
15 /** POST { provider, amount }: creates a payment intent and returns the gateway redirect URL / auto-submit
16 form */
17 export async function POST(req: Request) {
18

```

```

17     return handle(async () => {
18         const user = await requireUser(["buyer"]);
19         const b = await readJson(req);
20         return createCheckout(user, oneOf(b, "provider", ["stripe", "jazzcash", "easypaisa", "sandbox"] as con
21 st, true)!, num(b, "amount", { required: true, min: 1000, max: 50_000_000 }!), req);
22     }
23

```

src/app/api/payments/sandbox/[reference]/route.ts

```

1  import { ApiError, handle, oneOf, readJson } from "@lib/api";
2  import { requireUser } from "@lib/auth";
3  import { completeIntent, getIntent, providerStatus, sandboxSignature } from "@lib/services/payments";
4
5  type Ctx = { params: Promise<{ reference: string }> };
6
7  /**
8   * Test gateway: simulates the provider calling our callback.
9   * The callback is HMAC-signed server-side and goes through the same completeIntent() path as real gateway
10 s.
11 * Disable in production with PAYMENTS_SANDBOX=false.
12 */
13 export async function POST(req: Request, { params }: Ctx) {
14     return handle(async () => {
15         if (!providerStatus().sandbox.enabled) throw new ApiError(403, "Test gateway is disabled on this serve
16 r");
17         const user = await requireUser(["buyer"]);
18         const { reference } = await params;
19         const intent = await getIntent(user, reference);
20         if (intent.provider !== "sandbox") throw new ApiError(400, "Not a test-gateway payment");
21         const outcome = oneOf(await readJson(req), "outcome", ["success", "failed"] as const, true!);
22         const done = await completeIntent(reference, outcome === "success", `SBX-${Date.now()}`, { outcome, si
23 gnature: sandboxSignature(reference, outcome) });
24         return { reference, status: done.status };
25     });
26 }
27

```

src/app/api/payments/webhook/stripe/route.ts

```

1  import { completeIntent, verifyStripeSignature } from "@lib/services/payments";
2
3  export const dynamic = "force-dynamic";
4
5  /** Stripe webhook (configure endpoint: /api/payments/webhook/stripe, events: checkout.session.*) */
6  export async function POST(req: Request) {
7      const raw = await req.text();
8      if (!verifyStripeSignature(raw, req.headers.get("stripe-signature"))) return new Response("Invalid signa
9 ture", { status: 400 });
10     const event = JSON.parse(raw) as { type: string; data: { object: { id: string; client_reference_id?: str
11 ing; payment_status?: string; metadata?: { reference?: string } } } };
12     const s = event.data.object;
13     const reference = s.client_reference_id || s.metadata?.reference;
14     if (!reference) return Response.json({ received: true });
15     try {
16         if (event.type === "checkout.session.completed" || event.type === "checkout.session.async_payment_succ
17 eeded") {
18             if (s.payment_status === "paid") await completeIntent(reference, true, s.id, { type: event.type, id:
19 s.id });
20             } else if (event.type === "checkout.session.expired" || event.type === "checkout.session.async_payment
21 _failed") {
22                 await completeIntent(reference, false, s.id, { type: event.type, id: s.id });
23             }
24         } catch (e) {
25             console.error("[stripe webhook]", e);
26             return new Response("Error", { status: 500 });
27         }
28     }
29     return Response.json({ received: true });
30 }
31

```

src/app/api/payouts/[id]/route.ts

```
1 import { handle, oneOf, readJson, str } from "@lib/api";
2 import { requireUser } from "@lib/auth";
3 import { processPayout } from "@lib/services/payments";
4
5 type Ctx = { params: Promise<{ id: string }> };
6
7 /** PATCH { status: "paid" | "rejected", note?: admin settles a payout */
8 export async function PATCH(req: Request, { params }: Ctx) {
9   return handle(async () => {
10     const admin = await requireUser(["admin"]);
11     const b = await readJson(req);
12     return processPayout(admin, Number((await params).id), oneOf(b, "status", ["paid", "rejected"] as cons
13 t, true)!, str(b, "note", { max: 500 }));
14   });
15 }
```

src/app/api/payouts/route.ts

```
1 import { handle, num, oneOf, readJson, str } from "@lib/api";
2 import { requireUser } from "@lib/auth";
3 import { listPayouts, requestPayout } from "@lib/services/payments";
4
5 export const dynamic = "force-dynamic";
6
7 export async function GET() {
8   return handle(async () => listPayouts(await requireUser(["farmer", "admin"])));
9 }
10
11 /** POST { amount, method, accountTitle, accountNumber }: farmer withdraws wallet balance */
12 export async function POST(req: Request) {
13   return handle(async () => {
14     const farmer = await requireUser(["farmer"]);
15     const b = await readJson(req);
16     return requestPayout(farmer, {
17       amount: num(b, "amount", { required: true, min: 1000 }!),
18       method: oneOf(b, "method", ["bank", "jazzcash", "easypaisa"] as const, true)!,
19       accountTitle: str(b, "accountTitle", { required: true, max: 120 }!),
20       accountNumber: str(b, "accountNumber", { required: true, max: 60 }!),
21     });
22   }, 201);
23 }
24
```

src/app/api/reports/summary/route.ts

```
1 import { requireUser } from "@lib/auth";
2 import { fail } from "@lib/api";
3 import { adminStats, farmerStats } from "@lib/services/analytics";
4 import { formatPKR } from "@lib/constants";
5
6 export const dynamic = "force-dynamic";
7
8 type Cell = string | number | null | undefined;
9
10 /** Escape CSV values and prevent spreadsheet formula execution in free-text fields. */
11 function csvCell(value: Cell) {
12   let text = String(value ?? "").replace(/\r\n/g, " ");
13   if (/^[=+@t]/.test(text) || /^(?!\\d)/.test(text)) text = `_${text}`;
14   return `_${text.replace(/"/g, '"')}_`;
15 }
16
17 const toCsv = (rows: Cell[][] => "\uFEFF" + rows.map((row) => row.map(csvCell).join(",")).join("\r\n") +
18 "\r\n");
19
20 /** GET /api/reports/summary – downloads a live, role-scoped CSV summary for admin or farmer. */
21 export async function GET() {
22   try {
23     const user = await requireUser(["farmer", "admin"]);
24     const date = new Date().toLocaleString("en-GB", { timeZone: "Asia/Karachi", dateStyle: "medium", timeS
25 tyle: "short" });
26   }
27 }
```

```

24     const rows: Cell[][] = [
25       ["MARKETLINK AGRI-HUB", user.role === "admin" ? "Platform summary" : "Farmer sales summary"],
26       ["Generated (PKT)", date],
27       ["Prepared for", user.fullName],
28       [],
29       ["Metric", "Value"],
30     ];
31
32     if (user.role === "admin") {
33       const report = await adminStats();
34       rows.push(
35         ["Transaction volume", formatPKR(report.totalVolume)],
36         ["Orders", report.totalOrders],
37         ["Active bids", report.activeBids],
38         ["Shipments in progress", report.activeShipments],
39         ["Held in escrow", formatPKR(report.escrowHeld)],
40         ["Open disputes", report.openDisputes],
41         ["Active farmers", report.farmers],
42         ["Buyers", report.buyers],
43         ["Platform fees", formatPKR(report.platformFees)],
44         [],
45         ["Month", "Transaction volume (PKR)", "Orders"],
46         ...report.monthly.map((m) => [m.month, m.volume, m.orders]),
47         [],
48         ["Region", "Transaction volume (PKR)", "Orders"],
49         ...report.topRegions.map((r) => [r.region, r.volume, r.orders]),
50       );
51     } else {
52       const report = await farmerStats(user.id);
53       rows.push(
54         ["Active listings", report.activeListings],
55         ["Available inventory (kg)", report.inventoryKg],
56         ["Inventory value", formatPKR(report.inventoryValue)],
57         ["New bids to review", report.pendingBids],
58         ["Counter-offers awaiting buyer", report.counteredBids],
59         ["Active orders", report.activeOrders],
60         ["Funds in escrow", formatPKR(report.inEscrow)],
61         ["Total payout earnings", formatPKR(report.totalEarned)],
62         ["Earnings this month", formatPKR(report.earnedThisMonth)],
63         ["Verified buyer reviews", user.ratingCount],
64         ["Trust score (out of 5)", user.trustScore],
65         [],
66         ["Month", "Sales volume (PKR)", "Orders"],
67         ...report.monthly.map((m) => [m.month, m.volume, m.orders]),
68         [],
69         ["Crop", "Sales volume (PKR)"],
70         ...report.byCrop.map((crop) => [crop.name, crop.value]),
71       );
72     }
73
74     const filename = `marketlink-${user.role}-summary-${new Date().toISOString().slice(0, 10)}.csv`;
75     return new Response(toCsv(rows), {
76       status: 200,
77       headers: {
78         "content-type": "text/csv; charset=utf-8",
79         "content-disposition": `attachment; filename="${filename}"`,
80         "cache-control": "private, no-store",
81         "x-content-type-options": "nosniff",
82       },
83     });
84   } catch (error) {
85     return fail(error);
86   }
87 }
88

```

src/app/api/reviews/[id]/reply/route.ts

```

1  import { ApiError, handle, readJson, str } from "@lib/api";
2  import { requireUser } from "@lib/auth";
3  import { respondToReview } from "@lib/services/reviews";
4
5  type Context = { params: Promise<{ id: string }> };
6
7  /** POST /api/reviews/:id/reply { reply } – farmer responds once to a verified buyer review. */

```

```

8   export async function POST(request: Request, { params }: Context) {
9       return handle(async () => {
10           const farmer = await requireUser(["farmer"]);
11           const reviewId = Number((await params).id);
12           if (!Number.isSafeInteger(reviewId) || reviewId < 1) throw new ApiError(422, "Invalid review ID");
13           const body = await readJson(request);
14           return respondToReview(farmer, reviewId, str(body, "reply", { required: true, max: 600 }));
15       }, 201);
16   }
17

```

src/app/api/reviews/route.ts

```

1   import { ApiError, handle, num, readJson, str } from "@lib/api";
2   import { requireUser } from "@lib/auth";
3   import { createReview, listReviews, pendingReviews } from "@lib/services/reviews";
4
5   export const dynamic = "force-dynamic";
6
7   /** GET ?farmerId=: public seller reviews · GET ?pending=1: buyer's delivered orders awaiting a review */
8   export async function GET(req: Request) {
9       return handle(async () => {
10           const sp = new URL(req.url).searchParams;
11           if (sp.get("pending") === "1") return pendingReviews((await requireUser(["buyer"])).id);
12           const farmerId = Number(sp.get("farmerId"));
13           if (!farmerId) throw new ApiError(422, "farmerId is required");
14           return listReviews(farmerId);
15       });
16   }
17
18   /** POST { orderId, rating 1-5, comment? }: buyer reviews a delivered order (one per order) */
19   export async function POST(req: Request) {
20       return handle(async () => {
21           const buyer = await requireUser(["buyer"]);
22           const b = await readJson(req);
23           return createReview(buyer, num(b, "orderId", { required: true, min: 1 }), num(b, "rating", { required
24 : true, min: 1, max: 5 }), str(b, "comment", { max: 1000 }));
25       }, 201);
26   }
27

```

src/app/api/tracking/[trackingNumber]/route.ts

```

1   import { handle } from "@lib/api";
2   import { getCurrentUser } from "@lib/auth";
3   import { trackShipment } from "@lib/services/tracking";
4
5   export const dynamic = "force-dynamic";
6   type Ctx = { params: Promise<{ trackingNumber: string }> };
7
8   /** Public shipment tracking (party names & notes only for order parties / staff). */
9   export async function GET(_req: Request, { params }: Ctx) {
10       return handle(async () => {
11           const viewer = await getCurrentUser();
12           return trackShipment(decodeURIComponent((await params).trackingNumber), viewer ? { id: viewer.id, role
13 : viewer.role } : null);
14       });
15   }
16

```

src/app/api/wallet/route.ts

```

1   import { ApiError, handle } from "@lib/api";
2   import { requireUser } from "@lib/auth";
3   import { getWallet } from "@lib/services/orders";
4
5   export const dynamic = "force-dynamic";
6
7   export async function GET() {
8       return handle(async () => getWallet(await requireUser(["buyer", "farmer"])));
9   }
10
11   /** Direct top-ups are disabled: funds can only enter through a verified payment gateway callback (/api/pa

```

```

11   yments/)*). */
12   export async function POST() {
13     return handle(async () => {
14       throw new ApiError(410, "Direct top-ups are disabled. Use POST /api/payments/checkout.");
15     });
16   }
17

```

src/app/auth/google-test/page.tsx

```

1   import { redirect } from "next/navigation";
2   import { googleMockEnabled } from "@lib/google";
3   import { GoogleTestForm } from "../test-form";
4
5   export const dynamic = "force-dynamic";
6   export const metadata = { title: "Google sign-in" };
7
8   export default async function GoogleTestPage({ searchParams }: { searchParams: Promise<{ role?: string; ne
9   xt?: string }> }) {
10     if (!googleMockEnabled()) redirect("/api/auth/google");
11     const sp = await searchParams;
12     return <GoogleTestForm role={sp.role === "farmer" ? "farmer" : "buyer"} next={sp.next} />;
13   }

```

src/app/auth/google-test/test-form.tsx

```

1   "use client";
2
3   import Link from "next/link";
4   import { useRouter } from "next/navigation";
5   import { useState } from "react";
6   import { GoogleLogo } from "@components/auth/google-button";
7   import { Spinner } from "@components/ui";
8   import { useToast } from "@components/providers";
9
10  /** Simulated Google account chooser for servers without GOOGLE_CLIENT_ID. Uses the real provisioning logi
11  c. */
12  export function GoogleTestForm({ role, next }: { role: "buyer" | "farmer"; next?: string }) {
13    const router = useRouter();
14    const { push } = useToast();
15    const [f, setF] = useState({ name: "", email: "", role });
16    const [busy, setBusy] = useState(false);
17
18    async function submit(e: React.FormEvent) {
19      e.preventDefault();
20      setBusy(true);
21      try {
22        const res = await fetch("/api/auth/google/test", { method: "POST", headers: { "content-type": "appli
23        cation/json" }, body: JSON.stringify({ ...f, next }) });
24        const data = await res.json();
25        if (!res.ok || !data.success) throw new Error(data.error || "Sign-in failed");
26        push({ kind: "success", title: data.data.created ? "Account created with Google" : "Signed in with G
27        oogle" });
28        router.push(data.data.redirectTo);
29        router.refresh();
30      } catch (err) {
31        push({ kind: "error", title: "Google sign-in failed", body: (err as Error).message });
32        setBusy(false);
33      }
34    }
35
36    return (
37      <div className="grid min-h-screen place-items-center bg-[#f0f4f9] px-4 py-10 dark:bg-slate-950">
38        <div className="w-full max-w-[450px] rounded-[28px] bg-white p-8 shadow-sm sm:p-10 dark:bg-slate-900">
39          <GoogleLogo className="h-10 w-10" />
40          <h1 className="mt-5 text-3xl font-normal text-slate-900 dark:text-white">Sign in</h1>
41          <p className="mt-2 text-base text-slate-700 dark:text-slate-300">to continue to <b>MarketLink Agri
42          -Hub</b></p>
43          <p className="mt-4 rounded-xl bg-amber-50 p-3 text-xs text-amber-900 dark:bg-amber-950/30 dark:tex
44          t-amber-200">
45            <b>Test mode.</b> Google OAuth credentials aren't configured on this server, so this screen
46            simulates Google's consent step. It can only create new accounts or sign into accounts made here. Ad
47            d <code>GOOGLE_CLIENT_ID</code> / <code>GOOGLE_CLIENT_SECRET</code> for real Google sign-in.

```

```

41     </p>
42     <form onSubmit={submit} className="mt-6 space-y-4">
43       <input className="h-14 w-full rounded-md border border-slate-300 px-4 text-base outline-none focus:
  ↵  us:border-[#1a73e8] focus:ring-1 focus:ring-[#1a73e8] dark:border-slate-600 dark:bg-slate-800" placeholder
  ↵  ="Full name" value={f.name} onChange={(e) => setF({ ...f, name: e.target.value })} required aria-label="Full
  ↵  name" />
44       <input type="email" className="h-14 w-full rounded-md border border-slate-300 px-4 text-base out
  ↵  line-none focus:border-[#1a73e8] focus:ring-1 focus:ring-[#1a73e8] dark:border-slate-600 dark:bg-slate-800
  ↵  " placeholder="Email (Gmail address)" value={f.email} onChange={(e) => setF({ ...f, email: e.target.value
  ↵  })} required aria-label="Email" />
45       <fieldset className="flex gap-2 text-sm">
46         <legend className="mb-2 text-slate-600 dark:text-slate-400">New account type</legend>
47         {[["buyer", "farmer"] as const].map((r) => (
48           <label key={r} className={`flex-1 cursor-pointer rounded-lg border-2 px-3 py-2 text-center font-medium ${f.role === r ? "border-[#1a73e8] bg-blue-50 text-[#1a73e8] dark:bg-blue-500/10" : "border-slate-200 dark:border-slate-700"}`}>
  ↵           <input type="radio" className="sr-only" checked={f.role === r} onChange={() => setF({ ...f, role: r })} />
49           {r === "buyer" ? " Buyer" : " Farmer"}
50         </label>
51       )]}
52     </fieldset>
53     <p className="text-xs text-slate-500">Google will share your name, email address and profile picture with MarketLink.</p>
54     <div className="flex items-center justify-between pt-2">
55       <Link href="/login" className="rounded-full px-4 py-2 text-sm font-medium text-[#1a73e8] hover:bg-blue-50 dark:hover:bg-blue-500/10">Cancel</Link>
56       <button disabled={busy} className="inline-flex items-center gap-2 rounded-full bg-[#1a73e8] px-6 py-2.5 text-sm font-medium text-white hover:bg-[#1765cc] disabled:opacity-60">{busy} && <Spinner /> Continue</button>
57     </div>
58   </form>
59 </div>
60 </div>
61 </div>
62 );
63 }
64

```

src/app/deploy/page.tsx

```

1  import Link from "next/link";
2  import { Brand } from "@components/app-shell";
3
4  export const metadata = { title: "Run locally / Hosting" };
5
6  function Code({ children }: { children: string }) {
7    return <pre className="mt-2 overflow-x-auto rounded-xl bg-brand-950 p-3 text-xs leading-6 text-emerald-200">{children}</pre>;
8  }
9
10 function Step({ n, title, children }: { n: number; title: string; children: React.ReactNode }) {
11   return (
12     <li className="flex gap-4">
13       <span className="grid h-9 w-9 shrink-0 place-items-center rounded-full bg-brand-900 font-extrabold text-gold-400">{n}</span>
14       <div className="pt-1">
15         <p className="font-semibold">{title}</p>
16         <div className="mt-1 text-sm text-slate-600 dark:text-slate-400">{children}</div>
17       </div>
18     </li>
19   );
20 }
21
22 function Option({ tag, title, time, children }: { tag: string; title: string; time: string; children: React.ReactNode }) {
23   return (
24     <section className="card p-6">
25       <div className="flex items-center gap-3">
26         <span className="grid h-9 w-9 place-items-center rounded-xl bg-slate-100 font-bold text-brand-900 dark:bg-white/10 dark:text-gold-400">{tag}</span>
27         <div>
28           <h3 className="font-bold">{title}</h3>
29           <p className="text-xs text-slate-500"> {time}</p>
30         </div>
31       </div>

```

```

32     <div className="mt-3 space-y-2 text-sm text-slate-700 dark:text-slate-300">{children}</div>
33   </section>
34 );
35 }
36
37 export default function DeployPage() {
38   return (
39     <div className="min-h-screen">
40       <header className="pk-pattern relative bg-brand-900">
41         <div className="mx-auto flex max-w-5xl items-center justify-between px-4 py-4 sm:px-6">
42           <Brand light />
43           <Link href="/login" className="btn bg-gold-400 text-brand-950 hover:bg-gold-300">Open app</Link>
44         </div>
45         <div className="mx-auto max-w-5xl px-4 pt-6 pb-12 text-white sm:px-6">
46           <h1 className="text-3xl font-extrabold sm:text-4xl">Run MarketLink on your own PC</h1>
47           <p className="mt-2 max-w-2xl text-white/80">You only need Node.js. The database is built in, so
48           ↵ there's no PostgreSQL or Docker to install. Tables and Pakistani demo data are created automatically.
49           ↵ </p>
50           <div className="mt-6 flex flex-wrap gap-3">
51             <a href="/MarketLink-AgriHub-Pakistan.zip" className="btn bg-gold-400 px-5 py-3 text-base text
52             ↵ -brand-950 hover:bg-gold-300">⚡ Download complete project folder (.zip)</a>
53             <a href="https://nodejs.org" target="_blank" rel="noreferrer" className="btn bg-white/10 px-5
54             ↵ py-3 text-base text-white ring-1 ring-white/30 hover:bg-white/20">⚡ Get Node.js (LTS)</a>
55             <a href="/MarketLink-AgriHub-Complete.pdf" target="_blank" className="btn bg-white/10 px-5 py-
56             ↵ 3 text-base text-white ring-1 ring-white/30 hover:bg-white/20">Project PDF</a>
57           </div>
58         </div>
59       </header>
60
61       <main className="mx-auto max-w-5xl space-y-6 px-4 py-10 sm:px-6">
62         <div className="card border-amber-300 bg-amber-50 p-5 dark:border-amber-700 dark:bg-amber-950/30">
63           <p className="font-bold text-amber-900 dark:text-amber-200">⚠️ [Urdu] Why "http://localhost:3
64           ↵ 000" shows "This site can't be reached"</p>
65           <p className="mt-1 text-sm text-amber-900/80 dark:text-amber-200/80">
66             ↵ <b>localhost</b> means "this computer". The link only works <b>after</b> you start
67             ↵ the MarketLink server on your PC (steps below) and only <b>while</b> its black window stays open</b>. Opening
68             ↵ the link without starting the server always fails.
69           </p>
70         </div>
71
72         <section className="card relative overflow-hidden p-6 sm:p-8">
73           <span className="absolute inset-x-0 top-0 h-1 bg-gradient-to-r from-brand-900 to-gold-400" />
74           <div className="flex flex-wrap items-center gap-3">
75             <h2 className="text-xl font-extrabold">Local hosting in 4 steps</h2>
76             <span className="badge bg-gold-400 text-brand-950">Recommended</span>
77           </div>
78           <ol className="mt-6 space-y-5">
79             <Step n={1} title="Install Node.js 20 LTS (one time)">
80               Go to <a className="font-semibold text-brand-700 underline" href="https://nodejs.org" target
81               ↵ ="_blank" rel="noreferrer">nodejs.org</a>, click the green <b>LTS</b> button and install with the default
82               ↵ options.
83             </Step>
84             <Step n={2} title="Download and extract the source code">
85               Download the <a className="font-semibold text-brand-700 underline" href="/MarketLink-AgriHub
86               ↵ -Pakistan.zip">complete project folder (.zip)</a>, right-click it and choose <b>Extract All</b>. You get o
87               ↵ ne folder, <b>MarketLink-AgriHub-Pakistan</b>, with the source code, start files, PDF, SQL scripts and gui
88               ↵ des. Open <b>START-HERE.html</b> inside it first.
89             </Step>
90             <Step n={3} title="Double-click the start file">
91               <b>Windows</b>: <code>start-windows.bat</code> · <b>Mac / Linux</b>: run <code>bash start.sh
92               ↵ </code> in Terminal. The first run installs and builds (3-6 min); later runs take a few seconds.
93             </Step>
94             <Step n={4} title="Wait for MarketLink is running!">
95               Your browser opens <b>http://localhost:3000</b> automatically. <b>Keep</b> the black window open
96               ↵ <b>while</b> using the site, and press <code>Ctrl+C</code> in it to stop. On your phone (same Wi-Fi
97               ↵ ), open the "Phone" address shown in the window.
98             </Step>
99           </ol>
100          <div className="mt-6 overflow-hidden rounded-xl bg-brand-950 p-4 font-mono text-xs leading-6 tex
101          ↵ t-emerald-200">
102            <p>=====</p>
103            <p>MarketLink is running!</p>
104            <p>On this computer: http://localhost:3000</p>
105            <p>Phone / same Wi-Fi: http://192.168.1.5:3000</p>
106            <p>Demo login: farmer@marketlink.pk / password123</p>
107            <p>=====</p>

```

```

91         </div>
92     </section>
93
94     <section className="card p-6">
95         <h2 className="text-lg font-bold"> Troubleshooting</h2>
96         <div className="mt-3 overflow-x-auto">
97             <table className="w-full text-sm">
98                 <tbody className="divide-y divide-slate-100 dark:divide-white/5">
99                     {
100                         ["'node' is not recognized", "Node.js isn't installed. Install it, restart the PC, and t
101 ry again."],
102                         ["Browser: \"can't be reached\"", "The server window was closed or hasn't finished start
103 ing. Wait for MarketLink is running!"],
104                         ["Port 3000 already in use", "Handled automatically: the window shows the port it picked
105 (e.g. 3001)."],
106                         ["Fresh demo data", "Stop the server, delete the .data folder, start again."],
107                         ["Use my own PostgreSQL", "Set DATABASE_URL in .env. The database is created automatical
108 ly."],
109                         .map(([a, b]) => (
110                             <tr key={a}><td className="py-2.5 pr-4 font-semibold whitespace-nowrap">{a}</td><td clas
111 sName="py-2.5 text-slate-600 dark:text-slate-400">{b}</td></tr>
112                         )))
113                     </tbody>
114                 </table>
115             </div>
116         </section>
117
118         <h2 className="pt-2 text-lg font-bold">Other hosting options</h2>
119         <div className="grid gap-4 md:grid-cols-3">
120             <Option tag="B" title="Temporary public link" time="1 min · free · no account">
121                 <p>While the site runs on your PC, open a second terminal:</p>
122                 <Code>npx cloudflared tunnel --url http://localhost:3000</Code>
123                 <p>You get a public <code>https://...trycloudflare.com</code> link for anyone to open while you
124 r PC is on.</p>
125             </Option>
126             <Option tag="C" title="Docker + PostgreSQL" time="~5 min">
127                 <p>Install Docker Desktop, then double-click <b>start-docker.bat</b> or run:</p>
128                 <Code>docker compose up --build</Code>
129             </Option>
130             <Option tag="D" title="Free cloud (24/7)" time="~15 min · GitHub account">
131                 <p>Push the folder to GitHub, then on <b>render.com</b> choose <b>New → Blueprint</b>. <code>r
132 ender.yaml</code> sets up everything.</p>
133                 <p className="text-xs text-slate-500">Or Vercel + Neon: set <code>DATABASE_URL</code>, <code>S
134 ESSION_SECRET</code>, <code>COOKIE_SECURE=true</code>.</p>
135             </Option>
136         </div>
137
138         <div className="card flex flex-wrap items-center gap-3 p-4 text-sm">
139             <span className="text-xl"></span>
140             <p>Demo logins (password <b>password123</b>): <code>farmer@marketlink.pk</code> · <code>buyer@ma
141 rketlink.pk</code> · <code>ahmed.mustafa@admin.com</code></p>
142         </div>
143     </main>
144 </div>
145 );
146 }
147
148

```

src/app/docs/ddl-tabs.tsx

```

1  "use client";
2
3  import { useState } from "react";
4
5  export function DdlTabs({ postgres, mysql }: { postgres: string; mysql: string }) {
6      const [tab, setTab] = useState<"pg" | "mysql">("pg");
7      const [copied, setCopied] = useState(false);
8      const text = tab === "pg" ? postgres : mysql;
9      return (
10         <div className="card mt-4 overflow-hidden">
11             <div className="flex items-center gap-2 border-b border-slate-200 px-4 py-2 dark:border-slate-800">
12                 {[ "pg", "mysql" ] as const}.map((t) => (
13                     <button key={t} onClick={() => setTab(t)} className={`rounded-lg px-3 py-1.5 text-sm font-semibo
14 ld ${tab === t ? "bg-brand-600 text-white" : "text-slate-500"}`>
15                         {t === "pg" ? "PostgreSQL (runtime)" : "MySQL 8"}
16                     </button>
17                 ))
18             </div>
19         </div>
20     );
21 }

```

```

15         </button>
16     )})
17     <button
18         className="btn-secondary ml-auto py-1 text-xs"
19         onClick={() => {
20             navigator.clipboard.writeText(text);
21             setCopied(true);
22             setTimeout(() => setCopied(false), 1500);
23         }}
24     >
25         {copied ? "✓ Copied" : "Copy SQL"}
26     </button>
27 </div>
28 <pre className="max-h-[520px] overflow-auto bg-slate-950 p-4 text-xs leading-5 text-slate-200">{text
29 }</pre>
30 </div>
31 );
32 }

```

src/app/docs/page.tsx

```

1  import Link from "next/link";
2  import { Brand } from "@components/app-shell";
3  import { API_GROUPS, ERD, MYSQL_DDL, WORKFLOWS } from "@lib/docs-content";
4  import { POSTGRES_DDL } from "@lib/ddl-postgres";
5  import { DdlTabs } from "../ddl-tabs";
6
7  export const metadata = { title: "API & Schema" };
8
9  const METHOD_STYLE: Record<string, string> = {
10    GET: "bg-sky-100 text-sky-800 dark:bg-sky-500/15 dark:text-sky-300",
11    POST: "bg-emerald-100 text-emerald-800 dark:bg-emerald-500/15 dark:text-emerald-300",
12    PATCH: "bg-amber-100 text-amber-800 dark:bg-amber-500/15 dark:text-amber-300",
13    DELETE: "bg-rose-100 text-rose-800 dark:bg-rose-500/15 dark:text-rose-300",
14  };
15
16  export default function DocsPage() {
17    return (
18      <div className="min-h-screen">
19        <header className="sticky top-0 z-30 border-b border-slate-200 bg-white/85 backdrop-blur dark:border
20        -slate-800 dark:bg-slate-950/85">
21          <div className="mx-auto flex max-w-6xl items-center justify-between px-4 py-3 sm:px-6">
22            <Brand />
23            <nav className="flex gap-1 text-sm">
24              {[ "erd", "ddl", "api", "workflows" ].map((s) => (
25                <a key={s} href={`#${s}`} className="hidden rounded-lg px-3 py-1.5 font-medium text-slate-60
26                0 uppercase hover:bg-slate-100 sm:block dark:text-slate-300 dark:hover:bg-slate-800">{s}</a>
27              ))}
28              <Link href="/login" className="btn-primary ml-2 py-1.5">Open app</Link>
29            </nav>
30          </div>
31        </header>
32
33        <main className="mx-auto max-w-6xl space-y-14 px-4 py-10 sm:px-6">
34          <section>
35            <h1 className="text-3xl font-extrabold tracking-tight sm:text-4xl">Technical Specification</h1>
36            <p className="mt-2 max-w-3xl text-slate-600 dark:text-slate-400">
37              Architecture: Next.js App Router (React 19 + Tailwind CSS) front-end · Route Handlers as a RES
38              T API layer · service modules containing business logic with transactional row locking · PostgreSQL via Dr
39              izzle ORM · Recharts for visualization. All responses follow <code className="rounded bg-slate-100 px-1 da
40              rk:bg-slate-800">{`{ success, data | error }`}</code>.
41            </p>
42            <div className="mt-6 grid gap-3 sm:grid-cols-4">
43              {[["Presentation", "React Server + Client Components, Tailwind, dark mode, EN/[Urdu]", ["API"
44                , "REST route handlers · RBAC via signed session cookie"], ["Domain", "Bids state machine · escrow ledger
45                · logistics · mandi sync"], ["Data", "PostgreSQL · 9 tables · FKs, enums, unique & secondary indexes"]].ma
46                p([t, d]) => (
47                  <div key={t} className="card p-4"><p className="text-xs font-bold tracking-wider text-brand-
48                  600 uppercase">{t}</p><p className="mt-1 text-sm">{d}</p></div>
49                )]}
50            </div>
51          </section>
52
53          <section id="erd">

```

```

45     <h2 className="text-2xl font-bold">1 · Entity Relationship Diagram</h2>
46     <div className="mt-4 grid gap-4 lg:grid-cols-2">
47         <pre className="card overflow-x-auto p-5 text-xs leading-6">{ERD}</pre>
48         <div className="card grid grid-cols-2 gap-2 p-5 text-xs sm:grid-cols-3">
49             {[ "users", "crops_inventory", "mandi_rates", "bids_negotiations", "orders_logistics", "quali
ty_inspections", "order_events", "wallet_transactions", "disputes" ].map((t, i) => (
50                 <div key={t} className={`rounded-xl border p-3 font-mono ${i < 6 ? "border-brand-300 bg-br
and-50 dark:border-brand-800 dark:bg-brand-950/30" : "border-slate-200 dark:border-slate-700"}`}>
51                     <p className="font-bold">{t}</p>
52                     <p className="mt-1 text-[10px] text-slate-500">{i < 6 ? "SRS core" : "supporting"}</p>
53                 </div>
54             )]}
55         </div>
56     </div>
57 </section>
58
59     <section id="ddl">
60         <h2 className="text-2xl font-bold">2 · Database DDL</h2>
61         <p className="mt-1 text-sm text-slate-500">PostgreSQL DDL is generated directly from the Drizzle
schema (<code>src/db/schema.ts</code>) and is what runs in production. A MySQL 8 equivalent is provided f
or portability. Files: <code>database/schema.sql</code>, <code>database/mysql_schema.sql</code>.</p>
62         <DdlTabs postgres={POSTGRES_DDL} mysql={MYSQL_DDL} />
63     </section>
64
65     <section id="api">
66         <h2 className="text-2xl font-bold">3 · REST API Endpoints</h2>
67         <p className="mt-1 text-sm text-slate-500">Auth: httpOnly HMAC-signed <code>ml_session</code> co
okie. Errors: 400 bad JSON · 401 unauthenticated · 403 wrong role/ownership · 404 · 409 invalid state tran
sition · 402 insufficient funds · 422 validation.</p>
68         <div className="mt-5 space-y-8">
69             {API_GROUPS.map((g) => (
70                 <div key={g.name}>
71                     <h3 className="mb-3 text-lg font-semibold">{g.name}</h3>
72                     <div className="space-y-3">
73                         {g.endpoints.map((e) => (
74                             <details key={e.method + e.path} className="card group overflow-hidden">
75                                 <summary className="flex cursor-pointer flex-wrap items-center gap-3 px-4 py-3">
76                                     <span className={`badge w-16 justify-center font-mono ${METHOD_STYLE[e.method]}`}>
{e.method}</span>
77                                     <code className="text-sm font-semibold break-all">{e.path}</code>
78                                     <span className="ml-auto text-xs text-slate-500">{e.roles}</span>
79                                 </summary>
80                                 <div className="space-y-3 border-t border-slate-100 px-4 py-3 text-sm dark:border-sl
ate-800">
81                                     <p>{e.desc}</p>
82                                     {e.request && (<div><p className="label">Request body</p><pre className="overflow-
x-auto rounded-xl bg-slate-950 p-3 text-xs text-emerald-300">{e.request}</pre></div>)}
83                                     {e.response && (<div><p className="label">Response</p><pre className="overflow-x-a
uto rounded-xl bg-slate-950 p-3 text-xs text-sky-300">{e.response}</pre></div>)}
84                                 </div>
85                             </details>
86                         )]}
87                     </div>
88                 </div>
89             )]}
90         </div>
91     </section>
92
93     <section id="workflows">
94         <h2 className="text-2xl font-bold">4 · Business Logic Workflows</h2>
95         <div className="mt-5 grid gap-4 md:grid-cols-2">
96             {WORKFLOWS.map((w) => (
97                 <div key={w.title} className="card p-5">
98                     <h3 className="font-semibold">{w.icon} {w.title}</h3>
99                     <ol className="mt-3 space-y-2">
100                         {w.steps.map((s, i) => (
101                             <li key={i} className="flex gap-3 text-sm text-slate-600 dark:text-slate-300">
102                                 <span className="grid h-6 w-6 shrink-0 place-items-center rounded-full bg-brand-600
text-xs font-bold text-white">{i + 1}</span>
103                                 <span>{s}</span>
104                             </li>
105                         )]}
106                     </ol>
107                 </div>
108             )]}
109         </div>

```

```

110     <div className="card mt-4 overflow-x-auto p-5">
111       <p className="label">Bid state machine</p>
112       <pre className="text-xs leading-6">{`pending —farmer:accept————→ accepted —→ order(awa
113   iting_escrow)
114   pending —farmer:counter————→ countered —buyer:accept_counter→ accepted
115   pending —farmer:reject————→ rejected   countered —buyer:reject_counter→ rejected
116   pending|countered —buyer:withdraw▶ withdrawn   countered —buyer:revise→ pending
117   order: confirmed —(escrow_locked)→ quality_checked[admin] → dispatched[farmer] → in_transit[farmer] →
118   delivered[buyer → escrow released]
119   any locked stage —dispute→ disputed —admin→ released | refunded`}</pre>
120     </div>
121   </main>
122 </div>
123   );
124 }
125

```

src/app/error.tsx

```

1  "use client";
2
3  import Link from "next/link";
4
5  export default function GlobalError({ error, reset }: { error: Error & { digest?: string }; reset: () => v
6  oid }) {
7    return (
8      <div className="grid min-h-screen place-items-center px-4">
9        <div className="card max-w-lg p-8 text-center">
10          <div className="text-5xl"></div>
11          <h1 className="mt-4 text-xl font-bold">Something went wrong</h1>
12          <p className="mt-2 text-sm text-slate-600 dark:text-slate-400">
13            The server couldn't complete this request. If you are hosting MarketLink on your own comput
14            er, the most common cause is the database connection.
15          </p>
16          <ul className="mt-4 space-y-1.5 text-left text-sm text-slate-600 dark:text-slate-400">
17            <li>• Simplest fix: open <code>.env</code> and leave <code>DATABASE_URL=</code> empty to use the
18            built-in database, then restart.</li>
19            <li>• Using PostgreSQL? Make sure it is running and the password in <code>DATABASE_URL</code> is
20            correct.</li>
21          </ul>
22          {error.digest && <p className="mt-3 font-mono text-xs text-slate-400">Error ID: {error.digest}</p>
23          }
24          <div className="mt-6 flex justify-center gap-2">
25            <button className="btn-primary" onClick={reset}>Try again</button>
26            <Link href="/" className="btn-secondary">Home</Link>
27          </div>
28        </div>
29      </div>
30    );
31  }
32

```

src/app/forgot-password/page.tsx

```

1  import { ForgotPasswordForm } from "@/components/auth/otp-forms";
2
3  export const metadata = { title: "Forgot password" };
4
5  export default async function ForgotPasswordPage({ searchParams }: { searchParams: Promise<{ email?: strin
6  g }> }) {
7    const { email } = await searchParams;
8    return <ForgotPasswordForm initialEmail={email ?? ""} />;
9  }
10

```

src/app/globals.css

```
1  @import "tailwindcss";
2
3  @custom-variant dark (&:where(.dark, .dark *));
4
5  @theme {
6    --font-sans: "Inter", ui-sans-serif, system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
7    /* Pakistan flag green scale (#01411C) */
8    --color-brand-50: #edfbf2;
9    --color-brand-100: #d3f5df;
10   --color-brand-200: #a9e9c3;
11   --color-brand-300: #72d59f;
12   --color-brand-400: #3dba78;
13   --color-brand-500: #1d9d5c;
14   --color-brand-600: #117e48;
15   --color-brand-700: #0d653b;
16   --color-brand-800: #0b5030;
17   --color-brand-900: #01411c;
18   --color-brand-950: #002611;
19   --color-gold-300: #f7d774;
20   --color-gold-400: #f1c232;
21   --color-gold-500: #d9a514;
22 }
23
24 html {
25   color-scheme: light;
26 }
27 html.dark {
28   color-scheme: dark;
29 }
30
31 body {
32   @apply bg-[#f6f8f4] text-slate-900 dark:bg-[#07130d] dark:text-slate-100;
33 }
34
35 [dir="rtl"] body {
36   font-family: "Noto Nastaliq Urdu", "Jameel Noori Nastaleeq", var(--font-sans);
37 }
38
39 @utility card {
40   @apply rounded-2xl border border-brand-900/8 bg-white shadow-[0_1px_2px_rgba(1,65,28,0.04),0_4px_16px_-4
41   ↵ px_rgba(1,65,28,0.08)] dark:border-white/8 dark:bg-[#0d1f16];
42 }
43 @utility btn {
44   @apply inline-flex items-center justify-center gap-2 rounded-xl px-4 py-2 text-sm font-semibold transiti
45   ↵ on disabled:cursor-not-allowed disabled:opacity-50;
46 }
47 @utility btn-primary {
48   @apply btn bg-brand-600 text-white shadow-sm hover:bg-brand-700 focus-visible:outline-2 focus-visible:ou
49   ↵ tline-offset-2 focus-visible:outline-brand-600;
50 }
51 @utility btn-secondary {
52   @apply btn border border-slate-300 bg-white text-slate-700 hover:bg-slate-50 dark:border-slate-700 dark:
53   ↵ bg-slate-800 dark:text-slate-200 dark:hover:bg-slate-700;
54 }
55 @utility btn-danger {
56   @apply btn bg-rose-600 text-white hover:bg-rose-700;
57 }
58 @utility btn-amber {
59   @apply btn bg-amber-500 text-white hover:bg-amber-600;
60 }
61 @utility input {
62   @apply w-full rounded-xl border border-slate-300 bg-white px-3 py-2 text-sm text-slate-900 placeholder:t
63   ↵ ext-slate-400 focus:border-brand-500 focus:ring-2 focus:ring-brand-500/30 focus:outline-none dark:border-s
64   ↵ late-700 dark:bg-slate-800 dark:text-slate-100;
65 }
66 @utility label {
67   @apply mb-1 block text-xs font-semibold tracking-wide text-slate-600 uppercase dark:text-slate-400;
68 }
69 @utility badge {
70   @apply inline-flex items-center rounded-full px-2.5 py-0.5 text-xs font-semibold capitalize;
71 }
72 @utility table-head {
73   @apply border-b border-slate-200 bg-slate-50/80 text-left text-[11px] font-semibold tracking-wider text-
```

```

67     slate-500 uppercase dark:border-white/5 dark:bg-white/[0.03] dark:text-slate-400;
68   }
69
70   @keyframes slide-in-right {
71     from { transform: translateX(100%); }
72     to { transform: translateX(0); }
73   }
74   @keyframes fade-in {
75     from { opacity: 0; transform: translateY(6px); }
76     to { opacity: 1; transform: translateY(0); }
77   }
78   @keyframes pop {
79     0% { transform: scale(0.6); opacity: 0; }
80     60% { transform: scale(1.1); opacity: 1; }
81     100% { transform: scale(1); }
82   }
83   .animate-slide-in { animation: slide-in-right 0.28s cubic-bezier(0.2, 0.8, 0.2, 1); }
84   .animate-fade-in { animation: fade-in 0.25s ease-out; }
85   .animate-pop { animation: pop 0.45s ease-out; }
86
87   @keyframes marquee {
88     from { transform: translateX(0); }
89     to { transform: translateX(-50%); }
90   }
91   .animate-marquee { animation: marquee 60s linear infinite; }
92   .animate-marquee:hover { animation-play-state: paused; }
93   [dir="rtl"] .animate-marquee { animation-direction: reverse; }
94
95   /* Subtle crescent & star pattern on green surfaces.
96      Rendered as a ::before overlay (inside the components layer) so it never
97      overrides the element's own background colour / gradient utilities. */
98   @layer components {
99     .pk-pattern {
100       isolation: isolate;
101     }
102     .pk-pattern::before {
103       content: "";
104       position: absolute;
105       inset: 0;
106       z-index: -1;
107       pointer-events: none;
108       background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='80' height=
109       '80' viewBox='0 0 80 80'%3E%3Cg fill='%23ffffff' fill-opacity='0.05'%3E%3Cpath d='M30 18a14 14 0 1 0 12 22
110       '11 11 0 1 1 0-18 14 14 0 0 0-12-4z'/%3E%3Cpath d='M52 22l2 5h5l-4 3 2 5-5-3-4 3 1-5-4-3h5z'/%3E%3C/g%3E%3
111       C/svg%3E");
112     }
113   }

```

src/app/layout.tsx

```

1  import type { Metadata, Viewport } from "next";
2  import type { ReactNode } from "react";
3  import "@/globals.css";
4  import { Providers } from "@components/providers";
5
6  export const metadata: Metadata = {
7    title: {
8      default: "MARKETLINK AGRI-HUB | Farm to Market",
9      template: "%s | MARKETLINK AGRI-HUB",
10    },
11    description: "MARKETLINK AGRI-HUB connects Pakistani farmers directly with commercial buyers through gra
12    ded crop listings, transparent mandi rates, secure bidding, escrow and shipment tracking.",
13    applicationName: "MARKETLINK AGRI-HUB",
14    icons: {
15      icon: [{ url: "/icon.svg", type: "image/svg+xml" }],
16      shortcut: "/icon.svg",
17      apple: "/apple-icon.png",
18    },
19  };
20
21  export const viewport: Viewport = {
22    themeColor: [
23      { media: "(prefers-color-scheme: light)", color: "#01411c" },
24      { media: "(prefers-color-scheme: dark)", color: "#003e27" },
25    ],
26  };

```

```

24   ],
25   };
26
27   const themeScript = `(function(){try{var t=localStorage.getItem('ml-theme');if(t==='dark'||(!t&&window.mat
  ↵ chMedia('(prefers-color-scheme: dark)').matches)){document.documentElement.classList.add('dark')}}var l=loc
  ↵ alStorage.getItem('ml-lang');if(l==='ur'){document.documentElement.dir='rtl';document.documentElement.lang
  ↵ ='ur'}}catch(e){}})();`;
28
29   export default function RootLayout({ children }: { children: ReactNode }) {
30     return (
31       <html lang="en" suppressHydrationWarning>
32         <head>
33           <script dangerouslySetInnerHTML={{ __html: themeScript }} />
34         </head>
35         <body className="min-h-screen antialiased">
36           <Providers>{children}</Providers>
37         </body>
38       </html>
39     );
40   }
41

```

src/app/login/login-form.tsx

```

1  "use client";
2
3  import { useState } from "react";
4  import { useRouter } from "next/navigation";
5  import { Brand } from "@components/app-shell";
6  import { Logo } from "@components/Logo";
7  import { api, Spinner } from "@components/ui";
8  import { CityOptions } from "@components/city-options";
9  import { useToast } from "@components/providers";
10 import { GoogleButton, OrDivider } from "@components/auth/google-button";
11 import { useEffect } from "react";
12
13 const DEMOS = [
14   { label: "Farmer", sub: "Muhammad Aslam · Multan", email: "farmer@marketlink.pk", icon: "" },
15   { label: "Buyer", sub: "Qureshi Wholesale · Lahore", email: "buyer@marketlink.pk", icon: "" },
16   { label: "Quality Inspector", sub: "Dr. Farhan Malik · Faisalabad", email: "inspector@marketlink.pk", ic
  ↵ on: "" },
17 ];
18
19 export type LoginAlert = { kind: "error" | "info"; title: string; body?: string } | null;
20
21 export function LoginForm({ initialEmail, initialRole, startOnRegister, adminMode = false, next, alert = n
  ↵ ull, signedInAs = null }: { initialEmail: string; initialRole: "farmer" | "buyer"; startOnRegister: boolea
  ↵ n; adminMode?: boolean; next?: string; alert?: LoginAlert; signedInAs?: { fullName: string; role: string }
  ↵ | null }) {
22   const router = useRouter();
23   const { push } = useToast();
24   const [tab, setTab] = useState<"login" | "register">(startOnRegister ? "register" : "login");
25   const [loading, setLoading] = useState<string | null>(null);
26   const [email, setEmail] = useState(initialEmail);
27   const [password, setPassword] = useState(initialEmail && !adminMode ? "password123" : "");
28   const [reg, setReg] = useState({ fullName: "", email: "", password: "", phone: "", role: initialRole, ci
  ↵ ty: "Lahore", cnicId: "", businessName: "", address: "" });
29
30   async function login(e?: string, p?: string) {
31     setLoading(e ?? "form");
32     try {
33       const res = await fetch("/api/auth/login", { method: "POST", headers: { "content-type": "application
  ↵ /json" }, body: JSON.stringify({ email: e ?? email, password: p ?? password }) });
34       const data = await res.json();
35       if (res.status === 403 && data.details?.code === "EMAIL_NOT_VERIFIED") {
36         push({ kind: "info", title: "Verify your email to continue" });
37         router.push(`/verify-email?email=${encodeURIComponent(data.details.email)}${data.details.devCode ?
  ↵ `&dev=${data.details.devCode}` : ""}`);
38         return;
39       }
40       if (!res.ok || !data.success) throw new Error(data.error || "Sign in failed");
41       const r = data.data as { redirectTo: string; fullName: string };
42       push({ kind: "success", title: `Welcome, ${r.fullName}` });
43       router.push(next && next.startsWith("/") && !next.startsWith("//") ? next : r.redirectTo);
44       router.refresh();

```

```

45     } catch (err) {
46         push({ kind: "error", title: "Sign in failed", body: (err as Error).message });
47         setLoading(null);
48     }
49 }
50
51 async function register(ev: React.FormEvent) {
52     ev.preventDefault();
53     setLoading("register");
54     try {
55         const r = await api<{ email: string; devCode?: string }>("/api/auth/register", { method: "POST", jso
56 n: reg });
57         push({ kind: "success", title: "Account created", body: "Enter the verification code we emailed you.
58 " });
59         router.push(`/verify-email?email=${encodeURIComponent(r.email)}${r.devCode ? `&dev=${r.devCode}` : "
60 "}`);
61     } catch (err) {
62         push({ kind: "error", title: "Registration failed", body: (err as Error).message });
63         setLoading(null);
64     }
65 }
66
67 useEffect(() => {
68     if (alert) push({ kind: alert.kind === "error" ? "error" : "info", title: alert.title, body: alert.bod
69 y });
70 }, []); // eslint-disable-line react-hooks/exhaustive-deps
71
72 async function switchAccount() {
73     await fetch("/api/auth/logout", { method: "POST" });
74     router.refresh();
75 }
76
77 const set = (k: keyof typeof reg) => (e: React.ChangeEvent<HTMLInputElement | HTMLSelectElement>) => set
78 Reg({ ...reg, [k]: e.target.value });
79
80 return (
81     <div className="grid min-h-screen lg:grid-cols-2">
82         <div className="relative hidden overflow-hidden lg:block">
83             { /* eslint-disable-next-line @next/next/no-img-element */ }
84             
86             <div className="absolute inset-0 bg-gradient-to-t from-brand-950 via-brand-900/60 to-brand-900/10"
87 />
88             <div className="absolute inset-x-0 bottom-0 p-12 text-white">
89                 <p className="mb-4 text-xl text-gold-300" dir="rtl">[Urdu]– [Urdu]</p>
90                 <p className="text-3xl font-bold">&ldquo;I sold 125 maund of wheat at Rs. 3,800/maund – Rs. 280
91 more than the local arhti offered.&rdquo;</p>
92                 <p className="mt-3 text-brand-100">– Muhammad Aslam, wheat & Chaunsa mango grower, Multan, Punja
93 b</p>
94             </div>
95         </div>
96
97         <div className="flex items-center justify-center px-4 py-10 sm:px-8">
98             <div className="w-full max-w-md">
99                 <Brand />
100                 {alert && (
101                     <div role="alert" className={`mt-8 rounded-2xl border p-4 text-sm ${alert.kind === "error" ? "
102 border-rose-300 bg-rose-50 text-rose-900 dark:border-rose-800 dark:bg-rose-950/40 dark:text-rose-100" : "b
103 order-sky-300 bg-sky-50 text-sky-900 dark:border-sky-800 dark:bg-sky-950/40 dark:text-sky-100"}`}>
104                     <p className="flex items-center gap-2 font-bold">{alert.kind === "error" ? "" : "[Urdu]} {a
105 lert.title}</p>
106                     {alert.body && <p className="mt-1 opacity-90">{alert.body}</p>}
107                     {signedInAs && (
108                         <p className="mt-2 text-xs opacity-90">
109                             You&apos;re signed in as <b>{signedInAs.fullName}</b> ({signedInAs.role}). Sign in below
110                             with an administrator account, or{" "}
111                             <button type="button" onClick={switchAccount} className="font-semibold underline">sign o
112 ut</button>.
113                         </p>
114                     )}
115                 </div>
116             )}
117         </div>
118     )}
119     {adminMode && (
120         <div className="mt-8 flex items-center gap-3 rounded-2xl bg-brand-900 p-4 text-white">
121             <Logo variant="mark" size={40} />
122         </div>
123     )}

```

```

107         <p className="font-bold">MARKETLINK AGRI-HUB · Admin</p>
108         <p className="text-xs text-white/70">Authorized administrators only</p>
109     </div>
110 </div>
111 }}
112 <h1 className={` ${adminMode ? "mt-6" : "mt-8"} text-2xl font-bold`} >{adminMode ? "Admin sign in"
113 : tab === "login" ? "Sign in to your account" : "Create your MarketLink account"}</h1>
114 <div className={`mt-5 flex rounded-xl bg-slate-100 p-1 dark:bg-slate-800 ${adminMode ? "hidden"
115 : ""}`>
116     {[ "login", "register" ].map((t) => (
117         <button key={t} onClick={() => setTab(t)} className={`flex-1 rounded-lg py-2 text-sm font-se
118 mibold ${tab === t ? "bg-white shadow dark:bg-slate-900" : "text-slate-500"} `}>
119         {t === "login" ? "Sign in" : "Register"}
120     </button>
121     ))}
122 </div>
123 {tab === "login" ? (
124     <div>
125         {!adminMode && (
126             <div className="mt-6">
127                 <GoogleButton role={initialRole} next={next} />
128                 <OrDivider text="or sign in with email" />
129             </div>
130             <form className={` ${adminMode ? "mt-6" : ""} space-y-4`} onSubmit={(e) => { e.preventDefault
131 (); login(); }}>
132                 <div>
133                     <label className="label" htmlFor="email">Email</label>
134                     <input id="email" type="email" required className="input" value={email} onChange={(e) =>
135 setEmail(e.target.value)} placeholder="aap@company.pk" />
136                 </div>
137                 <div>
138                     <div className="mb-1 flex items-center justify-between">
139                         <label className="label !mb-0" htmlFor="password">Password</label>
140                         <a href={` /forgot-password${email ? `?email=${encodeURIComponent(email)}` : ""}` } clas
141 sName="text-xs font-semibold text-brand-700 hover:underline dark:text-brand-400">Forgot password?</a>
142                     </div>
143                     <input id="password" type="password" required className="input" value={password} onChang
144 e={(e) => setPassword(e.target.value)} placeholder="....." />
145                 </div>
146                 <button className="btn-primary w-full py-2.5" disabled={!loading}>{loading === "form" &&
147 <Spinner />} Sign in</button>
148             </form>
149             {adminMode ? (
150                 <div className="mt-6 rounded-xl border border-amber-200 bg-amber-50 p-3 text-xs text-amber
151 -900 dark:border-amber-800 dark:bg-amber-950/30 dark:text-amber-200">
152                     Super Admin: <b>Ahmed Mustafa</b> · <b>ahmed.mustafa@admin.com</b> / <b>password123</b>.
153                     After signing in, change it under <b>Account settings</b>.
154                 </div>
155             ) : null}
156             {!adminMode && (
157                 <p className="mt-4 text-center text-xs text-slate-500">
158                     Administrator? <a href="/admin-login" className="font-semibold text-brand-700 hover:unde
159 rline dark:text-brand-400">[Urdu]Admin panel login</a>
160                 </p>
161             )}
162             {adminMode && (
163                 <p className="mt-4 text-center text-xs text-slate-500">
164                     Not an admin? <a href="/login" className="font-semibold text-brand-700 hover:underline d
165 ark:text-brand-400">Farmer / buyer login</a>
166                 </p>
167             )}
168         <div className={`mt-8 ${adminMode ? "hidden" : ""}`>
169             <p className="text-center text-xs font-semibold tracking-wider text-slate-500 uppercase">Q
170 uick demo access</p>
171             <div className="mt-3 grid gap-2">
172                 {DEMOS.map((d) => (
173                     <button key={d.email} onClick={() => login(d.email, "password123")} disabled={!loadin
174 g} className="card flex items-center gap-3 p-3 text-left transition hover:border-brand-400 hover:shadow-md
175 ">
176                         <span className="text-2xl">{d.icon}</span>
177                         <span className="flex-1">
178                             <span className="block text-sm font-semibold">Continue as {d.label}</span>
179                             <span className="block text-xs text-slate-500">{d.sub}</span>
180                         </span>
181                     </button>
182                 ))}
183             </div>
184         </div>
185     </div>
186 }
187 </div>

```

```

168         {loading === d.email ? <Spinner /> : <span className="text-slate-400"></span>}
169         </button>
170     )}}
171 </div>
172 </div>
173 </>
174 ) : (
175     <>
176         <div className="mt-6">
177             <GoogleButton role={reg.role} next={next} label={`Sign up with Google as ${reg.role === "far
178 mer" ? "Farmer" : "Buyer"}}` />
179             <OrDivider text="or register with email" />
180             <form className="grid grid-cols-2 gap-3" onSubmit={register}>
181                 <div className="col-span-2 grid grid-cols-2 gap-2">
182                     {[["farmer", "buyer"] as const].map((r) => (
183                         <button type="button" key={r} onClick={() => setReg({ ...reg, role: r })} className={`ro
184 unded-xl border-2 p-3 text-sm font-semibold ${reg.role === r ? "border-brand-500 bg-brand-50 dark:bg-brand
185 -500/10" : "border-slate-200 dark:border-slate-700"}`}>
186                         {r === "farmer" ? "Farmer / Seller" : "Buyer / Wholesaler"}
187                     </button>
188                 )]}
189             </div>
190             <div className="col-span-2"><label className="label">Full name</label><input className="input"
191             type="text" required value={reg.fullName} onChange={set("fullName")} /></div>
192             <div className="col-span-2 sm:col-span-1"><label className="label">Email</label><input type="
193             email" className="input" required value={reg.email} onChange={set("email")} /></div>
194             <div className="col-span-2 sm:col-span-1"><label className="label">Password</label><input ty
195             pe="password" minLength={8} className="input" required value={reg.password} onChange={set("password")} /><
196             /div>
197             <div className="col-span-2 sm:col-span-1"><label className="label">Phone</label><input class
198             Name="input" required value={reg.phone} onChange={set("phone")} placeholder="+92 3xx xxxxxxx" /></div>
199             <div className="col-span-2 sm:col-span-1"><label className="label">CNIC</label><input classN
200             ame="input" value={reg.cnicId} onChange={set("cnicId")} placeholder="12345-1234567-1" pattern="\d{5}-\d{7}
201             -\d" /></div>
202             <div className="col-span-2 sm:col-span-1"><label className="label">City</label><select class
203             Name="input" value={reg.city} onChange={set("city")}><CityOptions /></select></div>
204             <div className="col-span-2 sm:col-span-1"><label className="label">{reg.role === "farmer" ?
205             "Farm name" : "Company"}</label><input className="input" value={reg.businessName} onChange={set("businessN
206             ame")} /></div>
207             <div className="col-span-2"><label className="label">Address</label><input className="input"
208             value={reg.address} onChange={set("address")} /></div>
209             <button className="btn-primary col-span-2 mt-2 py-2.5" disabled={!loading}>{loading === "re
210             gister" && <Spinner /> Create account</button>
211         </form>
212     </>
213     )}
214 </div>
215 </div>
216 );
217 }
218 }

```

src/app/login/page.tsx

```

1  import { redirect } from "next/navigation";
2  import { ADMIN_DENIED, getCurrentUser, homeFor } from "@/lib/auth";
3  import { googleConfigured } from "@/lib/google";
4  import { LoginForm, type LoginAlert } from "./login-form";
5
6  export const dynamic = "force-dynamic";
7
8  type SP = { email?: string; role?: string; next?: string; error?: string; msg?: string };
9
10 function alertFor(sp: SP): LoginAlert {
11     switch (sp.error) {
12         case "admin_required":
13             return { kind: "error", title: ADMIN_DENIED, body: "This area is restricted to MarketLink administra
14             tors." };
15         case "google_failed":
16             return { kind: "error", title: "Google sign-in failed", body: sp.msg?.slice(0, 200) };
17         case "google_not_configured":
18             return { kind: "info", title: "Google sign-in isn't set up on this server", body: "An administrator
19             must add GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET. Use email and password for now." };

```

```

18     default:
19         return null;
20     }
21 }
22
23 export default async function LoginPage({ searchParams }: { searchParams: Promise<SP> }) {
24     const [user, sp] = await Promise.all([getCurrentUser(), searchParams]);
25     const alert = alertFor(sp);
26     // Signed-in users go to their dashboard, unless they were bounced here by the admin guard
27     if (user && !alert) redirect(homeFor(user.role));
28     if (user && user.role === "admin" && sp.error === "admin_required") redirect(sp.next ?? homeFor("admin"))
29 );
30     return (
31         <LoginForm
32             initialEmail={sp.email ?? ""}
33             initialRole={sp.role === "farmer" ? "farmer" : "buyer"}
34             startOnRegister={sp.role === "farmer" || sp.role === "buyer"}
35             next={sp.next}
36             alert={alert}
37             signedInAs={user ? { fullName: user.fullName, role: user.role } : null}
38             key={googleConfigured() ? "g" : "n"}
39         />
40     );
41 }

```

src/app/page.tsx

```

1  import Link from "next/link";
2  import { getCurrentUser } from "@lib/auth";
3  import { SiteNav } from "@components/nav/site-nav";
4  import { getViewer } from "@lib/viewer";
5  import { MandiTicker } from "@components/mandi-ticker";
6  import { ChatBot } from "@components/ChatBot";
7  import { getTicker } from "@lib/ticker";
8  import { CROP_CATALOGUE, MANDIS, PROVINCES, CITIES } from "@lib/constants";
9
10 export const dynamic = "force-dynamic";
11
12 const FEATURES = [
13     { icon: "", title: "Direct Crop Listings", ur: "[Urdu]", body: "Kisan list graded produce with photos, harvest date and quantity in kg, maund or tons – visible to verified buyers across Pakistan." },
14     { icon: "", title: "Bidding & Negotiation", ur: "[Urdu]", body: "Wholesalers, supermarkets, exporters and processors send bulk offers. Farmers accept, reject or counter – no arhti commission." },
15     { icon: "", title: "Live Mandi Rates", ur: "[Urdu]", body: "Daily min / max / average rates from Badami Bagh, Multan Ghalla Mandi, Karachi Super Highway and 7 more mandis, per kg and per maund." },
16     { icon: "", title: "Escrow Payments (PKR)", ur: "[Urdu]", body: "Buyer money is locked in escrow and released to the farmer only after delivery is confirmed. Flat 1.5% fee – no hidden cuts." },
17     { icon: "", title: "Truck Tracking", ur: "[Urdu]", body: "Follow every consignment: Placed → Inspected → Dispatched → In Transit (e.g. M-2, N-5) → Delivered, with checkpoint history." },
18     { icon: "[Urdu]", title: "Quality Inspection", ur: "[Urdu]", body: "Inspectors verify CNIC & farm, grade crops A/B/C, record moisture and soil pH, and attach lab reports before dispatch." },
19 ];
20
21 const STEPS = [
22     { n: "1", title: "Kisan lists the crop", ur: "[Urdu]", body: "Photo, grade, quantity and a price suggested from today's mandi rates." },
23     { n: "2", title: "Buyers bid, farmer decides", ur: "[Urdu]", body: "Accept the best offer or send a counter-offer in one tap." },
24     { n: "3", title: "Escrow, inspect, deliver", ur: "[Urdu]", body: "Funds locked, quality checked, truck tracked, payment released on delivery." },
25 ];
26
27 export default async function Home() {
28     const user = await getCurrentUser();
29     const viewer = await getViewer(user);
30     const ticker = await getTicker();
31     const cityCount = Object.keys(CITIES).length;
32     return (
33         <div className="min-h-screen">
34             <SiteNav viewer={viewer} transparent />
35
36             <div>
37                 <div>
38                     <div>

```

```

39     
40     <div className="absolute inset-0 -z-10 bg-gradient-to-r from-brand-950/95 via-brand-900/80 to-bran
↳ d-900/10" />
41     <div className="mx-auto max-w-7xl px-4 pt-32 pb-20 sm:px-6 lg:pt-40 lg:pb-32">
42         <div className="max-w-2xl text-white">
43             <span className="inline-flex items-center gap-2 rounded-full bg-white/10 px-3 py-1 text-xs fon
↳ t-semibold ring-1 ring-white/20 backdrop-blur"> Pakistan's B2B Agri-Commerce Platform</span>
44             <h1 className="mt-6 text-4xl leading-[1.1] font-extrabold tracking-tight sm:text-5xl lg:text-6
↳ xl">
45                 Kisan se khareedar tak.
46                 <span className="block text-gold-400">No middlemen.</span>
47             </h1>
48             <p className="mt-3 text-2xl text-white/90" dir="rtl" lang="ur" style={{ fontFamily: "'Noto Nas
↳ taliq Urdu', serif" }}>[Urdu]– [Urdu]</p>
49             <p className="mt-5 max-w-xl text-lg text-white/80">
50                 MarketLink Agri-Hub connects farmers in Punjab, Sindh, KPK and Balochistan directly with who
↳ lesalers, supermarket chains, exporters and food processors, with transparent mandi rates, PKR escrow and
↳ truck tracking.
51             </p>
52             <div className="mt-8 flex flex-wrap gap-3">
53                 <Link href="/register?role=farmer" className="btn bg-gold-400 px-6 py-3 text-base text-brand
↳ -950 hover:bg-gold-300"> I'm a Kisan (Farmer)</Link>
54                 <Link href="/marketplace" className="btn bg-white/10 px-6 py-3 text-base text-white ring-1 r
↳ ing-white/30 backdrop-blur hover:bg-white/20"> Browse marketplace</Link>
55             </div>
56             <dl className="mt-12 grid max-w-lg grid-cols-3 gap-6 border-t border-white/15 pt-6">
57                 {[`${MANDIS.length}`, "Mandis tracked"], [`${cityCount}+`, "Cities covered"], ["1.5%", "Fla
↳ t fee, no arhti cut"]}.map(([v, l]) => (
58                     <div key={l}>
59                         <dt className="text-3xl font-extrabold text-gold-400">{v}</dt>
60                         <dd className="text-xs text-white/70">{l}</dd>
61                     </div>
62                 )))
63             </dl>
64         </div>
65     </div>
66 </section>
67
68 <MandiTicker items={ticker} variant="dark" />
69
70 {/* HOW IT WORKS */}
71 <section className="mx-auto max-w-7xl px-4 py-16 sm:px-6">
72     <p className="text-center text-sm font-bold tracking-widest text-brand-600 uppercase">How it works
↳ • [Urdu]</p>
73     <h2 className="mt-2 text-center text-3xl font-extrabold tracking-tight sm:text-4xl">Three steps fr
↳ om field to buyer</h2>
74     <div className="mt-10 grid gap-5 md:grid-cols-3">
75         {STEPS.map((s) => (
76             <div key={s.n} className="card relative overflow-hidden p-6">
77                 <span className="absolute -top-4 -right-2 text-8xl font-black text-brand-900/5 dark:text-whi
↳ te/5">{s.n}</span>
78                 <span className="grid h-10 w-10 place-items-center rounded-full bg-brand-900 font-bold text-
↳ gold-400">{s.n}</span>
79                 <h3 className="mt-4 text-lg font-bold">{s.title}</h3>
80                 <p className="text-sm text-brand-700 dark:text-brand-300" dir="rtl">{s.ur}</p>
81                 <p className="mt-2 text-sm text-slate-600 dark:text-slate-400">{s.body}</p>
82             </div>
83         )))
84     </div>
85 </section>
86
87 {/* FEATURES + MANDI IMAGE */}
88 <section className="bg-white py-16 dark:bg-[#0a1a12]">
89     <div className="mx-auto grid max-w-7xl gap-10 px-4 sm:px-6 lg:grid-cols-[1fr_380px]">
90         <div>
91             <h2 className="text-3xl font-extrabold tracking-tight">Built for Pakistani agriculture</h2>
92             <p className="mt-2 text-slate-500 dark:text-slate-400">Prices in PKR, rates per maund (40 kg),
↳ mandi names you know, and Urdu support.</p>
93             <div className="mt-8 grid gap-4 sm:grid-cols-2">
94                 {FEATURES.map((f) => (
95                     <div key={f.title} className="flex gap-4 rounded-2xl border border-slate-100 p-4 transitio
↳ n hover:border-brand-200 hover:bg-brand-50/40 dark:border-white/5 dark:hover:bg-white/5">
96                         <span className="grid h-11 w-11 shrink-0 place-items-center rounded-xl bg-brand-900 text
↳ -xl">{f.icon}</span>
97                     <div>

```

```

98         <h3 className="font-semibold">{f.title} <span className="text-xs font-normal text-bran
↳ d-600" dir="rtl">· {f.ur}</span></h3>
99         <p className="mt-1 text-sm text-slate-600 dark:text-slate-400">{f.body}</p>
100     </div>
101 </div>
102     )})
103 </div>
104 </div>
105     <div className="relative hidden overflow-hidden rounded-3xl shadow-xl lg:block">
106         {/* eslint-disable-next-line @next/next/no-img-element */}
107         
108         <div className="absolute inset-x-0 bottom-0 bg-gradient-to-t from-brand-950/90 to-transparent
↳ p-5 text-white">
109             <p className="text-sm font-semibold">Badami Bagh Mandi, Lahore</p>
110             <p className="text-xs text-white/70">Rates synced daily with 9 other mandis nationwide</p>
111         </div>
112     </div>
113 </div>
114 </section>
115
116     {/* CROPS */}
117     <section className="mx-auto max-w-7xl px-4 py-16 sm:px-6">
118         <h2 className="text-3xl font-extrabold tracking-tight">Crops traded on MarketLink</h2>
119         <div className="mt-8 grid grid-cols-2 gap-4 sm:grid-cols-3 lg:grid-cols-5">
120             {CROP_CATALOGUE.slice(0, 10).map((c) => (
121                 <div key={c.name} className="group relative overflow-hidden rounded-2xl shadow-sm">
122                     {/* eslint-disable-next-line @next/next/no-img-element */}
123                     <img src={c.image} alt={c.name} className="h-36 w-full object-cover transition duration-500
↳ group-hover:scale-110" />
124                     <div className="absolute inset-0 bg-gradient-to-t from-black/80 via-black/20 to-transparent"
↳ />
125                     <div className="absolute inset-x-0 bottom-0 p-3 text-white">
126                         <p className="text-sm font-bold">{c.name} <span className="font-normal opacity-80">{c.urdu
↳ }</span></p>
127                         <p className="text-[11px] text-gold-300">≈ Rs. {(c.mandiBase * 40).toLocaleString("en-US")
↳ }/maund</p>
128                     </div>
129                 </div>
130             )})
131         </div>
132     </section>
133
134     {/* PROVINCES */}
135     <section className="pk-pattern relative bg-brand-900 py-14 text-white">
136         <div className="mx-auto max-w-7xl px-4 sm:px-6">
137             <h2 className="text-center text-2xl font-extrabold sm:text-3xl">Serving every province of Pakist
↳ an</h2>
138             <div className="mt-8 grid grid-cols-2 gap-3 sm:grid-cols-4 lg:grid-cols-7">
139                 {PROVINCES.map((p) => (
140                     <div key={p} className="rounded-2xl bg-white/10 p-4 text-center ring-1 ring-white/10">
141                         <p className="text-sm font-bold">{p}</p>
142                         <p className="mt-1 text-xs text-white/60">{Object.values(CITIES).filter((c) => c.province
↳ === p).length} cities</p>
143                     </div>
144                 )})
145             </div>
146         </div>
147     </section>
148
149     {/* DEMO */}
150     <section className="mx-auto max-w-7xl px-4 py-16 sm:px-6">
151         <div className="card overflow-hidden border-0 bg-gradient-to-br from-gold-400 to-gold-500 p-8 text
↳ -brand-950 sm:p-12">
152             <h2 className="text-2xl font-extrabold sm:text-3xl">Try the live demo</h2>
153             <p className="mt-2 max-w-2xl">Pre-loaded with farmers from Multan, Sahiwal, Hyderabad, Okara & T
↳ urbat, buyers from Lahore, Karachi & Faisalabad. Password for all: <code className="rounded bg-brand-950/1
↳ 0 px-1.5 py-0.5 font-bold">password123</code></p>
154             <div className="mt-6 grid gap-3 sm:grid-cols-3">
155                 {[[" Kisan (Farmer)", "farmer@marketlink.pk"], [" Buyer / Wholesaler", "buyer@marketlink.pk"],
↳ ["[Urdu]Admin / Inspector", "ahmed.mustafa@admin.com"]].map(([r, e]) => (
156                     <Link key={e} href={`/login?email=${e}`} className="rounded-2xl bg-brand-950 p-4 text-white
↳ transition hover:-translate-y-0.5 hover:shadow-xl">
157                         <p className="font-semibold">{r}</p>
158                         <p className="text-sm text-gold-300">{e}</p>
159                     </Link>

```

```

160     )))
161   </div>
162 </div>
163 </section>
164
165   <footer className="border-t border-brand-900/10 dark:border-white/5">
166     <div className="mx-auto flex max-w-7xl flex-col items-center justify-between gap-3 px-4 py-6 text-
167     sm text-slate-500 sm:flex-row sm:px-6">
168       <p>© {new Date().getFullYear()} MarketLink Agri-Hub Pakistan</p>
169       <nav className="flex gap-5">
170         <Link href="/login" className="hover:text-brand-700 dark: hover: text-brand-400">Sign in</Link>
171         <Link href="/admin-login" className="hover:text-brand-700 dark: hover: text-brand-400">Admin</Li
172         nk>
173         <Link href="/deploy" className="hover:text-brand-700 dark: hover: text-brand-400">Developers</Li
174         nk>
175       </nav>
176     </div>
177   </footer>
178   {!!user || user.role === "buyer") && <ChatBot placement="public" userCity={user?.city} />
179   </div>
180 );
181 }
182

```

src/app/pay/sandbox/[reference]/page.tsx

```

1  "use client";
2
3  import { useParams, useRouter } from "next/navigation";
4  import { useEffect, useState } from "react";
5  import { Logo } from "@components/Logo";
6  import { Spinner } from "@components/ui";
7
8  /** Hosted test checkout that mimics a real gateway page. The result goes through the signed server callba
9  ck. */
10 export default function SandboxCheckout() {
11   const { reference } = useParams<{ reference: string }>();
12   const router = useRouter();
13   const [intent, setIntent] = useState<{ amount: number; status: string } | null>(null);
14   const [err, setErr] = useState<string | null>(null);
15   const [busy, setBusy] = useState<string | null>(null);
16
17   useEffect(() => {
18     fetch(`/api/payments/${reference}`).then((r) => r.json()).then((r) => (r.success ? setIntent(r.data) :
19     setErr(r.error))).catch(() => setErr("Could not load payment"));
20   }, [reference]);
21
22   async function pay(outcome: "success" | "failed") {
23     setBusy(outcome);
24     const r = await fetch(`/api/payments/sandbox/${reference}`, { method: "POST", headers: { "content-type
25     ": "application/json" }, body: JSON.stringify({ outcome }) }).then((x) => x.json());
26     if (!r.success) { setErr(r.error); setBusy(null); return; }
27     router.replace(`/wallet?payment=${reference}`);
28   }
29
30   return (
31     <div className="grid min-h-screen place-items-center bg-slate-100 px-4 dark:bg-[#07130d]">
32       <div className="w-full max-w-md overflow-hidden rounded-3xl bg-white shadow-2xl dark:bg-[#0d1f16]">
33         <div className="flex items-center justify-between bg-slate-900 px-6 py-4 text-white">
34           <Logo size={28} tone="light" tagline={false} />
35           <span className="rounded-full bg-amber-400 px-2.5 py-0.5 text-[11px] font-bold text-slate-900">T
36           EST MODE</span>
37         </div>
38         <div className="space-y-5 p-6">
39           {err && <p className="rounded-xl bg-rose-50 p-3 text-sm text-rose-800">{err}</p>}
40           {intent && (
41             <div>
42               <p className="text-sm text-slate-500">Pay MarketLink Agri-Hub</p>
43               <p className="text-3xl font-extrabold">Rs. {intent.amount.toLocaleString("en-US")}</p>
44               <p className="font-mono text-xs text-slate-500">Ref {reference}</p>
45             </div>
46             {intent.status !== "pending" ? (
47               <p className="rounded-xl bg-slate-100 p-3 text-sm dark:bg-white/5">This payment is already
48               <b>{intent.status}</b></p>
49             ) : (
50               <p>...
51             )}
52           )}
53         </div>
54       </div>
55     )
56   )
57 }

```

```

45         ) : (
46             <>
47                 <div className="space-y-3 rounded-2xl border border-slate-200 p-4 dark:border-white/10">
48                     <div><label className="label">Card number</label><input className="input font-mono" de
49 faultValue="4242 4242 4242 4242" readOnly /></div>
50                     <div className="grid grid-cols-2 gap-3">
51                         <div><label className="label">Expiry</label><input className="input font-mono" defau
52 ltValue="12 / 30" readOnly /></div>
53                         <div><label className="label">CVC</label><input className="input font-mono" defaultV
54 alue="123" readOnly /></div>
55                     </div>
56                     <button className="btn-primary w-full py-3 text-base" disabled={!busy} onClick={() => p
57 ay("success")}>{busy === "success" && <Spinner />} Pay Rs. {intent.amount.toLocaleString("en-US")}</button
58 >
59                     <button className="btn-secondary w-full" disabled={!busy} onClick={() => pay("failed")}
60 >{busy === "failed" && <Spinner />} Simulate declined payment</button>
61                 </div>
62             </div>
63             <p className="text-center text-[11px] text-slate-500">No real money is charged. Configure St
64 ripe, JazzCash or Easypaisa keys for live payments.</p>
65         </div>
66     </div>
67     </div>
68     </div>
69     </div>
70     </div>
71     </div>
72     </div>
73     </div>
74     </div>
75     </div>
76     </div>
77     </div>
78     </div>
79     </div>
80     </div>
81     </div>
82     </div>
83     </div>
84     </div>
85     </div>
86     </div>
87     </div>
88     </div>
89     </div>
90     </div>
91     </div>
92     </div>
93     </div>
94     </div>
95     </div>
96     </div>
97     </div>
98     </div>
99     </div>
100    </div>
101    </div>
102    </div>
103    </div>
104    </div>
105    </div>
106    </div>
107    </div>
108    </div>
109    </div>
110    </div>
111    </div>
112    </div>
113    </div>
114    </div>
115    </div>
116    </div>
117    </div>
118    </div>
119    </div>
120    </div>
121    </div>
122    </div>
123    </div>
124    </div>
125    </div>
126    </div>
127    </div>
128    </div>
129    </div>
130    </div>
131    </div>
132    </div>
133    </div>
134    </div>
135    </div>
136    </div>
137    </div>
138    </div>
139    </div>
140    </div>
141    </div>
142    </div>
143    </div>
144    </div>
145    </div>
146    </div>
147    </div>
148    </div>
149    </div>
150    </div>
151    </div>
152    </div>
153    </div>
154    </div>
155    </div>
156    </div>
157    </div>
158    </div>
159    </div>
160    </div>
161    </div>
162    </div>
163    </div>
164    </div>
165    </div>
166    </div>
167    </div>
168    </div>
169    </div>
170    </div>
171    </div>
172    </div>
173    </div>
174    </div>
175    </div>
176    </div>
177    </div>
178    </div>
179    </div>
180    </div>
181    </div>
182    </div>
183    </div>
184    </div>
185    </div>
186    </div>
187    </div>
188    </div>
189    </div>
190    </div>
191    </div>
192    </div>
193    </div>
194    </div>
195    </div>
196    </div>
197    </div>
198    </div>
199    </div>
200    </div>
201    </div>
202    </div>
203    </div>
204    </div>
205    </div>
206    </div>
207    </div>
208    </div>
209    </div>
210    </div>
211    </div>
212    </div>
213    </div>
214    </div>
215    </div>
216    </div>
217    </div>
218    </div>
219    </div>
220    </div>
221    </div>
222    </div>
223    </div>
224    </div>
225    </div>
226    </div>
227    </div>
228    </div>
229    </div>
230    </div>
231    </div>
232    </div>
233    </div>
234    </div>
235    </div>
236    </div>
237    </div>
238    </div>
239    </div>
240    </div>
241    </div>
242    </div>
243    </div>
244    </div>
245    </div>
246    </div>
247    </div>
248    </div>
249    </div>
250    </div>
251    </div>
252    </div>
253    </div>
254    </div>
255    </div>
256    </div>
257    </div>
258    </div>
259    </div>
260    </div>
261    </div>
262    </div>
263    </div>
264    </div>
265    </div>
266    </div>
267    </div>
268    </div>
269    </div>
270    </div>
271    </div>
272    </div>
273    </div>
274    </div>
275    </div>
276    </div>
277    </div>
278    </div>
279    </div>
280    </div>
281    </div>
282    </div>
283    </div>
284    </div>
285    </div>
286    </div>
287    </div>
288    </div>
289    </div>
290    </div>
291    </div>
292    </div>
293    </div>
294    </div>
295    </div>
296    </div>
297    </div>
298    </div>
299    </div>
300    </div>
301    </div>
302    </div>
303    </div>
304    </div>
305    </div>
306    </div>
307    </div>
308    </div>
309    </div>
310    </div>
311    </div>
312    </div>
313    </div>
314    </div>
315    </div>
316    </div>
317    </div>
318    </div>
319    </div>
320    </div>
321    </div>
322    </div>
323    </div>
324    </div>
325    </div>
326    </div>
327    </div>
328    </div>
329    </div>
330    </div>
331    </div>
332    </div>
333    </div>
334    </div>
335    </div>
336    </div>
337    </div>
338    </div>
339    </div>
340    </div>
341    </div>
342    </div>
343    </div>
344    </div>
345    </div>
346    </div>
347    </div>
348    </div>
349    </div>
350    </div>
351    </div>
352    </div>
353    </div>
354    </div>
355    </div>
356    </div>
357    </div>
358    </div>
359    </div>
360    </div>
361    </div>
362    </div>
363    </div>
364    </div>
365    </div>
366    </div>
367    </div>
368    </div>
369    </div>
370    </div>
371    </div>
372    </div>
373    </div>
374    </div>
375    </div>
376    </div>
377    </div>
378    </div>
379    </div>
380    </div>
381    </div>
382    </div>
383    </div>
384    </div>
385    </div>
386    </div>
387    </div>
388    </div>
389    </div>
390    </div>
391    </div>
392    </div>
393    </div>
394    </div>
3
```

src/app/register/page.tsx

```

1 import { redirect } from "next/navigation";
2 import { getCurrentUser, homeFor } from "@lib/auth";
3 import { LoginForm } from "@app/login/login-form";
4
5 export const dynamic = "force-dynamic";
6 export const metadata = { title: "Create account" };
7
8 export default async function RegisterPage({ searchParams }: { searchParams: Promise<{ role?: string }> })
9 {
10   const user = await getCurrentUser();
11   if (user) redirect(homeFor(user.role));
12   const { role } = await searchParams;
13   return <LoginForm initialEmail="" initialRole={role === "buyer" ? "buyer" : "farmer"} startOnRegister />
14   ;
15 }

```

src/app/reset-password/page.tsx

```

1 import { redirect } from "next/navigation";
2 import { ResetPasswordForm } from "@components/auth/otp-forms";
3
4 export const metadata = { title: "Reset password" };
5
6 export default async function ResetPasswordPage({ searchParams }: { searchParams: Promise<{ token?: string
7   }> }) {
8   const { token } = await searchParams;
9   if (!token) redirect("/forgot-password");
10  return <ResetPasswordForm token={token} />;
11 }

```

src/app/verify-email/page.tsx

```
1 import { redirect } from "next/navigation";
2 import { VerifyEmailForm } from "@components/auth/otp-forms";
3
4 export const metadata = { title: "Verify email" };
5
6 export default async function VerifyEmailPage({ searchParams }: { searchParams: Promise<{ email?: string;
7   dev?: string }> }) {
8   const { email, dev } = await searchParams;
9   if (!email) redirect("/register");
10  return <VerifyEmailForm email={email} initialDevCode={dev} />;
11 }
```

src/components/ChatBot.tsx

```
1 "use client";
2
3 import Link from "next/link";
4 import { useCallback, useEffect, useRef, useState } from "react";
5 import { Logo } from "@components/Logo";
6 import type { AssistantCard, AssistantReply, DealerCard, ListingCard, RateCard } from "@lib/services/assistant";
7
8 /* ----- */
9 /* Types & config */
10 /* ----- */
11 type Msg = { id: string; role: "user" | "bot"; text: string; cards?: AssistantCard[]; suggestions?: string
12   []; link?: { label: string; href: string }; offline?: boolean };
13
14 export type ChatBotProps = {
15   /** "app" leaves room for the mobile bottom navigation; "public" sits in the corner */
16   placement?: "app" | "public";
17   /** Buyer's home city: used for greeting context */
18   userCity?: string;
19   /** Endpoint that returns AssistantReply JSON (override to plug in an LLM backend) */
20   endpoint?: string;
21   title?: string;
22 };
23
24 const STORAGE_KEY = "ml-assistant-v1";
25 const HINT_KEY = "ml-assistant-hint-seen";
26 const STARTERS = ["Top-rated dealers", "High-quality fruits", "Cheapest wheat", "Mango mandi rate", "How d
27   oes escrow work?"];
28
29 const uid = () => Math.random().toString(36).slice(2, 10);
30 const rs = (n: number) => `Rs. ` + Math.round(n).toLocaleString("en-US");
31
32 /** Offline fallback used when the assistant API is unreachable. */
33 function mockReply(input: string): AssistantReply {
34   const t = input.toLowerCase();
35   if (/dealer|seller|farmer|supplier|rated/.test(t))
36     return { reply: "Top-rated sellers are verified farmers with the most completed deliveries and clean d
37       ispute records. Open the marketplace and look for the ✓ verified badge and inspected listings.", suggesti
38       ons: ["High-quality grains", "How does escrow work?"] };
39   if (/quality|grade|premium|best/.test(t))
40     return { reply: "For the best quality, filter the marketplace by Grade A and prefer listings marked I
41       nspected. Those passed a MarketLink quality check (moisture, foreign matter, size).", suggestions: ["Top-r
42       ated dealers", "Cheapest wheat"] };
43   if (/rate|mandi|price|bhav/.test(t))
44     return { reply: "Live mandi rates (per kg and per 40 kg maund) for 10 Pakistani mandis are on the Mand
45       i Insights page, and the ticker at the top shows today's averages.", suggestions: ["Top-rated dealers"] };
46   if (/escrow|pay|refund/.test(t))
47     return { reply: "Your money is held in escrow and released to the farmer only after you confirm delive
48       ry. Disputes freeze the funds until an admin resolves them.", suggestions: ["How do I place a bid?"] };
49   return { reply: "I'm having trouble reaching the marketplace right now. Please try again in a moment, or
50     browse the marketplace directly.", suggestions: STARTERS.slice(0, 3) };
51 }
52
53 /* ----- */
54 /* Result cards */
55 /* ----- */
56 function Stars({ value }: { value: number }) {
```

```

48     return (
49       <span className="inline-flex items-center gap-0.5" aria-label={` ${value} out of 5 stars`} >
50         {[1, 2, 3, 4, 5].map((i) => {
51           const fill = Math.max(0, Math.min(1, value - (i - 1)));
52           return (
53             <svg key={i} viewBox="0 0 20 20" className="h-3.5 w-3.5" aria-hidden>
54               <defs>
55                 <linearGradient id={` ${i}-${Math.round(fill * 100)} `}>
56                   <stop offset={` ${fill * 100}% `} stopColor="#f1c232" />
57                   <stop offset={` ${fill * 100}% `} stopColor="currentColor" stopOpacity="0.2" />
58                 </linearGradient>
59               </defs>
60               <path d="M10 1.8l2.5 5.2 5.7 8.4 4.1 4 1 5.7L10 14.8l-5.1 2.7 1-5.7-4.1-4 5.7-.8z" fill={` url(#s
61               ${i}-${Math.round(fill * 100)} `} />
62             </svg>
63           );
64         })}
65       </span>
66     );
67   }
68   function ListingItem({ c, onNavigate }: { c: ListingCard; onNavigate: () => void }) {
69     return (
70       <Link href={c.href} onClick={onNavigate} className="group flex gap-3 rounded-2xl border border-slate-2
71       00 bg-white p-2.5 transition hover:border-brand-400 hover:shadow-md dark:border-white/10 dark:bg-white/5">
72         <img src={c.image} alt="" className="h-16 w-16 shrink-0 rounded-xl object-cover" />
73         <div className="min-w-0 flex-1">
74           <div className="flex items-center gap-1.5">
75             <p className="truncate text-sm font-semibold">{c.title}</p>
76             <span className={` shrink-0 rounded px-1 text-[10px] font-bold text-white ${c.grade === "A" ? "bg
77             -emerald-600" : c.grade === "B" ? "bg-amber-500" : "bg-slate-500"} `}>{c.grade}</span>
78             {c.inspected && <span className="shrink-0 text-[10px] font-semibold text-emerald-700 dark:text-e
79             merald-400"> Inspected</span>}
80           </div>
81           <p className="truncate text-xs text-slate-500 dark:text-slate-400">
82             {c.seller}{c.verified && <span className="text-brand-600"> ✓</span>} · {c.location}{c.distanceKm
83             !== null ? ` · ${c.distanceKm} km` : ""}
84           </p>
85           <div className="mt-1 flex items-baseline justify-between gap-2">
86             <p className="text-sm font-bold text-brand-700 dark:text-brand-400">{rs(c.pricePerKg)}<span clas
87             sName="text-[11px] font-normal text-slate-500">/kg</span></p>
88             <p className="text-[11px] text-slate-500">{rs(c.pricePerMaunder)}maunder</p>
89           </div>
90         </div>
91       </Link>
92     );
93   }
94   function DealerItem({ c, onNavigate }: { c: DealerCard; onNavigate: () => void }) {
95     const initials = c.name.split(" ").map((p) => p[0]).slice(0, 2).join("");
96     return (
97       <Link href={c.href} onClick={onNavigate} className="flex gap-3 rounded-2xl border border-slate-200 bg-
98       white p-3 transition hover:border-brand-400 hover:shadow-md dark:border-white/10 dark:bg-white/5">
99         <span className="grid h-11 w-11 shrink-0 place-items-center rounded-full bg-brand-900 text-sm font-b
100         old text-gold-400">{initials}</span>
101         <div className="min-w-0 flex-1">
102           <p className="flex items-center gap-1 text-sm font-semibold">
103             <span className="truncate">{c.business ?? c.name}</span>
104             {c.verified && <span className="shrink-0 text-xs text-brand-600" title="Verified seller" aria-la
105             bel="Verified seller">✓</span>}
106           </p>
107           <div className="flex flex-wrap items-center gap-x-1.5 text-xs text-slate-500 dark:text-slate-400">
108             <Stars value={c.rating} />
109             <span className="font-semibold text-slate-700 dark:text-slate-200">{c.rating.toFixed(1)}</span>
110             <span className="whitespace-nowrap">· {c.deliveries} deliveries · {c.city}</span>
111           </div>
112           <p className="mt-0.5 truncate text-[11px] text-slate-500">{c.crops}</p>
113         </div>
114       </Link>
115     );
116   }
117   function RateItem({ c }: { c: RateCard }) {
118     return (
119       <div className="rounded-2xl border border-slate-200 bg-white p-3 dark:border-white/10 dark:bg-white/5"

```

```

115     <div className="flex items-baseline justify-between">
116       <p className="text-sm font-semibold">{c.crop}</p>
117       {c.changePct !== null && <span className={`text-xs font-semibold ${c.changePct >= 0 ? "text-emerald-600" : "text-rose-600"}`}>{c.changePct >= 0 ? "▲" : "▼"} {Math.abs(c.changePct).toFixed(1)}%</span>}
118     </div>
119     <p className="text-lg font-extrabold text-brand-700 dark:text-brand-400">{rs(c.avgPerMaund)}<span className="text-xs font-normal text-slate-500"> /maund avg</span></p>
120     <div className="mt-1.5 grid grid-cols-2 gap-2 text-[11px]">
121       <div className="rounded-lg bg-emerald-50 px-2 py-1 dark:bg-emerald-500/10"><p className="text-emerald-700 dark:text-emerald-300">Lowest · {rs(c.low.perMaund)}</p><p className="truncate text-slate-500">{c.low.mandi}</p></div>
122       <div className="rounded-lg bg-amber-50 px-2 py-1 dark:bg-amber-500/10"><p className="text-amber-700 dark:text-amber-300">Highest · {rs(c.high.perMaund)}</p><p className="truncate text-slate-500">{c.high.mandi}</p></div>
123     </div>
124   </div>
125   );
126 }
127
128 /* ----- */
129 /* Widget */
130 /* ----- */
131 export function ChatBot({ placement = "public", userCity, endpoint = "/api/assistant", title = "MARKETLINK AGRI-HUB" }: ChatBotProps) {
132   const greeting = useCallback(
133     (): Msg => ({
134       id: "hello",
135       role: "bot",
136       text: `Assalam-o-Alaikum! I can help you find trusted sellers, high-quality produce, the best price
137       s${userCity ? `near ${userCity}` : ""}, and today's mandi rates.` ,
138       suggestions: STARTERS,
139     }),
140     [userCity],
141   );
142   const [open, setOpen] = useState(false);
143   const [msgs, setMsgs] = useState<Msg[]>([]);
144   const [input, setInput] = useState("");
145   const [typing, setTyping] = useState(false);
146   const [hint, setHint] = useState(false);
147   const listRef = useRef<HTMLDivElement>(null);
148   const inputRef = useRef<HTMLTextAreaElement>(null);
149
150   // Restore conversation (per browser tab session)
151   useEffect(() => {
152     try {
153       const saved = sessionStorage.getItem(STORAGE_KEY);
154       // eslint-disable-next-line react-hooks/set-state-in-effect
155       setMsgs(saved ? (JSON.parse(saved) as Msg[]) : [greeting()]);
156     } catch {
157       setMsgs([greeting()]);
158     }
159     if (!localStorage.getItem(HINT_KEY)) {
160       const id = setTimeout(() => setHint(true), 4000);
161       return () => clearTimeout(id);
162     }
163   }, [greeting]);
164
165   useEffect(() => {
166     if (msgs.length) sessionStorage.setItem(STORAGE_KEY, JSON.stringify(msgs.slice(-40)));
167   }, [msgs]);
168
169   useEffect(() => {
170     listRef.current?.scrollTo({ top: listRef.current.scrollHeight, behavior: "smooth" });
171   }, [msgs, typing, open]);
172
173   useEffect(() => {
174     if (!open) return;
175     inputRef.current?.focus();
176     const onKey = (e: KeyboardEvent) => e.key === "Escape" && setOpen(false);
177     window.addEventListener("keydown", onKey);
178     return () => window.removeEventListener("keydown", onKey);
179   }, [open]);
180
181   function toggle() {

```

```

182     setOpen((v) => !v);
183     setHint(false);
184     localStorage.setItem(HINT_KEY, "1");
185   }
186
187   async function send(raw: string) {
188     const text = raw.trim().slice(0, 500);
189     if (!text || typing) return;
190     setInput("");
191     setMsgs((m) => [...m, { id: uid(), role: "user", text }]);
192     setTyping(true);
193     const started = Date.now();
194     let reply: AssistantReply;
195     let offline = false;
196     try {
197       const res = await fetch(endpoint, { method: "POST", headers: { "content-type": "application/json" },
198         body: JSON.stringify({ message: text, history: msgs.slice(-8).map((m) => ({ role: m.role === "bot" ? "assistant" : "user", content: m.text })) }) });
199       const data = await res.json();
200       if (!res.ok || !data.success) throw new Error(data.error || "Request failed");
201       reply = data.data as AssistantReply;
202     } catch {
203       reply = mockReply(text);
204       offline = true;
205     }
206     // Small natural delay so the typing indicator doesn't flash
207     const wait = Math.max(0, 450 - (Date.now() - started));
208     await new Promise((r) => setTimeout(r, wait));
209     setTyping(false);
210     setMsgs((m) => [...m, { id: uid(), role: "bot", text: reply.reply, cards: reply.cards, suggestions: reply.suggestions, link: reply.link, offline }]);
211   }
212
213   function reset() {
214     sessionStorage.removeItem(STORAGE_KEY);
215     setMsgs([greeting()]);
216   }
217
218   const bottom = placement === "app" ? "bottom-20 lg:bottom-6" : "bottom-5 sm:bottom-6";
219   const lastBot = [...msgs].reverse().find((m) => m.role === "bot");
220
221   return (
222     <div dir="ltr">
223       { /* Chat window */ }
224       { open && (
225         <section
226           role="dialog"
227           aria-label={title}
228           className={`animate-fade-in fixed right-3 z-[60] flex w-[calc(100vw-1.5rem)] origin-bottom-right
229             flex-col overflow-hidden rounded-3xl border border-slate-200 bg-slate-50 shadow-2xl shadow-brand-950/20 sm:right-6 sm:w-[400px] dark:border-white/10 dark:bg-[#0a1a12] ${
230               placement === "app" ? "bottom-36 h-[min(600px,calc(100dvh-10.5rem))] lg:bottom-24 lg:h-[min(640px,calc(100dvh-8rem))]" : "bottom-24 h-[min(640px,calc(100dvh-8rem))]"
231             }`}
232         >
233           <header className="pk-pattern relative flex items-center gap-3 bg-gradient-to-br from-brand-800 to-brand-950 px-4 py-3.5 text-white">
234             <Logo variant="mark" size={36} />
235             <div className="min-w-0 flex-1">
236               <p className="truncate text-sm font-bold">{title}</p>
237               <p className="flex items-center gap-1.5 text-[11px] text-white/70">
238                 <span className="h-1.5 w-1.5 rounded-full bg-emerald-400" /> Shopping assistant · Online
239               </p>
240             </div>
241             <button onClick={reset} className="rounded-lg p-2 text-white/70 hover:bg-white/10 hover:text-white" title="New conversation" aria-label="New conversation">
242               <svg viewBox="0 0 24 24" className="h-4 w-4" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"><path d="M3 12a9 9 0 1 0 3-6.7L3 8" /><path d="M3 3v5h5" /></svg>
243             </button>
244             <button onClick={() => setOpen(false)} className="rounded-lg p-2 text-white/70 hover:bg-white/10 hover:text-white" aria-label="Close chat">
245               <svg viewBox="0 0 24 24" className="h-4 w-4" fill="none" stroke="currentColor" strokeWidth="2.2" strokeLinecap="round"><path d="M6 6l12 12M18 6 6 18" /></svg>
246             </button>
247           </header>

```

```

247     <div ref={listRef} className="flex-1 space-y-4 overflow-y-auto px-4 py-4" aria-live="polite">
248       {msgs.map((m) =>
249         m.role === "user" ? (
250           <div key={m.id} className="flex justify-end">
251             <p className="max-w-[80%] rounded-2xl rounded-br-md bg-brand-700 px-3.5 py-2 text-sm whi
252 tespace-pre-line text-white shadow-sm">{m.text}</p>
253           </div>
254         ) : (
255           <div key={m.id} className="flex gap-2">
256             <span className="mt-0.5 shrink-0"><Logo variant="mark" size={26} /></span>
257             <div className="min-w-0 flex-1 space-y-2">
258               <p className="w-fit max-w-full rounded-2xl rounded-tl-md bg-white px-3.5 py-2 text-sm
259 whitespace-pre-line text-slate-800 shadow-sm ring-1 ring-slate-200/70 dark:bg-white/5 dark:text-slate-100
260 dark:ring-white/10">
261                 {m.text}
262                 {m.offline && <span className="mt-1 block text-[10px] text-amber-600">Offline answer
263               </span>}
264             </p>
265             {m.cards && m.cards.length > 0 && (
266               <div className="space-y-2">
267                 {m.cards.map((c, i) =>
268                   c.kind === "listing" ? <ListingItem key={i} c={c} onNavigate={() => setOpen(fals
269 e)} />
270                   : c.kind === "dealer" ? <DealerItem key={i} c={c} onNavigate={() => setOpen(fals
271 e)} />
272                   : <RateItem key={i} c={c} />,
273                 )}
274               </div>
275             )}
276             {m.link && (
277               <Link href={m.link.href} onClick={() => setOpen(false)} className="inline-flex items
278 -center gap-1 text-xs font-semibold text-brand-700 hover:underline dark:text-brand-400">
279                 {m.link.label} →
280               </Link>
281             )}
282             {m === lastBot && m.suggestions && m.suggestions.length > 0 && !typing && (
283               <div className="flex flex-wrap gap-1.5 pt-1">
284                 {m.suggestions.map((s) => (
285                   <button
286                     key={s}
287                     onClick={() => send(s)}
288                     className="rounded-full border border-brand-200 bg-white px-3 py-1 text-xs fon
289 t-medium text-brand-800 transition hover:border-brand-500 hover:bg-brand-50 dark:border-brand-800 dark:bg-
290 transparent dark:text-brand-300 dark:hover:bg-brand-950"
291                   >
292                     {s}
293                   </button>
294                 ))}
295               </div>
296             )}
297           </div>
298         ),
299       )}
300     </div>
301     {typing && (
302       <div className="flex gap-2">
303         <Logo variant="mark" size={26} />
304         <span className="flex items-center gap-1 rounded-2xl rounded-tl-md bg-white px-3.5 py-3 sh
305 adow-sm ring-1 ring-slate-200/70 dark:bg-white/5 dark:ring-white/10" aria-label="Assistant is typing">
306           {[0, 1, 2].map((i) => <span key={i} className="h-1.5 w-1.5 animate-bounce rounded-full b
307 g-brand-600" style={{ animationDelay: `${i * 120}ms` }} />)}
308         </span>
309       </div>
310     )}
311   </div>
312   <form
313     onSubmit={(e) => { e.preventDefault(); send(input); }}
314     className="flex items-end gap-2 border-t border-slate-200 bg-white p-3 dark:border-white/10 da
315 rk:bg-[#0d1f16]"
316   >
317     <textarea
318       ref={inputRef}
319       rows={1}
320       value={input}
321       maxLength={500}

```

```

311         onChange={(e) => setInput(e.target.value)}
312         onKeyDown={(e) => { if (e.key === "Enter" && !e.shiftKey) { e.preventDefault(); send(input);
    ↵     } }}
313         placeholder="Ask about crops, sellers or prices..."
314         aria-label="Message"
315         className="max-h-28 min-h-[40px] flex-1 resize-none rounded-xl border border-slate-200 bg-slate-50
    ↵         px-3 py-2.5 text-sm outline-none placeholder:text-slate-400 focus:border-brand-500 focus:ring-2 focus:ring-brand-500/20 dark:border-white/10 dark:bg-white/5"
316     />
317     <button type="submit" disabled={!input.trim() || typing} className="grid h-10 w-10 shrink-0 place-items-center rounded-xl bg-brand-700 text-white transition hover:bg-brand-800 disabled:opacity-40" aria-label="Send">
    ↵
    ↵     <svg viewBox="0 0 24 24" className="h-4 w-4" fill="currentColor"><path d="M3.4 20.4 21 12 3.4 3.6 3.4 10 12 6 2 12.6 2z" /></svg>
318     </button>
319   </form>
320 </section>
321 </div>
322 </div>
323
324   { /* Hint bubble */
325   { hint && !open && (
326     <button onClick={toggle} className={`animate-fade-in fixed right-24 z-[60] hidden max-w-[220px] rounded-2xl rounded-br-sm bg-white px-4 py-2.5 text-left text-sm shadow-xl ring-1 ring-slate-200 sm:block dark:bg-[#0d1f16] dark:ring-white/10 ${placement === "app" ? "bottom-24 lg:bottom-9" : "bottom-8"}`} >
    ↵
    ↵     <span className="font-semibold">Need help finding produce?</span>
327     <span className="block text-xs text-slate-500">Ask me for top sellers or today's rates.</span>
328   </button>
329   ) }
330 </div>
331
332   { /* Floating action button */
333   <button
334     onClick={toggle}
335     aria-label={open ? "Close assistant" : "Open shopping assistant"}
336     aria-expanded={open}
337     className={`fixed right-4 z-[60] grid h-14 w-14 place-items-center rounded-full bg-gradient-to-br from-brand-600 to-brand-900 text-white shadow-xl shadow-brand-900/30 ring-4 ring-white transition hover:scale-105 focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-gold-400 sm:right-6 dark:ring-[#07130d] ${bottom}`}
    ↵
    ↵
    ↵     >
338     { open ? (
339       <svg viewBox="0 0 24 24" className="h-6 w-6" fill="none" stroke="currentColor" strokeWidth="2.2" strokeLinecap="round"><path d="M6 6 12 12 18 6 6 18" /></svg>
    ↵
    ↵       ) : (
340       <svg viewBox="0 0 24 24" className="h-6 w-6" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
    ↵
    ↵         <path d="M21 12a8 8 0 0 1 -11.6 7.1L4 20 11.4 4.6A8 8 0 1 1 21 12Z" />
344         <path d="M8.5 11h.01M12 11h.01M15.5 11h.01" strokeWidth="2.8" />
345       </svg>
346       <span className="absolute -top-0.5 -right-0.5 h-3.5 w-3.5 rounded-full bg-gold-400 ring-2 ring-white dark:ring-[#07130d]" />
347     </>
348     ) }
349   </button>
350 </div>
351 </div>
352 );
353 }
354

```

src/components/Logo.tsx

```

1  type LogoProps = {
2    /** "full" displays the brand lockup; "mark" shows only the infinity/wheat symbol. */
3    variant?: "full" | "mark";
4    /** Height of the vector symbol in pixels. */
5    size?: number;
6    /** Force white text when placed over a dark/green surface. */
7    tone?: "auto" | "light";
8    /** Add a small Pakistan label beneath the wordmark for large placements. */
9    tagline?: boolean;
10   className?: string;
11 };
12
13 /**

```

```

14  * MARKETLINK AGRI-HUB logo. One transparent SVG symbol is shared by the
15  * navigation, dashboards, auth screens, chatbot, favicon and email templates.
16  * The lockup stays readable in both light and dark themes.
17  */
18  export function Logo({ variant = "full", size = 38, tone = "auto", tagline = false, className = "" }: Logo
  Props) {
19      const image = (
20          <span
21              className="inline-grid shrink-0 place-items-center overflow-hidden rounded-xl bg-white shadow-sm rin
  g-1 ring-brand-900/10"
22              style={{ width: Math.round(size * 1.64), height: size, padding: Math.max(2, Math.round(size * 0.04))
  }}
23          >
24              {/* eslint-disable-next-line @next/next/no-img-element */}
25              
32          </span>
33      );
34
35      if (variant === "mark") return <span className={`inline-flex ${className}`}>{image}</span>;
36
37      return (
38          <span className={`inline-flex items-center gap-2.5 whitespace-nowrap ${className}`}>
39              {image}
40              <span className="block leading-none">
41                  <span
42                      className={`block font-black tracking-[0.035em] ${tone === "light" ? "text-white" : "text-brand-
  950 dark:text-white"}`}
43                      style={{ fontSize: Math.max(14, Math.round(size * 0.42)) }}
44                  >
45                      MARKETLINK
46                  </span>
47                  <span
48                      className="mt-1 flex items-center gap-1.5 font-bold tracking-[0.27em] text-[#d69924]"
49                      style={{ fontSize: Math.max(9, Math.round(size * 0.25)) }}
50                  >
51                      AGRI-HUB
52                      {tagline && <span className={`font-medium tracking-[0.04em] ${tone === "light" ? "text-white/60"
  : "text-slate-400"}`}>PAKISTAN</span>}
53                  </span>
54              </span>
55          </span>
56      );
57  }
58

```

src/components/admin/orders-table.tsx

```

1  "use client";
2
3  import { useCallback, useEffect, useMemo, useState } from "react";
4  import { api, Badge, Drawer, EmptyState, PageHeader, Spinner } from "@components/ui";
5  import { LogisticsStepper } from "@components/stepper";
6  import { useToast } from "@components/providers";
7  import type { OrderRow } from "@components/orders-board";
8  import { CITIES, CITY_NAMES, DELIVERY_STAGES, STATUS_LABELS, cropImage, formatDate, formatKg, formatPKR, f
  ormatPKRShort, nextStage, stageIndex } from "@lib/constants";
9
10 type Filter = "all" | "inspection" | "transit" | "escrow" | "disputed" | "done";
11 type Sort = "newest" | "oldest" | "amount_desc" | "amount_asc";
12
13 const PAGE_SIZE = 10;
14
15 const FILTERS: { key: Filter; label: string; match: (o: OrderRow) => boolean }[] = [
16     { key: "all", label: "All", match: () => true },
17     { key: "inspection", label: "Needs inspection", match: (o) => o.paymentStatus === "escrow_locked" && o.d
  eliveryStage === "confirmed" },
18     { key: "transit", label: "In transit", match: (o) => o.paymentStatus === "escrow_locked" && ["quality_ch
  ecked", "dispatched", "in_transit"].includes(o.deliveryStage) },
19     { key: "escrow", label: "Awaiting escrow", match: (o) => o.paymentStatus === "awaiting_escrow" },

```

```

20   { key: "disputed", label: "Disputed", match: (o) => o.paymentStatus === "disputed" },
21   { key: "done", label: "Completed", match: (o) => o.paymentStatus === "released" || o.paymentStatus === "
  ↵ refunded" },
22 ];
23
24 const EVENT_LABEL: Record<string, string> = {
25   confirmed: "Order placed",
26   escrow_locked: "Escrow funded",
27   quality_checked: "Quality inspected",
28   dispatched: "Dispatched",
29   in_transit: "In transit",
30   location_update: "Checkpoint update",
31   delivered: "Delivered",
32   escrow_released: "Payment released",
33   escrow_refunded: "Buyer refunded",
34   dispute_opened: "Dispute opened",
35 };
36
37 /** Next action an admin can take on an order, or null. */
38 function adminAction(o: OrderRow): { label: string; icon: string } | null {
39   if (o.paymentStatus !== "escrow_locked") return null;
40   const next = nextStage(o.deliveryStage);
41   if (next === "quality_checked") return { label: "Inspect", icon: "" };
42   if (next === "dispatched") return { label: "Dispatch", icon: "" };
43   if (next === "in_transit") return { label: "In transit", icon: "" };
44   if (next === "delivered") return { label: "Deliver", icon: "" };
45   return null;
46 }
47
48 function ProgressBar({ o }: { o: OrderRow }) {
49   const idx = stageIndex(o.deliveryStage);
50   const disputed = o.paymentStatus === "disputed";
51   const refunded = o.paymentStatus === "refunded";
52   return (
53     <div className="w-32">
54       <div className="flex gap-1" aria-hidden>
55         {DELIVERY_STAGES.map((s, i) => (
56           <span
57             key={s.key}
58             className={`h-1.5 flex-1 rounded-full ${
59               i <= idx ? (disputed ? "bg-rose-500" : refunded ? "bg-violet-400" : "bg-brand-600") : "bg-slate-200 dark:bg-white/10"
60             }`}
61           >
62         ))}
63       </div>
64       <p className="mt-1.5 text-xs text-slate-500 dark:text-slate-400">{STATUS_LABELS[o.deliveryStage]}</p>
65     </div>
66   );
67 }
68
69 export function AdminOrdersTable() {
70   const { push } = useToast();
71   const [orders, setOrders] = useState<OrderRow[] | null>(null);
72   const [filter, setFilter] = useState<Filter>("all");
73   const [sort, setSort] = useState<Sort>("newest");
74   const [q, setQ] = useState("");
75   const [page, setPage] = useState(1);
76   const [openId, setOpenId] = useState<number | null>(null);
77
78   const load = useCallback(
79     () =>
80       api<OrderRow[]>("/api/orders")
81         .then(setOrders)
82         .catch((e) => push({ kind: "error", title: "Could not load orders", body: (e as Error).message })),
83     [push],
84   );
85   useEffect(() => {
86     load();
87   }, [load]);
88
89   const rows = useMemo(() => {
90     const f = FILTERS.find((x) => x.key === filter!);
91     const term = q.trim().toLowerCase();

```

```

92     const list = (orders ?? []).filter(
93       (o) =>
94         f.match(o) &&
95         (!term || [o.trackingNumber, o.cropName, o.farmerName, o.farmName, o.buyerName, o.buyerBusiness, o
96         .farmLocation, o.buyerCity].some((v) => v?.toLowerCase().includes(term))),
97     );
98     return list.sort((a, b) =>
99       sort === "newest" ? +new Date(b.createdAt) - +new Date(a.createdAt)
100       : sort === "oldest" ? +new Date(a.createdAt) - +new Date(b.createdAt)
101       : sort === "amount_desc" ? b.totalAmount - a.totalAmount
102       : a.totalAmount - b.totalAmount,
103     );
104   }, [orders, filter, q, sort]);
105   useEffect(() => setPage(1), [filter, q, sort]);
106
107   const pages = Math.max(1, Math.ceil(rows.length / PAGE_SIZE));
108   const pageRows = rows.slice((page - 1) * PAGE_SIZE, page * PAGE_SIZE);
109   const selected = orders?.find((o) => o.orderId === openId) ?? null;
110
111   const kpi = useMemo(() => {
112     const all = orders ?? [];
113     const count = (k: Filter) => all.filter(FILTERS.find((f) => f.key === k)!.match).length;
114     return {
115       total: all.length,
116       value: all.filter((o) => o.paymentStatus !== "refunded").reduce((s, o) => s + o.totalAmount, 0),
117       inspection: count("inspection"),
118       transit: count("transit"),
119       disputed: count("disputed"),
120       frozen: all.filter((o) => o.paymentStatus === "disputed").reduce((s, o) => s + o.totalAmount, 0),
121     };
122   }, [orders]);
123
124   return (
125     <div>
126       <PageHeader title="Orders" urdu="[Urdu]" subtitle="Inspect, dispatch and resolve every order on the
127       platform" />
128       <div className="mb-5 grid grid-cols-2 gap-3 lg:grid-cols-4">
129         <div>
130           {label: "Total orders", value: kpi.total, sub: formatPKRShort(kpi.value), f: "all" as Filter },
131           {label: "Needs inspection", value: kpi.inspection, sub: "Escrow funded", f: "inspection" as Fil
132           ter, accent: kpi.inspection > 0 },
133           {label: "In transit", value: kpi.transit, sub: "Inspected → on the road", f: "transit" as Filte
134           r },
135           {label: "Open disputes", value: kpi.disputed, sub: kpi.disputed ? `${formatPKRShort(kpi.frozen)
136           } frozen` : "None", f: "disputed" as Filter, danger: kpi.disputed > 0 },
137         </div>
138         <div>
139           <button
140             key={k.label}
141             onClick={() => setFilter(k.f)}
142             className={`card p-4 text-left transition hover:-translate-y-0.5 hover:shadow-md ${filter ===
143             k.f ? "ring-2 ring-brand-600" : ""}`}
144           >
145             <p className="text-[11px] font-semibold tracking-wider text-slate-500 uppercase dark:text-slat
146             e-400">{k.label}</p>
147             <p className={`mt-1 text-2xl font-extrabold tabular-nums ${k.danger ? "text-rose-600" : k.acce
148             nt ? "text-amber-600" : ""}`}>{orders ? k.value : "-"}</p>
149             <p className="text-xs text-slate-500 dark:text-slate-400">{k.sub}</p>
150           </button>
151         </div>
152       </div>
153       <div className="card overflow-hidden">
154         <div>
155           <div>
156             <div>
157               <div>

```

```

158         <select className="input sm:ml-auto sm:w-48" value={sort} onChange={(e) => setSort(e.target.va
159 lue as Sort)} aria-label="Sort orders">
160         <option value="newest">Newest first</option>
161         <option value="oldest">Oldest first</option>
162         <option value="amount_desc">Amount: high → low</option>
163         <option value="amount_asc">Amount: low → high</option>
164     </select>
165 </div>
166 <div className="-mx-1 flex gap-1 overflow-x-auto px-1 pb-0.5" role="tablist" aria-label="Filter
167 orders">
168     {FILTERS.map((f) => {
169         const n = orders ? orders.filter(f.match).length : 0;
170         return (
171             <button
172                 key={f.key}
173                 role="tab"
174                 aria-selected={filter === f.key}
175                 onClick={() => setFilter(f.key)}
176                 className={`inline-flex shrink-0 items-center gap-1.5 rounded-lg px-3 py-1.5 text-xs fon
177 t-semibold whitespace-nowrap transition ${
178     filter === f.key ? "bg-brand-900 text-white" : "text-slate-600 hover:bg-slate-100 dark
179 :text-slate-300 dark:hover:bg-white/5"
180 }`}
181             >
182                 {f.label}
183                 <span className={`rounded-md px-1.5 text-[10px] tabular-nums ${filter === f.key ? "bg-wh
184 ite/20" : "bg-slate-100 dark:bg-white/10"}`}>{n}</span>
185             </button>
186         );
187     })}
188 </div>
189 </div>
190 {!orders ? (
191     <div className="space-y-2 p-4">
192         {Array.from({ length: 6 }).map((_, i) => <div key={i} className="h-14 animate-pulse rounded-xl
193 bg-slate-100 dark:bg-white/5" />)}
194     </div>
195 ) : rows.length === 0 ? (
196     <div className="p-6"><EmptyState icon="" title="No orders match" body="Try a different filter or
197 search term." /></div>
198 ) : (
199     <>
200         <div className="hidden overflow-x-auto md:block">
201             <table className="w-full text-sm">
202                 <thead className="table-head">
203                     <tr>
204                         <th className="px-4 py-3 whitespace-nowrap">Order</th>
205                         <th className="px-4 py-3">Produce</th>
206                         <th className="px-4 py-3 whitespace-nowrap">Farmer → Buyer</th>
207                         <th className="px-4 py-3 text-right">Amount</th>
208                         <th className="px-4 py-3">Payment</th>
209                         <th className="px-4 py-3">Progress</th>
210                         <th className="px-4 py-3 text-right"><span className="sr-only">Actions</span></th>
211                     </tr>
212                 </thead>
213                 <tbody className="divide-y divide-slate-100 dark:divide-white/5">
214                     {pageRows.map((o) => {
215                         const act = adminAction(o);
216                         return (
217                             <tr key={o.orderId} onClick={() => setOpenId(o.orderId)} className="cursor-pointer a
218 lign-middle transition hover:bg-brand-50/50 dark:hover:bg-white/[0.03]">
219                                 <td className="px-4 py-3">
220                                     <p className="font-mono text-xs font-semibold whitespace-nowrap text-slate-900 d
221 ark:text-slate-100">{o.trackingNumber}</p>
222                                     <p className="text-xs whitespace-nowrap text-slate-500">{formatDate(o.createdAt)}

```

```

223         <p className="text-xs whitespace-nowrap text-slate-500">{formatKg(o.quantity
    Kg)} · Grade {o.qualityGrade}</p>
224     </div>
225 </div>
226 </td>
227 <td className="max-w-[220px] px-4 py-3">
228     <p className="truncate font-medium">{o.farmName ?? o.farmerName}</p>
229     <p className="truncate text-xs text-slate-500">→ {o.buyerBusiness ?? o.buyerName
    } · {o.buyerCity}</p>
230 </td>
231 <td className="px-4 py-3 text-right">
232     <p className="font-semibold whitespace-nowrap tabular-nums">{formatPKR(o.totalAm
    ount)}</p>
233     <p className="text-xs whitespace-nowrap text-slate-500 tabular-nums">{formatPKR(
    o.pricePerKg, 2)}/kg</p>
234 </td>
235 <td className="px-4 py-3"><Badge status={o.paymentStatus} /></td>
236 <td className="px-4 py-3"><ProgressBar o={o} /></td>
237 <td className="px-4 py-3 text-right whitespace-nowrap">
238     {o.paymentStatus === "disputed" ? (
239         <button className="btn-danger px-3 py-1.5 text-xs" onClick={(e) => { e.stopPro
    pagation(); setOpenId(o.orderId); }}>Resolve</button>
240     ) : act ? (
241         <button className="btn-primary px-3 py-1.5 text-xs" onClick={(e) => { e.stopPr
    opagation(); setOpenId(o.orderId); }}>{act.icon} {act.label}</button>
242     ) : (
243         <span className="text-xs font-semibold text-brand-700 dark:text-brand-400">Vie
    w →</span>
244     )}
245 </td>
246 </tr>
247 </tbody>
248 </table>
249 </div>
250 </div>
251 </div>
252
253 { /* Mobile list */}
254 <ul className="divide-y divide-slate-100 md:hidden dark:divide-white/5">
255     {pageRows.map((o) => (
256         <li key={o.orderId}>
257             <button onClick={() => setOpenId(o.orderId)} className="flex w-full items-center gap-3 p
    x-4 py-3 text-left">
258                 { /* eslint-disable-next-line @next/next/no-img-element */}
259                 <img src={cropImage(o.imagesJson, o.cropName, o.category)} alt="" className="h-11 w-11
    shrink-0 rounded-lg object-cover" />
260                 <div className="min-w-0 flex-1">
261                     <div className="flex items-center justify-between gap-2">
262                         <p className="truncate font-medium">{o.cropName}</p>
263                         <p className="text-sm font-semibold tabular-nums">{formatPKRShort(o.totalAmount)}<
    /p>
264                     </div>
265                     <p className="truncate text-xs text-slate-500">{o.trackingNumber} · {o.buyerBusiness
    ?? o.buyerName}</p>
266                     <div className="mt-1.5 flex items-center gap-2"><Badge status={o.paymentStatus} /><B
    adge status={o.deliveryStage} dot={false} /></div>
267                 </div>
268             </button>
269         </li>
270     )})}
271 </ul>
272
273 { /* Pagination */}
274 <div className="flex items-center justify-between gap-3 border-t border-slate-200 px-4 py-3 te
    xt-sm dark:border-white/5">
275     <p className="text-slate-500">
276         <span className="tabular-nums">{(page - 1) * PAGE_SIZE + 1}—{Math.min(page * PAGE_SIZE, ro
    ws.length)}</span> of <span className="tabular-nums">{rows.length}</span>
277     </p>
278     <div className="flex items-center gap-1">
279         <button className="btn-secondary px-3 py-1.5 text-xs" disabled={page === 1} onClick={() =>
    setPage((p) => p - 1)}>← Prev</button>
280         <span className="px-2 text-xs text-slate-500 tabular-nums">{page} / {pages}</span>
281         <button className="btn-secondary px-3 py-1.5 text-xs" disabled={page === pages} onClick={()
    => setPage((p) => p + 1)}>Next →</button>
282     </div>

```

```

283         </div>
284     </>
285     })
286 </div>
287
288     <OrderDrawer order={selected} onClose={() => setOpenId(null)} onChange={load} />
289 </div>
290 );
291 }
292
293 /* ----- */
294 /* Detail drawer */
295 /* ----- */
296 function OrderDrawer({ order, onClose, onChange }: { order: OrderRow | null; onClose: () => void; onChange: () => Promise<unknown> | void }) {
297     const { push } = useToast();
298     const [busy, setBusy] = useState<string | null>(null);
299     const [form, setForm] = useState({ grade: "A", moisture: "12", note: "", location: "", resolution: "" });
300
301     useEffect(() => {
302         if (order) setForm({ grade: order.qualityGrade, moisture: "12", note: "", location: "", resolution: order.disputes.find((d) => d.status === "open" || d.status === "investigating")?.resolution ?? "" });
303         }, [order?.orderId]); // eslint-disable-line react-hooks/exhaustive-deps
304
305         if (!order) return <Drawer open={false} onClose={onClose} title="">{null}</Drawer>;
306         const o = order;
307         const next = nextStage(o.deliveryStage);
308         const locked = o.paymentStatus === "escrow_locked";
309         const dispute = o.disputes.find((d) => d.status === "open" || d.status === "investigating");
310         const moving = locked && (o.deliveryStage === "dispatched" || o.deliveryStage === "in_transit");
311
312         async function run(key: string, fn: () => Promise<unknown>, ok: string) {
313             setBusy(key);
314             try {
315                 await fn();
316                 push({ kind: "success", title: ok });
317                 await onChange();
318                 setForm((f) => ({ ...f, note: "", location: "" }));
319             } catch (e) {
320                 push({ kind: "error", title: "Action failed", body: (e as Error).message });
321             } finally {
322                 setBusy(null);
323             }
324         }
325
326         const advance = () =>
327             run(
328                 "advance",
329                 () =>
330                     api(`/api/orders/${o.orderId}`, {
331                         method: "PATCH",
332                         json: {
333                             action: "advance",
334                             location: form.location || undefined,
335                             note: form.note || undefined,
336                             ...(next === "quality_checked" ? { grade: form.grade, moisture: Number(form.moisture) } : {}),
337                         },
338                     ),
339                 `Order marked ${STATUS_LABELS[next ?? ""]?.toLowerCase() ?? "updated"}`,
340             );
341
342         return (
343             <Drawer
344                 open
345                 wide
346                 onClose={onClose}
347                 title={<span className="flex items-center gap-2"><span className="font-mono text-sm">{o.trackingNumber}</span><span><Badge status={o.paymentStatus} /></span></span>
348             >
349                 <div className="space-y-6">
350                     {/* Summary */}
351                     <section className="flex items-center gap-4">
352                         {/* eslint-disable-next-line @next/next/no-img-element */}
353                         <img src={cropImage(o.imagesJson, o.cropName, o.category)} alt="" className="h-16 w-16 rounded-xl object-cover" />

```

```

354         <div className="min-w-0 flex-1">
355             <p className="text-lg font-bold">{o.cropName}</p>
356             <p className="text-sm text-slate-500">{formatKg(o.quantityKg)} · Grade {o.qualityGrade} · {for
357         ↵ matPKR(o.pricePerKg, 2)}/kg</p>
358             </div>
359             <div className="text-right">
360                 <p className="text-xl font-extrabold tabular-nums">{formatPKR(o.totalAmount)}</p>
361                 <p className="text-xs text-slate-500">Placed {formatDate(o.createdAt)}</p>
362             </div>
363         </section>
364
365         <section className="rounded-2xl border border-slate-200 p-4 dark:border-white/10">
366             <LogisticsStepper stage={o.deliveryStage} compact disputed={o.paymentStatus === "disputed"} />
367             {o.transitLocation && o.deliveryStage !== "confirmed" && <p className="mt-4 text-center text-xs
368         ↵ text-slate-500"> {o.transitLocation}</p>}
369         </section>
370
371         <section className="grid grid-cols-2 gap-3 text-sm">
372             <div className="rounded-xl bg-slate-50 p-3 dark:bg-white/5">
373                 <p className="text-[11px] font-semibold tracking-wider text-slate-500 uppercase">Farmer</p>
374                 <p className="mt-1 font-semibold">{o.farmName ?? o.farmerName}</p>
375                 <p className="text-xs text-slate-500">{o.farmLocation}{CITIES[o.farmLocation] ? `, ${CITIES[o.
376         ↵ farmLocation].province}` : ""}</p>
377             </div>
378             <div className="rounded-xl bg-slate-50 p-3 dark:bg-white/5">
379                 <p className="text-[11px] font-semibold tracking-wider text-slate-500 uppercase">Buyer</p>
380                 <p className="mt-1 font-semibold">{o.buyerBusiness ?? o.buyerName}</p>
381                 <p className="text-xs text-slate-500">{o.buyerCity} · ETA {formatDate(o.expectedDeliveryDate)}
382         ↵ </p>
383             </div>
384         </section>
385
386         {/* Dispute resolution */}
387         {dispute && (
388             <section className="rounded-2xl border border-rose-200 bg-rose-50/60 p-4 dark:border-rose-900 da
389         ↵ rk:bg-rose-950/20">
390                 <div className="flex items-center justify-between gap-2">
391                     <p className="font-semibold text-rose-900 dark:text-rose-200">Dispute</p>
392                     <Badge status={dispute.status} />
393                 </div>
394                 <p className="mt-2 text-sm text-rose-900/80 dark:text-rose-200/80">{dispute.reason}</p>
395                 <label className="label mt-3">Resolution notes</label>
396                 <textarea className="input" rows={3} value={form.resolution} onChange={(e) => setForm({ ...for
397         ↵ m, resolution: e.target.value })} placeholder="Findings from weighbridge slip, lab re-test, calls with bot
398         ↵ h parties..." />
399                 <div className="mt-3 grid grid-cols-3 gap-2">
400                     <button className="btn-secondary text-xs" disabled={!busy} onClick={() => run("inv", () =>
401         ↵ api(`/api/admin/disputes/${dispute.disputeId}`, { method: "PATCH", json: { outcome: "investigating", resol
402         ↵ ution: form.resolution || undefined } } ), "Marked as investigating")}>
403                         {busy === "inv" && <Spinner />} Investigating
404                     </button>
405                     <button className="btn-primary text-xs" disabled={!busy || !form.resolution.trim()} onClick=
406         ↵ ={() => run("rel", () => api(`/api/admin/disputes/${dispute.disputeId}`, { method: "PATCH", json: { outcom
407         ↵ e: "release", resolution: form.resolution } } ), "Payment released to farmer")}>
408                         {busy === "rel" && <Spinner />} Pay farmer
409                     </button>
410                     <button className="btn-danger text-xs" disabled={!busy || !form.resolution.trim()} onClick=
411         ↵ ={() => run("ref", () => api(`/api/admin/disputes/${dispute.disputeId}`, { method: "PATCH", json: { outcome
412         ↵ : "refund", resolution: form.resolution } } ), "Buyer refunded")}>
413                         {busy === "ref" && <Spinner />} Refund buyer
414                     </button>
415                 </div>
416             </section>
417         )}
418
419         {/* Next action */}
420         {locked && next && (
421             <section className="rounded-2xl border border-brand-200 bg-brand-50/50 p-4 dark:border-brand-900
422         ↵ dark:bg-brand-950/20">
423                 <p className="font-semibold">Next step: {DELIVERY_STAGES.find((s) => s.key === next)?.label}</
424         ↵ p>
425                 <div className="mt-3 grid grid-cols-2 gap-3">
426                     {next === "quality_checked" ? (
427                         <>
428                         <div>
429                             <label className="label">Grade</label>

```

```

415         <select className="input" value={form.grade} onChange={(e) => setForm({ ...form, grade
416 : e.target.value })}><option>A</option><option>B</option><option>C</option></select>
417     </div>
418     <label className="label">Moisture %</label>
419     <input type="number" step="0.1" min="0" max="100" className="input" value={form.moisture}
420     onChange={(e) => setForm({ ...form, moisture: e.target.value })} />
421 </div>
422 ) : (
423     <div className="col-span-2">
424         <label className="label">{next === "delivered" ? "Received at" : "Location"}</label>
425         <input className="input" list="admin-cities" value={form.location} onChange={(e) => setForm({ ...form, location: e.target.value })} placeholder={next === "delivered" ? "Buyer warehouse" : "e.g. M-2 Motorway, Sheikhpura"} />
426     </div>
427 )}
428     <div className="col-span-2">
429         <label className="label">Note (optional)</label>
430         <input className="input" value={form.note} onChange={(e) => setForm({ ...form, note: e.target.value })} placeholder={next === "dispatched" ? "Truck LES-4521, driver 0300-..." : ""} />
431     </div>
432 </div>
433     {next === "delivered" && <p className="mt-3 text-xs text-slate-600 dark:text-slate-400">Confirming delivery releases {formatPKR(o.totalAmount)} to the farmer (minus 1.5% fee).</p>}
434     <button className="btn-primary mt-3 w-full" disabled={!busy} onClick={advance}>
435         {busy === "advance" && <Spinner />} Mark as {STATUS_LABELS[next].toLowerCase()}
436     </button>
437 </section>
438 )}
439
440     {moving && (
441         <section className="flex gap-2">
442             <input className="input" list="admin-cities" placeholder="Update checkpoint location..." value={form.location} onChange={(e) => setForm({ ...form, location: e.target.value })} />
443             <button
444                 className="btn-secondary shrink-0"
445                 disabled={!busy || !form.location}
446                 onClick={() => run("loc", () => api(`/api/orders/${o.orderId}`, { method: "PATCH", json: { action: "update_location", location: form.location } }), "Checkpoint updated")}
447             >
448                 {busy === "loc" && <Spinner />} Update
449             </button>
450         </section>
451     )}
452
453     {o.paymentStatus === "awaiting_escrow" && (
454         <p className="rounded-xl bg-amber-50 p-3 text-sm text-amber-900 dark:bg-amber-950/30 dark:text-amber-200">Waiting for the buyer to fund escrow. No action required yet.</p>
455     )}
456
457     {/* Timeline */}
458     <section>
459         <p className="mb-3 text-[11px] font-semibold tracking-wider text-slate-500 uppercase">Activity</p>
460         <ol className="relative space-y-4 border-l border-slate-200 pl-5 dark:border-white/10">
461             {[...o.events].reverse().map((e) => (
462                 <li key={e.eventId} className="relative">
463                     <span className={`absolute top-1.5 -left-[25px] h-2.5 w-2.5 rounded-full ring-4 ring-white dark:ring-[#0d1f16] ${e.stage.includes("dispute") ? "bg-rose-500" : e.stage.includes("escrow") ? "bg-gold-400" : "bg-brand-600"}`} />
464                     <div className="flex items-baseline justify-between gap-3">
465                         <p className="text-sm font-semibold">{EVENT_LABEL[e.stage] ?? e.stage.replace(/_/g, " ")}</p>
466                         <time className="shrink-0 text-xs text-slate-500">{new Date(e.createdAt).toLocaleString("en-GB", { day: "2-digit", month: "short", hour: "2-digit", minute: "2-digit" })}</time>
467                     </div>
468                     {(e.location || e.note) && <p className="text-xs text-slate-500">{[e.location, e.note].filter(Boolean).join(" · ")}</p>}
469                 </li>
470             )]}
471         </ol>
472     </section>
473 </div>
474     <datalist id="admin-cities">{CITY_NAMES.map((c) => <option key={c} value={` ${c}`, ${CITIES[c].province}`}) />}</datalist>

```

```

475     </Drawer>
476   );
477 }
478

```

src/components/admin/payouts-panel.tsx

```

1  "use client";
2
3  import { useCallback, useEffect, useState } from "react";
4  import { api, Badge, Spinner } from "@components/ui";
5  import { useToast } from "@components/providers";
6  import { formatDate, formatPKR } from "@lib/constants";
7
8  type Payout = { payoutId: number; amount: number; method: string; accountTitle: string; accountNumber: str
  ↵  ing; status: string; createdAt: string; farmerName: string; farmName: string | null; adminNote: string | n
  ↵  ull };
9
10 /** Admin queue for farmer withdrawal requests. */
11 export function PayoutsPanel() {
12   const { push } = useToast();
13   const [rows, setRows] = useState<Payout[] | null>(null);
14   const [busy, setBusy] = useState<number | null>(null);
15   const load = useCallback(() => api<Payout[]>("/api/payouts").then(setRows).catch(() => setRows([])), [])
  ↵   ;
16   useEffect(() => { load(); }, [load]);
17
18   async function act(p: Payout, status: "paid" | "rejected") {
19     const note = status === "rejected" ? prompt("Reason for rejection (sent to the farmer):") ?? undefined
  ↵     : undefined;
20     if (status === "rejected" && !note) return;
21     if (status === "paid" && !confirm(`Confirm you transferred ${formatPKR(p.amount)} to ${p.accountTitle}
  ↵     (${p.accountNumber})?`)) return;
22     setBusy(p.payoutId);
23     try {
24       await api(`/api/payouts/${p.payoutId}`, { method: "PATCH", json: { status, note } });
25       push({ kind: "success", title: status === "paid" ? "Marked as paid" : "Payout rejected, funds return
  ↵     ed" });
26       load();
27     } catch (e) {
28       push({ kind: "error", title: (e as Error).message });
29     } finally {
30       setBusy(null);
31     }
32   }
33
34   const pending = rows?.filter((r) => r.status === "pending") ?? [];
35   return (
36     <section className="card overflow-hidden">
37       <div className="flex items-center justify-between border-b border-slate-100 px-5 py-3.5 dark:border-
  ↵     white/5">
38         <h2 className="font-bold">Farmer payout requests</h2>
39         <span className="text-xs text-slate-500">{pending.length} pending</span>
40       </div>
41       {!rows ? <div className="h-24 animate-pulse" /> : rows.length === 0 ? <p className="p-6 text-center
  ↵     text-sm text-slate-500">No payout requests yet.</p> : (
42         <div className="overflow-x-auto">
43           <table className="w-full text-sm">
44             <thead className="table-head"><tr><th className="px-4 py-3">Farmer</th><th className="px-4 py-
  ↵     3 text-right">Amount</th><th className="px-4 py-3">Destination</th><th className="px-4 py-3">Requested</th>
  ↵     <th className="px-4 py-3">Status</th><th className="px-4 py-3 text-right"><span className="sr-only">Actio
  ↵     ns</span></th></tr></thead>
45             <tbody className="divide-y divide-slate-100 dark:divide-white/5">
46               {rows.slice(0, 10).map((p) => (
47                 <tr key={p.payoutId}>
48                   <td className="px-4 py-3"><p className="font-medium">{p.farmName ?? p.farmerName}</p><p
  ↵     className="text-xs text-slate-500">{p.farmerName}</p></td>
49                   <td className="px-4 py-3 text-right font-semibold tabular-nums">{formatPKR(p.amount)}</t
  ↵     d>
50                   <td className="px-4 py-3"><p className="text-xs font-semibold uppercase">{p.method}</p><p
  ↵     p className="font-mono text-xs text-slate-500">{p.accountTitle} · {p.accountNumber}</p></td>
51                   <td className="px-4 py-3 text-xs text-slate-500">{formatDate(p.createdAt)}</td>
52                   <td className="px-4 py-3"><Badge status={p.status === "paid" ? "released" : p.status ===
  ↵     "rejected" ? "rejected" : "pending"} label={p.status} /></td>
53                   <td className="px-4 py-3 text-right whitespace-nowrap">
54                     {p.status === "pending" && (

```

```

55         <div className="flex justify-end gap-1.5">
56             <button className="btn-primary px-3 py-1 text-xs" disabled={busy === p.payoutId} o
57             nClick={() => act(p, "paid")}>{busy === p.payoutId && <Spinner />} Mark paid</button>
57             <button className="btn-secondary px-3 py-1 text-xs text-rose-600" disabled={busy =
58             == p.payoutId} onClick={() => act(p, "rejected")}>Reject</button>
59         </div>
60     )}
61 </td>
62 </tr>
63 ))}
64 </tbody>
65 </table>
66 </div>
67 )}
68 </section>
69 );
70 }

```

src/components/app-shell.tsx

```

1  "use client";
2
3  import Link from "next/link";
4  import { usePathname, useRouter } from "next/navigation";
5  import { useEffect, useState, type ReactNode } from "react";
6  import { useI18n } from "@components/providers";
7  import { MandiTicker, type TickerItem } from "@components/mandi-ticker";
8  import { Logo } from "@components/Logo";
9  import { NotificationBell, ProfileMenu, WalletChip, type NavViewer } from "@components/nav/nav-actions";
10 import { ChatBot } from "@components/ChatBot";
11 import { CITIES, type Role } from "@lib/constants";
12
13 type NavItem = { href: string; key: string; icon: string };
14 const NAV: Record<Role, NavItem[]> = {
15   farmer: [
16     { href: "/farmer/dashboard", key: "dashboard", icon: "" },
17     { href: "/farmer/listings", key: "listings", icon: "" },
18     { href: "/farmer/bids", key: "negotiations", icon: "" },
19     { href: "/farmer/orders", key: "orders", icon: "" },
20     { href: "/wallet", key: "wallet", icon: "" },
21     { href: "/farmer/reviews", key: "reviews", icon: "" },
22     { href: "/mandi-rates", key: "mandi", icon: "" },
23   ],
24   buyer: [
25     { href: "/buyer/dashboard", key: "dashboard", icon: "" },
26     { href: "/marketplace", key: "marketplace", icon: "" },
27     { href: "/buyer/bids", key: "myBids", icon: "" },
28     { href: "/buyer/orders", key: "logistics", icon: "" },
29     { href: "/wallet", key: "wallet", icon: "" },
30     { href: "/mandi-rates", key: "mandi", icon: "" },
31   ],
32   inspector: [
33     { href: "/inspector/dashboard", key: "inspections", icon: "" },
34     { href: "/tracking", key: "tracking", icon: "" },
35     { href: "/mandi-rates", key: "mandi", icon: "" },
36   ],
37   admin: [
38     { href: "/admin/dashboard", key: "analytics", icon: "" },
39     { href: "/admin/orders", key: "disputes", icon: "🔍[Urdu]" },
40     { href: "/admin/users", key: "users", icon: "" },
41     { href: "/admin/verification", key: "verification", icon: "[Urdu]" },
42     { href: "/admin/mandi", key: "priceController", icon: "" },
43     { href: "/inspector/dashboard", key: "inspections", icon: "" },
44     { href: "/admin/db-explorer", key: "dbExplorer", icon: "[Urdu]" },
45   ],
46 };
47
48 const ROLE_LABEL: Record<Role, { en: string; ur: string }> = {
49   farmer: { en: "Kisan · Seller", ur: "[Urdu]" },
50   buyer: { en: "Buyer · Wholesaler", ur: "[Urdu]" },
51   inspector: { en: "Quality Inspector", ur: "[Urdu]" },
52   admin: { en: "Administrator", ur: "[Urdu]" },
53 };
54

```

```

55 export type ShellUser = { fullName: string; role: Role; businessName: string | null; isVerified: boolean;
56   city: string; title: string | null; avatarUrl: string | null; userCode: string | null };
57
58 function PKClock() {
59   const [now, setNow] = useState<string>("");
60   useEffect(() => {
61     const tick = () =>
62       setNow(new Date().toLocaleString("en-GB", { timeZone: "Asia/Karachi", weekday: "short", day: "2-digit", month: "short", hour: "2-digit", minute: "2-digit" }));
63     tick();
64     const id = setInterval(tick, 30000);
65     return () => clearInterval(id);
66   }, []);
67   return <span suppressHydrationWarning>{now} PKT</span>;
68 }
69
70 export function AppShell({ user, ticker, viewer, children }: { user: ShellUser; ticker: TickerItem[]; viewer: NavViewer; children: ReactNode }) {
71   const pathname = usePathname();
72   const router = useRouter();
73   const { lang, t } = useI18n();
74   const [mobileOpen, setMobileOpen] = useState(false);
75   const items = NAV[user.role];
76   const isActive = (href: string) => pathname === href || pathname.startsWith(href + "/");
77   const province = CITIES[user.city]?.province;
78
79   async function logout() {
80     await fetch("/api/auth/logout", { method: "POST" });
81     router.push("/login");
82     router.refresh();
83   }
84
85   const navList = (onClick?: () => void) => (
86     <nav className="flex flex-col gap-1">
87       {items.map((it) => (
88         <Link
89           key={it.href}
90           href={it.href}
91           onClick={onClick}
92           className={`group flex items-center gap-3 rounded-xl px-3 py-2.5 text-sm font-medium transition
93             ${isActive(it.href) ? "bg-white text-brand-900 shadow-lg shadow-black/20" : "text-white/75 hover:bg-white/10 hover:text-white"}
94           `}
95         >
96           <span className={`grid h-8 w-8 place-items-center rounded-lg text-base ${isActive(it.href) ? "bg-brand-50" : "bg-white/10"}`} >{it.icon}</span>
97           {t(it.key)}
98           {isActive(it.href) && <span className="ml-auto h-1.5 w-1.5 rounded-full bg-gold-400" />}
99         </Link>
100       ))}
101       <div className="my-3 h-px bg-white/10" />
102       <Link href="/account" onClick={onClick} className={`flex items-center gap-3 rounded-xl px-3 py-2 text-sm font-medium ${pathname.startsWith("/account") ? "bg-white text-brand-900" : "text-white/60 hover:bg-white/10 hover:text-white"}`} >
103         <span className="grid h-8 w-8 place-items-center rounded-lg bg-white/5">{t("account")}</span>
104         {t("account")}
105       </Link>
106     </nav>
107   );
108
109   const initials = user.fullName.split(" ").map((p) => p[0]).slice(0, 2).join("");
110
111   const sidebarInner = (onClick?: () => void) => (
112     <>
113       <Brand light />
114       <div className="mt-6 flex-1 overflow-y-auto">{navList(onClick)}</div>
115       <div className="rounded-2xl bg-white/10 p-3 ring-1 ring-white/10">
116         <div className="flex items-center gap-3">
117           {user.avatarUrl ? (
118             // eslint-disable-next-line @next/next/no-img-element
119             <img src={user.avatarUrl} alt="" referrerPolicy="no-referrer" className="h-10 w-10 rounded-full object-cover ring-2 ring-gold-400" />
120           ) : (
121             <div className="grid h-10 w-10 place-items-center rounded-full bg-gold-400 text-sm font-extrabold text-brand-950">{initials}</div>

```

```

121     })
122     <div className="min-w-0 text-white">
123         <p className="truncate text-sm font-semibold">{user.fullName}</p>
124         <p className="truncate text-xs text-white/60" title={user.userCode ? `ID ${user.userCode}` : u
125         ndefined}>{user.role === "admin" ? (user.title ?? ROLE_LABEL.admin[lang]) : ROLE_LABEL[user.role][lang]}</p>
126         {user.role === "admin" && <span className="mt-1 inline-flex items-center gap-1 rounded-full bg
127         -gold-400 px-2 py-0.5 text-[10px] font-extrabold tracking-wide text-brand-950 uppercase">★ Super Admin</sp
128         an>}
129     </div>
130     <button onClick={logout} className="mt-3 w-full rounded-lg bg-white/10 py-1.5 text-xs font-semibol
131     d text-white hover:bg-white/20">
132         ⓧ {t("logout")}
133     </button>
134 </div>
135 </>
136 );
137
138 return (
139     <div className="min-h-screen lg:grid lg:grid-cols-[270px_1fr]">
140         <aside className="pk-pattern sticky top-0 hidden h-screen flex-col bg-gradient-to-b from-brand-900 v
141         ia-brand-900 to-brand-950 p-4 lg:flex">
142             {sidebarInner()}
143         </aside>
144
145         <div className="flex min-h-screen min-w-0 flex-col">
146             <div className="sticky top-0 z-40">
147                 <header className="flex items-center gap-2 border-b border-brand-900/10 bg-white/90 px-4 py-2.5
148                 backdrop-blur sm:px-6 dark:border-white/5 dark:bg-[#07130d]/90">
149                     <button className="rounded-lg p-2 hover:bg-slate-100 lg:hidden dark:hover:bg-slate-800" onClic
150                     k={() => setMobileOpen(true)} aria-label="Open menu">
151                         <svg width="22" height="22" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidt
152                         h="2"><path d="M4 6h16M4 12h16M4 18h16" /></svg>
153                     </button>
154                     <div className="lg:hidden"><Brand small /></div>
155                     <div className="hidden min-w-0 lg:block">
156                         <p className="truncate text-sm">
157                             <span className="font-semibold text-brand-800 dark:text-brand-300">{lang === "ur" ? "[Urdu
158                             ]" : "Assalam-o-Alaikum"}, {user.fullName.split(" ")[0]}</span>
159                             <span className="text-slate-500 dark:text-slate-400">
160                                 {user.businessName && <> · {user.businessName}</> · {user.city}{province ? `, ${provin
161                                 ce}` : ""}}
162                             </span>
163                             {user.role === "admin" ? (
164                                 <span className="ml-2 inline-flex items-center gap-1 rounded-full bg-gold-400 px-2 py-0.
165                                 5 text-xs font-bold text-brand-950">★ Super Admin · {user.title ?? "Platform Manager"}</span>
166                             ) : user.isVerified ? (
167                                 <span className="ml-2 rounded-full bg-brand-50 px-2 py-0.5 text-xs font-semibold text-br
168                                 and-700 dark:bg-brand-500/10 dark:text-brand-300">✓ {t("verified")}</span>
169                             ) : user.role === "farmer" ? (
170                                 <span className="ml-2 rounded-full bg-amber-50 px-2 py-0.5 text-xs font-semibold text-am
171                                 ber-700"> Verification pending</span>
172                             ) : null}
173                         </p>
174                         <p className="text-xs text-slate-400"><PKClock /> · Pakistan</p>
175                     </div>
176                     <div className="ml-auto flex items-center gap-2">
177                         <nav className="mr-2 hidden items-center gap-1 xl:flex" aria-label="Site">
178                             {[["/", "Home"], ["/marketplace", "Marketplace"], ["/mandi-rates", "Mandi Rates"], ["/trac
179                             king", "Tracking"]].map(([href, label]) => (
180                                 <Link key={href} href={href} className="rounded-lg px-2.5 py-1.5 text-sm font-medium tex
181                                 t-slate-600 hover:bg-slate-100 hover:text-slate-900 dark:text-slate-300 dark:hover:bg-slate-800 dark:ho
182                                 ver:text-white">{label}</Link>
183                             ))}
184                         </nav>
185                         {viewer && (viewer.role === "farmer" || viewer.role === "buyer") && <WalletChip balance={vie
186                         wer.walletBalance} escrow={viewer.escrowBalance} />}
187                         {viewer && <NotificationBell initialUnread={viewer.unread} />}
188                         {viewer && <ProfileMenu viewer={viewer} />}
189                     </div>
190                 </header>
191                 <MandiTicker items={ticker} />
192             </div>
193         </div>
194     </div>
195     <main className="flex-1 px-4 py-6 pb-24 sm:px-6 lg:px-8 lg:pb-10">{children}</main>

```

```

179
180     <nav className="fixed inset-x-0 bottom-0 z-40 grid border-t border-white/10 bg-brand-900 lg:hidden
181     " style={{ gridTemplateColumns: `repeat(${Math.min(items.length, 5)}, minmax(0, 1fr))` }}>
182         {items.slice(0, 5).map((it) => (
183             <Link key={it.href} href={it.href} className={`flex flex-col items-center gap-0.5 py-2 text-[1
184             0px] font-semibold ${isActive(it.href) ? "text-gold-400" : "text-white/70"}`}>
185                 <span className="text-lg">{it.icon}</span>
186                 <span className="truncate px-1">{t(it.key)}</span>
187             </Link>
188         ))}
189     </nav>
190 </div>
191
192     {user.role === "buyer" && <ChatBot placement="app" userCity={user.city} /> }
193
194     {mobileOpen && (
195         <div className="fixed inset-0 z-50 bg-black/50 lg:hidden onClick={() => setMobileOpen(false)}>
196         <div className="pk-pattern animate-fade-in relative flex h-full w-72 flex-col bg-brand-900 p-4"
197         onClick={(e) => e.stopPropagation()}>
198             <button onClick={() => setMobileOpen(false)} className="mb-2 self-end rounded-lg p-1 text-whit
199             e/70 hover:bg-white/10"><</button>
200             {sidebarInner(() => setMobileOpen(false))}
201         </div>
202         </div>
203     )}
204 </div>
205 );
206 }
207
208 export function Brand({ small = false, light = false }: { small?: boolean; light?: boolean }) {
209     return (
210         <Link href="/" className="inline-flex rounded-xl focus-visible:outline-2 focus-visible:outline-offset-
211         4 focus-visible:outline-gold-400" aria-label="MarketLink home">
212             <Logo size={small ? 30 : 38} tone={light ? "light" : "auto"} tagline={!small} />
213         </Link>
214     );
215 }

```

src/components/auth/google-button.tsx

```

1  "use client";
2
3  import { useState } from "react";
4  import { Spinner } from "@components/ui";
5
6  /** Official multicolour Google "G" mark (per Google's sign-in branding guidelines). */
7  export function GoogleLogo({ className = "h-5 w-5" }: { className?: string }) {
8      return (
9          <svg className={className} viewBox="0 0 48 48" aria-hidden>
10             <path fill="#FFC107" d="M43.611 20.083H42V20H24v8h11.303c-1.649 4.657-6.08 8-11.303 8-6.627 0-12-5.3
11             73-12-12s5.373-12 12-12c3.059 0 5.842 1.154 7.961 3.039l5.657-5.657C34.046 6.053 29.268 4 24 4 12.955 4 4
12             12.955 4 24s8.955 20 20 20-8.955 20-20c0-1.341-.138-2.65-.389-3.917z" />
13             <path fill="#FF3D00" d="m6.306 14.691 6.571 4.819C14.655 15.108 18.961 12 24 12c3.059 0 5.842 1.154
14             7.961 3.039l5.657-5.657C34.046 6.053 29.268 4 24 4 16.318 4 9.656 8.337 6.306 14.691z" />
15             <path fill="#4CAF50" d="M24 44c5.166 0 9.86-1.977 13.409-5.192l-6.19-5.238A11.91 11.91 0 0 1 24 36c-
16             5.202 0-9.619-3.317-11.283-7.946l-6.522 5.025C9.505 39.556 16.227 44 24 44z" />
17             <path fill="#1976D2" d="M43.611 20.083H42V20H24v8h11.303a12.04 12.04 0 0 1-4.087 5.571l.003-.002 6.1
18             9 5.238C36.971 39.205 44 34 44 24c0-1.341-.138-2.65-.389-3.917z" />
19         </svg>
20     );
21 }
22
23 export function GoogleButton({ role = "buyer", next, label = "Continue with Google" }: { role?: "buyer" |
24 "farmer"; next?: string; label?: string }) {
25     const [busy, setBusy] = useState(false);
26     const href = `/api/auth/google?role=${role}${next ? `&next=${encodeURIComponent(next)}` : ""}`;
27     return (
28         <a
29             href={href}
30             onClick={() => setBusy(true)}
31             className="flex h-11 w-full items-center justify-center gap-3 rounded-xl border border-slate-300 bg-
32             white px-4 text-sm font-semibold text-slate-700 shadow-sm transition hover:bg-slate-50 hover:shadow focus-
33             visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-brand-600 dark:border-slate-600 dar
34             k:bg-slate-900 dark:text-slate-100 dark:hover:bg-slate-800"

```

```

26     >
27     {busy ? <Spinner /> : <GoogleLogo />}
28     {label}
29   </a>
30 );
31 }
32
33 export function OrDivider({ text = "or" }: { text?: string }) {
34   return (
35     <div className="my-5 flex items-center gap-3 text-xs font-semibold tracking-wider text-slate-400 upper
36     case">
37       <span className="h-px flex-1 bg-slate-200 dark:bg-slate-700" />
38       {text}
39       <span className="h-px flex-1 bg-slate-200 dark:bg-slate-700" />
40     </div>
41   );
42 }

```

src/components/auth/otp-forms.tsx

```

1  "use client";
2
3  import Link from "next/link";
4  import { useRouter } from "next/navigation";
5  import { useEffect, useRef, useState } from "react";
6  import { Logo } from "@components/Logo";
7  import { api, Spinner } from "@components/ui";
8  import { useToast } from "@components/providers";
9
10 function Shell({ title, subtitle, children }: { title: string; subtitle: string; children: React.ReactNode
11 }) {
12   return (
13     <div className="grid min-h-screen place-items-center bg-[#f6f8f4] px-4 py-10 dark:bg-[#07130d]">
14       <div className="w-full max-w-md">
15         <Link href="/" className="inline-flex"><Logo size={38} /></Link>
16         <div className="card mt-6 p-6 sm:p-8">
17           <h1 className="text-2xl font-extrabold tracking-tight">{title}</h1>
18           <p className="mt-1 text-sm text-slate-500 dark:text-slate-400">{subtitle}</p>
19           <div className="mt-6">{children}</div>
20         </div>
21       </div>
22     </div>
23   );
24 }
25
26 /** 6-box one-time-code input with paste support. */
27 function CodeInput({ value, onChange }: { value: string; onChange: (v: string) => void }) {
28   const refs = useRef<(HTMLInputElement | null)[]>([]);
29   const digits = value.padEnd(6, " ").slice(0, 6).split("");
30   return (
31     <div className="flex justify-between gap-2" onPaste={(e) => { const t = e.clipboardData.getData("text"
32     ).replace(/\D/g, "").slice(0, 6); if (t) { e.preventDefault(); onChange(t); refs.current[Math.min(5, t.len
33     gth)]?.focus(); } }}>
34       {digits.map((d, i) => (
35         <input key={i} ref={el} => { refs.current[i] = el; } } inputMode="numeric" maxLength={1} aria-labe
36         l={`Digit ${i + 1}`} value={d.trim()}
37         onChange={(e) => { const c = e.target.value.replace(/\D/g, "").slice(-1); const arr = value.padE
38         nd(6, " ").split(""); arr[i] = c || " "; onChange(arr.join("").trimEnd()); if (c) refs.current[i + 1]?.foc
39         us(); } }
40         onKeyDown={(e) => { if (e.key === "Backspace" && !d.trim()) refs.current[i - 1]?.focus(); } }
41         className="h-14 w-12 rounded-xl border border-slate-300 bg-white text-center text-2xl font-bold
42         focus:border-brand-500 focus:ring-2 focus:ring-brand-500/30 focus:outline-none dark:border-slate-700 dark:
43         bg-slate-800" />
44       ))}
45     </div>
46   );
47 }
48
49 function DevCode({ code }: { code?: string }) {
50   if (!code) return null;
51   return <p className="mb-4 rounded-xl border border-amber-300 bg-amber-50 p-3 text-xs text-amber-900 dark
52   :border-amber-800 dark:bg-amber-950/30 dark:text-amber-200">Development mode: email delivery isn't co
53   nfigured, so your code is shown here: <b className="font-mono text-sm tracking-widest">{code}</b></p>;
54 }

```

```

45
46 export function VerifyEmailForm({ email, initialDevCode }: { email: string; initialDevCode?: string }) {
47   const router = useRouter();
48   const { push } = useToast();
49   const [code, setCode] = useState("");
50   const [dev, setDev] = useState(initialDevCode);
51   const [busy, setBusy] = useState(false);
52   const [cooldown, setCooldown] = useState(45);
53   useEffect(() => { const id = setInterval(() => setCooldown((c) => Math.max(0, c - 1)), 1000); return ()
    => clearInterval(id); }, []);
54
55   async function verify(e?: React.FormEvent) {
56     e?.preventDefault();
57     if (code.length !== 6) return;
58     setBusy(true);
59     try {
60       const r = await api<{ redirectTo: string }>("/api/auth/verify-email", { method: "POST", json: { email
    l, code } });
61       push({ kind: "success", title: "Email verified. Welcome to MarketLink!" });
62       router.push(r.redirectTo);
63       router.refresh();
64     } catch (err) {
65       push({ kind: "error", title: (err as Error).message });
66       setBusy(false);
67     }
68   }
69   async function resend() {
70     try {
71       const r = await api<{ devCode?: string }>("/api/auth/verify-email", { method: "POST", json: { email,
    resend: true } });
72       setDev(r.devCode);
73       setCooldown(45);
74       push({ kind: "info", title: "New code sent" });
75     } catch (err) {
76       push({ kind: "error", title: (err as Error).message });
77     }
78   }
79   useEffect(() => { if (code.length === 6) verify(); }, [code]); // eslint-disable-line react-hooks/exhaus
    tive-deps
80
81   return (
82     <Shell title="Verify your email" subtitle={`Enter the 6-digit code we sent to ${email}. It expires in
    10 minutes.`>
83       <DevCode code={dev} />
84       <form onSubmit={verify} className="space-y-5">
85         <CodeInput value={code} onChange={setCode} />
86         <button className="btn-primary w-full py-2.5" disabled={busy || code.length !== 6}>{busy && <Spinn
    er />} Verify & continue</button>
87       </form>
88       <p className="mt-5 text-center text-sm text-slate-500">
89         Didn't get it? {cooldown > 0 ? <span>Resend in {cooldown}s</span> : <button onClick={resend}
    className="font-semibold text-brand-700 hover:underline dark:text-brand-400">Resend code</button>}
90       </p>
91     </Shell>
92   );
93 }
94
95 export function ForgotPasswordForm({ initialEmail = "" }: { initialEmail?: string }) {
96   const { push } = useToast();
97   const [email, setEmail] = useState(initialEmail);
98   const [busy, setBusy] = useState(false);
99   const [sent, setSent] = useState<{ message: string; devResetUrl?: string } | null>(null);
100
101   async function request(e: React.FormEvent) {
102     e.preventDefault();
103     setBusy(true);
104     try {
105       const r = await api<{ message: string; devResetUrl?: string }>("/api/auth/forgot-password", { method
    : "POST", json: { email } });
106       setSent(r);
107       push({ kind: "success", title: "Reset link sent", body: "Check your inbox (and spam folder)."} );
108     } catch (err) {
109       push({ kind: "error", title: "Couldn't send reset link", body: (err as Error).message });
110     } finally {
111       setBusy(false);
112     }

```

```

113     }
114
115     return (
116       <Shell title={sent ? "Check your email" : "Forgot password?"} subtitle={sent ? sent.message : "Enter t
117 he email you signed up with. We'll send you a secure link to choose a new password."}>
118         {!sent ? (
119           <form onSubmit={request} className="space-y-4">
120             <div><label className="label" htmlFor="fp-email">Email</label><input id="fp-email" type="email"
121             required className="input" value={email} onChange={(e) => setEmail(e.target.value)} autoFocus placeholder=
122             "aap@company.pk" /></div>
123             <button className="btn-primary w-full py-2.5" disabled={busy}>{busy && <Spinner />} Send reset l
124             ink</button>
125           </form>
126         ) : (
127           <div className="space-y-4">
128             <div className="grid place-items-center rounded-2xl bg-brand-50 py-6 text-5xl dark:bg-brand-500/
129             10"></div>
130             <p className="text-sm text-slate-600 dark:text-slate-300">The link expires in <b>15 minutes</b>
131             and can be used once.</p>
132             {sent.devResetUrl && (
133               <div className="rounded-xl border border-amber-300 bg-amber-50 p-3 text-xs text-amber-900 dark
134               :border-amber-800 dark:bg-amber-950/30 dark:text-amber-200">
135                 <p><b>Simulated email:</b> email delivery isn't configured on this server (RESEND_API_K
136                 EY), so the link is shown here instead.</p>
137                 <Link href={sent.devResetUrl} className="btn-primary mt-2 w-full">Open reset link </Link>
138               </div>
139             )}
140             <button className="btn-secondary w-full" onClick={() => setSent(null)}>Use a different email</bu
141             tton>
142           </div>
143         )}
144         <p className="mt-5 text-center text-sm text-slate-500"><Link href="/login" className="font-semibold
145         text-brand-700 hover:underline dark:text-brand-400">< Back to sign in</Link></p>
146       </Shell>
147     );
148   }
149 }
150
151 export function ResetPasswordForm({ token }: { token: string }) {
152   const router = useRouter();
153   const { push } = useToast();
154   const [state, setState] = useState<{ status: "checking" | "valid" | "invalid"; email?: string; error?: s
155   tring }>({ status: "checking" });
156   const [pw, setPw] = useState({ a: "", b: "" });
157   const [show, setShow] = useState(false);
158   const [busy, setBusy] = useState(false);
159
160   useEffect(() => {
161     fetch(`/api/auth/reset-password?token=${encodeURIComponent(token)}`)
162       .then((r) => r.json())
163       .then((r) => setState(r.success ? { status: "valid", email: r.data.email } : { status: "invalid", er
164       ror: r.error }))
165       .catch(() => setState({ status: "invalid", error: "Could not verify the link. Check your connection.
166       " }));
167   }, [token]);
168
169   const strength = [pw.a.length >= 8, /[A-Z]/.test(pw.a) && /[a-z]/.test(pw.a), /\d/.test(pw.a), /^[A-Za-z
170   0-9]/.test(pw.a)].filter(Boolean).length;
171
172   async function submit(e: React.FormEvent) {
173     e.preventDefault();
174     if (pw.a !== pw.b) return push({ kind: "error", title: "Passwords don't match" });
175     setBusy(true);
176     try {
177       const r = await api<{ email: string }>("/api/auth/reset-password", { method: "POST", json: { token,
178       newPassword: pw.a } });
179       push({ kind: "success", title: "Password updated", body: "Sign in with your new password." });
180       router.push(`/login?email=${encodeURIComponent(r.email)}`);
181     } catch (err) {
182       push({ kind: "error", title: "Couldn't reset password", body: (err as Error).message });
183       setBusy(false);
184     }
185   }
186
187   if (state.status === "checking") return <Shell title="Reset password" subtitle="Checking your reset link
188   ..."><div className="h-24 animate-pulse rounded-xl bg-slate-100 dark:bg-white/5" /></Shell>;
189   if (state.status === "invalid")

```

```

173     return (
174       <Shell title="Link expired" subtitle={state.error ?? "This reset link is invalid or has expired."}>
175         <Link href="/forgot-password" className="btn-primary w-full">Request a new link</Link>
176       </Shell>
177     );
178     const type = show ? "text" : "password";
179     return (
180       <Shell title="Choose a new password" subtitle={`For ${state.email}. At least 8 characters with a letter and a number.`}>
181         <form onSubmit={submit} className="space-y-4">
182           <div>
183             <label className="label" htmlFor="np">New password</label>
184             <input id="np" type={type} minLength={8} required className="input" value={pw.a} onChange={(e) => setPw({ ...pw, a: e.target.value })} autoComplete="new-password" autoFocus />
185             <div className="mt-2 flex gap-1" aria-hidden>{[0, 1, 2, 3].map((i) => <span key={i} className={`h-1.5 flex-1 rounded-full ${i < strength ? (strength <= 1 ? "bg-rose-500" : strength <= 2 ? "bg-amber-500" : "bg-brand-600")} : "bg-slate-200 dark:bg-slate-700"}`} />)}</div>
186           </div>
187           <div><label className="label" htmlFor="cp">Confirm password</label><input id="cp" type={type} minLength={8} required className="input" value={pw.b} onChange={(e) => setPw({ ...pw, b: e.target.value })} autoComplete="new-password" /></div>
188           <label className="flex items-center gap-2 text-sm text-slate-600 dark:text-slate-400"><input type="checkbox" checked={show} onChange={(e) => setShow(e.target.checked)} /> Show passwords</label>
189           <button className="btn-primary w-full py-2.5" disabled={busy}>{busy && <Spinner />} Update password</button>
190         </form>
191       </Shell>
192     );
193   }
194 }

```

src/components/charts.tsx

```

1  "use client";
2
3  import {
4    ResponsiveContainer,
5    LineChart,
6    Line,
7    BarChart,
8    Bar,
9    AreaChart,
10   Area,
11   PieChart,
12   Pie,
13   Cell,
14   XAxis,
15   YAxis,
16   Tooltip,
17   Legend,
18   CartesianGrid,
19 } from "recharts";
20 import { useTheme } from "@components/providers";
21
22 export const PALETTE = ["#01411c", "#f1c232", "#1d9d5c", "#0ea5e9", "#d97706", "#8b5cf6", "#ef4444", "#64748b", "#14b8a6", "#ec4899"];
23
24 function useAxis() {
25   const { theme } = useTheme();
26   const dark = theme === "dark";
27   return {
28     grid: dark ? "#1e293b" : "#e2e8f0",
29     tick: { fill: dark ? "#94a3b8" : "#64748b", fontSize: 11 },
30     tooltip: {
31       contentStyle: {
32         background: dark ? "#0f172a" : "ffffff",
33         border: `1px solid ${dark ? "#334155" : "#e2e8f0"}`,
34         borderRadius: 12,
35         fontSize: 12,
36         color: dark ? "#e2e8f0" : "#0f172a",
37       },
38     },
39   };
40 }
41

```

```

42 const shortRs = (v: number) => (v >= 1e7 ? `${(v / 1e7).toFixed(1)} Cr` : v >= 1e5 ? `${(v / 1e5).toFixed(
43 1)} Lakh` : v >= 1e3 ? `${(v / 1e3).toFixed(0)}k` : `${v}`);
44 export function MultiLineChart({ data, keys, xKey = "date", height = 300, unit = "Rs./kg" }: { data: Recor
45 d<string, unknown>[]; keys: string[]; xKey?: string; height?: number; unit?: string }) {
46   const a = useAxis();
47   return (
48     <ResponsiveContainer width="100%" height={height}>
49       <LineChart data={data} margin={{ top: 10, right: 10, left: -10, bottom: 0 }}>
50         <CartesianGrid stroke={a.grid} strokeDasharray="3 3" />
51         <XAxis dataKey={xKey} tick={a.tick} tickFormatter={(v: string) => (typeof v === "string" && v.leng
52 th === 10 ? v.slice(5) : v)} />
53         <YAxis tick={a.tick} domain={["auto", "auto"]} />
54         <Tooltip {...a.tooltip} formatter={(v) => [ `Rs. ${Number(v).toFixed(2)}`, undefined]} labelFormatt
55 er={(l) => `${l} (${unit})` } />
56         <Legend wrapperStyle={{ fontSize: 12 }} />
57         {keys.map((k, i) => (
58           <Line key={k} type="monotone" dataKey={k} stroke={PALETTE[i % PALETTE.length]} strokeWidth={2} d
59 ot={false} activeDot={{ r: 4 }} />
60         ))}
61       </LineChart>
62     </ResponsiveContainer>
63   );
64 }
65
66 export function CompareBarChart({ data, height = 320 }: { data: { cropName: string; mandiAvg: number; plat
67 formAsk: number | null; platformDeal: number | null }[]; height?: number }) {
68   const a = useAxis();
69   return (
70     <ResponsiveContainer width="100%" height={height}>
71       <BarChart data={data} margin={{ top: 10, right: 10, left: -10, bottom: 0 }}>
72         <CartesianGrid stroke={a.grid} strokeDasharray="3 3" vertical={false} />
73         <XAxis dataKey="cropName" tick={a.tick} interval={0} angle={-20} textAnchor="end" height={50} />
74         <YAxis tick={a.tick} />
75         <Tooltip {...a.tooltip} formatter={(v) => (v === null || v === undefined ? "-" : `Rs. ${Number(v).
76 toFixed(2)}/kg`) } />
77         <Legend wrapperStyle={{ fontSize: 12 }} />
78         <Bar dataKey="mandiAvg" name="Mandi avg" fill="#94a3b8" radius={[6, 6, 0, 0]} />
79         <Bar dataKey="platformAsk" name="Platform ask" fill="#f1c232" radius={[6, 6, 0, 0]} />
80         <Bar dataKey="platformDeal" name="Platform deal" fill="#01411c" radius={[6, 6, 0, 0]} />
81       </BarChart>
82     </ResponsiveContainer>
83   );
84 }
85
86 export function VolumeAreaChart({ data, height = 260, dataKey = "volume", color = "#117e48" }: { data: Rec
87 ord<string, unknown>[]; height?: number; dataKey?: string; color?: string }) {
88   const a = useAxis();
89   return (
90     <ResponsiveContainer width="100%" height={height}>
91       <AreaChart data={data} margin={{ top: 10, right: 10, left: -5, bottom: 0 }}>
92         <defs>
93           <linearGradient id={`g-${dataKey}`} x1="0" y1="0" x2="0" y2="1">
94             <stop offset="0%" stopColor={color} stopOpacity={0.35} />
95             <stop offset="100%" stopColor={color} stopOpacity={0} />
96           </linearGradient>
97         </defs>
98         <CartesianGrid stroke={a.grid} strokeDasharray="3 3" vertical={false} />
99         <XAxis dataKey="month" tick={a.tick} />
100         <YAxis tick={a.tick} tickFormatter={shortRs} />
101         <Tooltip {...a.tooltip} formatter={(v) => [ `Rs. ${Number(v).toLocaleString()}`, "Volume"] } />
102         <Area type="monotone" dataKey={dataKey} stroke={color} strokeWidth={2.5} fill={`url(#g-${dataKey})`
103 } />
104       </AreaChart>
105     </ResponsiveContainer>
106   );
107 }
108
109 export function SimpleBarChart({ data, xKey, yKey, height = 260, horizontal = false }: { data: Record<stri
110 ng, unknown>[]; xKey: string; yKey: string; height?: number; horizontal?: boolean }) {
111   const a = useAxis();
112   return (
113     <ResponsiveContainer width="100%" height={height}>
114       <BarChart data={data} layout={horizontal ? "vertical" : "horizontal"} margin={{ top: 5, right: 15, l
115 eft: horizontal ? 20 : -5, bottom: 0 }}>
116         <CartesianGrid stroke={a.grid} strokeDasharray="3 3" horizontal={!horizontal} vertical={horizontal

```

```

107     } />
108     {horizontal ? (
109       <XAxis type="number" tick={a.tick} tickFormatter={shortRs} />
110       <YAxis type="category" dataKey={xKey} tick={a.tick} width={90} />
111     ) />
112   ) : (
113     <XAxis dataKey={xKey} tick={a.tick} />
114     <YAxis tick={a.tick} tickFormatter={shortRs} />
115   ) />
116   <Bar dataKey={yKey} radius={horizontal ? [0, 6, 6, 0] : [6, 6, 0, 0]}>
117     {data.map((_, i) => (
118       <Cell key={i} fill={PALETTE[i % PALETTE.length]} />
119     ))}
120   </Bar>
121 </BarChart>
122 </ResponsiveContainer>
123 );
124 }
125
126 export function DonutChart({ data, height = 240 }: { data: { name: string; value: number }[]; height?: number }) {
127   const a = useAxis();
128   return (
129     <ResponsiveContainer width="100%" height={height}>
130       <PieChart>
131         <Pie data={data} dataKey="value" nameKey="name" innerRadius="55%" outerRadius="85%" paddingAngle={
132           2}>
133           {data.map((_, i) => (
134             <Cell key={i} fill={PALETTE[i % PALETTE.length]} />
135           ))}
136         </Pie>
137         <Tooltip {...a.tooltip} formatter={(v) => `Rs. ${Number(v).toLocaleString()}`} />
138         <Legend wrapperStyle={{ fontSize: 12 }} />
139       </PieChart>
140     </ResponsiveContainer>
141   );
142 }
143
144 }
145

```

src/components/city-options.tsx

```

1  import { CITIES, PROVINCES } from "@lib/constants";
2
3  /** <option>s for every Pakistani city grouped by province. Use inside a <select>. */
4  export function CityOptions() {
5    return (
6      <
7        {PROVINCES.map((p) => {
8          const cities = Object.entries(CITIES).filter(([, c]) => c.province === p);
9          if (!cities.length) return null;
10         return (
11           <optgroup key={p} label={p}>
12             {cities.map(([name, c]) => (
13               <option key={name} value={name}>
14                 {name} - {c.urdu}
15             </option>
16             ))}
17           </optgroup>
18         );
19       })}
20     </>
21   );
22 }
23

```

src/components/crop/bid-panel.tsx

```
1  "use client";
2
3  import Link from "next/link";
4  import { useState } from "react";
5  import { api, Spinner } from "@components/ui";
6  import { useToast } from "@components/providers";
7  import { formatPKR, todayISO } from "@lib/constants";
8
9  type Props = {
10    crop: { cropId: number; cropName: string; basePricePerKg: number; totalQuantityKg: number; available: boolean };
11    viewerRole: string | null;
12    isOwner: boolean;
13  };
14
15  /** Direct "Submit Bid / RFQ" call-to-action on the crop details page. */
16  export function BidPanel({ crop, viewerRole, isOwner }: Props) {
17    const { push } = useToast();
18    const [f, setF] = useState({ price: String(Math.round(crop.basePricePerKg * 0.97 * 100) / 100), qty: String(Math.min(crop.totalQuantityKg, 1000)), date: todayISO(7), message: "" });
19    const [busy, setBusy] = useState(false);
20    const [done, setDone] = useState(false);
21    const total = Number(f.price || 0) * Number(f.qty || 0);
22
23    if (!crop.available) return <div className="rounded-2xl bg-slate-100 p-4 text-sm text-slate-600 dark:bg-white/5 dark:text-slate-300">This listing is not accepting bids right now.</div>;
24    if (isOwner) return <Link href="/farmer/listings" className="btn-secondary w-full">[Urdu]Manage this listing</Link>;
25    if (!viewerRole)
26      return (
27        <div className="rounded-2xl border border-brand-200 bg-brand-50 p-4 dark:border-brand-900 dark:bg-brand-950/30">
28          <p className="text-sm font-semibold text-brand-900 dark:text-brand-100">Sign in as a buyer to submit a bid or RFQ</p>
29          <div className="mt-3 flex gap-2">
30            <Link href={`/${login?next=${encodeURIComponent(`/crop/${crop.cropId}`)}}` className="btn-primary flex-1">Log in</Link>
31            <Link href="/register?role=buyer" className="btn-secondary flex-1">Create buyer account</Link>
32          </div>
33        </div>
34      );
35    if (viewerRole !== "buyer") return <div className="rounded-2xl bg-slate-100 p-4 text-sm text-slate-600 dark:bg-white/5 dark:text-slate-300">Only buyer accounts can submit bids.</div>;
36    if (done)
37      return (
38        <div className="animate-fade-in rounded-2xl border border-emerald-200 bg-emerald-50 p-5 text-center dark:border-emerald-900 dark:bg-emerald-950/30">
39          <p className="text-3xl"></p>
40          <p className="mt-2 font-bold">Bid submitted</p>
41          <p className="text-sm text-slate-600 dark:text-slate-300">The farmer has been notified. You'll get a notification when they accept or counter.</p>
42          <Link href="/buyer/bids" className="btn-primary mt-4">Track my bids</Link>
43        </div>
44      );
45
46    async function submit(e: React.FormEvent) {
47      e.preventDefault();
48      setBusy(true);
49      try {
50        await api("/api/bids", { method: "POST", json: { cropId: crop.cropId, bidPricePerKg: Number(f.price), bidQuantityKg: Number(f.qty), targetDeliveryDate: f.date, message: f.message || undefined } });
51        setDone(true);
52      } catch (err) {
53        push({ kind: "error", title: "Bid not submitted", body: (err as Error).message });
54      } finally {
55        setBusy(false);
56      }
57    }
58
59    return (
60      <form onSubmit={submit} className="space-y-3 rounded-2xl border border-slate-200 bg-white p-4 shadow-sm dark:border-white/10 dark:bg-white/5">
61        <p className="font-bold">Submit bid / RFQ</p>
62      </form>
63    );
64  }
```

```

62     <div className="grid grid-cols-2 gap-3">
63         <div><label className="label">Price (Rs./kg)</label><input type="number" step="0.01" min={crop.bas
        ePricePerKg * 0.5} className="input" value={f.price} onChange={(e) => setF({ ...f, price: e.target.value }
        )} required /></div>
64         <div><label className="label">Quantity (kg)</label><input type="number" min={1} max={crop.totalQua
        ntityKg} className="input" value={f.qty} onChange={(e) => setF({ ...f, qty: e.target.value })} required />
        </div>
65         <div className="col-span-2 sm:col-span-1"><label className="label">Delivery by</label><input type=
        "date" min={todayISO(1)} className="input" value={f.date} onChange={(e) => setF({ ...f, date: e.target.val
        ue })} required /></div>
66         <div className="col-span-2 sm:col-span-1"><label className="label">Terms (optional)</label><input
        className="input" value={f.message} onChange={(e) => setF({ ...f, message: e.target.value })} placeholder=
        "Packing, pickup..." /></div>
67     </div>
68     <div className="flex items-center justify-between rounded-xl bg-brand-700 px-4 py-2.5 text-white">
69         <span className="text-sm">Offer value · {Math.round(Number(f.qty || 0) / 40)} maund</span>
70         <span className="text-lg font-extrabold">{formatPKR(total)}</span>
71     </div>
72     <button className="btn-primary w-full py-2.5" disabled={busy}>{busy && <Spinner />} Submit bid</butt
    on>
73     <p className="text-center text-[11px] text-slate-500">Funds are only locked in escrow after the farm
    er accepts.</p>
74 </form>
75 );
76 }
77

```

src/components/crop/price-chart.tsx

```

1  "use client";
2
3  import { CartesianGrid, Legend, Line, LineChart, ResponsiveContainer, Tooltip, XAxis, YAxis } from "rechar
    ts";
4  import { useTheme } from "@components/providers";
5
6  export function CropPriceChart({ data }: { data: { date: string; mandi: number | null; platform: number |
    null; ask: number }[] }) {
7      const { theme } = useTheme();
8      const dark = theme === "dark";
9      const tick = { fill: dark ? "#94a3b8" : "#64748b", fontSize: 11 };
10     return (
11         <ResponsiveContainer width="100%" height={300}>
12             <LineChart data={data} margin={{ top: 8, right: 12, left: -8, bottom: 0 }}>
13                 <CartesianGrid stroke={dark ? "#1e293b" : "#e2e8f0"} strokeDasharray="3 3" />
14                 <XAxis dataKey="date" tick={tick} tickFormatter={(v: string) => v.slice(5)} minTickGap={24} />
15                 <YAxis tick={tick} domain={["auto", "auto"]} />
16                 <Tooltip
17                     contentStyle={{ background: dark ? "#0f172a" : "#fff", border: `1px solid ${dark ? "#334155" : "
        #e2e8f0"}`, borderRadius: 12, fontSize: 12 }}
18                     formatter={(v, n) => [v === null || v === undefined ? "-" : `Rs. ${Number(v).toFixed(2)}/kg`, n]
        }
19                 />
20                 <Legend wrapperStyle={{ fontSize: 12 }} />
21                 <Line type="monotone" dataKey="mandi" name="Govt. mandi avg" stroke="#94a3b8" strokeWidth={2} dot=
        {false} connectNulls />
22                 <Line type="monotone" dataKey="platform" name="MarketLink deals" stroke="#117e48" strokeWidth={2.5
        } dot={{ r: 3 }} connectNulls />
23                 <Line type="stepAfter" dataKey="ask" name="This listing" stroke="#d9a514" strokeWidth={2} strokeDa
        sharray="6 4" dot={false} />
24             </LineChart>
25         </ResponsiveContainer>
26     );
27 }
28

```

src/components/dashboard/workflow-trail.tsx

```

1  import Link from "next/link";
2
3  type Step = {
4      label: string;
5      detail: string;
6      href: string;
7      /** Number of items that need the user's attention at this step. */

```

```

8   pending?: number;
9   };
10
11  /** A compact, role-specific map from the user's task to the next marketplace module. */
12  export function WorkflowTrail({ title = "Trading workflow", steps }: { title?: string; steps: Step[] }) {
13    return (
14      <section className="card p-4 sm:p-5" aria-label={title}>
15        <div className="flex items-baseline justify-between gap-3">
16          <h2 className="text-sm font-bold text-brand-950 dark:text-white">{title}</h2>
17          <span className="hidden text-[10px] font-semibold tracking-widest text-brand-700 uppercase sm:blo
18            k dark:text-gold-400">MARKETLINK AGRI-HUB</span>
19        </div>
20        <ol className="mt-3 flex gap-2 overflow-x-auto pb-1">
21          {steps.map((step, index) => (
22            <li key={step.label} className="relative min-w-[152px] flex-1">
23              <Link href={step.href} className="group flex h-full flex-col rounded-xl border border-slate-20
24                0 bg-slate-50/70 p-3 transition hover:border-brand-400 hover:bg-brand-50 focus-visible:outline-2 focus-vis
25                ible:outline-brand-500 dark:border-slate-700 dark:bg-slate-800/40 dark:hover:bg-brand-950/30">
26                <div className="flex items-center justify-between gap-2">
27                  <span className="grid h-6 w-6 place-items-center rounded-lg bg-brand-900 text-[10px] font-
28                    bold text-white">{String(index + 1).padStart(2, "0")}</span>
29                  {step.pending ? <span className="rounded-full bg-gold-400 px-2 py-0.5 text-[10px] font-bol
30                    d text-brand-950">{step.pending} pending</span> : null}
31                </div>
32                <span className="mt-2 text-xs font-bold text-slate-900 group-hover:text-brand-800 dark:text-
33                  slate-100 dark:group-hover:text-gold-300">{step.label} <span aria-hidden></span></span>
34                <span className="mt-0.5 text-[11px] leading-snug text-slate-500 dark:text-slate-400">{step.d
35                  etail}</span>
36              </Link>
37            </li>
38          ))}
39        </ol>
40      </section>
41    );
42  }
43
44  }
45
46  }

```

src/components/mandi-insights.tsx

```

1  "use client";
2
3  import { useEffect, useMemo, useState } from "react";
4  import { api, LoadingBlock, PageHeader } from "@components/ui";
5  import { CompareBarChart, MultiLineChart } from "@components/charts";
6  import { useToast } from "@components/providers";
7  import { CITIES, CROP_CATALOGUE, MANDIS, MANDI_MARKETS, PROVINCES, formatPKR, mandiName, cropUrdu } from "
8    @/lib/constants";
9
10 type Latest = { rateId: number; cropName: string; marketLocation: string; minPricePerKg: number; maxPriceP
11   erKg: number; avgPricePerKg: number; changePct: number | null; source: string; dateUpdated: string };
12 type Compare = { cropName: string; mandiAvg: number; platformAsk: number | null; platformDeal: number | nu
13   ll; supplyKg: number; farmerGainPct: number | null };
14 type Trend = { markets: string[]; series: Record<string, number | string>[] };
15
16 export function MandiInsights() {
17   const { push } = useToast();
18   const [crop, setCrop] = useState("Wheat");
19   const [days, setDays] = useState(14);
20   const [market, setMarket] = useState("");
21   const [province, setProvince] = useState("");
22   const [latest, setLatest] = useState<Latest[] | null>(null);
23   const [compare, setCompare] = useState<Compare[]>([]);
24   const [trend, setTrend] = useState<Trend | null>(null);
25
26   useEffect(() => {
27     Promise.all([api<Latest[]>("/api/mandi-rates?view=latest"), api<Compare[]>("/api/mandi-rates?view=comp
28       are")])
29       .then(([l, c]) => { setLatest(l); setCompare(c); })
30       .catch((e) => push({ kind: "error", title: e.message }));
31   }, [push]);
32
33   useEffect(() => {
34     api<Trend>(`/api/mandi-rates?view=trend&crop=${encodeURIComponent(crop)}&days=${days}`).then(setTrend)
35     .catch(() => {});
36   }, [crop, days]);
37
38   }
39
40   }

```

```

32
33     const board = useMemo(() => (latest ?? []).filter((r) => (!market || r.marketLocation === market) && (!p
    province || CITIES[r.marketLocation]?.province === province) && r.cropName === crop), [latest, market, prov
    ince, crop]);
34     const movers = useMemo(() => {
35         const byCrop: Record<string, number[]> = {};
36         (latest ?? []).forEach((r) => r.changePct !== null && (byCrop[r.cropName] = [...(byCrop[r.cropName] ??
    []), r.changePct]));
37         return Object.entries(byCrop).map(([c, v]) => ({ crop: c, change: v.reduce((a, b) => a + b, 0) / v.len
    gth })).sort((a, b) => b.change - a.change);
38     }, [latest]);
39     const lastUpdated = latest?.[0]?.dateUpdated;
40
41     if (!latest) return <LoadingBlock />;
42     return (
43         <div className="space-y-6">
44             <PageHeader title="Live Mandi Insights" urdu="[Urdu]" subtitle={`Daily wholesale rates from ${MANDI_
    MARKETS.length} Pakistani mandis (Badami Bagh, Multan, Karachi, Quetta...) · last updated ${lastUpdated ?? "
    -"} PKT`} />
45
46             <div className="flex gap-2 overflow-x-auto pb-1">
47                 {movers.map((m) => (
48                     <button key={m.crop} onClick={() => setCrop(m.crop)} className={`card shrink-0 px-4 py-2.5 text-
    left ${crop === m.crop ? "ring-2 ring-brand-500" : ""}`}>
49                     <p className="text-xs font-semibold text-slate-500">{m.crop}</p>
50                     <p className={`text-sm font-bold ${m.change >= 0 ? "text-emerald-600" : "text-rose-600"}`}>{m.
    change >= 0 ? "▲" : "▼"} {Math.abs(m.change).toFixed(2)}%</p>
51                 </button>
52             )})
53         </div>
54
55         <div className="card p-5">
56             <div className="mb-4 flex flex-wrap items-center justify-between gap-3">
57                 <div>
58                     <h2 className="font-semibold">{crop} <span className="font-normal text-slate-500">{cropUrdu(cr
    op)}</span> – avg price by mandi</h2>
59                     <p className="text-xs text-slate-500">PKR per kg, daily average (× 40 for per-maund rate)</p>
60                 </div>
61                 <div className="flex gap-2">
62                     <select className="input w-40" value={crop} onChange={(e) => setCrop(e.target.value)}>{CROP_CA
    TALOGUE.map((c) => <option key={c.name}>{c.name}</option>)}</select>
63                     <select className="input w-28" value={days} onChange={(e) => setDays(Number(e.target.value))}>
    <option value={7}>7 days</option><option value={14}>14 days</option><option value={30}>30 days</option></s
    elect>
64                 </div>
65             </div>
66             {trend ? <MultiLineChart data={trend.series} keys={trend.markets} /> : <div className="h-[300px] a
    nimate-pulse rounded-xl bg-slate-100 dark:bg-slate-800" />}
67         </div>
68
69         <div className="grid gap-4 lg:grid-cols-5">
70             <div className="card p-5 lg:col-span-3">
71                 <h2 className="font-semibold">Government mandi rate vs MarketLink platform rate</h2>
72                 <p className="mb-3 text-xs text-slate-500">Mandi avg (latest) vs active listing ask vs realised
    platform deal price (90d)</p>
73                 <CompareBarChart data={compare} />
74             </div>
75             <div className="card overflow-hidden lg:col-span-2">
76                 <h2 className="border-b border-slate-200 px-5 py-3.5 font-semibold dark:border-slate-800">Farmer
    price advantage</h2>
77                 <ul className="divide-y divide-slate-100 dark:divide-slate-800">
78                     {compare.map((c) => (
79                         <li key={c.cropName} className="flex items-center justify-between px-5 py-2.5 text-sm">
80                             <span className="font-medium">{c.cropName}</span>
81                             <span className="text-slate-500">{formatPKR(c.mandiAvg * 40)} → {c.platformDeal ? formatPK
    R(c.platformDeal * 40) : "-"} /maund</span>
82                             <span className={`w-16 text-right font-bold ${c.farmerGainPct === null ? "text-slate-400"
    : c.farmerGainPct >= 0 ? "text-emerald-600" : "text-rose-600"}`}>
83                                 {c.farmerGainPct === null ? "-" : `${c.farmerGainPct >= 0 ? "+" : ""}${c.farmerGainPct.t
    oFixed(1)}%`}
84                             </span>
85                         </li>
86                     )})
87                 </ul>
88             </div>
89         </div>

```

```

90
91     <div className="card overflow-hidden">
92       <div className="flex flex-wrap items-center justify-between gap-3 border-b border-slate-200 px-5 p
93         y-3.5 dark:border-slate-800">
94         <h2 className="font-semibold">Today's rate board · {crop}</h2>
95         <select className="input w-40" value={province} onChange={(e) => { setProvince(e.target.value);
96           setMarket(""); }} aria-label="Province"><option value="">All provinces</option>{PROVINCES.filter((p) => MA
97           NDIS.some((m) => CITIES[m.key]?.province === p)).map((p) => <option key={p}>{p}</option>)}</select>
98         <select className="input w-44" value={market} onChange={(e) => setMarket(e.target.value)}><optio
99           n value="">All mandis</option>{MANDIS.filter((m) => !province || CITIES[m.key]?.province === province).map
100           ((m) => <option key={m.key} value={m.key}>{m.name}</option>)}</select>
101       </div>
102       <div className="overflow-x-auto">
103         <table className="w-full text-sm">
104           <thead className="table-head"><tr><th className="px-5 py-2.5">Mandi</th><th className="px-5 py
105             -2.5">Min</th><th className="px-5 py-2.5">Max</th><th className="px-5 py-2.5">Average /kg</th><th classNam
106             e="px-5 py-2.5">Per maund (40kg)</th><th className="px-5 py-2.5">Day change</th><th className="px-5 py-2.5
107             ">Source</th><th className="px-5 py-2.5">Date</th></tr></thead>
108           <tbody className="divide-y divide-slate-100 dark:divide-slate-800">
109             {board.map((r) => (
110               <tr key={r.rateId}>
111                 <td className="px-5 py-3 font-medium">{mandiName(r.marketLocation)}</td>
112                 <td className="px-5 py-3">{formatPKR(r.minPricePerKg, 2)}</td>
113                 <td className="px-5 py-3">{formatPKR(r.maxPricePerKg, 2)}</td>
114                 <td className="px-5 py-3 font-bold">{formatPKR(r.avgPricePerKg, 2)}</td>
115                 <td className="px-5 py-3 font-bold text-brand-700 dark:text-brand-400">{formatPKR(r.avgP
116                   ricePerKg * 40)}</td>
117                 <td className={`px-5 py-3 font-semibold ${r.changePct === null ? "" : r.changePct >= 0 ?
118                   "text-emerald-600" : "text-rose-600"}`}>{r.changePct === null ? "—" : `${r.changePct >= 0 ? "▲" : "▼"} ${
119                   Math.abs(r.changePct).toFixed(2)}%`}</td>
120                 <td className="px-5 py-3"><span className={`badge ${r.source === "feed" ? "bg-sky-100 te
121                   xt-sky-700 dark:bg-sky-500/15 dark:text-sky-300" : "bg-slate-100 text-slate-600 dark:bg-slate-800 dark:tex
122                   t-slate-300"}`}>{r.source}</span></td>
123                 <td className="px-5 py-3 text-slate-500">{r.dateUpdated}</td>
124               </tr>
125             ))}
126           </tbody>
127         </table>
128       </div>
129     </div>
130   );
131 }
132

```

src/components/mandi-ticker.tsx

```

1  export type TickerItem = { crop: string; urdu: string; avg: number; change: number | null };
2
3  /** Horizontally scrolling live mandi rates strip (PKR per 40kg maund). */
4  export function MandiTicker({ items, variant = "light" }: { items: TickerItem[]; variant?: "light" | "dark"
5    " }) {
6    if (!items.length) return null;
7    const row = [...items, ...items];
8    const dark = variant === "dark";
9    return (
10      <div className={`relative flex items-center overflow-hidden text-xs ${dark ? "bg-brand-950 text-white"
11        : "border-b border-brand-900/10 bg-brand-50/70 text-slate-700 dark:border-white/5 dark:bg-brand-950/60 da
12        rk:text-slate-200"}`} dir="ltr">
13        <span className={`z-10 flex shrink-0 items-center gap-1.5 px-3 py-1.5 font-bold tracking-wide upperc
14          ase ${dark ? "bg-gold-400 text-brand-950" : "bg-brand-900 text-white"}`}>
15          <span className="relative flex h-2 w-2"><span className="absolute inline-flex h-full w-full animat
16            e-ping rounded-full bg-red-400 opacity-75" /><span className="relative inline-flex h-2 w-2 rounded-full bg
17            -red-500" /></span>
18          Live Mandi
19        </span>
20        <div className="flex overflow-hidden">
21          <div className="animate-marquee flex shrink-0 gap-7 py-1.5 pl-6 whitespace-nowrap">
22            {row.map((t, i) => (
23              <span key={i} className="flex items-center gap-1.5">
24                <span className="font-semibold">{t.crop}</span>
25                <span className="opacity-60">{t.urdu}</span>
26                <span className="font-mono">Rs. {Math.round(t.avg * 40).toLocaleString("en-US")}/maund</span>
27              >
28                {t.change !== null && (

```

```

22         <span className={`font-semibold ${t.change >= 0 ? (dark ? "text-emerald-300" : "text-emera
↳ ld-600") : "text-rose-500"}`} >
23             {t.change >= 0 ? "▲" : "▼"} {Math.abs(t.change).toFixed(1)}%
24         </span>
25     )}
26 </span>
27 )})
28 </div>
29 </div>
30 </div>
31 );
32 }
33

```

src/components/marketplace/marketplace-client.tsx

```

1  "use client";
2
3  import Link from "next/link";
4  import { useEffect, useMemo, useState } from "react";
5  import { api, EmptyState, GradeBadge, LoadingBlock, Modal, PageHeader, Spinner } from "@components/ui";
6  import { useI18n, useToast } from "@components/providers";
7  import { CityOptions } from "@components/city-options";
8  import { CATEGORIES, PROVINCES, cropImage, perMaund, formatDate, formatKg, formatPKR, todayISO, type Categ
↳ ory, type Grade } from "@lib/constants";
9
10 type Listing = {
11   cropId: number; cropName: string; category: Category; qualityGrade: Grade; totalQuantityKg: number; base
↳ PricePerKg: number; harvestDate: string;
12   imagesJson: string[]; description: string | null; farmLocation: string; farmerName: string; farmName: st
↳ ring | null; farmerVerified: boolean;
13   openBids: number; highestBid: number | null; inspectionGrade: string | null; distanceKm: number | null;
↳ trustScore: number; ratingCount: number;
14 };
15 type Rate = { cropName: string; avgPricePerKg: number };
16
17 export type MarketFilters = { q?: string; category?: string; grade?: string; sort?: string; farmerId?: str
↳ ing };
18
19 export function MarketplaceClient({ homeCity, initial = {}, viewerRole = null }: { homeCity: string; initi
↳ al?: MarketFilters; viewerRole?: string | null }) {
20   const [view, setView] = useState<"grid" | "list">("grid");
21   const canBid = viewerRole === "buyer";
22   const { push } = useToast();
23   const { t } = useI18n();
24   const [items, setItems] = useState<Listing[] | null>(null);
25   const empty = { q: "", category: "", grade: "", minPrice: "", maxPrice: "", minQty: "", province: "", or
↳ iginCity: homeCity, radiusKm: "", sort: "newest", farmerId: "" };
26   const [f, setF] = useState({ ...empty, ...Object.fromEntries(Object.entries(initial).filter(([k, v]) => v
↳ )) });
27   const [mandi, setMandi] = useState<Record<string, number>>({});
28   const [target, setTarget] = useState<Listing | null>(null);
29   const [bid, setBid] = useState({ price: "", qty: "", date: todayISO(7), message: "" });
30   const [busy, setBusy] = useState(false);
31   const [showFilters, setShowFilters] = useState(false);
32
33   const query = useMemo(() => {
34     const p = new URLSearchParams();
35     Object.entries(f).forEach(([k, v]) => v && p.set(k, v));
36     return p.toString();
37   }, [f]);
38
39   useEffect(() => {
40     let alive = true;
41     const h = setTimeout(() => {
42       api<Listing[]>(`/api/crops?${query}`)
43         .then((d) => alive && setItems(d))
44         .catch((e) => push({ kind: "error", title: e.message }));
45     }, 250);
46     return () => { alive = false; clearTimeout(h); };
47   }, [query, push]);
48
49   useEffect(() => {
50     api<Rate[]>(`/api/mandi-rates?view=latest`).then((rows) => {
51       const acc: Record<string, { s: number; n: number }> = {};

```

```

52     rows.forEach((r) => { acc[r.cropName] = { s: (acc[r.cropName]?.s ?? 0) + r.avgPricePerKg, n: (acc[r.
    cropName]?.n ?? 0) + 1 }; });
53     setMandi(Object.fromEntries(Object.entries(acc).map(([k, v]) => [k, v.s / v.n])));
54     }).catch(() => {});
55     }, []);
56
57     function openBid(l: Listing) {
58         setTarget(l);
59         setBid({ price: String(Math.round(l.basePricePerKg * 0.97 * 100) / 100), qty: String(Math.min(l.totalQ
    uantityKg, 1000)), date: todayISO(7), message: "" });
60     }
61
62     async function submitBid(e: React.FormEvent) {
63         e.preventDefault();
64         if (!target) return;
65         setBusy(true);
66         try {
67             await api("/api/bids", { method: "POST", json: { cropId: target.cropId, bidPricePerKg: Number(bid.pr
    ice), bidQuantityKg: Number(bid.qty), targetDeliveryDate: bid.date, message: bid.message || undefined } })
        ;
68             push({ kind: "success", title: "Bid submitted", body: `${target.farmName ?? target.farmerName} will
    respond shortly.` });
69             setItems((prev) => prev?.map((x) => (x.cropId === target.cropId ? { ...x, openBids: x.openBids + 1,
    highestBid: Math.max(x.highestBid ?? 0, Number(bid.price)) } : x)) ?? null);
70             setTarget(null);
71         } catch (err) {
72             push({ kind: "error", title: "Bid rejected", body: (err as Error).message });
73         } finally {
74             setBusy(false);
75         }
76     }
77
78     const set = (k: keyof typeof f) => (e: React.ChangeEvent<HTMLInputElement | HTMLSelectElement>) => setF(
    { ...f, [k]: e.target.value });
79     const total = Number(bid.price || 0) * Number(bid.qty || 0);
80
81     return (
82         <div>
83             <PageHeader title={t("marketplace")} urdu="[Urdu]" subtitle="Buy directly from verified farms" actio
    ns={
84                 <div className="flex items-center gap-2">
85                     <div className="flex rounded-xl border border-slate-200 p-0.5 dark:border-slate-700" role="group
    " aria-label="View">
86                         {[ "grid", "list" ] as const}.map((v) => (
87                             <button key={v} onClick={() => setView(v)} className={`rounded-lg px-3 py-1.5 text-xs font-s
    emibold capitalize ${view === v ? "bg-brand-900 text-white" : "text-slate-500"}`} aria-pressed={view === v
    }>{v === "grid" ? "■ Grid" : "List"}</button>
88                         ))
89                     </div>
90                     {canBid && <Link href="/buyer/bids" className="btn-secondary"> My bids</Link>}
91                 </div> </div> </div>
92
93                 <div className="mb-4 flex gap-2 overflow-x-auto pb-1">
94                     <button onClick={() => setF({ ...f, category: "" })} className={`shrink-0 rounded-full px-4 py-2 t
    ext-sm font-semibold ${!f.category ? "bg-brand-600 text-white" : "bg-white ring-1 ring-slate-200 dark:bg-s
    late-900 dark:ring-slate-700"}`}> All</button>
95                     {CATEGORIES.map((c) => (
96                         <button key={c.value} onClick={() => setF({ ...f, category: c.value })} className={`shrink-0 rou
    nded-full px-4 py-2 text-sm font-semibold ${f.category === c.value ? "bg-brand-600 text-white" : "bg-white
    ring-1 ring-slate-200 dark:bg-slate-900 dark:ring-slate-700"}`}>
97                         {c.emoji} {c.label}
98                     </button>
99                     ))
100                 </div>
101
102                 <div className="card mb-5 p-4">
103                     <div className="flex gap-2">
104                         <input className="input flex-1" placeholder="Search crops (e.g. Wheat, Mango)..." value={f.q} onC
    hange={set("q")} />
105                         <button className="btn-secondary md:hidden" onClick={() => setShowFilters((v) => !v)}>⦿[Urdu]Fil
    ters</button>
106                         <select className="input hidden w-44 md:block" value={f.sort} onChange={set("sort")}>
107                             <option value="newest">Newest first</option><option value="price_asc">Price: low → high</optio
    n><option value="price_desc">Price: high → low</option><option value="distance">Nearest first</option>
108                         </select>
109                     </div>

```

```

110     <div className={`mt-3 grid-cols-2 gap-3 md:grid md:grid-cols-4 xl:grid-cols-8 ${showFilters ? "grid" : "hidden"}`}>
111       <div><label className="label">Min quantity</label><select className="input" value={f.minQty} onChange={set("minQty")}><option value="">Any</option><option value="1000">≥ 1 ton</option><option value="5000">≥ 5 tons</option><option value="10000">≥ 10 tons</option><option value="25000">≥ 25 tons</option></select></div>
112       <div><label className="label">Province</label><select className="input" value={f.province} onChange={set("province")}><option value="">All</option>{PROVINCES.map((p) => <option key={p}>{p}</option>)}</select></div>
113       <div><label className="label">Grade</label><select className="input" value={f.grade} onChange={set("grade")}><option value="">Any</option><option>A</option><option>B</option><option>C</option></select></div>
114       <div><label className="label">Min Rs./kg</label><input type="number" className="input" value={f.minPrice} onChange={set("minPrice")} /></div>
115       <div><label className="label">Max Rs./kg</label><input type="number" className="input" value={f.maxPrice} onChange={set("maxPrice")} /></div>
116       <div><label className="label">From city</label><select className="input" value={f.originCity} onChange={set("originCity")}><CityOptions /></select></div>
117       <div><label className="label">Radius</label><select className="input" value={f.radiusKm} onChange={set("radiusKm")}><option value="">Nationwide</option><option value="50">≤ 50 km</option><option value="150">≤ 150 km</option><option value="300">≤ 300 km</option><option value="600">≤ 600 km</option></select></div>
118       <div className="md:hidden"><label className="label">Sort</label><select className="input" value={f.sort} onChange={set("sort")}><option value="newest">Newest</option><option value="price_asc">Price ↑</option><option value="price_desc">Price ↓</option><option value="distance">Nearest</option></select></div>
119       <div className="flex items-end"><button className="btn-secondary w-full" onClick={() => setF(emp ty)}>Reset</button></div>
120     </div>
121   </div>
122
123   {!items ? <LoadingBlock /> : items.length === 0 ? (
124     <EmptyState icon="" title="No produce matches your filters" body="Try widening the radius or clearing the grade/price filters." />
125   ) : (
126     <>
127       {f.farmerId && items.length > 0 && (
128         <div className="mb-3 flex items-center gap-2 text-sm">
129           <span className="rounded-full bg-brand-50 px-3 py-1 font-medium text-brand-800 dark:bg-brand-500/10 dark:text-brand-300">Seller: {items[0].farmName ?? items[0].farmerName}</span>
130           <button className="text-xs font-semibold text-slate-500 hover:text-slate-800 dark:hover:text-slate-200" onClick={() => setF({ ...f, farmerId: "" })}>× Show all sellers</button>
131         </div>
132       )}
133       <p className="mb-3 text-sm text-slate-500">{items.length} listing(s) · distances from {f.originCity}</p>
134       <div className={view === "grid" ? "grid gap-4 sm:grid-cols-2 xl:grid-cols-3" : "grid gap-3">
135         {items.map((l) => {
136           const m = mandi[l.cropName];
137           const vs = m ? ((l.basePricePerKg - m) / m) * 100 : null;
138           return (
139             <article key={l.cropId} className={`card group overflow-hidden transition hover:shadow-lg ${view === "grid" ? "flex flex-col hover:-translate-y-0.5" : "flex flex-col sm:flex-row"}`>
140               <div className={`relative overflow-hidden ${view === "grid" ? "h-44" : "h-44 sm:h-auto sm:w-56 sm:shrink-0"}`>
141                 <!-- eslint-disable-next-line @next/next/no-img-element -->
142                 <img src={cropImage(l.imagesJson, l.cropName, l.category)} alt={l.cropName} className="h-full w-full object-cover transition duration-500 group-hover:scale-105" />
143                 <div className="absolute top-3 left-3 flex gap-1.5"><GradeBadge grade={l.qualityGrade} />{l.inspectionGrade} && <span className="badge bg-white/90 text-emerald-700"> Inspected</span></div>
144                 {l.distanceKm !== null && <span className="badge absolute top-3 right-3 bg-black/60 text-white"> {l.distanceKm} km</span>}
145               </div>
146               <div className="flex flex-1 flex-col p-4">
147                 <div className="flex items-start justify-between gap-2">
148                   <div className="min-w-0">
149                     <h3 className="font-semibold"><Link href={` /crop/${l.cropId}` } className="hover:text-brand-700 hover:underline dark:hover:text-brand-400">{l.cropName}</Link></h3>
150                     <p className="truncate text-xs text-slate-500">{l.farmName ?? l.farmerName} {l.farmerVerified && <span className="text-brand-600">✓</span>}{l.trustScore > 0 && <span className="ml-1 text-amber-600">★ {Number(l.trustScore).toFixed(1)}</span>} · {l.farmLocation}</p>
151                   </div>
152                   <div className="text-right">
153                     <p className="text-lg font-bold text-brand-700 dark:text-brand-400">{formatPKR(l.basePricePerKg, 2)}<span className="text-xs font-normal text-slate-500">/kg</span></p>
154                     <p className="text-[11px] text-slate-500">{perMaund(l.basePricePerKg)}</p>
155                     {vs !== null && <p className="text-[11px] font-semibold ${vs <= 0 ? "text-emerald

```

```

-600" : "text-amber-600"}>{vs <= 0 ? "▼" : "▲"} {Math.abs(vs).toFixed(1)}% vs mandi</p>
156 </div>
157 </div>
158 {l.description && <p className="mt-2 line-clamp-2 text-xs text-slate-600 dark:text-sla
te-400">{l.description}</p>}
159 <div className="mt-3 grid grid-cols-3 gap-2 rounded-xl bg-slate-50 p-2.5 text-center t
ext-xs dark:bg-slate-800/60">
160 <div><p className="text-slate-500">Available</p><p className="font-semibold">{format
Kg(l.totalQuantityKg)}</p></div>
161 <div><p className="text-slate-500">Harvested</p><p className="font-semibold">{format
Date(l.harvestDate).slice(0, 6)}</p></div>
162 <div><p className="text-slate-500">Bids</p><p className="font-semibold">{l.openBids}
{l.highestBid ? ` · ${Math.round(l.highestBid)}` : ""}</p></div>
163 </div>
164 <div className="mt-4 flex gap-2">
165 <Link href={` /crop/${l.cropId}`} className="btn-secondary flex-1">Details</Link>
166 {canBid ? (
167 <button className="btn-primary flex-1" onClick={() => openBid(l)}> {t("placeBid")}
</button>
168 ) : (
169 <Link href={viewerRole ? ` /crop/${l.cropId}` : ` /login?next=${encodeURIComponent(`
/crop/${l.cropId}`)} } className="btn-primary flex-1"> {viewerRole ? "View" : "Sign in to bid"}</Link>
170 )}
171 </div>
172 </div>
173 </article>
174 );
175 }}}
176 </div>
177 </>
178 )}
179
180 <Modal open={!target} onClose={() => setTarget(null)} title={target ? `Bulk offer · ${target.cropNa
me}` : ""}>
181 {target && (
182 <form onSubmit={submitBid} className="space-y-4">
183 <div className="flex items-center gap-3 rounded-xl bg-slate-50 p-3 text-sm dark:bg-slate-800">
184 { /* eslint-disable-next-line @next/next/no-img-element */ }
185 <img src={cropImage(target.imagesJson, target.cropName, target.category)} alt="" className="
h-12 w-12 rounded-lg object-cover" />
186 <div className="flex-1">
187 <p className="font-semibold">{target.farmName ?? target.farmerName} · Grade {target.qualit
yGrade}</p>
188 <p className="text-xs text-slate-500">Ask {formatPKR(target.basePricePerKg, 2)}/kg · {form
atKg(target.totalQuantityKg)} available{mandi[target.cropName] ? ` · Mandi avg ${formatPKR(mandi[target.cr
opName], 2)}` : ""}</p>
189 </div>
190 </div>
191 <div className="grid grid-cols-2 gap-3">
192 <div><label className="label">Your price (Rs./kg)</label><input type="number" step="0.01" mi
n={target.basePricePerKg * 0.5} className="input" value={bid.price} onChange={(e) => setBid({ ...bid, pric
e: e.target.value })} required /></div>
193 <div><label className="label">Quantity (kg)</label><input type="number" min="1" max={target.
totalQuantityKg} className="input" value={bid.qty} onChange={(e) => setBid({ ...bid, qty: e.target.value }
)} required /></div>
194 </div>
195 <div className="flex flex-wrap gap-1.5">
196 {[0.25, 0.5, 1].map((p) => (
197 <button type="button" key={p} className="rounded-full bg-slate-100 px-3 py-1 text-xs font-
semibold dark:bg-slate-800" onClick={() => setBid({ ...bid, qty: String(Math.floor(target.totalQuantityKg
* p)) }}>{p * 100}% ({formatKg(target.totalQuantityKg * p)})</button>
198 )})}
199 <button type="button" className="rounded-full bg-brand-50 px-3 py-1 text-xs font-semibold te
xt-brand-700 dark:bg-brand-500/10 dark:text-brand-300" onClick={() => setBid({ ...bid, price: String(targe
t.basePricePerKg) })}>Match ask price</button>
200 </div>
201 <div><label className="label">Target delivery date</label><input type="date" min={todayISO(1)}
className="input" value={bid.date} onChange={(e) => setBid({ ...bid, date: e.target.value })} required />
</div>
202 <div><label className="label">Message / RFQ terms</label><textarea className="input" rows={2}
value={bid.message} onChange={(e) => setBid({ ...bid, message: e.target.value })} placeholder="Packaging,
pickup, payment terms..." /></div>
203 <div className="flex items-center justify-between rounded-xl bg-brand-600 p-3 text-white"><spa
n className="text-sm">Offer value · {Math.round(Number(bid.qty || 0) / 40)} maund</span><span className="t
ext-xl font-extrabold">{formatPKR(total)}</span></div>
204 <p className="text-xs text-slate-500">If accepted, you&apos;ll lock this amount in escrow. Fun

```

```

ds are released to the farmer only after you confirm delivery.</p>
205     <div className="flex justify-end gap-2">
206         <button type="button" className="btn-secondary" onClick={() => setTarget(null)}>Cancel</butt
    on>
207         <button className="btn-primary" disabled={busy}>{busy && <Spinner />} Submit bid</button>
208     </div>
209 </form>
210   })
211 </Modal>
212 </div>
213   );
214 }
215

```

src/components/nav/mark-all-read.tsx

```

1  "use client";
2
3  import { useRouter } from "next/navigation";
4
5  export function MarkAllRead() {
6    const router = useRouter();
7    return (
8      <button className="btn-secondary" onClick={async () => { await fetch("/api/notifications", { method: "
    PATCH", headers: { "content-type": "application/json" }, body: "{}" }); router.refresh(); }}>
9        ✓ Mark all read
10     </button>
11   );
12 }
13

```

src/components/nav/nav-actions.tsx

```

1  "use client";
2
3  import Link from "next/link";
4  import { useRouter } from "next/navigation";
5  import { useCallback, useEffect, useRef, useState } from "react";
6  import { useI18n, useTheme } from "@components/providers";
7
8  export type NavViewer = {
9    fullName: string;
10    role: "farmer" | "buyer" | "admin" | "inspector";
11    walletBalance: number;
12    escrowBalance: number;
13    unread: number;
14    dashboard: string;
15    title: string | null;
16    avatarUrl: string | null;
17  } | null;
18
19  type Notif = { notificationId: number; title: string; body: string | null; link: string | null; isRead: bo
    olean; createdAt: string; type: string };
20
21  const ROLE_LABEL = { farmer: "Farmer", buyer: "Buyer", admin: "Admin", inspector: "Quality Inspector" };
22  const TYPE_ICON: Record<string, string> = { bid: "", bid_accepted: "", escrow_locked: "", shipment: "", de
    livered: "", inspection: "", dispute: "🗣️[Urdu]", payment: "", payout: "", review: "", review_reply: "", we
    lcome: "", security: "" };
23
24  const shortRs = (n: number) => (n >= 1e7 ? `${(n / 1e7).toFixed(2)} Cr` : n >= 1e5 ? `${(n / 1e5).toFixed(
    1)} Lakh` : Math.round(n).toLocaleString("en-US"));
25
26  function useOutside(ref: React.RefObject<HTMLElement | null>, onOut: () => void) {
27    useEffect(() => {
28      const h = (e: MouseEvent) => ref.current && !ref.current.contains(e.target as Node) && onOut();
29      const k = (e: KeyboardEvent) => e.key === "Escape" && onOut();
30      document.addEventListener("mousedown", h);
31      document.addEventListener("keydown", k);
32      return () => {
33        document.removeEventListener("mousedown", h);
34        document.removeEventListener("keydown", k);
35      };
36    }, [ref, onOut]);
37  }

```

```

38
39 export function NotificationBell({ initialUnread, tone = "light" }: { initialUnread: number; tone?: "light
  | "dark" }) {
40   const [open, setOpen] = useState(false);
41   const [unread, setUnread] = useState(initialUnread);
42   const [items, setItems] = useState<Notif[] | null>(null);
43   const ref = useRef<HTMLDivElement>(null);
44   const close = useCallback(() => setOpen(false), []);
45   useOutside(ref, close);
46
47   const load = useCallback(async () => {
48     const r = await fetch("/api/notifications", { cache: "no-store" }).then((x) => x.json()).catch(() => n
ull);
49     if (r?.success) {
50       setItems(r.data.items);
51       setUnread(r.data.unread);
52     }
53   }, []);
54
55   useEffect(() => {
56     const id = setInterval(load, 45_000);
57     return () => clearInterval(id);
58   }, [load]);
59
60   async function toggle() {
61     const next = !open;
62     setOpen(next);
63     if (next) await load();
64   }
65   async function markAll() {
66     await fetch("/api/notifications", { method: "PATCH", headers: { "content-type": "application/json" },
body: "{}" });
67     setUnread(0);
68     setItems((xs) => xs?.map((x) => ({ ...x, isRead: true })) ?? null);
69   }
70
71   return (
72     <div className="relative" ref={ref}>
73       <button onClick={toggle} aria-label={`Notifications${unread ? ` (${unread} unread)` : ""}`} aria-exp
anded={open}
74         className={`relative grid h-9 w-9 place-items-center rounded-xl transition ${tone === "dark" ? "te
xt-white/85 hover:bg-white/10" : "border border-slate-200 text-slate-600 hover:bg-slate-100 dark:border-sl
ate-700 dark:text-slate-300 dark:hover:bg-slate-800"}`}>
75         <svg viewBox="0 0 24 24" className="h-[18px] w-[18px]" fill="none" stroke="currentColor" strokeWid
th="2" strokeLinecap="round" strokeLinejoin="round"><path d="M6 8a6 6 0 1 1 12 0c0 7 3 9 3 9H3s3-2 3-9" />
<path d="M10.3 21a1.94 1.94 0 0 0 3.4 0" /></svg>
76         {unread > 0 && <span className="absolute -top-1 -right-1 grid h-[18px] min-w-[18px] place-items-ce
nter rounded-full bg-rose-600 px-1 text-[10px] font-bold text-white ring-2 ring-white dark:ring-slate-950"
>{unread > 9 ? "9+" : unread}</span>}
77       </button>
78       {open && (
79         <div className="animate-fade-in absolute right-0 z-50 mt-2 w-[min(92vw,360px)] overflow-hidden rou
nded-2xl border border-slate-200 bg-white text-slate-900 shadow-2xl dark:border-white/10 dark:bg-#0d1f16]
dark:text-slate-100">
80           <div className="flex items-center justify-between border-b border-slate-100 px-4 py-3 dark:borde
r-white/5">
81             <p className="text-sm font-bold">Notifications</p>
82             {unread > 0 && <button onClick={markAll} className="text-xs font-semibold text-brand-700 hover
:underline dark:text-brand-400">Mark all read</button>}
83           </div>
84           <ul className="max-h-[380px] divide-y divide-slate-100 overflow-y-auto dark:divide-white/5">
85             {items === null && <li className="p-6 text-center text-sm text-slate-500">Loading...</li>}
86             {items?.length === 0 && <li className="p-6 text-center text-sm text-slate-500">You&apos;re all
caught up</li>}
87             {items?.map((n) => (
88               <li key={n.notificationId}>
89                 <Link href={n.link ?? "/notifications"} onClick={close} className={`flex gap-3 px-4 py-3 t
ransition hover:bg-slate-50 dark:hover:bg-white/5 ${n.isRead ? "" : "bg-brand-50/60 dark:bg-brand-500/5"}`}
>
90                   <span className="text-lg leading-none">{TYPE_ICON[n.type] ?? ""}</span>
91                   <span className="min-w-0 flex-1">
92                     <span className="block text-sm font-semibold">{n.title}</span>
93                     {n.body && <span className="line-clamp-2 block text-xs text-slate-500 dark:text-slate-
400">{n.body}</span>}
94                     <span className="mt-0.5 block text-[11px] text-slate-400">{new Date(n.createdAt).toLoc
aleString("en-GB", { day: "2-digit", month: "short", hour: "2-digit", minute: "2-digit" })}</span>

```

```

95         </span>
96         {!n.isRead && <span className="mt-1.5 h-2 w-2 shrink-0 rounded-full bg-brand-600" />}
97         </Link>
98     </li>
99     )))
100 </ul>
101 <Link href="/notifications" onClick={close} className="block border-t border-slate-100 py-2.5 te
  xt-center text-xs font-semibold text-brand-700 hover:bg-slate-50 dark:border-white/5 dark:text-brand-400 d
  ark:hover:bg-white/5">View all</Link>
102 </div>
103 })
104 </div>
105 );
106 }
107
108 export function WalletChip({ balance, escrow, tone = "light" }: { balance: number; escrow: number; tone?:
  "light" | "dark" }) {
109     return (
110         <Link href="/wallet" title={`Available Rs. ${Math.round(balance).toLocaleString("en-US")} · In escrow
  Rs. ${Math.round(escrow).toLocaleString("en-US")}`}>
111         <div hidden items-center gap-2 rounded-xl px-3 py-1.5 text-xs font-semibold sm:inline-flex ${
  tone === "dark" ? "bg-white/10 text-white ring-1 ring-white/15 hover:bg-white/15" : "border border-slate-2
  00 text-slate-700 hover:bg-slate-100 dark:border-slate-700 dark:text-slate-200 dark:hover:bg-slate-800"}`>
112             <span aria-hidden></span>
113             <span className="tabular-nums">Rs. {shortRs(balance)}</span>
114             {escrow > 0 && <span className={`tabular-nums ${tone === "dark" ? "text-gold-300" : "text-sky-600 da
  rk:text-sky-400"}`}> · {shortRs(escrow)}</span>}
115         </Link>
116     );
117 }
118
119 export function ProfileMenu({ viewer, tone = "light" }: { viewer: NonNullable<NavViewer>; tone?: "light" |
  "dark" }) {
120     const [open, setOpen] = useState(false);
121     const ref = useRef<HTMLDivElement>(null);
122     const router = useRouter();
123     const { theme, toggle } = useTheme();
124     const { lang, setLang } = useI18n();
125     const close = useCallback(() => setOpen(false), []);
126     useOutside(ref, close);
127     const initials = viewer.fullName.split(" ").map((p) => p[0]).slice(0, 2).join("");
128
129     async function logout() {
130         await fetch("/api/auth/logout", { method: "POST" });
131         router.push("/");
132         router.refresh();
133     }
134     const item = "flex w-full items-center gap-2.5 rounded-lg px-3 py-2 text-left text-sm hover:bg-slate-100
  dark:hover:bg-white/5";
135     return (
136         <div className="relative" ref={ref}>
137             <button onClick={() => setOpen((v) => !v)} aria-expanded={open} aria-label="Account menu" className=
  {`flex items-center gap-2 rounded-xl p-1 pr-2 transition ${tone === "dark" ? "hover:bg-white/10" : "hover:
  bg-slate-100 dark:hover:bg-slate-800"}`}>
138                 {viewer.avatarUrl ? (
139                     // eslint-disable-next-line @next/next/no-img-element
140                     <img src={viewer.avatarUrl} alt="" referrerPolicy="no-referrer" className="h-8 w-8 rounded-full
  object-cover" />
141                 ) : (
142                     <span className="grid h-8 w-8 place-items-center rounded-full bg-gold-400 text-xs font-extrabold
  text-brand-950">{initials}</span>
143                 )}
144                 {viewer.role === "admin" && <span className={`hidden text-xs font-semibold md:inline ${tone === "d
  ark" ? "text-white" : "text-slate-700 dark:text-slate-200"}`}>{viewer.fullName}</span>}
145                 <svg viewBox="0 0 20 20" className={`h-4 w-4 ${tone === "dark" ? "text-white/70" : "text-slate-400
  "}`> fill="currentColor"><path d="M5.2 7.2a.75.75 0 0 1 1.06 0L10 9.4l3.74a.75.75 0 1 1 1.06 1.06l-
  4.27 4.27a.75.75 0 0 1 1.06 0L5.2 8.26a.75.75 0 0 1 0-1.06Z" /></svg>
146             </button>
147             {open && (
148                 <div className="animate-fade-in absolute right-0 z-50 mt-2 w-64 rounded-2xl border border-slate-20
  0 bg-white p-2 text-slate-900 shadow-2xl dark:border-white/10 dark:bg-[#0d1f16] dark:text-slate-100">
149                     <div className="border-b border-slate-100 px-3 pt-1 pb-3 dark:border-white/5">
150                         <p className="truncate text-sm font-bold">{viewer.fullName}</p>
151                         <p className="text-xs text-slate-500">{viewer.role === "admin" ? (viewer.title ?? "Administrat
  or") : ROLE_LABEL[viewer.role]}</p>

```

```

152         {viewer.role === "admin" && <span className="mt-1.5 inline-flex rounded-full bg-gold-400 px-2
    ↵ py-0.5 text-[10px] font-extrabold tracking-wide text-brand-950 uppercase">★ Super Admin</span>}
153         {(viewer.role === "farmer" || viewer.role === "buyer") && (
154             <p className="mt-2 rounded-lg bg-brand-50 px-2 py-1.5 text-xs text-brand-900 dark:bg-brand-5
    ↵ 00/10 dark:text-brand-200">Wallet: <b>Rs. {Math.round(viewer.walletBalance).toLocaleString("en-US")}</b></p>
    ↵
155         )}
156     </div>
157     <div className="py-1">
158         <Link href={viewer.dashboard} onClick={close} className={item}> Dashboard</Link>
159         {(viewer.role === "farmer" || viewer.role === "buyer") && <Link href="/wallet" onClick={close}
    ↵ className={item}> Escrow wallet</Link>}
160         <Link href="/notifications" onClick={close} className={item}> Notifications {viewer.unread > 0
    ↵ && <span className="ml-auto rounded-full bg-rose-600 px-1.5 text-[10px] font-bold text-white">{viewer.unr
    ↵ ead}</span></Link>
161         <Link href="/account" onClick={close} className={item}>⓪{Urdu}Account settings</Link>
162     </div>
163     <div className="border-t border-slate-100 py-1 dark:border-white/5">
164         <button onClick={toggle} className={item}>{theme === "dark" ? "Ⓜ{Urdu}Light mode" : " Dark mod
    ↵ e"}</button>
165         <button onClick={() => setLang(lang === "en" ? "ur" : "en")} className={item}> {lang === "en"
    ↵ ? "[Urdu]" : "English"}</button>
166         <button onClick={logout} className={` ${item} text-rose-600`} >Ⓢ Sign out</button>
167     </div>
168 </div>
169 )}
170 </div>
171 );
172 }
173

```

src/components/nav/site-nav.tsx

```

1  "use client";
2
3  import Link from "next/link";
4  import { usePathname } from "next/navigation";
5  import { useEffect, useState } from "react";
6  import { Logo } from "@components/Logo";
7  import { NotificationBell, ProfileMenu, WalletChip, type NavViewer } from "@components/nav/nav-actions";
8  import { useTheme } from "@components/providers";
9
10 const LINKS = [
11   { href: "/", label: "Home" },
12   { href: "/marketplace", label: "Marketplace" },
13   { href: "/mandi-rates", label: "Mandi Rates" },
14   { href: "/tracking", label: "Tracking" },
15 ];
16
17 /** Public top navigation: auth-aware (dashboard link, wallet, notifications, profile) and responsive. */
18 export function SiteNav({ viewer, transparent = false }: { viewer: NavViewer; transparent?: boolean }) {
19   const pathname = usePathname();
20   const { theme, toggle } = useTheme();
21   const [open, setOpen] = useState(false);
22   const [scrolled, setScrolled] = useState(false);
23   useEffect(() => {
24     const on = () => setScrolled(window.scrollY > 24);
25     on();
26     window.addEventListener("scroll", on, { passive: true });
27     return () => window.removeEventListener("scroll", on);
28   }, []);
29   // eslint-disable-next-line react-hooks/set-state-in-effect
30   useEffect(() => setOpen(false), [pathname]);
31
32   const glass = transparent && !scrolled && !open;
33   const tone = glass ? "dark" : "light";
34   const active = (href: string) => (href === "/" ? pathname === "/" : pathname.startsWith(href));
35   const links = viewer ? [...LINKS, { href: viewer.dashboard, label: "Dashboard" }] : LINKS;
36
37   return (
38     <header className={` ${transparent ? "fixed" : "sticky"} inset-x-0 top-0 z-40 transition-colors duratio
    ↵ n-300 ${glass ? "bg-transparent" : "border-b border-slate-200/80 bg-white/90 backdrop-blur-md dark:border-
    ↵ white/5 dark:bg-[#07130d]/90"} `>
39       <nav className="mx-auto flex h-16 max-w-7xl items-center gap-4 px-4 sm:px-6" aria-label="Main">
40         <Link href="/" aria-label="MarketLink home" className="shrink-0"><Logo size={34} tone={glass ? "li

```

```

40 ght" : "auto"} tagline={false} /></Link>
41
42 <ul className="ml-4 hidden items-center gap-1 lg:flex">
43   {links.map((l) => (
44     <li key={l.href}>
45       <Link href={l.href} aria-current={active(l.href) ? "page" : undefined}
46         className={`relative rounded-lg px-3 py-2 text-sm font-medium transition ${glass ? (active
47 (l.href) ? "text-white" : "text-white/75 hover:text-white") : active(l.href) ? "text-brand-800 dark:text-w
48 hite" : "text-slate-600 hover:text-slate-900 dark:text-slate-300 dark:hover:text-white"}`}>
49         {l.label}
50         {active(l.href) && <span className={`absolute inset-x-3 -bottom-[1px] h-0.5 rounded-full $
51 {glass ? "bg-gold-400" : "bg-brand-600"}`} />} />
52       </Link>
53     </li>
54   ))}
55 </ul>
56
57 <div className="ml-auto flex items-center gap-2">
58   {viewer ? (
59     <
60       {(viewer.role === "farmer" || viewer.role === "buyer") && <WalletChip balance={viewer.wallet
61 Balance} escrow={viewer.escrowBalance} tone={tone} />
62       <NotificationBell initialUnread={viewer.unread} tone={tone} />
63       <ProfileMenu viewer={viewer} tone={tone} />
64     </>
65   ) : (
66     <
67       <button onClick={toggle} aria-label="Toggle theme" className={`hidden h-9 w-9 place-items-ce
68 nter rounded-xl sm:grid ${glass ? "text-white/80 hover:bg-white/10" : "text-slate-500 hover:bg-slate-100 d
69 ark:hover:bg-slate-800"}`}>{theme === "dark" ? "[Urdu]" : ""}</button>
70       <Link href="/login" className={`hidden rounded-xl px-3.5 py-2 text-sm font-semibold sm:block
71 ${glass ? "text-white hover:bg-white/10" : "text-slate-700 hover:bg-slate-100 dark:text-slate-200 dark:ho
72 ver:bg-slate-800"}`}>Log in</Link>
73       <Link href="/register" className="btn bg-gold-400 text-brand-950 hover:bg-gold-300">Register
74 </Link>
75     </>
76   )}
77 </div>
78 <button onClick={() => setOpen((v) => !v)} className={`grid h-9 w-9 place-items-center rounded-x
79 l lg:hidden ${glass ? "text-white hover:bg-white/10" : "text-slate-700 hover:bg-slate-100 dark:text-slate-
80 200 dark:hover:bg-slate-800"}`} aria-label="Menu" aria-expanded={open}>
81   <svg viewBox="0 0 24 24" className="h-5 w-5" fill="none" stroke="currentColor" strokeWidth="2"
82   strokeLinecap="round">{open ? <path d="M6 6l12 12M18 6 6 18" /> : <path d="M4 7h16M4 12h16M4 17h16" />}</
83 svg>
84 </button>
85 </div>
86 </nav>
87
88 {open && (
89   <div className="animate-fade-in border-t border-slate-200 bg-white px-4 pb-4 lg:hidden dark:border
90 -white/5 dark:bg-[#07130d]">
91     <ul className="grid gap-1 pt-3">
92       {links.map((l) => (
93         <li key={l.href}><Link href={l.href} className={`block rounded-xl px-3 py-2.5 text-sm font-m
94 edium ${active(l.href) ? "bg-brand-50 text-brand-800 py-2 text-sm font-medium transition ${glass ? (active
95 (l.href) ? "text-white" : "text-white/75 hover:text-white") : active(l.href) ? "text-brand-800 dark:text-w
96 hite" : "text-slate-600 hover:text-slate-900 dark:text-slate-300 dark:hover:text-white"}`}>
97         {l.label}</Link></li>
98       ))}
99     </ul>
100     {!viewer && (
101       <div className="mt-3 grid grid-cols-2 gap-2">
102         <Link href="/login" className="btn-secondary">Log in</Link>
103         <Link href="/register" className="btn-primary">Register</Link>
104       </div>
105     )}
106   </div>
107 )}
108 </header>
109 );
110 }
111

```

src/components/negotiation.tsx

```
1  "use client";
2
3  import Link from "next/link";
4  import { useCallback, useEffect, useMemo, useState } from "react";
5  import { api, Badge, Drawer, EmptyState, GradeBadge, LoadingBlock, PageHeader, Spinner } from "@component
6  s/ui";
7  import { useI18n, useToast } from "@components/providers";
8  import { cropImage, formatDate, formatKg, formatPKR, type Category } from "@lib/constants";
9
10 type BidRow = {
11   bidId: number; cropId: number; buyerId: number; bidPricePerKg: number; bidQuantityKg: number; targetDeliv
12   eryDate: string | null; message: string | null;
13   status: "pending" | "accepted" | "rejected" | "countered" | "withdrawn"; counterPrice: number | null; co
14   unterNote: string | null; createdAt: string; updatedAt: string;
15   cropName: string; category: Category; qualityGrade: string; basePricePerKg: number; availableKg: number;
16   farmLocation: string; imagesJson: string[];
17   farmerName: string; buyerName: string; buyerBusiness: string | null; buyerCity: string; orderId: number
18   | null;
19 };
20 type Rate = { avgPricePerKg: number };
21 type Result = { kind: "accepted" | "countered" | "rejected" | "withdrawn" | "revised"; tracking?: string }
22 ;
23
24 const TABS = [
25   { key: "open", label: "Open", match: (s: string) => s === "pending" || s === "countered" },
26   { key: "accepted", label: "Accepted", match: (s: string) => s === "accepted" },
27   { key: "closed", label: "Closed", match: (s: string) => s === "rejected" || s === "withdrawn" },
28   { key: "all", label: "All", match: () => true },
29 ];
30
31 export function NegotiationBoard({ role }: { role: "farmer" | "buyer" }) {
32   const { push } = useToast();
33   const { t } = useI18n();
34   const [bids, setBids] = useState<BidRow[] | null>(null);
35   const [tab, setTab] = useState("open");
36   const [active, setActive] = useState<BidRow | null>(null);
37   const [mandiAvg, setMandiAvg] = useState<number | null>(null);
38   const [mode, setMode] = useState<"view" | "counter" | "revise">("view");
39   const [price, setPrice] = useState("");
40   const [note, setNote] = useState("");
41   const [busy, setBusy] = useState<string | null>(null);
42   const [result, setResult] = useState<Result | null>(null);
43
44   const load = useCallback(() => api<BidRow[]>("/api/bids").then(setBids).catch((e) => push({ kind: "error
45   ", title: e.message })), [push]);
46   useEffect(() => { load(); }, [load]);
47
48   function open(b: BidRow) {
49     setActive(b);
50     setMode("view");
51     setResult(null);
52     setNote("");
53     setPrice(role === "farmer" ? String(Math.round(((b.bidPricePerKg + b.basePricePerKg) / 2) * 100) / 100
54   ) : String(b.counterPrice ?? b.bidPricePerKg));
55     setMandiAvg(null);
56     api<Rate[]>(`/api/mandi-rates?view=latest&crop=${encodeURIComponent(b.cropName)}`)
57       .then((r) => setMandiAvg(r.length ? r.reduce((s, x) => s + x.avgPricePerKg, 0) / r.length : null))
58       .catch(() => {});
59   }
60
61   async function act(action: string, extra: Record<string, unknown> = {}) {
62     if (!active) return;
63     setBusy(action);
64     try {
65       const r = await api<{ bid: BidRow; order?: { trackingNumber: string } }>(`/api/bids/${active.bidId}`
66   , { method: "PATCH", json: { action, ...extra } });
67       const kind: Result["kind"] =
68         action === "accept" || action === "accept_counter" ? "accepted" : action === "counter" ? "countere
69   d" : action === "withdraw" ? "withdrawn" : action === "revise" ? "revised" : "rejected";
70       // Instant optimistic UI update
71       setBids((prev) =>
72         prev?.map((b) =>
73           b.bidId === active.bidId
```

```

64         ? {
65             ...b,
66             status: kind === "accepted" ? "accepted" : kind === "countered" ? "countered" : kind === "
        ↵ revised" ? "pending" : kind === "withdrawn" ? "withdrawn" : "rejected",
67             counterPrice: kind === "countered" ? Number(extra.counterPrice) : b.counterPrice,
68             bidPricePerKg: kind === "revised" ? Number(extra.bidPricePerKg) : kind === "accepted" && a
        ↵ ction === "accept_counter" ? (b.counterPrice ?? b.bidPricePerKg) : b.bidPricePerKg,
69         }
70         : b,
71     ) ?? null,
72 );
73 setResult({ kind, tracking: r.order?.trackingNumber });
74 push({
75     kind: kind === "accepted" ? "success" : "info",
76     title: kind === "accepted" ? "Deal closed! Order created" : kind === "countered" ? "Counter-offer
        ↵ sent" : kind === "revised" ? "Revised bid sent" : `Bid ${kind}`,
77     body: r.order ? `Tracking ${r.order.trackingNumber}` : undefined,
78 });
79 load();
80 } catch (e) {
81     push({ kind: "error", title: "Action failed", body: (e as Error).message });
82 } finally {
83     setBusy(null);
84 }
85 }
86
87 const tabDef = TABS.find((x) => x.key === tab)!;
88 const shown = useMemo(() => (bids ?? []).filter((b) => tabDef.match(b.status)), [bids, tabDef]);
89
90 const b = active;
91 const total = b ? b.bidQuantityKg * (b.status === "countered" && b.counterPrice ? b.counterPrice : b.bid
        ↵ PricePerKg) : 0;
92 const vsAsk = b ? ((b.bidPricePerKg - b.basePricePerKg) / b.basePricePerKg) * 100 : 0;
93
94 return (
95     <div>
96         <PageHeader
97             title={role === "farmer" ? t("negotiations") : t("myBids")}
98             subtitle={role === "farmer" ? "Review buyer offers – accept, reject or counter in one tap" : "Trac
        ↵ k your bulk purchase offers and respond to farmer counter-offers"}
99             actions={role === "buyer" ? <Link href="/marketplace" className="btn-primary"> Browse marketplace<
        ↵ /Link> : undefined}
100         />
101         <div className="mb-4 flex flex-wrap gap-2">
102             {TABS.map((x) => (
103                 <button key={x.key} onClick={() => setTab(x.key)} className={`rounded-full px-3.5 py-1.5 text-xs
        ↵ font-semibold ${tab === x.key ? "bg-slate-900 text-white dark:bg-white dark:text-slate-900" : "bg-white t
        ↵ ext-slate-600 ring-1 ring-slate-200 dark:bg-slate-900 dark:text-slate-300 dark:ring-slate-700"}`}>
104                 {x.label} {bids && <span className="opacity-60">{bids.filter((y) => x.match(y.status)).length
        ↵ }}</span>}
105             </button>
106         )]}
107         </div>
108
109         {!bids ? <LoadingBlock /> : shown.length === 0 ? (
110             <EmptyState icon="" title="Nothing here yet" body={role === "farmer" ? "When buyers bid on your li
        ↵ stings, they'll appear here." : "Place a bid from the marketplace to start negotiating."} />
111         ) : (
112             <div className="card overflow-hidden">
113                 <ul className="divide-y divide-slate-100 dark:divide-slate-800">
114                     {shown.map((x) => {
115                         const needsMe = (role === "farmer" && x.status === "pending") || (role === "buyer" && x.stat
        ↵ us === "countered");
116                         return (
117                             <li key={x.bidId}>
118                                 <button onClick={() => open(x)} className="flex w-full items-center gap-3 px-4 py-3.5 te
        ↵ xt-left transition hover:bg-slate-50 sm:px-5 dark:hover:bg-slate-800/50">
119                                     {/* eslint-disable-next-line @next/next/no-img-element */}
120                                     <img src={cropImage(x.imagesJson, x.cropName, x.category)} alt="" className="h-12 w-12
        ↵ shrink-0 rounded-xl object-cover" />
121                                     <div className="min-w-0 flex-1">
122                                         <p className="truncate text-sm font-semibold">
123                                             {x.cropName} <span className="font-normal text-slate-500">· {role === "farmer" ? (
        ↵ x.buyerBusiness ?? x.buyerName) : x.farmerName}</span>
124                                         </p>
125                                         <p className="text-xs text-slate-500">{formatKg(x.bidQuantityKg)} · target {formatDa

```

```

    te(x.targetDeliveryDate)} · {formatDate(x.updatedAt)}</p>
126     </div>
127     <div className="hidden text-right sm:block">
128         <p className="text-sm font-bold">{formatPKR(x.bidPricePerKg, 2)}<span className="tex
    t-xs font-normal text-slate-500">/kg</span></p>
129         {x.status === "countered" && x.counterPrice && <p className="text-xs font-semibold t
    ext-sky-600">Counter {formatPKR(x.counterPrice, 2)}</p>}
130     </div>
131     <div className="flex flex-col items-end gap-1">
132         <Badge status={x.status} />
133         {needsMe && <span className="text-[10px] font-bold text-amber-600 uppercase">Action
    needed</span>}
134     </div>
135 </button>
136 </li>
137 );
138 }}}
139 </ul>
140 </div>
141 )}
142
143 <Drawer open={!!b} onClose={() => setActive(null)} title={b ? `Bid #${b.bidId} · ${b.cropName}` : ""
    }>
144     {b && (
145         <div className="space-y-5">
146             {result ? (
147                 <div className="animate-fade-in py-6 text-center">
148                     <div className={`animate-pop mx-auto grid h-20 w-20 place-items-center rounded-full text-4
    xl ${result.kind === "accepted" ? "bg-emerald-100 dark:bg-emerald-500/20" : result.kind === "rejected" ||
    result.kind === "withdrawn" ? "bg-rose-100 dark:bg-rose-500/20" : "bg-sky-100 dark:bg-sky-500/20"}`}>
149                         {result.kind === "accepted" ? "" : result.kind === "rejected" ? "" : result.kind === "wi
    thdrawn" ? "↩[Urdu]" : ""}
150                     </div>
151                     <h3 className="mt-4 text-xl font-bold">
152                         {result.kind === "accepted" ? "Deal closed!" : result.kind === "countered" ? "Counter-of
    fer sent" : result.kind === "revised" ? "Revised bid sent" : result.kind === "withdrawn" ? "Bid withdrawn"
    : "Offer rejected"}
153                     </h3>
154                     <p className="mt-1 text-sm text-slate-500">
155                         {result.kind === "accepted"
156                             ? `Order ${result.tracking ?? ""} created. ${role === "buyer" ? "Lock funds in escrow
    to start fulfilment." : "Buyer has been notified to fund escrow."}`
157                             : result.kind === "countered"
158                             ? "The buyer can accept, reject or revise their offer."
159                             : result.kind === "revised"
160                             ? "The farmer will review your new price."
161                             : "This negotiation is now closed."}
162                     </p>
163                     <div className="mt-6 flex justify-center gap-2">
164                         {result.kind === "accepted" && <Link href={role === "buyer" ? "/buyer/orders" : "/farmer
    /orders"} className="btn-primary">View order </Link>}
165                         <button className="btn-secondary" onClick={() => setActive(null)}>Close</button>
166                     </div>
167                 </div>
168             ) : (
169                 <>
170                     <div className="flex items-center gap-3">
171                         </* eslint-disable-next-line @next/next/no-img-element */>
172                         <img src={cropImage(b.imagesJson, b.cropName, b.category)} alt="" className="h-16 w-16 r
    ounded-xl object-cover" />
173                         <div>
174                             <p className="font-semibold">{b.cropName} <GradeBadge grade={b.qualityGrade} /></p>
175                             <p className="text-xs text-slate-500">{b.farmLocation} · {formatKg(b.availableKg)} av
    ailable</p>
176                             <div className="mt-1"><Badge status={b.status} /></div>
177                         </div>
178                     </div>
179
180                     <div className="rounded-2xl bg-slate-50 p-4 dark:bg-slate-800/60">
181                         <p className="text-xs font-semibold text-slate-500 uppercase">{role === "farmer" ? "Buye
    r" : "Farmer"}</p>
182                         <p className="font-semibold">{role === "farmer" ? `${b.buyerBusiness ?? b.buyerName} · $
    {b.buyerCity}` : b.farmerName}</p>
183                         {b.message && <p className="mt-2 rounded-lg bg-white p-2.5 text-sm italic dark:bg-slate-
    900">&ldquo;{b.message}&rdquo;</p>}
184                     </div>

```

```

185
186         <dl className="grid grid-cols-2 gap-3 text-sm">
187             <div className="rounded-xl border border-slate-200 p-3 dark:border-slate-700"><dt classN
188             ame="text-xs text-slate-500">Bid price</dt><dd className="text-lg font-bold">{formatPKR(b.bidPricePerKg, 2
189             )}</dd><dd className={`text-xs font-semibold ${vsAsk >= 0 ? "text-emerald-600" : "text-rose-600"}`}>{vsAsk
190             >= 0 ? "+" : ""}{vsAsk.toFixed(1)}% vs ask</dd></div>
191             <div className="rounded-xl border border-slate-200 p-3 dark:border-slate-700"><dt classN
192             ame="text-xs text-slate-500">Asking price</dt><dd className="text-lg font-bold">{formatPKR(b.basePricePerK
193             g, 2)}</dd><dd className="text-xs text-slate-500">Mandi avg {mandiAvg ? formatPKR(mandiAvg, 2) : "..."}</dd>
194             </div>
195             <div className="rounded-xl border border-slate-200 p-3 dark:border-slate-700"><dt classN
196             ame="text-xs text-slate-500">Quantity</dt><dd className="font-bold">{formatKg(b.bidQuantityKg)}</dd></div>
197             <div className="rounded-xl border border-slate-200 p-3 dark:border-slate-700"><dt classN
198             ame="text-xs text-slate-500">Target delivery</dt><dd className="font-boId">{formatDate(b.targetDeliveryDat
199             e)}</dd></div>
200             {b.counterPrice && (
201             <div className="col-span-2 rounded-xl border border-sky-200 bg-sky-50 p-3 dark:border-
202             sky-800 dark:bg-sky-950/30">
203                 <dt className="text-xs font-semibold text-sky-700 dark:text-sky-300">Farmer counter-
204                 offer</dt>
205                 <dd className="text-lg font-bold text-sky-900 dark:text-sky-100">{formatPKR(b.counte
206                 rPrice, 2)}/kg</dd>
207                 {b.counterNote && <dd className="text-xs text-sky-800 dark:text-sky-300">&ldquo;{b.c
208                 ounterNote}&rdquo;</dd>}
209                 </div>
210             )}
211             <div className="col-span-2 flex items-center justify-between rounded-xl bg-brand-600 p-3
212             text-white">
213                 <span className="text-sm">Deal value</span><span className="text-xl font-extrabold">{f
214                 ormatPKR(total)}</span>
215             </div>
216         </dl>
217
218         {/* Farmer actions */}
219         {role === "farmer" && b.status === "pending" && (
220             mode === "counter" ? (
221                 <div className="animate-fade-in space-y-3 rounded-2xl border border-sky-200 p-4 dark:b
222                 order-sky-800">
223                     <div><label className="label">Counter price (Rs./kg)</label><input type="number" ste
224                     p="0.01" className="input" value={price} onChange={(e) => setPrice(e.target.value)} /></div>
225                     <p className="text-xs text-slate-500">New deal value: <b>{formatPKR(Number(price ||
226                     0) * b.bidQuantityKg)}</b></p>
227                     <div><label className="label">Note to buyer</label><input className="input" value={n
228                     ote} onChange={(e) => setNote(e.target.value)} placeholder="e.g. Includes bagging & loading" /></div>
229                     <div className="flex gap-2">
230                         <button className="btn-secondary flex-1" onClick={() => setMode("view")}>Back</but
231                         ton>
232                         <button className="btn-primary flex-1 !bg-sky-600 hover:!bg-sky-700" disabled={!b
233                         usy} onClick={() => act("counter", { counterPrice: Number(price), note })}>{busy === "counter" && <Spinner
234                         /> Send counter</button>
235                     </div>
236                 </div>
237             ) : (
238                 <div className="grid grid-cols-3 gap-2">
239                     <button className="btn-primary" disabled={!busy} onClick={() => act("accept")}>{bus
240                     y === "accept" ? <Spinner /> : "✓"} {t("accept")}</button>
241                     <button className="btn-secondary !border-sky-300 !text-sky-700 dark:!text-sky-300" d
242                     isabled={!busy} onClick={() => setMode("counter")}>{t("counter")}</button>
243                     <button className="btn-danger" disabled={!busy} onClick={() => act("reject")}>{busy
244                     === "reject" ? <Spinner /> : "x"} {t("reject")}</button>
245                 </div>
246             )
247         )}
248         {role === "farmer" && b.status === "countered" && (
249             <div className="space-y-2">
250                 <p className="rounded-xl bg-sky-50 p-3 text-sm text-sky-800 dark:bg-sky-950/30 dark:te
251                 xt-sky-300">Waiting for the buyer to respond to your counter-offer.</p>
252                 <button className="btn-secondary w-full text-rose-600" disabled={!busy} onClick={() =
253                 > act("reject")}>Withdraw counter & reject</button>
254             </div>
255         )}
256
257         {/* Buyer actions */}
258         {role === "buyer" && (b.status === "countered" || b.status === "pending") && (
259             mode === "revise" ? (
260                 <div className="animate-fade-in space-y-3 rounded-2xl border border-amber-200 p-4 dark

```

```

234 <div><label className="label">New bid price (Rs./kg)</label><input type="number" ste
p="0.01" className="input" value={price} onChange={(e) => setPrice(e.target.value)} /></div>
235 <div><label className="label">Message</label><input className="input" value={note} o
nChange={(e) => setNote(e.target.value)} /></div>
236 <div className="flex gap-2">
237 <button className="btn-secondary flex-1" onClick={() => setMode("view")}>Back</but
ton>
238 <button className="btn-amber flex-1" disabled={!busy} onClick={() => act("revise"
, { bidPricePerKg: Number(price), note: note || undefined })}>{busy === "revise" && <Spinner />} Send revi
sed bid</button>
239 </div>
240 </div>
241 ) : b.status === "countered" ? (
242 <div className="grid grid-cols-3 gap-2">
243 <button className="btn-primary" disabled={!busy} onClick={() => act("accept_counter
")}>{busy === "accept_counter" ? <Spinner /> : "✓"} Accept</button>
244 <button className="btn-secondary" disabled={!busy} onClick={() => setMode("revise")}
>➡ Revise</button>
245 <button className="btn-danger" disabled={!busy} onClick={() => act("reject_counter"
)}>✖ Reject</button>
246 </div>
247 ) : (
248 <div className="grid grid-cols-2 gap-2">
249 <button className="btn-secondary" disabled={!busy} onClick={() => setMode("revise")}
>=>[Urdu]Revise price</button>
250 <button className="btn-danger" disabled={!busy} onClick={() => act("withdraw")}>{bu
sy === "withdraw" && <Spinner />} Withdraw</button>
251 </div>
252 )
253 )}
254 {b.status === "accepted" && (
255 <Link href={role === "buyer" ? "/buyer/orders" : "/farmer/orders"} className="btn-primar
y w-full"> Track order</Link>
256 )}
257 </>
258 )}
259 </div>
260 )}
261 </Drawer>
262 </div>
263 );
264 }
265

```

src/components/orders-board.tsx

```

1 "use client";
2
3 import { useCallback, useEffect, useState } from "react";
4 import { api, Badge, EmptyState, LoadingBlock, Modal, PageHeader, Spinner } from "@components/ui";
5 import { LogisticsStepper } from "@components/stepper";
6 import { useToast } from "@components/providers";
7 import { CITIES, CITY_NAMES, DELIVERY_STAGES, cropImage, formatDate, formatKg, formatPKR, nextStage, PLATF
ORM_FEE_RATE, type Category, type DeliveryStage, type Role } from "@lib/constants";
8
9 export type OrderRow = {
10   orderId: number; trackingNumber: string; quantityKg: number; pricePerKg: number; totalAmount: number; pa
ymentStatus: string; deliveryStage: DeliveryStage;
11   transitLocation: string | null; expectedDeliveryDate: string | null; createdAt: string; cropName: string
; category: Category; qualityGrade: string; farmLocation: string; imagesJson: string[];
12   farmerName: string; farmName: string | null; buyerName: string; buyerBusiness: string | null; buyerCity:
string;
13   events: { eventId: number; stage: string; location: string | null; note: string | null; createdAt: strin
g }[];
14   disputes: { disputeId: number; reason: string; status: string; resolution: string | null }[];
15 };
16
17 const EVENT_ICON: Record<string, string> = { confirmed: "", escrow_locked: "", quality_checked: "", dispat
ched: "", in_transit: "", location_update: "", delivered: "", escrow_released: "", escrow_refunded: "↩[Urd
u]", dispute_opened: "⚠[Urdu]" };
18
19 type Dialog = { order: OrderRow; kind: "advance" | "location" | "dispute" | "escrow" } | null;
20
21 export function OrdersBoard({ role, title, subtitle }: { role: Role; title: string; subtitle: string }) {

```

```

22     const { push } = useToast();
23     const [orders, setOrders] = useState<OrderRow[] | null>(null);
24     const [tab, setTab] = useState<"active" | "done" | "all">("active");
25     const [expanded, setExpanded] = useState<number | null>(null);
26     const [dialog, setDialog] = useState<Dialog>(null);
27     const [form, setForm] = useState({ location: "", note: "", grade: "A", moisture: "12" });
28     const [busy, setBusy] = useState(false);
29
30     const load = useCallback(() => api<OrderRow[]>("/api/orders").then(setOrders).catch((e) => push({ kind:
    "error", title: e.message })), [push]);
31     useEffect(() => { load(); }, [load]);
32
33     const isDone = (o: OrderRow) => (o.deliveryStage === "delivered" && o.paymentStatus !== "disputed") || o
    .paymentStatus === "refunded";
34     const shown = (orders ?? []).filter((o) => (tab === "all" ? true : tab === "done" ? isDone(o) : !isDone(
    o)));
35
36     function openDialog(order: OrderRow, kind: NonNullable<Dialog>["kind"]) {
37         setForm({ location: kind === "advance" && nextStage(order.deliveryStage) === "in_transit" ? "" : (orde
    r.transitLocation ?? ""), note: "", grade: order.qualityGrade, moisture: "12" });
38         setDialog({ order, kind });
39     }
40
41     async function submit() {
42         if (!dialog) return;
43         const { order, kind } = dialog;
44         setBusy(true);
45         try {
46             if (kind === "dispute") {
47                 await api(`/api/orders/${order.orderId}/dispute`, { method: "POST", json: { reason: form.note } });
48                 push({ kind: "info", title: "Dispute raised", body: "Escrow is frozen until an admin resolves it."
49             });
50             } else if (kind === "escrow") {
51                 await api(`/api/orders/${order.orderId}`, { method: "PATCH", json: { action: "lock_escrow" } });
52                 push({ kind: "success", title: "Funds locked in escrow", body: `${formatPKR(order.totalAmount)} se
    cured for ${order.trackingNumber}` });
53             } else if (kind === "location") {
54                 await api(`/api/orders/${order.orderId}`, { method: "PATCH", json: { action: "update_location", lo
    cation: form.location } });
55                 push({ kind: "success", title: "Checkpoint updated" });
56             } else {
57                 const next = nextStage(order.deliveryStage)!;
58                 await api(`/api/orders/${order.orderId}`, {
59                     method: "PATCH",
60                     json: { action: "advance", location: form.location || undefined, note: form.note || undefined, .
    ..(next === "quality_checked" ? { grade: form.grade, moisture: Number(form.moisture) } : {}) },
61                 });
62                 push({ kind: "success", title: `Order ${DELIVERY_STAGES.find((s) => s.key === next)!.label}`, body
    : next === "delivered" ? "Escrow released to the farmer." : undefined });
63             }
64             setDialog(null);
65             load();
66         } catch (e) {
67             push({ kind: "error", title: "Action failed", body: (e as Error).message });
68         } finally {
69             setBusy(false);
70         }
71     }
72
73     function shareGps(o: OrderRow) {
74         if (!navigator.geolocation) return push({ kind: "error", title: "GPS not available on this device" });
75         navigator.geolocation.getCurrentPosition(
76             async (pos) => {
77                 try {
78                     await api(`/api/orders/${o.orderId}/gps`, { method: "POST", json: { lat: pos.coords.latitude, ln
    g: pos.coords.longitude } });
79                     push({ kind: "success", title: "Live location shared with the buyer" });
80                     load();
81                 } catch (e) {
82                     push({ kind: "error", title: (e as Error).message });
83                 }
84             },
85             () => push({ kind: "error", title: "Location permission denied" }),
86             { enableHighAccuracy: true, timeout: 10000 },

```

```

87 }
88
89 function actions(o: OrderRow) {
90   const next = nextStage(o.deliveryStage);
91   const btns: React.ReactNode[] = [];
92   const locked = o.paymentStatus === "escrow_locked";
93   if (role === "buyer" && o.paymentStatus === "awaiting_escrow") btns.push(<button key="esc" className="
94   btn-primary" onClick={() => openDialog(o, "escrow")}> Lock {formatPKR(o.totalAmount)} in escrow</button>);
95   if (locked && next === "quality_checked" && role === "admin") btns.push(<button key="qc" className="bt
96   n-primary" onClick={() => openDialog(o, "advance")}> Record quality inspection</button>);
97   if (locked && (next === "dispatched" || next === "in_transit") && (role === "farmer" || role === "admin"
98   ))
99     btns.push(<button key="adv" className="btn-primary" onClick={() => openDialog(o, "advance")}>{next ===
100   "dispatched" ? " Mark dispatched" : " Mark in transit"}</button>);
101   if (locked && (o.deliveryStage === "dispatched" || o.deliveryStage === "in_transit") && (role === "far
102   mer" || role === "admin"))
103     btns.push(<button key="loc" className="btn-secondary" onClick={() => openDialog(o, "location")}> Update
104   checkpoint</button>);
105   if (locked && next === "delivered" && (role === "buyer" || role === "admin"))
106     btns.push(<button key="del" className="btn-primary" onClick={() => openDialog(o, "advance")}> Confirm
107   delivery & release escrow</button>);
108   if ([ "dispatched", "in_transit", "quality_checked", "delivered" ].includes(o.deliveryStage)) btns.push(
109   <a key="trk" href={` /tracking?no=${o.trackingNumber} `} className="btn-secondary"> Live map</a>);
110   if (locked && role === "farmer" && (o.deliveryStage === "dispatched" || o.deliveryStage === "in_transi
111   t")) btns.push(<button key="gps" className="btn-secondary" onClick={() => shareGps(o)}> Share GPS</button>
112   );
113   if (locked && role !== "admin") btns.push(<button key="dis" className="btn-secondary text-rose-600" on
114   Click={() => openDialog(o, "dispute")}>⚠[Urdu]Raise dispute</button>);
115   return btns;
116 }
117
118 function waitingText(o: OrderRow) {
119   if (o.paymentStatus === "disputed") return "⚠[Urdu]Under dispute review – progression frozen";
120   if (o.paymentStatus === "refunded") return "⚠[Urdu]Refunded to buyer";
121   if (o.paymentStatus === "awaiting_escrow") return role === "buyer" ? null : " Waiting for buyer to loc
122   k funds in escrow";
123   const next = nextStage(o.deliveryStage);
124   if (!next) return " Completed – payment released";
125   const who = DELIVERY_STAGES.find((s) => s.key === next)!.actor;
126   if (who === role) return null;
127   return ` Next: ${DELIVERY_STAGES.find((s) => s.key === next)!.label} by ${who === "admin" ? "quality i
128   nspector" : who}`;
129 }
130
131 const d = dialog;
132 const dNext = d ? nextStage(d.order.deliveryStage) : null;
133
134 return (
135   <div>
136     <PageHeader title={title} subtitle={subtitle} />
137     <div className="mb-4 flex flex-wrap gap-2">
138       {[ "active", "done", "all" ] as const}.map((x) => (
139         <button key={x} onClick={() => setTab(x)} className={` rounded-full px-3.5 py-1.5 text-xs font-se
140         mibold capitalize ${tab === x ? "bg-slate-900 text-white dark:bg-white dark:text-slate-900" : "bg-white te
141         xt-slate-600 ring-1 ring-slate-200 dark:bg-slate-900 dark:text-slate-300 dark:ring-slate-700"} `>
142         {x === "done" ? "Completed" : x} {orders && <span className="opacity-60">{orders.filter((o) =
143         > (x === "all" ? true : x === "done" ? isDone(o) : !isDone(o))).length}</span>}
144       </button>
145     ))}
146   </div>
147
148   {!orders ? <LoadingBlock /> : shown.length === 0 ? (
149     <EmptyState icon="" title="No orders here" body="Orders appear once a bid is accepted." />
150   ) : (
151     <div className="space-y-4">
152       {shown.map((o) => {
153         const wait = waitingText(o);
154         const acts = actions(o);
155         return (
156           <article key={o.orderId} className="card overflow-hidden">
157             <div className="flex flex-col gap-3 border-b border-slate-100 p-4 sm:flex-row sm:items-cen
158             ter sm:p-5 dark:border-slate-800">
159               <div className="flex min-w-0 flex-1 items-center gap-3">
160                 { /* eslint-disable-next-line @next/next/no-img-element */ }
161                 <img src={cropImage(o.imagesJson, o.cropName, o.category)} alt="" className="h-14 w-14
162                 shrink-0 rounded-xl object-cover" />

```

```

145         <div className="min-w-0">
146             <p className="font-semibold">{o.cropName} · {formatKg(o.quantityKg)} <span className
147             = "font-mono text-xs text-slate-500">#{o.trackingNumber}</span></p>
148             <p className="truncate text-xs text-slate-500">
149                 {o.farmName ?? o.farmerName} ({o.farmLocation}) → {o.buyerBusiness ?? o.buyerNam
150                 e} ({o.buyerCity})
151             </p>
152             <p className="text-xs text-slate-500">Placed {formatDate(o.createdAt)} · ETA {format
153             Date(o.expectedDeliveryDate)}</p>
154         </div>
155         <div className="flex items-center gap-3 sm:flex-col sm:items-end sm:gap-1">
156             <p className="text-lg font-bold">{formatPKR(o.totalAmount)}</p>
157             <p className="text-xs text-slate-500">@ {formatPKR(o.pricePerKg, 2)}/kg</p>
158             <Badge status={o.paymentStatus} />
159         </div>
160         <div className="p-4 sm:p-5">
161             <LogisticsStepper stage={o.deliveryStage} disputed={o.paymentStatus === "disputed"} />
162             {o.transitLocation && o.deliveryStage !== "confirmed" && (
163                 <p className="mt-4 text-center text-sm text-slate-600 dark:text-slate-300">Last known
164                 location: <b>{o.transitLocation}</b></p>
165             )}
166             {o.disputes.filter((x) => x.status === "open" || x.status === "investigating").map((x) =
167             > (
168                 <p key={x.disputeId} className="mt-3 rounded-xl bg-rose-50 p-3 text-sm text-rose-800 d
169                 ark:bg-rose-950/30 dark:text-rose-300">▲[Urdu]Dispute ({x.status}): {x.reason}</p>
170             ))}
171             <div className="mt-4 flex flex-wrap items-center gap-2">
172                 {acts}
173                 {wait && <span className="text-sm text-slate-500">{wait}</span>}
174                 <button className="ml-auto text-sm font-semibold text-brand-600 hover:underline" onCli
175                 ck={() => setExpanded(expanded === o.orderId ? null : o.orderId)}>
176                     {expanded === o.orderId ? "Hide" : "Show"} tracking history ({o.events.length})
177                 </button>
178             </div>
179             {expanded === o.orderId && (
180                 <ol className="animate-fade-in mt-4 space-y-3 border-l-2 border-slate-200 pl-5 dark:bo
181                 rder-slate-700">
182                     {[...o.events].reverse().map((e) => (
183                         <li key={e.eventId} className="relative">
184                             <span className="absolute top-0 -left-[31px] grid h-6 w-6 place-items-center rou
185                             nded-full bg-white text-xs ring-2 ring-slate-200 dark:bg-slate-900 dark:ring-slate-700">{EVENT_ICON[e.stag
186                             e] ?? "•"}</span>
187                             <p className="text-sm font-semibold capitalize">{e.stage.replace(/_/g, " ")}</p>
188                             <p className="text-xs text-slate-500">{new Date(e.createdAt).toLocaleString("en-
189                             GB")} {e.location && `· ${e.location}`}</p>
190                             {e.note && <p className="text-xs text-slate-600 dark:text-slate-400">{e.note}</p>
191                         >}
192                     </li>
193                     )]}
194                 </ol>
195             )}
196         </div>
197     </article>
198 );
199 }}
200 </div>
201 ))
202
203 <Modal
204     open={!!d}
205     onClose={() => setDialog(null)}
206     title={
207         !d ? "" : d.kind === "escrow" ? "Lock funds in escrow" : d.kind === "dispute" ? "Raise a dispute
208         " : d.kind === "location" ? "Update shipment checkpoint" : `Advance to: ${DELIVERY_STAGES.find((s) => s.ke
209         y === dNext)?.label}`
210     }
211     >
212         {d && (
213             <div className="space-y-4">
214                 {d.kind === "escrow" && (
215                     <div className="space-y-3 text-sm">
216                         <p>MarketLink holds <b>{formatPKR(d.order.totalAmount)}</b> from your wallet in escrow. It
217                         is released to the farmer only when you confirm delivery.</p>
218                     <div className="rounded-xl bg-slate-50 p-3 dark:bg-slate-800">

```

```

206         <div className="flex justify-between"><span>Order value</span><b>{formatPKR(d.order.totalAmount)}</b></div>
207         <div className="flex justify-between text-slate-500"><span>Farmer receives (after {PLATFORM_FEE_RATE * 100}% fee)</span><span>{formatPKR(d.order.totalAmount * (1 - PLATFORM_FEE_RATE))}</span></div>
208     </div>
209 </div>
210 })
211 {d.kind === "dispute" && (
212     <div>
213         <label className="label">Describe the issue</label>
214         <textarea className="input" rows={4} value={form.note} onChange={(e) => setForm({ ...form, note: e.target.value })} placeholder="e.g. Quantity short by 200kg, moisture above agreed level..." />
215         <p className="mt-2 text-xs text-slate-500">Funds stay locked in escrow while an admin investigates.</p>
216     </div>
217 )}
218 {(d.kind === "location" || (d.kind === "advance" && (dNext === "dispatched" || dNext === "in_transit" || dNext === "delivered")) && (
219     <div>
220         <label className="label">{dNext === "delivered" ? "Received at" : "Current location / checkpoint"}</label>
221         <input className="input" list="cities" value={form.location} onChange={(e) => setForm({ ...form, location: e.target.value })} placeholder={dNext === "delivered" ? "Buyer warehouse" : "e.g. Motorway M-4, Pindi Bhattian toll"} />
222         <datalist id="cities">{CITY_NAMES.map((c) => <option key={c} value={` ${c} `, `${CITIES[c].province}` } />)}</datalist>
223     </div>
224 )}
225 {d.kind === "advance" && dNext === "quality_checked" && (
226     <div className="grid grid-cols-2 gap-3">
227         <div><label className="label">Grade assigned</label><select className="input" value={form.grade} onChange={(e) => setForm({ ...form, grade: e.target.value })}><option>A</option><option>B</option><option>C</option></select></div>
228         <div><label className="label">Moisture %</label><input type="number" step="0.1" className="input" value={form.moisture} onChange={(e) => setForm({ ...form, moisture: e.target.value })} /></div>
229     </div>
230 )}
231 {d.kind === "advance" && (
232     <div>
233         <label className="label">Note (optional)</label>
234         <input className="input" value={form.note} onChange={(e) => setForm({ ...form, note: e.target.value })} placeholder={dNext === "dispatched" ? "Truck LES-4521, driver Akram 0300-..." : ""} />
235     </div>
236 )}
237 {d.kind === "advance" && dNext === "delivered" && (
238     <p className="rounded-xl bg-emerald-50 p-3 text-sm text-emerald-800 dark:bg-emerald-950/30 dark:text-emerald-300">
239         Confirming delivery releases <b>{formatPKR(d.order.totalAmount)}</b> from escrow to the farmer. This cannot be undone.
240     </p>
241 )}
242 <div className="flex justify-end gap-2">
243     <button className="btn-secondary" onClick={() => setDialog(null)}>Cancel</button>
244     <button className={d.kind === "dispute" ? "btn-danger" : "btn-primary"} disabled={busy || (d.kind === "dispute" && form.note.trim().length < 5) || (d.kind === "location" && !form.location)} onClick={submit}>
245         {busy && <Spinner />} Confirm
246     </button>
247 </div>
248 </div>
249 )}
250 </Modal>
251 </div>
252 );
253 }
254

```

src/components/providers.tsx

```
1  "use client";
2
3  import { createContext, useCallback, useContext, useEffect, useState, type ReactNode } from "react";
4
5  /* ----- Theme ----- */
6  type Theme = "light" | "dark";
7  const ThemeCtx = createContext<{ theme: Theme; toggle: () => void }>({ theme: "light", toggle: () => {} })
8  ;
9  export const useTheme = () => useContext(ThemeCtx);
10
11 /* ----- i18n (English / Urdu placeholder) ----- */
12 export type Lang = "en" | "ur";
13 const DICT: Record<string, { en: string; ur: string }> = {
14   dashboard: { en: "Dashboard", ur: "[Urdu]" },
15   listings: { en: "My Listings", ur: "[Urdu]" },
16   negotiations: { en: "Negotiation Hub", ur: "[Urdu]" },
17   orders: { en: "Orders", ur: "[Urdu]" },
18   mandi: { en: "Mandi Insights", ur: "[Urdu]" },
19   marketplace: { en: "Marketplace", ur: "[Urdu]" },
20   myBids: { en: "My Bids & RFQs", ur: "[Urdu]" },
21   reviews: { en: "Buyer reviews", ur: "[Urdu]" },
22   logistics: { en: "Order Tracker", ur: "[Urdu]" },
23   wallet: { en: "Escrow Wallet", ur: "[Urdu]" },
24   analytics: { en: "Analytics", ur: "[Urdu]" },
25   verification: { en: "Verification", ur: "[Urdu]" },
26   priceController: { en: "Price Controller", ur: "[Urdu]" },
27   disputes: { en: "Orders", ur: "[Urdu]" },
28   apiDocs: { en: "API & Schema", ur: "API [Urdu]" },
29   logout: { en: "Sign out", ur: "[Urdu]" },
30   users: { en: "Users & Access", ur: "[Urdu]" },
31   inspections: { en: "Inspections", ur: "[Urdu]" },
32   tracking: { en: "Tracking", ur: "[Urdu]" },
33   dbExplorer: { en: "Database", ur: "[Urdu]" },
34   account: { en: "Account settings", ur: "[Urdu]" },
35   welcome: { en: "Welcome back", ur: "[Urdu]" },
36   tagline: { en: "Farm-gate to market, no middlemen.", ur: "[Urdu]" },
37   placeBid: { en: "Place Bid", ur: "[Urdu]" },
38   accept: { en: "Accept", ur: "[Urdu]" },
39   reject: { en: "Reject", ur: "[Urdu]" },
40   counter: { en: "Counter", ur: "[Urdu]" },
41   newListing: { en: "New Listing", ur: "[Urdu]" },
42   verified: { en: "Verified", ur: "[Urdu]" },
43 };
44 const LangCtx = createContext<{ lang: Lang; setLang: (l: Lang) => void; t: (k: string) => string }>({
45   lang: "en",
46   setLang: () => {},
47   t: (k) => k,
48 });
49 export const useI18n = () => useContext(LangCtx);
50
51 /* ----- Toasts ----- */
52 type Toast = { id: number; kind: "success" | "error" | "info"; title: string; body?: string };
53 const ToastCtx = createContext<{ push: (t: Omit<Toast, "id">) => void }>({ push: () => {} });
54 export const useToast = () => useContext(ToastCtx);
55
56 export function Providers({ children }: { children: ReactNode }) {
57   const [theme, setTheme] = useState<Theme>("light");
58   const [lang, setLangState] = useState<Lang>("en");
59   const [toasts, setToasts] = useState<Toast[]>([]);
60
61   useEffect(() => {
62     // eslint-disable-next-line react-hooks/set-state-in-effect
63     setTheme(document.documentElement.classList.contains("dark") ? "dark" : "light");
64     const l = (localStorage.getItem("ml-lang") as Lang) || "en";
65     setLangState(l);
66     document.documentElement.lang = l;
67     document.documentElement.dir = l === "ur" ? "rtl" : "ltr";
68   }, []);
69
70   const toggle = useCallback(() => {
71     setTheme((prev) => {
72       const next = prev === "dark" ? "light" : "dark";
73       document.documentElement.classList.toggle("dark", next === "dark");
74     });
75   }, []);
76 }
```

```

73     localStorage.setItem("ml-theme", next);
74     return next;
75   });
76 }, []);
77
78 const setLang = useCallback((l: Lang) => {
79   setLangState(l);
80   localStorage.setItem("ml-lang", l);
81   document.documentElement.lang = l;
82   document.documentElement.dir = l === "ur" ? "rtl" : "ltr";
83 }, []);
84
85 const t = useCallback((k: string) => DICT[k]?.[lang] ?? k, [lang]);
86
87 const push = useCallback((toast: Omit<Toast, "id">) => {
88   const id = Date.now() + Math.random();
89   setToasts((ts) => [...ts, { ...toast, id }]);
90   setTimeout(() => setToasts((ts) => ts.filter((x) => x.id !== id)), 4200);
91 }, []);
92
93 return (
94   <ThemeCtx.Provider value={{ theme, toggle }}>
95     <LangCtx.Provider value={{ lang, setLang, t }}>
96       <ToastCtx.Provider value={{ push }}>
97         {children}
98       <div className="pointer-events-none fixed right-4 bottom-4 z-[100] flex w-[min(92vw,380px)] flex
99         -col gap-2" dir="ltr">
100         {toasts.map((x) => (
101           <div
102             key={x.id}
103             className={`animate-fade-in pointer-events-auto rounded-xl border p-3 shadow-lg backdrop-b
104               lur ${
105                 x.kind === "success"
106                   ? "border-emerald-200 bg-emerald-50/95 text-emerald-900 dark:border-emerald-800 dark:b
107                     g-emerald-950/90 dark:text-emerald-100"
108                   : x.kind === "error"
109                     ? "border-rose-200 bg-rose-50/95 text-rose-900 dark:border-rose-800 dark:bg-rose-950
110                     /90 dark:text-rose-100"
111                   : "border-sky-200 bg-sky-50/95 text-sky-900 dark:border-sky-800 dark:bg-sky-950/90 d
112                     ark:text-sky-100"
113               }}
114           >
115             <div className="flex items-start gap-2">
116               <span className="text-lg leading-none">{x.kind === "success" ? "" : x.kind === "error" ?
117                 "▲[Urdu]" : "[Urdu]"}</span>
118               <div>
119                 <p className="text-sm font-semibold">{x.title}</p>
120                 {x.body && <p className="mt-0.5 text-xs opacity-80">{x.body}</p>}
121               </div>
122             </div>
123           </div>
124         ))}
125       </div>
126     </ToastCtx.Provider>
127   </LangCtx.Provider>
128 </ThemeCtx.Provider>
129 );
130 }
131

```

src/components/reviews/farmer-review-board.tsx

```

1  "use client";
2
3  import Link from "next/link";
4  import { useMemo, useState } from "react";
5  import { useRouter } from "next/navigation";
6  import { api, EmptyState, PageHeader, Spinner, StatCard } from "@components/ui";
7  import { useToast } from "@components/providers";
8  import { formatDate } from "@lib/constants";
9  import type { listReviews } from "@lib/services/reviews";
10
11 type Review = Awaited<ReturnType<typeof listReviews>>[number];
12
13 type Props = {

```

```

14   initialReviews: Review[];
15   trustScore: number;
16   ratingAverage: number;
17   ratingCount: number;
18 };
19
20 export function FarmerReviewBoard({ initialReviews, trustScore, ratingAverage, ratingCount }: Props) {
21   const router = useRouter();
22   const { push } = useToast();
23   const [reviews, setReviews] = useState(initialReviews);
24   const [filter, setFilter] = useState<"all" | "waiting" | "answered">("all");
25   const [drafts, setDrafts] = useState<Record<number, string>>({});
26   const [saving, setSaving] = useState<number | null>(null);
27
28   const unanswered = reviews.filter((review) => !review.farmerReply).length;
29   const shown = useMemo(() => reviews.filter((review) => {
30     if (filter === "waiting") return !review.farmerReply;
31     if (filter === "answered") return !!review.farmerReply;
32     return true;
33   }), [reviews, filter]);
34
35   async function respond(reviewId: number) {
36     const reply = (drafts[reviewId] ?? "").trim();
37     if (reply.length < 2 || reply.length > 600) return;
38     setSaving(reviewId);
39     try {
40       const updated = await api<{ farmerReply: string; repliedAt: string }>(`api/reviews/${reviewId}/reply`, { method: "POST", json: { reply } });
41       setReviews((current) => current.map((item) => item.reviewId === reviewId ? { ...item, farmerReply: updated.farmerReply, repliedAt: new Date(updated.repliedAt) } : item));
42       setDrafts((current) => ({ ...current, [reviewId]: "" }));
43       push({ kind: "success", title: "Reply published", body: "The buyer has been notified." });
44       router.refresh();
45     } catch (error) {
46       push({ kind: "error", title: "Could not publish reply", body: (error as Error).message });
47     } finally {
48       setSaving(null);
49     }
50   }
51
52   return (
53     <div className="space-y-5">
54       <PageHeader title="Buyer reviews" urdu="[Urdu]" subtitle="Respond to feedback from completed orders"
55       actions={<Link href="/account" className="btn-secondary">[Urdu]Manage profile</Link> } />
56       <div className="grid grid-cols-2 gap-3 sm:grid-cols-3">
57         <StatCard label="Trust score" value={ratingCount ? `${trustScore.toFixed(2)} / 5` : "New"} icon=""
58         sub="Verified sales & feedback" />
59         <StatCard label="Buyer rating" value={ratingCount ? `${ratingAverage.toFixed(1)} / 5` : "-"} icon=""
60         tone="amber" sub={`${ratingCount} verified ${ratingCount === 1 ? "review" : "reviews"}` />
61         <StatCard label="Awaiting response" value={unanswered} icon="" tone="sky" sub="Reply once per review" />
62       </div>
63       <div className="flex flex-wrap gap-2" role="group" aria-label="Filter reviews">
64         {[
65           ["all", "All", reviews.length],
66           ["waiting", "Needs reply", unanswered],
67           ["answered", "Replied", reviews.length - unanswered],
68         ] as const}.map(([key, label, count]) => (
69           <button key={key} type="button" onClick={() => setFilter(key)} aria-pressed={filter === key} className={
70             `rounded-full px-3.5 py-1.5 text-xs font-semibold transition ${filter === key ? "bg-brand-900 text-white" : "bg-white text-slate-600 ring-1 ring-slate-200 hover:bg-brand-50 dark:bg-slate-900 dark:text-slate-300 dark:ring-slate-700"}`
71             >{label} <span className="opacity-70">{count}</span>
72           </button>
73         ))
74       </div>
75       {!shown.length ? (
76         <EmptyState icon="" title={filter === "all" ? "No reviews yet" : "Nothing here"} body={filter === "all" ? "Buyer feedback appears after an order is delivered and paid." : "Try another filter to see more feedback."} />
77       ) : (
78         <div className="grid gap-4 lg:grid-cols-2">
79           {shown.map((review) => (

```

```

79         <article key={review.reviewId} className="card flex flex-col gap-4 p-5">
80             <div className="flex flex-wrap items-start justify-between gap-2">
81                 <div>
82                     <p className="text-sm font-semibold">{review.buyerName ?? review.buyerFullName}</p>
83                     <p className="text-xs text-slate-500">
84                         ✓ Verified purchase · <Link href={`\crop/${review.cropId}`} className="font-medium tex
85 t-brand-700 hover:underline dark:text-brand-400">{review.cropName}</Link> · {formatDate(review.createdAt)}
86                     </p>
87                     </div>
88                     <span className="text-gold-500" aria-label={`\${review.rating} out of 5 stars`} >
89                         {"*".repeat(review.rating)}<span className="text-slate-300 dark:text-slate-700">{"*".rep
90 eat(5 - review.rating)}</span>
91                     </span>
92                     </div>
93                     {review.comment && <p className="text-sm leading-relaxed text-slate-700 dark:text-slate-300"
94 >{review.comment}&rdquo;</p>}
95                     {review.farmerReply ? (
96                         <div className="rounded-xl border-l-2 border-brand-600 bg-brand-50/60 px-4 py-3 text-sm da
97 rk.bg-brand-950/30">
98                             <p className="text-xs font-semibold tracking-wide text-brand-800 uppercase dark:text-bra
99 nd-300">Your response · {formatDate(review.repliedAt)}</p>
100                             <p className="mt-1 whitespace-pre-wrap">{review.farmerReply}</p>
101                         </div>
102                     ) : (
103                         <div className="mt-auto space-y-2 border-t border-slate-100 pt-4 dark:border-slate-800">
104                             <label htmlFor={`reply-${review.reviewId}`} className="label">Reply to this buyer</label>
105                             <textarea
106                                 id={`reply-${review.reviewId}`}
107                                 rows={2}
108                                 maxLength={600}
109                                 value={drafts[review.reviewId] ?? ""}
110                                 onChange={(event) => setDrafts((current) => ({ ...current, [review.reviewId]: event.ta
111 rget.value })))}
112                                 placeholder="Thank you for your feedback."
113                                 className="input resize-y"
114                             />
115                             <div className="flex items-center justify-between gap-2">
116                                 <p className="text-xs text-slate-500">Visible to buyers · {(drafts[review.reviewId] ??
117 "" ).length}/600</p>
118                                 <button type="button" className="btn-primary" disabled={saving !== null || (drafts[rev
119 iew.reviewId] ?? "").trim().length < 2} onClick={() => respond(review.reviewId)}>
120                                     {saving === review.reviewId && <Spinner />} Publish reply
121                                 </button>
122                             </div>
123                         </div>
124                     )}
125                 </article>
126             </div>
127         </div>
128     );
129 }
130

```

src/components/reviews/review-prompts.tsx

```

1  "use client";
2
3  import { useRouter } from "next/navigation";
4  import { useState } from "react";
5  import { api, Spinner } from "@components/ui";
6  import { useToast } from "@components/providers";
7
8  type Item = { orderId: number; trackingNumber: string; cropName: string; farmer: string };
9
10 /** Post-delivery feedback loop: buyers rate sellers 1-5 and leave a review (updates the seller's trust sc
11 ore). */
12 export function ReviewPrompts({ items }: { items: Item[] }) {
13     const { push } = useToast();
14     const router = useRouter();
15     const [open, setOpen] = useState<number | null>(items[0]?.orderId ?? null);
16     const [rating, setRating] = useState(0);
17     const [hover, setHover] = useState(0);
18

```

```

17   const [comment, setComment] = useState("");
18   const [busy, setBusy] = useState(false);
19   const [done, setDone] = useState<number[]>([]);
20   const list = items.filter((i) => !done.includes(i.orderId));
21   if (!list.length) return null;
22
23   async function submit(orderId: number) {
24     if (!rating) return push({ kind: "error", title: "Choose a star rating" });
25     setBusy(true);
26     try {
27       const r = await api<{ trust: { trustScore: number } }>("/api/reviews", { method: "POST", json: { ord
28         erId, rating, comment: comment || undefined } });
29       push({ kind: "success", title: "Thanks for your review!", body: `Seller trust score is now ${r.trust
30         .trustScore.toFixed(2)}/5.` });
31       setDone((d) => [...d, orderId]);
32       setRating(0);
33       setComment("");
34       setOpen(list.find((i) => i.orderId !== orderId)?.orderId ?? null);
35       router.refresh();
36     } catch (e) {
37       push({ kind: "error", title: (e as Error).message });
38     } finally {
39       setBusy(false);
40     }
41   }
42
43   return (
44     <div className="rounded-2xl border border-gold-400/40 bg-gold-300/10 p-4">
45       <p className="text-sm font-bold"> Rate your recent deliveries ({list.length})</p>
46       <ul className="mt-3 space-y-2">
47         {list.map((i) => (
48           <li key={i.orderId} className="rounded-xl bg-white p-3 dark:bg-white/5">
49             <button className="flex w-full items-center justify-between text-left text-sm" onClick={() =>
50               setOpen(open === i.orderId ? null : i.orderId)}>
51               <span><b>{i.farmer}</b> · {i.cropName} <span className="font-mono text-xs text-slate-500">{i
52                 .trackingNumber}</span></span>
53               <span className="text-xs font-semibold text-brand-700 dark:text-brand-400">{open === i.order
54                 Id ? "Close" : "Rate"}</span>
55             </button>
56             {open === i.orderId && (
57               <div className="animate-fade-in mt-3 space-y-3">
58                 <div className="flex items-center gap-1" onMouseLeave={() => setHover(0)} role="radiogroup
59                   " aria-label="Rating">
60                   {[1, 2, 3, 4, 5].map((n) => (
61                     <button key={n} type="button" role="radio" aria-checked={rating === n} aria-label={`$
62                       {n} star${n > 1 ? "s" : ""}`} onMouseEnter={() => setHover(n)} onClick={() => setRating(n)}
63                       className={`text-3xl leading-none transition ${hover || rating} >= n ? "text-gold-5
64                         00" : "text-slate-300 dark:text-slate-600"}`}>*</button>
65                   )))}
66                 <span className="ml-2 text-xs text-slate-500">{["", "Poor", "Fair", "Good", "Very good",
67                   "Excellent"][hover || rating]}</span>
68               </div>
69               <textarea className="input" rows={2} maxLength={1000} value={comment} onChange={(e) => set
70                 Comment(e.target.value)} placeholder="Quality, weight accuracy, packing, communication..." />
71               <button className="btn-primary" disabled={busy} onClick={() => submit(i.orderId)}>{busy &&
72                 <Spinner />} Submit review</button>
73             </div>
74           </li>
75         ))}
76       </ul>
77     </div>
78   );
79 }

```

src/components/stepper.tsx

```

1   "use client";
2
3   import { DELIVERY_STAGES, stageIndex, type DeliveryStage } from "@lib/constants";
4
5   const ICONS = ["", "", "", "", ""];
6
7   export function LogisticsStepper({ stage, compact = false, disputed = false }: { stage: DeliveryStage; com

```

```

    pact?: boolean; disputed?: boolean }) {
    8     const current = stageIndex(stage);
    9     return (
    10     <ol className="flex w-full items-start" aria-label="Delivery progress">
    11     {DELIVERY_STAGES.map((s, i) => {
    12     const done = i < current || (i === current && s.key === "delivered");
    13     const active = i === current && s.key !== "delivered";
    14     return (
    15     <li key={s.key} className="relative flex flex-1 flex-col items-center text-center">
    16     {i > 0 && (
    17     <span
    18     className={`absolute top-4 right-1/2 h-1 w-full -translate-y-1/2 rounded ${
    19     i <= current ? "bg-brand-500" : "bg-slate-200 dark:bg-slate-700"
    20     }`}
    21     aria-hidden
    22     />
    23     )}
    24     <span
    25     className={`relative z-10 grid place-items-center rounded-full border-2 transition ${compact
    26     ? "h-8 w-8 text-sm" : "h-9 w-9 text-base"} ${
    27     done
    28     ? "border-brand-500 bg-brand-500 text-white"
    29     : active
    30     ? "border-rose-500 bg-rose-50 text-rose-600 ring-4 ring-rose-500/20 dark:bg-rose-950
    31     "
    32     : "border-brand-500 bg-white text-brand-600 ring-4 ring-brand-500/20 dark:bg-slate-9
    33     00"
    34     : "border-slate-300 bg-white text-slate-400 dark:border-slate-600 dark:bg-slate-900"
    35     }`}
    36     >
    37     {done ? "✓" : ICONS[i]}
    38     {active && !disputed && <span className="absolute inset-0 animate-ping rounded-full border-2
    39     border-brand-400 opacity-40" />}
    40     </span>
    41     <span
    42     className={`mt-2 px-0.5 text-[10px] leading-tight font-semibold sm:text-xs ${
    43     i <= current ? "text-slate-900 dark:text-slate-100" : "text-slate-400 dark:text-slate-500"
    44     }`}
    45     >
    46     {compact ? s.short : s.label}
    47     </span>
    48     </li>
    49     );
    50     });
    51 }

```

src/components/tracking/tracking-client.tsx

```

1  "use client";
2
3  import dynamic from "next/dynamic";
4  import { useRouter } from "next/navigation";
5  import { useCallback, useEffect, useState } from "react";
6  import { PageHeader, Spinner } from "@components/ui";
7  import { LogisticsStepper } from "@components/stepper";
8  import { formatKg, type DeliveryStage } from "@lib/constants";
9  import type { MapData } from "@components/tracking/tracking-map";
10
11  const TrackingMap = dynamic(() => import("@components/tracking/tracking-map").then((m) => m.TrackingMap),
12  {
13    ssr: false,
14    loading: () => <div className="h-[380px] animate-pulse rounded-2xl bg-slate-100 dark:bg-white/5" />,
15  });
16
17  type Track = {
18    trackingNumber: string; cropName: string; quantityKg: number; stage: DeliveryStage; stageLabel: string;
19    payment: string; disputed: boolean;
20    origin: MapData["origin"]; destination: MapData["destination"]; current: MapData["current"]; route: MapD
21    ata["route"];
22    distanceKm: number; eta: string | null; lastUpdate: string;
23    events: { id: number; stage: string; label: string; location: string | null; lat: number | null; lng: nu

```

```

20 mber | null; note: string | null; at: string []];
21 parties?: { farmer: string | null; buyer: string | null };
22 };
23
24 const ICON: Record<string, string> = { confirmed: "", escrow_locked: "", quality_checked: "", dispatched:
25   "", in_transit: "", gps_ping: "", location_update: "", arriving: "[Urdu]", delivered: "", escrow_released:
26   "", escrow_refunded: "⚡[Urdu]", dispute_opened: "⚠[Urdu]" };
27
28 export function TrackingClient({ initial, mine }: { initial: string; mine: { trackingNumber: string; cropName: string; stage: string }[] }) {
29   const router = useRouter();
30   const [input, setInput] = useState(initial);
31   const [no, setNo] = useState(initial);
32   const [data, setData] = useState<Track | null>(null);
33   const [error, setError] = useState<string | null>(null);
34   const [loading, setLoading] = useState(false);
35
36   const load = useCallback(async (tn: string, quiet = false) => {
37     if (!tn) return;
38     if (!quiet) setLoading(true);
39     try {
40       const r = await fetch(`/api/tracking/${encodeURIComponent(tn)}`, { cache: "no-store" }).then((x) =>
41         x.json());
42       if (!r.success) throw new Error(r.error);
43       setData(r.data);
44       setError(null);
45     } catch (e) {
46       setError((e as Error).message);
47       if (!quiet) setData(null);
48     } finally {
49       setLoading(false);
50     }
51   }, []);
52
53   useEffect(() => {
54     // eslint-disable-next-line react-hooks/set-state-in-effect
55     load(no);
56     if (!no) return;
57     const id = setInterval(() => load(no, true), 20_000);
58     return () => clearInterval(id);
59   }, [no, load]);
60
61   function submit(e: React.FormEvent) {
62     e.preventDefault();
63     const tn = input.trim().toUpperCase();
64     setNo(tn);
65     router.replace(`/tracking?no=${encodeURIComponent(tn)}`, { scroll: false });
66   }
67
68   return (
69     <div>
70       <PageHeader title="Shipment Tracking" urdu="[Urdu]" subtitle="Live GPS position, route and milestone
71         s for every MarketLink consignment" />
72       <form onSubmit={submit} className="card flex flex-col gap-3 p-4 sm:flex-row">
73         <input className="input flex-1 font-mono uppercase" placeholder="Tracking number, e.g. ML-2026-100
74         1" value={input} onChange={(e) => setInput(e.target.value)} aria-label="Tracking number" />
75         <button className="btn-primary px-6" disabled={loading || !input.trim()}>{loading && <Spinner />}
76         Track</button>
77       </form>
78       {mine.length > 0 && (
79         <div className="mt-3 flex flex-wrap items-center gap-2 text-xs">
80           <span className="text-slate-500">Your active shipments:</span>
81           {mine.map((m) => (
82             <button key={m.trackingNumber} onClick={() => { setInput(m.trackingNumber); setNo(m.trackingNu
83             mber); router.replace(`/tracking?no=${m.trackingNumber}`, { scroll: false }); }}
84             className={`rounded-full px-3 py-1 font-semibold ${no === m.trackingNumber ? "bg-brand-900 t
85             ext-white" : "bg-white ring-1 ring-slate-200 dark:bg-white/5 dark:ring-white/10"}`}>
86               {m.cropName} · {m.trackingNumber}
87             </button>
88           ))}
89         </div>
90       )}
91       {error && <p className="mt-6 rounded-2xl bg-rose-50 p-4 text-sm text-rose-800 dark:bg-rose-950/30 da
92       rk:text-rose-200">{error}</p>}
93       {!data && !error && !loading && <p className="mt-10 text-center text-sm text-slate-500">Enter a trac

```

```

    king number to see the shipment on the map.</p>
86
87    {data && (
88      <div className="mt-6 grid gap-6 lg:grid-cols-[1.6fr_1fr]">
89        <div className="space-y-5">
90          <div className="card p-5">
91            <div className="flex flex-wrap items-start justify-between gap-3">
92              <div>
93                <p className="font-mono text-xs font-semibold text-slate-500">{data.trackingNumber}</p>
94                <p className="text-xl font-bold">{data.cropName} · {formatKg(data.quantityKg)}</p>
95                <p className="text-sm text-slate-500">{data.origin.city} → {data.destination.city} · ~{d
    ata.distanceKm} km by road</p>
96              </div>
97              <div className="text-right">
98                <span className={`badge ${data.stage === "delivered" ? "bg-emerald-100 text-emerald-800"
    : data.disputed ? "bg-rose-100 text-rose-800" : "bg-sky-100 text-sky-800"}`}>{data.stageLabel}</span>
99                <p className="mt-1 text-xs text-slate-500">Payment: {data.payment}</p>
100              </div>
101            </div>
102            <div className="mt-5"><LogisticsStepper stage={data.stage} disputed={data.disputed} /></div>
103          </div>
104          <TrackingMap data={{ origin: data.origin, destination: data.destination, current: data.current
    , route: data.route, events: data.events, delivered: data.stage === "delivered" }} />
105          <div className="grid grid-cols-3 gap-3 text-center text-sm">
106            <div className="card p-3"><p className="text-[11px] font-semibold tracking-wider text-slate-
    500 uppercase">Progress</p><p className="text-lg font-bold">{data.current.progress}%</p></div>
107            <div className="card p-3"><p className="text-[11px] font-semibold tracking-wider text-slate-
    500 uppercase">ETA</p><p className="text-lg font-bold">{data.eta ? new Date(data.eta).toLocaleString("en-G
    B", { day: "2-digit", month: "short", hour: "2-digit", minute: "2-digit" }) : data.stage === "delivered" ?
    "Delivered" : "-"}</p></div>
108            <div className="card p-3"><p className="text-[11px] font-semibold tracking-wider text-slate-
    500 uppercase">Position</p><p className="text-lg font-bold">{data.current.source === "gps" ? " Live GPS" :
    data.current.source === "simulated" ? "≈ Estimated" : "At farm"}</p></div>
109          </div>
110        </div>
111        <aside className="card h-fit p-5">
112          <div className="flex items-center justify-between">
113            <h2 className="font-bold">Timeline</h2>
114            <span className="text-[11px] text-slate-500">Auto-refresh 20s</span>
115          </div>
116          {data.parties && <p className="mt-2 text-xs text-slate-500">{data.parties.farmer} → {data.part
    ies.buyer}</p>}
117          <ol className="relative mt-4 space-y-4 border-l border-slate-200 pl-6 dark:border-white/10">
118            {[...data.events].reverse().map((e) => (
119              <li key={e.id} className="relative">
120                <span className="absolute -left-[33px] grid h-6 w-6 place-items-center rounded-full bg-w
    hite text-xs ring-2 ring-slate-200 dark:bg-[#0d1f16] dark:ring-white/10">{ICON[e.stage]} ?? "•"</span>
121                <p className="text-sm font-semibold">{e.label}</p>
122                <p className="text-xs text-slate-500">{new Date(e.at).toLocaleString("en-GB", { day: "2-
    digit", month: "short", hour: "2-digit", minute: "2-digit" })}{e.location ? ` · ${e.location}` : ""}</p>
123                {e.note && <p className="text-xs text-slate-600 dark:text-slate-400">{e.note}</p>}
124              </li>
125            )]}
126          </ol>
127        </aside>
128      </div>
129    )}
130  </div>
131  </div>
132  );
133  }
134

```

src/components/tracking/tracking-map.tsx

```

1  "use client";
2
3  import { useEffect, useRef } from "react";
4  import "leaflet/dist/leaflet.css";
5  import type { Map as LeafletMap, LayerGroup } from "leaflet";
6
7  type Pt = { lat: number; lng: number };
8  export type MapData = {
9    origin: Pt & { city: string };
10   destination: Pt & { city: string };

```

```

11     current: Pt & { progress: number; source: string };
12     route: Pt[];
13     events: { id: number; label: string; lat: number | null; lng: number | null; location: string | null }[]
14     ;
15     delivered: boolean;
16 };
17
18 const pin = (bg: string, label: string) =>
19     `

MARKETLINK AGRI-HUB · Complete Project Book



Page 180 of 279


```

src/components/ui.tsx

```
1  "use client";
2
3  import { useEffect, type ReactNode } from "react";
4  import { STATUS_LABELS, STATUS_STYLES } from "@lib/constants";
5  import { useI18n } from "@components/providers";
6
7  export async function api<T = unknown>(url: string, init?: RequestInit & { json?: unknown }): Promise<T> {
8    const res = await fetch(url, {
9      ...init,
10     headers: { "content-type": "application/json", ...(init?.headers ?? {}) },
11     body: init?.json !== undefined ? JSON.stringify(init.json) : init?.body,
12     cache: "no-store",
13   });
14   const data = await res.json().catch(() => ({ success: false, error: "Invalid server response" }));
15   if (!res.ok || !data.success) throw new Error(data.error || `Request failed (${res.status})`);
16   return data.data as T;
17 }
18
19 export function Badge({ status, label, dot = true }: { status: string; label?: string; dot?: boolean }) {
20   const style = STATUS_STYLES[status] ?? "bg-slate-100 text-slate-700 dark:bg-slate-800 dark:text-slate-300";
21   return (
22     <span className={`inline-flex items-center gap-1.5 rounded-full px-2 py-0.5 text-[11px] font-semibold whitespace-nowrap ring-1 ring-current/15 ring-inset ${style}`}>
23       {dot && <span className="h-1.5 w-1.5 rounded-full bg-current opacity-70" aria-hidden />}
24       {label ?? STATUS_LABELS[status] ?? status.replace(/_/g, " ")}
25     </span>
26   );
27 }
28
29 export function GradeBadge({ grade }: { grade: string }) {
30   const c =
31     grade === "A"
32       ? "bg-emerald-600 text-white"
33       : grade === "B"
34         ? "bg-amber-500 text-white"
35         : "bg-slate-500 text-white";
36   return <span className={`inline-flex h-6 min-w-6 items-center justify-center rounded-md px-1.5 text-xs font-bold ${c}`}>Grade {grade}</span>;
37 }
38
39 export function StatCard({ label, value, sub, icon, tone = "brand" }: { label: string; value: ReactNode; sub?: ReactNode; icon: string; tone?: "brand" | "amber" | "sky" | "rose" | "violet" }) {
40   const tones = {
41     brand: { chip: "bg-brand-900 text-white", bar: "from-brand-500 to-brand-900" },
42     amber: { chip: "bg-gold-400 text-brand-950", bar: "from-gold-300 to-gold-500" },
43     sky: { chip: "bg-sky-600 text-white", bar: "from-sky-400 to-sky-700" },
44     rose: { chip: "bg-rose-600 text-white", bar: "from-rose-400 to-rose-700" },
45     violet: { chip: "bg-violet-600 text-white", bar: "from-violet-400 to-violet-700" },
46   };
47   return (
48     <div className="card relative overflow-hidden p-4 sm:p-5">
49       <span className={`absolute inset-x-0 top-0 h-1 bg-gradient-to-r ${tones[tone].bar}`} />
50       <div className="flex items-start justify-between gap-3">
51         <div className="min-w-0">
52           <p className="text-[11px] font-bold tracking-wider text-slate-500 uppercase dark:text-slate-400">{label}</p>
53           <p className="mt-2 text-lg leading-tight font-extrabold tracking-tight whitespace-nowrap tabular-nums sm:text-xl 2xl:text-2xl">{value}</p>
54           {sub && <p className="mt-1 text-xs text-slate-500 dark:text-slate-400">{sub}</p>}
55         </div>
56         <div className={`grid h-11 w-11 shrink-0 place-items-center rounded-2xl text-xl shadow-sm ${tones[tone].chip}`}>{icon}</div>
57       </div>
58     </div>
59   );
60 }
61
62 export function PageHeader({ title, subtitle, actions, urdu }: { title: string; subtitle?: string; actions?: ReactNode; urdu?: string }) {
63   const { lang } = useI18n();
64   return (
65     <div className="mb-6 flex flex-col gap-3 sm:flex-row sm:items-end sm:justify-between">
```

```

66     <div className="min-w-0">
67       <div className="mb-2 flex items-center gap-2">
68         <span className="h-0.5 w-7 rounded-full bg-gradient-to-r from-brand-900 to-gold-400" aria-hidden
69         />
70         <span className="text-[10px] font-bold tracking-[0.18em] text-brand-700 uppercase dark:text-gold
71         -400">MARKETLINK AGRI-HUB</span>
72       </div>
73       <h1 className="text-2xl font-extrabold tracking-tight text-brand-950 sm:text-3xl dark:text-white">
74       {lang === "ur" && urdu ? urdu : title}</h1>
75       {subtitle && <p className="mt-1 text-sm text-slate-500 dark:text-slate-400">{subtitle}</p>}
76     </div>
77     {actions && <div className="flex flex-wrap gap-2">{actions}</div>}
78   </div>
79   );
80 }
81
82 export function EmptyState({ icon, title, body, action }: { icon: string; title: string; body?: string; ac
83 tion?: ReactNode }) {
84   return (
85     <div className="card flex flex-col items-center px-6 py-14 text-center">
86       <div className="text-5xl">{icon}</div>
87       <p className="mt-3 font-semibold">{title}</p>
88       {body && <p className="mt-1 max-w-md text-sm text-slate-500 dark:text-slate-400">{body}</p>}
89       {action && <div className="mt-4">{action}</div>}
90     </div>
91   );
92 }
93
94 export function Spinner({ className = "" }: { className?: string }) {
95   return <span className={`inline-block h-4 w-4 animate-spin rounded-full border-2 border-current border-r
96   -transparent ${className}`} />;
97 }
98
99 export function LoadingBlock() {
100   return (
101     <div className="grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
102       {[0, 1, 2].map((i) => (
103         <div key={i} className="card h-40 animate-pulse bg-slate-100 dark:bg-slate-800/60" />
104       ))}
105     </div>
106   );
107 }
108
109 function useEscape(open: boolean, onClose: () => void) {
110   useEffect(() => {
111     if (!open) return;
112     const h = (e: KeyboardEvent) => e.key === "Escape" && onClose();
113     window.addEventListener("keydown", h);
114     document.body.style.overflow = "hidden";
115     return () => {
116       window.removeEventListener("keydown", h);
117       document.body.style.overflow = "";
118     };
119   }, [open, onClose]);
120 }
121
122 export function Modal({ open, onClose, title, children, wide }: { open: boolean; onClose: () => void; titl
123 e: string; children: ReactNode; wide?: boolean }) {
124   useEscape(open, onClose);
125   if (!open) return null;
126   return (
127     <div className="fixed inset-0 z-50 flex items-end justify-center bg-slate-950/50 p-0 backdrop-blur-sm
128     sm:items-center sm:p-4" onClick={onClose}>
129       <div
130         role="dialog"
131         aria-modal="true"
132         className={`animate-fade-in card max-h-[92vh] w-full overflow-y-auto rounded-b-none sm:rounded-2xl
133         ${wide ? "sm:max-w-3xl" : "sm:max-w-lg"}>
134         <h3 className="font-semibold">{title}</h3>
135         <button onClick={onClose} className="rounded-lg p-1.5 text-slate-500 hover:bg-slate-100 dark:hov
136         er:bg-slate-800" aria-label="Close">x</button>
137       </div>
138     </div>
139   );

```

```

132     <div className="p-5">{children}</div>
133   </div>
134 </div>
135   );
136 }
137
138 export function Drawer({ open, onClose, title, children, wide = false }: { open: boolean; onClose: () => v
  ↵ oid; title: ReactNode; children: ReactNode; wide?: boolean }) {
139   useEscape(open, onClose);
140   if (!open) return null;
141   return (
142     <div className="fixed inset-0 z-50 bg-slate-950/50 backdrop-blur-sm" onClick={onClose}>
143       <aside
144         className={`animate-slide-in absolute top-0 right-0 flex h-full w-full flex-col bg-white shadow-2x
  ↵ l dark:bg-[#0d1f16] ${wide ? "max-w-xl" : "max-w-md"}`}
145         onClick={(e) => e.stopPropagation()}
146         dir="ltr"
147       >
148         <div className="flex items-center justify-between border-b border-slate-200 px-5 py-4 dark:border-
  ↵ slate-800">
149           <h3 className="font-semibold">{title}</h3>
150           <button onClick={onClose} className="rounded-lg p-1.5 text-slate-500 hover:bg-slate-100 dark:hov
  ↵ er:bg-slate-800" aria-label="Close">x</button>
151         </div>
152         <div className="flex-1 overflow-y-auto p-5">{children}</div>
153       </aside>
154     </div>
155   );
156 }
157
158 /** Resize + JPEG-compress an image file client-side to a data URL. */
159 export function compressImage(file: File, maxDim = 960, quality = 0.72): Promise<string> {
160   return new Promise((resolve, reject) => {
161     const reader = new FileReader();
162     reader.onerror = () => reject(new Error("Could not read file"));
163     reader.onload = () => {
164       const img = new Image();
165       img.onerror = () => reject(new Error("Invalid image"));
166       img.onload = () => {
167         const scale = Math.min(1, maxDim / Math.max(img.width, img.height));
168         const canvas = document.createElement("canvas");
169         canvas.width = Math.round(img.width * scale);
170         canvas.height = Math.round(img.height * scale);
171         canvas.getContext("2d")!.drawImage(img, 0, 0, canvas.width, canvas.height);
172         resolve(canvas.toDataURL("image/jpeg", quality));
173       };
174       img.src = reader.result as string;
175     };
176     reader.readAsDataURL(file);
177   });
178 }
179

```

src/db/index.ts

```

1  import fs from "fs";
2  import path from "path";
3  import { Client, Pool } from "pg";
4  import { PGLite } from "@electric-sql/pglite";
5  import { drizzle, type NodePgDatabase } from "drizzle-orm/node-postgres";
6  import { drizzle as drizzlePglite } from "drizzle-orm/pglite";
7
8  /**
9   * Database connection.
10  *
11  * • DATABASE_URL=postgresql://... → real PostgreSQL server (production / Docker / cloud)
12  * • DATABASE_URL empty or unset → embedded PostgreSQL (PGLite) stored in ./data/pglite
13  *   – zero setup, perfect for local hosting on any PC.
14  * • DATABASE_URL=pglite:./my-dir → embedded PostgreSQL in a custom folder
15  *
16  * The client is created lazily on first use so `next build` never opens a connection.
17  */
18  export type DB = NodePgDatabase;
19
20  const g = globalThis as typeof globalThis & {

```

```

21   __mldb?: DB;
22   __mlPool?: Pool;
23   __mlPgLite?: PGlite;
24 };
25
26 const rawUrl = () => (process.env.DATABASE_URL ?? "").trim();
27
28 export function usingEmbeddedDb() {
29   const u = rawUrl();
30   return !u || u.startsWith("pglite:") || u.startsWith("file:");
31 }
32
33 function embeddedDir() {
34   const u = rawUrl();
35   if (u.startsWith("pglite:") || u.startsWith("file:")) {
36     const p = u.replace(/^(pglite|file):(\\/\//)?/, "");
37     return path.isAbsolute(p) ? p : path.join(/*turbopackIgnore: true*/ process.cwd(), p);
38   }
39   return path.join(/*turbopackIgnore: true*/ process.cwd(), ".data", "pglite");
40 }
41
42 function create(): DB {
43   if (usingEmbeddedDb()) {
44     const dir = embeddedDir();
45     fs.mkdirSync(dir, { recursive: true });
46     if (!g.__mlPgLite) {
47       g.__mlPgLite = new PGlite(dir);
48       console.log(`[db] Using built-in PostgreSQL (PGlite) → ${dir}`);
49     }
50     return drizzlePgLite(g.__mlPgLite) as unknown as DB;
51   }
52   if (!g.__mlPool) g.__mlPool = new Pool({ connectionString: rawUrl() });
53   return drizzle(g.__mlPool);
54 }
55
56 function getDb(): DB {
57   if (!g.__mldb) g.__mldb = create();
58   return g.__mldb;
59 }
60
61 export const db: DB = new Proxy({} as DB, {
62   get(_target, prop) {
63     const real = getDb() as unknown as Record<string | symbol, unknown>;
64     const value = real[prop];
65     return typeof value === "function" ? (value as (...a: unknown[]) => unknown).bind(real) : value;
66   },
67 });
68
69 /**
70  * For real PostgreSQL servers: if the database named in DATABASE_URL does not exist yet
71  * (error 3D000), connect to the default "postgres" database and create it.
72  */
73 export async function ensureDatabaseExists() {
74   if (usingEmbeddedDb()) return;
75   const url = rawUrl();
76   const probe = new Client({ connectionString: url });
77   try {
78     await probe.connect();
79     await probe.end();
80     return;
81   } catch (e) {
82     await probe.end().catch(() => {});
83     if ((e as { code?: string }).code !== "3D000") throw e;
84   }
85   const target = new URL(url);
86   const name = decodeURIComponent(target.pathname.replace(/^\//, ""));
87   if (!/^[A-Za-z0-9\_]+$/ .test(name)) throw new Error(`Database "${name}" does not exist and has an unsafe name to auto-create`);
88   const adminUrl = new URL(url);
89   adminUrl.pathname = "/postgres";
90   const admin = new Client({ connectionString: adminUrl.toString() });
91   await admin.connect();
92   try {
93     await admin.query(`CREATE DATABASE "${name}"`);
94     console.log(`[db] Created missing database "${name}"`);
95   } catch (e) {

```

```

96     if ((e as { code?: string }).code !== "42P04") throw e; // already exists (race)
97   } finally {
98     await admin.end();
99   }
100 }
101

```

src/db/schema.ts

```

1  import {
2    pgTable,
3    pgEnum,
4    serial,
5    integer,
6    varchar,
7    text,
8    numeric,
9    boolean,
10   date,
11   timestamp,
12   jsonb,
13   index,
14   uniqueIndex,
15   doublePrecision,
16 } from "drizzle-orm/pg-core";
17
18 /* ----- */
19 /* Enums */
20 /* ----- */
21 export const userRoleEnum = pgEnum("user_role", ["farmer", "buyer", "admin", "inspector"]);
22 export const cropCategoryEnum = pgEnum("crop_category", ["grains", "vegetables", "fruits", "cash_crops"]);
23 export const qualityGradeEnum = pgEnum("quality_grade", ["A", "B", "C"]);
24 export const cropStatusEnum = pgEnum("crop_status", ["active", "pending_inspection", "sold_out", "archived"]);
25 export const bidStatusEnum = pgEnum("bid_status", ["pending", "accepted", "rejected", "countered", "withdrawn"]);
26 export const paymentStatusEnum = pgEnum("payment_status", [
27   "awaiting_escrow",
28   "escrow_locked",
29   "released",
30   "refunded",
31   "disputed",
32 ]);
33 export const deliveryStageEnum = pgEnum("delivery_stage", [
34   "confirmed",
35   "quality_checked",
36   "dispatched",
37   "in_transit",
38   "delivered",
39 ]);
40 export const inspectionStatusEnum = pgEnum("inspection_status", ["pending", "passed", "failed"]);
41 export const walletTxnTypeEnum = pgEnum("wallet_txn_type", [
42   "deposit",
43   "escrow_lock",
44   "escrow_release",
45   "payout",
46   "refund",
47   "platform_fee",
48   "withdrawal",
49 ]);
50 export const disputeStatusEnum = pgEnum("dispute_status", ["open", "investigating", "resolved_release", "resolved_refund"]);
51
52 const money = (name: string) => numeric(name, { precision: 14, scale: 2, mode: "number" });
53 const qty = (name: string) => numeric(name, { precision: 14, scale: 2, mode: "number" });
54
55 /* ----- */
56 /* 1. users */
57 /* ----- */
58 export const users = pgTable(
59   "users",
60   {
61     id: serial("id").primaryKey(),
62     fullName: varchar("full_name", { length: 120 }).notNull(),
63     email: varchar("email", { length: 160 }).notNull(),
64     passwordHash: varchar("password_hash", { length: 255 }).notNull(),

```

```

65     phone: varchar("phone", { length: 30 }).notNull(),
66     role: userRoleEnum("role").notNull(),
67     cnicId: varchar("cnic_id", { length: 20 }),
68     businessName: varchar("business_name", { length: 160 }),
69     city: varchar("city", { length: 80 }).notNull(),
70     address: text("address"),
71     isVerified: boolean("is_verified").notNull().default(false),
72     emailVerified: boolean("email_verified").notNull().default(false),
73     title: varchar("title", { length: 120 }),
74     userCode: varchar("user_code", { length: 40 }),
75     avatarUrl: text("avatar_url"),
76     googleId: varchar("google_id", { length: 64 }),
77     authProvider: varchar("auth_provider", { length: 20 }).notNull().default("password"),
78     trustScore: numeric("trust_score", { precision: 3, scale: 2, mode: "number" }).notNull().default(0),
79     ratingAvg: numeric("rating_avg", { precision: 3, scale: 2, mode: "number" }).notNull().default(0),
80     ratingCount: integer("rating_count").notNull().default(0),
81     walletBalance: money("wallet_balance").notNull().default(0),
82     escrowBalance: money("escrow_balance").notNull().default(0),
83     createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
84   },
85   (t) => [
86     uniqueIndex("users_email_uq").on(t.email),
87     uniqueIndex("users_google_id_uq").on(t.googleId),
88     uniqueIndex("users_user_code_uq").on(t.userCode),
89     index("users_role_idx").on(t.role),
90     index("users_city_idx").on(t.city),
91   ],
92 );
93
94 /* ----- */
95 /* 2. crops_inventory */
96 /* ----- */
97 export const cropsInventory = pgTable(
98   "crops_inventory",
99   {
100     cropId: serial("crop_id").primaryKey(),
101     farmerId: integer("farmer_id")
102       .notNull()
103       .references(() => users.id, { onDelete: "cascade" }),
104     cropName: varchar("crop_name", { length: 80 }).notNull(),
105     category: cropCategoryEnum("category").notNull(),
106     qualityGrade: qualityGradeEnum("quality_grade").notNull(),
107     totalQuantityKg: qty("total_quantity_kg").notNull(),
108     basePricePerKg: money("base_price_per_kg").notNull(),
109     harvestDate: date("harvest_date", { mode: "string" }).notNull(),
110     imagesJson: jsonb("images_json").$type<string[]>().notNull().default([]),
111     description: text("description"),
112     farmLocation: varchar("farm_location", { length: 80 }).notNull(),
113     latitude: doublePrecision("latitude"),
114     longitude: doublePrecision("longitude"),
115     status: cropStatusEnum("status").notNull().default("active"),
116     createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
117   },
118   (t) => [
119     index("crops_farmer_idx").on(t.farmerId),
120     index("crops_category_idx").on(t.category),
121     index("crops_status_idx").on(t.status),
122     index("crops_name_idx").on(t.cropName),
123   ],
124 );
125
126 /* ----- */
127 /* 3. mandi_rates */
128 /* ----- */
129 export const mandiRates = pgTable(
130   "mandi_rates",
131   {
132     rateId: serial("rate_id").primaryKey(),
133     cropName: varchar("crop_name", { length: 80 }).notNull(),
134     marketLocation: varchar("market_location", { length: 80 }).notNull(),
135     minPricePerKg: money("min_price_per_kg").notNull(),
136     maxPricePerKg: money("max_price_per_kg").notNull(),
137     avgPricePerKg: money("avg_price_per_kg").notNull(),
138     source: varchar("source", { length: 20 }).notNull().default("manual"),
139     dateUpdated: date("date_updated", { mode: "string" }).notNull(),
140   },

```

```

141     (t) => [
142       uniqueIndex("mandi_crop_market_date_uq").on(t.cropName, t.marketLocation, t.dateUpdated),
143       index("mandi_date_idx").on(t.dateUpdated),
144     ],
145   );
146
147   /* ----- */
148   /* 4. bids_negotiations */
149   /* ----- */
150   export const bidsNegotiations = pgTable(
151     "bids_negotiations",
152     {
153       bidId: serial("bid_id").primaryKey(),
154       cropId: integer("crop_id")
155         .notNull()
156         .references(() => cropsInventory.cropId, { onDelete: "cascade" }),
157       buyerId: integer("buyer_id")
158         .notNull()
159         .references(() => users.id, { onDelete: "cascade" }),
160       bidPricePerKg: money("bid_price_per_kg").notNull(),
161       bidQuantityKg: qty("bid_quantity_kg").notNull(),
162       targetDeliveryDate: date("target_delivery_date", { mode: "string" }),
163       message: text("message"),
164       status: bidStatusEnum("status").notNull().default("pending"),
165       counterPrice: money("counter_price"),
166       counterNote: text("counter_note"),
167       createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
168       updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow(),
169     },
170     (t) => [
171       index("bids_crop_idx").on(t.cropId),
172       index("bids_buyer_idx").on(t.buyerId),
173       index("bids_status_idx").on(t.status),
174     ],
175   );
176
177   /* ----- */
178   /* 5. orders_logistics */
179   /* ----- */
180   export const ordersLogistics = pgTable(
181     "orders_logistics",
182     {
183       orderId: serial("order_id").primaryKey(),
184       bidId: integer("bid_id")
185         .notNull()
186         .references(() => bidsNegotiations.bidId, { onDelete: "restrict" }),
187       cropId: integer("crop_id")
188         .notNull()
189         .references(() => cropsInventory.cropId, { onDelete: "restrict" }),
190       farmerId: integer("farmer_id")
191         .notNull()
192         .references(() => users.id, { onDelete: "restrict" }),
193       buyerId: integer("buyer_id")
194         .notNull()
195         .references(() => users.id, { onDelete: "restrict" }),
196       quantityKg: qty("quantity_kg").notNull(),
197       pricePerKg: money("price_per_kg").notNull(),
198       totalAmount: money("total_amount").notNull(),
199       paymentStatus: paymentStatusEnum("payment_status").notNull().default("awaiting_escrow"),
200       deliveryStage: deliveryStageEnum("delivery_stage").notNull().default("confirmed"),
201       transitLocation: varchar("transit_location", { length: 120 }),
202       currentLat: doublePrecision("current_lat"),
203       currentLng: doublePrecision("current_lng"),
204       trackingNumber: varchar("tracking_number", { length: 40 }).notNull(),
205       expectedDeliveryDate: date("expected_delivery_date", { mode: "string" }),
206       createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
207       updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow(),
208     },
209     (t) => [
210       uniqueIndex("orders_bid_uq").on(t.bidId),
211       uniqueIndex("orders_tracking_uq").on(t.trackingNumber),
212       index("orders_farmer_idx").on(t.farmerId),
213       index("orders_buyer_idx").on(t.buyerId),
214       index("orders_stage_idx").on(t.deliveryStage),
215     ],
216   );

```

```

217
218 /* Order tracking history (audit trail for the logistics stepper) */
219 export const orderEvents = pgTable(
220   "order_events",
221   {
222     eventId: serial("event_id").primaryKey(),
223     orderId: integer("order_id")
224       .notNull()
225       .references(() => ordersLogistics.orderId, { onDelete: "cascade" }),
226     stage: varchar("stage", { length: 40 }).notNull(),
227     location: varchar("location", { length: 120 }),
228     lat: doublePrecision("lat"),
229     lng: doublePrecision("lng"),
230     note: text("note"),
231     actorId: integer("actor_id").references(() => users.id, { onDelete: "set null" }),
232     createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
233   },
234   (t) => [index("order_events_order_idx").on(t.orderId)],
235 );
236
237 /* ----- */
238 /* 6. quality_inspections */
239 /* ----- */
240 export const qualityInspections = pgTable(
241   "quality_inspections",
242   {
243     inspectionId: serial("inspection_id").primaryKey(),
244     cropId: integer("crop_id")
245       .notNull()
246       .references(() => cropsInventory.cropId, { onDelete: "cascade" }),
247     inspectorId: integer("inspector_id")
248       .notNull()
249       .references(() => users.id, { onDelete: "restrict" }),
250     gradeAssigned: qualityGradeEnum("grade_assigned").notNull(),
251     moistureLevelPercentage: numeric("moisture_level_percentage", { precision: 5, scale: 2, mode: "number"
252     }).notNull(),
252     soilPh: numeric("soil_ph", { precision: 4, scale: 2, mode: "number" }),
253     inspectionNotes: text("inspection_notes"),
254     reportAttachment: text("report_attachment"),
255     reportFileName: varchar("report_file_name", { length: 200 }),
256     certificateNo: varchar("certificate_no", { length: 40 }),
257     status: inspectionStatusEnum("status").notNull().default("pending"),
258     verifiedAt: timestamp("verified_at", { withTimezone: true }),
259   },
260   (t) => [index("inspections_crop_idx").on(t.cropId), index("inspections_status_idx").on(t.status)],
261 );
262
263 /* Escrow wallet ledger */
264 export const walletTransactions = pgTable(
265   "wallet_transactions",
266   {
267     txnId: serial("txn_id").primaryKey(),
268     userId: integer("user_id")
269       .notNull()
270       .references(() => users.id, { onDelete: "cascade" }),
271     orderId: integer("order_id").references(() => ordersLogistics.orderId, { onDelete: "set null" }),
272     type: walletTxnTypeEnum("type").notNull(),
273     amount: money("amount").notNull(),
274     description: text("description"),
275     createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
276   },
277   (t) => [index("wallet_user_idx").on(t.userId)],
278 );
279
280 /* Dispute resolution log */
281 export const disputes = pgTable(
282   "disputes",
283   {
284     disputeId: serial("dispute_id").primaryKey(),
285     orderId: integer("order_id")
286       .notNull()
287       .references(() => ordersLogistics.orderId, { onDelete: "cascade" }),
288     raisedBy: integer("raised_by")
289       .notNull()
290       .references(() => users.id, { onDelete: "restrict" }),
291     reason: text("reason").notNull(),

```

```

292     status: disputeStatusEnum("status").notNull().default("open"),
293     resolution: text("resolution"),
294     createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
295     resolvedAt: timestamp("resolved_at", { withTimezone: true }),
296   },
297   (t) => [index("disputes_order_idx").on(t.orderId), index("disputes_status_idx").on(t.status)],
298 );
299
300 /* ----- */
301 /* Reviews & seller trust */
302 /* ----- */
303 export const reviews = pgTable(
304   "reviews",
305   {
306     reviewId: serial("review_id").primaryKey(),
307     orderId: integer("order_id").notNull().references(() => ordersLogistics.orderId, { onDelete: "cascade"
308   ~   }},
309     buyerId: integer("buyer_id").notNull().references(() => users.id, { onDelete: "cascade" }),
310     farmerId: integer("farmer_id").notNull().references(() => users.id, { onDelete: "cascade" }),
311     rating: integer("rating").notNull(),
312     comment: text("comment"),
313     farmerReply: text("farmer_reply"),
314     repliedAt: timestamp("replied_at", { withTimezone: true }),
315     createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
316   },
317   (t) => [uniqueIndex("reviews_order_uq").on(t.orderId), index("reviews_farmer_idx").on(t.farmerId)],
318 );
319
320 /* In-app notifications + email/SMS delivery log */
321 export const notifications = pgTable(
322   "notifications",
323   {
324     notificationId: serial("notification_id").primaryKey(),
325     userId: integer("user_id").notNull().references(() => users.id, { onDelete: "cascade" }),
326     type: varchar("type", { length: 40 }).notNull(),
327     title: varchar("title", { length: 200 }).notNull(),
328     body: text("body"),
329     link: varchar("link", { length: 300 }),
330     channels: varchar("channels", { length: 60 }).notNull().default("in_app"),
331     isRead: boolean("is_read").notNull().default(false),
332     createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
333   },
334   (t) => [index("notifications_user_idx").on(t.userId, t.isRead)],
335 );
336
337 /* One-time codes: email verification & password reset */
338 export const authTokens = pgTable(
339   "auth_tokens",
340   {
341     tokenId: serial("token_id").primaryKey(),
342     userId: integer("user_id").notNull().references(() => users.id, { onDelete: "cascade" }),
343     purpose: varchar("purpose", { length: 20 }).notNull(), // verify_email | reset_password
344     codeHash: varchar("code_hash", { length: 128 }).notNull(),
345     attempts: integer("attempts").notNull().default(0),
346     expiresAt: timestamp("expires_at", { withTimezone: true }).notNull(),
347     usedAt: timestamp("used_at", { withTimezone: true }),
348     createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
349   },
350   (t) => [index("auth_tokens_user_idx").on(t.userId, t.purpose)],
351 );
352
353 /* Payment gateway intents (Stripe / JazzCash / Easypaisa / sandbox) */
354 export const paymentIntents = pgTable(
355   "payment_intents",
356   {
357     intentId: serial("intent_id").primaryKey(),
358     reference: varchar("reference", { length: 60 }).notNull(),
359     userId: integer("user_id").notNull().references(() => users.id, { onDelete: "cascade" }),
360     provider: varchar("provider", { length: 20 }).notNull(),
361     amount: money("amount").notNull(),
362     status: varchar("status", { length: 20 }).notNull().default("pending"), // pending | succeeded | failed
363     d | expired
364     providerRef: varchar("provider_ref", { length: 200 }),
365     rawCallback: text("raw_callback"),
366     createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
367     completedAt: timestamp("completed_at", { withTimezone: true }),

```

```

366     },
367     (t) => [uniqueIndex("payment_intents_ref_uq").on(t.reference), index("payment_intents_user_idx").on(t.us
    erId)],
368 );
369
370 /* Farmer payout (withdrawal) requests */
371 export const payoutRequests = pgTable(
372     "payout_requests",
373     {
374         payoutId: serial("payout_id").primaryKey(),
375         farmerId: integer("farmer_id").notNull().references(() => users.id, { onDelete: "cascade" }),
376         amount: money("amount").notNull(),
377         method: varchar("method", { length: 20 }).notNull(), // bank | jazzcash | easypaisa
378         accountTitle: varchar("account_title", { length: 120 }).notNull(),
379         accountNumber: varchar("account_number", { length: 60 }).notNull(),
380         status: varchar("status", { length: 20 }).notNull().default("pending"), // pending | paid | rejected
381         adminNote: text("admin_note"),
382         createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
383         processedAt: timestamp("processed_at", { withTimezone: true }),
384     },
385     (t) => [index("payout_requests_farmer_idx").on(t.farmerId), index("payout_requests_status_idx").on(t.sta
    tus)],
386 );
387
388 /* Audit trail for the admin DB explorer */
389 export const dbAuditLog = pgTable("db_audit_log", {
390     auditId: serial("audit_id").primaryKey(),
391     adminId: integer("admin_id").references(() => users.id, { onDelete: "set null" }),
392     mode: varchar("mode", { length: 10 }).notNull(), // read | write
393     query: text("query").notNull(),
394     rowCount: integer("row_count"),
395     durationMs: integer("duration_ms"),
396     error: text("error"),
397     createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
398 });
399
400 export type User = typeof users.$inferSelect;
401 export type Crop = typeof cropsInventory.$inferSelect;
402 export type MandiRate = typeof mandiRates.$inferSelect;
403 export type Bid = typeof bidsNegotiations.$inferSelect;
404 export type Order = typeof ordersLogistics.$inferSelect;
405 export type Inspection = typeof qualityInspections.$inferSelect;
406

```

src/lib/api.ts

```

1  export class ApiError extends Error {
2      status: number;
3      details?: unknown;
4      constructor(status: number, message: string, details?: unknown) {
5          super(message);
6          this.status = status;
7          this.details = details;
8      }
9  }
10
11 export function ok<T>(data: T, status = 200) {
12     return Response.json({ success: true, data }, { status });
13 }
14
15 export function fail(err: unknown) {
16     if (err instanceof ApiError) {
17         return Response.json({ success: false, error: err.message, details: err.details }, { status: err.statu
    s });
18     }
19     // Next.js redirect / notFound errors should bubble
20     if (err && typeof err === "object" && "digest" in err) throw err;
21     console.error("[api] unexpected error", err);
22     return Response.json({ success: false, error: "Internal server error" }, { status: 500 });
23 }
24
25 export async function handle<T>(fn: () => Promise<T>, status = 200) {
26     try {
27         const data = await fn();
28         return ok(data, status);
29     }
30 }

```

```

29     } catch (e) {
30         return fail(e);
31     }
32 }
33
34 export async function readJson(req: Request): Promise<Record<string, unknown>> {
35     try {
36         const body = await req.json();
37         if (!body || typeof body !== "object") throw new Error();
38         return body as Record<string, unknown>;
39     } catch {
40         throw new ApiError(400, "Invalid JSON body");
41     }
42 }
43
44 export function str(body: Record<string, unknown>, key: string, opts: { required?: boolean; max?: number }
45 ~ = {}) {
46     const v = body[key];
47     if (v === undefined || v === null || v === "") {
48         if (opts.required) throw new ApiError(422, `Field '${key}' is required`);
49         return undefined;
50     }
51     if (typeof v !== "string") throw new ApiError(422, `Field '${key}' must be a string`);
52     const t = v.trim();
53     if (opts.max && t.length > opts.max) throw new ApiError(422, `Field '${key}' exceeds ${opts.max} chars`);
54 ~ ;
55     return t;
56 }
57
58 export function num(body: Record<string, unknown>, key: string, opts: { required?: boolean; min?: number;
59 ~ max?: number } = {}) {
60     const v = body[key];
61     if (v === undefined || v === null || v === "") {
62         if (opts.required) throw new ApiError(422, `Field '${key}' is required`);
63         return undefined;
64     }
65     const n = typeof v === "number" ? v : Number(v);
66     if (!Number.isFinite(n)) throw new ApiError(422, `Field '${key}' must be a number`);
67     if (opts.min !== undefined && n < opts.min) throw new ApiError(422, `Field '${key}' must be ≥ ${opts.min
68 ~ ``);
69     if (opts.max !== undefined && n > opts.max) throw new ApiError(422, `Field '${key}' must be ≤ ${opts.max
70 ~ ``);
71     return n;
72 }
73
74 export function oneOf<T extends string>(body: Record<string, unknown>, key: string, values: readonly T[],
75 ~ required = false): T | undefined {
76     const v = body[key];
77     if (v === undefined || v === null || v === "") {
78         if (required) throw new ApiError(422, `Field '${key}' is required`);
79         return undefined;
80     }
81     if (typeof v !== "string" || !values.includes(v as T)) {
82         throw new ApiError(422, `Field '${key}' must be one of: ${values.join(", ")}`);
83     }
84     return v as T;
85 }
86
87 }
88
89 }

```

src/lib/auth.ts

```

1  import { cookies } from "next/headers";
2  import { redirect } from "next/navigation";
3  import { createHmac, timingSafeEqual } from "crypto";
4  import { eq } from "drizzle-orm";
5  import { db } from "@db";
6  import { users } from "@db/schema";
7  import type { Role } from "@lib/constants";
8  import { ApiError } from "@lib/api";
9  import { ensureSeeded } from "@lib/seed";
10 export { hashPassword, verifyPassword } from "@lib/password";
11
12 export const SESSION_COOKIE = "ml_session";
13 export const SESSION_MAX_AGE = 60 * 60 * 24 * 7;
14 const SECRET = process.env.SESSION_SECRET || "marketlink-agri-hub-dev-secret-change-me";

```

```

15  const ROLES: Role[] = ["farmer", "buyer", "admin", "inspector"];
16
17  function sign(value: string) {
18    return createHmac("sha256", SECRET).update(value).digest("hex");
19  }
20
21  /** Token format: `<userId>.<role>.<issuedAtMs>.<hmac>`; the role lets proxy.ts enforce RBAC before render
    ing. */
22  export function createSessionToken(userId: number, role: Role) {
23    const payload = `${userId}.${role}.${Date.now()}`;
24    return `${payload}.${sign(payload)}`;
25  }
26
27  export function parseSessionToken(token: string | undefined): { id: number; role: Role; issuedAt: number }
    | null {
28    if (!token) return null;
29    const parts = token.split(".");
30    if (parts.length !== 4) return null;
31    const [id, role, ts, sig] = parts;
32    const expected = sign(`${id}.${role}.${ts}`);
33    if (expected.length !== sig.length || !timingSafeEqual(Buffer.from(expected), Buffer.from(sig))) return
    null;
34    if (!ROLES.includes(role as Role)) return null;
35    if (Date.now() - Number(ts) > SESSION_MAX_AGE * 1000) return null;
36    return { id: Number(id), role: role as Role, issuedAt: Number(ts) };
37  }
38
39  export async function startSession(userId: number, role: Role) {
40    (await cookies()).set(SESSION_COOKIE, createSessionToken(userId, role), {
41      httpOnly: true,
42      sameSite: "lax",
43      path: "/",
44      maxAge: SESSION_MAX_AGE,
45      secure: process.env.COOKIE_SECURE === "true",
46    });
47  }
48
49  export type SessionUser = Omit<typeof users.$inferSelect, "passwordHash">;
50
51  export async function getCurrentUser(): Promise<SessionUser | null> {
52    await ensureSeeded();
53    const store = await cookies();
54    const session = parseSessionToken(store.get(SESSION_COOKIE)?.value);
55    if (!session) return null;
56    const [row] = await db.select().from(users).where(eq(users.id, session.id)).limit(1);
57    if (!row) return null;
58    // eslint-disable-next-line @typescript-eslint/no-unused-vars
59    const { passwordHash, ...safe } = row;
60    return safe;
61  }
62
63  /** For API route handlers: throws ApiError (401/403). Role is always re-checked against the database. */
64  export async function requireUser(roles?: Role[]): Promise<SessionUser> {
65    const user = await getCurrentUser();
66    if (!user) throw new ApiError(401, "Authentication required");
67    if (roles && !roles.includes(user.role)) {
68      throw new ApiError(403, roles.length === 1 && roles[0] === "admin" ? ADMIN_DENIED : `Requires role: ${
69        roles.join(" or ")`);
70    }
71    return user;
72  }
73
74  export const ADMIN_DENIED = "Access Denied: Admin privileges required";
75
76  /** For admin pages: anyone who isn't an admin is sent to /login with an access-denied alert. */
77  export async function requireAdminPage(): Promise<SessionUser> {
78    const user = await getCurrentUser();
79    if (!user || user.role !== "admin") redirect("/login?error=admin_required");
80    return user;
81  }
82
83  /** For server-rendered pages: redirects instead of throwing. */
84  export async function requirePageUser(roles?: Role[]): Promise<SessionUser> {
85    const user = await getCurrentUser();
86    if (!user) redirect("/login");
87    if (roles && !roles.includes(user.role)) redirect(homeFor(user.role));

```

```

87     return user;
88 }
89
90 export function homeFor(role: Role) {
91     return role === "farmer" ? "/farmer/dashboard" : role === "buyer" ? "/buyer/dashboard" : role === "inspe
92     ctor" ? "/inspector/dashboard" : "/admin/dashboard";
93 }

```

src/lib/base-url.ts

```

1  /**
2   * Public base-URL resolution – the single place that decides the absolute origin of this app.
3   *
4   * Why this exists: Next.js derives `request.url` from the socket the server listens on, so in a
5   * container/cloud sandbox the origin can come out as `http://0.0.0.0:3000`. Handing that to a
6   * browser produces ERR_ADDRESS_INVALID (the escrow checkout redirect bug). Every absolute URL
7   * that leaves the server for a browser – payment redirects, gateway return URLs, OAuth callbacks,
8   * email links – must therefore be built from, in order:
9   *
10  * 1. the host the visitor actually used (x-forwarded-host / host + x-forwarded-proto), or
11  * 2. a validated environment value (APP_URL, NEXT_PUBLIC_APP_URL, PUBLIC_URL, BASE_URL), or
12  * 3. a loopback fallback for local development.
13  *
14  * Wildcard/bind addresses (0.0.0.0, ::, *) are never used, and only well-formed absolute http(s)
15  * URLs are ever produced or returned to the browser.
16  *
17  * The module is environment-agnostic (safe to import from server and client components) and
18  * contains no Node.js dependencies.
19  */
20
21 export type OriginInfo = {
22     /** `https://host[:port]` */
23     origin: string;
24     /** Hostname without port, e.g. `app.example.com`, `127.0.0.1`, `[::1]` */
25     hostname: string;
26     protocol: "http" | "https";
27     /** A wildcard/unspecified address (0.0.0.0, ::) – never valid in a browser-facing URL. */
28     bind: boolean;
29     /** localhost / 127.0.0.0/8 / ::1 – only valid for the machine that runs the browser. */
30     loopback: boolean;
31     /** True when a browser on another device could plausibly open this host. */
32     routable: boolean;
33 };
34
35 /** Environment variables checked (in order) when a request host is unavailable or unusable. */
36 export const ENV_BASE_URL_KEYS = ["APP_URL", "NEXT_PUBLIC_APP_URL", "PUBLIC_URL", "BASE_URL"] as const;
37
38 const hasProcess = () => typeof process !== "undefined" && !!process.env;
39
40 const unbracket = (hostname: string) => hostname.trim().toLowerCase().replace(/^\[/, "").replace(/\]/, "");
41
42 /** Accepts either a hostname or `host[:port]` and returns the hostname part. */
43 function hostOnly(value: string): string {
44     const raw = (value ?? "").trim().toLowerCase();
45     if (!raw) return "";
46     if (raw.startsWith("[") {
47         const end = raw.indexOf("]");
48         return end === -1 ? raw.slice(1) : raw.slice(1, end);
49     }
50     const colon = raw.lastIndexOf(":");
51     if (colon !== -1 && colon === raw.indexOf(":") && /^d+$/.test(raw.slice(colon + 1))) return raw.slice(0
52     , colon);
53     return raw;
54 }
55
56 /** 0.0.0.0, ::, * – an interface bind address, never a real destination. */
57 export function isBindHost(hostname: string): boolean {
58     const h = hostOnly(hostname);
59     if (!h) return true;
60     if (h === "*" || h === "::" || h === "::0" || h === "0:0:0:0:0:0:0") return true;
61     // "0" and "0.0.0.0" are the same unspecified address (WHATWG URL normalises 0 → 0.0.0.0).
62     return h === "0" || /^0+(?:\.\d+){3}$/.test(h);
63 }

```

```

64  /** localhost / *.localhost / 127.0.0.0/8 / ::1 */
65  export function isLoopbackHost(hostname: string): boolean {
66      const h = hostOnly(hostname);
67      return h === "localhost" || h.endsWith(".localhost") || h === "::1" || /^127\./.test(h);
68  }
69
70  const isIpLiteral = (hostname: string) => hostname.startsWith("[") || /^d{1,3}(?:\.\d{1,3}){3}$/.test(unb
  ↵ racket(hostname));
71
72  /**
73   * Parse `host[:port]` or a full URL into a validated origin. Returns null for anything that is
74   * not an http(s) origin: bad syntax, credentials, paths, query strings, other schemes.
75   * Also protects against Host-header injection (a header must never smuggle a path or host list).
76   */
77  export function parseOriginInfo(candidate: string | null | undefined): OriginInfo | null {
78      const raw = (candidate ?? "").trim();
79      if (!raw || /\s@"'<>\\^{}]/.test(raw)) return null;
80      let url: URL;
81      try {
82          url = new URL(raw.includes("://") ? raw : `http://${raw}`);
83      } catch {
84          return null;
85      }
86      if (url.protocol !== "http:" && url.protocol !== "https:") return null;
87      if (url.username || url.password) return null;
88      if (url.pathname !== "/" || url.search || url.hash) return null;
89      if (!url.hostname) return null;
90      const bind = isBindHost(url.hostname);
91      const loopback = !bind && isLoopbackHost(url.hostname);
92      const hostname = url.hostname;
93      return {
94          origin: `${url.protocol}/${url.host}`,
95          hostname,
96          protocol: url.protocol as "http:" | "https:",
97          bind,
98          loopback,
99          routable: !bind && !loopback && (isIpLiteral(hostname) || unbracket(hostname).includes(".")),
100      };
101  }
102
103  /** Value of the first hop of a comma-separated proxy header. */
104  const firstHop = (value: string | null) => (value ?? "").split(",")[0].trim();
105
106  /**
107   * Which scheme can the visitor actually use?
108   *
109   * A proxy that terminates TLS reports `https`. Careful with the opposite case: a proxy that talks
110   * plain HTTP to the app (Next.js itself fills `x-forwarded-proto` in from the socket it listens on)
111   * reports `http` even though the browser is on https. So `http` is trusted only when the host
112   * carries an explicit non-default port (LAN/tunnel/dev, e.g. `192.168.1.5:3000`); a bare public
113   * hostname is kept on https. An http-only public deployment should set APP_URL.
114   */
115  function resolveProtocol(forwardedProto: string, info: OriginInfo, explicitNonDefaultPort: boolean): "http
  ↵ " | "https:" {
116      if (forwardedProto === "https") return "https:";
117      if (forwardedProto === "http") return explicitNonDefaultPort || !info.routable ? "http:" : "https:";
118      return info.loopback || explicitNonDefaultPort ? "http:" : "https:";
119  }
120
121  /**
122   * The origin the visitor actually used, from x-forwarded-host/x-forwarded-proto (falling back to
123   * host). The client port is taken from the host header itself: proxies may also send
124   * `x-forwarded-port` with their internal port (Next.js fills it in from the socket it listens on),
125   * and appending that to a public hostname produces URLs such as `https://app.example.com:3000`
126   * that no browser can reach.
127   */
128  export function requestOriginInfo(req: Request): OriginInfo | null {
129      let headers: Headers;
130      try {
131          headers = req.headers;
132      } catch {
133          return null;
134      }
135      const host = firstHop(headers.get("x-forwarded-host")) || firstHop(headers.get("host"));
136      const info = parseOriginInfo(host);
137      if (!info) return null;

```

```

138
139     const authority = new URL(info.origin).host;
140     const protocol = resolveProtocol(firstHop(headers.get("x-forwarded-proto")).toLowerCase(), info, new URL
141     (info.origin).port !== "");
142
143     return { ...info, origin: `${protocol}://${authority}`, protocol };
144 }
145
146 /** First usable configured origin. Bind addresses and malformed values are ignored. */
147 export function envOriginInfo(): OriginInfo | null {
148     if (!hasProcess()) return null;
149     for (const key of ENV_BASE_URL_KEYS) {
150         const info = parseOriginInfo(process.env[key]);
151         if (info && !info.bind) return info;
152     }
153     return null;
154 }
155
156 /** Case-insensitive hostname comparison (IPv6 brackets ignored). */
157 function sameHostname(a: string, b: string): boolean {
158     return !!a && !!b && unbracket(a).toLowerCase() === unbracket(b).toLowerCase();
159 }
160
161 /* The last public origin seen on a request: lets background work (emails, receipts) build links
162 * even when no APP_URL is configured. Only public hosts are remembered, never loopback/bind. */
163 let observedOrigin: string | null = null;
164
165 function remember(origin: string): string {
166     observedOrigin = origin;
167     return origin;
168 }
169
170 /** Last public origin seen by this server process (null before the first such request). */
171 export function observedPublicOrigin(): string | null {
172     return observedOrigin;
173 }
174
175 export function forgetObservedPublicOrigin() {
176     observedOrigin = null;
177 }
178
179 /**
180 * The absolute origin of this app for browser-facing URLs.
181 * Never returns `0.0.0.0`, `*`, `::` or an otherwise malformed URL.
182 */
183 export function publicOrigin(req?: Request | null): string {
184     const fromRequest = req ? requestOriginInfo(req) : null;
185     const fromEnv = envOriginInfo();
186
187     if (fromRequest?.routable) {
188         // A configured APP_URL for this very hostname wins: it carries the deployment's real scheme
189         // and port (e.g. an http-only domain). A stale/other APP_URL never overrides the live host.
190         if (fromEnv && !fromEnv.loopback && sameHostname(fromEnv.hostname, fromRequest.hostname)) return remember
191         (fromEnv.origin);
192         return remember(fromRequest.origin);
193     }
194
195     if (fromEnv && !fromEnv.loopback) return remember(fromEnv.origin);
196     if (fromEnv) return fromEnv.origin;
197
198     // Local development: keep the loopback host the developer actually typed (e.g. 127.0.0.1:8080).
199     // Single-label internal names (docker service names such as `app:3000`) are skipped: a browser
200     // elsewhere cannot resolve them – set APP_URL when a proxy rewrites the Host header that way.
201     if (fromRequest && !fromRequest.bind && fromRequest.loopback) return fromRequest.origin;
202
203     // Background work (notification emails, receipts) has no request: reuse the last public origin
204     // this server served, so links still match the live domain.
205     if (observedOrigin) return observedOrigin;
206
207     const port = hasProcess() ? process.env.PORT || "3000" : "3000";
208     return `http://localhost:${port}`;
209 }
210
211 /** Join a path to the resolved public origin, e.g. absoluteUrl(req, "/wallet?payment=MLT1"). */
212 export function absoluteUrl(req: Request | null | undefined, path: string): string {
213     const suffix = path.startsWith("/") ? path : `/${path}`;

```

```

212     return `${publicOrigin(req)}${suffix}`;
213 }
214
215 /** True for a well-formed absolute http(s) URL. */
216 export function isValidHttpUrl(value: unknown): value is string {
217     if (typeof value !== "string" || !value.trim()) return false;
218     try {
219         const url = new URL(value.trim());
220         return (url.protocol === "http:" || url.protocol === "https:") && !!url.hostname;
221     } catch {
222         return false;
223     }
224 }
225
226 export type RedirectResolution = { ok: true; url: string; rewritten: boolean } | { ok: false; error: string };
227
228 /**
229  * Browser-side guard for redirect/form URLs coming back from the API.
230  *
231  * Validates the scheme, and if a bind/loopback host is returned while the page itself is served
232  * from a public origin, the URL is re-pointed at `window.location.origin` (same path, so e.g.
233  * `http://0.0.0.0:3000/pay/sandbox/MLT1` becomes `https://3000-abc.e2b.app/pay/sandbox/MLT1`)
234  * instead of failing with ERR_ADDRESS_INVALID. Third-party gateway URLs (Stripe, JazzCash,
235  * Easypaisa) are never rewritten.
236  */
237 export function resolveBrowserRedirect(raw: unknown, pageOrigin: string): RedirectResolution {
238     if (typeof raw !== "string" || !raw.trim()) return { ok: false, error: "The payment gateway did not return a checkout URL. Please try again." };
239     let target: URL;
240     try {
241         target = new URL(raw.trim(), pageOrigin);
242     } catch {
243         return { ok: false, error: "The payment gateway returned a malformed checkout URL. Please try again." };
244     }
245     if (target.protocol !== "http:" && target.protocol !== "https:") {
246         return { ok: false, error: `Refused to open an unsupported link (${target.protocol}) from the payment gateway.` };
247     }
248
249     let page: URL | null = null;
250     try {
251         page = new URL(pageOrigin);
252     } catch {
253         page = null;
254     }
255     const pageUsable = !!page && /^https?:$/.test(page.protocol) && !!page.hostname && !isBindHost(page.hostname);
256     const targetUnusable = isBindHost(target.hostname) || (isLoopbackHost(target.hostname) && pageUsable && !isLoopbackHost(page!.hostname));
257
258     if (!targetUnusable) return { ok: true, url: target.toString(), rewritten: false };
259     if (!pageUsable) return { ok: false, error: "The site could not determine its own public address. Reload the page and try again." };
260
261     target.protocol = page!.protocol;
262     target.hostname = page!.hostname;
263     target.port = page!.port;
264     return { ok: true, url: target.toString(), rewritten: true };
265 }
266

```

src/lib/constants.ts

```

1  export type Role = "farmer" | "buyer" | "admin" | "inspector";
2  export type Category = "grains" | "vegetables" | "fruits" | "cash_crops";
3  export type Grade = "A" | "B" | "C";
4  export type DeliveryStage = "confirmed" | "quality_checked" | "dispatched" | "in_transit" | "delivered";
5  export type PaymentStatus = "awaiting_escrow" | "escrow_locked" | "released" | "refunded" | "disputed";
6
7  export const PLATFORM_FEE_RATE = 0.015; // 1.5% commission deducted on escrow release
8  export const MAUND_KG = 40; // 1 Mann (maund) = 40 kg – standard Pakistani mandi unit
9  export const CURRENCY = "PKR";
10

```

```

11 export type Province = "Punjab" | "Sindh" | "Khyber Pakhtunkhwa" | "Balochistan" | "Islamabad (ICT)" | "Gilgit-Baltistan" | "Azad Kashmir";
12
13 export const CITIES: Record<string, { lat: number; lng: number; province: Province; urdu: string }> = {
14   // Punjab
15   Lahore: { lat: 31.5204, lng: 74.3587, province: "Punjab", urdu: "[Urdu]" },
16   Faisalabad: { lat: 31.4504, lng: 73.135, province: "Punjab", urdu: "[Urdu]" },
17   Rawalpindi: { lat: 33.5651, lng: 73.0169, province: "Punjab", urdu: "[Urdu]" },
18   Multan: { lat: 30.1575, lng: 71.5249, province: "Punjab", urdu: "[Urdu]" },
19   Gujranwala: { lat: 32.1877, lng: 74.1945, province: "Punjab", urdu: "[Urdu]" },
20   Sialkot: { lat: 32.4945, lng: 74.5229, province: "Punjab", urdu: "[Urdu]" },
21   Bahawalpur: { lat: 29.3544, lng: 71.6911, province: "Punjab", urdu: "[Urdu]" },
22   Sargodha: { lat: 32.074, lng: 72.6861, province: "Punjab", urdu: "[Urdu]" },
23   Sahiwal: { lat: 30.6682, lng: 73.1114, province: "Punjab", urdu: "[Urdu]" },
24   Okara: { lat: 30.8138, lng: 73.4534, province: "Punjab", urdu: "[Urdu]" },
25   "Rahim Yar Khan": { lat: 28.4212, lng: 70.2989, province: "Punjab", urdu: "[Urdu]" },
26   Sheikhupura: { lat: 31.7131, lng: 73.9783, province: "Punjab", urdu: "[Urdu]" },
27   Kasur: { lat: 31.1187, lng: 74.4507, province: "Punjab", urdu: "[Urdu]" },
28   Jhang: { lat: 31.2681, lng: 72.3181, province: "Punjab", urdu: "[Urdu]" },
29   Vehari: { lat: 30.0452, lng: 72.3489, province: "Punjab", urdu: "[Urdu]" },
30   Khanewal: { lat: 30.3017, lng: 71.9321, province: "Punjab", urdu: "[Urdu]" },
31   "Dera Ghazi Khan": { lat: 30.0459, lng: 70.6403, province: "Punjab", urdu: "[Urdu]" },
32   "Toba Tek Singh": { lat: 30.9709, lng: 72.4826, province: "Punjab", urdu: "[Urdu]" },
33   // Sindh
34   Karachi: { lat: 24.8607, lng: 67.0011, province: "Sindh", urdu: "[Urdu]" },
35   Hyderabad: { lat: 25.396, lng: 68.3578, province: "Sindh", urdu: "[Urdu]" },
36   Sukkur: { lat: 27.7052, lng: 68.8574, province: "Sindh", urdu: "[Urdu]" },
37   Larkana: { lat: 27.557, lng: 68.2264, province: "Sindh", urdu: "[Urdu]" },
38   "Mirpur Khas": { lat: 25.5276, lng: 69.0111, province: "Sindh", urdu: "[Urdu]" },
39   Nawabshah: { lat: 26.2442, lng: 68.41, province: "Sindh", urdu: "[Urdu]" },
40   Umerkot: { lat: 25.3616, lng: 69.7362, province: "Sindh", urdu: "[Urdu]" },
41   Badin: { lat: 24.656, lng: 68.837, province: "Sindh", urdu: "[Urdu]" },
42   // Khyber Pakhtunkhwa
43   Peshawar: { lat: 34.0151, lng: 71.5249, province: "Khyber Pakhtunkhwa", urdu: "[Urdu]" },
44   Mardan: { lat: 34.1986, lng: 72.0404, province: "Khyber Pakhtunkhwa", urdu: "[Urdu]" },
45   Swat: { lat: 34.7717, lng: 72.36, province: "Khyber Pakhtunkhwa", urdu: "[Urdu]" },
46   "Dera Ismail Khan": { lat: 31.8314, lng: 70.9019, province: "Khyber Pakhtunkhwa", urdu: "[Urdu]" },
47   Charsadda: { lat: 34.1482, lng: 71.7406, province: "Khyber Pakhtunkhwa", urdu: "[Urdu]" },
48   Abbottabad: { lat: 34.1688, lng: 73.2215, province: "Khyber Pakhtunkhwa", urdu: "[Urdu]" },
49   // Balochistan
50   Quetta: { lat: 30.1798, lng: 66.975, province: "Balochistan", urdu: "[Urdu]" },
51   Turbat: { lat: 26.0023, lng: 63.044, province: "Balochistan", urdu: "[Urdu]" },
52   "Dera Murad Jamali": { lat: 28.546, lng: 68.223, province: "Balochistan", urdu: "[Urdu]" },
53   // Federal / North
54   Islamabad: { lat: 33.6844, lng: 73.0479, province: "Islamabad (ICT)", urdu: "[Urdu]" },
55   Gilgit: { lat: 35.9208, lng: 74.308, province: "Gilgit-Baltistan", urdu: "[Urdu]" },
56   Mirpur: { lat: 33.1484, lng: 73.7518, province: "Azad Kashmir", urdu: "[Urdu]" },
57 };
58 export const CITY_NAMES = Object.keys(CITIES);
59 export const PROVINCES: Province[] = ["Punjab", "Sindh", "Khyber Pakhtunkhwa", "Balochistan", "Islamabad (ICT)", "Gilgit-Baltistan", "Azad Kashmir"];
60
61 /** Wholesale mandis (market key = city) with their real market names. */
62 export const MANDIS: { key: string; name: string; urdu: string }[] = [
63   { key: "Lahore", name: "Badami Bagh Mandi, Lahore", urdu: "[Urdu]" },
64   { key: "Karachi", name: "Sabzi Mandi Super Highway, Karachi", urdu: "[Urdu]" },
65   { key: "Multan", name: "Ghalla Mandi, Multan", urdu: "[Urdu]" },
66   { key: "Faisalabad", name: "Grain Market, Faisalabad", urdu: "[Urdu]" },
67   { key: "Islamabad", name: "I-11 Sabzi Mandi, Islamabad", urdu: "[Urdu]" },
68   { key: "Hyderabad", name: "Sabzi Mandi, Hyderabad", urdu: "[Urdu]" },
69   { key: "Peshawar", name: "Ghalla Mandi, Peshawar", urdu: "[Urdu]" },
70   { key: "Quetta", name: "Fruit Mandi, Quetta", urdu: "[Urdu]" },
71   { key: "Sukkur", name: "Grain Market, Sukkur", urdu: "[Urdu]" },
72   { key: "Gujranwala", name: "Rice Market, Gujranwala", urdu: "[Urdu]" },
73 ];
74 export const MANDI_MARKETS = MANDIS.map((m) => m.key);
75 export const mandiName = (key: string) => MANDIS.find((m) => m.key === key)?.name ?? key;
76
77 export const CATEGORIES: { value: Category; label: string; urdu: string; emoji: string }[] = [
78   { value: "grains", label: "Grains & Pulses", urdu: "[Urdu]", emoji: "" },
79   { value: "vegetables", label: "Vegetables", urdu: "[Urdu]", emoji: "" },
80   { value: "fruits", label: "Fruits", urdu: "[Urdu]", emoji: "" },
81   { value: "cash_crops", label: "Cotton / Sugarcane", urdu: "[Urdu]", emoji: "" },
82 ];
83
84 const px = (id: number) => `https://images.pexels.com/photos/${id}/pexels-photo-${id}.jpeg?auto=compress&c

```

```

85 s=tinysrgb&fit=crop&h=627&w=1200`;
86 /** Base prices are PKR per kg, calibrated to typical Pakistani mandi rates. */
87 export const CROP_CATALOGUE: { name: string; urdu: string; category: Category; mandiBase: number; image: string }[] = [
88   { name: "Wheat", urdu: "[Urdu]", category: "grains", mandiBase: 95, image: px(38129076) },
89   { name: "Basmati Rice", urdu: "[Urdu]", category: "grains", mandiBase: 290, image: px(7421207) },
90   { name: "Maize", urdu: "[Urdu]", category: "grains", mandiBase: 70, image: px(8108170) },
91   { name: "Chickpea (Chana)", urdu: "[Urdu]", category: "grains", mandiBase: 240, image: px(10111952) },
92   { name: "Tomato", urdu: "[Urdu]", category: "vegetables", mandiBase: 120, image: px(37408649) },
93   { name: "Potato", urdu: "[Urdu]", category: "vegetables", mandiBase: 55, image: px(37091858) },
94   { name: "Onion", urdu: "[Urdu]", category: "vegetables", mandiBase: 110, image: px(37091856) },
95   { name: "Mango (Chaunsa)", urdu: "[Urdu]", category: "fruits", mandiBase: 180, image: px(30893290) },
96   { name: "Kinnow", urdu: "[Urdu]", category: "fruits", mandiBase: 90, image: px(14766928) },
97   { name: "Dates (Khajoor)", urdu: "[Urdu]", category: "fruits", mandiBase: 260, image: px(17877978) },
98   { name: "Cotton (Phutti)", urdu: "[Urdu]", category: "cash_crops", mandiBase: 210, image: px(5640079) },
99   { name: "Sugarcane", urdu: "[Urdu]", category: "cash_crops", mandiBase: 11, image: px(9622985) },
100  { name: "Red Chillli (Kunri)", urdu: "[Urdu]", category: "cash_crops", mandiBase: 480, image: px(6902055) },
101 ],
102
103 export const FALLBACK_IMAGE: Record<Category, string> = {
104   grains: CROP_CATALOGUE[0].image,
105   vegetables: CROP_CATALOGUE[5].image,
106   fruits: CROP_CATALOGUE[7].image,
107   cash_crops: CROP_CATALOGUE[10].image,
108 };
109
110 export function cropImage(images: string[] | null | undefined, cropName: string, category: Category) {
111   if (images && images.length > 0) return images[0];
112   const c = CROP_CATALOGUE.find((x) => x.name.toLowerCase() === cropName.toLowerCase());
113   return c?.image ?? FALLBACK_IMAGE[category];
114 }
115
116 export const cropUrdu = (name: string) => CROP_CATALOGUE.find((c) => c.name === name)?.urdu ?? "";
117
118 export const DELIVERY_STAGES: { key: DeliveryStage; label: string; short: string; actor: Role }[] = [
119   { key: "confirmed", label: "Order Placed", short: "Placed", actor: "farmer" },
120   { key: "quality_checked", label: "Quality Inspected", short: "Inspected", actor: "admin" },
121   { key: "dispatched", label: "Dispatched", short: "Dispatched", actor: "farmer" },
122   { key: "in_transit", label: "In Transit", short: "Transit", actor: "farmer" },
123   { key: "delivered", label: "Delivered", short: "Delivered", actor: "buyer" },
124 ];
125
126 export function stageIndex(stage: DeliveryStage) {
127   return DELIVERY_STAGES.findIndex((s) => s.key === stage);
128 }
129
130 export function nextStage(stage: DeliveryStage): DeliveryStage | null {
131   const i = stageIndex(stage);
132   return i >= 0 && i < DELIVERY_STAGES.length - 1 ? DELIVERY_STAGES[i + 1].key : null;
133 }
134
135 export function distanceKm(a: { lat: number; lng: number }, b: { lat: number; lng: number }) {
136   const R = 6371;
137   const dLat = ((b.lat - a.lat) * Math.PI) / 180;
138   const dLng = ((b.lng - a.lng) * Math.PI) / 180;
139   const s =
140     Math.sin(dLat / 2) ** 2 +
141     Math.cos((a.lat * Math.PI) / 180) * Math.cos((b.lat * Math.PI) / 180) * Math.sin(dLng / 2) ** 2;
142   return 2 * R * Math.asin(Math.sqrt(s));
143 }
144
145 /** PKR with Pakistani thousands grouping, e.g. "Rs. 1,250,000". */
146 export function formatPKR(n: number | null | undefined, digits = 0) {
147   const v = Number(n ?? 0);
148   return "Rs. " + v.toLocaleString("en-US", { minimumFractionDigits: digits, maximumFractionDigits: digits });
149 }
150
151 /** Compact PKR in Lakh / Crore (common in Pakistan), e.g. "Rs. 12.5 Lakh", "Rs. 1.78 Crore". */
152 export function formatPKRShort(n: number | null | undefined) {
153   const v = Number(n ?? 0);
154   if (Math.abs(v) >= 1e7) return `Rs. ${v / 1e7}.toFixed(2)} Crore`;
155   if (Math.abs(v) >= 1e5) return `Rs. ${v / 1e5}.toFixed(1)} Lakh`;
156   return formatPKR(v);

```

```

157 }
158
159 /** Price per maund (40 kg) – the unit Pakistani farmers and artists quote. */
160 export function perMaund(perKg: number | null | undefined) {
161   return formatPKR(Number(perKg ?? 0) * MAUND_KG) + "/maund";
162 }
163
164 export function formatKg(n: number | null | undefined) {
165   const v = Number(n ?? 0);
166   if (v >= 1000) return (v / 1000).toLocaleString("en-US", { maximumFractionDigits: 2 }) + " ton";
167   return v.toLocaleString("en-US", { maximumFractionDigits: 0 }) + " kg";
168 }
169
170 export function formatMaunds(n: number | null | undefined) {
171   return Math.round(Number(n ?? 0) / MAUND_KG).toLocaleString("en-US") + " maund";
172 }
173
174 export function formatDate(d: string | Date | null | undefined) {
175   if (!d) return "-";
176   const dt = typeof d === "string" ? new Date(d) : d;
177   return dt.toLocaleDateString("en-GB", { day: "2-digit", month: "short", year: "numeric", timeZone: "Asia
178     Karachi" });
179 }
180
181 export function todayISO(offsetDays = 0) {
182   const d = new Date(Date.now() + 5 * 3600 * 1000); // Pakistan Standard Time (UTC+5)
183   d.setUTCDate(d.getUTCDate() + offsetDays);
184   return d.toISOString().slice(0, 10);
185 }
186
187 export const categoryLabel = (c: string) => CATEGORIES.find((x) => x.value === c)?.label ?? c;
188
189 /** Human-readable status labels used by <Badge>. */
190 export const STATUS_LABELS: Record<string, string> = {
191   pending: "Pending",
192   accepted: "Accepted",
193   rejected: "Rejected",
194   countered: "Countered",
195   withdrawn: "Withdrawn",
196   active: "Active",
197   pending_inspection: "Pending inspection",
198   sold_out: "Sold out",
199   archived: "Archived",
200   awaiting_escrow: "Awaiting escrow",
201   escrow_locked: "In escrow",
202   released: "Paid out",
203   refunded: "Refunded",
204   disputed: "Disputed",
205   passed: "Passed",
206   failed: "Failed",
207   open: "Open",
208   investigating: "Investigating",
209   resolved_release: "Resolved · paid",
210   resolved_refund: "Resolved · refunded",
211   confirmed: "Placed",
212   quality_checked: "Inspected",
213   dispatched: "Dispatched",
214   in_transit: "In transit",
215   delivered: "Delivered",
216 };
217
218 export const STATUS_STYLES: Record<string, string> = {
219   pending: "bg-amber-100 text-amber-800 dark:bg-amber-500/15 dark:text-amber-300",
220   accepted: "bg-emerald-100 text-emerald-800 dark:bg-emerald-500/15 dark:text-emerald-300",
221   rejected: "bg-rose-100 text-rose-800 dark:bg-rose-500/15 dark:text-rose-300",
222   countered: "bg-sky-100 text-sky-800 dark:bg-sky-500/15 dark:text-sky-300",
223   withdrawn: "bg-slate-200 text-slate-700 dark:bg-slate-500/20 dark:text-slate-300",
224   active: "bg-emerald-100 text-emerald-800 dark:bg-emerald-500/15 dark:text-emerald-300",
225   pending_inspection: "bg-amber-100 text-amber-800 dark:bg-amber-500/15 dark:text-amber-300",
226   sold_out: "bg-slate-200 text-slate-700 dark:bg-slate-500/20 dark:text-slate-300",
227   archived: "bg-slate-200 text-slate-700 dark:bg-slate-500/20 dark:text-slate-300",
228   awaiting_escrow: "bg-amber-100 text-amber-800 dark:bg-amber-500/15 dark:text-amber-300",
229   escrow_locked: "bg-sky-100 text-sky-800 dark:bg-sky-500/15 dark:text-sky-300",
230   released: "bg-emerald-100 text-emerald-800 dark:bg-emerald-500/15 dark:text-emerald-300",
231   refunded: "bg-violet-100 text-violet-800 dark:bg-violet-500/15 dark:text-violet-300",
232   disputed: "bg-rose-100 text-rose-800 dark:bg-rose-500/15 dark:text-rose-300",

```

```

232 passed: "bg-emerald-100 text-emerald-800 dark:bg-emerald-500/15 dark:text-emerald-300",
233 failed: "bg-rose-100 text-rose-800 dark:bg-rose-500/15 dark:text-rose-300",
234 open: "bg-rose-100 text-rose-800 dark:bg-rose-500/15 dark:text-rose-300",
235 investigating: "bg-amber-100 text-amber-800 dark:bg-amber-500/15 dark:text-amber-300",
236 resolved_release: "bg-emerald-100 text-emerald-800 dark:bg-emerald-500/15 dark:text-emerald-300",
237 resolved_refund: "bg-violet-100 text-violet-800 dark:bg-violet-500/15 dark:text-violet-300",
238 confirmed: "bg-slate-100 text-slate-700 dark:bg-slate-500/15 dark:text-slate-300",
239 quality_checked: "bg-teal-100 text-teal-800 dark:bg-teal-500/15 dark:text-teal-300",
240 dispatched: "bg-indigo-100 text-indigo-800 dark:bg-indigo-500/15 dark:text-indigo-300",
241 in_transit: "bg-sky-100 text-sky-800 dark:bg-sky-500/15 dark:text-sky-300",
242 delivered: "bg-emerald-100 text-emerald-800 dark:bg-emerald-500/15 dark:text-emerald-300",
243 };
244

```

src/lib/docs-content.ts

```

1  export const MYSQL_DDL = `-- MarketLink Agri-Hub · MySQL 8.0 compatible DDL (InnoDB, utf8mb4)
2  -- Production runtime uses PostgreSQL (see PostgreSQL tab); this script is the portable equivalent.
3
4  CREATE TABLE users (
5      id                INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
6      full_name         VARCHAR(120)  NOT NULL,
7      email             VARCHAR(160)  NOT NULL,
8      password_hash     VARCHAR(255)  NOT NULL,
9      phone             VARCHAR(30)   NOT NULL,
10     role               ENUM('farmer','buyer','admin','inspector') NOT NULL,
11     cnic_id            VARCHAR(20)   NULL,
12     business_name      VARCHAR(160)  NULL,
13     city               VARCHAR(80)   NOT NULL,
14     address            TEXT          NULL,
15     is_verified        BOOLEAN       NOT NULL DEFAULT FALSE,
16     email_verified     BOOLEAN       NOT NULL DEFAULT FALSE,
17     title              VARCHAR(120)  NULL,          -- e.g. 'Super Admin & Systems Controller'
18     user_code          VARCHAR(40)   NULL,          -- e.g. 'admin_01'
19     avatar_url         TEXT          NULL,
20     google_id          VARCHAR(64)   NULL,          -- Google OAuth subject
21     auth_provider      VARCHAR(20)   NOT NULL DEFAULT 'password',
22     trust_score        DECIMAL(3,2)  NOT NULL DEFAULT 0,
23     rating_avg         DECIMAL(3,2)  NOT NULL DEFAULT 0,
24     rating_count       INT            NOT NULL DEFAULT 0,
25     wallet_balance     DECIMAL(14,2) NOT NULL DEFAULT 0,
26     escrow_balance     DECIMAL(14,2) NOT NULL DEFAULT 0,
27     created_at         TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
28     UNIQUE KEY users_email_uq (email),
29     UNIQUE KEY users_google_id_uq (google_id),
30     UNIQUE KEY users_user_code_uq (user_code),
31     KEY users_role_idx (role),
32     KEY users_city_idx (city)
33 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
34
35 CREATE TABLE crops_inventory (
36     crop_id            INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
37     farmer_id          INT UNSIGNED NOT NULL,
38     crop_name          VARCHAR(80)   NOT NULL,
39     category           ENUM('grains','vegetables','fruits','cash_crops') NOT NULL,
40     quality_grade      ENUM('A','B','C') NOT NULL,
41     total_quantity_kg  DECIMAL(14,2) NOT NULL CHECK (total_quantity_kg >= 0),
42     base_price_per_kg  DECIMAL(14,2) NOT NULL CHECK (base_price_per_kg > 0),
43     harvest_date       DATE          NOT NULL,
44     images_json        JSON          NOT NULL,
45     description        TEXT          NULL,
46     farm_location      VARCHAR(80)   NOT NULL,
47     latitude           DOUBLE        NULL,
48     longitude          DOUBLE        NULL,
49     status             ENUM('active','pending_inspection','sold_out','archived') NOT NULL DEFAULT 'active',
50     created_at         TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
51     KEY crops_farmer_idx (farmer_id),
52     KEY crops_category_idx (category),
53     KEY crops_status_idx (status),
54     KEY crops_name_idx (crop_name),
55     CONSTRAINT fk_crops_farmer FOREIGN KEY (farmer_id) REFERENCES users(id) ON DELETE CASCADE
56 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
57
58 CREATE TABLE mandi_rates (
59     rate_id            INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,

```

```

60 crop_name          VARCHAR(80)    NOT NULL,
61 market_location   VARCHAR(80)    NOT NULL,
62 min_price_per_kg   DECIMAL(14,2) NOT NULL,
63 max_price_per_kg   DECIMAL(14,2) NOT NULL,
64 avg_price_per_kg   DECIMAL(14,2) NOT NULL,
65 source             VARCHAR(20)    NOT NULL DEFAULT 'manual',
66 date_updated       DATE            NOT NULL,
67 UNIQUE KEY mandi_crop_market_date_uq (crop_name, market_location, date_updated),
68 KEY mandi_date_idx (date_updated),
69 CHECK (min_price_per_kg <= avg_price_per_kg AND avg_price_per_kg <= max_price_per_kg)
70 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
71
72 CREATE TABLE bids_negotiations (
73 bid_id              INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
74 crop_id             INT UNSIGNED NOT NULL,
75 buyer_id           INT UNSIGNED NOT NULL,
76 bid_price_per_kg    DECIMAL(14,2) NOT NULL,
77 bid_quantity_kg     DECIMAL(14,2) NOT NULL,
78 target_delivery_date DATE          NULL,
79 message             TEXT           NULL,
80 status              ENUM('pending','accepted','rejected','countered','withdrawn') NOT NULL DEFAULT 'pending',
81 counter_price        DECIMAL(14,2) NULL,
82 counter_note         TEXT          NULL,
83 created_at           TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
84 updated_at           TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
85 KEY bids_crop_idx (crop_id),
86 KEY bids_buyer_idx (buyer_id),
87 KEY bids_status_idx (status),
88 CONSTRAINT fk_bids_crop FOREIGN KEY (crop_id) REFERENCES crops_inventory(crop_id) ON DELETE CASCADE,
89 CONSTRAINT fk_bids_buyer FOREIGN KEY (buyer_id) REFERENCES users(id) ON DELETE CASCADE
90 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
91
92 CREATE TABLE orders_logistics (
93 order_id            INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
94 bid_id              INT UNSIGNED NOT NULL,
95 crop_id             INT UNSIGNED NOT NULL,
96 farmer_id           INT UNSIGNED NOT NULL,
97 buyer_id           INT UNSIGNED NOT NULL,
98 quantity_kg         DECIMAL(14,2) NOT NULL,
99 price_per_kg        DECIMAL(14,2) NOT NULL,
100 total_amount        DECIMAL(14,2) NOT NULL,
101 payment_status       ENUM('awaiting_escrow','escrow_locked','released','refunded','disputed') NOT NULL
102 DEFAULT 'awaiting_escrow',
103 delivery_stage       ENUM('confirmed','quality_checked','dispatched','in_transit','delivered') NOT NULL
104 DEFAULT 'confirmed',
105 transit_location     VARCHAR(120) NULL,
106 current_lat          DOUBLE       NULL,
107 current_lng          DOUBLE       NULL,
108 tracking_number      VARCHAR(40)  NOT NULL,
109 expected_delivery_date DATE       NULL,
110 created_at           TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
111 updated_at           TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
112 UNIQUE KEY orders_bid_uq (bid_id),
113 UNIQUE KEY orders_tracking_uq (tracking_number),
114 KEY orders_farmer_idx (farmer_id),
115 KEY orders_buyer_idx (buyer_id),
116 KEY orders_stage_idx (delivery_stage),
117 CONSTRAINT fk_orders_bid FOREIGN KEY (bid_id) REFERENCES bids_negotiations(bid_id),
118 CONSTRAINT fk_orders_crop FOREIGN KEY (crop_id) REFERENCES crops_inventory(crop_id),
119 CONSTRAINT fk_orders_farmer FOREIGN KEY (farmer_id) REFERENCES users(id),
120 CONSTRAINT fk_orders_buyer FOREIGN KEY (buyer_id) REFERENCES users(id)
121 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
122
123 CREATE TABLE quality_inspections (
124 inspection_id        INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
125 crop_id             INT UNSIGNED NOT NULL,
126 inspector_id         INT UNSIGNED NOT NULL,
127 grade_assigned       ENUM('A','B','C') NOT NULL,
128 moisture_level_percentage DECIMAL(5,2) NOT NULL,
129 soil_ph              DECIMAL(4,2) NULL,
130 inspection_notes     TEXT          NULL,
131 report_attachment    LONGTEXT     NULL,
132 report_file_name     VARCHAR(200) NULL,
133 certificate_no       VARCHAR(40)  NULL,
134 status              ENUM('pending','passed','failed') NOT NULL DEFAULT 'pending',

```

```

133     verified_at          TIMESTAMP      NULL,
134     KEY inspections_crop_idx (crop_id),
135     KEY inspections_status_idx (status),
136     CONSTRAINT fk_insp_crop FOREIGN KEY (crop_id) REFERENCES crops_inventory(crop_id) ON DELETE CA
137     SCADE,
138     CONSTRAINT fk_insp_inspector FOREIGN KEY (inspector_id) REFERENCES users(id)
139 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
140
141 -- Supporting tables: order_events (tracking audit), wallet_transactions (escrow ledger), disputes
142 CREATE TABLE order_events (
143     event_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
144     order_id INT UNSIGNED NOT NULL,
145     stage VARCHAR(40) NOT NULL,
146     location VARCHAR(120) NULL,
147     lat DOUBLE NULL,
148     lng DOUBLE NULL,
149     note TEXT NULL,
150     actor_id INT UNSIGNED NULL,
151     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
152     KEY order_events_order_idx (order_id),
153     CONSTRAINT fk_events_order FOREIGN KEY (order_id) REFERENCES orders_logistics(order_id) ON DELETE CASCAD
154     E,
155     CONSTRAINT fk_events_actor FOREIGN KEY (actor_id) REFERENCES users(id) ON DELETE SET NULL
156 ) ENGINE=InnoDB;
157
158 CREATE TABLE wallet_transactions (
159     txn_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
160     user_id INT UNSIGNED NOT NULL,
161     order_id INT UNSIGNED NULL,
162     type ENUM('deposit','escrow_lock','escrow_release','payout','refund','platform_fee','withdrawal')
163     NOT NULL,
164     amount DECIMAL(14,2) NOT NULL,
165     description TEXT NULL,
166     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
167     KEY wallet_user_idx (user_id),
168     CONSTRAINT fk_wallet_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
169     CONSTRAINT fk_wallet_order FOREIGN KEY (order_id) REFERENCES orders_logistics(order_id) ON DELETE SET NU
170     LL
171 ) ENGINE=InnoDB;
172
173 CREATE TABLE disputes (
174     dispute_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
175     order_id INT UNSIGNED NOT NULL,
176     raised_by INT UNSIGNED NOT NULL,
177     reason TEXT NOT NULL,
178     status ENUM('open','investigating','resolved_release','resolved_refund') NOT NULL DEFAULT 'open',
179     resolution TEXT NULL,
180     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
181     resolved_at TIMESTAMP NULL,
182     KEY disputes_order_idx (order_id),
183     KEY disputes_status_idx (status),
184     CONSTRAINT fk_disputes_order FOREIGN KEY (order_id) REFERENCES orders_logistics(order_id) ON DELETE CASC
185     ADE,
186     CONSTRAINT fk_disputes_user FOREIGN KEY (raised_by) REFERENCES users(id)
187 ) ENGINE=InnoDB;
188
189 CREATE TABLE reviews (
190     review_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
191     order_id INT UNSIGNED NOT NULL,
192     buyer_id INT UNSIGNED NOT NULL,
193     farmer_id INT UNSIGNED NOT NULL,
194     rating TINYINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
195     comment TEXT NULL,
196     farmer_reply TEXT NULL,
197     replied_at TIMESTAMP NULL,
198     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
199     UNIQUE KEY reviews_order_uq (order_id),
200     KEY reviews_farmer_idx (farmer_id),
201     CONSTRAINT fk_reviews_order FOREIGN KEY (order_id) REFERENCES orders_logistics(order_id) ON DELETE CAS
202     CADE,
203     CONSTRAINT fk_reviews_buyer FOREIGN KEY (buyer_id) REFERENCES users(id) ON DELETE CASCADE,
204     CONSTRAINT fk_reviews_farmer FOREIGN KEY (farmer_id) REFERENCES users(id) ON DELETE CASCADE
205 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
206
207 CREATE TABLE notifications (
208     notification_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,

```

```

203     user_id    INT UNSIGNED NOT NULL,
204     type       VARCHAR(40)  NOT NULL,
205     title      VARCHAR(200) NOT NULL,
206     body       TEXT         NULL,
207     link       VARCHAR(300) NULL,
208     channels   VARCHAR(60)  NOT NULL DEFAULT 'in_app',
209     is_read    BOOLEAN      NOT NULL DEFAULT FALSE,
210     created_at TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
211     KEY notifications_user_idx (user_id, is_read),
212     CONSTRAINT fk_notifications_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
213 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
214
215 CREATE TABLE auth_tokens (
216     token_id   INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
217     user_id    INT UNSIGNED NOT NULL,
218     purpose    VARCHAR(20)  NOT NULL,          -- verify_email | reset_link (15-min password reset token)
219     code_hash  VARCHAR(128) NOT NULL,          -- SHA-256 of the 6-digit code + secret
220     attempts   INT          NOT NULL DEFAULT 0,
221     expires_at TIMESTAMP    NOT NULL,
222     used_at    TIMESTAMP    NULL,
223     created_at TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
224     KEY auth_tokens_user_idx (user_id, purpose),
225     CONSTRAINT fk_auth_tokens_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
226 ) ENGINE=InnoDB;
227
228 CREATE TABLE payment_intents (
229     intent_id  INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
230     reference  VARCHAR(60)  NOT NULL,
231     user_id    INT UNSIGNED NOT NULL,
232     provider   VARCHAR(20)  NOT NULL,          -- stripe | jazzcash | easypaisa | sandbox
233     amount     DECIMAL(14,2) NOT NULL,
234     status     VARCHAR(20)  NOT NULL DEFAULT 'pending',
235     provider_ref VARCHAR(200) NULL,
236     raw_callback TEXT       NULL,
237     created_at TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
238     completed_at TIMESTAMP  NULL,
239     UNIQUE KEY payment_intents_ref_uq (reference),
240     KEY payment_intents_user_idx (user_id),
241     CONSTRAINT fk_payment_intents_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
242 ) ENGINE=InnoDB;
243
244 CREATE TABLE payout_requests (
245     payout_id   INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
246     farmer_id   INT UNSIGNED NOT NULL,
247     amount      DECIMAL(14,2) NOT NULL,
248     method      VARCHAR(20)  NOT NULL,          -- bank | jazzcash | easypaisa
249     account_title VARCHAR(120) NOT NULL,
250     account_number VARCHAR(60) NOT NULL,
251     status      VARCHAR(20)  NOT NULL DEFAULT 'pending',
252     admin_note   TEXT        NULL,
253     created_at  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
254     processed_at TIMESTAMP    NULL,
255     KEY payout_requests_farmer_idx (farmer_id),
256     KEY payout_requests_status_idx (status),
257     CONSTRAINT fk_payout_requests_farmer FOREIGN KEY (farmer_id) REFERENCES users(id) ON DELETE CASCADE
258 ) ENGINE=InnoDB;
259
260 CREATE TABLE db_audit_log (
261     audit_id    INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
262     admin_id    INT UNSIGNED NULL,
263     mode       VARCHAR(10)  NOT NULL,
264     query      TEXT         NOT NULL,
265     row_count   INT          NULL,
266     duration_ms INT          NULL,
267     error      TEXT         NULL,
268     created_at  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
269     CONSTRAINT fk_audit_admin FOREIGN KEY (admin_id) REFERENCES users(id) ON DELETE SET NULL
270 ) ENGINE=InnoDB;
271 `;
272
273 export const ERD = `users 1—* crops_inventory          (farmer_id)
274 users 1—* bids_negotiations          (buyer_id)
275 crops_inventory 1—* bids_negotiations (crop_id)
276 bids_negotiations 1—0..1 orders_logistics (bid_id UNIQUE)
277 users 1—* orders_logistics          (farmer_id, buyer_id)
278 crops_inventory 1—* orders_logistics (crop_id)

```

```

279 orders_logistics 1——* order_events (order_id)
280 crops_inventory 1——* quality_inspections (crop_id)
281 users 1——* quality_inspections (inspector_id)
282 users 1——* wallet_transactions (user_id)
283 orders_logistics 1——* disputes (order_id)
284 orders_logistics 1——0..1 reviews (order_id UNIQUE) → users (buyer_id, farmer_id)
285 users 1——* notifications · auth_tokens · payment_intents · payout_requests · db_audit_log`;
286
287 export type Endpoint = { method: "GET" | "POST" | "PATCH" | "DELETE"; path: string; roles: string; desc: string; request?: string; response?: string };
288
289 export const API_GROUPS: { name: string; endpoints: Endpoint[] }[] = [
290   {
291     name: "Authentication",
292     endpoints: [
293       { method: "POST", path: "/api/auth/register", roles: "public", desc: "Register a farmer or buyer. Sets httpOnly session cookie.", request: `{ "fullName": "Ali Raza", "email": "ali@farm.pk", "password": "*****", "phone": "+92 300 1234567", "role": "farmer", "city": "Multan", "cnicId": "36302-1234567-1", "businessName": "Raza Farms" }`, response: `{ "success": true, "data": { "id": 11, "role": "farmer", "redirectTo": "/farmer" } }` },
294       { method: "POST", path: "/api/auth/login", roles: "public", desc: "Email + password login (script hash, HMAC-signed cookie).", request: `{ "email": "farmer@marketlink.pk", "password": "password123" }`, response: `{ "success": true, "data": { "id": 2, "fullName": "Muhammad Aslam", "role": "farmer", "redirectTo": "/farmer" } }` },
295       { method: "POST", path: "/api/auth/logout", roles: "any", desc: "Clears the session cookie." },
296       { method: "GET", path: "/api/auth/me", roles: "any", desc: "Current user profile (no password hash)."},
297     ],
298   },
299   {
300     name: "Crop Inventory",
301     endpoints: [
302       { method: "GET", path: "/api/crops?category=&grade=&minPrice=&maxPrice=&q=&originCity=&radiusKm=&sort=&mine=1", roles: "any", desc: "Marketplace catalogue with haversine radius filtering, open-bid counts, highest bid and inspection grade.", response: `{ "success": true, "data": [{ "cropId": 1, "cropName": "Wheat", "category": "grains", "qualityGrade": "A", "totalQuantityKg": 24000, "basePricePerKg": 96, "farmLocation": "Multan", "distanceKm": 312, "openBids": 2, "highestBid": 94, "farmerVerified": true }] }` },
303       { method: "POST", path: "/api/crops", roles: "farmer", desc: "Create listing. Unverified farmers' listings enter pending inspection.", request: `{ "cropName": "Wheat", "category": "grains", "qualityGrade": "A", "totalQuantityKg": 5000, "basePricePerKg": 96.5, "harvestDate": "2025-04-20", "farmLocation": "Multan", "description": "Galaxy-2013", "imagesJson": ["data:image/jpeg;base64,..."] }`, response: `{ "success": true, "data": { "cropId": 14, "status": "active", ... } }` },
304       { method: "GET", path: "/api/crops/:id", roles: "any", desc: "Listing detail with inspection history"},
305       { method: "PATCH", path: "/api/crops/:id", roles: "farmer (owner), admin", desc: "Update price, quantity, photos, status (active|archived).", request: `{ "basePricePerKg": 98, "totalQuantityKg": 20000 }` },
306       { method: "DELETE", path: "/api/crops/:id", roles: "farmer (owner), admin", desc: "Hard delete, or soft-archive if orders reference it (open bids auto-rejected).", response: `{ "success": true, "data": { "archived": true } }` },
307     ],
308   },
309   {
310     name: "Bidding & Negotiation",
311     endpoints: [
312       { method: "GET", path: "/api/bids?status=&cropId=", roles: "farmer | buyer | admin", desc: "Bids scoped to role (farmer: bids on own crops; buyer: own bids)."},
313       { method: "POST", path: "/api/bids", roles: "buyer", desc: "Submit bulk offer / RFQ. Validates stock, lowball guard (≥50% of ask), one open bid per crop per buyer.", request: `{ "cropId": 1, "bidPricePerKg": 92, "bidQuantityKg": 10000, "targetDeliveryDate": "2025-06-01", "message": "Pickup from farm" }`, response: `{ "success": true, "data": { "bidId": 41, "status": "pending", ... } }` },
314       { method: "PATCH", path: "/api/bids/:id", roles: "farmer | buyer", desc: "State machine action: accept | reject | counter (farmer) · accept_counter | reject_counter | revise | withdraw (buyer). Accepting creates an order atomically.", request: `{ "action": "counter", "counterPrice": 98, "note": "Includes bagging" }`, response: `{ "success": true, "data": { "bid": { "status": "accepted" }, "order": { "orderId": 37, "trackingNumber": "ML-2025-1038-X7K", "paymentStatus": "awaiting_escrow" } } }` },
315     ],
316   },
317   {
318     name: "Orders, Logistics & Escrow",
319     endpoints: [
320       { method: "GET", path: "/api/orders?active=1", roles: "any", desc: "Orders with joined crop/party info, tracking events and disputes." },
321       { method: "GET", path: "/api/orders/:id", roles: "party | admin", desc: "Single order detail." },
322       { method: "PATCH", path: "/api/orders/:id", roles: "role-dependent", desc: "lock_escrow (buyer) · advance (admin→quality_checked, farmer→dispatched/in_transit, buyer→delivered + auto escrow release) · update_location (farmer).", request: `{ "action": "advance", "location": "Khanewal Bypass, N-5", "note": "Truck" }` },

```

```

321     LES-4521" }`, response: `{ "success": true, "data": { "orderId": 1, "deliveryStage": "in_transit", "payme
322     ntStatus": "escrow_locked" } }`,
323     { method: "POST", path: "/api/orders/:id/dispute", roles: "buyer | farmer", desc: "Freeze escrow and
324     open a dispute.", request: `{ "reason": "Moisture 17% vs agreed 14%" }` },
325     { method: "GET", path: "/api/wallet", roles: "farmer | buyer", desc: "Balances + ledger.", response:
326     `{ "success": true, "data": { "walletBalance": 4500000, "escrowBalance": 1044000, "pendingInEscrow": 1044
327     000, "transactions": [...] } }`,
328     { method: "POST", path: "/api/wallet", roles: "buyer", desc: "Simulated bank top-up.", request: `{ "
329     amount": 500000 }` },
330     ],
331     },
332     {
333     name: "Mandi Rates",
334     endpoints: [
335     { method: "GET", path: "/api/mandi-rates?view=latest&crop=&market=", roles: "any", desc: "Latest rat
336     e per crop x mandi with day-on-day % change (window function)."},
337     { method: "GET", path: "/api/mandi-rates?view=trend&crop=Wheat&days=14", roles: "any", desc: "Pivote
338     d daily series for charting.", response: `{ "success": true, "data": { "cropName": "Wheat", "markets": ["L
339     ahore", ...], "series": [{ "date": "2025-05-01", "Lahore": 98.2, "Multan": 91.4 }] } }`,
340     { method: "GET", path: "/api/mandi-rates?view=compare", roles: "any", desc: "Mandi avg vs platform a
341     sk vs realised platform deal price per crop." },
342     { method: "POST", path: "/api/mandi-rates", roles: "admin", desc: "Manual upsert (unique crop+market
343     +date).", request: `{ "cropName": "Wheat", "marketLocation": "Lahore", "minPricePerKg": 92, "maxPricePerKg
344     ": 101, "dateUpdated": "2025-05-14" }` },
345     { method: "POST", path: "/api/mandi-rates/sync", roles: "admin", desc: "Mock government feed sync (b
346     ounded ±3% random walk, source=feed).", response: `{ "success": true, "data": { "updated": 60, "date": "20
347     25-05-14" } }` },
348     ],
349     },
350     {
351     name: "Payments, Payouts & Reviews",
352     endpoints: [
353     { method: "GET", path: "/api/payments/checkout", roles: "buyer | farmer", desc: "Enabled gateways (s
354     tripe / jazzcash / easypaisa / sandbox) and their mode." },
355     { method: "POST", path: "/api/payments/checkout", roles: "buyer", desc: "Create a payment intent; re
356     turns a redirect URL (Stripe Checkout / sandbox) or an auto-submit form (JazzCash / Easypaisa). The wallet
357     is credited only by a verified callback.", request: `{ "provider": "stripe", "amount": 500000 }`, respons
358     e: `{ "success": true, "data": { "reference": "MLT1790...", "redirectUrl": "https://checkout.stripe.com/..." }
359     }` },
360     { method: "POST", path: "/api/payments/webhook/stripe", roles: "Stripe (signed)", desc: "Stripe webh
361     ook; verifies Stripe-Signature (HMAC-SHA256, 5-min tolerance) and settles checkout.session.* events idempo
362     tently." },
363     { method: "POST", path: "/api/payments/callback/jazzcash", roles: "JazzCash (signed)", desc: "JazzCa
364     sh return URL; verifies pp_SecureHash with the integrity salt, pp_ResponseCode 000 = success." },
365     { method: "GET", path: "/api/payments/callback/easypaisa", roles: "Easypaisa", desc: "Easypaisa post
366     Back; result is confirmed server-to-server via Inquire Transaction before crediting." },
367     { method: "GET", path: "/api/payments/:reference", roles: "owner", desc: "Poll payment status after
368     returning from the gateway." },
369     { method: "POST", path: "/api/payouts", roles: "farmer", desc: "Withdraw wallet balance to bank / Ja
370     zzCash / Easypaisa.", request: `{ "amount": 50000, "method": "jazzcash", "accountTitle": "Bashir Soomro",
371     "accountNumber": "03451234567" }` },
372     { method: "PATCH", path: "/api/payouts/:id", roles: "admin", desc: "Mark payout paid, or reject (fun
373     ds returned).", request: `{ "status": "paid" }` },
374     { method: "POST", path: "/api/reviews", roles: "buyer", desc: "Rate the seller of a delivered & paid
375     order (1 per order); recalculates the seller trust score.", request: `{ "orderId": 33, "rating": 5, "comm
376     ent": "Fresh onions, perfect weight" }`, response: `{ "success": true, "data": { "trust": { "trustScore":
377     4.41, "ratingAvg": 4.6, "ratingCount": 5 } } }`,
378     { method: "GET", path: "/api/reviews?farmerId=2 &?pending=1", roles: "public & buyer", desc: "Selle
379     r reviews (including farmer replies), or the buyer's delivered orders awaiting review." },
380     { method: "POST", path: "/api/reviews/:id/reply", roles: "farmer (seller on that order)", desc: "Res
381     pond once to a verified buyer review. Buyer receives an in-app notification; response appears on crop deta
382     ils.", request: `{ "reply": "Thank you for your feedback. We look forward to supplying you again." }` },
383     { method: "GET", path: "/api/reports/summary", roles: "farmer | admin", desc: "Download a live, role
384     -scoped CSV with KPIs, six monthly periods, and crop or regional breakdowns. Buyers and inspectors cannot
385     access it." },
386     ],
387     },
388     {
389     name: "Tracking, Notifications & Auth",
390     endpoints: [
391     { method: "GET", path: "/api/tracking/:trackingNumber", roles: "public", desc: "Route, live/estimate
392     d GPS position, progress, ETA and milestones. Party names & notes only for order parties/staff. Fires the
393     'arriving soon' milestone at 85%." },
394     { method: "POST", path: "/api/orders/:id/gps", roles: "farmer | admin", desc: "Driver GPS ping while
395     dispatched / in transit.", request: `{ "lat": 25.9, "lng": 68.1, "label": "M-9 near Nooriabad" }` },
396     { method: "GET", path: "/api/notifications", roles: "any", desc: "Latest 30 notifications + unread c

```

```

ount." },
361     { method: "PATCH", path: "/api/notifications", roles: "any", desc: "Mark given ids (or all) as read.
    ", request: `{ "ids": [12, 13] }` },
362     { method: "POST", path: "/api/auth/verify-email", roles: "public", desc: "Verify the 6-digit signup
    code (signs in), or { resend: true }.", request: `{ "email": "ali@farm.pk", "code": "816958" }` },
363     { method: "POST", path: "/api/auth/forgot-password", roles: "public", desc: "Generates a single-use
    reset token (15-minute expiry, stored as SHA-256) and emails a /reset-password?token=... link. Same response
    whether or not the account exists.", request: `{ "email": "buyer@marketlink.pk" }`, response: `{ "success
    ": true, "data": { "sent": true, "message": "If an account exists..." } }` },
364     { method: "GET", path: "/api/auth/reset-password?token=...", roles: "public", desc: "Checks a reset li
    nk before the form is shown (masked email + expiry)."},
365     { method: "POST", path: "/api/auth/reset-password", roles: "public", desc: "Validates the token, sto
    res the new password as a bcrypt hash and deletes the token.", request: `{ "token": "...", "newPassword": "N
    ewPass#2026" }` },
366     { method: "GET", path: "/api/auth/google?role=buyer|farmer", roles: "public", desc: "Starts Google O
    Auth 2.0 (authorization code + PKCE, signed state cookie) → Google consent screen." },
367     { method: "GET", path: "/api/auth/google/callback", roles: "Google", desc: "Verifies state, exchange
    s the code, requires a verified Google email; signs in existing users (linking Google) or creates a new bu
    yer/farmer with name, email and profile picture." },
368     { method: "PATCH", path: "/api/account/profile", roles: "any", desc: "Update name, phone, city, busi
    ness, CNIC, address (used to complete Google sign-ups).", request: `{ "fullName": "Ali Raza", "phone": "03
    00 1234567", "city": "Multan" }` },
369   ],
370 },
371 {
372   name: "Database Explorer (admin)",
373   endpoints: [
374     { method: "GET", path: "/api/db/schema", roles: "admin", desc: "Tables, columns, PK/FK, indexes, row
    counts and recent audit log." },
375     { method: "POST", path: "/api/db/query", roles: "admin", desc: "Single SQL statement. read = READ ON
    LY transaction + 8s timeout + 500-row cap; write requires confirm: true. Dangerous statements blocked; eve
    ry query audited in db_audit_log.", request: `{ "sql": "SELECT full_name, trust_score FROM users WHERE rol
    e = 'farmer'", "mode": "read" }`, response: `{ "success": true, "data": { "columns": ["full_name", "trust_s
    core"], "rows": [{"Ghulam Rasool", 4.53}], "rowCount": 1, "durationMs": 4 } }` },
376   ],
377 },
378 {
379   name: "AI Shopping Assistant",
380   endpoints: [
381     { method: "POST", path: "/api/assistant", roles: "public (visitors + buyers)", desc: "LLM assistant
    (OpenAI or Gemini when OPENAI_API_KEY / GEMINI_API_KEY is set; rule engine otherwise) grounded in live DB
    results: top-rated dealers (rating from deliveries, verification, inspections, disputes), high-quality lis
    tings by crop/category, cheapest/nearest produce, mandi rates and FAQ (escrow, bidding, tracking). Underst
    ands English + Roman Urdu. Rate-limited to 30 msgs/min.", request: `{ "message": "Where can I find top-rat
    ed dealers?" }`, response: `{ "success": true, "data": { "reply": "Top-rated sellers...", "cards": [{ "kind
    ": "dealer", "name": "Muhammad Aslam", "business": "Aslam Agri Farms", "city": "Multan", "rating": 4.6, "de
    liveries": 8, "href": "/buyer?farmerId=2" }], "suggestions": ["High-quality vegetables", "Cheapest wheat"]
    } }` },
382   ],
383 },
384 {
385   name: "Admin & Quality",
386   endpoints: [
387     { method: "GET", path: "/api/admin/users?role=farmer", roles: "admin", desc: "Users with listing/ord
    er counts." },
388     { method: "PATCH", path: "/api/admin/users/:id", roles: "admin", desc: "Verify / revoke. Verifying a
    farmer activates held listings.", request: `{ "isVerified": true }`, response: `{ "success": true, "data"
    : { "id": 4, "isVerified": true, "listingsActivated": 1 } }` },
389     { method: "GET", path: "/api/inspections", roles: "admin", desc: "Inspection report log." },
390     { method: "POST", path: "/api/inspections", roles: "admin", desc: "Upload soil/crop inspection. pass
    ed → grade updated & listing active; failed → archived & bids rejected.", request: `{ "cropId": 7, "gradeA
    ssigned": "B", "moistureLevelPercentage": 68, "soilPh": 7.8, "inspectionNotes": "Brix 19.2", "status": "pa
    ssed", "reportAttachment": "data:application/pdf;base64,..." }` },
391     { method: "GET", path: "/api/inspections/:id/report", roles: "admin", desc: "Download attached repor
    t file." },
392     { method: "GET", path: "/api/admin/disputes", roles: "admin", desc: "Dispute resolution log." },
393     { method: "PATCH", path: "/api/admin/disputes/:id", roles: "admin", desc: "investigating | release (
    pay farmer) | refund (return to buyer).", request: `{ "outcome": "refund", "resolution": "Lab retest confi
    rmed 17% moisture" }` },
394     { method: "GET", path: "/api/analytics", roles: "any", desc: "Role-aware KPIs: farmer earnings, buye
    r spend, or platform-wide volume/regions/stages." },
395     { method: "GET", path: "/api/health", roles: "public", desc: "DB connectivity healthcheck." },
396   ],
397 },
398 ];
399

```

```

400 export const WORKFLOWS: { title: string; icon: string; steps: string[] }[] = [
401   {
402     title: "Role-Based Marketplace Flow",
403     icon: "",
404     steps: [
405       "Authenticate with password or Google OAuth; the signed session determines whether the user enters A
406       ↪ dmin, Farmer/Vendor, Buyer/Customer or Inspector modules. Every page and API repeats the role/ownership ch
407       ↪ eck.",
408       "Farmer: manage profile and graded crop listings → review and counter bids → fulfil accepted orders
409       ↪ → receive escrow payout → respond once to each verified buyer review.",
410       "Buyer: search the marketplace and inspect crop/quality details → place bids and negotiate → fund es
411       ↪ crow after acceptance → track shipment → confirm delivery and submit a verified rating.",
412       "Inspector: certify crop quality and record moisture, soil pH, grade and report before dispatch. Adm
413       ↪ in: approve users, monitor bids and transactions, manage mandi rates, and resolve disputes.",
414       "Data processing: accepting a bid creates an order and decrements inventory transactionally; escrow
415       ↪ lock precedes inspection, dispatch and tracking; verified delivery releases payment and enables a review."
416     ],
417     "Output: Admin and Farmer dashboards show live metrics and charts; their role-scoped CSV summary dow
418     ↪ nloads reflect current sales, orders and trading regions/crops.",
419   },
420   {
421     title: "Bidding & Negotiation",
422     icon: "",
423     steps: [
424       "Buyer submits bid (price, qty, target date). Guards: listing active, qty ≤ stock, price ≥ 50% of as
425       ↪ k, one open bid per buyer per listing.",
426       "Farmer reviews in Negotiation Hub drawer → Accept, Reject, or Counter (counter must exceed bid pric
427       ↪ e).",
428       "On counter: buyer may accept_counter, reject_counter, or revise (bid returns to pending with new pr
429       ↪ ice) – unlimited rounds.",
430       "On acceptance (SELECT ... FOR UPDATE on bid + crop): stock decremented, listing → sold_out at 0, othe
431       ↪ r open bids exceeding remaining stock auto-rejected, order created with unique tracking number in 'awaitin
432       ↪ g_escrow'.",
433     ],
434   },
435   {
436     title: "Escrow & Order Status Progression",
437     icon: "",
438     steps: [
439       "Buyer locks full order value: wallet_balance -= total, escrow_balance += total, ledger entry 'escro
440       ↪ w_lock'. Nothing progresses before this.",
441       "Quality inspector records pre-dispatch inspection → stage 'quality_checked' (creates a quality_insp
442       ↪ ections row).",
443       "Farmer marks 'dispatched' then 'in_transit', posting checkpoint locations (order_events audit trail
444       ↪ ).",
445       "Buyer confirms delivery → 'delivered'; escrow released atomically: farmer credited total – 1.5% pla
446       ↪ tform fee, ledger entries payout + platform_fee + escrow_release.",
447       "Either party can raise a dispute while escrow is locked → progression frozen; admin resolves by rel
448       ↪ ease (farmer paid) or refund (buyer credited).",
449     ],
450   },
451   {
452     title: "Mandi Rate Sync",
453     icon: "",
454     steps: [
455       "Rates are stored per (crop, market, date) with a UNIQUE constraint → all writes are idempotent upse
456       ↪ rts.",
457       "Admin manual entry: validates min ≤ avg ≤ max (avg defaults to midpoint), source='manual'.",
458       "Mock feed sync: reads the latest rate for every crop × mandi, applies a bounded ±3% random walk, up
459       ↪ serts today's row with source='feed' in one transaction; missing pairs are seeded from catalogue base pric
460       ↪ es.",
461       "Insights: day-on-day change via ROW_NUMBER() window; platform comparison joins latest mandi avg vs
462       ↪ active listing ask vs 90-day realised deal price; farmer listing form suggests grade-adjusted price (A ×1.
463       ↪ 00, B ×1.00, C ×0.90).",
464     ],
465   },
466   {
467     title: "Verification & Quality",
468     icon: "[Urdu]",
469     steps: [
470       "New farmers register unverified; their listings enter 'pending_inspection' and are hidden from the
471       ↪ marketplace.",
472       "Admin verifies CNIC/farm → farmer verified and held listings activated.",
473       "Inspection passed → listing grade overwritten with assigned grade and activated; failed → listing a
474       ↪ rchived and open bids rejected; pending → held.",
475     ],
476   },
477 ]

```

```

451     },
452   },
453 ];
454

```

src/lib/google.ts

```

1  import { createHash, createHmac, randomBytes, timingSafeEqual } from "crypto";
2  import { eq } from "drizzle-orm";
3  import { db } from "@db";
4  import { users } from "@db/schema";
5  import { ApiError } from "@lib/api";
6  import { absoluteUrl, isValidHttpUrl, parseOriginInfo } from "@lib/base-url";
7  import { homeFor, startSession } from "@lib/auth";
8  import { notify } from "@lib/notify";
9  import { unusablePasswordHash } from "@lib/password";
10
11  /* ----- */
12  /* Configuration */
13  /* ----- */
14  export const googleConfigured = () => !!process.env.GOOGLE_CLIENT_ID && process.env.GOOGLE_CLIENT_SECRET
15  ;
16  /** Test mode (no Google credentials): enabled unless GOOGLE_OAUTH MOCK=false. Can only create/sign into i
17  ts own test accounts. */
18  export const googleMockEnabled = () => !googleConfigured() && process.env.GOOGLE_OAUTH MOCK !== "false";
19
20  // Endpoint overrides exist only for automated tests.
21  const AUTH_URL = () => process.env.GOOGLE_AUTH_URL || "https://accounts.google.com/o/oauth2/v2/auth";
22  const TOKEN_URL = () => process.env.GOOGLE_TOKEN_URL || "https://oauth2.googleapis.com/token";
23  const USERINFO_URL = () => process.env.GOOGLE_USERINFO_URL || "https://openidconnect.googleapis.com/v1/use
24  rinfo";
25
26  const SECRET = process.env.SESSION_SECRET || "marketlink-agri-hub-dev-secret-change-me";
27  export const OAUTH_COOKIE = "ml_oauth";
28
29  /**
30   * Google requires the redirect URI to match the one registered in the Google console, so an
31   * explicit GOOGLE_CALLBACK_URL wins – but only when it is a well-formed, non-bind absolute URL.
32   * Otherwise the public origin of the current request is used (never the bind address).
33   */
34  export const callbackUrl = (req: Request) => {
35    const configured = parseOriginInfo(process.env.GOOGLE_CALLBACK_URL ?? null);
36    if (configured && !configured.bind && isValidHttpUrl(process.env.GOOGLE_CALLBACK_URL)) {
37      return new URL(process.env.GOOGLE_CALLBACK_URL!.trim()).toString();
38    }
39    return absoluteUrl(req, "/api/auth/google/callback");
40  };
41
42  export type SignupRole = "buyer" | "farmer";
43  export const safeNext = (n: string | null | undefined) => (n && n.startsWith("/") && !n.startsWith("//") &
44  & !n.startsWith("/api/") ? n : null);
45
46  /* ----- */
47  /* Signed, short-lived state cookie (CSRF state + PKCE verifier) */
48  /* ----- */
49  type OAuthState = { state: string; verifier: string; role: SignupRole; next: string | null; ts: number };
50
51  export function sealState(s: OAuthState) {
52    const body = Buffer.from(JSON.stringify(s)).toString("base64url");
53    return `${body}.${createHmac("sha256", SECRET).update(body).digest("base64url")}`;
54  }
55
56  export function openState(cookie: string | undefined): OAuthState | null {
57    if (!cookie) return null;
58    const [body, sig] = cookie.split(".");
59    if (!body || !sig) return null;
60    const expected = createHmac("sha256", SECRET).update(body).digest("base64url");
61    if (expected.length !== sig.length || !timingSafeEqual(Buffer.from(expected), Buffer.from(sig))) return
62    null;
63    const s = JSON.parse(Buffer.from(body, "base64url").toString()) as OAuthState;
64    return Date.now() - s.ts < 10 * 60_000 ? s : null;
65  }
66
67  /** Step 1: build the Google consent-screen URL (Authorization Code flow + PKCE S256). */
68  export function buildAuthRequest(req: Request, role: SignupRole, next: string | null) {
69    const state = randomBytes(16).toString("base64url");
70    const verifier = randomBytes(48).toString("base64url");

```

```

65     const challenge = createHash("sha256").update(verifier).digest("base64url");
66     const url = new URL(AUTH_URL());
67     url.search = new URLSearchParams({
68         client_id: process.env.GOOGLE_CLIENT_ID!,
69         redirect_uri: callbackUrl(req),
70         response_type: "code",
71         scope: "openid email profile",
72         state,
73         code_challenge: challenge,
74         code_challenge_method: "S256",
75         prompt: "select_account",
76         access_type: "online",
77     }).toString();
78     return { url: url.toString(), cookie: sealState({ state, verifier, role, next, ts: Date.now() }) };
79 }
80
81 export type GoogleProfile = { sub: string; email: string; email_verified: boolean; name?: string; picture?
    : string };
82
83 /** Step 2: exchange the authorization code for tokens and fetch the verified profile. */
84 export async function exchangeCode(req: Request, code: string, verifier: string): Promise<GoogleProfile> {
85     const tokenRes = await fetch(TOKEN_URL(), {
86         method: "POST",
87         headers: { "Content-Type": "application/x-www-form-urlencoded" },
88         body: new URLSearchParams({
89             code,
90             client_id: process.env.GOOGLE_CLIENT_ID!,
91             client_secret: process.env.GOOGLE_CLIENT_SECRET!,
92             redirect_uri: callbackUrl(req),
93             grant_type: "authorization_code",
94             code_verifier: verifier,
95         })),
96     });
97     const tokens = (await tokenRes.json().catch(() => ({}))) as { access_token?: string; error?: string; err
    or_description?: string };
98     if (!tokenRes.ok || !tokens.access_token) throw new ApiError(502, `Google token exchange failed: ${token
    s.error_description ?? tokens.error ?? tokenRes.status}`);
99     const infoRes = await fetch(USERINFO_URL(), { headers: { Authorization: `Bearer ${tokens.access_token}`
    } });
100    const p = (await infoRes.json().catch(() => ({}))) as Partial<GoogleProfile>;
101    if (!infoRes.ok || !p.sub || !p.email) throw new ApiError(502, "Could not read your Google profile");
102    if (p.email_verified !== true) throw new ApiError(403, "Your Google email address is not verified");
103    return { sub: p.sub, email: p.email.toLowerCase(), email_verified: true, name: p.name, picture: p.pictur
    e };
104 }
105
106 /* ----- */
107 /* User provisioning */
108 /* ----- */
109 /**
110  * Existing Google ID → sign in. Existing email → link Google to that account and sign in.
111  * New email → create an account (role buyer/farmer, auto-verified email, Google profile picture).
112  * In test mode (`mock`), only accounts created by test mode can be accessed.
113  */
114 export async function provisionGoogleUser(p: GoogleProfile, role: SignupRole, mock = false) {
115     let [u] = await db.select().from(users).where(eq(users.googleId, p.sub)).limit(1);
116     let created = false;
117     if (!u) {
118         const [byEmail] = await db.select().from(users).where(eq(users.email, p.email)).limit(1);
119         if (byEmail) {
120             if (mock) throw new ApiError(409, "This email already has a MarketLink account. Sign in with your pa
    ssword (Google test mode can't access existing accounts).");
121             [u] = await db
122                 .update(users)
123                 .set({ googleId: p.sub, avatarUrl: byEmail.avatarUrl ?? p.picture ?? null, emailVerified: true })
124                 .where(eq(users.id, byEmail.id))
125                 .returning();
126         } else {
127             [u] = await db
128                 .insert(users)
129                 .values({
130                     fullName: (p.name || p.email.split("@")[0]).slice(0, 120),
131                     email: p.email,
132                     passwordHash: unusablePasswordHash(),
133                     phone: "",
134                     city: "",

```

```

135         role,
136         isVerified: role === "buyer",
137         emailVerified: true,
138         walletBalance: 0,
139         avatarUrl: p.picture ?? null,
140         googleId: p.sub,
141         authProvider: mock ? "google_test" : "google",
142     })
143     .returning();
144     created = true;
145     await notify(u.id, {
146         type: "welcome",
147         title: "Welcome to MarketLink",
148         body: role === "farmer" ? "Your account was created with Google. Add your phone and farm location
149         to start listing crops." : "Your account was created with Google. Add your phone and city, then fund your
150         wallet to start buying.",
151         link: "/account?complete=1",
152         email: true,
153     });
154     } else if (mock && u.authProvider !== "google_test") {
155         throw new ApiError(409, "This account requires real Google sign-in.");
156     }
157     await startSession(u.id, u.role);
158     const incomplete = !u.phone || !u.city;
159     return { user: u, created, redirectTo: incomplete ? "/account?complete=1" : homeFor(u.role) };
160 }
161 /** Relative redirect that keeps cookies set via next/headers (Location may be relative per RFC 7231). */
162 export const redirectTo = (location: string, status = 302) => new Response(null, { status, headers: { Location: location, "Cache-Control": "no-store" } });
163
164 export const mockSub = (email: string) => `test-${createHash("sha256").update(email.toLowerCase()).digest(
165     "hex").slice(0, 24)}`;

```

src/lib/llm.ts

```

1  /**
2   * LLM connector for the shopping assistant.
3   * • OPENAI_API_KEY → OpenAI Chat Completions (OPENAI_MODEL, default gpt-4o-mini)
4   * • GEMINI_API_KEY → Google Gemini generateContent (GEMINI_MODEL, default gemini-1.5-flash)
5   * Returns null when no key is configured or the call fails, so callers can fall back to the rule engine.
6   */
7   export type ChatTurn = { role: "user" | "assistant"; content: string };
8   export type LlmEngine = "openai" | "gemini";
9
10  export const llmEngine = (): LlmEngine | null => (process.env.OPENAI_API_KEY ? "openai" : process.env.GEMINI_API_KEY ? "gemini" : null);
11
12  const SYSTEM = `You are "MarketLink Assistant", the shopping assistant of MarketLink Agri-Hub, a Pakistani
13  B2B agricultural marketplace connecting farmers with wholesalers, supermarkets, exporters and food processors.
14  Rules:
15  - Reply in the user's language: English, or Roman Urdu if the user writes in Roman Urdu (e.g. "gandum ka rate kya hai"). Never reply in Urdu script unless the user uses it.
16  - Use ONLY the facts in the CONTEXT JSON (live database results). Never invent sellers, prices, ratings or stock. If the context has no data, say so and suggest a related query.
17  - Prices are in Pakistani Rupees (Rs.), per kg and per maund (40 kg). Mention the seller's trust score/rating when relevant.
18  - Be concise: 1-4 short sentences, or a short list. The UI already shows result cards from CONTEXT.cards, so summarise and compare rather than repeating every field.
19  - Platform facts: payments are held in escrow and released to the farmer only after the buyer confirms delivery (1.5% fee from the farmer); bids can be countered; orders go Placed → Quality Inspected → Dispatched → In Transit → Delivered; disputes freeze escrow until an admin resolves them.`;
20
21  async function withTimeout<T>(p: Promise<T>, ms: number) {
22      return Promise.race([p, new Promise<never>((_, rej) => setTimeout(() => rej(new Error("LLM timeout")), ms))]);
23  }
24
25  export async function llmReply(message: string, context: unknown, history: ChatTurn[] = []): Promise<Text> {
26      const engine = llmEngine();
27      if (!engine) return null;

```

```

27     const ctx = `CONTEXT (live marketplace data, JSON):\n${JSON.stringify(context).slice(0, 12000)}`;
28     const turns = history.slice(-6);
29     try {
30         if (engine === "openai") {
31             const res = await withTimeout(
32                 fetch("https://api.openai.com/v1/chat/completions", {
33                     method: "POST",
34                     headers: { Authorization: `Bearer ${process.env.OPENAI_API_KEY}`, "Content-Type": "application/j
35 son" },
36                     body: JSON.stringify({
37                         model: process.env.OPENAI_MODEL || "gpt-4o-mini",
38                         temperature: 0.3,
39                         max_tokens: 350,
40                         messages: [{ role: "system", content: SYSTEM }, { role: "system", content: ctx }, ...turns, {
41 role: "user", content: message }],
42                     })),
43                     15000,
44                 );
45                 if (!res.ok) throw new Error(`OpenAI ${res.status}: ${res.text().slice(0, 200)}`);
46                 const data = (await res.json()) as { choices?: { message?: { content?: string } }[] };
47                 const text = data.choices?.[0]?.message?.content?.trim();
48                 return text ? { text, engine } : null;
49             }
50             const model = process.env.GEMINI_MODEL || "gemini-1.5-flash";
51             const res = await withTimeout(
52                 fetch(`https://generativelanguage.googleapis.com/v1beta/models/${model}:generateContent?key=${process
53 env.GEMINI_API_KEY}`, {
54                     method: "POST",
55                     headers: { "Content-Type": "application/json" },
56                     body: JSON.stringify({
57                         systemInstruction: { parts: [{ text: `${SYSTEM}\n\n${ctx}` } ] },
58                         contents: [...turns.map((t) => ({ role: t.role === "assistant" ? "model" : "user", parts: [{ tex
59 t: t.content } ] })), { role: "user", parts: [{ text: message } ] }],
60                         generationConfig: { temperature: 0.3, maxOutputTokens: 350 },
61                     })),
62                     15000,
63                 );
64                 if (!res.ok) throw new Error(`Gemini ${res.status}: ${res.text().slice(0, 200)}`);
65                 const data = (await res.json()) as { candidates?: { content?: { parts?: { text?: string } }[] }[] };
66                 const text = data.candidates?.[0]?.content?.parts?.map((p) => p.text ?? "").join("").trim();
67                 return text ? { text, engine } : null;
68             } catch (e) {
69                 console.error("[llm]", (e as Error).message);
70                 return null;
71             }
72         }
73     }

```

src/lib/migrate.ts

```

1  import { sql } from "drizzle-orm";
2  import { db } from "@db";
3  import { POSTGRES_DDL } from "@lib/ddl-postgres";
4  import { hashPassword } from "@lib/password";
5
6  /**
7   * Idempotent schema upgrades for databases created by earlier versions
8   * (e.g. the built-in local database of a previous download).
9   * Fresh databases get the full DDL in seed.ts instead.
10  */
11  const IGNOREABLE = new Set(["42P07", "42710", "42701", "42P06", "42P16", "23505"]); // already exists / dup
12  licate
13  async function tolerant(stmt: string) {
14      try {
15          await db.execute(sql.raw(stmt));
16      } catch (e) {
17          const code = (e as { code?: string }).code ?? (e as { cause?: { code?: string } }).cause?.code;
18          if (!code || !IGNOREABLE.has(code)) throw e;
19      }
20  }
21
22  export async function needsUpgrade() {

```

```

23     const r = await db.execute(sql`
24         select to_regclass('public.db_audit_log') is not null as t,
25         exists (select 1 from information_schema.columns where table_name = 'users' and column_name = '
↳ trust_score')
26         and exists (select 1 from information_schema.columns where table_name = 'users' and column_na
↳ me = 'google_id')
27         and exists (select 1 from information_schema.columns where table_name = 'reviews' and column_
↳ name = 'farmer_reply')
28         and exists (select 1 from information_schema.columns where table_name = 'reviews' and column_
↳ name = 'replied_at') as c,
29         exists (select 1 from pg_enum e join pg_type t on t.oid = e.enumtypid where t.typname = 'user_r
↳ ole' and e.enumlabel = 'inspector') as r`);
30     const row = r.rows[0] as { t: boolean; c: boolean; r: boolean };
31     return !(row.t && row.c && row.r);
32 }
33
34 export async function upgradeSchema() {
35     console.log("[migrate] upgrading database schema to the latest version");
36     // Enum values must be added outside a transaction and before use.
37     await tolerant(`ALTER TYPE "public"."user_role" ADD VALUE IF NOT EXISTS 'inspector'`);
38     await tolerant(`ALTER TYPE "public"."wallet_txn_type" ADD VALUE IF NOT EXISTS 'withdrawal'`);
39     // New columns on existing tables
40     for (const stmt of [
41         `ALTER TABLE "users" ADD COLUMN IF NOT EXISTS "email_verified" boolean DEFAULT false NOT NULL`,
42         `ALTER TABLE "users" ADD COLUMN IF NOT EXISTS "trust_score" numeric(3, 2) DEFAULT 0 NOT NULL`,
43         `ALTER TABLE "users" ADD COLUMN IF NOT EXISTS "rating_avg" numeric(3, 2) DEFAULT 0 NOT NULL`,
44         `ALTER TABLE "users" ADD COLUMN IF NOT EXISTS "rating_count" integer DEFAULT 0 NOT NULL`,
45         `ALTER TABLE "users" ADD COLUMN IF NOT EXISTS "title" varchar(120)`,
46         `ALTER TABLE "users" ADD COLUMN IF NOT EXISTS "user_code" varchar(40)`,
47         `ALTER TABLE "users" ADD COLUMN IF NOT EXISTS "avatar_url" text`,
48         `ALTER TABLE "users" ADD COLUMN IF NOT EXISTS "google_id" varchar(64)`,
49         `ALTER TABLE "users" ADD COLUMN IF NOT EXISTS "auth_provider" varchar(20) DEFAULT 'password' NOT NULL`
↳ ,
50         `ALTER TABLE "orders_logistics" ADD COLUMN IF NOT EXISTS "current_lat" double precision`,
51         `ALTER TABLE "orders_logistics" ADD COLUMN IF NOT EXISTS "current_lng" double precision`,
52         `ALTER TABLE "order_events" ADD COLUMN IF NOT EXISTS "lat" double precision`,
53         `ALTER TABLE "order_events" ADD COLUMN IF NOT EXISTS "lng" double precision`,
54         `ALTER TABLE "quality_inspections" ADD COLUMN IF NOT EXISTS "report_file_name" varchar(200)`,
55         `ALTER TABLE "quality_inspections" ADD COLUMN IF NOT EXISTS "certificate_no" varchar(40)`,
56         `ALTER TABLE "reviews" ADD COLUMN IF NOT EXISTS "farmer_reply" text`,
57         `ALTER TABLE "reviews" ADD COLUMN IF NOT EXISTS "replied_at" timestamp with time zone`,
58     ]) await tolerant(stmt);
59     // New tables, indexes and foreign keys: replay the full DDL, skipping objects that already exist.
60     for (const stmt of POSTGRES_DDL.split(/;\s*(?:\n|$)/).map((s) => s.trim()).filter(Boolean)) await tolera
↳ nt(stmt);
61
62     console.log("[migrate] done");
63 }
64
65 /**
66  * Idempotent data backfill, safe to run on every start:
67  * • legacy accounts (created before OTP verification existed, i.e. no auth_tokens rows) are marked email
↳ -verified
68  * • a Quality Inspector account exists
69  */
70 export const SUPER_ADMIN = {
71     userCode: "admin_01",
72     fullName: "Ahmed Mustafa",
73     email: "ahmed.mustafa@admin.com",
74     title: "Super Admin & Systems Controller",
75 } as const;
76
77 export async function backfillData() {
78     // Primary admin → Ahmed Mustafa (renames the legacy demo admin once; skipped if ADMIN_EMAIL is customis
↳ ed)
79     if (!process.env.ADMIN_EMAIL) {
80         const has = await db.execute(sql`select 1 from users where email = ${SUPER_ADMIN.email} limit 1`);
81         if (!has.rows.length) {
82             await db.execute(sql`update users set full_name = ${SUPER_ADMIN.fullName}, email = ${SUPER_ADMIN.ema
↳ il}, title = ${SUPER_ADMIN.title},
83             user_code = ${SUPER_ADMIN.userCode}, business_name = 'MarketLink Platform Management'
84             where email = 'admin@marketlink.pk' and role = 'admin'`);
85         }
86     }
87     await db.execute(sql`update users set title = ${SUPER_ADMIN.title}, user_code = ${SUPER_ADMIN.userCode}
88     where user_code is null and title is null and role = 'admin'`);

```

```

89     and id = (select min(id) from users where role = 'admin')
90     and not exists (select 1 from users where user_code = ${SUPER_ADMIN.userCode})`);
91
92     await db.execute(sql`update users set email_verified = true
93     where email_verified = false and not exists (select 1 from auth_tokens t where t.user_id = users.id)`);
94     ;
95     const hasInspector = await db.execute(sql`select 1 from users where role = 'inspector' limit 1`);
96     if (!hasInspector.rows.length) {
97         await db.execute(sql`insert into users (full_name, email, password_hash, phone, role, cnic_id, city, address, is_verified, email_verified, business_name)
98         values ('Dr. Farhan Malik', 'inspector@marketlink.pk', ${hashPassword("password123")}, '+92 300 7788 990', 'inspector', '35202-9988776-3', 'Faisalabad',
99         'Ayub Agricultural Research Institute, Faisalabad', true, true, 'MarketLink Quality Lab')
100         on conflict do nothing`);
101     }
102 }

```

src/lib/notify.ts

```

1  import { readFile } from "node:fs/promises";
2  import { join } from "node:path";
3  import { eq } from "drizzle-orm";
4  import { db } from "@db";
5  import { notifications, users } from "@db/schema";
6  import { envOriginInfo, observedPublicOrigin } from "@lib/base-url";
7
8  /**
9   * Notification service
10   * • in-app → stored in notifications
11   * • email → Resend when RESEND_API_KEY is configured (CID-embedded brand logo)
12   * • SMS → Twilio when its credentials are configured
13   * Without provider keys, email/SMS go to the development server outbox.
14   */
15  export const emailConfigured = () => !!process.env.RESEND_API_KEY;
16  export const smsConfigured = () => !!(process.env.TWILIO_ACCOUNT_SID && process.env.TWILIO_AUTH_TOKEN && process.env.TWILIO_FROM);
17
18  const APP_NAME = "MARKETLINK AGRI-HUB";
19  const LOGO_CID = "marketlink-brand";
20  let logoBase64: string | null | undefined;
21
22  async function loadEmailLogo() {
23     if (logoBase64 !== undefined) return logoBase64;
24     try {
25         const file = join(/* turbopackIgnore: true */ process.cwd(), "public", "brand", "marketlink-email.png");
26         logoBase64 = (await readFile(file)).toString("base64");
27     } catch (error) {
28         console.error("[notify] email brand asset missing", error);
29         logoBase64 = null;
30     }
31     return logoBase64;
32 }
33
34 /** Don't interpolate user-provided review text or messages directly into email HTML. */
35 const escapeHtml = (value: string) =>
36     value.replace(/&lt;"/g, (char) => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", "'": "&quot;", '"': "&#39";
37     ;" })[char]!);
38
39 function emailHtml(title: string, body: string, link: string | undefined, logoAttached: boolean) {
40     const safeLink = link?.startsWith("/") && !link.startsWith("//") ? link : undefined;
41     // Prefer a configured APP_URL; otherwise fall back to the last public origin this server saw,
42     // so notification links never point at a bind address or a stale localhost default.
43     const base = envOriginInfo()?.origin ?? observedPublicOrigin() ?? undefined;
44     const destination = safeLink && base ? escapeHtml(`${base}${safeLink}`) : null;
45     const logo = logoAttached
46         ? ``
47         : `<span style="color:#073e2b;font-size:20px;font-weight:800;letter-spacing:1px">MARKETLINK <span style="color:#d69924">AGRI-HUB</span></span>`;
48
49     return `<div style="background:#f6f8f4;padding:24px;font-family:Arial,Helvetica,sans-serif;color:#172b24">
50     <div style="max-width:520px;margin:auto;border:1px solid #dce9df;border-radius:16px;overflow:hidden;ba

```

```

    ckground:#fff">
50     <div style="padding:18px 22px;background:#fff;border-bottom:3px solid #d69924">${logo}</div>
51     <div style="padding:24px 22px;line-height:1.6">
52         <h1 style="color:#01411c;font-size:19px;line-height:1.3;margin:0 0 12px">${escapeHtml(title)}</h1>
53         <p style="margin:0;color:#334155;white-space:normal">${escapeHtml(body).replace(/\n/g, "<br>")}</p>
    >
54         ${destination ? `<p style="margin:20px 0 0"><a href="${destination}" style="display:inline-block;b
    ackground:#117e48;color:#fff;text-decoration:none;padding:11px 18px;border-radius:9px;font-weight:700">Vie
    w on MarketLink</a></p>` : ""}
55     </div>
56     <div style="padding:12px 22px;color:#64748b;font-size:12px;border-top:1px solid #edf2ee">You receive
    d this because you have a ${APP_NAME} account.</div>
57 </div>
58 </div>`;
59 }
60
61 export async function sendEmail(to: string, subject: string, text: string, link?: string): Promise<"sent"
    | "logged" | "failed"> {
62     if (!emailConfigured()) {
63         console.log(`[outbox:email] to=${to} subject="${subject}" | ${text.replace(/\n/g, " ")}`);
64         return "logged";
65     }
66     try {
67         const logo = await loadEmailLogo();
68         const res = await fetch("https://api.resend.com/emails", {
69             method: "POST",
70             headers: { Authorization: `Bearer ${process.env.RESEND_API_KEY}`, "Content-Type": "application/json"
    },
71             body: JSON.stringify({
72                 from: process.env.EMAIL_FROM || `${APP_NAME} <onboarding@resend.dev>`,
73                 to: [to],
74                 subject,
75                 text,
76                 html: emailHtml(subject, text, link, !!logo),
77                 ...(logo ? { attachments: [{ content: logo, filename: "marketlink-email.png", content_id: LOGO_CID
    }} : {}),
78             })),
79         });
80         if (!res.ok) throw new Error(`${res.status} ${await res.text()}`);
81         return "sent";
82     } catch (error) {
83         console.error("[notify] email failed", error);
84         return "failed";
85     }
86 }
87
88 /** Normalise Pakistani numbers (03xx...) to E.164 (+923xx...). */
89 export function toE164(phone: string) {
90     const d = phone.replace(/^[^d+]/g, "");
91     if (d.startsWith("+")) return d;
92     if (d.startsWith("92")) return `+${d}`;
93     if (d.startsWith("0")) return `+92${d.slice(1)}`;
94     return `+92${d}`;
95 }
96
97 export async function sendSms(phone: string, text: string): Promise<"sent" | "logged" | "failed"> {
98     if (!smsConfigured()) {
99         console.log(`[outbox:sms] to=${toE164(phone)} | ${text}`);
100         return "logged";
101     }
102     try {
103         const sid = process.env.TWILIO_ACCOUNT_SID!;
104         const res = await fetch(`https://api.twilio.com/2010-04-01/Accounts/${sid}/Messages.json`, {
105             method: "POST",
106             headers: { Authorization: "Basic " + Buffer.from(`${sid}:${process.env.TWILIO_AUTH_TOKEN}`).toString
    ("base64"), "Content-Type": "application/x-www-form-urlencoded" },
107             body: new URLSearchParams({ To: toE164(phone), From: process.env.TWILIO_FROM!, Body: text.slice(0, 4
    80) })),
108         });
109         if (!res.ok) throw new Error(`${res.status} ${await res.text()}`);
110         return "sent";
111     } catch (error) {
112         console.error("[notify] sms failed", error);
113         return "failed";
114     }
115 }

```

```

116
117 export type NotifyInput = { type: string; title: string; body: string; link?: string; email?: boolean; sms
  ↳ ??: boolean };
118
119 /** Notification failure never rolls back bidding, order or review transactions. */
120 export async function notify(userId: number, n: NotifyInput) {
121   try {
122     const [user] = await db.select({ email: users.email, phone: users.phone }).from(users).where(eq(users.
  ↳ id, userId)).limit(1);
123     if (!user) return;
124     const channels = ["in_app"];
125     if (n.email) channels.push(`email:${await sendEmail(user.email, n.title, n.body, n.link)}`);
126     if (n.sms && user.phone) channels.push(`sms:${await sendSms(user.phone, `${APP_NAME}: ${n.title}. ${n.
  ↳ body}`)}`);
127     await db.insert(notifications).values({ userId, type: n.type, title: n.title, body: n.body, link: n.li
  ↳ nk, channels: channels.join(",") });
128   } catch (error) {
129     console.error("[notify] failed", error);
130   }
131 }
132
133 export function notifyMany(list: [number, NotifyInput][] ) {
134   return Promise.all(list.map(([id, n]) => notify(id, n)));
135 }
136

```

src/lib/otp.ts

```

1  import { createHash, randomInt } from "crypto";
2  import { and, desc, eq, gt, isNull } from "drizzle-orm";
3  import { db } from "@db";
4  import { authTokens } from "@db/schema";
5  import { ApiError } from "@lib/api";
6  import { emailConfigured, sendEmail, sendSms } from "@lib/notify";
7
8  export type OtpPurpose = "verify_email" | "reset_password";
9  const TTL_MIN = 10;
10 const MAX_ATTEMPTS = 5;
11 const RESEND_COOLDOWN_S = 45;
12 const SECRET = process.env.SESSION_SECRET || "marketlink-agri-hub-dev-secret-change-me";
13
14 const hash = (code: string) => createHash("sha256").update(`${code}:${SECRET}`).digest("hex");
15
16 /**
17  * Issues a 6-digit code, invalidating older ones. Sends it by email (+ SMS copy if phone given).
18  * Returns `devCode` only when no email provider is configured, so local/dev installs remain usable.
19  */
20 export async function issueOtp(user: { id: number; email: string; phone?: string | null }, purpose: OtpPur
  ↳ pose) {
21   const [last] = await db
22     .select({ createdAt: authTokens.createdAt })
23     .from(authTokens)
24     .where(and(eq(authTokens.userId, user.id), eq(authTokens.purpose, purpose)))
25     .orderBy(desc(authTokens.createdAt))
26     .limit(1);
27   if (last && Date.now() - new Date(last.createdAt).getTime() < RESEND_COOLDOWN_S * 1000) {
28     throw new ApiError(429, `Please wait ${RESEND_COOLDOWN_S} seconds before requesting another code`);
29   }
30   const code = String(randomInt(0, 1_000_000)).padStart(6, "0");
31   await db.update(authTokens).set({ usedAt: new Date() }).where(and(eq(authTokens.userId, user.id), eq(auth
  ↳ Tokens.purpose, purpose), isNull(authTokens.usedAt)));
32   await db.insert(authTokens).values({ userId: user.id, purpose, codeHash: hash(code), expiresAt: new Date
  ↳ (Date.now() + TTL_MIN * 60_000) });
33
34   const subject = purpose === "verify_email" ? "Verify your MarketLink account" : "Reset your MarketLink p
  ↳ assword";
35   const text = `Your MarketLink ${purpose === "verify_email" ? "verification" : "password reset"} code is
  ↳ ${code}. It expires in ${TTL_MIN} minutes. If you didn't request this, ignore this message.`;
36   await sendEmail(user.email, subject, text);
37   if (user.phone && purpose === "reset_password") await sendSms(user.phone, `MarketLink code: ${code} (val
  ↳ id ${TTL_MIN} min)`);
38   return { sent: true, expiresInMinutes: TTL_MIN, ...(emailConfigured() ? {} : { devCode: code }) };
39 }
40
41 export async function consumeOtp(userId: number, purpose: OtpPurpose, code: string) {

```

```

42     const [tok] = await db
43       .select()
44       .from(authTokens)
45       .where(and(eq(authTokens.userId, userId), eq(authTokens.purpose, purpose), isNull(authTokens.usedAt),
46         ↵ gt(authTokens.expiresAt, new Date()))))
47       .orderBy(desc(authTokens.createdAt))
48       .limit(1);
49     if (!tok) throw new ApiError(400, "Code expired or not found. Request a new one.");
50     if (tok.attempts >= MAX_ATTEMPTS) throw new ApiError(429, "Too many wrong attempts. Request a new code."
51     ↵ );
52     if (tok.codeHash !== hash(code.trim())) {
53       await db.update(authTokens).set({ attempts: tok.attempts + 1 }).where(eq(authTokens.tokenId, tok.token
54       ↵ Id));
55       throw new ApiError(400, `Incorrect code (${MAX_ATTEMPTS - tok.attempts - 1} attempts left)`);
56     }
57     await db.update(authTokens).set({ usedAt: new Date() }).where(eq(authTokens.tokenId, tok.tokenId));
58   }
59 }

```

src/lib/password.ts

```

1  import bcrypt from "bcryptjs";
2  import { randomBytes, scryptSync, timingSafeEqual } from "crypto";
3
4  const BCrypt_Rounds = 10;
5
6  /** New passwords are hashed with bcrypt (cost 10). */
7  export function hashPassword(pw: string) {
8    return bcrypt.hashSync(pw, BCrypt_Rounds);
9  }
10
11 /** Hash that can never match a typed password (used for Google-only accounts). */
12 export function unusablePasswordHash() {
13   return hashPassword(randomBytes(32).toString("hex"));
14 }
15
16 export const isLegacyHash = (stored: string) => !stored.startsWith("$2");
17
18 /** Verifies bcrypt hashes and legacy scrypt `salt:hash` values from earlier versions. */
19 export function verifyPassword(pw: string, stored: string) {
20   if (!stored) return false;
21   if (!isLegacyHash(stored)) return bcrypt.compareSync(pw, stored);
22   const [salt, hash] = stored.split(":");
23   if (!salt || !hash) return false;
24   const candidate = scryptSync(pw, salt, 64);
25   const expected = Buffer.from(hash, "hex");
26   return expected.length === candidate.length && timingSafeEqual(candidate, expected);
27 }
28

```

src/lib/reset.ts

```

1  import { createHash, randomBytes } from "crypto";
2  import { and, desc, eq, gt, isNull } from "drizzle-orm";
3  import { db } from "@db";
4  import { authTokens, users } from "@db/schema";
5  import { ApiError } from "@lib/api";
6  import { publicOrigin } from "@lib/base-url";
7  import { emailConfigured, notify, sendEmail } from "@lib/notify";
8  import { hashPassword } from "@lib/password";
9
10 const PURPOSE = "reset_link";
11 export const RESET_TTL_MIN = 15;
12 const COOLDOWN_S = 30;
13 const sha = (t: string) => createHash("sha256").update(t).digest("hex");
14
15 /**
16  * On-screen "simulated email" links are a development aid used ONLY when no email provider is configured.
17  * They are never shown for staff accounts (admin / inspector), and can be switched off with AUTH_DEV_LINK
18  * S=false.
19  */
20 function devLinksAllowed(role: string) {
21   return !emailConfigured() && process.env.AUTH_DEV_LINKS !== "false" && role !== "admin" && role !== "ins
22   ↵ pector";
23 }

```

```

21 }
22
23 // Public origin is resolved centrally (request headers → validated env → loopback fallback),
24 // so reset links always point at the host the user is actually browsing.
25 const baseUrl = (req: Request) => publicOrigin(req);
26
27 /** POST /api/auth/forgot-password: always returns the same message so account existence isn't revealed. */
28 /
29 export async function requestPasswordReset(emailRaw: string, req: Request) {
30   const email = emailRaw.trim().toLowerCase();
31   const generic = { sent: true, message: `If an account exists for ${email}, a reset link valid for ${RESET_TTL_MIN} minutes has been sent.` };
32   const [u] = await db.select({ id: users.id, email: users.email, role: users.role }).from(users).where(eq(users.email, email)).limit(1);
33   if (!u) return generic;
34
35   const [last] = await db.select({ createdAt: authTokens.createdAt }).from(authTokens)
36     .where(and(eq(authTokens.userId, u.id), eq(authTokens.purpose, PURPOSE))).orderBy(desc(authTokens.createdAt)).limit(1);
37   if (last && Date.now() - new Date(last.createdAt).getTime() < COOLDOWN_S * 1000) {
38     throw new ApiError(429, `A reset link was just sent. Please wait ${COOLDOWN_S} seconds before requesting another.`);
39   }
40
41   const token = randomBytes(32).toString("base64url");
42   await db.delete(authTokens).where(and(eq(authTokens.userId, u.id), eq(authTokens.purpose, PURPOSE)));
43   await db.insert(authTokens).values({ userId: u.id, purpose: PURPOSE, codeHash: sha(token), expiresAt: new Date(Date.now() + RESET_TTL_MIN * 60_000) });
44
45   const path = `/reset-password?token=${token}`;
46   const link = `${baseUrl(req)}${path}`;
47   await sendEmail(
48     u.email,
49     "Reset your MarketLink password",
50     `We received a request to reset your MarketLink password.\n\nOpen this link to choose a new password (valid for ${RESET_TTL_MIN} minutes, single use):\n${link}\n\nIf you didn't request this, you can ignore this email. Your password won't change.`
51   );
52   return devLinksAllowed(u.role) ? { ...generic, devResetUrl: path } : generic;
53 }
54
55 async function findValid(token: string) {
56   if (!/^[A-Za-z0-9_-]{20,100}$/.test(token)) return null;
57   const [row] = await db
58     .select({ tokenId: authTokens.tokenId, userId: authTokens.userId, expiresAt: authTokens.expiresAt, email: users.email })
59     .from(authTokens)
60     .innerJoin(users, eq(users.id, authTokens.userId))
61     .where(and(eq(authTokens.codeHash, sha(token)), eq(authTokens.purpose, PURPOSE), isNull(authTokens.usedAt), gt(authTokens.expiresAt, new Date()))))
62     .limit(1);
63   return row ?? null;
64 }
65
66 /** GET /api/auth/reset-password?token=... : lets the page show "link expired" before the user types anything. */
67 export async function inspectResetToken(token: string) {
68   const row = await findValid(token);
69   if (!row) throw new ApiError(400, "This reset link is invalid or has expired. Please request a new one.");
70   const [name, domain] = row.email.split("@");
71   return { valid: true, email: `${name.slice(0, 2)}${".".repeat(Math.max(1, name.length - 2))}@${domain}`, expiresAt: row.expiresAt };
72 }
73
74 /** POST /api/auth/reset-password: validates the token, stores a bcrypt hash, and clears the token (single use). */
75 export async function resetPassword(token: string, newPassword: string) {
76   if (newPassword.length < 8) throw new ApiError(422, "Password must be at least 8 characters");
77   if (!/[A-Za-z]/.test(newPassword) || !/\d/.test(newPassword)) throw new ApiError(422, "Use at least one letter and one number");
78   const row = await findValid(token);
79   if (!row) throw new ApiError(400, "This reset link is invalid or has expired. Please request a new one.");
80   await db.transaction(async (tx) => {
81     await tx.update(users).set({ passwordHash: hashPassword(newPassword), emailVerified: true }).where(eq(

```

```

    users.id, row.userId));
81     await tx.delete(authTokens).where(and(eq(authTokens.userId, row.userId), eq(authTokens.purpose, PURPOSE)));
    E));
82   });
83   await notify(row.userId, { type: "security", title: "Your password was changed", body: "If this wasn't you, reset your password immediately and contact MarketLink support.", email: true });
    ou, reset your password immediately and contact MarketLink support.", email: true });
84   return { reset: true, email: row.email };
85 }
86

```

src/lib/seed.ts

```

1  import { sql } from "drizzle-orm";
2  import { db, ensureDatabaseExists } from "@db";
3  import {
4    users,
5    cropsInventory,
6    mandiRates,
7    bidsNegotiations,
8    ordersLogistics,
9    orderEvents,
10   qualityInspections,
11   walletTransactions,
12   disputes,
13   reviews,
14 } from "@db/schema";
15 import { CITIES, CROP_CATALOGUE, MANDI_MARKETS, DELIVERY_STAGES, PLATFORM_FEE_RATE, type DeliveryStage, type PaymentStatus } from "@lib/constants";
    pe PaymentStatus } from "@lib/constants";
16 import { hashPassword } from "@lib/password";
17 import { POSTGRES_DDL } from "@lib/ddl-postgres";
18 import { SUPER_ADMIN, backfillData, needsUpgrade, upgradeSchema } from "@lib/migrate";
19 import { recomputeAllTrust } from "@lib/services/reviews";
20
21 let seedPromise: Promise<void> | null = null;
22
23 export function ensureSeeded() {
24   if (!seedPromise) {
25     seedPromise = runSeed().catch((e) => {
26       seedPromise = null;
27       console.error("[seed] failed", e);
28     });
29   }
30   return seedPromise;
31 }
32
33 function rng(seed: number) {
34   let s = seed;
35   return () => {
36     s = (s * 16807) % 2147483647;
37     return (s - 1) / 2147483646;
38   };
39 }
40
41 const iso = (d: Date) => d.toISOString().slice(0, 10);
42 const daysAgo = (n: number) => {
43   const d = new Date();
44   d.setDate(d.getDate() - n);
45   return d;
46 };
47 const round2 = (n: number) => Math.round(n * 100) / 100;
48
49 /** Creates all tables/enums/indexes on an empty database (fresh local or cloud Postgres). */
50 async function ensureSchema() {
51   await ensureDatabaseExists();
52   const res = await db.execute(sql`select to_regclass('public.users') is not null as ok`);
53   if ((res.rows[0] as { ok: boolean }).ok) return;
54   console.log("[bootstrap] empty database detected - creating MarketLink schema");
55   // Run statements one by one (prepared-statement drivers reject multi-statement strings).
56   const statements = POSTGRES_DDL.split(/;\s*(?:\n|$)/).map((s) => s.trim()).filter(Boolean);
57   await db.transaction(async (tx) => {
58     await tx.execute(sql`select pg_advisory_xact_lock(424241)`);
59     const again = await tx.execute(sql`select to_regclass('public.users') is not null as ok`);
60     if ((again.rows[0] as { ok: boolean }).ok) return;
61     for (const stmt of statements) await tx.execute(sql.raw(stmt));
62   });

```

```

63     console.log("[bootstrap] schema created");
64 }
65
66 async function runSeed() {
67     await ensureSchema();
68     const [{ count }] = (await db.execute(sql`select count(*)::int as count from users`)).rows as { count: number }[];
69     if (count > 0) {
70         if (await needsUpgrade()) await upgradeSchema();
71         await backfillData();
72         const [{ n }] = (await db.execute(sql`select count(*)::int as n from reviews`)).rows as { n: number }[];
73         if (Number(n) === 0) {
74             await seedReviewsIfEmpty();
75             await recomputeAllTrust();
76         }
77         return;
78     }
79
80     await db.transaction(async (tx) => {
81         await tx.execute(sql`select pg_advisory_xact_lock(424242)`);
82         const again = (await tx.execute(sql`select count(*)::int as count from users`)).rows as { count: number }[];
83         if (again[0].count > 0) return;
84
85         const rand = rng(20240611);
86         const pw = hashPassword("password123");
87         // Self-hosters can set their own admin login in .env before the first start.
88         const adminEmail = (process.env.ADMIN_EMAIL || SUPER_ADMIN.email).trim().toLowerCase();
89         const adminPw = process.env.ADMIN_PASSWORD ? hashPassword(process.env.ADMIN_PASSWORD) : pw;
90
91         /* ----- Users ----- */
92         const u = await tx
93             .insert(users)
94             .values([
95                 { fullName: SUPER_ADMIN.fullName, email: adminEmail, passwordHash: adminPw, phone: "+92 300 1112233", role: "admin" as const, cnicId: "35202-1234567-1", city: "Lahore", address: "MarketLink HQ, Gulberg II I, Lahore", isVerified: true, businessName: "MarketLink Platform Management", title: SUPER_ADMIN.title, userCode: SUPER_ADMIN.userCode },
96                 { fullName: "Dr. Farhan Malik", email: "inspector@marketlink.pk", passwordHash: pw, phone: "+92 300 7788990", role: "inspector" as const, cnicId: "35202-9988776-3", city: "Faisalabad", address: "Ayub Agri cultural Research Institute, Faisalabad", isVerified: true, businessName: "MarketLink Quality Lab" },
97                 { fullName: "Muhammad Aslam", email: "farmer@marketlink.pk", passwordHash: pw, phone: "+92 301 445 5667", role: "farmer" as const, cnicId: "36302-7654321-3", city: "Multan", address: "Chak 5 Faiz, Shujabad Road, Multan", isVerified: true, businessName: "Aslam Agri Farms" },
98                 { fullName: "Ghulam Rasool", email: "rasool@marketlink.pk", passwordHash: pw, phone: "+92 302 9988 776", role: "farmer" as const, cnicId: "36502-1122334-5", city: "Sahiwal", address: "Village 88/6-R, Sahiwal", isVerified: true, businessName: "Rasool & Sons Kheti" },
99                 { fullName: "Rana Tariq", email: "tariq@marketlink.pk", passwordHash: pw, phone: "+92 333 5566778", role: "farmer" as const, cnicId: "33100-5566778-9", city: "Faisalabad", address: "Jaranwala Road, Faisalabad", isVerified: false, businessName: "Rana Organic Fields" },
100                { fullName: "Bashir Ahmed Soomro", email: "bashir@marketlink.pk", passwordHash: pw, phone: "+92 345 2233445", role: "farmer" as const, cnicId: "41303-9988776-1", city: "Hyderabad", address: "Tando Jam Road, Hyderabad", isVerified: true, businessName: "Soomro Tomato Growers" },
101                { fullName: "Nasreen Bibi", email: "nasreen@marketlink.pk", passwordHash: pw, phone: "+92 312 6677 889", role: "farmer" as const, cnicId: "35401-3344556-2", city: "Okara", address: "Renala Khurd, Okara", isVerified: false, businessName: "Nasreen Potato Farm" },
102                { fullName: "Abdul Wahid Baloch", email: "wahid@marketlink.pk", passwordHash: pw, phone: "+92 333 8123456", role: "farmer" as const, cnicId: "52101-4455667-7", city: "Turbat", address: "Date Palm Orchards, Kech Valley, Turbat", isVerified: true, businessName: "Kech Valley Dates" },
103                { fullName: "Imran Qureshi", email: "buyer@marketlink.pk", passwordHash: pw, phone: "+92 321 12345 67", role: "buyer" as const, cnicId: "35202-8877665-5", city: "Lahore", address: "Badami Bagh Grain Market, Lahore", isVerified: true, businessName: "Qureshi Wholesale Traders", walletBalance: 4500000 },
104                { fullName: "Sana Mirza", email: "metro@marketlink.pk", passwordHash: pw, phone: "+92 322 7654321", role: "buyer" as const, cnicId: "42101-1234321-8", city: "Karachi", address: "Korangi Industrial Area, Karachi", isVerified: true, businessName: "FreshMart Supermarket Supply", walletBalance: 3200000 },
105                { fullName: "Faisal Sheikh", email: "exports@marketlink.pk", passwordHash: pw, phone: "+92 300 876 5432", role: "buyer" as const, cnicId: "42201-5678765-3", city: "Karachi", address: "Port Qasim, Karachi", isVerified: true, businessName: "Indus Agro Exports", walletBalance: 8000000 },
106                { fullName: "Hamza Butt", email: "processor@marketlink.pk", passwordHash: pw, phone: "+92 311 2345 678", role: "buyer" as const, cnicId: "33100-8765678-1", city: "Faisalabad", address: "Sargodha Road Industrial Estate, Faisalabad", isVerified: true, businessName: "Punjab Food Processors Ltd", walletBalance: 2500000 },
107            ])
108            .returning({ id: users.id, email: users.email, city: users.city });
109     const id = (email: string) => u.find((x) => x.email === email)!.id;

```

```

110     const admin = id(adminEmail);
111     const inspector = id("inspector@marketlink.pk");
112     await tx.execute(sql`update users set email_verified = true`);
113     const aslam = id("farmer@marketlink.pk");
114     const rasool = id("rasool@marketlink.pk");
115     const tariq = id("tariq@marketlink.pk");
116     const bashir = id("bashir@marketlink.pk");
117     const nasreen = id("nasreen@marketlink.pk");
118     const wahid = id("wahid@marketlink.pk");
119     const imran = id("buyer@marketlink.pk");
120     const metro = id("metro@marketlink.pk");
121     const exportsB = id("exports@marketlink.pk");
122     const processor = id("processor@marketlink.pk");
123
124     /* ----- Mandi rates (14 days x 10 crops x 6 markets) ----- */
125     const marketBias: Record<string, number> = { Lahore: 1.04, Multan: 0.96, Faisalabad: 0.98, Karachi: 1.
126     08, Hyderabad: 0.97, Islamabad: 1.1, Peshawar: 1.05, Quetta: 1.07, Sukkur: 0.95, Gujranwala: 1.0 };
127     const rateRows: (typeof mandiRates.$inferInsert)[] = [];
128     for (const crop of CROP_CATALOGUE) {
129         for (const m of MANDI_MARKETS) {
130             let price = crop.mandiBase * marketBias[m];
131             for (let d = 13; d >= 0; d--) {
132                 price = price * (1 + (rand() - 0.48) * 0.05);
133                 const spread = price * (0.06 + rand() * 0.06);
134                 rateRows.push({
135                     cropName: crop.name,
136                     marketLocation: m,
137                     minPricePerKg: round2(price - spread),
138                     maxPricePerKg: round2(price + spread),
139                     avgPricePerKg: round2(price),
140                     source: d === 0 ? "feed" : "manual",
141                     dateUpdated: iso(daysAgo(d)),
142                 });
143             }
144         }
145     }
146     await tx.insert(mandiRates).values(rateRows);
147
148     /* ----- Crops ----- */
149     const img = (name: string) => [CROP_CATALOGUE.find((c) => c.name === name)!.image];
150     const geo = (city: string) => ({ farmLocation: city, latitude: CITIES[city].lat, longitude: CITIES[city].lng });
151     const crops = await tx
152         .insert(cropsInventory)
153         .values([
154             { farmerId: aslam, cropName: "Wheat", category: "grains" as const, qualityGrade: "A" as const, totalQuantityKg: 24000, basePricePerKg: 96, harvestDate: iso(daysAgo(20)), imagesJson: img("Wheat"), description: "Galaxy-2013 variety, sun-dried, cleaned & bagged in 50kg PP bags.", ...geo("Multan") },
155             { farmerId: aslam, cropName: "Mango (Chaunsa)", category: "fruits" as const, qualityGrade: "A" as const, totalQuantityKg: 8000, basePricePerKg: 185, harvestDate: iso(daysAgo(3)), imagesJson: img("Mango (Chaunsa)"), description: "Sindhri & Chaunsa mix, export-grade, packed in 10kg cartons.", ...geo("Multan") },
156             { farmerId: aslam, cropName: "Cotton (Phutti)", category: "cash_crops" as const, qualityGrade: "B" as const, totalQuantityKg: 15000, basePricePerKg: 205, harvestDate: iso(daysAgo(30)), imagesJson: img("Cotton (Phutti)"), description: "Phutti (seed cotton), trash below 5%.", ...geo("Multan") },
157             { farmerId: rasool, cropName: "Basmati Rice", category: "grains" as const, qualityGrade: "A" as const, totalQuantityKg: 18000, basePricePerKg: 295, harvestDate: iso(daysAgo(40)), imagesJson: img("Basmati Rice"), description: "Super Kernel Basmati, 1121 grade, aged 6 months.", ...geo("Sahiwal") },
158             { farmerId: rasool, cropName: "Maize", category: "grains" as const, qualityGrade: "B" as const, totalQuantityKg: 30000, basePricePerKg: 68, harvestDate: iso(daysAgo(15)), imagesJson: img("Maize"), description: "Yellow hybrid maize for feed mills, moisture ~14%.", ...geo("Sahiwal") },
159             { farmerId: rasool, cropName: "Potato", category: "vegetables" as const, qualityGrade: "A" as const, totalQuantityKg: 20000, basePricePerKg: 52, harvestDate: iso(daysAgo(8)), imagesJson: img("Potato"), description: "Cardinal red potatoes, cold-store ready.", ...geo("Sahiwal") },
160             { farmerId: tariq, cropName: "Sugarcane", category: "cash_crops" as const, qualityGrade: "B" as const, totalQuantityKg: 120000, basePricePerKg: 11, harvestDate: iso(daysAgo(5)), imagesJson: img("Sugarcane"), description: "CPF-249 variety, high sucrose content.", status: "pending_inspection" as const, ...geo("Faisalabad") },
161             { farmerId: tariq, cropName: "Wheat", category: "grains" as const, qualityGrade: "C" as const, totalQuantityKg: 10000, basePricePerKg: 88, harvestDate: iso(daysAgo(25)), imagesJson: img("Wheat"), description: "Mixed variety, suitable for flour mills.", ...geo("Faisalabad") },
162             { farmerId: bashir, cropName: "Tomato", category: "vegetables" as const, qualityGrade: "A" as const, totalQuantityKg: 6000, basePricePerKg: 115, harvestDate: iso(daysAgo(2)), imagesJson: img("Tomato"), description: "Fresh Sindh tomatoes, crated (25kg).", ...geo("Hyderabad") },
163             { farmerId: bashir, cropName: "Onion", category: "vegetables" as const, qualityGrade: "B" as const, totalQuantityKg: 14000, basePricePerKg: 105, harvestDate: iso(daysAgo(10)), imagesJson: img("Onion"), de

```

```

163     scription: "Red onion, Phulkara variety, netted 40kg bags.", ...geo("Hyderabad") },
164     { farmerId: bashir, cropName: "Mango (Chaunsa)", category: "fruits" as const, qualityGrade: "B" as
165     const, totalQuantityKg: 5000, basePricePerKg: 160, harvestDate: iso(daysAgo(4)), imagesJson: img("Mango (
166     Chaunsa)"), description: "Sindhri mango from Mirpur Khas orchards.", ...geo("Hyderabad") },
167     { farmerId: nasreen, cropName: "Potato", category: "vegetables" as const, qualityGrade: "B" as con
168     st, totalQuantityKg: 25000, basePricePerKg: 48, harvestDate: iso(daysAgo(12)), imagesJson: img("Potato"),
169     description: "Diamant white potatoes for chips processing.", ...geo("Okara") },
170     { farmerId: wahid, cropName: "Dates (Khajoor)", category: "fruits" as const, qualityGrade: "A" as
171     const, totalQuantityKg: 12000, basePricePerKg: 255, harvestDate: iso(daysAgo(9)), imagesJson: img("Dates (
172     Khajoor)"), description: "Begum Jangi & Muzawati dates, sun-cured, 5kg boxes.", ...geo("Turbat") },
173     { farmerId: bashir, cropName: "Red Chilli (Kunri)", category: "cash_crops" as const, qualityGrade:
174     "A" as const, totalQuantityKg: 4000, basePricePerKg: 470, harvestDate: iso(daysAgo(14)), imagesJson: img(
175     "Red Chilli (Kunri)"), description: "Kunri long red chilli, sun-dried, stemless.", ...geo("Umerkot") },
176     { farmerId: tariq, cropName: "Chickpea (Chana)", category: "grains" as const, qualityGrade: "B" as
177     const, totalQuantityKg: 9000, basePricePerKg: 235, harvestDate: iso(daysAgo(18)), imagesJson: img("Chickp
178     ea (Chana)"), description: "Thal desi chana, cleaned, 100kg bags.", ...geo("Jhang") },
179     { farmerId: nasreen, cropName: "Kinnow", category: "fruits" as const, qualityGrade: "A" as const,
180     totalQuantityKg: 9000, basePricePerKg: 92, harvestDate: iso(daysAgo(6)), imagesJson: img("Kinnow"), descri
181     ption: "Waxed and graded kinnow, export count 88-100.", ...geo("Okara") },
182   ])
183   .returning();
184   const crop = (farmerId: number, name: string) => crops.find((c) => c.farmerId === farmerId && c.cropName === name)!;
185
186   /* ----- Inspections ----- */
187   await tx.insert(qualityInspections).values([
188     { cropId: crop(aslam, "Wheat").cropId, inspectorId: inspector, gradeAssigned: "A" as const, moisture
189     LevelPercentage: 11.5, soilPh: 7.4, inspectionNotes: "Uniform grain size, no pest damage, low foreign matt
190     er (0.8%).", status: "passed" as const, verifiedAt: daysAgo(18) },
191     { cropId: crop(rasool, "Basmati Rice").cropId, inspectorId: inspector, gradeAssigned: "A" as const,
192     moistureLevelPercentage: 12.1, soilPh: 7.1, inspectionNotes: "Average grain length 8.3mm, broken < 2%.", s
193     tatus: "passed" as const, verifiedAt: daysAgo(35) },
194     { cropId: crop(tariq, "Sugarcane").cropId, inspectorId: inspector, gradeAssigned: "B" as const, mois
195     tureLevelPercentage: 70, soilPh: 7.9, inspectionNotes: "Awaiting brix-level lab result.", status: "pending
196     " as const, verifiedAt: null },
197     { cropId: crop(bashir, "Tomato").cropId, inspectorId: inspector, gradeAssigned: "A" as const, moistu
198     reLevelPercentage: 93.5, soilPh: 6.8, inspectionNotes: "Firm, uniform colour, shelf life est. 7 days.", st
199     atus: "passed" as const, verifiedAt: daysAgo(1) },
200   ]);
201
202   /* ----- Live bids ----- */
203   await tx.insert(bidsNegotiations).values([
204     { cropId: crop(aslam, "Wheat").cropId, buyerId: imran, bidPricePerKg: 92, bidQuantityKg: 10000, targ
205     etDeliveryDate: iso(daysAgo(-7)), message: "Need for flour mill. Can pick up from farm.", status: "pending
206     " as const },
207     { cropId: crop(aslam, "Wheat").cropId, buyerId: processor, bidPricePerKg: 94, bidQuantityKg: 8000, t
208     argetDeliveryDate: iso(daysAgo(-10)), message: "Regular monthly requirement.", status: "pending" as const
209     },
210     { cropId: crop(aslam, "Mango (Chaunsa)").cropId, buyerId: exportsB, bidPricePerKg: 175, bidQuantityK
211     g: 5000, targetDeliveryDate: iso(daysAgo(-5)), message: "For Dubai shipment, need hot-water treatment.", s
212     tatus: "countered" as const, counterPrice: 182, counterNote: "Best I can do is 182 with cartons included."
213     },
214     { cropId: crop(aslam, "Cotton (Phutti)").cropId, buyerId: processor, bidPricePerKg: 198, bidQuantity
215     Kg: 6000, targetDeliveryDate: iso(daysAgo(-14)), status: "pending" as const },
216     { cropId: crop(bashir, "Tomato").cropId, buyerId: metro, bidPricePerKg: 110, bidQuantityKg: 2000, ta
217     rgetDeliveryDate: iso(daysAgo(-2)), message: "Daily supermarket supply.", status: "pending" as const },
218     { cropId: crop(rasool, "Potato").cropId, buyerId: imran, bidPricePerKg: 45, bidQuantityKg: 5000, tar
219     getDeliveryDate: iso(daysAgo(-6)), status: "rejected" as const },
220     { cropId: crop(nasreen, "Kinnow").cropId, buyerId: exportsB, bidPricePerKg: 88, bidQuantityKg: 4000,
221     targetDeliveryDate: iso(daysAgo(-9)), status: "countered" as const, counterPrice: 91, counterNote: "Waxin
222     g cost included." },
223   ]);
224
225   /* ----- Orders (current pipeline + 6-month history) ----- */
226   type OrderSeed = { farmer: number; buyer: number; cropName: string; qty: number; price: number; stage:
227     DeliveryStage; payment: PaymentStatus; ago: number; transit?: string };
228   const orderSeeds: OrderSeed[] = [
229     { farmer: aslam, buyer: imran, cropName: "Wheat", qty: 5000, price: 95, stage: "in_transit", payment
230     : "escrow_locked", ago: 4, transit: "Khanewal Bypass, N-5" },
231     { farmer: rasool, buyer: exportsB, cropName: "Basmati Rice", qty: 6000, price: 292, stage: "dispatch
232     ed", payment: "escrow_locked", ago: 3, transit: "Sahiwal Dry Port" },
233     { farmer: bashir, buyer: metro, cropName: "Tomato", qty: 1500, price: 112, stage: "quality_checked",
234     payment: "escrow_locked", ago: 2 },
235     { farmer: aslam, buyer: processor, cropName: "Cotton (Phutti)", qty: 3000, price: 202, stage: "confi
236     rmed", payment: "awaiting_escrow", ago: 1 },
237     { farmer: rasool, buyer: imran, cropName: "Maize", qty: 8000, price: 67, stage: "delivered", payment

```

```

    : "disputed", ago: 9, transit: "Badami Bagh, Lahore" },
200     { farmer: nasreen, buyer: processor, cropName: "Potato", qty: 7000, price: 47, stage: "confirmed", p
    ayment: "escrow_locked", ago: 1 },
201   ];
202   // History: delivered + released orders spread over past 6 months
203   const histCrops: [number, string, number[][]] = [
204     [aslam, "Wheat", [imran, processor]],
205     [aslam, "Mango (Chaunsa)", [exportsB, metro]],
206     [rasool, "Basmati Rice", [exportsB, imran]],
207     [bashir, "Onion", [metro, imran]],
208     [nasreen, "Kinnow", [exportsB]],
209     [wahid, "Dates (Khajoor)", [exportsB, metro]],
210     [rasool, "Potato", [processor, metro]],
211   ];
212   for (let m = 5; m >= 0; m--) {
213     const n = 3 + Math.floor(rand() * 3);
214     for (let k = 0; k < n; k++) {
215       const [farmer, cropName, buyers] = histCrops[Math.floor(rand() * histCrops.length)];
216       const c = crop(farmer, cropName);
217       orderSeeds.push({
218         farmer,
219         buyer: buyers[Math.floor(rand() * buyers.length)],
220         cropName,
221         qty: Math.round((1000 + rand() * 5000) / 100) * 100,
222         price: round2(c.basePricePerKg * (1.03 + rand() * 0.07)),
223         stage: "delivered",
224         payment: "released",
225         ago: m * 30 + 12 + Math.floor(rand() * 15),
226         transit: c.farmLocation,
227       });
228     }
229   }
230
231   let seq = 1000;
232   for (const o of orderSeeds) {
233     const c = crop(o.farmer, o.cropName);
234     const createdAt = daysAgo(o.ago);
235     const [bid] = await tx
236       .insert(bidsNegotiations)
237       .values({ cropId: c.cropId, buyerId: o.buyer, bidPricePerKg: o.price, bidQuantityKg: o.qty, target
    DeliveryDate: iso(daysAgo(o.ago - 7)), status: "accepted", createdAt, updatedAt: createdAt })
238       .returning();
239     const total = round2(o.qty * o.price);
240     const [order] = await tx
241       .insert(ordersLogistics)
242       .values({
243         bidId: bid.bidId,
244         cropId: c.cropId,
245         farmerId: o.farmer,
246         buyerId: o.buyer,
247         quantityKg: o.qty,
248         pricePerKg: o.price,
249         totalAmount: total,
250         paymentStatus: o.payment,
251         deliveryStage: o.stage,
252         transitLocation: o.transit ?? c.farmLocation,
253         trackingNumber: `ML-${createdAt.getFullYear()}-${++seq}`,
254         expectedDeliveryDate: iso(daysAgo(o.ago - 7)),
255         createdAt,
256         updatedAt: createdAt,
257       })
258       .returning();
259     const upTo = DELIVERY_STAGES.findIndex((s) => s.key === o.stage);
260     const events = DELIVERY_STAGES.slice(0, upTo + 1).map((s, i) => ({
261       orderId: order.orderId,
262       stage: s.key,
263       location: i === 0 ? c.farmLocation : i === upTo ? (o.transit ?? c.farmLocation) : c.farmLocation,
264       note: s.label,
265       createdAt: new Date(createdAt.getTime() + i * 18 * 3600 * 1000),
266     }));
267     await tx.insert(orderEvents).values(events);
268     if (o.payment !== "awaiting_escrow") {
269       await tx.insert(walletTransactions).values({ userId: o.buyer, orderId: order.orderId, type: "escro
    w_lock", amount: total, description: `Escrow locked for ${order.trackingNumber}`, createdAt });
270     }
271     if (o.payment === "released") {

```

```

272     const fee = round2(total * PLATFORM_FEE_RATE);
273     const at = new Date(createdAt.getTime() + 5 * 86400000);
274     await tx.insert(walletTransactions).values([
275         { userId: o.buyer, orderId: order.orderId, type: "escrow_release", amount: total, description: `
↳ Escrow released for ${order.trackingNumber}`, createdAt: at },
276         { userId: o.farmer, orderId: order.orderId, type: "payout", amount: round2(total - fee), descrip
↳ tion: `Payout for ${order.trackingNumber}`, createdAt: at },
277         { userId: o.farmer, orderId: order.orderId, type: "platform_fee", amount: fee, description: `1.5
↳ % MarketLink platform fee`, createdAt: at },
278     ]);
279     }
280     if (o.payment === "disputed") {
281         await tx.insert(disputes).values({ orderId: order.orderId, raisedBy: o.buyer, reason: "Received ma
↳ ize with moisture above agreed 14% (measured 17%). Requesting partial refund.", status: "open" });
282     }
283     }
284
285     // One resolved dispute for the log
286     const [firstHist] = (await tx.execute(sql`select order_id from orders_logistics where payment_status =
↳ 'released' order by order_id limit 1`)).rows as { order_id: number }[];
287     if (firstHist) {
288         await tx.insert(disputes).values({ orderId: firstHist.order_id, raisedBy: metro, reason: "Short deli
↳ very of 120kg reported at warehouse.", status: "resolved_release", resolution: "Weighbridge slip verified
↳ full quantity. Escrow released to farmer.", resolvedAt: daysAgo(40), createdAt: daysAgo(43) });
289     }
290
291     // Deposits for buyers + reconcile balances with ledger
292     for (const b of [imran, metro, exportsB, processor]) {
293         await tx.insert(walletTransactions).values({ userId: b, type: "deposit", amount: 10000000, descripti
↳ on: "Opening IBFT transfer from HBL / Meezan Bank (simulated)", createdAt: daysAgo(200) });
294     }
295     await tx.execute(sql`
296         update users set escrow_balance = coalesce((
297             select sum(total_amount) from orders_logistics o
298             where o.buyer_id = users.id and o.payment_status in ('escrow_locked','disputed')
299             ), 0) where role = 'buyer');
300     await tx.execute(sql`
301         update users set wallet_balance = coalesce((
302             select sum(amount) from wallet_transactions w where w.user_id = users.id and w.type = 'payout'
303             ), 0) where role = 'farmer');
304     });
305     await seedReviewsIfEmpty();
306     await recomputeAllTrust();
307     console.log("[seed] MarketLink demo data seeded");
308 }
309
310 const REVIEW_TEXT: [number, string][] = [
311     [5, "Excellent quality, exactly as graded. Truck arrived on time."],
312     [5, "Clean produce, correct weight at our weighbridge. Will order again."],
313     [4, "Good quality, a day late but communication was clear."],
314     [5, "Very professional farmer, packing was export standard."],
315     [4, "Moisture slightly above spec but within tolerance. Fair price."],
316     [3, "Quality OK, some bags had foreign matter. Resolved quickly."],
317     [5, "Best rate compared to our usual arhti. Highly recommended."],
318 ];
319
320 /** Adds realistic buyer reviews to ~75% of paid-out orders (demo data). */
321 export async function seedReviewsIfEmpty() {
322     const [{ n }] = (await db.execute(sql`select count(*)::int as n from reviews`)).rows as { n: number }[];
323     if (Number(n) > 0) return;
324     const rows = (await db.execute(sql`select order_id, buyer_id, farmer_id, updated_at from orders_logistic
↳ s where payment_status = 'released' order by order_id`)).rows as {
325         order_id: number; buyer_id: number; farmer_id: number; updated_at: string;
326     }[];
327     const values = rows
328         .filter((_, i) => i % 4 !== 3)
329         .map((r, i) => {
330             const [rating, comment] = REVIEW_TEXT[i % REVIEW_TEXT.length];
331             return { orderId: Number(r.order_id), buyerId: Number(r.buyer_id), farmerId: Number(r.farmer_id), ra
↳ ting, comment, createdAt: new Date(new Date(r.updated_at).getTime() + 2 * 86400000) };
332         });
333     if (values.length) await db.insert(reviews).values(values);
334 }
335

```

src/lib/services/admin.ts

```
1 import { and, desc, eq, sql } from "drizzle-orm";
2 import { alias } from "drizzle-orm/pg-core";
3 import { db } from "@db";
4 import { users, cropsInventory, qualityInspections } from "@db/schema";
5 import { ApiError } from "@lib/api";
6 import type { SessionUser } from "@lib/auth";
7 import { hashPassword } from "@lib/password";
8
9 export async function listUsers(role?: "farmer" | "buyer" | "admin" | "inspector") {
10   return db
11     .select({
12       id: users.id,
13       fullName: users.fullName,
14       email: users.email,
15       phone: users.phone,
16       role: users.role,
17       cnicId: users.cnicId,
18       businessName: users.businessName,
19       city: users.city,
20       address: users.address,
21       isVerified: users.isVerified,
22       walletBalance: users.walletBalance,
23       createdAt: users.createdAt,
24       trustScore: users.trustScore,
25       ratingAvg: users.ratingAvg,
26       ratingCount: users.ratingCount,
27       emailVerified: users.emailVerified,
28       listings: sql<number>`(select count(*) from crops_inventory c where c.farmer_id = ${users.id})`.mapW
29         ith(Number),
30       orders: sql<number>`(select count(*) from orders_logistics o where o.farmer_id = ${users.id} or o.bu
31         yer_id = ${users.id})`.mapWith(Number),
32     })
33     .from(users)
34     .where(role ? eq(users.role, role) : undefined)
35     .orderBy(users.isVerified, desc(users.createdAt));
36 }
37
38 /** Verifying a farmer activates their listings that were held for verification. */
39 export async function setVerified(userId: number, verified: boolean) {
40   return db.transaction(async (tx) => {
41     const [u] = await tx.update(users).set({ isVerified: verified }).where(eq(users.id, userId)).returning
42     ({ id: users.id, role: users.role, isVerified: users.isVerified, fullName: users.fullName });
43     if (!u) throw new ApiError(404, "User not found");
44     let activated = 0;
45     if (u.role === "farmer" && verified) {
46       const res = await tx
47         .update(cropsInventory)
48         .set({ status: "active" })
49         .where(and(eq(cropsInventory.farmerId, userId), eq(cropsInventory.status, "pending_inspection")))
50         .returning({ id: cropsInventory.cropId });
51       activated = res.length;
52     }
53     return { ...u, listingsActivated: activated };
54   });
55 }
56
57 export async function listInspections() {
58   const inspector = alias(users, "inspector");
59   const farmer = alias(users, "farmer");
60   const rows = await db
61     .select({
62       inspection: qualityInspections,
63       cropName: cropsInventory.cropName,
64       farmLocation: cropsInventory.farmLocation,
65       farmerName: farmer.fullName,
66       inspectorName: inspector.fullName,
67     })
68     .from(qualityInspections)
69     .innerJoin(cropsInventory, eq(cropsInventory.cropId, qualityInspections.cropId))
70     .innerJoin(farmer, eq(farmer.id, cropsInventory.farmerId))
71     .innerJoin(inspector, eq(inspector.id, qualityInspections.inspectorId))
72     .orderBy(desc(qualityInspections.inspectionId))
73     .limit(100);
74 }
```

```

71     return rows.map(({ inspection, ...rest }) => ({ ...inspection, reportAttachment: inspection.reportAttach
72     ↵ ment ? "attached" : null, ...rest }));
73 }
74 export type InspectionInput = {
75     cropId: number;
76     gradeAssigned: "A" | "B" | "C";
77     moistureLevelPercentage: number;
78     soilPh?: number;
79     inspectionNotes?: string;
80     reportAttachment?: string;
81     reportFileName?: string;
82     status: "pending" | "passed" | "failed";
83 };
84
85 /**
86  * Recording an inspection:
87  * passed -> listing grade updated to assigned grade & listing activated
88  * failed -> listing archived and open bids rejected
89  * pending -> listing held in pending_inspection
90  */
91 export async function createInspection(inspection: SessionUser, input: InspectionInput) {
92     if (input.reportAttachment && input.reportAttachment.length > 2_000_000) throw new ApiError(422, "Report
93     ↵ attachment too large (max ~1.5MB)");
94     return db.transaction(async (tx) => {
95         const [crop] = await tx.select().from(cropsInventory).where(eq(cropsInventory.cropId, input.cropId)).f
96         ↵ or("update");
97         if (!crop) throw new ApiError(404, "Crop not found");
98         const [row] = await tx
99             .insert(qualityInspections)
100             .values({ ...input, inspectorId: inspection.id, certificateNo: input.status === "passed" ? `QC-${new
101             ↵ Date().getFullYear()}-${String(Date.now()).slice(-6)}` : null, verifiedAt: input.status === "pending" ? nu
102             ↵ ll : new Date() })
103             .returning();
104         if (input.status === "passed") {
105             await tx
106                 .update(cropsInventory)
107                 .set({ qualityGrade: input.gradeAssigned, status: crop.status === "sold_out" ? "sold_out" : crop.s
108                 ↵ tatus === "archived" ? "archived" : "active" })
109                 .where(eq(cropsInventory.cropId, crop.cropId));
110         } else if (input.status === "failed") {
111             await tx.update(cropsInventory).set({ status: "archived" }).where(eq(cropsInventory.cropId, crop.cro
112             ↵ pId));
113             await tx.execute(sql`update bids_negotiations set status = 'rejected', counter_note = 'Listing faile
114             ↵ d quality inspection', updated_at = now() where crop_id = ${crop.cropId} and status in ('pending','counter
115             ↵ ed')`);
116         } else if (crop.status === "active") {
117             await tx.update(cropsInventory).set({ status: "pending_inspection" }).where(eq(cropsInventory.cropId
118             ↵ , crop.cropId));
119         }
120         return row;
121     });
122 }
123
124 export async function getInspectionReport(id: number) {
125     const [r] = await db.select({ a: qualityInspections.reportAttachment }).from(qualityInspections).where(e
126     ↵ q(qualityInspections.inspectionId, id));
127     if (!r?.a) throw new ApiError(404, "No report attached");
128     return r.a;
129 }
130
131 export type NewUserInput = {
132     fullName: string;
133     email: string;
134     password: string;
135     phone: string;
136     role: "farmer" | "buyer" | "admin" | "inspector";
137     city: string;
138     cnicId?: string;
139     businessName?: string;
140 };
141
142 /** Admin creates any account (e.g. extra admins / quality inspectors). */
143 export async function createUserByAdmin(input: NewUserInput) {
144     if (input.password.length < 8) throw new ApiError(422, "Password must be at least 8 characters");
145     if (!/^[^@s]+@[^@s]+\.[^@s]+$/i.test(input.email)) throw new ApiError(422, "Invalid email address");

```

```

136     if (input.cnicId && !/^d{5}-d{7}-d$/.test(input.cnicId)) throw new ApiError(422, "CNIC must be in for
    ↪ mat 12345-1234567-1");
137     const [exists] = await db.select({ id: users.id }).from(users).where(eq(users.email, input.email)).limit
    ↪ (1);
138     if (exists) throw new ApiError(409, "An account with this email already exists");
139     const [u] = await db
140       .insert(users)
141       .values({
142         fullName: input.fullName,
143         email: input.email,
144         passwordHash: hashPassword(input.password),
145         phone: input.phone,
146         role: input.role,
147         city: input.city,
148         cnicId: input.cnicId,
149         businessName: input.businessName,
150         isVerified: true,
151         emailVerified: true,
152         walletBalance: input.role === "buyer" ? 1000000 : 0,
153       })
154       .returning({ id: users.id, fullName: users.fullName, email: users.email, role: users.role });
155     return u;
156   }
157
158   export async function resetUserPassword(userId: number, newPassword: string) {
159     if (newPassword.length < 8) throw new ApiError(422, "Password must be at least 8 characters");
160     const [u] = await db.update(users).set({ passwordHash: hashPassword(newPassword) }).where(eq(users.id, u
    ↪ serId)).returning({ id: users.id, email: users.email });
161     if (!u) throw new ApiError(404, "User not found");
162     return { ...u, passwordReset: true };
163   }
164
165   export async function changeRole(actor: SessionUser, userId: number, role: "farmer" | "buyer" | "admin" |
    ↪ "inspector") {
166     if (actor.id === userId && role !== "admin") throw new ApiError(409, "You cannot remove your own admin a
    ↪ ccess");
167     const [u] = await db.update(users).set({ role }).where(eq(users.id, userId)).returning({ id: users.id, r
    ↪ ole: users.role });
168     if (!u) throw new ApiError(404, "User not found");
169     return u;
170   }
171

```

src/lib/services/analytics.ts

```

1   import { sql } from "drizzle-orm";
2   import { db } from "@db";
3
4   const num = (v: unknown) => Number(v ?? 0);
5
6   function lastMonths(k = 6) {
7     const out: string[] = [];
8     const d = new Date();
9     d.setDate(1);
10    for (let i = k - 1; i >= 0; i--) {
11      const x = new Date(d.getFullYear(), d.getMonth() - i, 1);
12      out.push(`${x.getFullYear()}-${String(x.getMonth() + 1).padStart(2, "0")}`);
13    }
14    return out;
15  }
16
17  const monthLabel = (ym: string) => {
18    const [y, m] = ym.split("-").map(Number);
19    return new Date(y, m - 1, 1).toLocaleDateString("en-GB", { month: "short", year: "2-digit" });
20  };
21
22  function fillMonths(rows: { month: string; volume: number; orders: number }[]) {
23    return lastMonths(6).map((m) => {
24      const r = rows.find((x) => x.month === m);
25      return { month: monthLabel(m), volume: r ? Math.round(r.volume) : 0, orders: r?.orders ?? 0 };
26    });
27  }
28
29  async function rows<T = Record<string, unknown>>(q: ReturnType<typeof sql>) {
30    return (await db.execute(q)).rows as T[];
31  }

```

```

31
32 export async function farmerStats(farmerId: number) {
33   const [inv] = await rows(sql`
34     select count(*) filter (where status = 'active') as active_listings,
35           count(*) filter (where status = 'pending_inspection') as pending_inspection,
36           coalesce(sum(total_quantity_kg) filter (where status = 'active'), 0) as inventory_kg,
37           coalesce(sum(total_quantity_kg * base_price_per_kg) filter (where status = 'active'), 0) as inv
    ↵   inventory_value
38     from crops_inventory where farmer_id = ${farmerId}`);
39   const [bids] = await rows(sql`
40     select count(*) filter (where b.status = 'pending') as pending, count(*) filter (where b.status = 'cou
    ↵   ntered') as countered
41     from bids_negotiations b join crops_inventory c on c.crop_id = b.crop_id where c.farmer_id = ${farmerI
    ↵   d}`);
42   const [earn] = await rows(sql`
43     select coalesce(sum(amount) filter (where type = 'payout'), 0) as earned,
44           coalesce(sum(amount) filter (where type = 'payout' and created_at > date_trunc('month', now())))
    ↵   , 0) as earned_month
45     from wallet_transactions where user_id = ${farmerId}`);
46   const [esc] = await rows(sql`
47     select coalesce(sum(total_amount) filter (where payment_status in ('escrow_locked','disputed')), 0) as
    ↵   in_escrow,
48           count(*) filter (where delivery_stage <> 'delivered' and payment_status not in ('refunded')) as
    ↵   active_orders
49     from orders_logistics where farmer_id = ${farmerId}`);
50   const monthly = await rows<{ month: string; volume: string; orders: string }>(sql`
51     select to_char(date_trunc('month', created_at), 'YYYY-MM') as month, sum(total_amount) as volume, coun
    ↵   t(*) as orders
52     from orders_logistics where farmer_id = ${farmerId} and payment_status <> 'refunded' and created_at >
    ↵   now() - interval '7 months'
53     group by 1`);
54   const byCrop = await rows<{ crop_name: string; volume: string }>(sql`
55     select c.crop_name, sum(o.total_amount) as volume from orders_logistics o join crops_inventory c on c.
    ↵   crop_id = o.crop_id
56     where o.farmer_id = ${farmerId} and o.payment_status <> 'refunded' group by 1 order by 2 desc limit 6`
    ↵   );
57   return {
58     activeListings: num(inv.active_listings),
59     pendingInspection: num(inv.pending_inspection),
60     inventoryKg: num(inv.inventory_kg),
61     inventoryValue: num(inv.inventory_value),
62     pendingBids: num(bids.pending),
63     counteredBids: num(bids.countered),
64     totalEarned: num(earn.earned),
65     earnedThisMonth: num(earn.earned_month),
66     inEscrow: num(esc.in_escrow),
67     activeOrders: num(esc.active_orders),
68     monthly: fillMonths(monthly.map((m) => ({ month: m.month, volume: num(m.volume), orders: num(m.orders)
    ↵   }))),
69     byCrop: byCrop.map((c) => ({ name: c.crop_name, value: Math.round(num(c.volume)) })),
70   };
71 }
72
73 export async function buyerStats(buyerId: number) {
74   const [b] = await rows(sql`
75     select count(*) filter (where status = 'pending') as pending, count(*) filter (where status = 'counter
    ↵   ed') as countered
76     from bids_negotiations where buyer_id = ${buyerId}`);
77   const [o] = await rows(sql`
78     select count(*) filter (where delivery_stage <> 'delivered' and payment_status <> 'refunded') as activ
    ↵   e_orders,
79           count(*) filter (where payment_status = 'awaiting_escrow') as awaiting_escrow,
80           coalesce(sum(total_amount) filter (where payment_status <> 'refunded'), 0) as spend,
81           coalesce(sum(quantity_kg) filter (where payment_status <> 'refunded'), 0) as kg
82     from orders_logistics where buyer_id = ${buyerId}`);
83   const [w] = await rows(sql`select wallet_balance, escrow_balance from users where id = ${buyerId}`);
84   const monthly = await rows<{ month: string; volume: string; orders: string }>(sql`
85     select to_char(date_trunc('month', created_at), 'YYYY-MM') as month, sum(total_amount) as volume, coun
    ↵   t(*) as orders
86     from orders_logistics where buyer_id = ${buyerId} and payment_status <> 'refunded' and created_at > no
    ↵   w() - interval '7 months'
87     group by 1`);
88   return {
89     pendingBids: num(b.pending),
90     counteredBids: num(b.countered),
91     activeOrders: num(o.active_orders),

```

```

92     awaitingEscrow: num(o.awaiting_escrow),
93     totalSpend: num(o.spend),
94     totalKg: num(o.kg),
95     walletBalance: num(w.wallet_balance),
96     escrowBalance: num(w.escrow_balance),
97     monthly: fillMonths(monthly.map((m) => ({ month: m.month, volume: num(m.volume), orders: num(m.orders)
    ↵      }))),
98   };
99 }
100
101 export async function adminStats() {
102   const [tot] = await rows(sql`
103     select coalesce(sum(total_amount) filter (where payment_status <> 'refunded'), 0) as volume,
104           count(*) as orders,
105           coalesce(sum(total_amount) filter (where payment_status in ('escrow_locked','disputed')), 0) as
    ↵     escrow_held,
106           coalesce(sum(quantity_kg) filter (where payment_status <> 'refunded'), 0) as kg,
107           count(*) filter (where delivery_stage <> 'delivered' and payment_status <> 'refunded') as activ
    ↵     e_shipments
108   from orders_logistics`);
109   const [bids] = await rows(sql`select count(*) filter (where status in ('pending','countered')) as active
    ↵   from bids_negotiations`);
110   const [usr] = await rows(sql`
111     select count(*) filter (where role = 'farmer') as farmers, count(*) filter (where role = 'buyer') as b
    ↵     uyers,
112           count(*) filter (where role = 'farmer' and not is_verified) as unverified
113   from users`);
114   const [fees] = await rows(sql`select coalesce(sum(amount), 0) as fees from wallet_transactions where typ
    ↵     e = 'platform_fee'`);
115   const [disp] = await rows(sql`select count(*) filter (where status in ('open','investigating')) as open
    ↵   from disputes`);
116   const [insp] = await rows(sql`select count(*) as pending from crops_inventory where status = 'pending_in
    ↵     spection`);
117   const regions = await rows<{ region: string; volume: string; orders: string }>(sql`
118     select c.farm_location as region, sum(o.total_amount) as volume, count(*) as orders
119     from orders_logistics o join crops_inventory c on c.crop_id = o.crop_id
120     where o.payment_status <> 'refunded' group by 1 order by 2 desc limit 8`);
121   const categories = await rows<{ category: string; volume: string }>(sql`
122     select c.category, sum(o.total_amount) as volume from orders_logistics o join crops_inventory c on c.c
    ↵     rop_id = o.crop_id
123     where o.payment_status <> 'refunded' group by 1 order by 2 desc`);
124   const monthly = await rows<{ month: string; volume: string; orders: string }>(sql`
125     select to_char(date_trunc('month', created_at), 'YYYY-MM') as month, sum(total_amount) as volume, coun
    ↵     t(*) as orders
126   from orders_logistics where payment_status <> 'refunded' and created_at > now() - interval '7 months'
    ↵   group by 1`);
127   const stages = await rows<{ stage: string; n: string }>(sql`select delivery_stage as stage, count(*) as
    ↵     n from orders_logistics group by 1`);
128   return {
129     totalVolume: num(tot.volume),
130     totalOrders: num(tot.orders),
131     escrowHeld: num(tot.escrow_held),
132     totalKg: num(tot.kg),
133     activeShipments: num(tot.active_shipments),
134     activeBids: num(bids.active),
135     farmers: num(usr.farmers),
136     buyers: num(usr.buyers),
137     unverifiedFarmers: num(usr.unverified),
138     platformFees: num(fees.fees),
139     openDisputes: num(disp.open),
140     pendingInspections: num(insp.pending),
141     topRegions: regions.map((r) => ({ region: r.region, volume: Math.round(num(r.volume)), orders: num(r.o
    ↵     rders) })),
142     categories: categories.map((c) => ({ name: c.category, value: Math.round(num(c.volume)) })),
143     monthly: fillMonths(monthly.map((m) => ({ month: m.month, volume: num(m.volume), orders: num(m.orders)
    ↵      }))),
144     stages: stages.map((s) => ({ stage: s.stage, count: num(s.n) })),
145   };
146 }
147

```

src/lib/services/assistant.ts

```
1 import { sql } from "drizzle-orm";
2 import { db } from "@db";
3 import { listCrops } from "@lib/services/crops";
4 import { latestRates } from "@lib/services/mandi";
5 import { CATEGORIES, CITY_NAMES, CROP_CATALOGUE, MAUND_KG, categoryLabel, cropImage, mandiName, type Category, type Grade } from "@lib/constants";
6
7 /* ----- */
8 /* Types shared with the ChatBot widget */
9 /* ----- */
10 export type ListingCard = {
11   kind: "listing";
12   id: number;
13   title: string;
14   image: string;
15   pricePerKg: number;
16   pricePerMaund: number;
17   grade: Grade;
18   inspected: boolean;
19   seller: string;
20   verified: boolean;
21   location: string;
22   distanceKm: number | null;
23   availableKg: number;
24   href: string;
25 };
26 export type DealerCard = {
27   kind: "dealer";
28   id: number;
29   name: string;
30   business: string | null;
31   city: string;
32   rating: number;
33   deliveries: number;
34   crops: string;
35   verified: boolean;
36   href: string;
37 };
38 export type RateCard = {
39   kind: "rate";
40   crop: string;
41   avgPerMaund: number;
42   low: { mandi: string; perMaund: number };
43   high: { mandi: string; perMaund: number };
44   changePct: number | null;
45 };
46 export type AssistantCard = ListingCard | DealerCard | RateCard;
47 export type AssistantReply = { reply: string; cards?: AssistantCard[]; suggestions?: string[]; link?: { label: string; href: string } };
48
49 /* ----- */
50 /* Entity extraction (English + common Roman Urdu terms) */
51 /* ----- */
52 const CROP_ALIASES: Record<string, string> = {
53   wheat: "Wheat", gandum: "Wheat", gehun: "Wheat",
54   rice: "Basmati Rice", basmati: "Basmati Rice", chawal: "Basmati Rice",
55   maize: "Maize", corn: "Maize", makai: "Maize", makki: "Maize",
56   chickpea: "Chickpea (Chana)", chana: "Chickpea (Chana)", gram: "Chickpea (Chana)",
57   tomato: "Tomato", tamatar: "Tomato",
58   potato: "Potato", aloo: "Potato", aalu: "Potato",
59   onion: "Onion", pyaz: "Onion", piyaz: "Onion",
60   mango: "Mango (Chaunsa)", aam: "Mango (Chaunsa)", chaunsa: "Mango (Chaunsa)", sindhri: "Mango (Chaunsa)"
61   ,
62   kinnow: "Kinnow", kinno: "Kinnow", orange: "Kinnow", santra: "Kinnow",
63   dates: "Dates (Khajoor)", date: "Dates (Khajoor)", khajoor: "Dates (Khajoor)", khajur: "Dates (Khajoor)"
64   ,
65   cotton: "Cotton (Phutti)", kapas: "Cotton (Phutti)", phutti: "Cotton (Phutti)",
66   sugarcane: "Sugarcane", ganna: "Sugarcane", cane: "Sugarcane",
67   chilli: "Red Chilli (Kunri)", chili: "Red Chilli (Kunri)", mirch: "Red Chilli (Kunri)",
68 };
69 const CATEGORY_ALIASES: Record<string, Category> = {
70   grain: "grains", grains: "grains", pulses: "grains", anaaj: "grains", cereal: "grains",
71   vegetable: "vegetables", vegetables: "vegetables", veggies: "vegetables", sabzi: "vegetables", sabziyan:
```

```

69     "vegetables",
70     fruit: "fruits", fruits: "fruits", phal: "fruits",
71     "cash crop": "cash_crops", "cash crops": "cash_crops", fibre: "cash_crops", fiber: "cash_crops",
72 };
73
74 /** Whole-word match; a trailing "*" allows prefix matching (e.g. "negotiat*"). */
75 const has = (text: string, words: string[]) =>
76     words.some((w) => new RegExp(w.endsWith("*") ? `\\b${w.slice(0, -1)}\\` : `\\b${w}\\b\\`, "i").test(text));
77
78 function extract(message: string) {
79     const text = `${message.toLowerCase().replace(/[\^p{L}p{N}s-]/gu, " ")} `;
80     let crop: string | undefined;
81     for (const [alias, name] of Object.entries(CROP_ALIASES)) if (new RegExp(`\\b${alias}\\b\\`).test(text)) {
82         crop = name; break; }
83     let category: Category | undefined;
84     for (const [alias, cat] of Object.entries(CATEGORY_ALIASES)) if (new RegExp(`\\b${alias}\\b\\`).test(text)) {
85         category = cat; break; }
86     if (!category && crop) category = CROP_CATALOGUE.find((c) => c.name === crop)?.category;
87     const city = CITY_NAMES.find((c) => new RegExp(`\\b${c.toLowerCase()}\\b\\`).test(text));
88     const gradeMatch = text.match(/\\bgrade\\s*([abc])\\b/);
89     const quality = has(text, ["high quality", "high-quality", "best quality", "top quality", "premium", "export", "grade a", "a grade", "behtareen", "acha", "achha", "best"]);
90     const grade: Grade | undefined = gradeMatch ? (gradeMatch[1].toUpperCase() as Grade) : quality ? "A" : undefined;
91     return { text, crop, category, city, grade, quality };
92 }
93
94 /** ----- */
95 /** Data helpers */
96 /** ----- */
97 const marketplace = (p: Record<string, string | number | undefined>, isBuyer: boolean) => {
98     const qs = new URLSearchParams(Object.entries(p).filter(([, v]) => v !== undefined && v !== ""));
99     return `/marketplace${qs.toString()} ? ` + qs.toString();
100 };
101
102 async function findListings(
103     f: { crop?: string; category?: Category; grade?: Grade; city?: string; sort?: "price_asc" | "distance" | "quality" },
104     isBuyer: boolean,
105     limit = 4,
106 ): Promise<ListingCard[]> {
107     const rows = await listCrops({
108         q: f.crop,
109         category: f.crop ? undefined : f.category,
110         grade: f.grade,
111         originCity: f.city,
112         sort: f.sort === "price_asc" ? "price_asc" : f.sort === "distance" ? "distance" : "newest",
113     });
114     const ranked = f.sort === "quality" || (!f.sort && f.grade)
115         ? [...rows].sort((a, b) => Number(!b.inspectionGrade) - Number(!a.inspectionGrade) || Number(b.farmerVerified) - Number(a.farmerVerified) || a.qualityGrade.localeCompare(b.qualityGrade))
116         : rows;
117     return ranked.slice(0, limit).map((c) => ({
118         kind: "listing",
119         id: c.cropId,
120         title: c.cropName,
121         image: cropImage(c.imagesJson, c.cropName, c.category),
122         pricePerKg: c.basePricePerKg,
123         pricePerMaund: Math.round(c.basePricePerKg * MAUND_KG),
124         grade: c.qualityGrade,
125         inspected: !!c.inspectionGrade,
126         seller: c.farmName ?? c.farmerName,
127         verified: c.farmerVerified,
128         location: c.farmLocation,
129         distanceKm: c.distanceKm,
130         availableKg: c.totalQuantityKg,
131         href: `/crop/${c.cropId}`,
132     }));
133 }
134
135 /**
136  * Seller rating (1-5) derived from real platform signals:
137  * delivered orders, verification, passed inspections, dispute rate.
138  */
139 async function topDealers(f: { crop?: string; category?: Category; city?: string }, isBuyer: boolean, limit: number) {

```

```

138     const res = await db.execute(sql`
139       select u.id, u.full_name, u.business_name, u.city, u.is_verified, u.trust_score as trust, u.rating_cou
140       nt as reviews,
141       (select count(*) from orders_logistics o where o.farmer_id = u.id and o.payment_status = 'released')
142       ::int as delivered,
143       (select count(*) from disputes d join orders_logistics o on o.order_id = d.order_id where o.farmer_i
144       d = u.id)::int as disputes,
145       (select count(*) from quality_inspections qi join crops_inventory c on c.crop_id = qi.crop_id where
146       c.farmer_id = u.id and qi.status = 'passed')::int as passed,
147       (select string_agg(distinct c.crop_name, ', ') from crops_inventory c where c.farmer_id = u.id and c
148       .status = 'active') as crops
149       from users u
150       where u.role = 'farmer'
151       ${f.crop ? sql`and exists (select 1 from crops_inventory c where c.farmer_id = u.id and c.status = '
152       active' and c.crop_name = ${f.crop})` : sql``}
153       ${!f.crop && f.category ? sql`and exists (select 1 from crops_inventory c where c.farmer_id = u.id a
154       nd c.status = 'active' and c.category = ${f.category})` : sql``}
155     `);
156     const dealers = (res.rows as { trust: number; reviews: number; id: number; full_name: string; business_n
157     ame: string | null; city: string; is_verified: boolean; delivered: number; disputes: number; passed: numbe
158     r; crops: string | null }[]).
159     .filter((d) => d.crops)
160     .map((d) => {
161       const delivered = Number(d.delivered);
162       const disputeRate = delivered ? Number(d.disputes) / delivered : 0;
163       const score = Number(d.trust) > 0 ? Number(d.trust) : 3.4 + Math.min(1, delivered / 10) + (d.is_veri
164       fied ? 0.3 : 0) + Math.min(0.3, Number(d.passed) * 0.1) - disputeRate * 1.5;
165       return {
166         kind: "dealer" as const,
167         id: Number(d.id),
168         name: d.full_name,
169         business: d.business_name,
170         city: d.city,
171         rating: Math.round(Math.max(1, Math.min(5, score)) * 10) / 10,
172         deliveries: delivered,
173         crops: d.crops ?? "",
174         verified: d.is_verified,
175         href: marketHref({ farmerId: d.id }, isBuyer),
176       };
177     });
178     if (f.city) dealers.sort((a, b) => Number(b.city === f.city) - Number(a.city === f.city));
179     return dealers.sort((a, b) => (f.city ? Number(b.city === f.city) - Number(a.city === f.city) : 0) || b.
180     rating - a.rating || b.deliveries - a.deliveries).slice(0, limit);
181   }
182
183   async function rateCard(crop: string): Promise<RateCard | null> {
184     const rates = await latestRates({ crop });
185     if (!rates.length) return null;
186     const sorted = [...rates].sort((a, b) => a.avgPricePerKg - b.avgPricePerKg);
187     const avg = rates.reduce((s, r) => s + r.avgPricePerKg, 0) / rates.length;
188     const changes = rates.map((r) => r.changePct).filter((c): c is number => c !== null);
189     return {
190       kind: "rate",
191       crop,
192       avgPerMaund: Math.round(avg * MAUND_KG),
193       low: { mandi: mandiName(sorted[0].marketLocation), perMaund: Math.round(sorted[0].avgPricePerKg * MAUN
194       D_KG) },
195       high: { mandi: mandiName(sorted[sorted.length - 1].marketLocation), perMaund: Math.round(sorted[sorted
196       .length - 1].avgPricePerKg * MAUND_KG) },
197       changePct: changes.length ? Math.round((changes.reduce((a, b) => a + b, 0) / changes.length) * 100) /
198       100 : null,
199     };
200   }
201
202   /* ----- */
203   /* Intent routing */
204   /* ----- */
205   export const DEFAULT_SUGGESTIONS = ["Top-rated dealers", "High-quality fruits", "Cheapest wheat", "Mango m
206   andi rate", "How does escrow work?"];
207
208   export async function answer(message: string, ctx: { isBuyer: boolean; city?: string }): Promise<Assistant
209   Reply> {
210     const e = extract(message);
211     const t = e.text;
212     const place = e.city ?? ctx.city;

```

```

197     const what = e.crop ?? (e.category ? categoryLabel(e.category).toLowerCase() : "produce");
198
199     // Greetings / help
200     if (has(t, ["hi", "hello", "hey", "salam", "assalam", "aoa", "help", "start"]) && !e.crop && !e.category
201     && t.trim().split(/\s+/).length <= 4) {
202         return {
203             reply: "Assalam-o-Alaikum! I can find trusted sellers, high-quality produce, the best prices, and to
204             day's mandi rates. What are you looking for?",
205             suggestions: DEFAULT_SUGGESTIONS,
206         };
207     }
208
209     // FAQ: escrow / payment
210     if (has(t, ["escrow", "payment", "pay", "wallet", "refund", "safe", "paisa", "paise"])) {
211         return {
212             reply:
213             "Payments are protected by escrow:\n1. When a farmer accepts your bid, you lock the order amount f
214             rom your wallet.\n2. The crop is inspected, dispatched and tracked.\n3. You confirm delivery, and only the
215             n is the farmer paid (a flat 1.5% fee is deducted from the farmer).\nIf something is wrong, raise a disput
216             e and the funds stay frozen until an admin resolves it.",
217             suggestions: ["How do I place a bid?", "Track my order", "Top-rated dealers"],
218             link: ctx.isBuyer ? { label: "Open wallet", href: "/wallet" } : undefined,
219         };
220     }
221
222     // FAQ: bidding
223     if (has(t, ["bid", "offer", "negotiat*", "counter", "rfq", "bargain", "boli"])) {
224         return {
225             reply:
226             "To buy in bulk: open a listing, tap Place Bid, and enter your price per kg, quantity and delivery
227             date. The farmer can accept, reject or counter. You can then accept the counter-offer or revise your pric
228             e. Accepted bids become orders automatically.",
229             suggestions: ["High-quality grains", "Cheapest onion", "How does escrow work?"],
230             link: ctx.isBuyer ? { label: "My bids", href: "/buyer/bids" } : { label: "Create buyer account", hre
231             f: "/login?role=buyer" },
232         };
233     }
234
235     // FAQ: tracking
236     if (has(t, ["track", "delivery", "shipment", "truck", "where is my", "status", "order"])) {
237         return {
238             reply: "Every order moves through 5 stages: Placed → Quality Inspected → Dispatched → In Transit → D
239             elivered. You can see live checkpoints and the full history on your orders page.",
240             suggestions: ["How does escrow work?", "Top-rated dealers"],
241             link: ctx.isBuyer ? { label: "Track orders", href: "/buyer/orders" } : { label: "Sign in", href: "/l
242             ogin" },
243         };
244     }
245
246     // Dealers / sellers
247     if (has(t, ["dealer", "dealers", "seller", "sellers", "supplier", "suppliers", "farmer", "farmers", "ven
248     dor", "trusted", "reliable", "top rated", "top-rated", "rated", "kisan", "beopari"])) {
249         const cards = await topDealers({ crop: e.crop, category: e.category, city: e.city }, ctx.isBuyer);
250         if (!cards.length) return { reply: `No active sellers found for ${what} right now. Try another crop or
251         category.`, suggestions: DEFAULT_SUGGESTIONS };
252         return {
253             reply: `Top-rated sellers${e.crop || e.category ? ` for ${what}` : ""}${e.city ? ` (${e.city} first)
254             ` : ""}. Trust scores combine verified buyer reviews, completed deliveries, identity verification and disp
255             ute history`,
256             cards,
257             suggestions: [e.crop ? `High-quality ${e.crop.split(" ")[0].toLowerCase()}` : "High-quality vegetabl
258             es", "Cheapest wheat", "How does escrow work?"],
259         };
260     }
261
262     // Mandi rates
263     if (has(t, ["rate", "rates", "mandi", "bhav", "bhao", "market price", "price today", "qeemat", "kya chal
264     "])) {
265         const crops = e.crop ? [e.crop] : e.category ? CROP_CATALOGUE.filter((c) => c.category === e.category)
266         .map((c) => c.name) : ["Wheat", "Basmati Rice", "Mango (Chaunsa)"];
267         const cards = (await Promise.all(crops.slice(0, 3).map(rateCard))).filter((c): c is RateCard => !!c);
268         if (!cards.length) return { reply: `I couldn't find mandi rates for ${what} yet.`, suggestions: DEFAUL
269         T_SUGGESTIONS };
270         return {
271             reply: `Today's mandi rates per maund (40 kg)${e.crop ? ` for ${e.crop}` : ""}. The lowest mandi is
272             usually the best place to source`,
273             cards,
274             suggestions: e.crop ? [`Cheapest ${e.crop.split(" ")[0].toLowerCase()} listings`, `High-quality ${e.`

```

```

    crop.split(" ")[0].toLowerCase()}`, `Top ${e.crop.split(" ")[0].toLowerCase()} dealers`] : DEFAULT_SUGGEST
IONS,
254     link: { label: "All mandi rates", href: "/mandi-rates" },
255   };
256 }
257
258 // Listings: cheapest / nearby / quality / generic crop or category
259 const cheap = has(t, ["cheap", "cheapest", "lowest", "low price", "budget", "affordable", "sasta", "sast
i", "kam qeemat"]);
260 const near = has(t, ["near", "nearby", "nearest", "close to", "around", "qareeb", "paas"]) || (!!e.city
&& !cheap);
261 const quality = !!e.grade || e.quality;
262 if (cheap || near || quality || e.crop || e.category || has(t, ["buy", "find", "show", "list", "availabl
e", "stock", "khareed"])) {
263   const sort = cheap ? "price_asc" : near ? "distance" : quality ? "quality" : undefined;
264   let cards = await findListings({ crop: e.crop, category: e.category, grade: e.grade, city: near || che
ap ? place : undefined, sort }, ctx.isBuyer);
265   let relaxed = false;
266   if (!cards.length && e.grade) {
267     cards = await findListings({ crop: e.crop, category: e.category, city: place, sort: "quality" }, ctx
.isBuyer);
268     relaxed = cards.length > 0;
269   }
270   if (!cards.length) {
271     return { reply: `There are no active listings for ${what} right now. I can show you similar produce
or today's mandi rates instead.`, suggestions: DEFAULT_SUGGESTIONS };
272   }
273   const intro = relaxed
    ? `No Grade ${e.grade} ${what} is listed right now. Here are the best available options:`
    : cheap ? `Lowest-priced ${what}${place ? ` (distance from ${place})` : ""}:`
    : quality ? `Highest-quality ${what}. Grade ${e.grade ?? "A"} and inspected listings come first:`
    : near && place ? `${what[0].toUpperCase() + what.slice(1)} closest to ${place}:`
    : `Here's what's available for ${what}:`;
274   return {
275     reply: intro,
276     cards,
277     suggestions: [
278       e.crop ? `${e.crop.split(" ")[0]} mandi rate` : "Mandi rates today",
279       e.crop ? `Top ${e.crop.split(" ")[0].toLowerCase()} dealers` : "Top-rated dealers",
280       cheap ? `High-quality ${e.crop?.split(" ")[0].toLowerCase()} ?? "produce"` : `Cheapest ${e.crop?.s
plit(" ")[0].toLowerCase()} ?? "vegetables"`,
281     ],
282     link: { label: "Open marketplace", href: marketHref({ q: e.crop, category: e.crop ? undefined : e.ca
tegrory, grade: e.grade, sort: cheap ? "price_asc" : undefined }, ctx.isBuyer) },
283   };
284 }
285
286 // Fallback
287 return {
288   reply: `I'm not sure I understood. Try asking about a crop (e.g. "Grade A basmati"), a category (${CAT
EGORIES.map((c) => c.label.toLowerCase()).join(", ")}), sellers, or mandi rates.`,
289   suggestions: DEFAULT_SUGGESTIONS,
290 };
291 }
292
293 }
294
295 }
296
297 }

```

src/lib/services/bids.ts

```

1  import { and, desc, eq, gt, inArray, ne, sql } from "drizzle-orm";
2  import { alias } from "drizzle-orm/pg-core";
3  import { db } from "@db";
4  import { bidsNegotiations, cropsInventory, users, ordersLogistics, orderEvents } from "@db/schema";
5  import { ApiError } from "@lib/api";
6  import type { SessionUser } from "@lib/auth";
7  import { notify } from "@lib/notify";
8
9  type Tx = Parameters<Parameters<typeof db.transaction>[0]>[0];
10
11 const farmer = alias(users, "farmer");
12 const buyer = alias(users, "buyer");
13
14 export async function listBids(user: SessionUser, opts: { status?: string; cropId?: number } = {}) {
15   const conds = [];
16   if (user.role === "farmer") conds.push(eq(cropsInventory.farmerId, user.id));
17   if (user.role === "buyer") conds.push(eq(bidsNegotiations.buyerId, user.id));

```

```

18   if (opts.status) conds.push(eq(bidsNegotiations.status, opts.status as "pending"));
19   if (opts.cropId) conds.push(eq(bidsNegotiations.cropId, opts.cropId));
20   const rows = await db
21     .select({
22       bid: bidsNegotiations,
23       cropName: cropsInventory.cropName,
24       category: cropsInventory.category,
25       qualityGrade: cropsInventory.qualityGrade,
26       basePricePerKg: cropsInventory.basePricePerKg,
27       availableKg: cropsInventory.totalQuantityKg,
28       farmLocation: cropsInventory.farmLocation,
29       imagesJson: cropsInventory.imagesJson,
30       farmerId: cropsInventory.farmerId,
31       farmerName: farmer.fullName,
32       buyerName: buyer.fullName,
33       buyerBusiness: buyer.businessName,
34       buyerCity: buyer.city,
35       orderId: ordersLogistics.orderId,
36     })
37     .from(bidsNegotiations)
38     .innerJoin(cropsInventory, eq(cropsInventory.cropId, bidsNegotiations.cropId))
39     .innerJoin(farmer, eq(farmer.id, cropsInventory.farmerId))
40     .innerJoin(buyer, eq(buyer.id, bidsNegotiations.buyerId))
41     .leftJoin(ordersLogistics, eq(ordersLogistics.bidId, bidsNegotiations.bidId))
42     .where(conds.length ? and(...conds) : undefined)
43     .orderBy(desc(bidsNegotiations.updatedAt))
44     .limit(300);
45   return rows.map(({ bid, ...rest }) => ({ ...bid, ...rest }));
46 }
47
48 export type BidInput = { cropId: number; bidPricePerKg: number; bidQuantityKg: number; targetDeliveryDate?
↳ : string; message?: string };
49
50 async function createBidTx(buyerUser: SessionUser, input: BidInput) {
51   const [crop] = await db.select().from(cropsInventory).where(eq(cropsInventory.cropId, input.cropId)).lim
↳ it(1);
52   if (!crop) throw new ApiError(404, "Crop listing not found");
53   if (crop.status !== "active") throw new ApiError(409, "This listing is not accepting bids");
54   if (input.bidQuantityKg > crop.totalQuantityKg) throw new ApiError(422, `Only ${crop.totalQuantityKg} kg
↳ available`);
55   if (input.bidPricePerKg < crop.basePricePerKg * 0.5) throw new ApiError(422, "Bid is below 50% of the as
↳ king price and was blocked as a lowball offer");
56   if (input.targetDeliveryDate && new Date(input.targetDeliveryDate) < new Date(new Date().toISOString()).s
↳ lice(0, 10))
57     throw new ApiError(422, "Target delivery date must be in the future");
58   const [dupe] = await db
59     .select({ id: bidsNegotiations.bidId })
60     .from(bidsNegotiations)
61     .where(and(eq(bidsNegotiations.cropId, input.cropId), eq(bidsNegotiations.buyerId, buyerUser.id), inAr
↳ ray(bidsNegotiations.status, ["pending", "countered"])))
62     .limit(1);
63   if (dupe) throw new ApiError(409, "You already have an open bid on this listing. Revise or withdraw it f
↳ irst.");
64   const [row] = await db
65     .insert(bidsNegotiations)
66     .values({ ...input, buyerId: buyerUser.id, status: "pending" })
67     .returning();
68   return row;
69 }
70
71 export type BidAction = "accept" | "reject" | "counter" | "accept_counter" | "reject_counter" | "revise" |
↳ "withdraw";
72
73 /**
74  * Negotiation state machine
75  * pending --farmer:accept--> accepted (order created @ bid price)
76  * pending --farmer:reject--> rejected
77  * pending --farmer:counter--> countered (counter_price set)
78  * countered --buyer:accept_counter--> accepted (order created @ counter price)
79  * countered --buyer:reject_counter--> rejected
80  * countered --buyer:revise--> pending (new bid price, counter cleared)
81  * pending|countered --buyer:withdraw--> withdrawn
82  */
83 async function actOnBidTx(user: SessionUser, bidId: number, action: BidAction, payload: { counterPrice?: n
↳ umber; note?: string; bidPricePerKg?: number }) {
84   return db.transaction(async (tx) => {

```

```

85     const [bid] = await tx.select().from(bidsNegotiations).where(eq(bidsNegotiations.bidId, bidId)).for("u
    pdate").limit(1);
86     if (!bid) throw new ApiError(404, "Bid not found");
87     const [crop] = await tx.select().from(cropsInventory).where(eq(cropsInventory.cropId, bid.cropId)).for
    ("update").limit(1);
88     const isFarmer = user.role === "farmer" && crop.farmerId === user.id;
89     const isBuyer = user.role === "buyer" && bid.buyerId === user.id;
90     const now = new Date();
91
92     const need = (cond: boolean, msg: string, status = 409) => {
93         if (!cond) throw new ApiError(status, msg);
94     };
95
96     switch (action) {
97         case "accept": {
98             need(isFarmer, "Only the listing owner can accept bids", 403);
99             need(bid.status === "pending", `Cannot accept a bid that is ${bid.status}`);
100            const order = await convertToOrder(tx, bid, crop, bid.bidPricePerKg, user.id);
101            return { bid: { ...bid, status: "accepted" }, order };
102        }
103        case "reject": {
104            need(isFarmer, "Only the listing owner can reject bids", 403);
105            need(bid.status === "pending" || bid.status === "countered", `Cannot reject a bid that is ${bid.st
    atus}`);
106            const [b] = await tx.update(bidsNegotiations).set({ status: "rejected", counterNote: payload.note
    ?? bid.counterNote, updatedAt: now }).where(eq(bidsNegotiations.bidId, bidId)).returning();
107            return { bid: b };
108        }
109        case "counter": {
110            need(isFarmer, "Only the listing owner can counter bids", 403);
111            need(bid.status === "pending", `Cannot counter a bid that is ${bid.status}`);
112            need(payload.counterPrice !== undefined && payload.counterPrice > 0, "counterPrice is required", 4
    22);
113            need(payload.counterPrice! > bid.bidPricePerKg, "Counter price should be higher than the buyer's b
    id", 422);
114            const [b] = await tx.update(bidsNegotiations).set({ status: "countered", counterPrice: payload.cou
    nterPrice!, counterNote: payload.note ?? null, updatedAt: now }).where(eq(bidsNegotiations.bidId, bidId)).
    returning();
115            return { bid: b };
116        }
117        case "accept_counter": {
118            need(isBuyer, "Only the bidder can accept a counter-offer", 403);
119            need(bid.status === "countered" && bid.counterPrice !== null, "No counter-offer to accept");
120            const order = await convertToOrder(tx, bid, crop, bid.counterPrice!, user.id);
121            return { bid: { ...bid, status: "accepted" }, order };
122        }
123        case "reject_counter": {
124            need(isBuyer, "Only the bidder can reject a counter-offer", 403);
125            need(bid.status === "countered", "No counter-offer to reject");
126            const [b] = await tx.update(bidsNegotiations).set({ status: "rejected", updatedAt: now }).where(eq
    (bidsNegotiations.bidId, bidId)).returning();
127            return { bid: b };
128        }
129        case "revise": {
130            need(isBuyer, "Only the bidder can revise a bid", 403);
131            need(bid.status === "countered" || bid.status === "pending", `Cannot revise a bid that is ${bid.st
    atus}`);
132            need(payload.bidPricePerKg !== undefined && payload.bidPricePerKg > 0, "bidPricePerKg is required"
    , 422);
133            const [b] = await tx
134                .update(bidsNegotiations)
135                .set({ status: "pending", bidPricePerKg: payload.bidPricePerKg!, message: payload.note ?? bid.me
    ssage, updatedAt: now })
136                .where(eq(bidsNegotiations.bidId, bidId))
137                .returning();
138            return { bid: b };
139        }
140        case "withdraw": {
141            need(isBuyer, "Only the bidder can withdraw", 403);
142            need(bid.status === "pending" || bid.status === "countered", `Cannot withdraw a bid that is ${bid.
    status}`);
143            const [b] = await tx.update(bidsNegotiations).set({ status: "withdrawn", updatedAt: now }).where(e
    q(bidsNegotiations.bidId, bidId)).returning();
144            return { bid: b };
145        }
146        default:

```

```

147         throw new ApiError(422, "Unknown action");
148     }
149 });
150 }
151
152 async function convertToOrder(tx: Tx, bid: typeof bidsNegotiations.$inferSelect, crop: typeof cropsInventory.$inferSelect, price: number, actorId: number) {
153     if (crop.status !== "active") throw new ApiError(409, "Listing is no longer active");
154     if (bid.bidQuantityKg > crop.totalQuantityKg) throw new ApiError(409, `Insufficient stock: only ${crop.totalQuantityKg} kg left`);
155     const remaining = Math.round((crop.totalQuantityKg - bid.bidQuantityKg) * 100) / 100;
156     const now = new Date();
157
158     await tx.update(bidsNegotiations).set({ status: "accepted", bidPricePerKg: price, updatedAt: now }).where(
159         eq(bidsNegotiations.bidId, bid.bidId));
160     await tx
161         .update(cropsInventory)
162         .set({ totalQuantityKg: remaining, status: remaining <= 0 ? "sold_out" : "active" })
163         .where(eq(cropsInventory.cropId, crop.cropId));
164
165     // Auto-reject other open bids that can no longer be fulfilled
166     await tx
167         .update(bidsNegotiations)
168         .set({ status: "rejected", counterNote: "Auto-rejected: insufficient remaining stock", updatedAt: now })
169         .where(
170             and(
171                 eq(bidsNegotiations.cropId, crop.cropId),
172                 ne(bidsNegotiations.bidId, bid.bidId),
173                 inArray(bidsNegotiations.status, ["pending", "countered"]),
174                 gt(bidsNegotiations.bidQuantityKg, remaining)
175             )
176         );
177
178     const total = Math.round(bid.bidQuantityKg * price * 100) / 100;
179     const seqRes = await tx.execute(sql`select coalesce(max(order_id), 0) + 1001 as n from orders_logistics`);
180     const n = Number((seqRes.rows[0] as { n: number | string }).n);
181     const [order] = await tx
182         .insert(ordersLogistics)
183         .values({
184             bidId: bid.bidId,
185             cropId: crop.cropId,
186             farmerId: crop.farmerId,
187             buyerId: bid.buyerId,
188             quantityKg: bid.bidQuantityKg,
189             pricePerKg: price,
190             totalAmount: total,
191             paymentStatus: "awaiting_escrow",
192             deliveryStage: "confirmed",
193             transitLocation: crop.farmLocation,
194             trackingNumber: `ML-${now.getFullYear()}-${n}-${Math.random().toString(36).slice(2, 5).toUpperCase()}`,
195             expectedDeliveryDate: bid.targetDeliveryDate,
196         })
197         .returning();
198     await tx.insert(orderEvents).values({ orderId: order.orderId, stage: "confirmed", location: crop.farmLocation, note: `Deal closed at Rs. ${price}/kg for ${bid.bidQuantityKg} kg. Awaiting buyer escrow deposit.`, actorId });
199     return order;
200 }
201
202 /* ----- */
203 /* Public API with automated notifications */
204 /* ----- */
205 const rs = (n: number) => "Rs. " + Math.round(n).toLocaleString("en-US");
206
207 export async function createBid(buyerUser: SessionUser, input: BidInput) {
208     const bid = await createBidTx(buyerUser, input);
209     const [crop] = await db.select({ farmerId: cropsInventory.farmerId, cropName: cropsInventory.cropName })
210         .from(cropsInventory).where(eq(cropsInventory.cropId, input.cropId)).limit(1);
211     if (crop) {
212         await notify(crop.farmerId, {
213             type: "bid",
214             title: `New bid on your ${crop.cropName}`,
215             body: `${buyerUser.businessName ?? buyerUser.fullName} offered ${rs(input.bidPricePerKg)}/kg for ${i

```

```

    nput.bidQuantityKg.toLocaleString("en-US")} kg.` ,
214     link: "/farmer/bids",
215     sms: true,
216   });
217   }
218   return bid;
219 }
220
221 export async function actOnBid(user: SessionUser, bidId: number, action: BidAction, payload: { counterPrice?: number; note?: string; bidPricePerKg?: number }) {
222   const result = await actOnBidTx(user, bidId, action, payload);
223   const [info] = await db
224     .select({ buyerId: bidsNegotiations.buyerId, farmerId: cropsInventory.farmerId, cropName: cropsInventory.cropName, qty: bidsNegotiations.bidQuantityKg })
225     .from(bidsNegotiations)
226     .innerJoin(cropsInventory, eq(cropsInventory.cropId, bidsNegotiations.cropId))
227     .where(eq(bidsNegotiations.bidId, bidId))
228     .limit(1);
229   if (info) {
230     const order = (result as { order?: { trackingNumber: string; totalAmount: number } }).order;
231     if (order) {
232       await notify(info.buyerId, { type: "bid_accepted", title: "Bid accepted: fund escrow to start", body: `Your ${info.cropName} deal (${order.trackingNumber}) is confirmed at ${rs(order.totalAmount)}. Lock the amount in escrow to begin fulfilment.`, link: "/buyer/orders", email: true, sms: true });
233       await notify(info.farmerId, { type: "bid_accepted", title: "Deal closed", body: `Order ${order.trackingNumber} for ${info.qty.toLocaleString("en-US")} kg ${info.cropName} created. Waiting for the buyer's escrow deposit.`, link: "/farmer/orders" });
234     } else if (action === "counter") {
235       await notify(info.buyerId, { type: "bid", title: "Counter-offer received", body: `The farmer countered your ${info.cropName} bid at ${rs(payload.counterPrice ?? 0)}/kg.`, link: "/buyer/bids", sms: true });
236     } else if (action === "reject") {
237       await notify(info.buyerId, { type: "bid", title: "Bid declined", body: `Your bid on ${info.cropName} was declined.`, link: "/buyer/bids" });
238     } else if (action === "revise") {
239       await notify(info.farmerId, { type: "bid", title: "Revised bid received", body: `A buyer revised their ${info.cropName} offer to ${rs(payload.bidPricePerKg ?? 0)}/kg.`, link: "/farmer/bids" });
240     } else if (action === "reject_counter" || action === "withdraw") {
241       await notify(info.farmerId, { type: "bid", title: action === "withdraw" ? "Bid withdrawn" : "Counter-offer declined", body: `The buyer ${action === "withdraw" ? "withdrew their bid" : "declined your counter-offer"} on ${info.cropName}.`, link: "/farmer/bids" });
242     }
243   }
244   return result;
245 }
246

```

src/lib/services/crops.ts

```

1  import { and, desc, eq, gte, ilike, inArray, lte, sql, type SQL } from "drizzle-orm";
2  import { db } from "@db";
3  import { cropsInventory, users, qualityInspections, ordersLogistics } from "@db/schema";
4  import { ApiError } from "@lib/api";
5  import { CITIES, distanceKm, type Category, type Grade } from "@lib/constants";
6  import type { SessionUser } from "@lib/auth";
7
8  export type CropFilters = {
9    category?: Category;
10   grade?: Grade;
11   minPrice?: number;
12   maxPrice?: number;
13   q?: string;
14   originCity?: string;
15   radiusKm?: number;
16   farmerId?: number;
17   minQty?: number;
18   province?: string;
19   status?: "active" | "all";
20   sort?: "newest" | "price_asc" | "price_desc" | "distance";
21 };
22
23 export async function listCrops(f: CropFilters) {
24   const where: SQL[] = [];
25   if (f.category) where.push(eq(cropsInventory.category, f.category));
26   if (f.grade) where.push(eq(cropsInventory.qualityGrade, f.grade));
27   if (f.minPrice !== undefined) where.push(gte(cropsInventory.basePricePerKg, f.minPrice));

```

```

28   if (f.minQty !== undefined) where.push(gte(cropsInventory.totalQuantityKg, f.minQty));
29   if (f.province) where.push(inArray(cropsInventory.farmLocation, Object.keys(CITIES).filter((c) => CITIES
  ↳ [c].province === f.province)));
30   if (f.maxPrice !== undefined) where.push(lte(cropsInventory.basePricePerKg, f.maxPrice));
31   if (f.q) where.push(ilike(cropsInventory.cropName, `${f.q}%`));
32   if (f.farmerId) where.push(eq(cropsInventory.farmerId, f.farmerId));
33   if (f.status !== "all") where.push(eq(cropsInventory.status, "active"));
34   else where.push(inArray(cropsInventory.status, ["active", "pending_inspection", "sold_out"]));
35
36   const rows = await db
37     .select({
38       crop: cropsInventory,
39       farmerName: users.fullName,
40       farmName: users.businessName,
41       farmerVerified: users.isVerified,
42       trustScore: users.trustScore,
43       ratingCount: users.ratingCount,
44       openBids: sql<number>`(select count(*) from bids_negotiations b where b.crop_id = ${cropsInventory.c
  ↳ ropId} and b.status in ('pending','countered'))`.mapWith(Number),
45       highestBid: sql<number | null>`(select max(b.bid_price_per_kg) from bids_negotiations b where b.crop
  ↳ _id = ${cropsInventory.cropId} and b.status in ('pending','countered'))`.mapWith((v) => (v === null ? null
  ↳ : Number(v))),
46       inspectionGrade: sql<string | null>`(select qi.grade_assigned from quality_inspections qi where qi.c
  ↳ rop_id = ${cropsInventory.cropId} and qi.status = 'passed' order by qi.verified_at desc limit 1)`,
47     })
48     .from(cropsInventory)
49     .innerJoin(users, eq(users.id, cropsInventory.farmerId))
50     .where(and(...where))
51     .orderBy(desc(cropsInventory.createdAt));
52
53   const origin = f.originCity ? CITIES[f.originCity] : undefined;
54   let result = rows.map((r) => {
55     const dist =
56       origin && r.crop.latitude !== null && r.crop.longitude !== null
57       ? Math.round(distanceKm(origin, { lat: r.crop.latitude, lng: r.crop.longitude }))
58       : null;
59     return { ...r.crop, farmerName: r.farmerName, farmName: r.farmName, farmerVerified: r.farmerVerified,
  ↳ trustScore: r.trustScore, ratingCount: r.ratingCount, openBids: r.openBids, highestBid: r.highestBid, insp
  ↳ ectionGrade: r.inspectionGrade, distanceKm: dist };
60   });
61   if (origin && f.radiusKm) result = result.filter((c) => c.distanceKm === null || c.distanceKm <= f.radiu
  ↳ sKm!);
62   if (f.sort === "price_asc") result.sort((a, b) => a.basePricePerKg - b.basePricePerKg);
63   if (f.sort === "price_desc") result.sort((a, b) => b.basePricePerKg - a.basePricePerKg);
64   if (f.sort === "distance") result.sort((a, b) => (a.distanceKm ?? 1e9) - (b.distanceKm ?? 1e9));
65   return result;
66 }
67
68 export async function getCrop(cropId: number) {
69   const [row] = await db
70     .select({ crop: cropsInventory, farmerName: users.fullName, farmName: users.businessName, farmerVerifi
  ↳ ed: users.isVerified, phone: users.phone, farmerCity: users.city, trustScore: users.trustScore, ratingAvg:
  ↳ users.ratingAvg, ratingCount: users.ratingCount, memberSince: users.createdAt })
71     .from(cropsInventory)
72     .innerJoin(users, eq(users.id, cropsInventory.farmerId))
73     .where(eq(cropsInventory.cropId, cropId))
74     .limit(1);
75   if (!row) throw new ApiError(404, "Crop listing not found");
76   const inspections = await db
77     .select()
78     .from(qualityInspections)
79     .where(eq(qualityInspections.cropId, cropId))
80     .orderBy(desc(qualityInspections.inspectionId));
81   return { ...row.crop, farmerName: row.farmerName, farmName: row.farmName, farmerVerified: row.farmerVeri
  ↳ fied, farmerCity: row.farmerCity, trustScore: row.trustScore, ratingAvg: row.ratingAvg, ratingCount: row.r
  ↳ atingCount, memberSince: row.memberSince, inspections: inspections.map((i) => ({ ...i, reportAttachment: i
  ↳ .reportAttachment ? "attached" : null })) };
82 }
83
84 export type CropInput = {
85   cropName: string;
86   category: Category;
87   qualityGrade: Grade;
88   totalQuantityKg: number;
89   basePricePerKg: number;
90   harvestDate: string;

```

```

91   farmLocation: string;
92   description?: string;
93   imagesJson?: string[];
94 };
95
96 function validateImages(images: unknown): string[] {
97   if (!images) return [];
98   if (!Array.isArray(images)) throw new ApiError(422, "imagesJson must be an array");
99   if (images.length > 4) throw new ApiError(422, "Maximum 4 images per listing");
100  for (const i of images) {
101    if (typeof i !== "string") throw new ApiError(422, "Invalid image");
102    if (!(i.startsWith("data:image/") || i.startsWith("https://"))) throw new ApiError(422, "Images must be
    e uploaded files or https URLs");
103    if (i.length > 1_500_000) throw new ApiError(422, "Image too large (max ~1MB after compression)");
104  }
105  return images as string[];
106 }
107
108 export async function createCrop(farmer: SessionUser, input: CropInput) {
109   if (new Date(input.harvestDate) > new Date(Date.now() + 86400000)) throw new ApiError(422, "Harvest date
    cannot be in the future");
110   const loc = CITIES[input.farmLocation];
111   const [row] = await db
112     .insert(cropsInventory)
113     .values({
114       farmerId: farmer.id,
115       cropName: input.cropName,
116       category: input.category,
117       qualityGrade: input.qualityGrade,
118       totalQuantityKg: input.totalQuantityKg,
119       basePricePerKg: input.basePricePerKg,
120       harvestDate: input.harvestDate,
121       farmLocation: input.farmLocation,
122       latitude: loc?.lat ?? null,
123       longitude: loc?.lng ?? null,
124       description: input.description,
125       imagesJson: validateImages(input.imagesJson),
126       // Business rule: listings from unverified farmers require inspection before going live
127       status: farmer.isVerified ? "active" : "pending_inspection",
128     })
129     .returning();
130   return row;
131 }
132
133 export async function updateCrop(user: SessionUser, cropId: number, patch: Partial<CropInput> & { status?:
    "active" | "archived" }) {
134   const [existing] = await db.select().from(cropsInventory).where(eq(cropsInventory.cropId, cropId)).limit
    (1);
135   if (!existing) throw new ApiError(404, "Crop listing not found");
136   if (user.role !== "admin" && existing.farmerId !== user.id) throw new ApiError(403, "You can only edit y
    our own listings");
137   if (patch.status === "active" && existing.status === "pending_inspection" && user.role !== "admin")
138     throw new ApiError(409, "Listing is awaiting quality inspection");
139   const loc = patch.farmLocation ? CITIES[patch.farmLocation] : undefined;
140   const [row] = await db
141     .update(cropsInventory)
142     .set({
143       ...(patch.cropName && { cropName: patch.cropName }),
144       ...(patch.category && { category: patch.category }),
145       ...(patch.qualityGrade && { qualityGrade: patch.qualityGrade }),
146       ...(patch.totalQuantityKg !== undefined && { totalQuantityKg: patch.totalQuantityKg, ...(existing.st
    atus === "sold_out" && patch.totalQuantityKg > 0 ? { status: "active" as const } : {}) }),
147       ...(patch.basePricePerKg !== undefined && { basePricePerKg: patch.basePricePerKg }),
148       ...(patch.harvestDate && { harvestDate: patch.harvestDate }),
149       ...(patch.description !== undefined && { description: patch.description }),
150       ...(patch.imagesJson && { imagesJson: validateImages(patch.imagesJson) }),
151       ...(patch.farmLocation && { farmLocation: patch.farmLocation, latitude: loc?.lat ?? null, longitude:
    loc?.lng ?? null }),
152       ...(patch.status && { status: patch.status }),
153     })
154     .where(eq(cropsInventory.cropId, cropId))
155     .returning();
156   return row;
157 }
158
159 export async function deleteCrop(user: SessionUser, cropId: number) {

```

```

160 const [existing] = await db.select().from(cropsInventory).where(eq(cropsInventory.cropId, cropId)).limit
    (1);
161 if (!existing) throw new ApiError(404, "Crop listing not found");
162 if (user.role !== "admin" && existing.farmerId !== user.id) throw new ApiError(403, "You can only delete
    your own listings");
163 const [{ n }] = await db.select({ n: sql<number>`count(*)`.mapWith(Number) }).from(ordersLogistics).wher
    e(eq(ordersLogistics.cropId, cropId));
164 if (n > 0) {
165     // Soft-delete: orders reference this crop
166     await db.update(cropsInventory).set({ status: "archived" }).where(eq(cropsInventory.cropId, cropId));
167     await db.execute(sql`update bids_negotiations set status = 'rejected', updated_at = now() where crop_i
    d = ${cropId} and status in ('pending','countered')`);
168     return { archived: true };
169 }
170 await db.delete(cropsInventory).where(eq(cropsInventory.cropId, cropId));
171 return { deleted: true };
172 }
173

```

src/lib/services/mandi.ts

```

1 import { and, asc, eq, gte, sql } from "drizzle-orm";
2 import { db } from "@db";
3 import { mandiRates } from "@db/schema";
4 import { ApiError } from "@lib/api";
5 import { CROP_CATALOGUE, MANDI_MARKETS, todayISO } from "@lib/constants";
6
7 const n = (v: unknown) => (v === null || v === undefined ? null : Number(v));
8 const r2 = (x: number) => Math.round(x * 100) / 100;
9
10 export type LatestRate = {
11     rateId: number;
12     cropName: string;
13     marketLocation: string;
14     minPricePerKg: number;
15     maxPricePerKg: number;
16     avgPricePerKg: number;
17     prevAvg: number | null;
18     changePct: number | null;
19     source: string;
20     dateUpdated: string;
21 };
22
23 export async function latestRates(filters: { crop?: string; market?: string } = {}): Promise<LatestRate[]>
    {
24     const res = await db.execute(sql`
25         with ranked as (
26             select *, row_number() over (partition by crop_name, market_location order by date_updated desc) as
27             rn
28             from mandi_rates
29         )
30         select a.rate_id, a.crop_name, a.market_location, a.min_price_per_kg, a.max_price_per_kg, a.avg_price_
31         per_kg,
32         a.source, a.date_updated::text as date_updated, b.avg_price_per_kg as prev_avg
33         from ranked a
34         left join ranked b on b.crop_name = a.crop_name and b.market_location = a.market_location and b.rn = 2
35         where a.rn = 1
36         ${filters.crop ? sql`and a.crop_name = ${filters.crop}` : sql``}
37         ${filters.market ? sql`and a.market_location = ${filters.market}` : sql``}
38         order by a.crop_name, a.market_location`;
39     return (res.rows as Record<string, unknown>[]).map((r) => {
40         const avg = Number(r.avg_price_per_kg);
41         const prev = n(r.prev_avg);
42         return {
43             rateId: Number(r.rate_id),
44             cropName: String(r.crop_name),
45             marketLocation: String(r.market_location),
46             minPricePerKg: Number(r.min_price_per_kg),
47             maxPricePerKg: Number(r.max_price_per_kg),
48             avgPricePerKg: avg,
49             prevAvg: prev,
50             changePct: prev ? r2(((avg - prev) / prev) * 100) : null,
51             source: String(r.source),
52             dateUpdated: String(r.date_updated),
53         };
54     });
55 }

```

```

52     });
53 }
54
55 /** Daily avg price per market for a crop – pivoted for charting. */
56 export async function rateTrend(cropName: string, days = 14) {
57     const since = todayISO(-days + 1);
58     const rows = await db
59         .select()
60         .from(mandiRates)
61         .where(and(eq(mandiRates.cropName, cropName), gte(mandiRates.dateUpdated, since)))
62         .orderBy(asc(mandiRates.dateUpdated));
63     const byDate = new Map<string, Record<string, number | string>>();
64     for (const r of rows) {
65         const d = byDate.get(r.dateUpdated) ?? { date: r.dateUpdated };
66         d[r.marketLocation] = r.avgPricePerKg;
67         byDate.set(r.dateUpdated, d);
68     }
69     return { cropName, markets: MANDI_MARKETS, series: [...byDate.values()] };
70 }
71
72 /** Mandi (market) rate vs. MarketLink platform rates per crop. */
73 export async function platformComparison() {
74     const res = await db.execute(sql`
75         with latest as (
76             select distinct on (crop_name, market_location) crop_name, avg_price_per_kg
77             from mandi_rates order by crop_name, market_location, date_updated desc
78         ),
79         mandi as (select crop_name, avg(avg_price_per_kg) as mandi_avg from latest group by crop_name),
80         ask as (select crop_name, avg(base_price_per_kg) as ask_avg, sum(total_quantity_kg) as supply_kg from
81         crops_inventory where status = 'active' group by crop_name),
82         deal as (
83             select c.crop_name, sum(o.total_amount) / nullif(sum(o.quantity_kg), 0) as deal_avg
84             from orders_logistics o join crops_inventory c on c.crop_id = o.crop_id
85             where o.created_at > now() - interval '90 days' and o.payment_status <> 'refunded'
86             group by c.crop_name
87         )
88         select m.crop_name, m.mandi_avg, a.ask_avg, a.supply_kg, d.deal_avg
89         from mandi m left join ask a on a.crop_name = m.crop_name left join deal d on d.crop_name = m.crop_name
90     `);
91     return (res.rows as Record<string, unknown>[]).map((r) => {
92         const mandi = r2(Number(r.mandi_avg));
93         const ask = n(r.ask_avg);
94         const deal = n(r.deal_avg);
95         return {
96             cropName: String(r.crop_name),
97             mandiAvg: mandi,
98             platformAsk: ask === null ? null : r2(ask),
99             platformDeal: deal === null ? null : r2(deal),
100             supplyKg: n(r.supply_kg) ?? 0,
101             farmerGainPct: deal ? r2(((deal - mandi) / mandi) * 100) : null,
102         };
103     });
104 }
105
106 export type RateInput = { cropName: string; marketLocation: string; minPricePerKg: number; maxPricePerKg:
107     number; avgPricePerKg?: number; dateUpdated?: string; source?: string };
108
109 export async function upsertRate(input: RateInput) {
110     if (input.minPricePerKg > input.maxPricePerKg) throw new ApiError(422, "Min price cannot exceed max price");
111     const avg = input.avgPricePerKg ?? r2((input.minPricePerKg + input.maxPricePerKg) / 2);
112     if (avg < input.minPricePerKg || avg > input.maxPricePerKg) throw new ApiError(422, "Average must be between min and max");
113     const values = {
114         cropName: input.cropName,
115         marketLocation: input.marketLocation,
116         minPricePerKg: input.minPricePerKg,
117         maxPricePerKg: input.maxPricePerKg,
118         avgPricePerKg: avg,
119         source: input.source ?? "manual",
120         dateUpdated: input.dateUpdated ?? todayISO(),
121     };
122     const [row] = await db
123         .insert(mandiRates)
124         .values(values)

```

```

123     .onConflictDoUpdate({
124       target: [mandiRates.cropName, mandiRates.marketLocation, mandiRates.dateUpdated],
125       set: { minPricePerKg: values.minPricePerKg, maxPricePerKg: values.maxPricePerKg, avgPricePerKg: valu
126     ↵ es.avgPricePerKg, source: values.source },
127     })
128     .returning();
129     return row;
130   }
131 }
132 /**
133  * Mock external feed (e.g. AMIS / Bureau of Statistics) sync:
134  * takes each crop x market latest rate and applies a bounded random walk (±3%)
135  * then upserts today's rate with source = 'feed'.
136  */
137 export async function syncMockFeed() {
138   const latest = await latestRates();
139   const today = todayISO();
140   const known = new Set(latest.map((l) => `${l.cropName}|${l.marketLocation}`));
141   let updated = 0;
142   await db.transaction(async (tx) => {
143     const rows: (typeof mandiRates.$inferInsert)[] = [];
144     for (const l of latest) {
145       const drift = 1 + (Math.random() - 0.5) * 0.06;
146       const avg = r2(l.avgPricePerKg * drift);
147       const spread = r2(avg * (0.05 + Math.random() * 0.05));
148       rows.push({ cropName: l.cropName, marketLocation: l.marketLocation, minPricePerKg: r2(avg - spread),
149     ↵ maxPricePerKg: r2(avg + spread), avgPricePerKg: avg, source: "feed", dateUpdated: today });
150     }
151     // Seed any catalogue crop/market combination missing from history
152     for (const c of CROP_CATALOGUE)
153       for (const m of MANDI_MARKETS)
154         if (!known.has(`${c.name}|${m}`)) {
155           const avg = c.mandiBase;
156           rows.push({ cropName: c.name, marketLocation: m, minPricePerKg: r2(avg * 0.93), maxPricePerKg: r
157     ↵ 2(avg * 1.07), avgPricePerKg: avg, source: "feed", dateUpdated: today });
158         }
159     if (rows.length) {
160       await tx
161         .insert(mandiRates)
162         .values(rows)
163         .onConflictDoUpdate({
164           target: [mandiRates.cropName, mandiRates.marketLocation, mandiRates.dateUpdated],
165           set: {
166             minPricePerKg: sql`excluded.min_price_per_kg`,
167             maxPricePerKg: sql`excluded.max_price_per_kg`,
168             avgPricePerKg: sql`excluded.avg_price_per_kg`,
169             source: sql`excluded.source`,
170           },
171         });
172     }
173     updated = rows.length;
174   });
175   return { updated, date: today, syncedAt: new Date().toISOString() };
176 }

```

src/lib/services/orders.ts

```

1  import { and, asc, desc, eq, inArray, or, sql } from "drizzle-orm";
2  import { alias } from "drizzle-orm/pg-core";
3  import { db } from "@db";
4  import { ordersLogistics, orderEvents, cropsInventory, users, walletTransactions, disputes, qualityInspect
5  ↵ ions } from "@db/schema";
6  import { ApiError } from "@lib/api";
7  import type { SessionUser } from "@lib/auth";
8  import { DELIVERY_STAGES, PLATFORM_FEE_RATE, nextStage, type DeliveryStage } from "@lib/constants";
9  import { notify } from "@lib/notify";
10 import { geocode } from "@lib/services/tracking";
11 import { recomputeTrust } from "@lib/services/reviews";
12
13 type Tx = Parameters<Parameters<typeof db.transaction>[0]>[0];
14 const farmer = alias(users, "farmer");
15 const buyer = alias(users, "buyer");
16 const r2 = (n: number) => Math.round(n * 100) / 100;

```

```

17 export async function listOrders(user: SessionUser, opts: { active?: boolean; limit?: number } = {}) {
18   const conds = [];
19   if (user.role === "farmer") conds.push(eq(ordersLogistics.farmerId, user.id));
20   if (user.role === "buyer") conds.push(eq(ordersLogistics.buyerId, user.id));
21   if (opts.active) conds.push(or(inArray(ordersLogistics.deliveryStage, ["confirmed", "quality_checked", "
  ↳ dispatched", "in_transit"]), eq(ordersLogistics.paymentStatus, "disputed"))!);
22   const rows = await db
23     .select({
24       order: ordersLogistics,
25       cropName: cropsInventory.cropName,
26       category: cropsInventory.category,
27       qualityGrade: cropsInventory.qualityGrade,
28       farmLocation: cropsInventory.farmLocation,
29       imagesJson: cropsInventory.imagesJson,
30       farmerName: farmer.fullName,
31       farmName: farmer.businessName,
32       buyerName: buyer.fullName,
33       buyerBusiness: buyer.businessName,
34       buyerCity: buyer.city,
35     })
36     .from(ordersLogistics)
37     .innerJoin(cropsInventory, eq(cropsInventory.cropId, ordersLogistics.cropId))
38     .innerJoin(farmer, eq(farmer.id, ordersLogistics.farmerId))
39     .innerJoin(buyer, eq(buyer.id, ordersLogistics.buyerId))
40     .where(conds.length ? and(...conds) : undefined)
41     .orderBy(desc(ordersLogistics.createdAt))
42     .limit(opts.limit ?? 200);
43
44   const ids = rows.map((r) => r.order.orderId);
45   const events = ids.length
46     ? await db.select().from(orderEvents).where(inArray(orderEvents.orderId, ids)).orderBy(asc(orderEvents
  ↳ .createdAt))
47     : [];
48   const openDisputes = ids.length
49     ? await db.select().from(disputes).where(inArray(disputes.orderId, ids)).orderBy(desc(disputes.created
  ↳ At))
50     : [];
51   return rows.map(({ order, ...rest }) => ({
52     ...order,
53     ...rest,
54     events: events.filter((e) => e.orderId === order.orderId),
55     disputes: openDisputes.filter((d) => d.orderId === order.orderId),
56   }));
57 }
58
59 export type OrderView = Awaited<ReturnType<typeof listOrders>>[number];
60
61 async function lockOrder(tx: Tx, orderId: number) {
62   const [o] = await tx.select().from(ordersLogistics).where(eq(ordersLogistics.orderId, orderId)).for("upd
  ↳ ate").limit(1);
63   if (!o) throw new ApiError(404, "Order not found");
64   return o;
65 }
66
67 function assertParty(user: SessionUser, o: typeof ordersLogistics.$inferSelect) {
68   if (user.role === "admin" || user.role === "inspector") return;
69   if (user.id !== o.farmerId && user.id !== o.buyerId) throw new ApiError(403, "Not a party to this order"
  ↳ );
70 }
71
72 /** Buyer locks the order amount from wallet into escrow. */
73 async function lockEscrowTx(user: SessionUser, orderId: number) {
74   return db.transaction(async (tx) => {
75     const o = await lockOrder(tx, orderId);
76     if (user.role !== "buyer" || o.buyerId !== user.id) throw new ApiError(403, "Only the buyer can fund e
  ↳ scrow");
77     if (o.paymentStatus !== "awaiting_escrow") throw new ApiError(409, `Escrow already ${o.paymentStatus.r
  ↳ eplace("_", " ")}`);
78     const [b] = await tx.select().from(users).where(eq(users.id, user.id)).for("update").limit(1);
79     if (b.walletBalance < o.totalAmount) throw new ApiError(402, `Insufficient wallet balance. Need Rs. ${
  ↳ o.totalAmount.toLocaleString()}, have Rs. ${b.walletBalance.toLocaleString()}`);
80     await tx.update(users).set({ walletBalance: r2(b.walletBalance - o.totalAmount), escrowBalance: r2(b.e
  ↳ scrowBalance + o.totalAmount) }).where(eq(users.id, b.id));
81     await tx.insert(walletTransactions).values({ userId: b.id, orderId, type: "escrow_lock", amount: o.tot
  ↳ alAmount, description: `Escrow locked for ${o.trackingNumber}` });
82     const [updated] = await tx.update(ordersLogistics).set({ paymentStatus: "escrow_locked", updatedAt: ne

```

```

82 w Date() }).where(eq(ordersLogistics.orderId, orderId)).returning();
83     await tx.insert(orderEvents).values({ orderId, stage: "escrow_locked", note: "Buyer funds secured in M
84     arketLink escrow", actorId: user.id });
85     return updated;
86   });
87 }
88 async function releaseEscrow(tx: Tx, o: typeof ordersLogistics.$inferSelect, actorId: number, note: string
89 ) {
90   const [b] = await tx.select().from(users).where(eq(users.id, o.buyerId)).for("update").limit(1);
91   const [f] = await tx.select().from(users).where(eq(users.id, o.farmerId)).for("update").limit(1);
92   const fee = r2(o.totalAmount * PLATFORM_FEE_RATE);
93   const payout = r2(o.totalAmount - fee);
94   await tx.update(users).set({ escrowBalance: r2(Math.max(0, b.escrowBalance - o.totalAmount)) }).where(eq
95   (users.id, b.id));
96   await tx.update(users).set({ walletBalance: r2(f.walletBalance + payout) }).where(eq(users.id, f.id));
97   await tx.insert(walletTransactions).values([
98     { userId: b.id, orderId: o.orderId, type: "escrow_release", amount: o.totalAmount, description: `Escro
99     w released for ${o.trackingNumber}` },
100    { userId: f.id, orderId: o.orderId, type: "payout", amount: payout, description: `Payout for ${o.track
101    ingNumber}` },
102    { userId: f.id, orderId: o.orderId, type: "platform_fee", amount: fee, description: "1.5% platform fee
103    " },
104  ]);
105  await tx.insert(orderEvents).values({ orderId: o.orderId, stage: "escrow_released", note, actorId });
106 }
107
108 async function refundEscrow(tx: Tx, o: typeof ordersLogistics.$inferSelect, actorId: number, note: string)
109 {
110   const [b] = await tx.select().from(users).where(eq(users.id, o.buyerId)).for("update").limit(1);
111   await tx.update(users).set({ escrowBalance: r2(Math.max(0, b.escrowBalance - o.totalAmount)), walletBala
112   nce: r2(b.walletBalance + o.totalAmount) }).where(eq(users.id, b.id));
113   await tx.insert(walletTransactions).values({ userId: b.id, orderId: o.orderId, type: "refund", amount: o
114   .totalAmount, description: `Refund for ${o.trackingNumber}` });
115   await tx.insert(orderEvents).values({ orderId: o.orderId, stage: "escrow_refunded", note, actorId });
116 }
117
118 /**
119  * Order status progression:
120  * confirmed -> quality_checked (admin / quality inspector; requires escrow_locked)
121  * quality_checked -> dispatched (farmer or admin)
122  * dispatched -> in_transit (farmer or admin, with transit location)
123  * in_transit -> delivered (buyer confirms receipt -> escrow auto-released to farmer)
124  */
125 async function advanceOrderTx(
126   user: SessionUser,
127   orderId: number,
128   input: { location?: string; note?: string; moisture?: number; grade?: "A" | "B" | "C" },
129 ) {
130   return db.transaction(async (tx) => {
131     const o = await lockOrder(tx, orderId);
132     assertParty(user, o);
133     if (o.paymentStatus === "disputed") throw new ApiError(409, "Order is under dispute; progression is fr
134     ozen");
135     if (o.paymentStatus === "refunded") throw new ApiError(409, "Order was refunded");
136     const next = nextStage(o.deliveryStage as DeliveryStage);
137     if (!next) throw new ApiError(409, "Order already delivered");
138     if (o.paymentStatus !== "escrow_locked") throw new ApiError(409, "Buyer must lock funds in escrow befo
139     re the order can progress");
140
141     const allowed: Record<DeliveryStage, SessionUser["role"][]> = {
142       confirmed: [],
143       quality_checked: ["inspector", "admin"],
144       dispatched: ["farmer", "admin"],
145       in_transit: ["farmer", "admin"],
146       delivered: ["buyer", "admin"],
147     };
148     if (!allowed[next].includes(user.role)) {
149       const label = DELIVERY_STAGES.find((s) => s.key === next)!.label;
150       throw new ApiError(403, `${label} must be performed by: ${allowed[next].join(" / ")}`);
151     }
152     if (user.role === "farmer" && o.farmerId !== user.id) throw new ApiError(403, "Not your order");
153     if (user.role === "buyer" && o.buyerId !== user.id) throw new ApiError(403, "Not your order");
154
155     const location = input.location || (next === "delivered" ? "Buyer warehouse" : o.transitLocation) || n
156     ull;

```

```

146     const now = new Date();
147
148     if (next === "quality_checked") {
149         await tx.insert(qualityInspections).values({
150             cropId: o.cropId,
151             inspectorId: user.id,
152             gradeAssigned: input.grade ?? "A",
153             moistureLevelPercentage: input.moisture ?? 12,
154             inspectionNotes: input.note ?? `Pre-dispatch inspection for ${o.trackingNumber}`,
155             status: "passed",
156             verifiedAt: now,
157         });
158     }
159
160     const [updated] = await tx
161         .update(ordersLogistics)
162         .set({ deliveryStage: next, transitLocation: location, updatedAt: now, ...(next === "delivered" ? {
163             ↵ paymentStatus: "released" as const } : {}) })
164         .where(eq(ordersLogistics.orderId, orderId))
165         .returning();
166     const pt = geocode(location);
167     if (pt) await tx.update(ordersLogistics).set({ currentLat: pt.lat, currentLng: pt.lng }).where(eq(orde
168     ↵ rsLogistics.orderId, orderId));
169     await tx.insert(orderEvents).values({
170         orderId,
171         stage: next,
172         location,
173         lat: pt?.lat ?? null,
174         lng: pt?.lng ?? null,
175         note: input.note || DELIVERY_STAGES.find((s) => s.key === next)?.label,
176         actorId: user.id,
177     });
178     if (next === "delivered") {
179         await releaseEscrow(tx, o, user.id, "Delivery verified by buyer. Escrow released to farmer (1.5% pla
180         ↵ tform fee deducted).");
181     }
182     return updated;
183 }
184
185 export async function updateTransitLocation(user: SessionUser, orderId: number, location: string) {
186     return db.transaction(async (tx) => {
187         const o = await lockOrder(tx, orderId);
188         if (!(user.role === "admin" || user.id === o.farmerId)) throw new ApiError(403, "Only the farmer or ad
189         ↵ min can update location");
190         if (!["dispatched", "in_transit"].includes(o.deliveryStage)) throw new ApiError(409, "Location updates
191         ↵ are only available while shipment is moving");
192         const [u] = await tx.update(ordersLogistics).set({ transitLocation: location, updatedAt: new Date() })
193         ↵ .where(eq(ordersLogistics.orderId, orderId)).returning();
194         await tx.insert(orderEvents).values({ orderId, stage: "location_update", location, note: `Shipment che
195         ↵ ckpoint: ${location}`, actorId: user.id });
196         return u;
197     });
198 }
199
200 async function raiseDisputeTx(user: SessionUser, orderId: number, reason: string) {
201     return db.transaction(async (tx) => {
202         const o = await lockOrder(tx, orderId);
203         if (user.id !== o.buyerId && user.id !== o.farmerId) throw new ApiError(403, "Only order parties can r
204         ↵ aise disputes");
205         if (o.paymentStatus !== "escrow_locked") throw new ApiError(409, "Disputes can only be raised while fu
206         ↵ nds are held in escrow");
207         await tx.update(ordersLogistics).set({ paymentStatus: "disputed", updatedAt: new Date() }).where(eq(or
208         ↵ dersLogistics.orderId, orderId));
209         const [d] = await tx.insert(disputes).values({ orderId, raisedBy: user.id, reason, status: "open" }).r
210         ↵ eturning();
211         await tx.insert(orderEvents).values({ orderId, stage: "dispute_opened", note: reason, actorId: user.id
212         ↵ });
213         return d;
214     });
215 }
216
217 export async function listDisputes() {
218     const raiser = alias(users, "raiser");
219     return db
220         .select({

```

```

210     dispute: disputes,
211     trackingNumber: ordersLogistics.trackingNumber,
212     totalAmount: ordersLogistics.totalAmount,
213     paymentStatus: ordersLogistics.paymentStatus,
214     deliveryStage: ordersLogistics.deliveryStage,
215     raisedByName: raiser.fullName,
216     raisedByRole: raiser.role,
217     cropName: cropsInventory.cropName,
218   })
219   .from(disputes)
220   .innerJoin(ordersLogistics, eq(ordersLogistics.orderId, disputes.orderId))
221   .innerJoin(raiser, eq(raiser.id, disputes.raisedBy))
222   .innerJoin(cropsInventory, eq(cropsInventory.cropId, ordersLogistics.cropId))
223   .orderBy(sql`case when ${disputes.status} in ('open','investigating') then 0 else 1 end`, desc(dispute
  ↵ s.createdAt))
224   .then((rows) => rows.map(({ dispute, ...rest }) => ({ ...dispute, ...rest })));
225 }
226
227 async function resolveDisputeTx(admin: SessionUser, disputeId: number, outcome: "investigating" | "release
  ↵ " | "refund", resolution?: string) {
228   return db.transaction(async (tx) => {
229     const [d] = await tx.select().from(disputes).where(eq(disputes.disputeId, disputeId)).for("update").li
  ↵ mit(1);
230     if (!d) throw new ApiError(404, "Dispute not found");
231     if (d.status === "resolved_release" || d.status === "resolved_refund") throw new ApiError(409, "Disput
  ↵ e already resolved");
232     if (outcome === "investigating") {
233       const [u] = await tx.update(disputes).set({ status: "investigating", resolution: resolution ?? d.res
  ↵ olution }).where(eq(disputes.disputeId, disputeId)).returning();
234       return u;
235     }
236     const o = await lockOrder(tx, d.orderId);
237     if (!resolution) throw new ApiError(422, "Resolution notes are required");
238     if (o.paymentStatus === "disputed") {
239       if (outcome === "release") {
240         await releaseEscrow(tx, o, admin.id, `Dispute resolved in farmer's favour: ${resolution}`);
241         await tx.update(ordersLogistics).set({ paymentStatus: "released", deliveryStage: "delivered", upda
  ↵ tedAt: new Date() }).where(eq(ordersLogistics.orderId, o.orderId));
242       } else {
243         await refundEscrow(tx, o, admin.id, `Dispute resolved in buyer's favour: ${resolution}`);
244         await tx.update(ordersLogistics).set({ paymentStatus: "refunded", updatedAt: new Date() }).where(e
  ↵ q(ordersLogistics.orderId, o.orderId));
245       }
246     }
247     const [u] = await tx
248       .update(disputes)
249       .set({ status: outcome === "release" ? "resolved_release" : "resolved_refund", resolution, resolvedA
  ↵ t: new Date() })
250       .where(eq(disputes.disputeId, disputeId))
251       .returning();
252     return u;
253   });
254 }
255
256 export async function getWallet(user: SessionUser) {
257   const [u] = await db.select({ walletBalance: users.walletBalance, escrowBalance: users.escrowBalance }).
  ↵ from(users).where(eq(users.id, user.id));
258   const txns = await db.select().from(walletTransactions).where(eq(walletTransactions.userId, user.id)).or
  ↵ derBy(desc(walletTransactions.createdAt)).limit(100);
259   const pending = await db
260     .select({ total: sql<number>`coalesce(sum(${ordersLogistics.totalAmount}),0)`}.mapWith(Number) })
261     .from(ordersLogistics)
262     .where(and(user.role === "farmer" ? eq(ordersLogistics.farmerId, user.id) : eq(ordersLogistics.buyerId
  ↵ , user.id), inArray(ordersLogistics.paymentStatus, ["escrow_locked", "disputed"])));
263   return { ...u, pendingInEscrow: pending[0].total, transactions: txns };
264 }
265
266 export async function deposit(user: SessionUser, amount: number) {
267   return db.transaction(async (tx) => {
268     const [u] = await tx.select().from(users).where(eq(users.id, user.id)).for("update");
269     await tx.update(users).set({ walletBalance: r2(u.walletBalance + amount) }).where(eq(users.id, user.id
  ↵ ));
270     await tx.insert(walletTransactions).values({ userId: user.id, type: "deposit", amount, description: "W
  ↵ allet top-up via IBFT / JazzCash / Easypaisa (simulated) });
271     return { walletBalance: r2(u.walletBalance + amount) };
272   });

```

```

273 }
274
275 /* ----- */
276 /* Public API with automated milestone notifications */
277 /* ----- */
278 const rs = (n: number) => "Rs. " + Math.round(n).toLocaleString("en-US");
279
280 async function orderInfo(orderId: number) {
281   const [r] = await db
282     .select({ o: ordersLogistics, cropName: cropsInventory.cropName })
283     .from(ordersLogistics)
284     .innerJoin(cropsInventory, eq(cropsInventory.cropId, ordersLogistics.cropId))
285     .where(eq(ordersLogistics.orderId, orderId))
286     .limit(1);
287   return r;
288 }
289
290 export async function lockEscrow(user: SessionUser, orderId: number) {
291   const res = await lockEscrowTx(user, orderId);
292   const i = await orderInfo(orderId);
293   if (i) {
294     await notify(i.o.farmerId, { type: "escrow_locked", title: "Escrow funded: prepare for inspection", bo
295 dy: `The buyer secured ${rs(i.o.totalAmount)} in escrow for ${i.cropName} (${i.o.trackingNumber}). A quali
296 ty inspector will verify the lot before dispatch.`, link: "/farmer/orders", email: true, sms: true });
297     const inspectors = await db.select({ id: users.id }).from(users).where(eq(users.role, "inspector"));
298     await Promise.all(inspectors.map((x) => notify(x.id, { type: "inspection", title: "Pre-dispatch inspec
299 tion needed", body: `${i.cropName}, ${i.o.quantityKg.toLocaleString("en-US")} kg (${i.o.trackingNumber}) i
300 s ready for quality inspection.`, link: "/inspector/dashboard" })));
301   }
302   return res;
303 }
304
305 export async function advanceOrder(user: SessionUser, orderId: number, input: { location?: string; note?:
306 string; moisture?: number; grade?: "A" | "B" | "C" }) {
307   const res = await advanceOrderTx(user, orderId, input);
308   const i = await orderInfo(orderId);
309   if (i) {
310     const { o, cropName } = i;
311     const track = `/tracking?no=${o.trackingNumber}`;
312     if (o.deliveryStage === "quality_checked") {
313       await notify(o.farmerId, { type: "inspection", title: "Quality inspection passed", body: `${cropName
314 } (${o.trackingNumber}) passed inspection. You can dispatch now.`, link: "/farmer/orders" });
315       await notify(o.buyerId, { type: "inspection", title: "Your order passed quality inspection", body: `
316 ${cropName} (${o.trackingNumber}) was graded ${input.grade ?? "A"} by a MarketLink inspector.`, link: trac
317 k });
318     } else if (o.deliveryStage === "dispatched") {
319       await notify(o.buyerId, { type: "shipment", title: "Shipment dispatched", body: `${cropName} (${o.tr
320 ackingNumber}) left ${o.transitLocation ?? "the farm"}.${input.note ? ` ${input.note}` : ""}`, link: track
321 , email: true, sms: true });
322     } else if (o.deliveryStage === "in_transit") {
323       await notify(o.buyerId, { type: "shipment", title: "Shipment on the road", body: `${cropName} (${o.t
324 rackingNumber}) is in transit. Follow it live on the map.`, link: track });
325     } else if (o.deliveryStage === "delivered") {
326       await notify(o.farmerId, { type: "delivered", title: "Delivered: payment released", body: `${cropNam
327 e} (${o.trackingNumber}) was delivered. ${rs(o.totalAmount * (1 - PLATFORM_FEE_RATE))} has been credited t
328 o your wallet.`, link: "/wallet", email: true, sms: true });
329       await notify(o.buyerId, { type: "delivered", title: "Order delivered: rate your seller", body: `Than
330 ks for confirming delivery of ${o.trackingNumber}. Tell other buyers how it went.`, link: "/buyer/dashboar
331 d", email: true });
332       await recomputeTrust(o.farmerId);
333     }
334   }
335   return res;
336 }
337
338 export async function raiseDispute(user: SessionUser, orderId: number, reason: string) {
339   const res = await raiseDisputeTx(user, orderId, reason);
340   const i = await orderInfo(orderId);
341   if (i) {
342     const other = user.id === i.o.buyerId ? i.o.farmerId : i.o.buyerId;
343     await notify(other, { type: "dispute", title: "A dispute was opened on your order", body: `${i.o.track
344 ingNumber}: "${reason.slice(0, 140)}". Escrow is frozen until an admin resolves it.`, link: user.id === i.
345 o.buyerId ? "/farmer/orders" : "/buyer/orders", email: true });
346     const admins = await db.select({ id: users.id }).from(users).where(eq(users.role, "admin"));
347     await Promise.all(admins.map((a) => notify(a.id, { type: "dispute", title: "New dispute", body: `${i.o
348 .trackingNumber} (${rs(i.o.totalAmount)}): ${reason.slice(0, 140)}`, link: "/admin/orders" })));
349   }
350 }

```

```

331     }
332     return res;
333 }
334
335 export async function resolveDispute(admin: SessionUser, disputeId: number, outcome: "investigating" | "re
    lease" | "refund", resolution?: string) {
336     const res = await resolveDisputeTx(admin, disputeId, outcome, resolution);
337     if (outcome !== "investigating") {
338         const [d] = await db.select({ orderId: disputes.orderId }).from(disputes).where(eq(disputes.disputeId,
    disputeId)).limit(1);
339         const i = d && (await orderInfo(d.orderId));
340         if (i) {
341             const msg = outcome === "release" ? "resolved in the farmer's favour. Payment released." : "resolved
    in the buyer's favour. The buyer was refunded.";
342             await notify(i.o.farmerId, { type: "dispute", title: "Dispute resolved", body: `${i.o.trackingNumber
    } ${msg}`, link: "/farmer/orders", email: true });
343             await notify(i.o.buyerId, { type: "dispute", title: "Dispute resolved", body: `${i.o.trackingNumber}
    ${msg}`, link: "/buyer/orders", email: true });
344             await recomputeTrust(i.o.farmerId);
345         }
346     }
347     return res;
348 }
349

```

src/lib/services/payments.ts

```

1  import { createCipheriv, createHmac, randomBytes, timingSafeEqual } from "crypto";
2  import { desc, eq, sql } from "drizzle-orm";
3  import { db } from "@db";
4  import { paymentIntents, payoutRequests, users, walletTransactions } from "@db/schema";
5  import { ApiError } from "@lib/api";
6  import { absoluteUrl, isValidHttpUrl, publicOrigin } from "@lib/base-url";
7  import type { SessionUser } from "@lib/auth";
8  import { notify } from "@lib/notify";
9
10 export type Provider = "stripe" | "jazzcash" | "easypaisa" | "sandbox";
11 const r2 = (n: number) => Math.round(n * 100) / 100;
12 const live = () => process.env.PAYMENTS_MODE === "live";
13
14 export function providerStatus(): Record<Provider, { enabled: boolean; label: string; mode: string }> {
15     return {
16         stripe: { enabled: !!process.env.STRIPE_SECRET_KEY, label: "Card (Stripe)", mode: process.env.STRIPE_S
    ECRET_KEY?.startsWith("sk_live") ? "live" : "test" },
17         jazzcash: { enabled: !(process.env.JAZZCASH_MERCHANT_ID && process.env.JAZZCASH_PASSWORD && process.e
    nv.JAZZCASH_INTEGRITY_SALT), label: "JazzCash", mode: live() ? "live" : "sandbox" },
18         easypaisa: { enabled: !(process.env.EASYPAISA_STORE_ID && process.env.EASYPAISA_HASH_KEY), label: "Ea
    sypaisa", mode: live() ? "live" : "sandbox" },
19         sandbox: { enabled: process.env.PAYMENTS_SANDBOX !== "false", label: "Test gateway (no real money)", m
    ode: "test" },
20     };
21 }
22
23 export type Checkout = { reference: string; redirectUrl?: string; form?: { action: string; fields: Record<
    string, string> } };
24
25 /**
26  * Step 1: create a pending payment intent and hand the browser to the gateway.
27  * The wallet is credited ONLY by a verified callback.
28  * Every URL returned here is validated: it must be an absolute http(s) URL, so a bad request host
29  * or proxy header can never send the buyer to a bind address (ERR_ADDRESS_INVALID).
30  */
31 export async function createCheckout(user: SessionUser, provider: Provider, amount: number, req: Request):
    Promise<Checkout> {
32     const checkout = await buildCheckout(user, provider, amount, req);
33     if (checkout.redirectUrl && !isValidHttpUrl(checkout.redirectUrl)) {
34         throw new ApiError(500, "Could not build a valid payment redirect URL. Please check the server's APP_U
    RL / forwarded host headers.");
35     }
36     if (checkout.form && !isValidHttpUrl(checkout.form.action)) {
37         throw new ApiError(500, "Could not build a valid gateway address. Please check the server's APP_URL /
    forwarded host headers.");
38     }
39     return checkout;
40 }

```

```

41
42 async function buildCheckout(user: SessionUser, provider: Provider, amount: number, req: Request): Promise
43 <Checkout> {
44   if (!providerStatus()[provider]?.enabled) throw new ApiError(400, `${provider} is not configured on this
45   server`);
46   if (amount < 1000 || amount > 50_000_000) throw new ApiError(422, "Amount must be between Rs. 1,000 and
47   Rs. 5 Crore");
48   const reference = `MLT${Date.now()}${randomBytes(3).toString("hex").toUpperCase()}`;
49   await db.insert(paymentIntents).values({ reference, userId: user.id, provider, amount: r2(amount) });
50   // The visitor's own host (x-forwarded-host / host) decides the origin – never the bind address
51   // the server listens on, and never a hardcoded localhost:3000.
52   const base = publicOrigin(req);
53   const successUrl = `${base}/wallet?payment=${reference}`;
54
55   if (provider === "sandbox") return { reference, redirectUrl: absoluteUrl(req, `/pay/sandbox/${reference}
56   `) };
57
58   if (provider === "stripe") {
59     const body = new URLSearchParams({
60       mode: "payment",
61       success_url: successUrl,
62       cancel_url: `${base}/wallet?payment=${reference}&cancelled=1`,
63       client_reference_id: reference,
64       "metadata[reference]": reference,
65       "line_items[0][quantity]": "1",
66       "line_items[0][price_data][currency]": "pkr",
67       "line_items[0][price_data][unit_amount]": String(Math.round(amount * 100)),
68       "line_items[0][price_data][product_data][name]": "MarketLink wallet top-up",
69       customer_email: user.email,
70     });
71     const res = await fetch("https://api.stripe.com/v1/checkout/sessions", {
72       method: "POST",
73       headers: { Authorization: `Bearer ${process.env.STRIPE_SECRET_KEY}`, "Content-Type": "application/x-
74       www-form-urlencoded" },
75       body,
76     });
77     const data = (await res.json()) as { id?: string; url?: string; error?: { message: string } };
78     if (!res.ok || !data.url) throw new ApiError(502, `Stripe error: ${data.error?.message ?? res.status}`
79     );
80     if (!isValidHttpUrl(data.url)) throw new ApiError(502, "Stripe returned an invalid checkout URL");
81     await db.update(paymentIntents).set({ providerRef: data.id }).where(eq(paymentIntents.reference, refer
82     ence));
83     return { reference, redirectUrl: data.url };
84   }
85
86   if (provider === "jazzcash") {
87     const now = new Date(Date.now() + 5 * 3600_000); // PKT
88     const fmt = (d: Date) => d.toISOString().replace(/[-:]/g, "").slice(0, 14);
89     const fields: Record<string, string> = {
90       pp_Version: "1.1",
91       pp_TxnType: "",
92       pp_Language: "EN",
93       pp_MerchantID: process.env.JAZZCASH_MERCHANT_ID!,
94       pp_SubMerchantID: "",
95       pp_Password: process.env.JAZZCASH_PASSWORD!,
96       pp_BankID: "",
97       pp_ProductID: "",
98       pp_TxnRefNo: reference,
99       pp_Amount: String(Math.round(amount * 100)),
100       pp_TxnCurrency: "PKR",
101       pp_TxnDateTime: fmt(now),
102       pp_BillReference: "walletTopup",
103       pp_Description: "MarketLink wallet top-up",
104       pp_TxnExpiryDateTime: fmt(new Date(now.getTime() + 60 * 60_000)),
105       pp_ReturnURL: `${base}/api/payments/callback/jazzcash`,
106       ppmpf_1: String(user.id),
107     };
108     fields.pp_SecureHash = jazzcashHash(fields);
109     return { reference, form: { action: live() ? "https://payments.jazzcash.com.pk/CustomerPortal/transact
110     ionmanagement/merchantform/" : "https://sandbox.jazzcash.com.pk/CustomerPortal/transactionmanagement/merch
111     antform/", fields } };
112   }
113
114   // Easypaisa hosted checkout
115   const expiry = new Date(Date.now() + 5 * 3600_000 + 60 * 60_000).toISOString().replace(/[-:]/g, "").repl
116   ace("T", " ").slice(0, 15);

```

```

107     const params: Record<string, string> = {
108         amount: amount.toFixed(1),
109         autoRedirect: "1",
110         emailAddr: user.email,
111         expiryDate: expiry,
112         mobileNum: user.phone.replace(/\D/g, "").replace(/^92/, "0"),
113         orderRefNum: reference,
114         paymentMethod: "MA_PAYMENT_METHOD",
115         postBackURL: `${base}/api/payments/callback/easypaisa`,
116         storeId: process.env.EASYPAlSA_STORE_ID!,
117     };
118     const plain = Object.keys(params).sort().map((k) => `${k}=${params[k]}`).join("&");
119     const cipher = createCipheriv("aes-128-ecb", Buffer.from(process.env.EASYPAlSA_HASH_KEY!, "utf8").subarr
120     ay(0, 16), null);
121     params.merchantHashedReq = Buffer.concat([cipher.update(plain, "utf8"), cipher.final()]).toString("base6
122     4");
123     return { reference, form: { action: live() ? "https://easypay.easypaisa.com.pk/easypay/Index.jsf" : "htt
124     ps://easypaystg.easypaisa.com.pk/easypay/Index.jsf", fields: params } };
125 }
126
127 /** JazzCash pp_SecureHash: HMAC-SHA256(salt, salt&v1&v2... of non-empty pp_* fields sorted by key). */
128 export function jazzcashHash(fields: Record<string, string>) {
129     const salt = process.env.JAZZCASH_INTEGRITY_SALT!;
130     const values = Object.keys(fields).filter((k) => k.startsWith("pp") && k !== "pp_SecureHash" && fields[k]
131     !== "").sort().map((k) => fields[k]);
132     return createHmac("sha256", salt).update([salt, ...values].join("&")).digest("hex").toUpperCase();
133 }
134
135 /** Stripe-Signature verification: t=<ts>,v1=<hmac(secret, `${t}.${rawBody}`)>, 5-minute tolerance. */
136 export function verifyStripeSignature(rawBody: string, header: string | null) {
137     const secret = process.env.STRIPE_WEBHOOK_SECRET;
138     if (!secret || !header) return false;
139     const parts = Object.fromEntries(header.split(",").map((p) => p.split("=") as [string, string]));
140     const t = Number(parts.t);
141     if (!t || Math.abs(Date.now() / 1000 - t) > 300) return false;
142     const expected = createHmac("sha256", secret).update(`${t}.${rawBody}`).digest("hex");
143     const sigs = header.split(",").filter((p) => p.startsWith("v1=")).map((p) => p.slice(3));
144     return sigs.some((s) => s.length === expected.length && timingSafeEqual(Buffer.from(s), Buffer.from(expe
145     cted)));
146 }
147
148 /** Sandbox callbacks are HMAC-signed server-side so they can't be forged from the browser. */
149 export function sandboxSignature(reference: string, outcome: string) {
150     return createHmac("sha256", process.env.SESSION_SECRET || "marketlink-agri-hub-dev-secret-change-me").up
151     date(`sandbox:${reference}:${outcome}`).digest("hex");
152 }
153
154 /** Step 2 (webhook / callback): idempotently settle the intent and credit the wallet on success. */
155 export async function completeIntent(reference: string, success: boolean, providerRef?: string, raw?: unkn
156     own) {
157     const result = await db.transaction(async (tx) => {
158         const [pi] = await tx.select().from(paymentIntents).where(eq(paymentIntents.reference, reference)).for
159         ("update").limit(1);
160         if (!pi) throw new ApiError(404, "Unknown payment reference");
161         if (pi.status !== "pending") return { intent: pi, changed: false };
162         await tx
163             .update(paymentIntents)
164             .set({ status: success ? "succeeded" : "failed", providerRef: providerRef ?? pi.providerRef, rawCall
165             back: raw ? JSON.stringify(raw).slice(0, 4000) : null, completedAt: new Date() })
166             .where(eq(paymentIntents.intentId, pi.intentId));
167         if (success) {
168             await tx.update(users).set({ walletBalance: sql`${users.walletBalance} + ${pi.amount}` }).where(eq(u
169             sers.id, pi.userId));
170             await tx.insert(walletTransactions).values({ userId: pi.userId, type: "deposit", amount: pi.amount,
171             description: `Top-up via ${pi.provider} · ref ${reference}` });
172         }
173         return { intent: { ...pi, status: success ? "succeeded" : "failed" }, changed: true };
174     });
175     if (result.changed) {
176         const pi = result.intent;
177         await notify(pi.userId, success
178             ? { type: "payment", title: "Wallet funded", body: `Rs. ${pi.amount.toLocaleString("en-US")} was add
179             ed to your MarketLink wallet (ref ${reference}).`, link: "/wallet", email: true }
180             : { type: "payment", title: "Payment failed", body: `Your top-up of Rs. ${pi.amount.toLocaleString("
181             en-US")} did not go through (ref ${reference}). No money was deducted.`, link: "/wallet" });
182     }
183 }

```

```

170     return result.intent;
171 }
172
173 export async function getIntent(user: SessionUser, reference: string) {
174     const [pi] = await db.select().from(paymentIntents).where(eq(paymentIntents.reference, reference)).limit
175     (1);
176     if (!pi || (pi.userId !== user.id && user.role !== "admin")) throw new ApiError(404, "Payment not found"
177     );
178     return { reference: pi.reference, provider: pi.provider, amount: pi.amount, status: pi.status, createdAt
179     : pi.createdAt, completedAt: pi.completedAt };
180 }
181
182 /** Easypaisa: confirm the final status server-to-server before crediting (callback params alone are not t
183 rusted). */
184 export async function easypaisaInquire(reference: string): Promise<boolean> {
185     const { EASYPAlSA_USERNAME: u, EASYPAlSA_PASSWORD: p, EASYPAlSA_ACCOUNT_NUM: acc, EASYPAlSA_STORE_ID: st
186 ore } = process.env;
187     if (!u || !p || !acc) return false;
188     const url = live() ? "https://easypay.easypaisa.com.pk/easypay-service/rest/v4/inquire-transaction" : "h
189 ttps://easypaystg.easypaisa.com.pk/easypay-service/rest/v4/inquire-transaction";
190     const res = await fetch(url, { method: "POST", headers: { "Content-Type": "application/json", Credential
191 s: Buffer.from(`${u}:${p}`).toString("base64") }, body: JSON.stringify({ orderId: reference, storeId: stor
192 e, accountNum: acc }) });
193     const data = (await res.json()).catch(() => ({})) as { transactionStatus?: string };
194     return data.transactionStatus === "PAID";
195 }
196
197 /* ----- */
198 /* Farmer payouts */
199 /* ----- */
200 export async function requestPayout(farmer: SessionUser, input: { amount: number; method: "bank" | "jazzca
201 sh" | "easypaisa"; accountTitle: string; accountNumber: string }) {
202     if (input.amount < 1000) throw new ApiError(422, "Minimum payout is Rs. 1,000");
203     const row = await db.transaction(async (tx) => {
204         const [u] = await tx.select().from(users).where(eq(users.id, farmer.id)).for("update");
205         if (u.walletBalance < input.amount) throw new ApiError(402, `Insufficient balance. Available: Rs. ${u.
206 walletBalance.toLocaleString("en-US")}`);
207         const [p] = await tx.insert(payoutRequests).values({ farmerId: farmer.id, ...input }).returning();
208         await tx.update(users).set({ walletBalance: r2(u.walletBalance - input.amount) }).where(eq(users.id, u
209 .id));
210         await tx.insert(walletTransactions).values({ userId: u.id, type: "withdrawal", amount: input.amount, d
211 escription: `Payout request #${p.payoutId} to ${input.method.toUpperCase()} ${input.accountNumber.slice(-4
212 ).padStart(8, "•")}` });
213     });
214     return p;
215 }
216
217 const admins = await db.select({ id: users.id }).from(users).where(eq(users.role, "admin"));
218 await Promise.all(admins.map((a) => notify(a.id, { type: "payout", title: "New payout request", body: `
219 {farmer.fullName} requested Rs. ${input.amount.toLocaleString("en-US")} via ${input.method}.`, link: "/adm
220 in/dashboard" })));
221 return row;
222 }
223
224 export async function listPayouts(user: SessionUser) {
225     const q = db
226         .select({ p: payoutRequests, farmerName: users.fullName, farmName: users.businessName })
227         .from(payoutRequests)
228         .innerJoin(users, eq(users.id, payoutRequests.farmerId))
229         .orderBy(sql`case when ${payoutRequests.status} = 'pending' then 0 else 1 end`, desc(payoutRequests.cr
230 eatedAt))
231         .limit(100);
232     const rows = user.role === "admin" ? await q : await q.where(eq(payoutRequests.farmerId, user.id));
233     return rows.map(({ p, ...rest }) => ({ ...p, ...rest }));
234 }
235
236 export async function processPayout(admin: SessionUser, payoutId: number, status: "paid" | "rejected", not
237 e?: string) {
238     const p = await db.transaction(async (tx) => {
239         const [row] = await tx.select().from(payoutRequests).where(eq(payoutRequests.payoutId, payoutId)).for(
240 "update");
241         if (!row) throw new ApiError(404, "Payout not found");
242         if (row.status !== "pending") throw new ApiError(409, `Payout already ${row.status}`);
243         await tx.update(payoutRequests).set({ status, adminNote: note ?? null, processedAt: new Date() }).wher
244 e(eq(payoutRequests.payoutId, payoutId));
245         if (status === "rejected") {
246             await tx.update(users).set({ walletBalance: sql`${users.walletBalance} + ${row.amount}` }).where(eq(
247 users.id, row.farmerId));

```

```

226     await tx.insert(walletTransactions).values({ userId: row.farmerId, type: "refund", amount: row.amount,
227     ↵      description: `Payout #${payoutId} rejected, funds returned` });
228     return row;
229   });
230   await notify(p.farmerId, status === "paid"
231     ↵   ? { type: "payout", title: "Payout sent", body: `Rs. ${p.amount.toLocaleString("en-US")} was transferred to your ${p.method} account ${p.accountNumber}.`, link: "/wallet", email: true, sms: true }
232     : { type: "payout", title: "Payout rejected", body: `Your payout of Rs. ${p.amount.toLocaleString("en-US")} was rejected${note ? `: ${note}` : ""}. The amount is back in your wallet.`, link: "/wallet", email:
233     ↵      true });
234   return { payoutId, status, processedBy: admin.id };
235 }

```

src/lib/services/reviews.ts

```

1  import { and, desc, eq, sql } from "drizzle-orm";
2  import { db } from "@db";
3  import { ordersLogistics, reviews, users, cropsInventory } from "@db/schema";
4  import { ApiError } from "@lib/api";
5  import type { SessionUser } from "@lib/auth";
6  import { notify } from "@lib/notify";
7
8  /**
9   * Seller trust score (1.00–5.00):
10   *   Bayesian average of verified reviews (prior 4.0 weighted as 3 reviews)
11   *   + up to 0.20 for completed deliveries, + 0.10 if identity-verified
12   *   - 1.5 × dispute rate.  Recomputed on every new review / delivery / dispute resolution.
13   */
14  export async function recomputeTrust(farmerId: number) {
15     const res = await db.execute(sql`
16       select
17         (select count(*) from reviews r where r.farmer_id = ${farmerId})::int as n,
18         (select coalesce(sum(rating),0) from reviews r where r.farmer_id = ${farmerId})::int as total,
19         (select count(*) from orders_logistics o where o.farmer_id = ${farmerId} and o.payment_status = 'released')::int as delivered,
20         (select count(*) from disputes d join orders_logistics o on o.order_id = d.order_id where o.farmer_id = ${farmerId})::int as disputes,
21         (select is_verified from users where id = ${farmerId}) as verified`;
22     const r = res.rows[0] as { n: number; total: number; delivered: number; disputes: number; verified: boolean };
23     const n = Number(r.n), total = Number(r.total), delivered = Number(r.delivered), disputes = Number(r.disputes);
24     const avg = n ? total / n : 0;
25     const bayes = (3 * 4.0 + total) / (3 + n);
26     const score = Math.max(1, Math.min(5, bayes + Math.min(0.2, delivered * 0.02) + (r.verified ? 0.1 : 0) - (delivered ? (disputes / delivered) * 1.5 : 0)));
27     await db.update(users).set({ trustScore: Math.round(score * 100) / 100, ratingAvg: Math.round(avg * 100) / 100, ratingCount: n }).where(eq(users.id, farmerId));
28     return { trustScore: score, ratingAvg: avg, ratingCount: n };
29   }
30
31  export async function recomputeAllTrust() {
32     const farmers = await db.select({ id: users.id }).from(users).where(eq(users.role, "farmer"));
33     for (const f of farmers) await recomputeTrust(f.id);
34   }
35
36  export async function createReview(buyer: SessionUser, orderId: number, rating: number, comment?: string) {
37     ↵   if (!Number.isInteger(rating) || rating < 1 || rating > 5) throw new ApiError(422, "Rating must be 1 to 5 stars");
38     const [o] = await db.select().from(ordersLogistics).where(eq(ordersLogistics.orderId, orderId)).limit(1);
39     ↵   if (!o) throw new ApiError(404, "Order not found");
40     if (o.buyerId !== buyer.id) throw new ApiError(403, "Only the buyer of this order can review it");
41     if (o.deliveryStage !== "delivered" || o.paymentStatus !== "released") throw new ApiError(409, "You can review a seller after the order is delivered and paid");
42     const [existing] = await db.select({ id: reviews.reviewId }).from(reviews).where(eq(reviews.orderId, orderId)).limit(1);
43     ↵   if (existing) throw new ApiError(409, "You already reviewed this order");
44     const [row] = await db.insert(reviews).values({ orderId, buyerId: buyer.id, farmerId: o.farmerId, rating, comment: comment?.trim() || null }).returning();
45     const trust = await recomputeTrust(o.farmerId);
46     await notify(o.farmerId, { type: "review", title: `New ${rating}★ review`, body: `${buyer.businessName ?

```

```

46   ? buyer.fullName} rated order ${o.trackingNumber}${comment ? `: "${comment.slice(0, 120)}"`` : "."}` , link:
47   "/farmer/dashboard" });
48   return { review: row, trust };
49 }
50 export async function listReviews(farmerId: number, limit = 20) {
51   return db
52     .select({
53       reviewId: reviews.reviewId,
54       orderId: reviews.orderId,
55       rating: reviews.rating,
56       comment: reviews.comment,
57       farmerReply: reviews.farmerReply,
58       repliedAt: reviews.repliedAt,
59       createdAt: reviews.createdAt,
60       buyerName: users.businessName,
61       buyerFullName: users.fullName,
62       cropId: cropsInventory.cropId,
63       cropName: cropsInventory.cropName,
64     })
65     .from(reviews)
66     .innerJoin(users, eq(users.id, reviews.buyerId))
67     .innerJoin(ordersLogistics, eq(ordersLogistics.orderId, reviews.orderId))
68     .innerJoin(cropsInventory, eq(cropsInventory.cropId, ordersLogistics.cropId))
69     .where(eq(reviews.farmerId, farmerId))
70     .orderBy(desc(reviews.createdAt))
71     .limit(limit);
72 }
73
74 /** Delivered orders of this buyer that don't have a review yet. */
75 export async function pendingReviews(buyerId: number) {
76   const rows = await db.execute(sql`
77     select o.order_id, o.tracking_number, c.crop_name, coalesce(u.business_name, u.full_name) as farmer, o
78     .updated_at
79     from orders_logistics o
80     join crops_inventory c on c.crop_id = o.crop_id
81     join users u on u.id = o.farmer_id
82     where o.buyer_id = ${buyerId} and o.delivery_stage = 'delivered' and o.payment_status = 'released'
83     and not exists (select 1 from reviews r where r.order_id = o.order_id)
84     order by o.updated_at desc limit 10`);
85   return (rows.rows as { order_id: number; tracking_number: string; crop_name: string; farmer: string; upd
86     ated_at: string }[]).map((r) => ({
87     orderId: Number(r.order_id), trackingNumber: r.tracking_number, cropName: r.crop_name, farmer: r.farme
88     r, deliveredAt: r.updated_at,
89   }));
90 }
91
92 export async function respondToReview(farmer: SessionUser, reviewId: number, message: string) {
93   const reply = message.trim();
94   if (reply.length < 2 || reply.length > 600) throw new ApiError(422, "Reply must be between 2 and 600 cha
95   racters");
96   const saved = await db.transaction(async (tx) => {
97     const [review] = await tx.select().from(reviews).where(eq(reviews.reviewId, reviewId)).for("update").l
98     imit(1);
99     if (!review) throw new ApiError(404, "Review not found");
100     if (review.farmerId !== farmer.id) throw new ApiError(403, "You may respond only to reviews of your ow
101     n sales");
102     if (review.farmerReply !== null) throw new ApiError(409, "You have already responded to this review");
103     const [updated] = await tx
104       .update(reviews)
105       .set({ farmerReply: reply, repliedAt: new Date() })
106       .where(eq(reviews.reviewId, reviewId))
107       .returning();
108     return updated;
109   });
110   const [order] = await db.select({ cropId: ordersLogistics.cropId }).from(ordersLogistics).where(eq(order
111   logistics.orderId, saved.orderId)).limit(1);
112   await notify(saved.buyerId, {
113     type: "review_reply",
114     title: "Farmer replied to your review",
115     body: `${farmer.businessName ?? farmer.fullName}: "${reply.slice(0, 180)}"`,
116     link: order ? `/crop/${order.cropId}` : "/buyer/dashboard",
117   });
118   return saved;

```

```

114 }
115
116 export async function hasReviewed(orderIds: number[]) {
117   if (!orderIds.length) return new Set<number>();
118   const rows = await db.select({ orderId: reviews.orderId }).from(reviews).where(and(sql`${reviews.orderId`
119   } in (${sql.join(orderIds.map((i) => sql`${i}`), sql`, `)}))`));
120 }
121

```

src/lib/services/tracking.ts

```

1  import { asc, eq, and } from "drizzle-orm";
2  import { db } from "@db";
3  import { ordersLogistics, orderEvents, cropsInventory, users } from "@db/schema";
4  import { alias } from "drizzle-orm/pg-core";
5  import { ApiError } from "@lib/api";
6  import { CITIES, DELIVERY_STAGES, distanceKm, type DeliveryStage } from "@lib/constants";
7  import { notify } from "@lib/notify";
8
9  type Pt = { lat: number; lng: number };
10 const AVG_SPEED_KMH = 45; // loaded truck on Pakistani highways incl. stops
11
12 /** Resolve "Multan", "Multan, Punjab" or "Khanewal Bypass, N-5" to coordinates when possible. */
13 export function geocode(label: string | null | undefined): Pt | null {
14   if (!label) return null;
15   const hit = Object.keys(CITIES).find((c) => label.toLowerCase().includes(c.toLowerCase()));
16   return hit ? { lat: CITIES[hit].lat, lng: CITIES[hit].lng } : null;
17 }
18
19 /** Gently curved road-like path (quadratic Bézier) between two points. */
20 export function routePath(a: Pt, b: Pt, steps = 40): Pt[] {
21   const mid = { lat: (a.lat + b.lat) / 2, lng: (a.lng + b.lng) / 2 };
22   const dx = b.lng - a.lng, dy = b.lat - a.lat;
23   const ctrl = { lat: mid.lat - dx * 0.12, lng: mid.lng + dy * 0.12 };
24   return Array.from({ length: steps + 1 }, (_, i) => {
25     const t = i / steps;
26     return { lat: (1 - t) ** 2 * a.lat + 2 * (1 - t) * t * ctrl.lat + t ** 2 * b.lat, lng: (1 - t) ** 2 *
27     a.lng + 2 * (1 - t) * t * ctrl.lng + t ** 2 * b.lng };
28   });
29 }
30
31 const pointAt = (path: Pt[], t: number) => path[Math.round(Math.max(0, Math.min(1, t)) * (path.length - 1))];
32
33 const EVENT_LABEL: Record<string, string> = {
34   confirmed: "Order placed", escrow_locked: "Payment secured in escrow", quality_checked: "Quality inspect
35   ed", dispatched: "Dispatched from farm",
36   in_transit: "In transit", location_update: "Checkpoint", gps_ping: "GPS update", arriving: "Arriving soo
37   n", delivered: "Delivered",
38   escrow_released: "Payment released to farmer", escrow_refunded: "Buyer refunded", dispute_opened: "Dispu
39   te opened",
40 };
41
42 /**
43  * Shipment snapshot for the tracking page.
44  * Position source: latest driver GPS ping if present, otherwise a time-based simulation along the route
45  * (progress = time since "in transit" / expected travel time at 45 km/h).
46  * Crossing 85% of the route fires a one-time "Arriving soon" milestone + buyer notification.
47  */
48 export async function trackShipment(trackingNumber: string, viewer?: { id: number; role: string } | null) {
49   {
50     const farmer = alias(users, "farmer");
51     const buyer = alias(users, "buyer");
52     const [row] = await db
53       .select({ o: ordersLogistics, cropName: cropsInventory.cropName, farmLocation: cropsInventory.farmLoca
54       tion, buyerCity: buyer.city, farmerName: farmer.businessName, buyerName: buyer.businessName })
55       .from(ordersLogistics)
56       .innerJoin(cropsInventory, eq(cropsInventory.cropId, ordersLogistics.cropId))
57       .innerJoin(farmer, eq(farmer.id, ordersLogistics.farmerId))
58       .innerJoin(buyer, eq(buyer.id, ordersLogistics.buyerId))
59       .where(eq(ordersLogistics.trackingNumber, trackingNumber.trim().toUpperCase()))
60       .limit(1);
61     if (!row) throw new ApiError(404, "No shipment found with this tracking number");
62     const o = row.o;

```

```

57     const events = await db.select().from(orderEvents).where(eq(orderEvents.orderId, o.orderId)).orderBy(asc
    (orderEvents.createdAt));
58
59     const origin = geocode(row.farmLocation) ?? { lat: 30.37, lng: 69.34 };
60     const dest = geocode(row.buyerCity) ?? origin;
61     const path = routePath(origin, dest);
62     const km = Math.round(distanceKm(origin, dest) * 1.25); // road factor
63     const travelMs = Math.max(2, km / AVG_SPEED_KMH) * 3600_000;
64     const stage = o.deliveryStage as DeliveryStage;
65     const transitEvt = [...events].reverse().find((e) => e.stage === "in_transit");
66     const lastPing = [...events].reverse().find((e) => e.stage === "gps_ping" && e.lat !== null && e.lng !==
    null);
67
68     let progress = 0;
69     let source: "gps" | "simulated" | "fixed" = "fixed";
70     let current: Pt = origin;
71     if (stage === "delivered") { progress = 1; current = dest; }
72     else if (stage === "in_transit" && o.paymentStatus !== "disputed") {
73         if (lastPing && (!transitEvt || lastPing.createdAt >= transitEvt.createdAt)) {
74             current = { lat: lastPing.lat!, lng: lastPing.lng! };
75             const total = distanceKm(origin, dest) || 1;
76             progress = Math.max(0, Math.min(0.99, 1 - distanceKm(current, dest) / total));
77             source = "gps";
78         } else {
79             const since = transitEvt ? Date.now() - new Date(transitEvt.createdAt).getTime() : 0;
80             progress = Math.min(0.97, since / travelMs);
81             current = pointAt(path, progress);
82             source = "simulated";
83         }
84     }
85     const eta = stage === "delivered" ? null : stage === "in_transit" ? new Date(Date.now() + (1 - progress)
    * travelMs) : o.expectedDeliveryDate ? new Date(o.expectedDeliveryDate) : null;
86
87     // Automated milestone: arriving soon
88     if (stage === "in_transit" && progress >= 0.85 && !events.some((e) => e.stage === "arriving")) {
89         const [ev] = await db.insert(orderEvents).values({ orderId: o.orderId, stage: "arriving", location: `~
    ${Math.max(1, Math.round((1 - progress) * km))} km from ${row.buyerCity}`, lat: current.lat, lng: current.
    lng, note: "Shipment is close to the destination" }).returning();
90         events.push(ev);
91         await notify(o.buyerId, { type: "shipment", title: "Your shipment is arriving soon", body: `${row.crop
    Name} (${o.trackingNumber}) is about ${Math.round((1 - progress) * km)} km from ${row.buyerCity}.`, link:
    `/tracking?no=${o.trackingNumber}`, sms: true });
92     }
93     if (source !== "fixed") await db.update(ordersLogistics).set({ currentLat: current.lat, currentLng: curr
    ent.lng }).where(eq(ordersLogistics.orderId, o.orderId));
94
95     const isParty = !!viewer && (viewer.id === o.farmerId || viewer.id === o.buyerId || viewer.role === "adm
    in" || viewer.role === "inspector");
96     return {
97         trackingNumber: o.trackingNumber,
98         cropName: row.cropName,
99         quantityKg: o.quantityKg,
100        stage,
101        stageLabel: DELIVERY_STAGES.find((s) => s.key === stage)?.label ?? stage,
102        payment: o.paymentStatus === "escrow_locked" ? "Secured in escrow" : o.paymentStatus === "released" ?
    "Paid" : o.paymentStatus === "disputed" ? "On hold (dispute)" : o.paymentStatus === "refunded" ? "Refunded
    " : "Awaiting escrow",
103        disputed: o.paymentStatus === "disputed",
104        origin: { city: row.farmLocation, ...origin },
105        destination: { city: row.buyerCity, ...dest },
106        current: { ...current, progress: Math.round(progress * 1000) / 10, source },
107        route: path,
108        distanceKm: km,
109        eta: eta?.toISOString() ?? null,
110        lastUpdate: events.at(-1)?.createdAt ?? o.updatedAt,
111        events: events.map((e) => ({ id: e.eventId, stage: e.stage, label: EVENT_LABEL[e.stage] ?? e.stage.rep
    lace(/_/g, " "), location: e.location, lat: e.lat, lng: e.lng, note: isParty ? e.note : null, at: e.create
    dAt })),
112        parties: isParty ? { farmer: row.farmerName, buyer: row.buyerName } : undefined,
113    };
114 }
115
116 /** Driver/farmer GPS ping while the truck is moving. */
117 export async function recordGpsPing(user: { id: number; role: string }, orderId: number, lat: number, lng:
    number, label?: string) {
118     if (lat < 23 || lat > 37.5 || lng < 60 || lng > 78) throw new ApiError(422, "Coordinates must be inside

```

```

119     const [o] = await db.select().from(ordersLogistics).where(eq(ordersLogistics.orderId, orderId)).limit(1)
120     ;
121     if (!o) throw new ApiError(404, "Order not found");
122     if (!(user.role === "admin" || user.id === o.farmerId)) throw new ApiError(403, "Only the shipping farmer or an admin can send GPS updates");
123     if (!["dispatched", "in_transit"].includes(o.deliveryStage)) throw new ApiError(409, "GPS updates are accepted only while the shipment is moving");
124     await db.update(ordersLogistics).set({ currentLat: lat, currentLng: lng, transitLocation: label ?? o.transitLocation, updatedAt: new Date() }).where(eq(ordersLogistics.orderId, orderId));
125     const [ev] = await db.insert(orderEvents).values({ orderId, stage: "gps_ping", lat, lng, location: label ?? `${lat.toFixed(4)}, ${lng.toFixed(4)}`, actorId: user.id }).returning();
126     return ev;
127 }
128 export async function orderIdByTracking(trackingNumber: string) {
129     const [o] = await db.select({ id: ordersLogistics.orderId }).from(ordersLogistics).where(and(eq(ordersLogistics.trackingNumber, trackingNumber))).limit(1);
130     return o?.id ?? null;
131 }
132

```

src/lib/ticker.ts

```

1 import { latestRates } from "@lib/services/mandi";
2 import { cropUrdu } from "@lib/constants";
3 import type { TickerItem } from "@components/mandi-ticker";
4
5 /** Per-crop national average of the latest mandi rates with average day change. */
6 export async function getTicker(): Promise<TickerItem[]> {
7     try {
8         const rows = await latestRates();
9         const acc = new Map<string, { s: number; n: number; c: number; cn: number }>();
10        for (const r of rows) {
11            const a = acc.get(r.cropName) ?? { s: 0, n: 0, c: 0, cn: 0 };
12            a.s += r.avgPricePerKg;
13            a.n += 1;
14            if (r.changePct !== null) {
15                a.c += r.changePct;
16                a.cn += 1;
17            }
18            acc.set(r.cropName, a);
19        }
20        return [...acc.entries()].map(([crop, a]) => ({ crop, urdu: cropUrdu(crop), avg: a.s / a.n, change: a.cn ? a.c / a.cn : null }));
21    } catch {
22        return [];
23    }
24 }
25

```

src/lib/viewer.ts

```

1 import { and, eq, sql } from "drizzle-orm";
2 import { db } from "@db";
3 import { notifications } from "@db/schema";
4 import { getCurrentUser, homeFor, type SessionUser } from "@lib/auth";
5 import type { NavViewer } from "@components/nav/nav-actions";
6
7 /** Data for the auth-aware navigation (wallet balance, unread notifications, dashboard link). */
8 export async function getViewer(existing?: SessionUser | null): Promise<NavViewer> {
9     const u = existing === undefined ? await getCurrentUser() : existing;
10    if (!u) return null;
11    const [n] = await db
12        .select({ n: sql<number>`count(*)`.mapWith(Number) })
13        .from(notifications)
14        .where(and(eq(notifications.userId, u.id), eq(notifications.isRead, false)));
15    return { fullName: u.fullName, role: u.role, walletBalance: u.walletBalance, escrowBalance: u.escrowBalance, unread: n, dashboard: homeFor(u.role), title: u.title, avatarUrl: u.avatarUrl };
16 }
17

```

src/proxy.ts

```
1  import { NextResponse, type NextRequest } from "next/server";
2  import { createHmac, timingSafeEqual } from "crypto";
3
4  /**
5   * Edge-of-app RBAC guard (Next.js 16 Proxy, Node.js runtime).
6   * Verifies the signed session cookie and enforces role isolation before any page renders.
7   * Pages and API handlers re-check the role against the database (defence in depth).
8   */
9  const SECRET = process.env.SESSION_SECRET || "marketlink-agri-hub-dev-secret-change-me";
10 const MAX_AGE_MS = 60 * 60 * 24 * 7 * 1000;
11
12 type Role = "farmer" | "buyer" | "admin" | "inspector";
13 const RULES: { prefix: string; roles: Role[] }[] = [
14   { prefix: "/admin", roles: ["admin"] },
15   { prefix: "/api/db", roles: ["admin"] },
16   { prefix: "/api/admin", roles: ["admin"] },
17   { prefix: "/inspector", roles: ["inspector", "admin"] },
18   { prefix: "/farmer", roles: ["farmer"] },
19   { prefix: "/buyer", roles: ["buyer"] },
20   { prefix: "/account", roles: ["farmer", "buyer", "admin", "inspector"] },
21   { prefix: "/wallet", roles: ["farmer", "buyer"] },
22   { prefix: "/notifications", roles: ["farmer", "buyer", "admin", "inspector"] },
23 ];
24 const HOME: Record<Role, string> = { farmer: "/farmer/dashboard", buyer: "/buyer/dashboard", admin: "/admin/dashboard", inspector: "/inspector/dashboard" };
25
26 function readRole(token: string | undefined): Role | null {
27   if (!token) return null;
28   const parts = token.split(".");
29   if (parts.length !== 4) return null;
30   const [id, role, ts, sig] = parts;
31   const expected = createHmac("sha256", SECRET).update(`${id}.${role}.${ts}`).digest("hex");
32   if (expected.length !== sig.length || !timingSafeEqual(Buffer.from(expected), Buffer.from(sig))) return null;
33   if (Date.now() - Number(ts) > MAX_AGE_MS) return null;
34   return role in HOME ? (role as Role) : null;
35 }
36
37 export function proxy(req: NextRequest) {
38   const { pathname, search } = req.nextUrl;
39   const rule = RULES.find((r) => pathname === r.prefix || pathname.startsWith(r.prefix + "/"));
40   if (!rule) return NextResponse.next();
41
42   const role = readRole(req.cookies.get("ml_session")?.value);
43   const isApi = pathname.startsWith("/api/");
44   const adminArea = rule.roles.length === 1 && rule.roles[0] === "admin";
45
46   // Admin area: guests AND signed-in non-admins → /login with "Access Denied" alert (API → 403 JSON)
47   if (adminArea && role !== "admin") {
48     if (isApi) return NextResponse.json({ success: false, error: "Access Denied: Admin privileges required" }, { status: 403 });
49     const url = req.nextUrl.clone();
50     url.pathname = "/login";
51     url.search = `?error=admin_required&next=${encodeURIComponent(pathname + search)}`;
52     return NextResponse.redirect(url);
53   }
54
55   if (!role) {
56     if (isApi) return NextResponse.json({ success: false, error: "Authentication required" }, { status: 401 });
57     const url = req.nextUrl.clone();
58     url.pathname = "/login";
59     url.search = `?next=${encodeURIComponent(pathname + search)}`;
60     return NextResponse.redirect(url);
61   }
62   if (!rule.roles.includes(role)) {
63     if (isApi) return NextResponse.json({ success: false, error: `Requires role: ${rule.roles.join(" or ")}` }, { status: 403 });
64     const url = req.nextUrl.clone();
65     url.pathname = HOME[role];
66     url.search = "";
67     return NextResponse.redirect(url);
68   }
```

```

69     return NextResponse.next();
70 }
71
72 export const config = {
73   matcher: ["/admin", "/admin/:path*", "/api/db/:path*", "/api/admin/:path*", "/inspector/:path*", "/farmer/:path*", "/buyer/:path*", "/account/:path*", "/wallet/:path*", "/notifications/:path*"],
74 };
75

```

database/mysql_schema.sql

```

1  -- MarketLink Agri-Hub · MySQL 8.0 compatible DDL (InnoDB, utf8mb4)
2  -- Production runtime uses PostgreSQL (see PostgreSQL tab); this script is the portable equivalent.
3
4  CREATE TABLE users (
5    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
6    full_name VARCHAR(120) NOT NULL,
7    email VARCHAR(160) NOT NULL,
8    password_hash VARCHAR(255) NOT NULL,
9    phone VARCHAR(30) NOT NULL,
10   role ENUM('farmer','buyer','admin','inspector') NOT NULL,
11   cnic_id VARCHAR(20) NULL,
12   business_name VARCHAR(160) NULL,
13   city VARCHAR(80) NOT NULL,
14   address TEXT NULL,
15   is_verified BOOLEAN NOT NULL DEFAULT FALSE,
16   email_verified BOOLEAN NOT NULL DEFAULT FALSE,
17   title VARCHAR(120) NULL, -- e.g. 'Super Admin & Systems Controller'
18   user_code VARCHAR(40) NULL, -- e.g. 'admin_01'
19   avatar_url TEXT NULL,
20   google_id VARCHAR(64) NULL, -- Google OAuth subject
21   auth_provider VARCHAR(20) NOT NULL DEFAULT 'password',
22   trust_score DECIMAL(3,2) NOT NULL DEFAULT 0,
23   rating_avg DECIMAL(3,2) NOT NULL DEFAULT 0,
24   rating_count INT NOT NULL DEFAULT 0,
25   wallet_balance DECIMAL(14,2) NOT NULL DEFAULT 0,
26   escrow_balance DECIMAL(14,2) NOT NULL DEFAULT 0,
27   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
28   UNIQUE KEY users_email_uq (email),
29   UNIQUE KEY users_google_id_uq (google_id),
30   UNIQUE KEY users_user_code_uq (user_code),
31   KEY users_role_idx (role),
32   KEY users_city_idx (city)
33 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
34
35 CREATE TABLE crops_inventory (
36   crop_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
37   farmer_id INT UNSIGNED NOT NULL,
38   crop_name VARCHAR(80) NOT NULL,
39   category ENUM('grains','vegetables','fruits','cash_crops') NOT NULL,
40   quality_grade ENUM('A','B','C') NOT NULL,
41   total_quantity_kg DECIMAL(14,2) NOT NULL CHECK (total_quantity_kg >= 0),
42   base_price_per_kg DECIMAL(14,2) NOT NULL CHECK (base_price_per_kg > 0),
43   harvest_date DATE NOT NULL,
44   images_json JSON NOT NULL,
45   description TEXT NULL,
46   farm_location VARCHAR(80) NOT NULL,
47   latitude DOUBLE NULL,
48   longitude DOUBLE NULL,
49   status ENUM('active','pending_inspection','sold_out','archived') NOT NULL DEFAULT 'active',
50   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
51   KEY crops_farmer_idx (farmer_id),
52   KEY crops_category_idx (category),
53   KEY crops_status_idx (status),
54   KEY crops_name_idx (crop_name),
55   CONSTRAINT fk_crops_farmer FOREIGN KEY (farmer_id) REFERENCES users(id) ON DELETE CASCADE
56 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
57
58 CREATE TABLE mandi_rates (
59   rate_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
60   crop_name VARCHAR(80) NOT NULL,
61   market_location VARCHAR(80) NOT NULL,
62   min_price_per_kg DECIMAL(14,2) NOT NULL,
63   max_price_per_kg DECIMAL(14,2) NOT NULL,
64   avg_price_per_kg DECIMAL(14,2) NOT NULL,
65   source VARCHAR(20) NOT NULL DEFAULT 'manual',

```

```

66     date_updated      DATE          NOT NULL,
67     UNIQUE KEY mandi_crop_market_date_uq (crop_name, market_location, date_updated),
68     KEY mandi_date_idx (date_updated),
69     CHECK (min_price_per_kg <= avg_price_per_kg AND avg_price_per_kg <= max_price_per_kg)
70 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
71
72 CREATE TABLE bids_negotiations (
73     bid_id             INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
74     crop_id            INT UNSIGNED NOT NULL,
75     buyer_id          INT UNSIGNED NOT NULL,
76     bid_price_per_kg   DECIMAL(14,2) NOT NULL,
77     bid_quantity_kg    DECIMAL(14,2) NOT NULL,
78     target_delivery_date DATE        NULL,
79     message            TEXT          NULL,
80     status             ENUM('pending','accepted','rejected','countered','withdrawn') NOT NULL DEFAULT 'pending',
81     counter_price      DECIMAL(14,2) NULL,
82     counter_note       TEXT          NULL,
83     created_at         TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
84     updated_at         TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
85     KEY bids_crop_idx (crop_id),
86     KEY bids_buyer_idx (buyer_id),
87     KEY bids_status_idx (status),
88     CONSTRAINT fk_bids_crop FOREIGN KEY (crop_id) REFERENCES crops_inventory(crop_id) ON DELETE CASCADE,
89     CONSTRAINT fk_bids_buyer FOREIGN KEY (buyer_id) REFERENCES users(id) ON DELETE CASCADE
90 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
91
92 CREATE TABLE orders_logistics (
93     order_id           INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
94     bid_id             INT UNSIGNED NOT NULL,
95     crop_id            INT UNSIGNED NOT NULL,
96     farmer_id          INT UNSIGNED NOT NULL,
97     buyer_id          INT UNSIGNED NOT NULL,
98     quantity_kg        DECIMAL(14,2) NOT NULL,
99     price_per_kg       DECIMAL(14,2) NOT NULL,
100    total_amount       DECIMAL(14,2) NOT NULL,
101    payment_status     ENUM('awaiting_escrow','escrow_locked','released','refunded','disputed') NOT NULL
102    DEFAULT 'awaiting_escrow',
103    delivery_stage     ENUM('confirmed','quality_checked','dispatched','in_transit','delivered') NOT NULL
104    DEFAULT 'confirmed',
105    transit_location   VARCHAR(120) NULL,
106    current_lat        DOUBLE        NULL,
107    current_lng        DOUBLE        NULL,
108    tracking_number    VARCHAR(40) NOT NULL,
109    expected_delivery_date DATE      NULL,
110    created_at         TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
111    updated_at         TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
112    UNIQUE KEY orders_bid_uq (bid_id),
113    UNIQUE KEY orders_tracking_uq (tracking_number),
114    KEY orders_farmer_idx (farmer_id),
115    KEY orders_buyer_idx (buyer_id),
116    KEY orders_stage_idx (delivery_stage),
117    CONSTRAINT fk_orders_bid FOREIGN KEY (bid_id) REFERENCES bids_negotiations(bid_id),
118    CONSTRAINT fk_orders_crop FOREIGN KEY (crop_id) REFERENCES crops_inventory(crop_id),
119    CONSTRAINT fk_orders_farmer FOREIGN KEY (farmer_id) REFERENCES users(id),
120    CONSTRAINT fk_orders_buyer FOREIGN KEY (buyer_id) REFERENCES users(id)
121 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
122
123 CREATE TABLE quality_inspections (
124     inspection_id      INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
125     crop_id            INT UNSIGNED NOT NULL,
126     inspector_id       INT UNSIGNED NOT NULL,
127     grade_assigned     ENUM('A','B','C') NOT NULL,
128     moisture_level_percentage DECIMAL(5,2) NOT NULL,
129     soil_ph            DECIMAL(4,2) NULL,
130     inspection_notes   TEXT          NULL,
131     report_attachment  LONGTEXT     NULL,
132     report_file_name   VARCHAR(200) NULL,
133     certificate_no     VARCHAR(40) NULL,
134     status             ENUM('pending','passed','failed') NOT NULL DEFAULT 'pending',
135     verified_at        TIMESTAMP     NULL,
136     KEY inspections_crop_idx (crop_id),
137     KEY inspections_status_idx (status),
138     CONSTRAINT fk_insp_crop FOREIGN KEY (crop_id) REFERENCES crops_inventory(crop_id) ON DELETE CASCADE,
139     CONSTRAINT fk_insp_inspector FOREIGN KEY (inspector_id) REFERENCES users(id)

```

```

138 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
139
140 -- Supporting tables: order_events (tracking audit), wallet_transactions (escrow ledger), disputes
141 CREATE TABLE order_events (
142   event_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
143   order_id INT UNSIGNED NOT NULL,
144   stage VARCHAR(40) NOT NULL,
145   location VARCHAR(120) NULL,
146   lat DOUBLE NULL,
147   lng DOUBLE NULL,
148   note TEXT NULL,
149   actor_id INT UNSIGNED NULL,
150   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
151   KEY order_events_order_idx (order_id),
152   CONSTRAINT fk_events_order FOREIGN KEY (order_id) REFERENCES orders_logistics(order_id) ON DELETE CASCAD
153   E,
154   CONSTRAINT fk_events_actor FOREIGN KEY (actor_id) REFERENCES users(id) ON DELETE SET NULL
155 ) ENGINE=InnoDB;
156
157 CREATE TABLE wallet_transactions (
158   txn_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
159   user_id INT UNSIGNED NOT NULL,
160   order_id INT UNSIGNED NULL,
161   type ENUM('deposit','escrow_lock','escrow_release','payout','refund','platform_fee','withdrawal')
162   NOT NULL,
163   amount DECIMAL(14,2) NOT NULL,
164   description TEXT NULL,
165   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
166   KEY wallet_user_idx (user_id),
167   CONSTRAINT fk_wallet_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
168   CONSTRAINT fk_wallet_order FOREIGN KEY (order_id) REFERENCES orders_logistics(order_id) ON DELETE SET NU
169   LL
170 ) ENGINE=InnoDB;
171
172 CREATE TABLE disputes (
173   dispute_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
174   order_id INT UNSIGNED NOT NULL,
175   raised_by INT UNSIGNED NOT NULL,
176   reason TEXT NOT NULL,
177   status ENUM('open','investigating','resolved_release','resolved_refund') NOT NULL DEFAULT 'open',
178   resolution TEXT NULL,
179   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
180   resolved_at TIMESTAMP NULL,
181   KEY disputes_order_idx (order_id),
182   KEY disputes_status_idx (status),
183   CONSTRAINT fk_disputes_order FOREIGN KEY (order_id) REFERENCES orders_logistics(order_id) ON DELETE CASC
184   ADE,
185   CONSTRAINT fk_disputes_user FOREIGN KEY (raised_by) REFERENCES users(id)
186 ) ENGINE=InnoDB;
187
188 CREATE TABLE reviews (
189   review_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
190   order_id INT UNSIGNED NOT NULL,
191   buyer_id INT UNSIGNED NOT NULL,
192   farmer_id INT UNSIGNED NOT NULL,
193   rating TINYINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
194   comment TEXT NULL,
195   farmer_reply TEXT NULL,
196   replied_at TIMESTAMP NULL,
197   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
198   UNIQUE KEY reviews_order_uq (order_id),
199   KEY reviews_farmer_idx (farmer_id),
200   CONSTRAINT fk_reviews_order FOREIGN KEY (order_id) REFERENCES orders_logistics(order_id) ON DELETE CAS
201   CADE,
202   CONSTRAINT fk_reviews_buyer FOREIGN KEY (buyer_id) REFERENCES users(id) ON DELETE CASCADE,
203   CONSTRAINT fk_reviews_farmer FOREIGN KEY (farmer_id) REFERENCES users(id) ON DELETE CASCADE
204 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
205
206 CREATE TABLE notifications (
207   notification_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
208   user_id INT UNSIGNED NOT NULL,
209   type VARCHAR(40) NOT NULL,
210   title VARCHAR(200) NOT NULL,
211   body TEXT NULL,
212   link VARCHAR(300) NULL,
213   channels VARCHAR(60) NOT NULL DEFAULT 'in_app',

```

```

209     is_read    BOOLEAN        NOT NULL DEFAULT FALSE,
210     created_at TIMESTAMP      NOT NULL DEFAULT CURRENT_TIMESTAMP,
211     KEY notifications_user_idx (user_id, is_read),
212     CONSTRAINT fk_notifications_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
213 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
214
215 CREATE TABLE auth_tokens (
216     token_id   INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
217     user_id    INT UNSIGNED NOT NULL,
218     purpose    VARCHAR(20) NOT NULL,          -- verify_email | reset_link (15-min password reset token)
219     code_hash  VARCHAR(128) NOT NULL,         -- SHA-256 of the 6-digit code + secret
220     attempts   INT          NOT NULL DEFAULT 0,
221     expires_at TIMESTAMP     NOT NULL,
222     used_at    TIMESTAMP     NULL,
223     created_at TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
224     KEY auth_tokens_user_idx (user_id, purpose),
225     CONSTRAINT fk_auth_tokens_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
226 ) ENGINE=InnoDB;
227
228 CREATE TABLE payment_intents (
229     intent_id  INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
230     reference  VARCHAR(60) NOT NULL,
231     user_id    INT UNSIGNED NOT NULL,
232     provider   VARCHAR(20) NOT NULL,          -- stripe | jazzcash | easypaisa | sandbox
233     amount     DECIMAL(14,2) NOT NULL,
234     status     VARCHAR(20) NOT NULL DEFAULT 'pending',
235     provider_ref VARCHAR(200) NULL,
236     raw_callback TEXT        NULL,
237     created_at TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
238     completed_at TIMESTAMP    NULL,
239     UNIQUE KEY payment_intents_ref_uq (reference),
240     KEY payment_intents_user_idx (user_id),
241     CONSTRAINT fk_payment_intents_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
242 ) ENGINE=InnoDB;
243
244 CREATE TABLE payout_requests (
245     payout_id   INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
246     farmer_id   INT UNSIGNED NOT NULL,
247     amount      DECIMAL(14,2) NOT NULL,
248     method      VARCHAR(20) NOT NULL,         -- bank | jazzcash | easypaisa
249     account_title VARCHAR(120) NOT NULL,
250     account_number VARCHAR(60) NOT NULL,
251     status      VARCHAR(20) NOT NULL DEFAULT 'pending',
252     admin_note   TEXT        NULL,
253     created_at  TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
254     processed_at TIMESTAMP    NULL,
255     KEY payout_requests_farmer_idx (farmer_id),
256     KEY payout_requests_status_idx (status),
257     CONSTRAINT fk_payout_farmer FOREIGN KEY (farmer_id) REFERENCES users(id) ON DELETE CASCADE
258 ) ENGINE=InnoDB;
259
260 CREATE TABLE db_audit_log (
261     audit_id    INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
262     admin_id    INT UNSIGNED NULL,
263     mode        VARCHAR(10) NOT NULL,
264     query       TEXT        NOT NULL,
265     row_count   INT          NULL,
266     duration_ms INT          NULL,
267     error       TEXT        NULL,
268     created_at  TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
269     CONSTRAINT fk_audit_admin FOREIGN KEY (admin_id) REFERENCES users(id) ON DELETE SET NULL
270 ) ENGINE=InnoDB;
271

```

database/schema.sql

```

1  CREATE TYPE "public"."bid_status" AS ENUM('pending', 'accepted', 'rejected', 'countered', 'withdrawn');
2  CREATE TYPE "public"."crop_category" AS ENUM('grains', 'vegetables', 'fruits', 'cash_crops');
3  CREATE TYPE "public"."crop_status" AS ENUM('active', 'pending_inspection', 'sold_out', 'archived');
4  CREATE TYPE "public"."delivery_stage" AS ENUM('confirmed', 'quality_checked', 'dispatched', 'in_transit',
5  'delivered');
6  CREATE TYPE "public"."dispute_status" AS ENUM('open', 'investigating', 'resolved_release', 'resolved_refun
7  d');
8  CREATE TYPE "public"."inspection_status" AS ENUM('pending', 'passed', 'failed');
9  CREATE TYPE "public"."payment_status" AS ENUM('awaiting_escrow', 'escrow_locked', 'released', 'refunded',

```

```

7  'disputed');
8  CREATE TYPE "public"."quality_grade" AS ENUM('A', 'B', 'C');
9  CREATE TYPE "public"."user_role" AS ENUM('farmer', 'buyer', 'admin', 'inspector');
10 CREATE TYPE "public"."wallet_txn_type" AS ENUM('deposit', 'escrow_lock', 'escrow_release', 'payout', 'refund', 'platform_fee', 'withdrawal');
11 CREATE TABLE "auth_tokens" (
12     "token_id" serial PRIMARY KEY NOT NULL,
13     "user_id" integer NOT NULL,
14     "purpose" varchar(20) NOT NULL,
15     "code_hash" varchar(128) NOT NULL,
16     "attempts" integer DEFAULT 0 NOT NULL,
17     "expires_at" timestamp with time zone NOT NULL,
18     "used_at" timestamp with time zone,
19     "created_at" timestamp with time zone DEFAULT now() NOT NULL
20 );
21
22 CREATE TABLE "bids_negotiations" (
23     "bid_id" serial PRIMARY KEY NOT NULL,
24     "crop_id" integer NOT NULL,
25     "buyer_id" integer NOT NULL,
26     "bid_price_per_kg" numeric(14, 2) NOT NULL,
27     "bid_quantity_kg" numeric(14, 2) NOT NULL,
28     "target_delivery_date" date,
29     "message" text,
30     "status" "bid_status" DEFAULT 'pending' NOT NULL,
31     "counter_price" numeric(14, 2),
32     "counter_note" text,
33     "created_at" timestamp with time zone DEFAULT now() NOT NULL,
34     "updated_at" timestamp with time zone DEFAULT now() NOT NULL
35 );
36
37 CREATE TABLE "crops_inventory" (
38     "crop_id" serial PRIMARY KEY NOT NULL,
39     "farmer_id" integer NOT NULL,
40     "crop_name" varchar(80) NOT NULL,
41     "category" "crop_category" NOT NULL,
42     "quality_grade" "quality_grade" NOT NULL,
43     "total_quantity_kg" numeric(14, 2) NOT NULL,
44     "base_price_per_kg" numeric(14, 2) NOT NULL,
45     "harvest_date" date NOT NULL,
46     "images_json" jsonb DEFAULT '[]'::jsonb NOT NULL,
47     "description" text,
48     "farm_location" varchar(80) NOT NULL,
49     "latitude" double precision,
50     "longitude" double precision,
51     "status" "crop_status" DEFAULT 'active' NOT NULL,
52     "created_at" timestamp with time zone DEFAULT now() NOT NULL
53 );
54
55 CREATE TABLE "db_audit_log" (
56     "audit_id" serial PRIMARY KEY NOT NULL,
57     "admin_id" integer,
58     "mode" varchar(10) NOT NULL,
59     "query" text NOT NULL,
60     "row_count" integer,
61     "duration_ms" integer,
62     "error" text,
63     "created_at" timestamp with time zone DEFAULT now() NOT NULL
64 );
65
66 CREATE TABLE "disputes" (
67     "dispute_id" serial PRIMARY KEY NOT NULL,
68     "order_id" integer NOT NULL,
69     "raised_by" integer NOT NULL,
70     "reason" text NOT NULL,
71     "status" "dispute_status" DEFAULT 'open' NOT NULL,
72     "resolution" text,
73     "created_at" timestamp with time zone DEFAULT now() NOT NULL,
74     "resolved_at" timestamp with time zone
75 );
76
77 CREATE TABLE "mandi_rates" (
78     "rate_id" serial PRIMARY KEY NOT NULL,
79     "crop_name" varchar(80) NOT NULL,
80     "market_location" varchar(80) NOT NULL,
81     "min_price_per_kg" numeric(14, 2) NOT NULL,

```

```

82     "max_price_per_kg" numeric(14, 2) NOT NULL,
83     "avg_price_per_kg" numeric(14, 2) NOT NULL,
84     "source" varchar(20) DEFAULT 'manual' NOT NULL,
85     "date_updated" date NOT NULL
86 );
87
88 CREATE TABLE "notifications" (
89     "notification_id" serial PRIMARY KEY NOT NULL,
90     "user_id" integer NOT NULL,
91     "type" varchar(40) NOT NULL,
92     "title" varchar(200) NOT NULL,
93     "body" text,
94     "link" varchar(300),
95     "channels" varchar(60) DEFAULT 'in_app' NOT NULL,
96     "is_read" boolean DEFAULT false NOT NULL,
97     "created_at" timestamp with time zone DEFAULT now() NOT NULL
98 );
99
100 CREATE TABLE "order_events" (
101     "event_id" serial PRIMARY KEY NOT NULL,
102     "order_id" integer NOT NULL,
103     "stage" varchar(40) NOT NULL,
104     "location" varchar(120),
105     "lat" double precision,
106     "lng" double precision,
107     "note" text,
108     "actor_id" integer,
109     "created_at" timestamp with time zone DEFAULT now() NOT NULL
110 );
111
112 CREATE TABLE "orders_logistics" (
113     "order_id" serial PRIMARY KEY NOT NULL,
114     "bid_id" integer NOT NULL,
115     "crop_id" integer NOT NULL,
116     "farmer_id" integer NOT NULL,
117     "buyer_id" integer NOT NULL,
118     "quantity_kg" numeric(14, 2) NOT NULL,
119     "price_per_kg" numeric(14, 2) NOT NULL,
120     "total_amount" numeric(14, 2) NOT NULL,
121     "payment_status" "payment_status" DEFAULT 'awaiting_escrow' NOT NULL,
122     "delivery_stage" "delivery_stage" DEFAULT 'confirmed' NOT NULL,
123     "transit_location" varchar(120),
124     "current_lat" double precision,
125     "current_lng" double precision,
126     "tracking_number" varchar(40) NOT NULL,
127     "expected_delivery_date" date,
128     "created_at" timestamp with time zone DEFAULT now() NOT NULL,
129     "updated_at" timestamp with time zone DEFAULT now() NOT NULL
130 );
131
132 CREATE TABLE "payment_intents" (
133     "intent_id" serial PRIMARY KEY NOT NULL,
134     "reference" varchar(60) NOT NULL,
135     "user_id" integer NOT NULL,
136     "provider" varchar(20) NOT NULL,
137     "amount" numeric(14, 2) NOT NULL,
138     "status" varchar(20) DEFAULT 'pending' NOT NULL,
139     "provider_ref" varchar(200),
140     "raw_callback" text,
141     "created_at" timestamp with time zone DEFAULT now() NOT NULL,
142     "completed_at" timestamp with time zone
143 );
144
145 CREATE TABLE "payout_requests" (
146     "payout_id" serial PRIMARY KEY NOT NULL,
147     "farmer_id" integer NOT NULL,
148     "amount" numeric(14, 2) NOT NULL,
149     "method" varchar(20) NOT NULL,
150     "account_title" varchar(120) NOT NULL,
151     "account_number" varchar(60) NOT NULL,
152     "status" varchar(20) DEFAULT 'pending' NOT NULL,
153     "admin_note" text,
154     "created_at" timestamp with time zone DEFAULT now() NOT NULL,
155     "processed_at" timestamp with time zone
156 );
157

```

```

158 CREATE TABLE "quality_inspections" (
159     "inspection_id" serial PRIMARY KEY NOT NULL,
160     "crop_id" integer NOT NULL,
161     "inspector_id" integer NOT NULL,
162     "grade_assigned" "quality_grade" NOT NULL,
163     "moisture_level_percentage" numeric(5, 2) NOT NULL,
164     "soil_ph" numeric(4, 2),
165     "inspection_notes" text,
166     "report_attachment" text,
167     "report_file_name" varchar(200),
168     "certificate_no" varchar(40),
169     "status" "inspection_status" DEFAULT 'pending' NOT NULL,
170     "verified_at" timestamp with time zone
171 );
172
173 CREATE TABLE "reviews" (
174     "review_id" serial PRIMARY KEY NOT NULL,
175     "order_id" integer NOT NULL,
176     "buyer_id" integer NOT NULL,
177     "farmer_id" integer NOT NULL,
178     "rating" integer NOT NULL,
179     "comment" text,
180     "farmer_reply" text,
181     "replied_at" timestamp with time zone,
182     "created_at" timestamp with time zone DEFAULT now() NOT NULL
183 );
184
185 CREATE TABLE "users" (
186     "id" serial PRIMARY KEY NOT NULL,
187     "full_name" varchar(120) NOT NULL,
188     "email" varchar(160) NOT NULL,
189     "password_hash" varchar(255) NOT NULL,
190     "phone" varchar(30) NOT NULL,
191     "role" "user_role" NOT NULL,
192     "cnic_id" varchar(20),
193     "business_name" varchar(160),
194     "city" varchar(80) NOT NULL,
195     "address" text,
196     "is_verified" boolean DEFAULT false NOT NULL,
197     "email_verified" boolean DEFAULT false NOT NULL,
198     "title" varchar(120),
199     "user_code" varchar(40),
200     "avatar_url" text,
201     "google_id" varchar(64),
202     "auth_provider" varchar(20) DEFAULT 'password' NOT NULL,
203     "trust_score" numeric(3, 2) DEFAULT 0 NOT NULL,
204     "rating_avg" numeric(3, 2) DEFAULT 0 NOT NULL,
205     "rating_count" integer DEFAULT 0 NOT NULL,
206     "wallet_balance" numeric(14, 2) DEFAULT 0 NOT NULL,
207     "escrow_balance" numeric(14, 2) DEFAULT 0 NOT NULL,
208     "created_at" timestamp with time zone DEFAULT now() NOT NULL
209 );
210
211 CREATE TABLE "wallet_transactions" (
212     "txn_id" serial PRIMARY KEY NOT NULL,
213     "user_id" integer NOT NULL,
214     "order_id" integer,
215     "type" "wallet_txn_type" NOT NULL,
216     "amount" numeric(14, 2) NOT NULL,
217     "description" text,
218     "created_at" timestamp with time zone DEFAULT now() NOT NULL
219 );
220
221 ALTER TABLE "auth_tokens" ADD CONSTRAINT "auth_tokens_user_id_users_id_fk" FOREIGN KEY ("user_id") REFEREN
    ↵ CES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
222 ALTER TABLE "bids_negotiations" ADD CONSTRAINT "bids_negotiations_crop_id_crops_inventory_crop_id_fk" FORE
    ↵ IGN KEY ("crop_id") REFERENCES "public"."crops_inventory"("crop_id") ON DELETE cascade ON UPDATE no action
    ↵ ;
223 ALTER TABLE "bids_negotiations" ADD CONSTRAINT "bids_negotiations_buyer_id_users_id_fk" FOREIGN KEY ("buye
    ↵ r_id") REFERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
224 ALTER TABLE "crops_inventory" ADD CONSTRAINT "crops_inventory_farmer_id_users_id_fk" FOREIGN KEY ("farmer_
    ↵ id") REFERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
225 ALTER TABLE "db_audit_log" ADD CONSTRAINT "db_audit_log_admin_id_users_id_fk" FOREIGN KEY ("admin_id") REF
    ↵ ERENCES "public"."users"("id") ON DELETE set null ON UPDATE no action;
226 ALTER TABLE "disputes" ADD CONSTRAINT "disputes_order_id_orders_logistics_order_id_fk" FOREIGN KEY ("order
    ↵ _id") REFERENCES "public"."orders_logistics"("order_id") ON DELETE cascade ON UPDATE no action;

```

```

227 ALTER TABLE "disputes" ADD CONSTRAINT "disputes_raised_by_users_id_fk" FOREIGN KEY ("raised_by") REFERENCE
↳ S "public"."users"("id") ON DELETE restrict ON UPDATE no action;
228 ALTER TABLE "notifications" ADD CONSTRAINT "notifications_user_id_users_id_fk" FOREIGN KEY ("user_id") REF
↳ ERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
229 ALTER TABLE "order_events" ADD CONSTRAINT "order_events_order_id_orders_logistics_order_id_fk" FOREIGN KEY
↳ ("order_id") REFERENCES "public"."orders_logistics"("order_id") ON DELETE cascade ON UPDATE no action;
230 ALTER TABLE "order_events" ADD CONSTRAINT "order_events_actor_id_users_id_fk" FOREIGN KEY ("actor_id") REF
↳ ERENCES "public"."users"("id") ON DELETE set null ON UPDATE no action;
231 ALTER TABLE "orders_logistics" ADD CONSTRAINT "orders_logistics_bid_id_bids_negotiations_bid_id_fk" FOREIGN
↳ N KEY ("bid_id") REFERENCES "public"."bids_negotiations"("bid_id") ON DELETE restrict ON UPDATE no action;
232 ALTER TABLE "orders_logistics" ADD CONSTRAINT "orders_logistics_crop_id_crops_inventory_crop_id_fk" FOREIGN
↳ N KEY ("crop_id") REFERENCES "public"."crops_inventory"("crop_id") ON DELETE restrict ON UPDATE no action;
233 ALTER TABLE "orders_logistics" ADD CONSTRAINT "orders_logistics_farmer_id_users_id_fk" FOREIGN KEY ("farmer
↳ id") REFERENCES "public"."users"("id") ON DELETE restrict ON UPDATE no action;
234 ALTER TABLE "orders_logistics" ADD CONSTRAINT "orders_logistics_buyer_id_users_id_fk" FOREIGN KEY ("buyer
↳ id") REFERENCES "public"."users"("id") ON DELETE restrict ON UPDATE no action;
235 ALTER TABLE "payment_intents" ADD CONSTRAINT "payment_intents_user_id_users_id_fk" FOREIGN KEY ("user_id")
↳ REFERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
236 ALTER TABLE "payout_requests" ADD CONSTRAINT "payout_requests_farmer_id_users_id_fk" FOREIGN KEY ("farmer
↳ id") REFERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
237 ALTER TABLE "quality_inspections" ADD CONSTRAINT "quality_inspections_crop_id_crops_inventory_crop_id_fk"
↳ FOREIGN KEY ("crop_id") REFERENCES "public"."crops_inventory"("crop_id") ON DELETE cascade ON UPDATE no ac
↳ tion;
238 ALTER TABLE "quality_inspections" ADD CONSTRAINT "quality_inspections_inspector_id_users_id_fk" FOREIGN KE
↳ Y ("inspector_id") REFERENCES "public"."users"("id") ON DELETE restrict ON UPDATE no action;
239 ALTER TABLE "reviews" ADD CONSTRAINT "reviews_order_id_orders_logistics_order_id_fk" FOREIGN KEY ("order_i
↳ d") REFERENCES "public"."orders_logistics"("order_id") ON DELETE cascade ON UPDATE no action;
240 ALTER TABLE "reviews" ADD CONSTRAINT "reviews_buyer_id_users_id_fk" FOREIGN KEY ("buyer_id") REFERENCES "p
↳ ublic"."users"("id") ON DELETE cascade ON UPDATE no action;
241 ALTER TABLE "reviews" ADD CONSTRAINT "reviews_farmer_id_users_id_fk" FOREIGN KEY ("farmer_id") REFERENCES
↳ "public"."users"("id") ON DELETE cascade ON UPDATE no action;
242 ALTER TABLE "wallet_transactions" ADD CONSTRAINT "wallet_transactions_user_id_users_id_fk" FOREIGN KEY ("u
↳ ser_id") REFERENCES "public"."users"("id") ON DELETE cascade ON UPDATE no action;
243 ALTER TABLE "wallet_transactions" ADD CONSTRAINT "wallet_transactions_order_id_orders_logistics_order_id_f
↳ k" FOREIGN KEY ("order_id") REFERENCES "public"."orders_logistics"("order_id") ON DELETE set null ON UPDAT
↳ E no action;
244 CREATE INDEX "auth_tokens_user_idx" ON "auth_tokens" USING btree ("user_id","purpose");
245 CREATE INDEX "bids_crop_idx" ON "bids_negotiations" USING btree ("crop_id");
246 CREATE INDEX "bids_buyer_idx" ON "bids_negotiations" USING btree ("buyer_id");
247 CREATE INDEX "bids_status_idx" ON "bids_negotiations" USING btree ("status");
248 CREATE INDEX "crops_farmer_idx" ON "crops_inventory" USING btree ("farmer_id");
249 CREATE INDEX "crops_category_idx" ON "crops_inventory" USING btree ("category");
250 CREATE INDEX "crops_status_idx" ON "crops_inventory" USING btree ("status");
251 CREATE INDEX "crops_name_idx" ON "crops_inventory" USING btree ("crop_name");
252 CREATE INDEX "disputes_order_idx" ON "disputes" USING btree ("order_id");
253 CREATE INDEX "disputes_status_idx" ON "disputes" USING btree ("status");
254 CREATE UNIQUE INDEX "mandi_crop_market_date_uq" ON "mandi_rates" USING btree ("crop_name","market_location
↳ ","date_updated");
255 CREATE INDEX "mandi_date_idx" ON "mandi_rates" USING btree ("date_updated");
256 CREATE INDEX "notifications_user_idx" ON "notifications" USING btree ("user_id","is_read");
257 CREATE INDEX "order_events_order_idx" ON "order_events" USING btree ("order_id");
258 CREATE UNIQUE INDEX "orders_bid_uq" ON "orders_logistics" USING btree ("bid_id");
259 CREATE UNIQUE INDEX "orders_tracking_uq" ON "orders_logistics" USING btree ("tracking_number");
260 CREATE INDEX "orders_farmer_idx" ON "orders_logistics" USING btree ("farmer_id");
261 CREATE INDEX "orders_buyer_idx" ON "orders_logistics" USING btree ("buyer_id");
262 CREATE INDEX "orders_stage_idx" ON "orders_logistics" USING btree ("delivery_stage");
263 CREATE UNIQUE INDEX "payment_intents_ref_uq" ON "payment_intents" USING btree ("reference");
264 CREATE INDEX "payment_intents_user_idx" ON "payment_intents" USING btree ("user_id");
265 CREATE INDEX "payout_requests_farmer_idx" ON "payout_requests" USING btree ("farmer_id");
266 CREATE INDEX "payout_requests_status_idx" ON "payout_requests" USING btree ("status");
267 CREATE INDEX "inspections_crop_idx" ON "quality_inspections" USING btree ("crop_id");
268 CREATE INDEX "inspections_status_idx" ON "quality_inspections" USING btree ("status");
269 CREATE UNIQUE INDEX "reviews_order_uq" ON "reviews" USING btree ("order_id");
270 CREATE INDEX "reviews_farmer_idx" ON "reviews" USING btree ("farmer_id");
271 CREATE UNIQUE INDEX "users_email_uq" ON "users" USING btree ("email");
272 CREATE UNIQUE INDEX "users_google_id_uq" ON "users" USING btree ("google_id");
273 CREATE UNIQUE INDEX "users_user_code_uq" ON "users" USING btree ("user_code");
274 CREATE INDEX "users_role_idx" ON "users" USING btree ("role");
275 CREATE INDEX "users_city_idx" ON "users" USING btree ("city");
276 CREATE INDEX "wallet_user_idx" ON "wallet_transactions" USING btree ("user_id");
277

```

scripts/generate-brand-assets.cjs

```
1  /* eslint-disable */
2  /**
3   * Renders the brand assets that the site, e-mails and the project PDF expect:
4   *   public/apple-icon.png           180x180 iOS home-screen icon
5   *   public/favicon.ico             16/32/48 browser tab icon
6   *   public/brand/marketlink-email.png e-mail header logo (used as a CID attachment)
7   *
8   * Usage: NODE_PATH=./node_modules node scripts/generate-brand-assets.cjs
9   * Requires `sharp` (SVG rasteriser) and Python + Pillow for the .ico container.
10  */
11  const fs = require("fs");
12  const path = require("path");
13  const { execFileSync } = require("child_process");
14  const sharp = require("sharp");
15
16  const ROOT = path.resolve(__dirname, "..");
17  const BRAND = path.join(ROOT, "public", "brand");
18  const GREEN = { r: 1, g: 65, b: 28, alpha: 1 }; // brand-950 #01411c
19
20  const symbol = fs.readFileSync(path.join(BRAND, "marketlink-symbol.svg"));
21  const logo = fs.readFileSync(path.join(BRAND, "marketlink-logo.svg"));
22
23  (async () => {
24    // 1. Square app icon: symbol centred on the brand-green field.
25    const symbolPng = await sharp(symbol, { density: 600 }).resize({ width: 432, fit: "contain", background:
26    { r: 0, g: 0, b: 0, alpha: 0 } }).png().toBuffer();
27    const square = await sharp({ create: { width: 540, height: 540, channels: 4, background: GREEN } })
28    .composite([{ input: symbolPng, gravity: "center" }])
29    .png()
30    .toBuffer();
31
32    const apple = await sharp(square).resize(180, 180).png({ compressionLevel: 9 }).toBuffer();
33    fs.writeFileSync(path.join(ROOT, "public", "apple-icon.png"), apple);
34    fs.writeFileSync("/tmp/ml-icon-512.png", await sharp(square).resize(512, 512).png().toBuffer());
35
36    // 2. Multi-resolution .ico (Pillow builds the container from the 512px master).
37    execFileSync("python3", ["-c", `
38    from PIL import Image
39    img = Image.open("/tmp/ml-icon-512.png").convert("RGBA")
40    img.save(${JSON.stringify(path.join(ROOT, "public", "favicon.ico"))}, sizes=[(16,16),(32,32),(48,48)])
41    `], { stdio: "inherit" });
42
43    // 3. E-mail / PDF logo: the full lockup, 4x the 260px e-mail width for retina screens.
44    const email = await sharp(logo, { density: 600 }).resize({ width: 1040, fit: "contain", background: { r:
45    0, g: 0, b: 0, alpha: 0 } }).png().toBuffer();
46    fs.writeFileSync(path.join(BRAND, "marketlink-email.png"), email);
47
48    for (const f of ["apple-icon.png", "favicon.ico", "brand/marketlink-email.png"]) {
49      const p = path.join(ROOT, "public", f);
50      const meta = f.endsWith(".ico") ? "" : ` (${(await sharp(p).metadata()).width}px wide)`;
51      console.log(`✓ public/${f} - ${((fs.statSync(p).size / 1024).toFixed(1))} KB${meta}`);
52    }
53  })();
```

scripts/generate-pdf.cjs

```
1  /* eslint-disable */
2  // Generates public/MarketLink-AgriHub-Complete.pdf – the complete project book.
3  // Usage: NODE_PATH=/path/with/pdfkit/node_modules node scripts/generate-pdf.cjs [liveUrl]
4  const fs = require("fs");
5  const path = require("path");
6  const PDFDocument = require("pdfkit");
7  const fontkit = require("fontkit");
8  const ROOT = path.resolve(__dirname, "..");
9  const ts = require(path.join(ROOT, "node_modules/typescript"));
10
11  const OUT = path.join(ROOT, "public/MarketLink-AgriHub-Complete.pdf");
12  const LIVE_URL = process.argv[2] || "";
13  const F = "/usr/share/fonts/truetype/dejavu/";
14  const FONTS = { sans: F + "DejaVuSans.ttf", bold: F + "DejaVuSans-Bold.ttf", mono: F + "DejaVuSansMono.ttf",
15  monoB: F + "DejaVuSansMono-Bold.ttf" };
16
17  const doc = new PDFDocument({
18    fontkit,
19    fonts: FONTS,
20    liveUrl: LIVE_URL,
21    out: OUT,
22    type: "application/pdf"
23  });
24
25  doc.addPage(function () {
26    // ... (content generation logic) ...
27  });
28
29  doc.end();
```

```

15 const monoFont = fontkit.openSync(FONTS.mono);
16 const sansFont = fontkit.openSync(FONTS.sans);
17
18 function loadTs(file) {
19   const src = fs.readFileSync(path.join(ROOT, file), "utf8");
20   const js = ts.transpileModule(src, { compilerOptions: { module: ts.ModuleKind.CommonJS, target: ts.ScriptTarget.ES2019 } }).outputText;
21   const m = { exports: {} };
22   new Function("module", "exports", "require", js)(m, m.exports, require);
23   return m.exports;
24 }
25 const docs = loadTs("src/lib/docs-content.ts");
26 const { POSTGRES_DDL } = loadTs("src/lib/ddl-postgres.ts");
27
28 /** Keep text renderable by DejaVu: Urdu → "[Urdu]", drop emoji / unsupported glyphs, keep newlines. */
29 function clean(text, font) {
30   let out = "";
31   let inUrdu = false;
32   for (const ch of text) {
33     const cp = ch.codePointAt(0);
34     const isArabic = (cp >= 0x0600 && cp <= 0x06ff) || (cp >= 0xfb50 && cp <= 0xfefe);
35     if (isArabic) { if (!inUrdu) { out += "[Urdu]"; inUrdu = true; } continue; }
36     if (inUrdu && ch === " ") continue;
37     inUrdu = false;
38     if (ch === "\n") { out += "\n"; continue; }
39     if (ch === "\t") { out += " "; continue; }
40     if (cp === 0xfe0f || cp === 0x200d || cp === 0x200c || cp === 0xfeff) continue;
41     if (cp < 32 || cp >= 0x1f000) continue;
42     if (font.hasGlyphForCodePoint(cp)) out += ch;
43   }
44   return out;
45 }
46
47 const GREEN = "#01411c", GOLD = "#d9a514", GREY = "#64748b", LIGHT = "#edfbf2";
48 const doc = new PDFDocument({
49   size: "A4",
50   margins: { top: 56, bottom: 56, left: 48, right: 48 },
51   bufferPages: true,
52   info: { Title: "MARKETLINK AGRI-HUB – Complete Project", Author: "MarketLink Agri-Hub", Subject: "B2B Agricultural Marketplace – hosting, source code, schema, API & workflows" },
53 });
54 Object.entries(FONTS).forEach(([k, v]) => doc.registerFont(k, v));
55 doc.pipe(fs.createWriteStream(OUT));
56 const W = doc.page.width - 96;
57
58 function ensure(h) { if (doc.y + h > doc.page.height - 60) doc.addPage(); }
59 function h1(title) {
60   doc.addPage();
61   doc.rect(0, 0, doc.page.width, 90).fill(GREEN);
62   doc.rect(0, 90, doc.page.width, 4).fill(GOLD);
63   doc.fillColor("white").font("bold").fontSize(22).text(title, 48, 36, { width: W });
64   doc.fillColor("black").font("sans").fontSize(10);
65   doc.x = 48; doc.y = 118;
66   return doc.outline.addItem(title);
67 }
68 function h2(t) { doc.moveDown(0.6); ensure(40); doc.x = 48; doc.fillColor(GREEN).font("bold").fontSize(14).text(clean(t, sansFont), { width: W }); doc.moveTo(48, doc.y + 2).lineTo(108, doc.y + 2).lineWidth(2).strokeColor(GOLD).stroke(); doc.moveDown(0.5); }
69 function h3(t) { doc.moveDown(0.4); ensure(30); doc.x = 48; doc.fillColor("#0b5030").font("bold").fontSize(11.5).text(clean(t, sansFont), { width: W }); doc.moveDown(0.2); }
70 function p(t, o = {}) { doc.x = 48; doc.font(o.bold ? "bold" : "sans").fontSize(o.size || 10).fillColor(o.color || "#1e293b").text(clean(t, sansFont), { width: W, lineGap: 2 }); }
71 function bullet(t) { ensure(16); const y = doc.y; doc.circle(54, y + 5, 1.8).fill(GREEN); doc.fillColor("#1e293b").font("sans").fontSize(10).text(clean(t, sansFont), 62, y, { width: W - 14, lineGap: 2 }); doc.x = 48; }
72 function numbered(steps) {
73   steps.forEach((s, i) => { ensure(20); const y = doc.y; doc.circle(56, y + 6, 7).fill(GREEN); doc.fillColor("white").font("bold").fontSize(8).text(String(i + 1), 49, y + 2.5, { width: 14, align: "center" }); doc.fillColor("#1e293b").font("sans").fontSize(10).text(clean(s, sansFont), 70, y, { width: W - 22, lineGap: 2 }); doc.x = 48; doc.moveDown(0.4); });
74 }
75 function codeBlock(text, { size = 7.2, numbers = true } = {}) {
76   const lineH = size * 1.32, charW = size * 0.602, gutter = numbers ? 30 : 0;
77   const maxChars = Math.floor(W - gutter - 8) / charW;
78   const lines = clean(text.replace(/\r/g, ""), monoFont).split("\n");
79   lines.forEach((raw, idx) => {

```

```

80     const n = idx + 1;
81     const parts = [];
82     let rest = raw;
83     if (!rest.length) parts.push("");
84     while (rest.length) { parts.push(rest.slice(0, maxChars)); rest = rest.slice(maxChars); }
85     parts.forEach((seg, i) => {
86         if (doc.y + lineH > doc.page.height - 60) doc.addPage();
87         const y = doc.y;
88         if (n % 2 === 0) doc.rect(48, y - 1, W, lineH).fill("#f8faf8");
89         if (numbers) doc.font("mono").fontSize(size - 0.8).fillColor("#94a3b8").text(i === 0 ? String(n).pad
90 Start(4, " ") : "  ", 48, y, { width: gutter - 4, lineBreak: false });
91         doc.font("mono").fontSize(size).fillColor("#0f172a").text(seg || " ", 48 + gutter, y, { width: W - g
92 utter, lineBreak: false });
93         doc.y = y + lineH;
94     });
95 });
96 doc.x = 48;
97 doc.moveDown(0.5);
98 }
99
100 /* ----- Collect files ----- */
101 const files = [];
102 function walk(dir) {
103     for (const f of fs.readdirSync(path.join(ROOT, dir)).sort()) {
104         const rel = path.join(dir, f);
105         if (fs.statSync(path.join(ROOT, rel)).isDirectory()) walk(rel);
106         else if (/\. (ts|tsx|css|sql|mjs|cjs)$/ .test(f) && rel !== "src/lib/ddl-postgres.ts") files.push(rel);
107     }
108 }
109 ["README.md", "schema.sql", "schema.mysql.sql", "package.json", "tsconfig.json", "next.config.ts", "drizzle
110 e.config.json", "postcss.config.mjs", "eslint.config.mjs", "Dockerfile", "docker-compose.yml", "render.yam
111 l", ".env.example", "start-windows.bat", "start-docker.bat", "start.sh"].forEach((f) => fs.existsSync(path
112 .join(ROOT, f)) && files.push(f));
113 walk("src");
114 walk("database");
115 walk("scripts");
116 const read = (f) => fs.readFileSync(path.join(ROOT, f), "utf8");
117 const totalLines = files.reduce((s, f) => s + read(f).split("\n").length, 0);
118
119 /* ----- Cover ----- */
120 doc.rect(0, 0, doc.page.width, doc.page.height).fill(GREEN);
121 doc.rect(0, doc.page.height * 0.62, doc.page.width, 6).fill(GOLD);
122 doc.roundedRect(doc.page.width - 248, 83, 200, 80, 12).fill("white");
123 doc.image(path.join(ROOT, "public", "brand", "marketlink-email.png"), doc.page.width - 240, 92, { fit: [18
124 4, 62] });
125 doc.fillColor(GOLD).font("bold").fontSize(12).text("PROJECT 3 · SRS IMPLEMENTATION", 48, 150, { characterS
126 pacing: 2 });
127 doc.fillColor("white").font("bold").fontSize(38).text("MARKETLINK", 48, 180);
128 doc.text("AGRI-HUB", 48, 226);
129 doc.fillColor("#d3f5df").font("sans").fontSize(14).text("B2B Agricultural Marketplace connecting Pakistani
130 farmers (kisan) directly with wholesalers, supermarket chains, exporters and food processors.", 48, 290,
131 { width: W - 60, lineGap: 4 });
132 doc.fillColor("white").font("bold").fontSize(12).text("Complete Project Book", 48, 380);
133 doc.font("sans").fontSize(11).fillColor("#d3f5df");
134 ["Hosting guide: local, temporary public link and free cloud", "Database ERD & DDL (PostgreSQL + MySQL)",
135 "REST API endpoint specification", "Business logic workflows (Bidding · Escrow · Mandi Sync)", "Full sourc
136 e code: Farmer, Buyer & Admin dashboards"].forEach((l, i) => doc.text("▶ " + l, 48, 402 + i * 18));
137 doc.fillColor("white").font("bold").fontSize(11).text("Tech stack", 48, doc.page.height * 0.62 + 30);
138 doc.font("sans").fontSize(10).fillColor("#d3f5df").text("Next.js 16 (React 19) · Tailwind CSS 4 · PostgreS
139 QL · Drizzle ORM · Recharts · REST API · RBAC · Docker", 48, doc.y + 4, { width: W });
140 if (LIVE_URL) { doc.fillColor("white").font("bold").fontSize(11).text("Live preview (temporary)", 48, doc.
141 y + 16); doc.font("sans").fontSize(10).fillColor(GOLD).text(LIVE_URL, 48, doc.y + 4, { link: LIVE_URL, und
142 erline: true }); }
143 doc.fillColor("white").font("bold").fontSize(11).text("Demo logins (password: password123)", 48, doc.y + 1
144 6);
145 doc.font("sans").fontSize(10).fillColor("#d3f5df").text("Farmer: farmer@marketlink.pk   Buyer: buyer@mark
146 etlink.pk   Admin: ahmed.mustafa@admin.com", 48, doc.y + 4);
147 doc.fillColor("#a9e9c3").fontSize(9).text("Generated " + new Date().toLocaleString("en-GB", { timeZone: "A
148 sia/Karachi" }) + " PKT", 48, doc.page.height - 70);
149
150 /* ----- Contents ----- */
151 hl("Contents");
152 ["0", "Hosting Guide (local · temporary link · free cloud)", ["1", "Project Overview & Features"], ["2",
153 "Database: ERD & DDL Scripts"], ["3", "REST API Endpoint Specification"], ["4", "Business Logic Workflows
154 "], ["5", "Project Structure"], ["6", "Complete Source Code (${files.length} files, ${totalLines.toLocaleS
155 tring()} lines)`]].forEach(([n, t]) => {

```

```

136 doc.x = 48; doc.font("bold").fontSize(13).fillColor(GREEN).text(n + "    ", { continued: true }).fillCol
    or("#1e293b").font("sans").text(t); doc.moveDown(0.5);
137 });
138 doc.moveDown(1);
139 p("Tip: use your PDF viewer's bookmarks panel to jump to any section or source file.", { color: GREY, size
    : 9 });
140
141 /* ----- 0 Hosting ----- */
142 h1("0 · Hosting Guide");
143 h2("Why http://localhost:3000 does not open by itself");
144 p("localhost means 'this computer'. The link only works AFTER you start the MarketLink server on your own
    PC, and only while its server window stays open. Opening the link without starting the server always shows
    'This site can't be reached'.");
145 h2("Option 1: Run on your PC (recommended, only Node.js needed)");
146 p("The app includes a built-in PostgreSQL-compatible database (PGlite) stored in the .data folder, so ther
    e's no PostgreSQL or Docker to install. Tables and demo data are created automatically.");
147 numbered(["Install Node.js 20 LTS or newer from nodejs.org (green LTS button, default options).", "Downloa
    d MarketLink-AgriHub-Pakistan.zip from the /deploy page, right-click → Extract All. Don't run it from insi
    de the zip.", "Windows: double-click start-windows.bat. Mac/Linux: run 'bash start.sh' in Terminal. The fi
    rst run installs and builds (3-6 minutes).", "Wait for 'MarketLink is running!'. The browser opens http:/
    /localhost:3000 automatically. Keep the window open; press Ctrl+C to stop.", "Phone on the same Wi-Fi: ope
    n the 'Phone / same Wi-Fi' address printed in the window (allow Windows Firewall access if asked)."]);
148 h3("Troubleshooting");
149 [["'node' is not recognized", "Install Node.js, restart the PC, try again."], ["Browser: can't be reached"
    , "The server window was closed or is still starting. Wait for the message."], ["Port 3000 busy", "Handle
    d automatically: the window shows the port used (e.g. 3001)."], ["Fresh demo data", "Stop the server, dele
    te the .data folder, start again."], ["Updated code", "Run start-windows.bat --rebuild"], ["Own PostgreSQL
    ", "Set DATABASE_URL in .env. The database is created automatically."].forEach(([k, v]) => bullet(k + ":
    " + v));
150 h2("Option 2: Temporary public link from your PC");
151 numbered(["Keep Option 1 running.", "In a second terminal run: npx cloudflared tunnel --url http://localh
    ost:3000", "Share the printed https://xxxx.trycloudflare.com link. It works while your PC stays on. Altern
    ative: npx localtunnel --port 3000"]);
152 h2("Option 3: Docker (with a real PostgreSQL server)");
153 numbered(["Install Docker Desktop.", "Double-click start-docker.bat, or run: docker compose up --build", "
    Open http://localhost:3000"]);
154 h2("Option 4: Free cloud hosting (24/7)");
155 bullet("Render.com: push to GitHub → New → Blueprint (render.yaml creates the web service + PostgreSQL). F
    ree tier sleeps after 15 min idle.");
156 bullet("Vercel + Neon: free Neon Postgres, import the repo on Vercel, set DATABASE_URL, SESSION_SECRET, CO
    OKIE_SECURE=true.");
157 bullet("Railway.app: deploy from GitHub + PostgreSQL plugin, set DATABASE_URL and SESSION_SECRET.");
158 h2("Admin panel access");
159 numbered(["Open /admin-login (e.g. http://localhost:3000/admin-login) and sign in: ahmed.mustafa@admin.com
    / password123.", "Change the default password right away: sidebar → Account settings. The admin dashboard
    shows a red warning until you do.", "Admin pages: /admin (analytics), /admin/users (create admins & inspe
    ctors, reset passwords, change roles), /admin/verification, /admin/mandi, /admin/orders.", "Own admin logi
    n from the first start: set ADMIN_EMAIL and ADMIN_PASSWORD in .env before the first run (or delete the .da
    ta folder to start over)."]);
160 h2("Environment variables (.env)");
161 [["DATABASE_URL", "empty = built-in database; or a PostgreSQL connection string"], ["ADMIN_EMAIL / ADMIN_P
    ASSWORD", "admin login created on first start"], ["SESSION_SECRET", "long random text that signs login coo
    kies"], ["COOKIE_SECURE", "'true' when served over HTTPS"], ["PORT", "preferred port (default 3000, auto-i
    ncrements if busy)"]].forEach(([k, v]) => { ensure(14); doc.x = 48; doc.font("monoB").fontSize(9).fillColo
    r(GREEN).text(k, { continued: true }).font("sans").fillColor("#1e293b").text(" " + v); });
162
163 /* ----- 1 Overview ----- */
164 h1("1 · Project Overview & Features");
165 p("MarketLink Agri-Hub is a hyper-local and regional B2B agri-commerce platform for Pakistan. It lets farm
    ers sell graded produce directly to commercial buyers without arhti (commission agent) middlemen, with tra
    nsparent mandi pricing, PKR escrow payments, quality inspection and truck tracking.");
166 h2("Pakistan localisation");
167 ["Currency: Pakistani Rupee (Rs. / PKR), with Lakh / Crore short forms for large amounts.", "Mandi rates p
    er kg and per maund (40 kg), the unit Pakistani farmers and arhtis use.", "38 cities across Punjab, Sindh,
    Khyber Pakhtunkhwa, Balochistan, Islamabad, Gilgit-Baltistan and Azad Kashmir, grouped by province.", "10
    real wholesale mandis: Badami Bagh Lahore, Super Highway Karachi, Ghalla Mandi Multan, Faisalabad, I-11 I
    slamabad, Hyderabad, Peshawar, Quetta, Sukkur, Gujranwala.", "Local crops: Wheat, Basmati, Maize, Chana, T
    omato, Potato, Onion, Chaunsa Mango, Kinnow, Khajoor, Cotton (Phutti), Sugarcane, Kunri Red Chillli.", "Eng
    lish / Urdu toggle with right-to-left layout, Pakistan Standard Time clock, CNIC validation (12345-1234567
    -1), JazzCash / Easypaisa / IBFT top-up (simulated)."].forEach(bullet);
168 h2("User roles (RBAC)");
169 h3("Farmer (Kisan / Seller)");
170 ["Dashboard: inventory value, active listings, incoming bids, earnings & escrow, monthly sales chart, sale
    s-by-crop donut.", "Crop listing: photos, grade A/B/C, harvest date, quantity in kg / maund / tons, price
    suggested from live mandi rates.", "Live Mandi Insights and Negotiation Hub: accept / reject / counter fro
    m a slide-in drawer with instant feedback.", "Order dispatch, transit checkpoints, escrow payouts wallet."

```

```

170   ].forEach(bullet);
171   h3("Buyer (Wholesaler / Supermarket / Exporter / Processor)");
172   ["Marketplace filtered by category, grade, price and distance from any Pakistani city.", "Bidding & RFQ: p
    rice, quantity, target delivery date. Respond to counter-offers.", "Order pipeline with a 5-stage logistic
    s stepper and checkpoint timeline.", "Escrow wallet: lock funds at deal time, release on delivery, raise d
    isputes."].forEach(bullet);
173   h3("Admin / Quality Inspector");
174   ["Verification portal: verify farmer CNIC/farm, upload inspection reports (grade, moisture %, soil pH, PDF
    /image).", "Mandi price controller: manual daily rates and a mock government feed sync.", "System analytic
    s: volume, active bids, top regions, category mix, logistics pipeline, dispute log.", "Pre-dispatch qualiti
    y inspection and dispute resolution (release to farmer / refund buyer)."].forEach(bullet);
175   h2("Architecture");
176   ["Presentation: React Server Components + Client Components (drawers, modals, Recharts charts). Tailwind C
    SS 4, dark/light mode, mobile navigation.", "API: Next.js Route Handlers exposing REST endpoints with a {
    success, data | error } envelope.", "Domain: service modules (crops, bids, orders, mandi, analytics, admin
    ) with transactional row locking.", "Data: PostgreSQL (server) or built-in PGlite (local, zero-setup) via
    Drizzle ORM. 9 tables, enums, foreign keys, indexes. Schema + demo data are created automatically on an em
    pty database.", "Security: scrypt password hashing, HMAC-signed httpOnly session cookie, role checks on ev
    ery endpoint and page."].forEach(bullet);
177
178   /* ----- 2 Database ----- */
179   h1("2 · Database: ERD & DDL Scripts");
180   h2("Entity relationships");
181   codeBlock(docs.ERD, { numbers: false, size: 8.5 });
182   h2("PostgreSQL DDL (runtime, generated from src/db/schema.ts)");
183   codeBlock(PGSQL_DDL.replace(/--> statement-breakpoint/g, ""));
184   h2("MySQL 8 DDL (portable equivalent)");
185   codeBlock(docs.MYSQL_DDL);
186
187   /* ----- 3 API ----- */
188   const apiItem = h1("3 · REST API Endpoint Specification");
189   p('Authentication uses the httpOnly ml_session cookie set by /api/auth/login. Responses: { "success": true
    , "data": ... } or { "success": false, "error": "message" }.');
190   p("Errors: 400 bad JSON · 401 unauthenticated · 402 insufficient balance · 403 wrong role / not owner · 40
    4 not found · 409 invalid state transition · 422 validation.", { color: GREY, size: 9 });
191   const METHOD_COLOR = { GET: "#0284c7", POST: "#059669", PATCH: "#d97706", DELETE: "#e11d48" };
192   for (const g of docs.API_GROUPS) {
193     apiItem.addItem(g.name);
194     h2(g.name);
195     for (const e of g.endpoints) {
196       ensure(50);
197       const y = doc.y;
198       doc.roundedRect(48, y, 50, 14, 3).fill(METHOD_COLOR[e.method]);
199       doc.fillColor("white").font("monoB").fontSize(8).text(e.method, 48, y + 3, { width: 50, align: "center
    " });
200       doc.fillColor("#0f172a").font("monoB").fontSize(8.5).text(e.path, 104, y + 2.5, { width: W - 56 });
201       doc.x = 48;
202       doc.font("sans").fontSize(8.5).fillColor(GREY).text("Roles: " + e.roles, 48, doc.y + 2);
203       p(e.desc, { size: 9.5 });
204       if (e.request) { doc.font("bold").fontSize(8).fillColor(GREEN).text("REQUEST", 48, doc.y + 2); codeBlo
    ck(e.request, { numbers: false, size: 7.5 }); }
205       if (e.response) { doc.font("bold").fontSize(8).fillColor(GREEN).text("RESPONSE", 48, doc.y); codeBlock
    (e.response, { numbers: false, size: 7.5 }); }
206       doc.moveTo(48, doc.y).lineTo(48 + W, doc.y).lineWidth(0.5).strokeColor("#e2e8f0").stroke();
207       doc.moveDown(0.6);
208     }
209   }
210
211   /* ----- 4 Workflows ----- */
212   h1("4 · Business Logic Workflows");
213   for (const w of docs.WORKFLOWS) { h2(w.title); numbered(w.steps); }
214   h2("State machines");
215   codeBlock(`
    BID
    pending —farmer:accept→ accepted → order (awaiting_escrow)
    pending —farmer:counter→ countered
    countered —buyer:accept_counter→ accepted → order @ counter price
    countered —buyer:reject_counter→ rejected
    countered —buyer:revise→ pending (new price)
    pending —farmer:reject→ rejected
    pending|countered —buyer:withdraw→ withdrawn

    ORDER / LOGISTICS
    confirmed —(buyer locks escrow)→ quality_checked [inspector]
    → dispatched [farmer] → in_transit [farmer, checkpoints]
    → delivered [buyer] → escrow released: farmer gets total – 1.5% fee
  `);
228

```

```

229 PAYMENT
230 awaiting_escrow → escrow_locked → released
231 escrow_locked -(dispute)→ disputed -(admin)→ released | refunded`, { numbers: false, size: 8 });
232
233 /* ----- 5 Structure ----- */
234 h1("5 · Project Structure");
235 p("All files reproduced in section 6. (src/lib/ddl-postgres.ts is auto-generated from the schema and appears in section 2.)", { color: GREY, size: 9 });
236 doc.moveDown(0.5);
237 codeBlock(files.map((f) => " ".repeat(f.split("/").length - 1) + "└─ " + f + " (" + read(f).split("\n").length + " lines)").join("\n"), { numbers: false, size: 7.5 });
238
239 /* ----- 6 Source ----- */
240 const srcItem = h1("6 · Complete Source Code");
241 p(`${files.length} files · ${totalLines.toLocaleString()} lines, with line numbers. Long lines wrap with ↵`, { color: GREY, size: 9 });
242 const groups = {};
243 for (const f of files) {
244     const group = f.startsWith("src/app/api") ? "API route handlers" : f.startsWith("src/app") ? "Pages & layouts" : f.startsWith("src/components") ? "UI components" : f.startsWith("src/lib") || f.startsWith("src/db") ? "Backend services, DB & libraries" : f.startsWith("database") ? "Database scripts" : "Hosting, scripts & configuration";
245     groups[group] = groups[group] || srcItem.addItem(group);
246     ensure(90);
247     if (doc.y > 140) doc.moveDown(0.8);
248     groups[group].addItem(f);
249     const y = doc.y;
250     doc.rect(48, y, W, 20).fill(LIGHT);
251     doc.rect(48, y, 3, 20).fill(GREEN);
252     doc.fillColor(GREEN).font("monoB").fontSize(9.5).text(f, 58, y + 5.5, { width: W - 20 });
253     doc.y = y + 26; doc.x = 48;
254     codeBlock(read(f));
255 }
256
257 /* ----- Page footers ----- */
258 const range = doc.bufferedPageRange();
259 for (let i = 1; i < range.count; i++) {
260     doc.switchToPage(i);
261     const mb = doc.page.margins.bottom;
262     doc.page.margins.bottom = 0;
263     doc.font("sans").fontSize(7.5).fillColor(GREY).text("MARKETLINK AGRI-HUB · Complete Project Book", 48, doc.page.height - 34, { width: W / 2, lineBreak: false });
264     doc.text(`Page ${i + 1} of ${range.count}`, 48 + W / 2, doc.page.height - 34, { width: W / 2, align: "right", lineBreak: false });
265     doc.page.margins.bottom = mb;
266 }
267 doc.end();
268 console.log("PDF written:", OUT, "| pages:", range.count, "| files:", files.length, "| lines:", totalLines);
269

```

scripts/package-project.cjs

```

1  #!/usr/bin/env node
2  /* eslint-disable */
3  /**
4   * Builds the complete, organised project folder and zips it:
5   *   public/MarketLink-AgriHub-Pakistan.zip → MarketLink-AgriHub-Pakistan/
6   *
7   * Usage: node scripts/package-project.cjs [liveUrl]
8   */
9  const fs = require("fs");
10 const path = require("path");
11 const { spawnSync } = require("child_process");
12
13 const ROOT = path.resolve(__dirname, "..");
14 const NAME = "MarketLink-AgriHub-Pakistan";
15 const STAGE_PARENT = path.join(require("os").tmpdir(), "ml-package");
16 const STAGE = path.join(STAGE_PARENT, NAME);
17 const OUT_ZIP = path.join(ROOT, "public", `${NAME}.zip`);
18 const LIVE_URL = process.argv[2] || "";
19 const ts = require(path.join(ROOT, "node_modules/typescript"));
20
21 function loadTs(file) {
22     const src = fs.readFileSync(path.join(ROOT, file), "utf8");

```

```

23     const js = ts.transpileModule(src, { compilerOptions: { module: ts.ModuleKind.CommonJS, target: ts.Scrip
    ↵ tTarget.ES2019 } }).outputText;
24     const m = { exports: {} };
25     new Function("module", "exports", "require", js)(m, m.exports, require);
26     return m.exports;
27 }
28 const docs = loadTs("src/lib/docs-content.ts");
29 const { POSTGRES_DDL } = loadTs("src/lib/ddl-postgres.ts");
30 const C = loadTs("src/lib/constants.ts");
31
32 /* ----- helpers ----- */
33 function copy(src, dest, filter = () => true) {
34     const s = path.join(ROOT, src);
35     if (!fs.existsSync(s)) return;
36     const st = fs.statSync(s);
37     if (st.isDirectory()) {
38         for (const f of fs.readdirSync(s)) {
39             const rel = path.join(src, f);
40             if (filter(rel)) copy(rel, path.join(dest, f), filter);
41         }
42     } else {
43         fs.mkdirSync(path.dirname(dest), { recursive: true });
44         fs.copyFileSync(s, dest);
45     }
46 }
47 const write = (rel, content) => {
48     const p = path.join(STAGE, rel);
49     fs.mkdirSync(path.dirname(p), { recursive: true });
50     fs.writeFileSync(p, content);
51 };
52 const countFiles = (dir) => fs.readdirSync(dir, { withFileTypes: true }).reduce((n, d) => n + (d.isDirecto
    ↵ ry() ? countFiles(path.join(dir, d.name)) : 1), 0);
53
54 /* ----- 1. fresh stage ----- */
55 fs.rmSync(STAGE_PARENT, { recursive: true, force: true });
56 fs.mkdirSync(STAGE, { recursive: true });
57
58 /* ----- 2. application source (runs from the folder root) ----- */
59 const skip = (rel) => !/\.zip$/.test(rel) && !rel.includes("node_modules") && !rel.startsWith(".next") &&
    ↵ !rel.startsWith(".data");
60 ["src", "public", "database", "scripts"].forEach((d) => copy(d, path.join(STAGE, d), skip));
61 [
62     "package.json", "package-lock.json", "tsconfig.json", "next.config.ts", "drizzle.config.json", "postcss.
    ↵ config.mjs", "eslint.config.mjs",
63     "Dockerfile", ".dockerignore", "docker-compose.yml", "render.yaml", ".env.example", "README.md", "schema
    ↵ .sql", "schema.mysql.sql",
64     "start-windows.bat", "start-docker.bat", "start.sh",
65 ].forEach((f) => copy(f, path.join(STAGE, f)));
66 write(".gitignore", "node_modules\n.next\n.data\n.env\n*.log\n");
67 try { fs.chmodSync(path.join(STAGE, "start.sh"), 0o755); } catch {}
68
69 /* ----- 3. Documentation folder ----- */
70 const DOC = "Documentation";
71 copy("public/MarketLink-AgriHub-Complete.pdf", path.join(STAGE, DOC, "1-MarketLink-Complete-Project-Book.p
    ↵ df"));
72 write(`${DOC}/2-Database-Schema-PostgreSQL.sql`, POSTGRES_DDL.replace(/--> statement-breakpoint\n?/g, ""))
    ↵ ;
73 write(`${DOC}/3-Database-Schema-MySQL.sql`, docs.MYSQL_DDL);
74
75 // API spec
76 let api = `# MarketLink Agri-Hub Pakistan: REST API Endpoints\n\nAuthentication: httpOnly \n\nml_session` c
    ↵ ookie set by \n\nPOST /api/auth/login\n\nResponses: \n\n{ "success": true, "data": ... }\n\nor \n\n{ "success":
    ↵ false, "error": "message" }\n\nErrors: 400 bad JSON · 401 unauthenticated · 402 insufficient balance ·
    ↵ 403 wrong role · 404 not found · 409 invalid state · 422 validation.\n\n`;
77 for (const g of docs.API_GROUPS) {
78     api += `\n## ${g.name}\n`;
79     for (const e of g.endpoints) {
80         api += `\n### \`${e.method} \`${e.path}\n\n**Roles:** ${e.roles}\n\n${e.desc}\n`;
81         if (e.request) api += `\n**Request**\n\n\`\`\`json\n${e.request}\n\`\`\`\n`;
82         if (e.response) api += `\n**Response**\n\n\`\`\`json\n${e.response}\n\`\`\`\n`;
83     }
84 }
85 api += `\n## Account & admin (user management)\n\n### \n\nPATCH /api/account/password\n\n**Roles:** any sign
    ↵ ed-in user\n\n\`\`\`json\n{ "currentPassword": "password123", "newPassword": "MyNewPass#2025" }\n\n\`\`\`\n\n
    ↵ ### \n\nPOST /api/admin/users\n\n**Roles:** admin: create admin / inspector / farmer / buyer accounts\n\n\`
    ↵ \n\n\`\`\`json\n{ "fullName": "Inspector Kamran", "email": "kamran@marketlink.pk", "password": "Inspect@2025", "

```

```

phone": "+92 300 5556677", "role": "admin", "city": "Multan" }\n\`\`\`\n\n### \`PATCH /api/admin/users/:id
\`\n**Roles:** admin: \`${ "isVerified": true }\` · \`${ "newPassword": "..." }\` · \`${ "role": "admin" }\`\n
n`;
86 write(`${DOC}/4-API-Endpoints.md`, api);
87
88 // Workflows
89 let wf = `# MarketLink Agri-Hub Pakistan: Business Logic Workflows\n`;
90 for (const w of docs.WORKFLOWS) wf += `\n## ${w.title}\n\n${w.steps.map((s, i) => `${i + 1}. ${s}`).join("\n")}\n`;
91 wf += `\n## State machines\n\n\`\`\`\nBID\npending —farmer:accept————→ accepted —→ order (awaiting
_escrow)\npending —farmer:counter————→ countered\ncountered —buyer:accept_counter→ accepted —→ or
der @ counter price\ncountered —buyer:reject_counter→ rejected\ncountered —buyer:revise————→ pendi
ng (new price)\npending —farmer:reject————→ rejected\npending|countered —buyer:withdraw→ withdrawn
\n\nORDER / LOGISTICS\nconfirmed —(buyer locks escrow)→ quality_checked [inspector]\n          → dispatc
hed [farmer] → in_transit [farmer]\n          → delivered [buyer] → escrow released (farmer gets total -
1.5% fee)\n\nPAYMENT\nawaiting_escrow → escrow_locked → released\nescrow_locked —(dispute)→ disputed —
(admin)→ released | refunded\n\`\`\`\n`;
92 write(`${DOC}/5-Business-Workflows.md`, wf);
93
94 // ERD
95 write(`${DOC}/6-Database-ERD.md`, `# Database: Entity Relationship Diagram\n\n\`\`\`\n\n${docs.ERD}\n\`\`\`\n
\n\n Table | Purpose |\n|---|---|\n\n users | Farmers, buyers, admins; CNIC, city, verification, wallet & e
scrow balances |\n\n crops_inventory | Produce listings: grade, quantity, price, harvest date, images, loca
tion, status |\n\n mandi_rates | Daily min/max/avg per crop × mandi (unique per day) |\n\n bids_negotiations
| Offers & counter-offers (status state machine) |\n\n orders_logistics | Orders from accepted bids: payme
nt status, delivery stage, tracking no. |\n\n quality_inspections | Inspector grade, moisture %, soil pH, n
otes, report file |\n\n order_events | Tracking history for the logistics stepper |\n\n wallet_transactions
| Escrow ledger: deposit, lock, release, payout, refund, fee |\n\n disputes | Dispute resolution log |\n`);
96
97 // Admin guide
98 write(`${DOC}/7-Admin-Guide.md`, `# Admin Panel Guide\n\n## Sign in\n- Open **/admin-login** (e.g. http://
localhost:3000/admin-login)\n- Default: **ahmed.mustafa@admin.com** / **password123**\n- **Change it immed
iately:** sidebar → @ [Urdu] Account settings\n\n## Admin pages\n| Page | URL | What you can do |\n|---|---|
---|\n\n Analytics | /admin | Trade volume (PKR), active bids, top regions, logistics pipeline, dispute log
|\n\n Users & Access | /admin/users | Create admins/inspectors, reset passwords, verify users, change role
s |\n\n Verification | /admin/verification | Verify farmer CNIC, upload soil/crop inspection reports |\n\n P
rice Controller | /admin/mandi | Enter daily mandi rates, sync the mock government feed |\n\n Shipments & D
isputes | /admin/orders | Record quality inspection, resolve disputes (release / refund) |\n\n## Custom ad
min login on first start\nSet in \`.env\` **before the first run**:\n\`\`\`\nADMIN_EMAIL=you@yourcompany.p
k\nADMIN_PASSWORD=YourStrongPassword\n\`\`\`\nAlready started once? Stop the server, delete the \`.data\`
folder, start again (demo data is recreated).\n\n## Forgot the admin password (local install)\nSame steps:
set ADMIN_PASSWORD in .env → delete .data → restart.\n`);
99
100 // Demo accounts
101 write(`${DOC}/8-Demo-Accounts.txt`, `MarketLink Agri-Hub Pakistan: demo accounts (password for all: passwo
rd123)\n\nADMIN\n  ahmed.mustafa@admin.com      Ahmed Mustafa (Super Admin)      → /admin-login\n\nQUALIT
Y INSPECTOR\n  inspector@marketlink.pk      Dr. Farhan Malik (Faisalabad)      → /inspector/dashboard\n\nFARMER
S (KISAN)\n  farmer@marketlink.pk      Muhammad Aslam (Multan)      wheat, Chaunsa mango, cotton\n  ra
sool@marketlink.pk      Ghulam Rasool (Sahiwal)      basmati, maize, potato\n  tariq@marketlink.pk
Rana Tariq (Faisalabad)      sugarcane, wheat, chana (unverified)\n  bashir@marketlink.pk      Bas
hir Ahmed Soomro (Hyderabad) tomato, onion, mango, red chilli\n  nasreen@marketlink.pk      Nasreen Bibi (
Okara)      potato, kinnow (unverified)\n  wahid@marketlink.pk      Abdul Wahid Baloch (Turbat)
dates (khajoor)\n\nBUYERS\n  buyer@marketlink.pk      Qureshi Wholesale Traders (Lahore)\n  metro@mark
etlink.pk      FreshMart Supermarket Supply (Karachi)\n  exports@marketlink.pk      Indus Agro Exports (
Karachi)\n  processor@marketlink.pk      Punjab Food Processors Ltd (Faisalabad)\n`);
102
103 /* ----- 4. START-HERE.html (offline guide) ----- */
104 const tree = [
105   ["START-HERE.html", "This page"],
106   ["► [Urdu] start-windows.bat", "Double-click to run on Windows"],
107   ["► [Urdu] start.sh", "Run on Mac / Linux: bash start.sh"],
108   ["start-docker.bat", "Run with Docker (optional)"],
109   ["README.md", "Full setup & hosting instructions"],
110   ["Documentation/", "PDF project book, SQL scripts, API, workflows, ERD, admin guide, demo accounts"],
111   ["src/", "Application source code: pages, dashboards, REST API, business logic"],
112   ["database/", "Database DDL scripts (PostgreSQL + MySQL)"],
113   ["public/", "Images and downloadable files"],
114   ["scripts/", "Local server launcher, PDF & package generators"],
115   ["[Urdu] schema.sql / schema.mysql.sql", "Database DDL: PostgreSQL (runtime) and MySQL 8, all 15 tables"]
116   ,
117   ["@ [Urdu].env.example", "Settings: database, admin login, payment gateways, email/SMS, AI keys"],
118   ["@ [Urdu] package.json ..", "Project configuration (Next.js, Tailwind, Drizzle)"],
119   ["_ [Urdu] Dockerfile, docker-compose.yml, render.yaml", "Docker & free cloud hosting"],
120 ];
121 const docsList = fs.readdirSync(path.join(STAGE, DOC)).sort();
122 const html = `<!doctype html>

```

```

122 <html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scal
123 <title>START HERE: MarketLink Agri-Hub Pakistan</title>
124 <style>
125 *{box-sizing:border-box}body{margin:0;font-family:Segoe UI,Roboto,Arial,sans-serif;background:#f6f8f4;colo
126 header{background:#01411c;color:#fff;padding:36px 24px 44px;border-bottom:6px solid #f1c232}
127 .wrap{max-width:980px;margin:0 auto}h1{margin:0;font-size:34px}header p{color:#d3f5df;margin:.4em 0 0}
128 .ur{font-size:20px;color:#f7d774;direction:rtl;margin-top:8px}
129 main{padding:28px 24px 60px}.card{background:#fff;border:1px solid #e3ebe5;border-radius:18px;padding:22px
130 24px;margin:0 0 18px;box-shadow:0 4px 16px -8px rgba(1,65,28,.15)}
130 h2{margin:0 0 12px;color:#01411c;font-size:20px}.warn{background:#fffbef;border-color:#fcd34d}
131 ol{padding-left:22px}li{margin:6px 0}code,kbd{background:#edfbf2;border:1px solid #cfe9d8;border-radius:6p
132 x;padding:1px 6px;font-family:Consolas,monospace;font-size:13px}
132 table{width:100%;border-collapse:collapse;font-size:14px}td{padding:8px 6px;border-bottom:1px solid #eef2e
133 f;vertical-align:top}td:first-child{font-weight:600;white-space:nowrap;padding-right:18px}
133 .grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(220px,1fr));gap:12px}.pill{background:#014
134 11c;color:#fff;border-radius:14px;padding:14px 16px}
134 .pill b{color:#f1c232;display:block}.pill span{font-size:13px;color:#d3f5df}a{color:#0d653b;font-weight:60
135 0}
135 .term{background:#002611;color:#a9e9c3;border-radius:12px;padding:14px 16px;font-family:Consolas,monospace
136 ;font-size:13px;white-space:pre;overflow:auto}
136 footer{text-align:center;color:#64748b;font-size:13px;padding:20px}
137 </style></head><body>
138 <header><div class="wrap">
139 <div style="display:inline-flex;align-items:center;gap:12px;background:white;border-radius:14px;padding:8p
140 x 16px;margin-bottom:18px"></div>
142 <h1>MARKETLINK AGRI-HUB</h1>
143 <p>B2B agricultural marketplace: farmers ↔ wholesalers, supermarkets, exporters & food processors. PKR · p
144 er-maund mandi rates · escrow · truck tracking.</p>
145 <div class="ur">[Urdu]– [Urdu]</div>
146 </div></header>
147 <main class="wrap">
148 <div class="card warn"><h2>▲[Urdu]Important</h2>
149 <b>http://localhost:3000</b> only works <b>after</b> you start the server (steps below) and <b>while its b
150 lack window stays open</b>.
151 Also: extract this folder from the zip first, because it can't run from inside the zip.</div>
152 <div class="card"><h2>▶[Urdu]Run the website on this PC (4 steps)</h2>
153 <ol>
154 <li>Install <b>Node.js 20 LTS</b> or newer from <a href="https://nodejs.org" target="_blank">nodejs.org</a>
155 (one time, default options).</li>
156 <li><b>Windows:</b> double-click <code>start-windows.bat</code> &nbsp;·&nbsp; <b>Mac/Linux:</b> open Termi
157 nal here and run <code>bash start.sh</code></li>
158 <li>First run installs & builds (3-6 minutes). Wait for <b>MarketLink is running!</b></li>
159 <li>Your browser opens <a href="http://localhost:3000">http://localhost:3000</a> automatically. Keep the w
160 indow open; press <b>Ctrl</b>+<b>C</b> to stop.</li>
161 </ol>
162 <div class="term">=====
163 MarketLink is running!
164 On this computer: http://localhost:3000
165 Phone / same Wi-Fi: http://192.168.x.x:3000
166 </div>
167 <p style="font-size:14px;color:#475569">No PostgreSQL needed: a built-in database is created automatically
168 in the <code>.data</code> folder, with Pakistani demo data.</p>
169 </div>
170 <div class="card"><h2>Logins (password: <code>password123</code>)</h2>
171 <div class="grid">
172 <div class="pill"><b>[Urdu]Admin panel</b><br>ahmed.mustafa@admin.com<br><span>Open <a style="color:#f7d774" h
173 ref="http://localhost:3000/admin-login">localhost:3000/admin-login</a></span></div>
174 <div class="pill"><b>Inspector</b><br>inspector@marketlink.pk<br><span>Quality lab desk</span></div>
175 <div class="pill"><b>Farmer</b><br>farmer@marketlink.pk<br><span>Muhammad Aslam · Multan</span></div>
176 <div class="pill"><b>Buyer</b><br>buyer@marketlink.pk<br><span>Qureshi Wholesale · Lahore</span></div>
177 </div>
178 <p style="font-size:14px">After signing in as admin, change the password in <b>@[Urdu]Account settings</b>
179 . All demo accounts: <a href="Documentation/8-Demo-Accounts.txt">Documentation/8-Demo-Accounts.txt</a></p>
180 </div>
181 <div class="card"><h2>What's in this folder</h2><table>
182 ${tree.map(([a, b]) => `<tr><td>${a}</td><td>${b}</td></tr>`).join("\n")}
183 </table></div>
184 <div class="card"><h2>Documentation</h2><table>

```

```

180   ${docsList.map((f) => `<tr><td><a href="Documentation/${f}">${f}</a></td><td>${
181     f.endsWith(".pdf") ? "Complete project book: hosting guide, features, ERD & DDL, REST API, workflows, fu
    ↵ ll source code" :
182     f.includes("PostgreSQL") ? "PostgreSQL DDL (used by the app)" : f.includes("MySQL") ? "MySQL 8 DDL (port
    ↵ able version)" :
183     f.includes("API") ? "REST API endpoints with request/response JSON" : f.includes("Workflows") ? "Bidding
    ↵ , escrow/order progression, mandi rate sync" :
184     f.includes("ERD") ? "Entity relationship diagram & table purposes" : f.includes("Admin") ? "Admin panel:
    ↵ login, pages, custom admin password" : "All demo logins"
185   }</td></tr>`).join("\n")}
186 </table></div>
187
188 <div class="card"><h2> Share it online</h2>
189 <table>
190 <tr><td>Temporary public link</td><td>While running, in a 2nd terminal: <code>npx cloudflared tunnel --url
    ↵ http://localhost:3000</code> → share the https://...trycloudflare.com link</td></tr>
191 <tr><td>Phone (same Wi-Fi)</td><td>Open the "Phone / same Wi-Fi" address shown in the black window</td></tr>
    ↵ </tr>
192 <tr><td>Free cloud 24/7</td><td>Upload folder to GitHub → render.com → New → Blueprint (uses <code>render.
    ↵ yaml</code></td></tr>
193 <tr><td>Docker</td><td>Double-click <code>start-docker.bat</code> (includes PostgreSQL)</td></tr>
194 </table>
195 <p style="font-size:14px">Full details: <a href="README.md">README.md</a></p>
196 </div>
197
198 <div class="card"><h2> Troubleshooting</h2><table>
199 <tr><td>'node' is not recognized</td><td>Install Node.js, restart the PC, try again</td></tr>
200 <tr><td>"This site can't be reached"</td><td>The server window is closed or still starting. Wait for </td>
    ↵ </tr>
201 <tr><td>Port 3000 busy</td><td>Automatic: the window shows the port used (e.g. 3001)</td></tr>
202 <tr><td>Fresh demo data</td><td>Stop the server, delete the <code>.data</code> folder, start again</td></tr>
    ↵ </tr>
203 <tr><td>Edited the code</td><td>Run <code>start-windows.bat --rebuild</code></td></tr>
204 </table></div>
205 ${LIVE_URL ? `<div class="card"><h2> Online preview (temporary)</h2><a href="${LIVE_URL}" target="_blank">
    ↵ ${LIVE_URL}</a></div>` : ""}
206 </main>
207 <footer>MarketLink Agri-Hub Pakistan · Next.js · Tailwind CSS · PostgreSQL / PGlite · Drizzle ORM · Rechar
    ↵ ts · packaged ${new Date().toLocaleString("en-GB", { timeZone: "Asia/Karachi" })} PKT</footer>
208 </body></html>`;
209 write("START-HERE.html", html);
210
211 /* ----- 5. zip ----- */
212 fs.rmSync(OUT_ZIP, { force: true });
213 for (const old of ["MarketLink-AgriHub-Source.zip"]) fs.rmSync(path.join(ROOT, "public", old), { force: tr
    ↵ ue });
214 const r = spawnSync("zip", ["-rq", "-X", OUT_ZIP, NAME], { cwd: STAGE_PARENT, stdio: "inherit" });
215 if (r.status !== 0) { console.error("zip failed"); process.exit(1); }
216 console.log(`Package ready: ${OUT_ZIP} | ${countFiles(STAGE)} files | ${((fs.statSync(OUT_ZIP).size / 1024
    ↵ / 1024).toFixed(2))} MB`);
217 console.log(`Folder preview: ${STAGE}`);
218

```

scripts/serve.cjs

```

1  #!/usr/bin/env node
2  /* eslint-disable */
3  /**
4   * MarketLink Agri-Hub Pakistan: one-command local server.
5   *
6   *   node scripts/serve.cjs           build (if needed) → start → open browser
7   *   node scripts/serve.cjs --rebuild force a fresh build
8   *   NO_OPEN=1 node scripts/serve.cjs don't open the browser
9   *   PORT=4000 node scripts/serve.cjs preferred port (auto-increments if busy)
10  *
11  * Works on Windows, macOS and Linux. Needs only Node.js 20.9+; uses the built-in
12  * database automatically unless DATABASE_URL is set in .env.
13  */
14  const { spawn, spawnSync } = require("child_process");
15  const fs = require("fs");
16  const net = require("net");
17  const os = require("os");
18  const path = require("path");
19  const http = require("http");
20

```

```

21 const ROOT = path.resolve(__dirname, "..");
22 process.chdir(ROOT);
23 const isWin = process.platform === "win32";
24 const npm = isWin ? "npm.cmd" : "npm";
25 const G = (s) => `\x1b[32m${s}\x1b[0m`, Y = (s) => `\x1b[33m${s}\x1b[0m`, R = (s) => `\x1b[31m${s}\x1b[0m`
  ↵ , B = (s) => `\x1b[1m${s}\x1b[0m`;
26
27 function fail(msg) {
28   console.error("\n" + R("✖ " + msg) + "\n");
29   process.exit(1);
30 }
31
32 // 1. Node version
33 const [maj, min] = process.versions.node.split(".").map(Number);
34 if (maj < 20 || (maj === 20 && min < 9)) fail(`Node.js ${process.versions.node} is too old. Install Node.js
  ↵ 20 LTS or newer from https://nodejs.org`);
35
36 console.log(B("\n MarketLink Agri-Hub Pakistan: local server\n"));
37
38 // 2. .env (optional). Without DATABASE_URL the built-in database is used.
39 if (!fs.existsSync(".env") && fs.existsSync(".env.example")) {
40   fs.copyFileSync(".env.example", ".env");
41   console.log(G("✓ Created .env (using built-in database, no PostgreSQL needed)"));
42 }
43
44 // 3. Dependencies
45 if (!fs.existsSync("node_modules/next")) {
46   console.log(Y("• Installing packages (first run only, 2-5 minutes)..."));
47   const r = spawnSync(npm, ["install", "--no-audit", "--no-fund"], { stdio: "inherit", shell: isWin });
48   if (r.status !== 0) fail("npm install failed. Check your internet connection and try again.");
49 }
50
51 // 4. Build
52 const needBuild = process.argv.includes("--rebuild") || !fs.existsSync(".next/BUILD_ID");
53 if (needBuild) {
54   console.log(Y("• Building the website (1-2 minutes)..."));
55   const r = spawnSync(npm, ["run", "build"], { stdio: "inherit", shell: isWin });
56   if (r.status !== 0) fail("Build failed. See the messages above.");
57 }
58
59 // 5. Free port
60 function portFree(port) {
61   return new Promise((resolve) => {
62     const srv = net.createServer();
63     srv.once("error", () => resolve(false));
64     srv.once("listening", () => srv.close(() => resolve(true)));
65     srv.listen(port, "0.0.0.0");
66   });
67 }
68 async function findPort(start) {
69   for (let p = start; p < start + 20; p++) if (await portFree(p)) return p;
70   fail(`No free port found between ${start} and ${start + 19}.`);
71 }
72
73 function get(url, timeout = 60000) {
74   return new Promise((resolve) => {
75     const req = http.get(url, { timeout }, (res) => {
76       res.resume();
77       res.on("end", () => resolve(res.statusCode));
78     });
79     req.on("error", () => resolve(0));
80     req.on("timeout", () => { req.destroy(); resolve(0); });
81   });
82 }
83
84 function lanAddresses() {
85   const out = [];
86   for (const list of Object.values(os.networkInterfaces())) for (const a of list || []) if (a.family === "
  ↵ IPv4" && !a.internal) out.push(a.address);
87   return out;
88 }
89
90 function openBrowser(url) {
91   if (process.env.NO_OPEN) return;
92   try {
93     if (isWin) spawn("cmd", ["/c", "start", "", url], { detached: true, stdio: "ignore" }).unref();

```

```

94     else if (process.platform === "darwin") spawn("open", [url], { detached: true, stdio: "ignore" }).unre
95     f();
96     else spawn("xdg-open", [url], { detached: true, stdio: "ignore" }).unref();
97   } catch {}
98 }
99
100 (async () => {
101   const preferred = Number(process.env.PORT) || 3000;
102   // Already running? (e.g. the start file was double-clicked twice) → just open it.
103   for (let p = preferred; p < preferred + 5; p++) {
104     const running = await new Promise((resolve) => {
105       const req = http.get(`http://127.0.0.1:${p}/api/health`, { timeout: 1500 }, (res) => {
106         let body = "";
107         res.on("data", (c) => (body += c));
108         res.on("end", () => resolve(res.statusCode === 200 && body.includes('"ok":true')));
109       });
110       req.on("error", () => resolve(false));
111       req.on("timeout", () => { req.destroy(); resolve(false); });
112     });
113     if (running) {
114       console.log(G(`✓ MarketLink is already running → http://localhost:${p}`) + " (opening browser)");
115       openBrowser(`http://localhost:${p}`);
116       process.exit(0);
117     }
118     const port = await findPort(preferred);
119     if (port !== preferred) console.log(Y(`• Port ${preferred} is busy, using ${port} instead`));
120
121     const nextBin = path.join(ROOT, "node_modules", "next", "dist", "bin", "next");
122     const child = spawn(process.execPath, [nextBin, "start", "-p", String(port), "-H", "0.0.0.0"], { stdio:
123     ["ignore", "pipe", "pipe"], env: process.env });
124     child.stdout.on("data", (d) => process.stdout.write(d));
125     child.stderr.on("data", (d) => process.stderr.write(d));
126     child.on("exit", (code) => { if (code) console.error(R(`Server stopped (exit code ${code}).`)); process.
127     exit(code || 0); });
128     const stop = () => { child.kill(); process.exit(0); };
129     process.on("SIGINT", stop);
130     process.on("SIGTERM", stop);
131
132     // 6. Wait until the server answers, then warm up (creates tables + demo data on first run)
133     const local = `http://localhost:${port}`;
134     let ok = false;
135     for (let i = 0; i < 120 && !ok; i++) {
136       ok = (await get(`http://127.0.0.1:${port}/api/health`, 3000)) === 200;
137       if (!ok) await new Promise((r) => setTimeout(r, 1000));
138     }
139     if (!ok) fail("Server did not become healthy. If you set DATABASE_URL in .env, check PostgreSQL is runni
140     ng, or remove that line to use the built-in database.");
141     console.log(Y(`• Preparing database & demo data (first run takes ~10 s)...`));
142     await get(`http://127.0.0.1:${port}/login`, 120000);
143
144     const lan = lanAddresses();
145     console.log("\n" + G("=".repeat(58)));
146     console.log(B(G(" MarketLink is running!")));
147     console.log(` On this computer:   ${B(local)}`);
148     lan.forEach((ip) => console.log(` Phone / same Wi-Fi:  ${B(`http://${ip}:${port}`)}));
149     console.log(` Demo login:       farmer@marketlink.pk / password123`);
150     console.log(` Stop the server:   press Ctrl + C (keep this window open while using the site)`);
151     console.log(G("=".repeat(58)) + "\n");
152     openBrowser(local);
153   })();
154 }

```

scripts/verify-payment-redirect.cjs

```

1  #!/usr/bin/env node
2  /* eslint-disable */
3  /**
4   * Checks that the escrow/buyer checkout redirect is a usable absolute HTTPS URL.
5   *
6   * node scripts/verify-payment-redirect.cjs # against http://127.0.0.1:3000
7   * node scripts/verify-payment-redirect.cjs --base=https://my.app
8   * node scripts/verify-payment-redirect.cjs --host=3000-abc.e2b.app # simulate the public Host header
9   * node scripts/verify-payment-redirect.cjs --host=3000-abc.e2b.app --full
10  *

```

```

11  * Regression guard for ERR_ADDRESS_INVALID: a redirect URL must never point at a bind or loopback
12  * address (0.0.0.0, ::, *) and must be a well-formed absolute http(s) URL.
13  *
14  * Uses the demo buyer account (buyer@marketlink.pk / password123) – see .env.example / README.
15  * Requires a running server; no dependencies beyond Node.js 20+.
16  */
17  const http = require("http");
18  const https = require("https");
19  const { URL } = require("url");
20
21  const arg = (name, fallback) => {
22    const hit = process.argv.find((a) => a === `--${name}` || a.startsWith(`--${name}=`));
23    if (!hit) return fallback;
24    const value = hit.includes("=") ? hit.slice(hit.indexOf("=") + 1) : "";
25    return value || true;
26  };
27
28  const BASE = String(arg("base", process.env.BASE_URL || "http://127.0.0.1:3000")).replace(/\/$/, "");
29  const HOST_HEADER = arg("host", "");
30  const FULL = !!arg("full", false);
31  const EMAIL = String(arg("email", "buyer@marketlink.pk"));
32  const PASSWORD = String(arg("password", "password123"));
33  const AMOUNT = Number(arg("amount", 250000));
34
35  let failures = 0;
36  const pass = (msg) => console.log(` ${msg}`);
37  const fail = (msg) => { failures++; console.log(` ${msg}`); };
38  const check = (ok, msg) => (ok ? pass(msg) : fail(msg));
39
40  /** Minimal request helper: node:http is used so a custom Host header can be sent (fetch forbids it). */
41  function request(method, path, { body, cookie, hostHeader } = {}) {
42    const url = new URL(path, BASE);
43    const payload = body === undefined ? null : JSON.stringify(body);
44    const headers = { accept: "application/json" };
45    if (payload) { headers["content-type"] = "application/json"; headers["content-length"] = Buffer.byteLength(payload); }
46    if (cookie) headers.cookie = cookie;
47    if (hostHeader) { headers.host = hostHeader; headers["x-forwarded-proto"] = url.protocol === "https:" ? "https" : "http"; }
48    const client = url.protocol === "https:" ? https : http;
49    return new Promise((resolve, reject) => {
50      const req = client.request({ method, protocol: url.protocol, hostname: url.hostname, port: url.port || undefined, path: url.pathname + url.search, headers }, (res) => {
51        let text = "";
52        res.on("data", (c) => (text += c));
53        res.on("end", () => {
54          let json = null;
55          try { json = JSON.parse(text); } catch { /* non-JSON page */ }
56          resolve({ status: res.statusCode, headers: res.headers, text, json });
57        });
58      });
59      req.on("error", reject);
60      if (payload) req.write(payload);
61      req.end();
62    });
63  }
64
65  const isBindHost = (hostname) => /^(0\.0\.0\.0|0|::|.*|[:\])$/i.test(hostname);
66
67  (async () => {
68    const suffix = HOST_HEADER ? ` (Host: ${HOST_HEADER})` : "";
69    console.log(`\n Escrow payment redirect check – ${BASE}${suffix}\n`);
70
71    const health = await request("GET", "/api/health", { hostHeader: HOST_HEADER }).catch((e) => e);
72    if (!health || health.status !== 200) {
73      console.error(`* Server not reachable at ${BASE}. Start it first (npm run dev / npm start).\n`);
74      process.exit(1);
75    }
76
77    const login = await request("POST", "/api/auth/login", { body: { email: EMAIL, password: PASSWORD }, hostHeader: HOST_HEADER });
78    const setCookie = login.headers["set-cookie"] || [];
79    const cookie = (Array.isArray(setCookie) ? setCookie : [setCookie]).map((c) => c.split(";")[0]).join(";");
80    if (!login.json?.success) {
81      console.error(`* Login failed for ${EMAIL}: ${login.json?.error || login.status}. Use --email/--passwo

```

```

81     rd for another account.\n`);
82     process.exit(1);
83 }
84 pass(`signed in as ${EMAIL}`);
85
86 const walletBefore = await request("GET", "/api/wallet", { cookie, hostHeader: HOST_HEADER });
87 const balanceBefore = walletBefore.json?.data?.walletBalance ?? 0;
88
89 const checkout = await request("POST", "/api/payments/checkout", { cookie, hostHeader: HOST_HEADER, body
90 : { provider: "sandbox", amount: AMOUNT } });
91 const data = checkout.json?.data;
92 if (!data?.redirectUrl) { fail(`checkout did not return a redirect URL: ${checkout.text.slice(0, 200)}`)
93 ; process.exit(1); }
94
95 const target = new URL(data.redirectUrl);
96 const looksAbsolute = /^https?:\/\//i.test(data.redirectUrl);
97 check(looksAbsolute, `redirect URL is an absolute ${target.protocol.replace(":", "")} URL: ${data.redire
98 ctUrl}`);
99 check(!isBindHost(target.hostname), `redirect host "${target.hostname}" is not a bind address (0.0.0.0 /
100 :: / *)`);
101 const localhost = target.hostname === "localhost" || /^127\./.test(target.hostname) || /^d+\.\d+\.\d+\.
102 \d+$/i.test(target.hostname) || target.hostname.endsWith(".local");
103 check(target.protocol === "https:" || localhost, "scheme matches the host (https for public hosts)");
104 if (HOST_HEADER) check(target.host.split(":")[0] === HOST_HEADER.split(":")[0], `redirect host matches t
105 he requested host "${HOST_HEADER}"`);
106
107 const page = await request("GET", `${target.pathname}${target.search}`, { cookie, hostHeader: HOST_HEADE
108 R });
109 check(page.status === 200 && page.text.includes(data.reference), `checkout page opens and shows referenc
110 e ${data.reference}`);
111
112 if (FULL) {
113     const pay = await request("POST", `/api/payments/sandbox/${data.reference}`, { cookie, hostHeader: HOS
114 T_HEADER, body: { outcome: "success" } });
115     check(pay.json?.data?.status === "succeeded", "signed gateway callback settled the payment");
116     const walletAfter = await request("GET", "/api/wallet", { cookie, hostHeader: HOST_HEADER });
117     const balanceAfter = walletAfter.json?.data?.walletBalance ?? 0;
118     check(balanceAfter === balanceBefore + AMOUNT, `wallet credited Rs. ${AMOUNT.toLocaleString("en-US")}
119 (${balanceBefore} → ${balanceAfter})`);
120 }
121
122 console.log(failures === 0 ? "\n Payment redirect resolution looks correct.\n" : `\n ${failures} check(s
123 ) failed.\n`);
124 process.exit(failures === 0 ? 0 : 1);
125 }().catch((e) => {
126     console.error(`\n* ${e.message}\n`);
127     process.exit(1);
128 });

```